基于多源数据与结构水热耦合的吉隆泥石流触发及传播研究

摘　要

针对高寒山区冰崩向泥石流演变的链式过程，本文建立“气象因素识别—结构水热稳定性—启动时间预测—传播响应”的多模型研究体系。

首先，基于 MET、RADM、SOIL 和 FLUX 构造 5526 个窗口、112 项特征，通过边际异常、条件贡献与相似天气过程三路线识别因素族，得到土壤水分、空气湿状态、土壤热状态、空气热状态和平均风场的共识 TOP5，并量化第五席竞争与排序稳定性。其次，在共同结构与参数设计下构建结构、结构加温度、结构加水及结构水热耦合四机制，完成 2048 个情景，形成条件临界关系；解析热核与细化有限差分的最大有限启动差为 0.347 小时，原始 141 条试验为本构竞争提供外部依据。

时间预测采用机制首次过阈进行条件状态回放，以逆速度/状态空间和动态生存模型在 13 个外部终末失稳组及 1 个真实右删失站上分组评价 24、48、72 小时预测。固定提前 24 小时发布 72 小时预测时，逆速度和状态空间方法的完整标签包含为 3/13 和 6/13，显示信息尺度与发布设计的重要作用。传播部分完成 34 个简化和 31 个原生计算，以全源面 ST、120 米格网为主展示，比较孔压、复合固水、夹带及数值尺度响应；7 个选定运行联接 4 个机械设计、49 条目标记录。

研究以区域背景、条件机制、外部时间评价和给定释放传播分别回答四问，给出因素排序、临界关系、时间区间及过程场，并据此提出环境背景—源区状态—沟谷响应的分层监测配置。

关键词：因素族识别；结构水热耦合；首次过阈；时间预测；传播仿真

AI 辅助说明：本文部分文字整理、程序和图表使用 OpenAI Codex 辅助；具体贡献及使用记录见文后声明与配套说明。

目录

1 问题分析与研究框架 1

2 多源数据、预处理与统一符号 2

3 多源气象特征与区域异常因素识别 4

4 结构水热耦合的触发条件与机制评估 8

5 三引擎时间预测与分层预警响应 12

6 给定释放条件下的泥石流传播与模型比较 17

7 模型评价、适用范围与防灾启示 22

8 结论 24

AI 工具使用声明 27

A 补充结果、软件与复跑说明 28

B 论文采用的自编程序 29

1 问题分析与研究框架

1.1 灾害链与研究目标

吉隆高寒山区地形起伏强烈，冰雪、破碎岩体与沟谷水流共同构成物质和能量传递系统。赛题描述的 2026 年 8 月 26 日灾害具有“冰崩—碎屑流—泥石流—山洪”的链式特征。对于这类过程，气象背景、源区稳定性和下游运动分别对应不同的空间支持与时间尺度：区域观测反映环境演变，结构与水热状态决定源区的抗力和驱动力，释放后的地形、摩阻、孔压及组分控制传播响应。

本文以这一分工组织四问。首先从 MET、RADM、SOIL 和 FLUX 提取因素族特征，识别事件前突出的区域环境状态；随后在共同结构几何和参数设计下比较温度、水与损伤机制，建立条件临界关系；再将物理首次过阈、连续变形加速和动态生存三类时间预测模型并列评价；最后通过沿程、二维、孔压和复合相计算刻画释放后的传播场及目标响应。四部分之间交换有物理含义的状态和参数，而不是将某一项统计排序直接当作触发方程。

1.2 三个时刻与两类空间尺度

记源区首次机械启动时刻为 t_{m}，进入传播模型的整体释放时刻为 t_{r}，目标 j 达到给定响应阈值的时刻为 $t_{\mathrm{a},j}$。其关系写为

$$t_{\mathrm{a},j} = t_{\mathrm{m}} + \Delta_{\mathrm{mr}} + \tau_j, \qquad \Delta_{\mathrm{mr}} = t_{\mathrm{r}} - t_{\mathrm{m}}, \quad \tau_j = t_{\mathrm{a},j} - t_{\mathrm{r}}. \tag{1}$$

其中 Δ_{mr} 描述机械启动到整体释放的阶段转换，τ_j 由给定释放状态和传播模型计算。本文分别保存这三个对象，并以对应的资料与模型评价它们。题面参考时刻用于环境窗口定位，目录时刻及波形资料用于事件阶段分析，目标响应采用释放后的相对时钟。

空间上，气象站作为区域背景支撑，源区和河谷使用米制投影下的几何与地形；对口岸、桥位和设施分别登记代表对象。本文以“区域背景因素—源区条件机制—河谷传播响应”的层级联系这些尺度，使站点异常、物理临界条件和影响足迹具有各自清晰的解释对象。

1.3 共同实验设计与技术路线

模型竞争采用共同输入和明确对照。气象因素比较围绕背景年份、天气过程和窗口支持展开；触发机制使用配对参数点，逐项加入温度、水和耦合；时间预测固定发布机会和预测时域；传播模型固定可比地形、物源及边界，并单独检查原生初态、网格与时间步。

本文的主要贡献体现在三个方面。第一，以因素族聚合和多路线竞争减少同一物理因素的重复入选，并同时报告方向与排序稳定性。第二，将机制方程、响应时标、参数补偿和可观测预测放在同一分析中，形成结构—温度—水的条件临界图景。第三，将时间预测和传播实验按信息类型、物理阶段与评价尺度关联，形成可追溯的模型比较框架。由此，复杂模型的价值通过新增过程量和可检验响应呈现。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

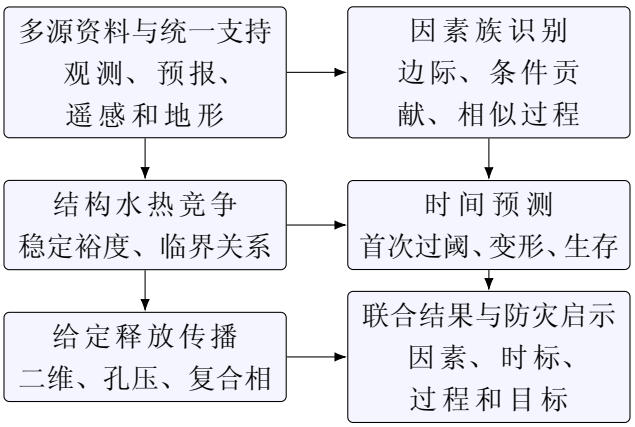


图 1 研究技术路线。跨问交换采用同一参数或情景标识；环境因素、机械启动和给定释放后的运动分别建模。

2 多源数据、预处理与统一符号

2.1 资料组成与研究用途

小时资料覆盖 MET、RADM、SOIL、FLUX 四类，共 24 份年度 CSV、63 个测量字段和 37 个原始质量标记。统一时间轴包含 52 584 个小时；数值记录的实际边界为 2021 年 4 月 23 日 11 时至 2026 年 8 月 27 日 8 时（原始 UTC+8 标签），各变量按其有效观测和质量轨使用。区域站元数据位置为 28.86°N、85.29°E，海拔 4140 m，适合描述高寒山区的背景变化 [1]。背景排序采用 2021—2025 年对应日历日期前后 30 日，事件窗口保留 24、72 和 168 小时。

Copernicus GLO-30 地形以可追溯 DSM 构建共同 60 和 120 m 计算网格；源产品主体采集年代为 2011—2015 年，垂直参考 EGM2008[2]。遥感资料用于几何、表面状态和变化线索，地形用于条件传播计算。县域逐日温度资料独立保留长期背景角色。GFS 按起报、预报时效和有效时刻组织，将气象预报输入与区域站历史观测分别管理 [3]。

2.2 质量视图与时空预处理

数值表、质量标记表和来源记录保留相同行序。记原始测量为 $x_{v,t}$，数值有效掩膜为 $m^{\mathrm{num}}_{v,t}$，原始质量标记为 $q_{v,t}$，主轨定义为

$$m^{(0)}_{v,t} = m^{\mathrm{num}}_{v,t}\,\mathbf{1}(q_{v,t}=0), \qquad x^{(0)}_{v,t} = \begin{cases} x_{v,t}, & m^{(0)}_{v,t}=1, \\ \mathrm{NA}, & \text{其余位置.} \end{cases} \tag{2}$$

其中数值有效表示有限值并排除来源哨兵 9999.9。原始文本缺测、哨兵、来源插补和质量类别分别保留。FLUX 及降水采用原质量标记未随附的探索轨，通过纳入与排除比较评估其对因素排序的影响。此处理使质量选择能够复查，同时保留不同物理变量的符号与量纲。

小时来源按 UTC+8 约定换算为 UTC，事件叙述明确标注显示时区；各窗口采用半开时间区间，并通过 ±1 小时平移实际改变样本集合。县域逐日表保持其日期标签和空间聚合层级。模型计算统一使用 UTC、米、秒或小时及明确的温度/压力单位，图表同时

表 1　数据体系与分析支持。小时数和字段数表示记录结构，具体统计使用对应质量轨的有效样本。

资料	结构与空间支持	主要用途
MET	22 字段，21 质量标记；1、2、4、10、20 m 梯度	温湿风状态、垂直差异、气压与降水探索
RADM	4 字段，4 质量标记	上下行短波、长波与辐射背景
SOIL	12 字段，12 质量标记；0.1—1.6 m 土层	含水率、土温、热通量及层间梯度
FLUX	25 字段，单列探索质量轨	湍流、风分量、声温及通量特征
GFS	3 个起报，f000—f096 共 579 个选定字段	假设延迟和理想输入下的逐小时机制回放
遥感与地形	SAR 速度对、光学像元、COP-DEM 及对象几何	物源/几何约束、路径与传播地形
外部监测	13 失稳组、1 真实右删失站	变形加速、生存模型与时间区间分组评价

标明其物理支持。

空间分析使用 WGS84/UTM 45N（EPSG:32645）。栅格地形、源面掩膜、路径与目标几何以版本关联；网格间比较固定总释放体积后调整厚度。对影像分别检查有效覆盖、分类掩膜和读入范围，对对象几何保留其产品或重建来源。物理阶段、资料用途与几何版本共同决定每项结果的解释范围。

2.3　共享符号、信息集与评价单位

本文分别组织区域观测分析、外部站点评价、给定条件情景和数值基准。因素重采样以年份和过程为层级，时间预测以物理站点或失稳组为分组单位，传播以配对情景为比较单位。重复发布、传感器和目标响应均在原分组内汇总。后续章节使用这些单位报告样本数、区间覆盖和敏感性，使不同模型的比较具有一致基准。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

表 2　跨问主要符号；分问专用参数在对应模型中定义。

符号	含义	单位/支持
t, t_0, H	时刻、预测发布时刻、预测时域	UTC；$H = 24, 48, 72$ h
$t_{\mathrm{m}}, t_{\mathrm{r}}, \tau_j$	首次机械启动、整体释放、目标相对传播时延	h 或 s，分别注明
$\boldsymbol{\theta}$	给定机制的结构、热、水及材料参数	各分量按物理量给出
T, α	温度、热扩散率	°C、m^2/s
σ_n, p	法向应力、孔隙水压力	kPa
$h, \boldsymbol{u}$	流深、深度平均水平速度	m、m/s
V, M	体积、质量	m^3、kg
$\mathcal{I}(t_0)$	发布时刻可使用的信息集	按观测/假设延迟分列

3　多源气象特征与区域异常因素识别

3.1　比较对象与多尺度特征

本问识别事件参考时段相对于同季节背景的气象异常，得到供结构水热分析使用的候选因素。综合三种统计路线，主质量轨的前五项依次为土壤水分、空气湿状态、土壤热状态、空气热状态和平均风；其中第五席与土热通量存在竞争。排序表示区域观测的相对异常及其稳定性，物理作用还需结合源区条件解释。

使用吉隆地气相互作用逐小时综合观测的气象、辐射、土壤及通量四类资料 [1]。元数据站址为 28.86°N、85.29°E，海拔 4140 m，对应区域背景站。主质量轨仅接纳有限且原始 QC 为 0 的值；无配对 QC 的降水和 FLUX 变量另列探索轨，原有非零 QC 值仍剔除。由 63 个原始变量构造 24 个驱动量、112 个排序特征，其中 82 个属于主质量轨，另 30 个用于探索（表3）。按物理对象归族，可防止同一变量的多个高度或统计量获得额外选票。

表 3　四类资料与特征库规模

资料	原变量	驱动量	主轨特征	探索增量	主要表征
MET	22	10	37	5	温湿、平均风、气压、降水标签
RADM	4	4	20	0	长短波及上下行差
SOIL	12	5	25	0	浅层水热、垂向梯度、土热通量
FLUX	25	5	0	25	VPD、感潜热、摩擦速度、湍动能
合计	63	24	82	30	9 个主轨族、3 个新增探索族

注：探索轨保留主轨并加入 30 个特征；VPD 并入空气湿状态。降水标签的积分时段、通量符号及处理口径按原资料保留，因此不计算累计降水或积分能量。

参考时刻取题面所述时刻，按源时钟 UTC+8 换算为 2026 年 8 月 26 日 02:30 UTC。取其前 24、72 和 168 小时的标签窗口；小时标签的平均区间边界仍作为时间口径条件。2021–2025 年各取 8 月 26 日前后 30 天、每日一个同钟点背景，共 305 个窗口，按 7 日

块划分 45 个天气过程代理。三日和十四日块用于敏感性分析，过程块只表征相关性，不计作灾害事件。

普通驱动量提取中位数、5% 和 95% 分位数、每 24 小时线性趋势、最长连续上升链占窗口标签数的比例；风向以正余弦中位数表示。降水另取中位数、95% 分位数、最大值、有效值中的正值比例和最长正值链比例。缺测打断连续链，主设置要求窗口有效支持率至少 75%。这种设计同时保留状态、尾部与变化过程，避免只用单一极值代表整个事前窗口。

3.2 三条异常路线与共识排序

三条路线分别衡量边际偏离、条件异常和类比背景差异。对年份 y 中的过程 p 及其第 i 个有效窗口，令权重 $w_i = 1/(Y P_y n_{yp})$，其中 Y 为有效年份数，P_y 为该年有效过程数，n_{yp} 为过程内有效窗口数；遇缺测重新归一。路线 A 使用加权中点经验分布：

$$F_j(x) = \sum_i w_i \big[\mathbf{1}(x_{ij} < x) + \tfrac{1}{2}\mathbf{1}(x_{ij} = x)\big], \qquad a_j = 2\left|F_j(x_j^{*}) - \tfrac{1}{2}\right|. \tag{3}$$

先在驱动量内取特征分数中位数，再在族内取驱动量分数中位数。中点处理允许常值背景，双侧距离则同时识别偏高和偏低异常。

路线 B 选取背景加权观测率至少 80%、覆盖至少三个年份且目标有值的列。以训练加权中位数为中心、1.4826 MAD 为尺度，退化时改用 IQR/1.349，仍为零则剔除。训练中位数填补缺测，截断至 $[-8, 8]$ 并减去训练加权均值。借鉴协方差收缩思想 [4]，固定单位阵目标及收缩强度：

$$\Sigma_\lambda = (1-\lambda)C + \lambda I, \quad P = \Sigma_\lambda^{-1}, \quad b_g = \frac{\eta_g^{\mathsf{T}} (P_{gg})^{-1} \eta_g}{d_g}, \quad \eta = P\delta, \qquad \lambda = 0.5. \tag{4}$$

这里 C 为加权协方差，δ 为中心化后的目标向量，d_g 为族 g 的有效特征数。b_g 表示给定其他族后的单位维数条件距离，再用逐年留出的背景贡献分布转换成上侧百分位；各族贡献不要求可加。每折重新确定选列、中心、尺度、填补和协方差，留出行须实际观测到该族全部活跃列才参与校准。

路线 C 以自然类比思想 [5] 比较相近的其他气象状态：匹配空间沿用 B 的有效列，距离排除待评估族及其派生量，各剩余族等权；匹配维度实际观测率至少 80%，每年选取两个不同过程的最近窗口，至多 10 个类比。随后计算待评估族在类比集合中的双侧中点分布偏离，仍取驱动量、物理族两层中位数。该路线比较其他状态相近时，待评族的异常是否仍然突出。

三路线各自在每个窗口内形成名次 r_{gmw}，同分用中间名次；跨路线、跨窗口的共识为

$$S_g = \frac{1}{9} \sum_{w \in \{24,72,168\}} \sum_{m \in \{A,B,C\}} \left(1 - \frac{r_{gmw} - 1}{G_{mw} - 1}\right). \tag{5}$$

主轨 $G_{mw} = 9$。TOP5 边界出现并列时按剩余席位分配入选份额，避免用变量顺序打破并列。先做留年与留过程检查，后者按最大 168 小时实际标签支持清除相邻重叠窗口；两

质量轨、三窗口共 300 个评估折，复用同样的 50 种分组。目标时段只用于评分，不参与尺度或参数选择。

五个留年折各以 244 个窗口训练、61 个窗口留出；45 个留过程折各以 286–294 个窗口训练、5–7 个留出，并清除 6–12 个相邻窗口。分割单位保持年份与天气过程的组织结构，可减少重叠长窗口造成的近邻重复。它们服务于背景评分与排序检查，统计对象始终是同一案例的气象背景比较。

3.3　排序结果与水热状态解释

表4给出三路线跨窗口排序和最终共识。A 首先识别土水、空气热与短波，B 更突出土热和空气湿，C 更突出土水与平均风。路线差异说明边际偏离与其他气象状态给定后的异常并不等价；采用等权名次共识，可使不同量纲的路线分数具有一致的汇总尺度。

表 4　主质量轨的三路线跨窗口名次、共识及稳定性

物理族	A 名次	B 名次	C 名次	共识 S_g	平均入选频率
土壤水分	1	5	1	0.743	0.814
空气湿状态	5	2	3	0.667	0.814
土壤热状态	6	1	4	0.639	0.843
空气热状态	2.5	3	5	0.611	0.890
平均风	8	4	2	0.583	0.527
土热通量	4	6	6	0.465	0.559
短波辐射	2.5	8.5	7	0.410	0.304
长波辐射	7	7	8	0.264	0.232
气压	9	8.5	9	0.118	0.015

注：各路线名次由三窗归一名次得分平均后再排序，S_g 由式(5)给出。频率为三窗内有效共识重采样的 TOP5 份额均值，每窗分母 44、请求数 100；它描述排序稳定性。

图2将实际轨迹与相对背景位置并列，体积含水率均以 $\mathrm{m^3\,m^{-3}}$ 计。0.1 米深度的 168 小时中位数为 0.12103，处于季节背景约 0.822 的位置；同时首末值由 0.1647 降至 0.08275，线性拟合显示每 24 小时下降约 0.01222。因此，“背景位置较高”与“近期下降”同时成立，土水排名靠前应结合含水状态、梯度和趋势理解，不据此推导近期持续增湿。

2 米气温的 24、72 小时中位数背景百分位分别为 0.883 和 0.867，168 小时为 0.678；0.1 米土温却由 24 小时的 0.521 降至 168 小时的 0.196 背景位置。空气与土壤热状态呈现不同尺度的响应。相对湿度对应百分位为 0.347、0.392、0.759，平均风速为 0.357、0.301、0.262，也体现窗口依赖。族分数包含梯度、尾部和持续性，因此某个代表中位数居中并不排除整个物理族具有高条件异常。

3.4　稳定性、第五席竞争及后续使用

采用按年份和过程分层的 100 次重采样。A、C 每窗均有 89 次有效结果；B 的逐年交叉校准要求至少四个不同重采样年份，因而 B 及三路线共识每窗均为 44 次有效，其余结果保留为空。图3按各自有效分母展示三窗平均频率。共识前四项的入选频率为 0.814–0.890，第五项平均风为 0.527，而第六项土热通量为 0.559。共识顺序反映原样本名次，

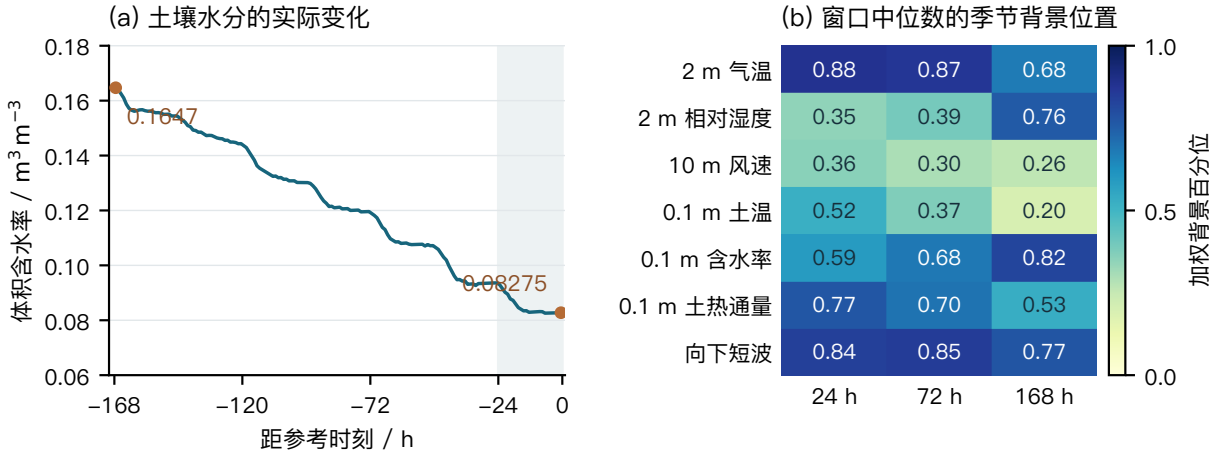


图 2　土壤水分的实际变化与代表变量的背景位置。参考时刻为 2026 年 8 月 26 日 02:30 UTC；右图为各变量窗口中位数的加权背景百分位，固定代表量用于说明方向，不替代全族排序。

重采样频率反映条件稳定性，两者共同支持将前四项作为核心候选，并保留平均风与土热通量的竞争关系。

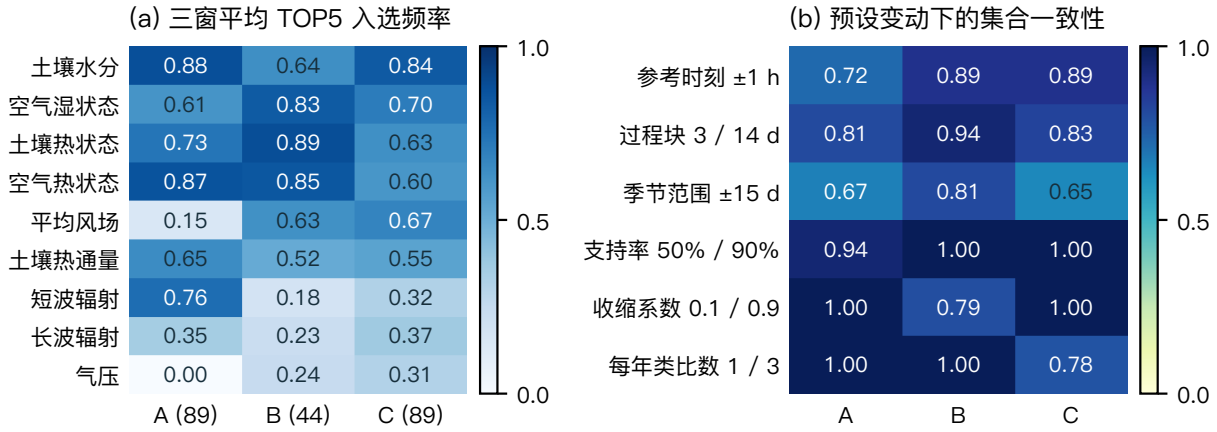


图 3　三路线的重采样稳定性与预设敏感性。左图括号为每窗有效重采样数，请求数均为 100；右图按三窗及行内所列变动取 TOP5 集合 Jaccard 系数的均值，取值越大表示与主设置越一致。

敏感性同时覆盖参考时刻前后 1 小时、过程块长度、季节范围、有效支持率、收缩系数与类比配额，全部保留三种窗口，不依据结果挑选最佳窗口。探索轨加入降水及通量信息后，前五项集合保持一致，空气湿与土水的次序互换；部分湍流族的 B 校准缺少有效贡献，分数保留为空，通量信息据其实际支持范围解释。

短波辐射在 A 中并列第二，在 B 和 C 中的名次下降，提示其异常对比较条件敏感；平均风则由 A 的第八升至 C 的第二，体现条件对照的价值。第五席的分歧因此应作为后续机制比较的切入点，分别考察动力交换与地表传热路径，而非按单一路线删去其一。

本问据此向后续分析提供“空气温湿—土壤水热—风与土热通量”的候选结构、变化方向和尺度依赖。区域站观测提供背景约束；源区地形、物源、冰岩结构及孔隙水压力需由空间资料与机理模型共同描述。候选前五项用于安排水热路径的比较，同时保留降水等机制相关输入，由结构稳定性分析判断其在给定源区条件下的作用。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

4　结构水热耦合的触发条件与机制评估

气象因素通过界面温度、裂隙储水和结构损伤作用于坡体稳定性。为区分这些传递路径，本文建立共享几何与初态的结构基础模型 S、结构加热模型 ST、结构加水模型 SH，以及结构热水耦合模型 STH。四组统一输出稳定裕度、临界水头和首次机械越阈区间，使因素关联进一步转化为可计算的力学条件。

4.1　共同力学骨架与条件参数

以潜在结构面上的单位面积平面板表示源区承载体。设滑面倾角为 θ，法向厚度为 h，岩石体积分数为 r，均质密度为 $\rho = 2700r + 917(1-r)$。岩桥面积比例 f_b 与整体组分 r 分别描述承载结构和质量组成。法向应力 σ_n 与驱动剪应力 τ_g 为

$$\sigma_n = 10^{-3}\rho g h\cos\theta, \qquad \tau_g = 10^{-3}\rho g h\sin\theta, \quad g = 9.80665\ \mathrm{m\,s^{-2}}. \tag{6}$$

式中应力单位为 kPa。引入界面温度 T、黏聚损伤比例 $D \in [0,1)$ 和孔压 u，将冰填节理与岩桥的黏聚、摩擦贡献按面积混合：

$$\begin{aligned} C(T,D) &= (1-D)\{f_b c_b + (1-f_b)c_j(T)\},\\ \mu_e(T) &= f_b\mu_b + (1-f_b)\mu_j(T),\\ R &= C(T,D) + \max(\sigma_n - u, 0)\mu_e(T), \qquad M = \frac{R-\tau_g}{\tau_g}. \end{aligned} \tag{7}$$

$M > 0$ 表示抗力高于驱动力，$M = 0$ 为临界面。损伤只衰减黏聚项；有效应力的非负截断表示失去压接后不继续产生负摩擦抗力。

冰填节理采用 Mamot 等的快速剪切试验关系 [6]：

$$c_j(T) = 53.3 - 73.5T \quad (\mathrm{kPa}), \qquad \mu_j(T) = 0.42 - 0.21T. \tag{8}$$

其实验对象为冰填石灰岩节理，温度为 -8 至 $-0.5\,^{\circ}\mathrm{C}$、名义法向应力为 100–400 kPa，平均应变率为 $(4.8 \pm 1.4)\times 10^{-3}\,\mathrm{s^{-1}}$。计算同时保留线性延伸和暖端岩接触两种策略；后者在 $T > -0.5\,^{\circ}\mathrm{C}$ 时置 $c_j = 0$、$\mu_j = 0.60$，用于检验接触形式变化的影响。两策略沿用同一计算引擎，实验数值域标记随状态输出。

表 5　共同参数设计及单位；范围用于空间填充比较，非概率先验

过程	参数及设计范围	参数及设计范围
结构几何	θ : 30–60°	h : 10–60 m
组成与岩桥	r : 0.2–0.8；f_b : 0–0.25	c_b : 300–3000 kPa；μ_b : 0.45–0.85
热初态与边界	T_0 : −6–−0.8 °C	ΔT : −18–−6 °C
传热路径	α : (0.9–2.1) $\times 10^{-6}$ m^2/s	z : 0.5–20 m
储排水	S_y : 0.002–0.08；τ_0 : 3–240 h	H_{conn} : 2–40 m
水量输入	m_R : 0.25–3	k_m : 0.5–6 mm/(°C d)
损伤演化	A : 10^{-7}–2×10^{-4} h^{-1}	n : 2–6；D_0 : 0.05–0.7；κ : 0–3

表5给出 19 维条件参数域。其中 α 的端点取自外部基岩热扩散率中位数及 MAD 范围 [7]；几何、岩桥、站源映射和水力参数用于情景对照。固定种子生成 256 个共同空间填充设计点，逐点配对四机制与两强度策略，形成 $256 \times 4 \times 2 = 2048$ 条情景。边界覆盖 2026 年 6 月 1 日至 8 月 27 日（UTC）的 2089 个小时状态，$W_0 = 0$。温度使用 QC0 观测；降水按条件小时率解释，经站源倍率和液态分配进入模型。

4.2 热传导、储排水与损伤的状态耦合

令站点空气温度为 T_a，映射到源区的空气边界为 $T_s = T_a + \Delta T$，冰接触表面边界取 $B = \min(T_s, 0)$。对常扩散率、均匀初温和一维半无限导热，本文采用分段常值边界的阶跃叠加作为线性传热数值参考：

$$T(z, t_k) = T_0 + \sum_{j<k} \Delta B_j \, \operatorname{erfc}\left(\frac{z}{2\sqrt{\alpha (t_k - t_j)}} \right), \qquad \Delta B_0 = B_0 - T_0, \quad \Delta B_j = B_j - B_{j-1}. \tag{9}$$

时间差在式(9)中换算为秒，零时差核取零；第 j 小时边界只作用于之后的状态。因果卷积采用充分零填充，消除循环卷积和热空间网格误差。以 $\alpha = 1.5 \times 10^{-6}\,\mathrm{m^2/s}$、72 h 地表阶跃为例，2 m、5 m 深度温变仅分别为表面幅度的 0.0233、1.43×10^{-8}，表明短期纯导热对深部变化具有强过滤作用。

裂隙水路将降雨、融水与孔压相连 [8]。以 W 表示水当量储量、S_y 表示储量与水头之比，条件有效输入为

$$I_i = \frac{m_R P_i}{1000}\, \chi(T_{s,i}) + \frac{k_m \max(T_{s,i}, 0)}{24000}, \qquad \chi(T_s) = \min\{1, \max[0, (T_s + 1)/2]\}. \tag{10}$$

P 以 mm/h、I 以 m/h 计。融水使用截断前的 T_s，故 SH 中仍包含温度驱动的外部融水输入，而其内部界面温度固定为 T_0。定义水的单位水头压力 $\gamma_w = 10^{-3}\rho_w g = 9.80665\,\mathrm{kPa/m}$，其中 $\rho_w = 1000\,\mathrm{kg/m^3}$。水力与损伤状态满足

$$\begin{aligned} \dot{W} &= I - W/\tau_d - Q_{\mathrm{ov}}, \quad u = \gamma_w W / S_y, \quad \tau_{d,i} = \tau_0 e^{-\kappa D_i}, \\ W_{\max} &= S_y \min(H_{\mathrm{conn}}, 0.98\sigma_n/\gamma_w), \\ D_{i+1} &= 1 - (1 - D_i) e^{-A q_i^n \Delta t_h}, \qquad q_i = \min\left(\frac{\tau_g}{\max(R_i, 10^{-6})}, 10 \right). \end{aligned} \tag{11}$$

Q_{ov} 是达到容量后的连续溢流率；水头以 m、孔压以 kPa 计，$\Delta t_h = 1$ h。每步固定输入和排水时常，采用水库分段解析更新并逐步记入排水、溢流；损伤上限取 $1 - 10^{-6}$，抗力分母下限取 10^{-6} kPa。现象学应力比损伤描述累积黏聚损失，不等同于慢蠕变材料定律。温压降低抗力，抗力影响损伤，损伤再改变排水时常，构成闭合反馈，而非将温度评分和降水评分相加。计算同步检验“末态储水等于初态储水加输入减排水与溢流”的水量账，全部情景最大绝对残差为 1.15×10^{-14} m。这一记账将储水抬升与快速泄排两种响应分开，使孔压变化能够追溯到给定补给和连通条件。

4.3 临界水头与首次机械越阈

在给定 T, D 下，式(7)可化为条件临界水头：

$$H_c(T,D)=\frac{1}{\gamma_w}\left[\sigma_n-\frac{\tau_g-C(T,D)}{\mu_e(T)}\right],\quad C(T,D)\le\tau_g,\ \mu_e(T)>0. \tag{12}$$

正根还须满足 $H_c\le\min(H_{\mathrm{conn}},0.98\sigma_n/\gamma_w)$ 才能由本水路达到；负根表示干态已临界，$C>\tau_g$ 时仅减小压接摩擦不能越阈。由此将“有根”“容量可达”“干态已越阈”区分，避免把任意正水头视为触发条件。例如固定初始损伤的代表设计 p0036 在 $-6\,^{\circ}\mathrm{C}$ 时属于无单独水压根；升至 $-3\,^{\circ}\mathrm{C}$ 时临界水头为 11.36 m，高于该设计的 7.83 m 连通容量；在 $-1\,^{\circ}\mathrm{C}$ 时则转为干态已临界。相同结构在不同热状态下跨越不同临界类型，体现了温压路径共同决定的条件性。

各情景从统一起点寻找 M 首次由正转为非正的相邻小时区间，区间内线性插值只提供数值定位；初始 $M\le 0$ 单列为左删失，整段保持正裕度则记为未越阈。S 固定 T_0 并取 $u=0$；ST 开启界面导热，SH 开启储排水，STH 同时开启两路径。四组均保留损伤，使对比对应新增过程，而非不同的结构底座。

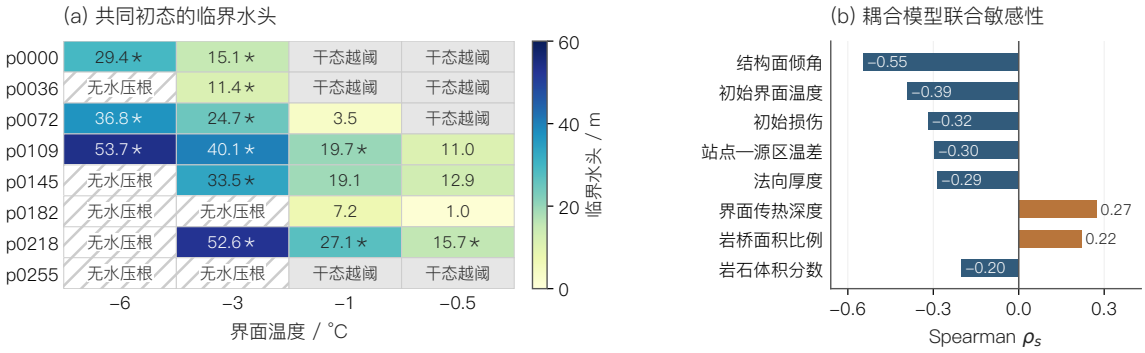


图 4 条件临界水头与耦合敏感性。左图取八个固定初始损伤的设计代表；星号表示正根超出连通容量，斜线格无单独水压根，灰格为干态已临界。右图为解析热 STH 线性分支中评估期最小裕度的联合设计 Spearman 关联。

4.4 共同设计的机制比较与数值支持

表6汇总解析热线性强度分支。相同结构基础下，开启内部热路径使后续越阈设计点由 4 增至 28，单独水力路径为 6，耦合路径为 31；相应评估期最小裕度中位数由 0.6901 降至 0.4718。热路径在该参数域改变更多设计点的稳定状态，水力路径则通过有效应力和损伤反馈进一步调节临界接近程度。

两强度策略合计得到 137 个后续越阈、248 个初始左删失和 1663 个未越阈情景。暖端接触策略仅将 ST 的后续越阈数由 28 改为 27，其余组计数相同。图4的联合设计中，STH 最小裕度与倾角、初温、初损伤、站源温差、厚度的相关系数依次约为 -0.545、-0.390、-0.318、-0.296、-0.285；这些方向说明较陡、较暖及较高初损伤的条件更接近临界，排序随规定参数范围解释。

解析热与 0.0625 m 有限差分参考的 137 个有限时刻配对中，最大越阈时间差为 0.347 h，类别保持一致；与 0.125 m 网格的最大差为 19.0724 h。因而本文以解析热作为所声明

表 6　解析热线性强度分支的共同设计结果，每行 256 点

机制	后续越阈	初始左删失	未越阈	最小裕度中位数
S	4	31	221	0.6901
ST	28	31	197	0.4853
SH	6	31	219	0.6623
STH	31	31	194	0.4718

越阈分类覆盖全计算期；最小裕度取 8 月 19–27 日。设计点计数和分位数描述条件参数域，不赋予事件概率；本地独立灾害验证事件数为 0。

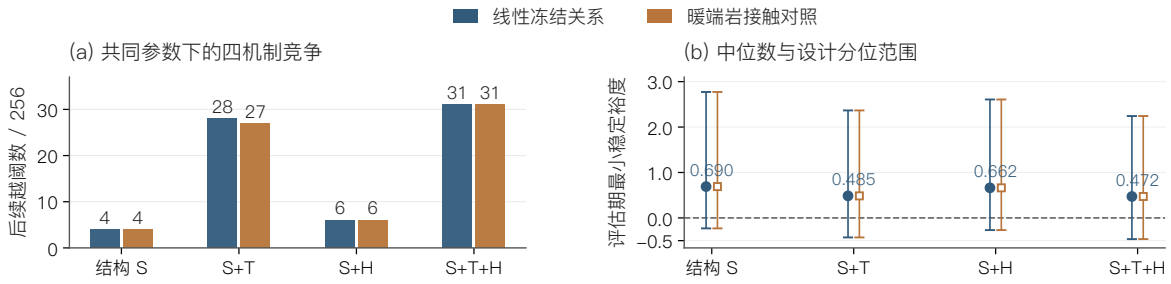


图 5　四机制在相同设计下的越阈与稳定裕度响应；两种暖端强度策略并列，区间表示设计点的 5%–95% 分位。

线性导热问题的参考，同时保留小时水量和损伤更新的离散支持。暖端不连续切换仍可造成最大裕度差 0.1790，热模块精度与整体机械时刻精度分别讨论。

在本设计的深度、扩散率和边界序列下，将半无限解与 40 m 无通量底边界比较，边界总变差乘首反射核得到的最大温差界为 $2.38\times10^{-26}\,^{\circ}\mathrm{C}$。因此这一计算期内的理想底边界选择影响很小，现场解释主要依赖表面映射、初始热状态以及裂隙水热输运等过程条件。

进一步对岩桥强度、损伤、水头、温度四未知量作局部雅可比分析，单裕度、同条件重复裕度、增添温度、温度加压力、再增理想损伤观测时的秩依次为 $1,1,2,3,4$。这一结果将监测重点指向同一结构面的温压与岩桥损伤联合约束，而非仅增加相同条件下的采样次数。

4.5　暖域高应力的外部本构挑战

为检验式(8)的应力迁移，本文完整读取官方补充表的 141 次试验和 14 条派生温层摘要，后者用于作者关系复核，逐试验数据用于新拟合。名义 800 kPa 组有 20 次试验，面积修正后实测应力为 808.89–868.63 kPa；其温度设计仅覆盖 -4 至 $-0.5\,^{\circ}\mathrm{C}$。按名义温应力组划分数据，拟合使用破坏时实测列，保留重复标签的身份歧义和相关性敏感性。

四种固定候选在四域、两权重下形成 32 个拟合、576 个整组留出折。五暖温层的全应力共同域含 83 次试验；图6按整温层、整应力层比较跨条件泛化。温度线性 MC 的训练 RMSE 为 187.45 kPa，两类留出的组等权宏 RMSE 为 198.41、296.34 kPa。应力二次和 400 kPa 铰链虽将训练误差降至 174.32、177.41 kPa，仅留 800 组时却分别增至 925.78、2065.13 kPa，显示高应力分支外推的敏感性。

表 7　共同暖域本构复杂度与应力层外推误差（kPa）

候选	参数数	训练 RMSE	仅留 800 组 RMSE
温度线性 MC	4	187.45	443.94
温度二次 MC	6	185.39	421.07
应力二次	6	174.32	925.78
400 kPa 铰链	6	177.41	2065.13

主权重为逐试验 OLS；仅留 800 组时，其余 63 次暖域试验训练、20 次高应力试验评价。

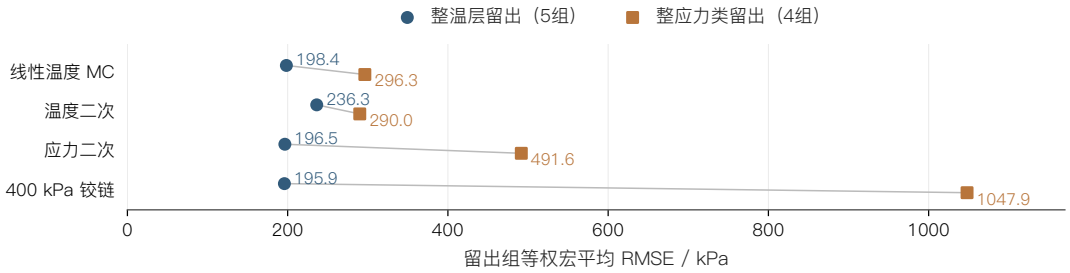


图 6　83 次暖域外部试验的分组留出比较。宏 RMSE 为各折 RMSE 平方的等权均值再开方；作者 Eq6 另作描述性复核，不纳入留出比较。

作者在共同暖域内的低应力与全应力拟合分别选取三、四个显著温层，因此系数变化同时包含应力覆盖与筛温影响。本文另以完整五暖层进行比较，保留温度线性 MC 作为低复杂度外部挑战；本地计算继续使用原式与原版本。条件 bootstrap 均值范围、试验重复相关性和跨材料迁移分别记录，外部试验数不加入本地灾害验证分母。以原实验的名义坐标 $T=-2\,^{\circ}\mathrm{C}$、$\sigma=800\,\mathrm{kPa}$ 为例，暖域线性 MC 的条件均值为 659.30 kPa；单元内逐试验与整标签重采样的均值 2.5%–97.5% 分位范围分别为 614.49–714.29、607.58–691.88 kPa。

八个重复标签的物理试件映射保持未知，两种方案用于展示相关结构敏感性；这些范围描述给定模型下的均值变动，个体散差与现场迁移另作解释。

综上，结构水热框架给出了从气象输入到界面状态、稳定裕度和条件临界水头的完整链条。STH 用于统一状态推进，S、ST、SH 用于辨别传递路径；源区首次机械越阈与整体释放、下游到达分别交接。该设计将机制比较与后续时刻预测相联，并将最有价值的新增观测明确为界面温度、连通水压和岩桥损伤信息。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

5　三引擎时间预测与分层预警响应

5.1　预测对象、信息集与评价设计

针对源区启动时间及下游预警需求，建立机械首次过阈、变形零点外推和动态风险三个互补引擎。本地对象是源区首次机械启动；外部位移资料的标签是终末或主体失稳；传播响应以给定整体释放为起点。本地日历时刻 t 统一用 UTC，外部记录以原资料相对小时 s 计算，不补写未知的 UTC 偏移。对发布时刻 t_0 或 s_0，B/C 的信息集 $\mathcal{F}_0$ 仅含发布

前观测；A 以给定历史重建状态为起点，分列假设可得天气、理想对照和严格首发审核。统一报告 $H \in \{24, 48, 72\}$ h 内的事件质量、不发生质量及条件时间区间。若 τ 为距发布时刻的等待时间，模型内中央区间定义为

$$p_H = \mathbb{P}(0 < \tau \le H \,|\, \mathcal{F}_0), \qquad I_H^{95} = [Q_{.025}, Q_{.975}]\,(\tau \,|\, 0 < \tau \le H, \mathcal{F}_0). \tag{13}$$

区间端点是相对时长，需加各自发布时刻；其名义水平与观测标签的实际覆盖率分别计算。

外部资料由历史岩坡监测集和 Brienz 公开记录构成 [9, 10]，按物理过程合并为 13 个终末失稳组，另以 Stampa 的 725 天真实观测作为一个右删失组 [11]；Brienz 两传感器属于同一组。风险模型的确知生存暴露合计56 381.84小时，失稳标签的时间不确定性单独进入似然。采用外层留组、训练侧选参，并在同一传感器、发布锚点和时域上配对比较；连续小时及传感器数量均不增加独立事件分母。本文按本地条件计算、外部分组评价与释放后传播组织三层证据，各层采用对应的时间对象与评价资料。

5.2　三条时间预测路线

A：机械首次过阈与信息上下界。　沿用第二问的失稳裕度 $M_\theta = F_{S,\theta} - 1$，将结构、解析导热、储排水和损伤状态共同暖启动，定义

$$\tau_\theta(t_0) = \inf\{h > 0 : M_\theta(t_0 + h) \le 0\}, \qquad \inf \varnothing = +\infty. \tag{14}$$

实际以逐小时首次非正裕度定位跨阈，并在相邻小时间线性插值；完整随访 H 内无跨阈才判定该时域不启动，随访不足归入未知。256 组共同参数、4 种机制和 2 种强度策略组成 2048 条设计路径。未来边界采用三份 18UTC 起报的实际 GFS 产品 [12]。75 次逐小时发布各设 10 条输入轨：两种空间摘要各配 0/3/6/9 小时假设延迟，另设 1 条理想轨和 1 条严格首发轨。严格轨的首次公开可用时间为空时保持未知。由此产生1 536 000条参数路径记录，以及 $75 \times 10 \times (8 + 1) \times 3 \times 3 = 60\,750$ 组分布；后三项为机制政策及混合、设计先验和时域。

以归一权重 w_θ 表示给定设计先验，固定原条件历史下的风险集 $\mathcal{R}$，令 $R = \sum_{\mathcal{R}} w_\theta$。其中 E_H, N_H, U_H 分别为域内已知启动、域内已知不启动和结局未知的质量，满足 $E_H + N_H + U_H = R$。A 的 p_H 条件于 $\theta \in \mathcal{R}$，从而

$$\frac{E_H}{R} \le p_H \le \frac{E_H + U_H}{R}, \qquad R > 0. \tag{15}$$

原已启动质量保持在风险集外；只有 $U_H = 0$ 时给出精确概率，并在事件质量为正时求式(13)的分位数。材料域按发布后实际随访的 $L + 1$ 个状态离线诊断，要求全部满足 $T \in [-8, -0.5]\,^\circ$C 及有效正应力 100–400 kPa，包含跨阈后状态 [6]；原未来跨阈支撑集中于 p0149-S，其有效正应力为 820.31 kPa，相应结局质量归入 U_H。三时域各20 250组主对照均保留正未知质量；启动下界均为 0，上界的总体最小值在 24/48 小时为 0.488584，72 小时为 0.497717，最大值均为 1。这给出了指定设计的**信息界**，精确概率与条件分位数记空。同随访天气对照进一步分解边界替换的作用：在 72 小时终点，各设计中的最大界面

温差为 2.78954 °C，无量纲裕度差为 1.07956；随访内峰值孔压差最大为 40.86915 kPa。终点状态与过程峰值分别比较，说明天气能够改变水热状态，而当前发布窗的离散跨阈支撑仍由同一结构分支主导。因此用状态响应评价天气驱动、用信息界评价启动结局，能够保留两种输出各自的分辨能力。

B：逆速度与状态空间外推。 由发布前最近 5 点位移回归得到速度绝对值；Weissmiess 直接将已给日速度换算为小时速度。取最近 16 个有限正速度的归一逆速度 y_i。逆速度线性化依据加速失稳的零点关系 [13]，本研究分别实现普通最小二乘 IV 与局部趋势状态空间 SS：

$$\begin{aligned} &\text{IV}: \quad y_i = a + b(s_i - s_0) + \varepsilon_i, \qquad \tau = -a/b \ (a>0, b<0), \\ &\text{SS}: \quad z_i = F(\Delta_i) z_{i-1} + \eta_i, \quad y_i = (1,0) z_i + \epsilon_i, \\ &\qquad F(\Delta) = \begin{pmatrix} 1 & \Delta \\ 0 & 1 \end{pmatrix}, \quad Q(\Delta) = r\rho \begin{pmatrix} \Delta^3/3 & \Delta^2/2 \\ \Delta^2/2 & \Delta \end{pmatrix}. \end{aligned} \tag{16}$$

SS 按实际不等间隔 Kalman 递推 [14]；$r = \max\{\text{median}_i[(y_i - y_{i-1})^2]/2, 10^{-8}\}$，$\rho = 0.01\,\text{h}^{-3}$。前 3 点初始化，其余点顺序更新至发布时刻。IV 采用回归系数协方差，SS 采用发布时状态协方差，各从联合高斯估计分布中抽取 512 个样本，条件为当前逆速度为正；非负趋势对应 $\tau = +\infty$，落入 $(0, H]$ 的比例给出 p_H，该时域内的根给出条件区间。该分布表达当前估计不确定性，SS 不另模拟未来过程扰动。历史不足、观测过旧或当前逆速度均值非正时保留弃权。

C：无记录年龄的动态风险。 令 v_j 为最近 6 次观测的回归斜率绝对值，b_j 为最近至多 64 次观测的相邻速率绝对值中位数，c_j 为最近 5 段之前的 6 段速率中位数（不足 11 段时取 b_j），$\epsilon_j = \max(10^{-6} b_j, 10^{-12})$。构造两个无量纲特征

$$x_j = \text{clip}_{[-12,12]}\left(\log \frac{v_j + \epsilon_j}{b_j + \epsilon_j},\ \log \frac{v_j + \epsilon_j}{c_j + \epsilon_j} \right). \tag{17}$$

再经训练侧中位数及 $\max(\text{IQR}, 0.25)$ 缩放并截断为 $z \in [-6, 6]^2$。线性风险与非线性 RBF 竞争，其中 RBF 基为

$$\phi(z) = \left(1, z_1, z_2, \{e^{-(z_j - c)^2/2}\}_{j=1,2;\ c=-1,0,1}, z_1 z_2\right), \quad \lambda(s) = e^{\text{clip}(\beta^\top \phi(z(s)), -35, 20)}\,\text{h}^{-1}. \tag{18}$$

风险暴露从第 6 次观测起算，失稳区间内固定标签下界前最后状态。以暴露起点为累计风险零点，$\Lambda_i(u) = \int_{s_{i,\text{entry}}}^{u} \lambda_i(s)\,\mathrm{d}s$，$S_i(u) = e^{-\Lambda_i(u)}$。失稳区间 $[L_i, U_i]$、精确标签 T_i 和右删失 C_i 的似然贡献分别为

$$\mathcal{L}_i = \begin{cases} S_i(L_i) - S_i(U_i), & L_i < U_i, \\ \lambda_i(T_i) S_i(T_i), & L_i = U_i = T_i, \\ S_i(C_i), & \text{右删失}. \end{cases} \tag{19}$$

分段指数风险以局部常值积分连接暴露与结局 [15]，岭惩罚参数由训练组内留组选取。预测时固定发布前的最后特征，因而

$$p_H = 1 - e^{-\lambda_0 H}, \qquad Q_q(\tau \mid 0 < \tau \le H) = -\lambda_0^{-1} \log\!\left[1 - q(1 - e^{-\lambda_0 H})\right]. \tag{20}$$

主候选为无记录年龄的 RBF raw；记录年龄支线只诊断事后选窗的时钟效应。512 与 1024 节点对照保留同一风险集，原定义锚点上 RBF raw 的最大概率变化为 0.010412，48/72 小时 Brier 变化为 0.002959/0.002147。该对照保持同一物理样本，量化积分网格带来的数值敏感性。

5.3 分时域结果、覆盖率与提前量

发布锚点按各组随访终点减 H 回顾选取。从 14 个物理组中，仅在两模型均发布概率且结局已知的相同机会比较，结果见表8；弃权保留为空，72 小时 Weissmiess 无可发布锚点。24 小时的 5 个可评分组均为阴性，分数主要反映该时域的不报警质量；48/72 小时各有 4 个阳性、4 个阴性。SS 在 48 小时分数较低，RBF raw 在 72 小时较低，支持按监测条件和时域分工。C 全部 10 个已知结局组的 48/72 小时分数为 0.373917/0.307760；B/C 配对比较采用各时域共同组，24 小时为 5 组，48/72 小时各为 8 组。表中分数按组等权计算为 $\mathrm{BS}_H = n_H^{-1}\sum_{i=1}^{n_H}(p_{i,H} - o_{i,H})^2$，其中 $o_{i,H}$ 仅取确知的 0 或 1；该比较描述所选锚点的有效配对子集。

表 8 外部同发布锚点、同有效组的 Brier 分数（越小越好）

H/h	共同独立组数	阳性/阴性	SS	无年龄 RBF raw
24	5	0/5	0.046924	0.005908
48	8	4/4	0.242887	0.308827
72	8	4/4	0.346922	0.243395

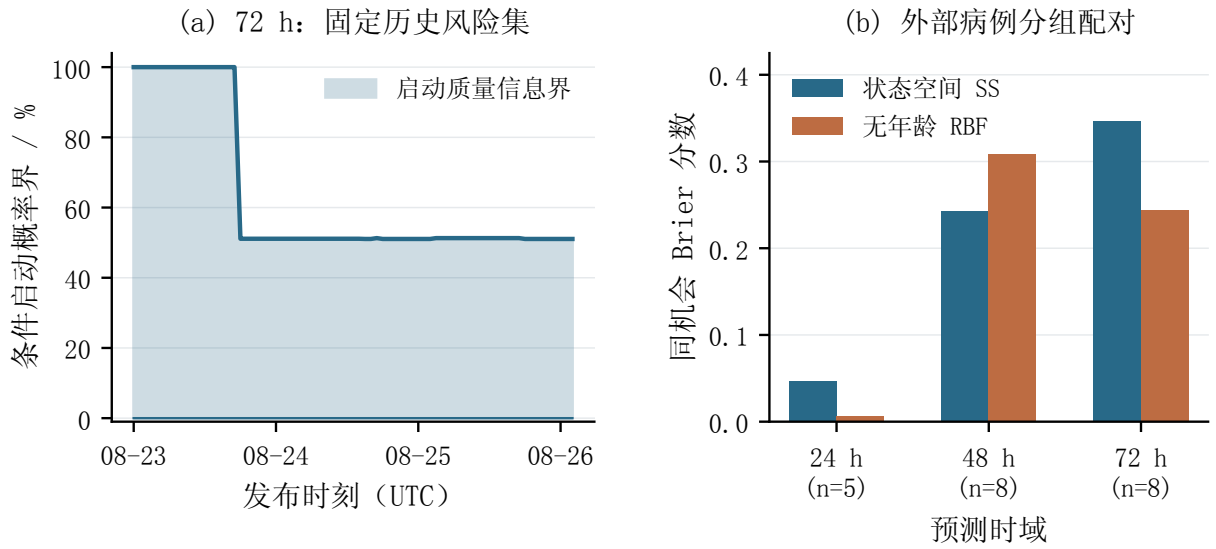


图 7 不同证据层的风险表达。(a) 设计等权、机制混合、AOI 等权平均及假设零延迟下的 72 小时信息界，24/48 小时在该组合中同形；阴影为条件于原历史风险集的启动概率信息界。(b) 表8的外部配对分数，n 为有效独立组数。两图对象分别为本地条件启动与外部终末失稳。

发布尺度同样影响方法比较。固定共同 10 个组后，48 小时 IV/SS 的等组 Brier 在每小时回放为 0.17199/0.22659，在最后 48 小时原生发布为 0.53427/0.49009；72 小时分别为 0.20396/0.24340 及 0.45039/0.42196。两种回放还覆盖不同的观测时期，因此将其解释

为发布支持与尺度的联合敏感性，而非单一采样频率效应。

将等待时长转换为源时钟区间 $\widetilde{I}_i = s_{0,i} + I_{H,i}^{95}$。对失稳标签 $J_i = [L_i, U_i]$，完整包含为 $J_i \subseteq \widetilde{I}_i$，相交为 $J_i \cap \widetilde{I}_i \neq \varnothing$；完整覆盖、相交覆盖给出区间标签下的保守与宽松统计量。表9在标签下界前 24 小时固定发布 72 小时预测，每组固定主传感器，无区间者仍留在 13 个病例机会分母中。

表 9　固定截面中央 95% 模型区间的外部表现

模型	有效概率	有限区间	完整包含	相交	点误差中位数/小时
IV	10/13	7/13	3/13	5/13	21.15（7 组）
SS	11/13	8/13	6/13	6/13	12.55（8 组）

点误差为预测中位数到标签区间的距离，表列为跨组中位数；括号为有限点分母。

该截面的有限区间平均宽度分别为 39.28 小时（7 组）及 44.96 小时（8 组）；24 小时发布提前量来自锚点设计。另在标签前 30 天每小时回放，以 $p_{72} \geq 0.5$ 产生告警。表10区分首个**时域内正确告警**与首个任意告警；前者要求完整标签已落入未来 72 小时，后者包括远早于该时域的报警。提前量距标签下界计算，仅在对应有告警组中取中位数；滚动 Brienz 按两传感器最早告警合并，固定截面使用主传感器。这种并列报告将及时性与早报分别计量，且总体仍是事先选入的失稳病例。

表 10　72 小时滚动规则下两种首警统计

模型	时域内正确告警	中位提前量/小时	任意首警	中位提前量/小时
IV	9/13	60.24（9 组）	12/13	464.50（12 组）
SS	9/13	62.24（9 组）	11/13	696.00（11 组）

病例分母为 13 个失稳组；括号为相应提前量中位数的有效组数。

共同锚点还存在标签上界恰等于时域末端的情形：连续条件分布的 97.5% 分位小于 H，使完整包含率为 0 而相交率可较高。故同锚点评分用于公平概率比较，表9另列完整标签覆盖。共形层按独立校准分数的秩 $k = \lceil 0.95(m+1) \rceil$ 检验可达范围 [16]。实际另留独立校准组为 0；即便把外层训练的至多 13 组作为乐观上界，也有 $k = 14 > 13$。该计算给出无界的秩可行性结果；本文据此分别呈现模型内条件 95% 区间、实际外部覆盖与校准秩。

5.4　从机械时钟到给定释放后的空间响应

以同一 base 情景连接机械状态和释放物源，最终形成 7 个运行、4 个机械设计、49 条目标记录。模型、法向厚度、密度等 20 个物理参数逐项保持；紧凑源与全源面、60/120 米网格分别记录。目标到达时钟满足

$$t_{\mathrm{arr},k}^{(\theta)} = t_{\mathrm{init}}^{(\theta)} + \delta_{\mathrm{rel}}^{(\theta)} + \tau_{\mathrm{prop},k}^{(\theta)}. \tag{21}$$

其中机械启动至整体释放的 δ_{rel} 保持未知，传播结果按给定整体释放的相对时钟呈现；图8展示的是给定释放后、100 米代表点缓冲内格中心厚度达到 0.1 米的保存帧括号。

p0000 和 p0072 的机械模型未跨阈，仍只作给定释放响应。p0255 的全历史跨阈位于 7 月 1 日 11–12UTC，后续评价窗则为已不稳左删失，二者分列，不把评价首样本变成窗内新启动。

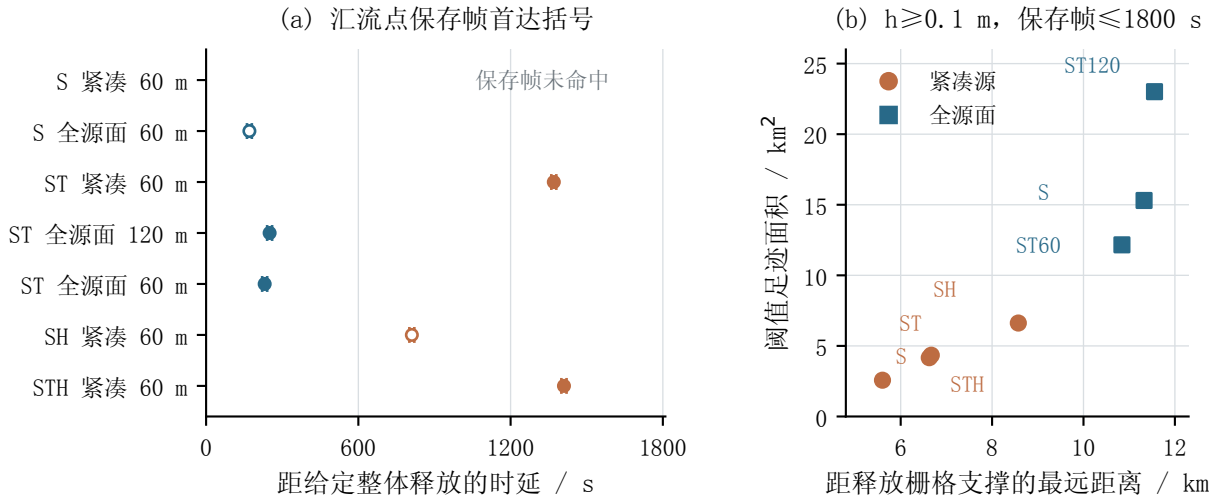


图 8　同 base 情景的条件传播响应。(a) 汇流点括号，空心点为仅给定释放、实心点为机械模型已跨阈；“保存帧未命中”保留为观测状态。(b) 请求 1800 秒内、厚度阈值 0.1 米的足迹面积与距实际正释放栅格足迹的最大格心距离，ST120/ST60 为同体积的两个网格。

同体积 p0036 全源面在 120/60 米下的足迹为 23.0112/12.1608 平方千米，支持距离为 11.554/10.841 千米；紧凑源对应 4.3416 平方千米和 6.670 千米。这些量刻画物源与格网条件下的传播差异，距离采用释放足迹到受影响格心的欧氏定义，物源边界、原生初态与网格差异共同解释；时间联接按同一情景组合机械状态、阶段转换与传播响应。由此，本问输出机械状态驱动的信息界、外部可比较的条件时间区间，以及给定释放后的设施响应三类结果，为气象与变形资料逐步到位时选择相应预警层提供一致接口。业务更新时，气象引擎先更新水热状态与未知质量；新增变形观测再更新零点与动态风险。各层同时携带发布时刻、时域和空间对象，使后续观测能够修正相应分布，而不混合不同阶段的时间标签。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

6　给定释放条件下的泥石流传播与模型比较

6.1　空间构型与释放设计

本节将前述稳定性计算中的物源状态转为流动初始条件，研究流深、速度、展布与下游响应。共完成 24 项一维沿程、10 项自编二维计算，以及 31 项原生求解：单混合物 18 项、复合固水 3 项、D-Claw 孔压 9 项，另有 1 项通用材料诊断。原生队列包含 30 项目标材料计算；7 项材料映射不适用的配置另列。由此形成简化动力学、均质混合物、孔压耦合和相间运动四个层次的过程比较。

地形采用 Copernicus GLO-30，统一至 UTM 45N 及 60、120 m 格网；产品垂直基准为 EGM2008，主体获取于 2011—2015 年 [17]。在候选源面与河谷构型上得到 20.70、21.42、21.66 km 三条地形路径，源边缘接河的局部连接长约 0.710—0.939 km。该构型为源面、

河道与代表目标提供一致的计算坐标；地形年代和图上对象定位作为几何条件保留。

表11将 4 个机械设计映射为 7 个传播运行。紧凑源按 Q2 给出的面积、厚度与体积投影；全源面使用共同候选掩膜。60 m 源格面积为 0.9252 km^2，120 m 为 0.9648 km^2，后者按相同体积归一厚度。主过程选取全源面 p0036–ST / 120 m：体积 31.184139×10^6 m^3，初始垂直厚度 32.322 m，等效密度 2158.626 kg/m^3。各释放初速为零。H 描述结构内的储排水作用，SH、STH 的释放自由水体积分数仍为 0；在这组单混合物运行中，岩冰组分通过体积和等效密度进入传播。

表 11 四个机械设计的释放映射；体积单位为 10^6 m^3

设计	密度 (kg/m^3)	紧凑源体积	全源面体积	运行数	格网 (m)
p0000–S	1298.435	0.559400	51.755729	2	60；60
p0036–ST	2158.626	1.348212	31.184139	3	60；60、120
p0072–SH	1293.058	5.100007	—	1	60
p0255–STH	1708.464	1.255515	—	1	60

格网列依次对应紧凑源与全源面；破折号表示该映射未纳入此组运行。7 次运行共享 4 个机械设计，形成 49 条目标记录，不赋予统计权重。S、SH 保留“若整体释放”的条件。

6.2 深度平均动力学与响应定义

采用 r.avaflow 4.0 Rev7 组织单混合物及相模型计算 [18, 19]，自编有限体积模型提供一维、二维对照。其共同质量形式为

$$\frac{\partial h}{\partial \tau} + \nabla\cdot(h\boldsymbol{u}) = E, \qquad \tau = t - t_{\text{release}}, \tag{22}$$

自编对照采用有限体积形式交换厚度和动量：二维计算保留两个水平分量，一维则沿三条路线转换距离—高程，并用恒定等效宽度组织质量账。相模型对各相保存 $(h_k, h_k u_k, h_k v_k)$，相加得到总厚度；因此，一维的等效宽度面积、二维平面足迹及分相状态分别解释。

其中 h 为流深，$\boldsymbol{u}$ 为水平速度，E 为单位面积夹带源项；t 沿用全文 UTC 时间，τ 为给定整体释放后的秒数。动量过程可归纳为通量、重力、阻力、相间作用及物源交换：

$$\begin{aligned} \frac{\partial \boldsymbol{q}}{\partial \tau} + \nabla\cdot\boldsymbol{F}(\boldsymbol{q}) &= \boldsymbol{S}_g + \boldsymbol{S}_f + \boldsymbol{S}_{\text{inter}} + \boldsymbol{S}_E, \\ \frac{\boldsymbol{\tau}_b}{\rho} &= \left(\mu g_n h + \frac{g\|\boldsymbol{u}\|^2}{\xi}\right)\frac{\boldsymbol{u}}{\|\boldsymbol{u}\| + \varepsilon}. \end{aligned} \tag{23}$$

$\boldsymbol{q}$ 为厚度与水平动量组成的状态向量。第二行给出单混合物对照的 Coulomb–Voellmy 阻力表达；主设置 $\mu = 0.15$、$\xi = 1000$ m/s^2，小量 ε 仅用于零速方向正则化。原生求解器保留自身地形、压力和数值闭合，跨模型比较以各自实际状态量为准。请求时域统一为 1800 s，名义输出间隔 20 s，时间步由 CFL 控制；实际存帧时刻直接读取原生记录，避免以固定间隔替代真实积分时间。

孔压路线采用 D-Claw 1.0[20, 21]，以固相体积分数、剪胀及基底孔压 p_b 耦合流动，有效抗力由 $(\rho g_n h - p_b)\tan\phi$ 控制。本组孔压参考的初始固相体积分数为 0.63，参考渗透率取 10^{-10} m^2；它们是所采用算例设置的迁移参数。复合相路线将岩 0.6、冰 0.1 合为体

积分数 0.7 的固体，其密度为 2445.286 kg/m^3，水相占 0.3；中间固相为空。它描述复合固体与水的相对运动，冰的作用限于密度闭合，相变与岩冰相对速度不在该配置中求解。孔压及复合相参考释放体积均为 13.878×10^6 m^3，与主 ST 物源分别定义。

所有目标使用代表点周围 100 m 内的格心，总厚度阈值统一为 0.1 m。令 $\mathcal{B}_j$ 为该邻域，首个存帧响应为

$$\widehat{\tau}_j = \min\{\tau_k : \max_{\boldsymbol{x}_i\in\mathcal{B}_j} h_i(\tau_k) \geq 0.1\}, \qquad \tau_j \in (\tau_{k-1}, \tau_k]. \tag{24}$$

图中 J、P 分别为汇流与口岸代表点；另 5 个 OSM 几何对应 3 个实体。累计足迹按请求 1800 s 内的保存帧最大厚度统计，速度为湿区水平模长（相模型取湿相最大值），区别于原生逐步累计的地形标量速度。机械启动至整体释放的延迟保持未知，结果据此解释为相对释放响应。

6.3 沿谷传播过程与模型状态

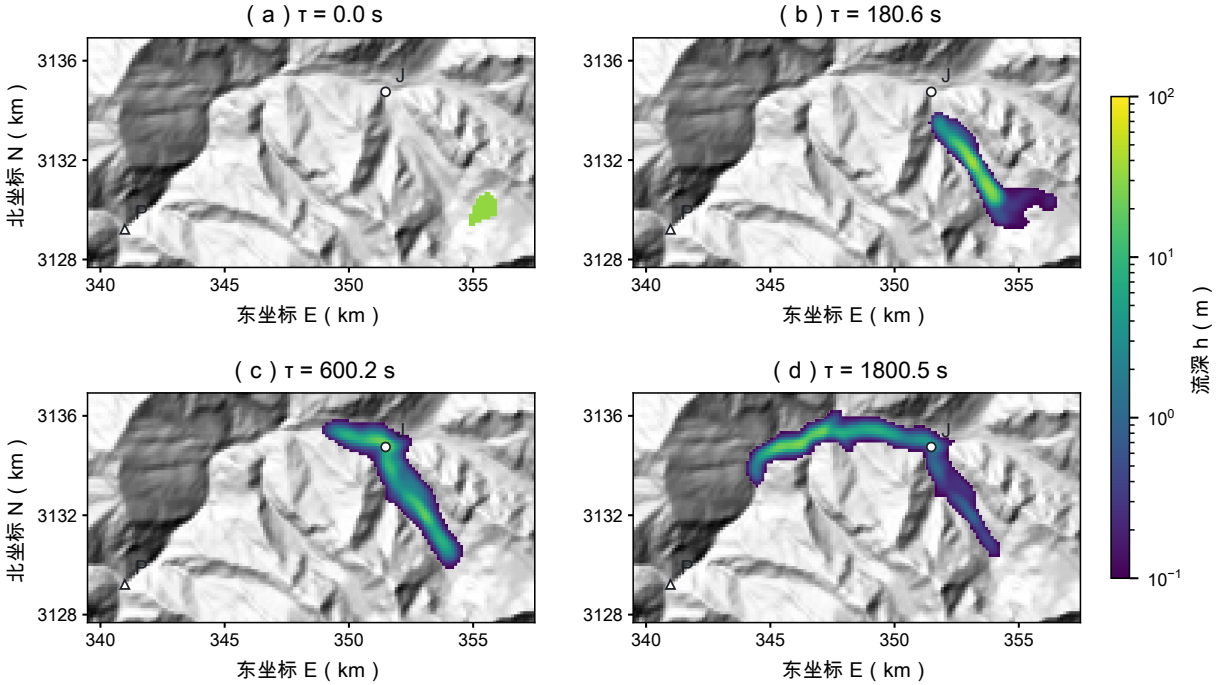


图 9 全源面 ST / 120 m 的流深过程。灰底为地形阴影，色标为对数流深，J、P 为目标代表点；时刻取原生实际存帧，末帧 1800.5 s 单列展示。

图9展示主 ST 运行由源区展布、进入主沟到汇流后沿谷运动的过程。0、180.6、600.2、1800.5 s 的当时湿区面积依次为 0.9648、7.2576、10.4400、13.3056 km^2。180.6 s 时流体仍主要分布在源区下方沟段；600.2 s 已越过汇流代表点并向西展布，末态沿谷形成连续流动带。四帧局部最大流深为 32.322、40.337、22.739、28.366 m，说明局部增厚与整体扩散同时发生；当时湿区面积与全程累计足迹承担不同的过程描述。

主 ST 运行的保存帧峰深为 61.616 m，湿区水平峰速 74.066 m/s，汇流响应位于释放后 (240.6,260.1] s。保持体积、密度与阻力，将格网改为 60 m，峰深增至 80.906 m、水平峰速降至 67.437 m/s，累计足迹由 23.0112 降至 12.1608 km^2；两个汇流括号有 0.1 s 交叠。

图10上排显示，加密格网使主要运动带更窄，局部厚度更集中。因此，局部极值、平面足迹和前缘响应共同刻画传播强度及其尺度变化。

表 12　代表运行的保存帧过程指标

运行	体积 ($10^6 m^3$)	峰深 (m)	峰速 (m/s)	足迹 (km^2)	汇流括号 (s)
ST 全源面 / 120 m	31.184	61.616	74.066	23.0112	(240.6,260.1]
ST 全源面 / 60 m	31.184	80.906	67.437	12.1608	(221.0,240.7]
D-Claw 扩域 / 120 m	13.878	60.899	180.494	83.3904	(40,60]
复合固水 / 120 m	13.878	22.005	66.082	29.3904	(200.0,220.4]

足迹为 1800 s 内保存帧总厚最大值达到 0.1 m 的平面面积，峰速采用湿区水平速度；D-Claw 为整个扩展域，图中仅显示共同核心区域。不同物源、初始化和相闭合分别保持，表内数值用于结构对照。

D-Claw 增加了压力与固相状态的空间解释：扩域运行的汇流响应为 (40,60] s，口岸代表点为 (160,180] s；图10下排给出 180 s 的基底压力场。复合固水运行则输出各相厚度及水相份额，汇流响应为 (200.0,220.4] s。主 ST 的口岸代表点在本时域内没有厚度达阈记录。这些响应共同反映了给定物源和闭合结构的差异；其中 D-Claw 初态积分使源厚支持由 67 格扩为 111 格，厚度归一 L_1 差 13.899%，跨框架速度差因而同时包含初态映射与动力过程的作用。

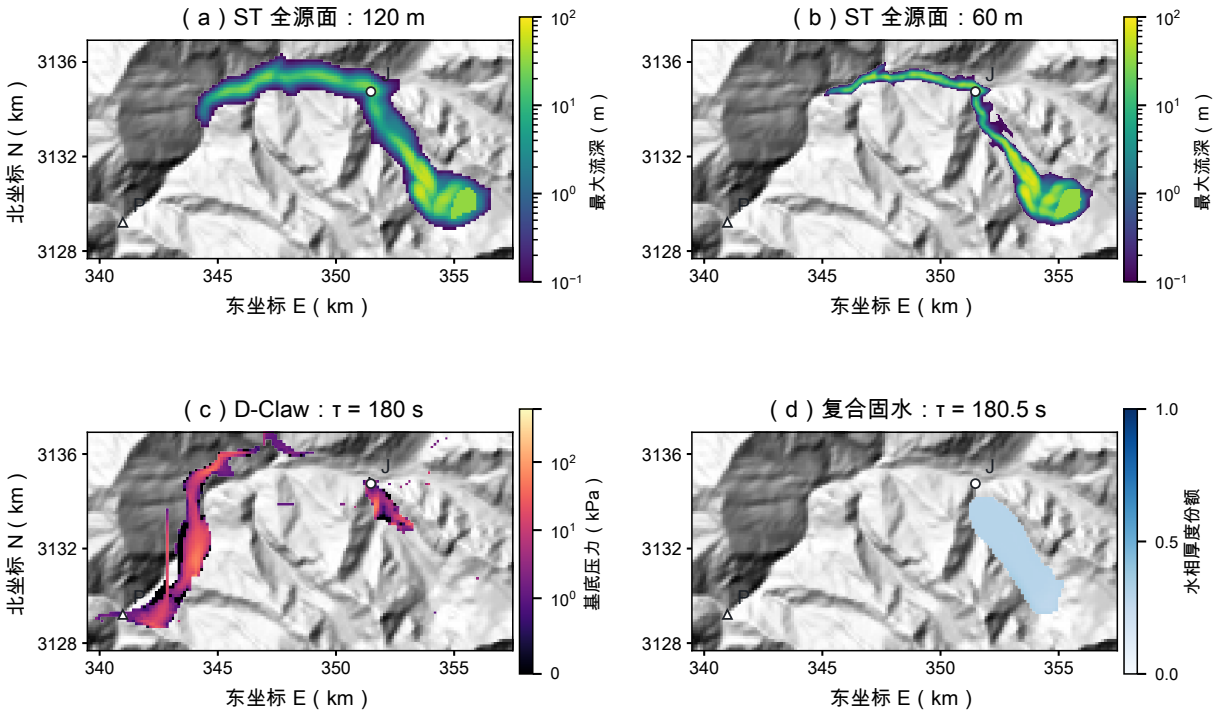


图 10　格网与物理结构对照。上排为 ST 双格网 1800 s 内累计最大流深；下排分别为扩域 D-Claw 的 180 s 基底压力和复合固水的 180.5 s 水相厚度份额。压力色阶在 1 kPa 以上采用对数间隔；下排使用各自 13.878×10^6 m^3 参考释放。

七个跨问运行中，全源面 S 的汇流响应为 (161.1,180.2] s；紧凑源 ST、SH、STH 分别为 (1360.5,1380.1]、(801.1,820.6]、(1400.1,1420.5] s，紧凑源 S 在时域内无达阈记录。几何和物源规模改变了运动带及响应时标，因而 49 条记录按父设计、源面与格网关联保

存，不将不同物源的时延直接合并成概率分布。

6.4 源项、数值响应与过程解释

表13集中比较数值和物理干预。15 m 释放厚度的 120 m 格网参考只将 CFL 从 0.5 减至 0.25，水平保存峰速即由 92.394 降至 53.256 m/s，原生累计地形标量也由 152.356 降至 92.633 m/s。两类速度均呈响应，时间步效应包含存帧采样之外的数值变化。60 m 参考的质量账为 +11.506%，与 120 m 的 -1.026% 相对照，说明格网、初态映射和源面陡坎共同影响预算。自编模型提供另一组证据：含边界项的体积账最大相对残差为 8.05×10^{-16}，而常坡薄流 20、10、5 m 格网速度相对误差依次为 0.755、0.377、0.188；守恒与传播速度应作为互补评价量。

表 13 参考物源的源项、格网与耦合对照

运行 / 干预	水平峰速 (m/s)	末态质量账 (%)	过程响应
单相 120 m，CFL=0.5	92.394	-1.026	汇流 (420.9,441.3] s
单相 120 m，CFL=0.25	53.256	-1.551	汇流 (461.0,480.7] s
单相 60 m，CFL=0.5	59.596	$+11.506$	汇流 (540.1,560.8] s
复合固水，保留耦合	66.082	-10.301	汇流 (200.0,220.4] s
复合固水，关闭 drag / VM	—	-1.984	汇流 (181.1,200.4] s
全岩固体＋水	—	-13.145	汇流 (220.1,240.2] s
D-Claw 扩域	180.494	-4.147	口岸 (160,180] s
单相 120 m，1 m 侵蚀床	—	-1.157	夹带 5.61843×10^{6} m^3

单相参考 $V = 13.878 \times 10^{6}$ m^3、$\rho = 2105$ kg/m^3；相模型及孔压模型采用各自组分密度。r.avaflow 列原生终态预算，单列边缘储库及侵蚀源；D-Claw 列扩域总质量变化。实际末时为 1800—1802.4 s。横线表示本表不重复列该量。

物理干预提供了可分解的过程量。复合固水与全岩配对保持初始相厚及总体积，固相密度改变使总质量随之变化；关闭的仅为 drag 与 virtual-mass 两项，压力和浮力等项继续保留。全岩与复合参考的汇流括号交叠 0.3 s，关闭 drag / VM 与复合参考交叠 0.4 s，应结合响应和质量账解释。D-Claw 扩大域后所有保存外缘的厚度与水平动量均为零，口岸括号保持不变，同时总质量在 180 s 和 1800 s 分别变化 -2.161%、-4.147%，为边界敏感性与域内预算提供了独立信息。

有界夹带将路径缓冲区内 1 m 床层作为物源，容量 9.5472×10^{6} m^3，实际加入 5.61843×10^{6} m^3。扣除该新增源后再讨论预算，可区分侵蚀增长与数值残差。末态以 $h \geq 0.1$ m、湿相（厚度下限 10^{-6} m）最大水平速度小于 0.1 m/s 定义静止残余：参考及夹带两例此集合为空，$\mu = 0.25$ 运行则得到约 0.3805×10^{6} m^3 有限时刻残余。因此，累计最大流深、末态流深与静止残余分别对应最大过流厚度、当时展布与停滞状态，永久堆积需要更长时段及沉积观测解释。

本节以全源面 ST 过程串联物源、沿谷展布与代表目标响应，双格网、孔压、复合相及有界夹带分别增加空间分辨率、有效应力、组分和源项的信息。本文以相对释放时钟、给定参数与源面组织情景，形成可复查的传播过程、目标响应和结构敏感性结果；与源区机械状态的联系按同一情景标识保留阶段转换项。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

7　模型评价、适用范围与防灾启示

7.1　从区域异常到源区机制的跨尺度解释

因素族排序与机制计算共同提供了一条可检验的解释路径。气象分析中，土壤水分的季节背景位置较高，而事件前 168 小时的变化方向为下降；这一组合提示应同时关注环境状态的历史积累和近期演变。源区水作用模型则以储水、排水、连通性和有效应力为状态，表达裂隙内的水力响应。两者分别对应区域土层和源区水路，联系它们的关键是空间代表性及流路支持，而非要求同名变量具有相同趋势。

热状态也具有明确的时标分工。在一维半无限、初始均温的线性热扩散条件下，$\alpha = 1.5 \times 10^{-6}\,\mathrm{m^2/s}$ 的阶跃边界示例中，72 小时后 2 m 和 5 m 深处的温度响应与表面阶跃之比分别约为 0.0233 和 1.43×10^{-8}。这一深度依赖把解释重点分成浅部短时热扰动、多年初始热状态以及连通水路的传热效应。气象异常说明边界变化，热响应计算进一步回答该变化能够在何种深度与时间支持上影响强度。

因此，本文将 TOP5 作为环境背景输入的优先集合，将条件临界关系作为状态到稳定性的物理映射。结构几何与既有损伤决定共同基础，温度、水及耦合项通过配对设计表达各自贡献。该组织方式既保留了多源数据的统计信息，又将机制解释落实到应力、温度、压力与时间尺度。

7.2　数值敏感性与新增过程的解释收益

传播模型比较中，守恒、初态投影、时间步及边界分别代表不同的数值问题。本文同时报告过程场和这些响应量，比较新增物理过程所解释的现象。表14集中列出具有代表性的量化结果，作为读取图像和到达括号的尺度参照。

表 14　代表性数值对照。各行沿用其原实验配置，表内不同路线的量不作跨配置精度排名；目标括号来自实际存帧、相对整体释放。

对照	主要响应	解释作用
解析热核/细 FD	137 组有限配对，0.0625 m 细 FD 最大差 0.347 h	区分热空间离散与其他状态更新影响
单混合物 CFL	0.5 降至 0.25，存帧水平峰速 92.394 降至 53.256 m/s	将时间离散响应纳入物性效应解释
D-Claw 扩域	口岸代表目标响应保持 $(160, 180]$ s；1800 s 质量变化约 -4.147%	同时读取边界响应与密度加权质量账
全源面 ST 网格	120/60 m、同 31.184 $\mathrm{Mm^3}$，汇流响应 $(240.6, 260.1]$ / $(221.0, 240.7]$ s	保持体积，量化空间离散与足迹变化
有界夹带	新增 5.61843 $\mathrm{Mm^3}$；ASCII 扣源质量残差约 -1.17%	对增长物源建立独立源项账

D-Claw 提供固相状态和孔压演化，复合相模型提供相厚度、密度及相间作用，夹带实验提供侵蚀床供给量。这些输出扩大了模型可解释的过程集合。同时，共同栅格输入进入不同框架后形成各自原生初态；例如 D-Claw 参考初始化的厚度归一 L^1 差约 13.90%，

支持由 67 格变为 111 格。跨框架差异据此分解为初态、物理过程、边界和离散响应，体现了共同实验设计的作用。

本文采用单混合物 120 m 作为主要过程展示，以孔压及复合相路线解释额外状态。复杂度收益通过相应场量、配对变化与源项账呈现；全源面与紧凑源、不同格网以及不同组成属于不同情景，分别保留结果。这样可以把清晰的空间过程展示与量化模型评价放在同一篇论文中。

7.3 预测信息、区间与评价分母

时间预测按信息类型分工。物理首次过阈引擎传播初态和参数给定下的状态演化；逆速度与状态空间方法利用发布前连续变形；生存模型通过暴露与失稳组学习动态风险。外部监测评价以 13 个终末失稳组和 1 个真实右删失站分组，报告固定发布机会及 24、48、72 小时上的区间覆盖、宽度和概率分数。不同尺度下的排名变化说明模型选择应与监测频率、发布时刻和预测对象共同确定。

概率输出同时描述时域内启动、时域内不启动和资料/适用域带来的未知质量。机制关系的数值适用域通过固定历史风险集比较，以信息界和条件结果表达其影响。条件 95% 时间区间、参数情景范围及外部经验覆盖率分别使用相应定义和分母。对于小样本共形校准，秩计算得到的无界区间作为校准结果保留，模型内区间则用于展示预测分布的形状与尺度。

跨问关联采用同一情景标识，将启动条件、释放状态和目标时延连在一起。7 个选定传播运行对应 4 个机械设计和 49 条目标记录；重复格网与几何用于配对敏感性。绝对目标时刻按式(1)保留阶段转换项，空间范围按实际释放足迹及厚度阈值定义。由此形成“机制状态—时间预测—给定释放传播”的可复查链条。

7.4 监测配置与分级响应的研究建议

由上述结果可以推导出三类互补的监测和信息配置。第一，区域背景层持续汇集温湿风、辐射和土层水热，按同季节背景及多窗口更新因素族状态；这有利于识别异常过程的持续性。第二，源区状态层优先观测高频变形、裂隙深度温度和水头/排水状态，并校准各设备时钟；它们直接对应机械启动方程及可辨识性分析中的关键状态。第三，沟谷传播层围绕入河位置、汇流段、桥位和设施分别设置响应对象，使传播时钟与工程对象保持一致。

响应组织上，环境背景用于持续跟踪，源区变形与温压用于识别机械状态，整体释放后的沟谷信号用于更新下游响应时窗。各层保留其独立信息来源，再按同一事件和情景关联。本文给出的监测优先级来自模型参数对输出的敏感性及过程可辨识性，可为后续观测设计和防灾策略提供量化依据。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

表 15 由模型结构导出的信息配置与响应分工。表中为本文研究建议，阈值配置以具体监测点、工程对象和信息时效为依据。

信息层	优先观测与更新	对应研究输出
环境背景	多高度温湿风、辐射、土层水热、质量轨	因素族背景位置、持续时间与排名变化
源区状态	变形加速、裂隙温度、水压、结构与物源	条件稳定裕度、首次过阈、时间分布
传播与对象	释放状态、河谷响应、桥位和设施几何	相对到达括号、流深、影响足迹
信息发布	校准时钟、起报/有效/可用时刻、实际延迟	同发布机会的 24/48/72 h 评价

8 结论

围绕吉隆泥石流灾害链，本文建立了从多源环境资料到结构水热机制、时间预测与传播响应的分析体系，主要结论如下。

（1）因素族聚合使关键因素识别同时具有物理含义与统计对照。基于 5526 个窗口和 112 项特征，边际异常、条件贡献和相似过程三路线形成土壤水分、空气湿状态、土壤热状态、空气热状态及平均风场的共识 TOP5。土壤热通量构成第五席的竞争因素，排序的解释结合变化方向、背景位置和有效重采样分母。

（2）共同参数下的四机制竞争建立了结构、热、水及耦合的条件稳定性图景。2048 个设计情景给出稳定裕度、首过阈类别与临界关系；解析热核与细化有限差分的最大有限启动时刻差为 0.347 小时。温度响应的深度依赖、储排水与损伤的参数补偿，以及 141 条原始实验的本构对照，为机制区分提供了量化支持。

（3）三类时间预测引擎对应不同的信息条件。其中变形与生存模型以 13 个外部失稳组和 1 个右删失站进行分组评价，固定提前 24 小时发布 72 小时预测时，逆速度与状态空间方法的完整标签包含分别为 3/13 和 6/13；24/48/72 小时及监测尺度比较进一步显示模型表现与发布设计相关。机制预测同时保留已启动、未来启动、不启动和未知质量，使参数条件、概率区间和外部覆盖具有清晰角色。

（4）34 个简化区域计算和 31 个原生计算形成了传播模型竞争体系。以给定释放下的流深、速度、足迹、源项账及目标时间括号为共同输出，二维、孔压、复合固水及夹带过程分别贡献了新的解释量。7 个选定运行与 4 个机械设计关联，得到 49 条目标记录；全源面 ST 的同体积格网对照及有界夹带计算展示了几何、离散尺度和新增物源对传播的影响。

（5）跨问联合分析把环境异常、机械启动和释放后的目标响应连成可追溯链条。区域背景、源区温压/变形和沟谷对象响应构成互补监测层，时间与空间尺度、信息可用性及模型敏感性共同支撑防灾信息配置。

本节文字整理、程序与图表制作使用了 OpenAI Codex 辅助，贡献记录见文后声明及配套说明。

参考文献

[1] 马耀明，马伟强，谢志鹏，王宾宾，陈学龙，韩存博. 西藏吉隆高寒山区地气相互作用逐小时综合观测数据集（2021—2026）[DB/OL]. 国家青藏高原科学数据中心，2026. https://doi.org/10.11888/Atmos.tpdc.303600. 元数据镜像：https://gwfnet.net/Metadata/Index/T-2026-08-28-p1f3e5qIqTEyibfPAp3smBeA，访问日期：2026-10-24.

[2] Copernicus Data Space Ecosystem. Copernicus DEM[EB/OL]. https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM. 2021 COG 发布资料：https://registry.opendata.aws/copernicus-dem/，访问日期：2026-10-24.

[3] NOAA/NCEP. GFS NOMADS Products[EB/OL]. https://www.nco.ncep.noaa.gov/pmb/products/gfs/nomads/. 数据访问登记：https://registry.opendata.aws/noaa-gfs-bdp-pds/，访问日期：2026-10-24.

[4] Ledoit O, Wolf M. A well-conditioned estimator for large-dimensional covariance matrices[J]. Journal of Multivariate Analysis, 2004, 88(2): 365–411. https://doi.org/10.1016/S0047-259X(03)00096-4.

[5] Lorenz E N. Atmospheric predictability as revealed by naturally occurring analogues[J]. Journal of the Atmospheric Sciences, 1969, 26(4): 636–646. https://doi.org/10.1175/1520-0469(1969)26<636:APARBN>2.0.CO;2.

[6] Mamot P, Weber S, Schröder T, Krautblatter M. A temperature- and stress-controlled failure criterion for ice-filled permafrost rock joints[J]. The Cryosphere, 2018, 12: 3333–3353. DOI: https://doi.org/10.5194/tc-12-3333-2018. 官方全文：https://tc.copernicus.org/articles/12/3333/2018/（查阅日期：2026-10-24）.

[7] Weber S, Vieli A, Phillips M, Cicoira A. Thermal diffusivity of mountain permafrost derived from borehole temperature data in the Swiss Alps[J]. The Cryosphere, 2025, 19: 6727–6748. DOI: https://doi.org/10.5194/tc-19-6727-2025. 官方全文：https://tc.copernicus.org/articles/19/6727/2025/（查阅日期：2026-10-24）.

[8] Ben-Asher M, Chabas A, Josnin J.-Y., et al. Water flow timing, quantity, and sources in a fractured high mountain permafrost rock wall[J]. Hydrology and Earth System Sciences, 2026, 30: 1735–1754. DOI: https://doi.org/10.5194/hess-30-1735-2026. 官方全文：https://hess.copernicus.org/articles/30/1735/2026/（查阅日期：2026-10-24）.

[9] Leinauer J, Weber S, Cicoira A, et al. An approach for prospective forecasting of rock slope failure time[J]. Communications Earth & Environment, 2023, 4: 253. DOI: 10.1038/s43247-023-00909-z.

[10] Loew S. Monitoring Data from the Insel Compartment at Brienz/Brinzauls Switzerland[DS]. ETH Zurich, 2024. DOI: 10.3929/ethz-b-000670494.

[11] Schild L, Scheiber T, Snook P, et al. Exposing the potential of XAI-based causal discovery for analysing unstable rock slopes[J]. Scientific Reports, 2026, 16: 18935. DOI: 10.1038/s41598-026-48268-x.

[12] NOAA/NCEP/NCO. NCEP Data Products: GFS and GDAS[EB/OL]. https://www.nco.ncep.noaa.gov/pmb/products/gfs/. 访问日期：2026-10-24.

[13] Fukuzono T. A Method to Predict the Time of Slope Failure Caused by Rainfall Using the Inverse Number of Velocity of Surface Displacement[J]. Landslides (Japan Landslide Society), 1985, 22(2): 8–13. DOI: 10.3313/jls1964.22.2_8.

[14] Kalman R E. A New Approach to Linear Filtering and Prediction Problems[J]. Journal of Basic Engineering, 1960, 82(1): 35–45. DOI: 10.1115/1.3662552.

[15] Friedman M. Piecewise Exponential Models for Survival Data with Covariates[J]. The Annals of Statistics, 1982, 10(1): 101–113. DOI: 10.1214/aos/1176345693.

[16] Angelopoulos A N, Bates S. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification[EB/OL]. arXiv:2107.07511, 2021, v6 修订于 2022-12-07. https://arxiv.org/abs/2107.07511. 访问日期：2026-10-24.

[17] European Space Agency, Airbus. Copernicus DEM[DS/OL]. European Space Agency, 2022[2026-10-24]. doi:10.5270/ESA-c5d3d65.

[18] Mergili M, Pfeffer H, Kellerer-Pirklbauer A, Zangerl C, Pudasaini S P. r.avaflow v4, a multi-purpose landslide simulation framework[J]. Geoscientific Model Development, 2025, 18: 9879–9896. doi:10.5194/gmd-18-9879-2025 （访问日期：2026-10-24）.

[19] Pudasaini S P, Mergili M. A multi-phase mass flow model[J]. Journal of Geophysical Research: Earth Surface, 2019, 124(12): 2920–2942. doi:10.1029/2019JF005204 （访问日期：2026-10-24）.

[20] Iverson R M, George D L. A depth-averaged debris-flow model that includes the effects of evolving dilatancy. I. Physical basis[J]. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 2014, 470(2170): 20130819. doi:10.1098/rspa.2013.0819 （访问日期：2026-10-24）.

[21] George D L, Barnhart K R, Iverson R M. D-Claw—A library for simulation of granular-fluid flows, version 1.0.0[CP/OL]. U.S. Geological Survey software release, 2025[2026-10-24]. doi:10.5066/P13GUXUT.

[22] OpenAI Codex，GPT-6 系列（本次会话系统身份；精确模型快照未由会话接口提供），OpenAI，使用日期：2026-10-23 至 2026-10-24. 工具说明：`https://developers.openai.com/api/docs/guides/code-generation`，访问日期：2026-10-24.

AI 工具使用声明

本文使用 OpenAI Codex 辅助资料整理、数据处理、模型代码与实验配置、计算图表、公式与程序复核、论文文字整理及 LaTeX 排版 [22]。相关章节及图表的辅助贡献统一登记于配套《AI 工具使用详情说明》，其中列出工具身份、使用日期、关键提示与回复索引、实际产物和采用记录。用户已对第三轮研究成果完成评审并批准本初稿制作；具体队员人工修改与后续采用情况按实际记录更新。

研究代码、配置、输入和结果索引随支撑工程保存。论文中观测分析、外部站点评价、条件情景及数值基准按各自对象呈现，技术交叉复核与用户评审分别记录。

A 补充结果、软件与复跑说明

A.1 材料组织与结果索引

本论文工程的 `evidence/claim_index.json` 登记正文论断、源文件哈希和图表锚点。原始研究表格保留在各问 `round3/results/`，论文图及其绘图脚本位于论文工程 `figures/`和 `scripts/`。每个模型、机制、情景和几何版本沿用原标识，程序附录对应论文实际采用的自编计算与后处理代码。

A.2 软件、版本与命令

资料处理采用 Python 3.12、NumPy、pandas、SciPy、matplotlib 及按用途锁定的统计/地理环境；原共享环境含 44 个锁定包。传播计算采用本地 Linux 容器中的 r.avaflow 4.0 Revision 7、D-Claw 1.0.0 及 Clawpack 5.11.0。r.avaflow 资源收尾派生修复只涉及 45 处资源清理调用，有效共同配置的 70 项 ASCII 输出逐字一致，物理方程沿用固定源码。原版与派生镜像、每 run 实际源码/构建及配置快照分别登记。

论文的复跑从已有结果开始，依次执行证据/来源核对、论文绘图、参考文献排序、程序附录构建和 XeLaTeX 编译。完整命令在工程 `README.md`。历史模型的重放命令见各问 `REPRODUCE.md`，其输入版本和缓存索引与论文证据表关联。

表 16 研究与论文重放入口。命令在仓库根执行，各路径指向相应版本的说明。

模块	入口
Q1 因素识别	`track_b/research/q1/round3/REPRODUCE.md`
Q2 机制及实验	`track_b/research/q2/round3/REPRODUCE.md`
Q3 时间预测	`track_b/research/q3/round3/REPRODUCE.md`
Q4 传播模型	`track_b/research/q4/round3/REPRODUCE.md`
论文集成	`track_b/paper/README.md`

A.3 数据与模型配置的层级

气象排序同时保留主质量轨与探索质量轨；结构水热计算以共同 256 参数设计、四机制及两强度政策组织；时间预测将历史信息、假设延迟和理想天气输入分别登记；传播给出几何、释放体积、组分、密度、摩阻、孔压、格网、时间步及目标定义。详细参数表、分组评价和每次运行字段由证据索引链接原表，供逐项核对。

A.4 结构水热共同参数设计

下表给出空间填充设计的 19 项参数上下限及取样尺度。四机制共享 256 个参数点，两强度政策形成 2048 个设计情景；参数设计与概率先验分别登记。实际各参数点见相应参数表。

表 17 共同参数设计范围。

参数	单位	下限	上限	取样尺度
结构面倾角	°	30	60	线性
法向厚度	m	10	60	线性
岩体体积分数	1	0.2	0.8	线性
岩桥承载面积比例	1	0	0.25	线性
岩桥有效黏聚力	kPa	300	3000	对数
岩面摩擦系数	1	0.45	0.85	线性

参数	单位	下限	上限	取样尺度
初始裂隙温度	°C	-6	-0.8	线性
热扩散率	m^2/s	9e-07	2.1e-06	线性
热路径深度	m	0.5	20	对数
站点至源区温度偏移	°C	-18	-6	线性
储水量/水头比	1	0.002	0.08	对数
排水时间常数	h	3	240	对数
最大连通水头	m	2	40	对数
有效入渗倍率	1	0.25	3	对数
度日融化系数	mm/(°C·d)	0.5	6	线性
损伤速率系数	h^{-1}	1e-07	0.0002	对数
应力比幂次	1	2	6	线性
初始损伤	1	0.05	0.7	线性
损伤排水耦合系数	1	0	3	线性

A.5　解析热机制分类与裕度补充

分类按相应初态与首次过阈定义，在同一设计表内汇总；初始越阈与未来越阈分列。

表 18　最终解析热版本的机制分类与最小稳定裕度中位数。

机制	强度政策	未来越阈	初始越阈	时域内保持	裕度中位数
S	线性延伸	4	31	221	0.6901
S+T	线性延伸	28	31	197	0.4853
S+H	线性延伸	6	31	219	0.6623
S+T+H	线性延伸	31	31	194	0.4718
S	暖端岩接触	4	31	221	0.6901
S+T	暖端岩接触	27	31	198	0.4853
S+H	暖端岩接触	6	31	219	0.6623
S+T+H	暖端岩接触	31	31	194	0.4718

地形派生成果使用 Copernicus WorldDEM-30（DLR e.V. 2010—2014、Airbus Defence and Space GmbH 2014—2018，由欧盟与 ESA 通过 COPERNICUS 提供）；光学图件包含经修改的 Copernicus Sentinel 2026 数据。对象与河网资料署名为 OpenStreetMap contributors，依 ODbL 登记。

B　论文采用的自编程序

本附录按采用清单列出完整自编源码。完整原程序在配套工程的 `code/`中按原字节保存；本附录的显示副本仅将本机用户目录前缀替换为 `<LOCAL_HOME>`，公式、参数和程序逻辑保持原样。源码行号与折行便于核查；第三方求解器以版本和配置索引。

序号	程序与用途	行数
1	build_features.py	405
2	run_models.py	405
3	summarize_results.py	205
4	q1_figures.py	151
5	q1_evidence.py	458

序号	程序与用途	行数
6	共同力学、损伤及储排水推进	182
7	因果 erfc 热核与解析热接口	143
8	共同 19 维参数设计与情景记录	246
9	全 2048 解析热竞争与参考配对后处理	399
10	局部雅可比与参数补偿基准	226
11	完整外部实验本构拟合、分组留出与条件不确定性	365
12	本章节三幅矢量图重绘	272
13	paper plotting, preview compilation or evidence assembly; no model refit	259
14	paper plotting, preview compilation or evidence assembly; no model refit	163
15	paper plotting, preview compilation or evidence assembly; no model refit	48
16	erfc 线性因果卷积热提供者	143
17	结构/损伤/储水与温应力强度关系自编引擎	182
18	同输入热提供者与同随访天气状态对照	99
19	实际解析热与逐 issue 天气耦合消费端	279
20	Primarypublic data acquisition/provenance andcanonical time/label/measurement audit	133
21	GFS 实际字段获取、原生累计解析与区域摘要	289
22	固定历史风险适用性质量划分	80
23	原先验与适用性分组汇总	149
24	self-written common sensor/issue/H B-vs-C opportunity construction and group-paired scoring	108
25	OLS/SS models, causalvelocity, MonteCarlo mixture, replay/evaluation andStampa challenge	188
26	self-written dimensionless causal features, piecewise exponential/RBF likelihood, group-heldout tuning, rate scaling and predictions; 512/1024 run entry	806
27	join/check frozen seven-run conditional target response	245
28	self-written final integration of already computed paired scores	104
29	first conditional propagation join	75
30	compact/base parameter handoff	603
31	full source-mask join	679
32	机械状态更新、首过阈、权重、信息界与导出通用逻辑	672
33	reduced_solver.py	175
34	reduced_region.py	80
35	reduced_benchmarks.py	68
36	reduced_validate.py	52
37	spatial_build.py	95
38	map_q2_releases.py	125
39	official_q2_v2_cases.py	16
40	extend_computational_domain.py	74
41	official_composite_cases.py	11
42	official_entrainment_case.py	18

序号	程序与用途	行数
43	official_container.py	44
44	official_launch_container.py	26
45	official_analyse.py	205
46	official_targets.py	24
47	integrate_comparison.py	1412
48	audit_dclaw_mass.py	282
49	compare_native_sensitivity.py	148
50	official_result_matrix.py	126
51	q4_plot_results.py	170
52	论文论断与数值来源核对	219
53	参考文献按首次引用排序	114
54	自编程序清单与完整源码附录	203
55	参数范围与机制分类附表生成	57
56	论文结构、引用、匿名和研究完整性检查	235
57	PDF 逐页渲染与总览图生成	44
58	论文可重复构建入口	31

第三方框架版本索引

- r.avaflow，版本 4.0 Revision7，发布 20250902。源码：`https://www.landslidemodels.org/r.avaflow/software/r.avaflow.40G.zip`。
- D-Claw，版本 1.0.0，提交 `7d48663bbed9`；数值依赖：Clawpack，版本 v5.11.0，提交 `285b7de98377`。源码：`https://github.com/geoflows/dclaw.git`。
- ravaflow resourcefix1。

完整版本元数据、源记录路径与哈希见配套工程的`evidence/program_manifest.json`。

程序 1：build_features.py

来源：track_b/research/q1/round3/scripts/build_features.py
SHA256: dbea3bca9f519e43b065481b433372d7abdab3e8f789cf1a1804d09589a3aeb0

```
#!/usr/bin/env python3
"""Extract the preregistered Q1 R3 features without changing public inputs.

Uses the original locked 44-package environment.  Every driver is evaluated on
accepted hourly labels before derivatives; no interpolation or coverage cut is
applied here. Published v1 bytes are immutable: reruns compare before writing.
AI assistance: OpenAI Codex, 2026-10-24; human scientific adoption pending.
"""
from __future__ import annotations

import csv
import hashlib
import io
import json
from pathlib import Path
import sys

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[5]
R3 = ROOT / 'track_b/research/q1/round3'
TEMP = ROOT / 'track_b/data/processed/temporal'


def digest(data):
    return hashlib.sha256(data).hexdigest()


def emit(path, data):
    path.parent.mkdir(parents=True, exist_ok=True)
    if path.exists():
        if path.read_bytes() != data:
            raise RuntimeError(f'Published v1 file differs; create a new version, do not
            ↪  overwrite: {path}')
    else:
        path.write_bytes(data)


def json_bytes(value):
    return (json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n').encode()


def emit_csv(path, frame):
    emit(path, frame.to_csv(index=False, lineterminator='\n').encode())


def emit_parquet(path, frame):
    stream = io.BytesIO()
    frame.to_parquet(stream, index=False, engine='pyarrow', compression='zstd')
    emit(path, stream.getvalue())


def drivers():
    specifications = [
        ('air_temperature_2m', 'air_thermal', ['MET__Ta_2m'], 'identity', 'degC', 'Ta_2m'),
```

```
        ('air_temperature_gradient', 'air_thermal', ['MET__Ta_20m','MET__Ta_2m'], 'difference18', 'degC m-1', '(Ta_20m - Ta_2m)/18 m; nominal measurement heights'),
        ('relative_humidity_2m', 'air_moisture', ['MET__RH_2m'], 'identity', '%', 'RH_2m'),
        ('relative_humidity_gradient', 'air_moisture', ['MET__RH_20m','MET__RH_2m'], 'difference18', 'percentage_point m-1', '(RH_20m - RH_2m)/18 m; nominal measurement heights'),
        ('wind_speed_10m', 'mean_wind', ['MET__WS_10m'], 'identity', 'm s-1', 'WS_10m'),
        ('wind_speed_gradient', 'mean_wind', ['MET__WS_20m','MET__WS_2m'], 'difference18', 's-1', '(WS_20m - WS_2m)/18 m; not friction velocity'),
        ('wind_direction_sin_10m', 'mean_wind', ['MET__WD_10m'], 'sin_degrees', '1', 'sin(WD_10m*pi/180); not a vector-mean direction'),
        ('wind_direction_cos_10m', 'mean_wind', ['MET__WD_10m'], 'cos_degrees', '1', 'cos(WD_10m*pi/180); not a vector-mean direction'),
        ('air_pressure', 'air_pressure', ['MET__Pressure'], 'identity', 'hPa', 'Pressure'),
        ('shortwave_down', 'shortwave', ['RADM__Rsd'], 'identity', 'W m-2', 'Rsd'),
        ('shortwave_difference', 'shortwave', ['RADM__Rsd','RADM__Rsu'], 'difference', 'W m-2', 'Rsd - Rsu; instrument sign convention remains unverified'),
        ('longwave_down', 'longwave', ['RADM__Rld'], 'identity', 'W m-2', 'Rld'),
        ('longwave_difference', 'longwave', ['RADM__Rld','RADM__Rlu'], 'difference', 'W m-2', 'Rld - Rlu; instrument sign convention remains unverified'),
        ('soil_temperature_0p1m', 'soil_thermal', ['SOIL__soilT_0.1m'], 'identity', 'degC', 'soilT_0.1m'),
        ('soil_temperature_gradient', 'soil_thermal', ['SOIL__soilT_0.1m','SOIL__soilT_0.8m'], 'difference0p7', 'degC m-1', '(soilT_0.1m - soilT_0.8m)/0.7 m; shallow-minus-deep nominal gradient'),
        ('soil_vwc_0p1m', 'soil_moisture', ['SOIL__VWC_0.1m'], 'identity', 'm3 m-3', 'VWC_0.1m'),
        ('soil_vwc_gradient', 'soil_moisture', ['SOIL__VWC_0.1m','SOIL__VWC_0.8m'], 'difference0p7', 'm3 m-4', '(VWC_0.1m - VWC_0.8m)/0.7 m; shallow-minus-deep nominal gradient'),
        ('soil_heat_flux_0p1m', 'soil_heat_flux', ['SOIL__G_0.1m'], 'identity', 'W m-2', 'G_0.1m; sign convention remains unverified'),
        ('vapour_pressure_deficit', 'air_moisture', ['FLUX__VPD'], 'identity', 'hPa', 'VPD; unknown FLUX QC, processing and averaging interval'),
        ('precipitation_label', 'precipitation', ['MET__Precip'], 'identity', 'mm (label value; integration period unknown)', 'Precip label; not cumulative rainfall'),
        ('latent_heat_flux', 'turbulent_heat', ['FLUX__LE'], 'identity', 'W m-2', 'LE; unknown QC/sign/processing; not integrated energy'),
        ('sensible_heat_flux', 'turbulent_heat', ['FLUX__H'], 'identity', 'W m-2', 'H; unknown QC/sign/processing; not integrated energy'),
        ('friction_velocity', 'turbulent_mechanics', ['FLUX__USTAR'], 'identity', 'm s-1', 'USTAR; unknown QC/processing'),
        ('turbulent_kinetic_energy', 'turbulent_mechanics', ['FLUX__TKE'], 'identity', 'm2 s-2', 'TKE; unknown QC/processing'),
    ]
    result = []
    for driver_id, family, variables, operation, unit, formula in specifications:
        unknown = driver_id in {'vapour_pressure_deficit','precipitation_label','latent_heat_flux','sensible_heat_flux','friction_velocity','turbulent_kinetic_energy'}
        result.append({'driver_id':driver_id, 'family_id':family, 'source_variables':variables,
            'operation':operation, 'unit':unit, 'formula':formula,
            'quality_requirement':'numeric_present for supplied no-QC variables; unknown never promoted to QC0' if unknown else 'all source values finite, numeric-present and original QC=0 at same hour',
            'availability_track':'unknown_qc_exploration_only' if unknown else 'both_tracks_original_qc0',
            'statistics':['median'] if 'wind_direction_' in driver_id else ['median','p95','max','positive_fraction','longest_positive_fraction'] if driver_id=='precipitation_label' else ['median','p05','p95','trend_per24h','longest_increase_fraction']})
    return result

```

```

def longest_true(mask):
    if not np.any(mask):
        return 0
    padded = np.concatenate(([False], np.asarray(mask, dtype=bool), [False]))
    edges = np.flatnonzero(padded[1:] != padded[:-1])
    return int(np.max(edges[1::2] - edges[::2]))


def extract(values, hours, stats):
    finite = np.isfinite(values)
    count = int(finite.sum())
    n = len(values)
    result = {s:np.nan for s in stats}
    minimum = float(np.min(values[finite])) if count else np.nan
    maximum = float(np.max(values[finite])) if count else np.nan
    if count >= 2:
        selected = values[finite]
        ordered = np.sort(selected)
        for stat, q in [('median',.5),('p05',.05),('p95',.95)]:
            if stat in result:
                position = (count-1)*q
                lower = int(position)
                upper = min(lower+1,count-1)
                fraction = position-lower
                result[stat] = float(ordered[lower] + (ordered[upper]-ordered[lower])*fraction)
        if 'max' in result:
            result['max'] = maximum
        if 'positive_fraction' in result:
            result['positive_fraction'] = float(np.count_nonzero(selected > 0) / count)
        if 'longest_positive_fraction' in result:
            result['longest_positive_fraction'] = longest_true(finite & (values > 0)) / n
        if 'longest_increase_fraction' in result:
            increases = finite[1:] & finite[:-1] & (np.diff(hours)==1) & (np.diff(values)>0)
            result['longest_increase_fraction'] = longest_true(increases) / n
    if count >= 6 and 'trend_per24h' in result:
        x = hours[finite].astype(float)
        x -= np.mean(x)
        y = values[finite]
        result['trend_per24h'] = float(np.dot(x, y-np.mean(y)) / np.dot(x,x) * 24)
    return result, {'n_valid':count,'n_labels':n,'coverage':count/n,'max_gap_hours':longest_tr⌋
    ↪  ue(~finite)}, minimum, maximum


def main():
    assert Path(sys.prefix).resolve() == (ROOT/'track_b/.venv').resolve()
    experiment_path = R3/'config/experiment.json'
    experiment = json.loads(experiment_path.read_text())
    binding = json.loads((R3/'workspace_binding.json').read_text())
    assert Path(binding['execution_root']) == ROOT
    assert experiment['common_baseline_commit'] == binding['common_baseline_commit']
    assert experiment['baseline_manifest_sha256'] == binding['manifest_sha256']
    assert json.loads((R3/'workspace_verification.json').read_text())['passed']
    definitions = drivers()
    configuration = {'schema_version':'q1-r3-features-v1',
        'input_version':{'common_baseline_commit':binding['common_baseline_commit'],
                         'baseline_manifest_sha256':binding['manifest_sha256'],
                         'experiment_config_sha256':digest(experiment_path.read_bytes()),
                         'experiment_protocol_sha256':digest((R3/'EXPERIMENT_PROTOCOL.md').rea⌋
                         ↪  d_bytes())},
        'feature_id_pattern':'<driver_id>__<statistic>',
        'quality_tracks':experiment['quality_tracks'], 'drivers':definitions,
        'minimum_valid_for_distribution_or_persistence':2,'minimum_valid_for_trend':6,
```

```
        'coverage_filter_applied':False,
        'positive_run_definition':'adjacent accepted positive labels; missing breaks; chain label count / nominal window labels',
        'increase_run_definition':'adjacent +1h accepted pairs with positive difference; maximum consecutive increment hours / nominal window labels',
        'time_support':'[anchor-window, anchor) on source hourly labels; timestamp integration semantics unverified',
        'export_time_support_source_clock':['2026-07-01T00:00:00+08:00','2026-08-26T11:30:00+08:00'],
        'v1_immutability':'Existing bytes must match on rerun; changes require a new version and fresh export path'}
    emit(R3/'config/features.json',json_bytes(configuration))
    source_paths=[experiment_path,R3/'EXPERIMENT_PROTOCOL.md',R3/'workspace_binding.json',
        ROOT/'track_b/config/stage4_round3_contract.json',TEMP/'data_dictionary.json',
        TEMP/'hourly_raw_numeric.parquet',TEMP/'hourly_strict_qc0.parquet',
        TEMP/'hourly_mask_is_numeric_present.parquet',TEMP/'hourly_time_axis.parquet',
        ROOT/'track_b/research/q1/results/window_descriptives.csv']
    before={str(p.relative_to(ROOT)):digest(p.read_bytes()) for p in source_paths}
    dictionary=json.loads((TEMP/'data_dictionary.json').read_text())['hourly']
    all_variables=[v['variable_id'] for v in dictionary]
    known={v['variable_id'] for v in dictionary if v['qc_raw_column'] is not None}
    unknown=set(all_variables)-known
    assert len(known)==37 and len(unknown)==26
    raw=pd.read_parquet(TEMP/'hourly_raw_numeric.parquet')
    strict=pd.read_parquet(TEMP/'hourly_strict_qc0.parquet')
    present=pd.read_parquet(TEMP/'hourly_mask_is_numeric_present.parquet')
    clock=pd.read_parquet(TEMP/'hourly_time_axis.parquet')
    assert len(raw)==len(strict)==len(present)==len(clock)==52584
    time_ns=clock['time_utc'].astype('int64').to_numpy()
    hour_indices=clock['hour_index'].to_numpy()
    assert np.all(np.diff(time_ns)==3_600_000_000_000) and np.array_equal(hour_indices,np.arange(len(clock)))
    assert strict[list(unknown)].isna().all().all()
    expanded=strict.copy()
    for variable in unknown:
        expanded[variable]=raw[variable].where(present[variable] & np.isfinite(raw[variable]))
    assert expanded[sorted(known)].equals(strict[sorted(known)])
    hourly={}
    for track, table in zip(experiment['quality_tracks'],[strict,expanded]):
        arrays={}
        for d in definitions:
            source=[table[v].to_numpy(dtype=float) for v in d['source_variables']]
            operation=d['operation']
            if operation=='identity': values=source[0].copy()
            elif operation=='sin_degrees':values=np.sin(np.deg2rad(source[0]))
            elif operation=='cos_degrees':values=np.cos(np.deg2rad(source[0]))
            else:
                divisor={'difference':1.,'difference18':18.,'difference0p7':.7}[operation]
                values=(source[0]-source[1])/divisor
            values[~np.isfinite(values)]=np.nan
            arrays[d['driver_id']]=values
        hourly[track]=arrays

    metadata=[]
    for track in experiment['quality_tracks']:
        anchors=[]
        for year in experiment['background_years']:
            for day_offset in range(-30,31):
                date=pd.Timestamp(f'{year}-08-26T10:30:00',tz='Asia/Shanghai')+pd.Timedelta(days=day_offset)
                for shift in experiment['anchor_shifts_hours']:
                    anchors.append((year,day_offset,shift,'problem_1030',False,date+pd.Timedelta(hours=shift)))
```

```
        for anchor in experiment['anchor_sources']:
            for shift in experiment['anchor_shifts_hours']:
                anchors.append((2026,0,shift,anchor['anchor_id'],True,pd.Timestamp(anchor['sou
                ↪  rce_time'])+pd.Timedelta(hours=shift)))
        for year,day_offset,shift,anchor_id,target,anchor in anchors:
            for window in experiment['windows_hours']:
                start=anchor-pd.Timedelta(hours=window)
                first=int(np.searchsorted(time_ns,start.tz_convert('UTC').value,side='left'))
                stop=int(np.searchsorted(time_ns,anchor.tz_convert('UTC').value,side='left'))
                assert stop-first==window
                sid=f'{track}|{year}|d{day_offset:+03d}|h{shift:+02d}|{anchor_id}|w{window}'
                metadata.append({'sample_id':sid,'quality_track':track,'year':year,'day_offset
                ↪  ':day_offset,
                    'shift_hours':shift,'anchor_id':anchor_id,'window_hours':window,'is_target
                    ↪  ':target,
                    'anchor_time_utc':anchor.tz_convert('UTC').isoformat(),
                    'support_start_utc':start.tz_convert('UTC').isoformat(),
                    'support_end_utc':anchor.tz_convert('UTC').isoformat(),
                    'first_hour_index':first,'last_hour_index':stop-1,
                    'process_id':f'{year}:calendar7:{(day_offset+30)//7:02d}',
                    ↪  'n_labels':stop-first})
    metadata_frame=pd.DataFrame(metadata)
    assert len(metadata)==5526 and metadata_frame['sample_id'].is_unique
    feature_rows=[]; support_rows=[]; extrema_rows=[]; extraction_cache={}
    for sample in metadata:
        first,last=sample['first_hour_index'],sample['last_hour_index']
        track=sample['quality_track']; sid=sample['sample_id']
        features={'sample_id':sid}
        for d in definitions:
            driver_id=d['driver_id']
            known_driver=d['availability_track']=='both_tracks_original_qc0'
            cache_key=(driver_id,'known_qc' if known_driver else track,first,last)
            if not known_driver and track=='original_qc_view':
                cache_key=(driver_id,'all_unknown_null',last-first+1)
            if cache_key not in extraction_cache:
                extraction_cache[cache_key]=extract(hourly[track][driver_id][first:last+1],hou
                ↪  r_indices[first:last+1],d['statistics'])
            extracted,support,minimum,maximum=extraction_cache[cache_key]
            features.update({driver_id+'__'+s:value for s,value in extracted.items()})
            support_rows.append({'sample_id':sid,'driver_id':driver_id,**support})
            extrema_rows.append({'sample_id':sid,'driver_id':driver_id,'n_valid':support['n_va
            ↪  lid'],'minimum':minimum,'maximum':maximum})
        feature_rows.append(features)
    feature_frame=pd.DataFrame(feature_rows)
    for column in
    ↪  feature_frame.columns[1:]:feature_frame[column]=feature_frame[column].astype('float64')
    support_frame=pd.DataFrame(support_rows)
    emit_parquet(R3/'results/features.parquet',feature_frame)
    emit_csv(R3/'results/sample_metadata.csv',metadata_frame)
    emit_csv(R3/'results/feature_support.csv',support_frame)
    emit_csv(R3/'results/feature_extrema.csv',pd.DataFrame(extrema_rows))

    contract=[]
    for d in definitions:
        for stat in d['statistics']:
            unit='1' if stat.endswith('_fraction') else d['unit']+' per 24 nominal hours' if
            ↪  stat=='trend_per24h' else d['unit']
            statistic_formula={'median':'linear sample quantile q=0.5','p05':'linear sample
            ↪  quantile q=0.05','p95':'linear sample quantile q=0.95',
                'max':'maximum accepted label value; no accumulation',
                'trend_per24h':'OLS on accepted value versus true hourly index, slope
                ↪  multiplied by 24; >=6 accepted labels',
                'longest_increase_fraction':configuration['increase_run_definition'],
```

```
            'positive_fraction':'accepted labels >0 / count(accepted labels)',
            'longest_positive_fraction':configuration['positive_run_definition']}[stat]
        contract.append({'feature_id':d['driver_id']+'__'+stat,'driver_id':d['driver_id'],
            'family_id':d['family_id'],'statistic':stat,'ranking_enabled':True,'unit':unit,
            'source_variables':json.dumps(d['source_variables']), 'formula':d['formula']+';
            ↪ '+statistic_formula,
            'quality_requirement':d['quality_requirement']+'; '+d['availability_track'],
            'available_time':None,'space_object':'Gyirong_regional_background; metadata
            ↪ longitude85.29 latitude28.86 elevation4140m; coordinate datum and error
            ↪ unknown; local equivalence and source-area representativeness unverified',
            'physical_phase':'background','data_role':'exploration','timestamp_integration
            ↪ _interval':None})
contract_frame=pd.DataFrame(contract)
assert list(feature_frame.columns[1:])==contract_frame['feature_id'].tolist()
emit_csv(R3/'results/feature_contract.csv',contract_frame)
mappings=[]
for v in dictionary:
    variable=v['variable_id']; selected=[d for d in definitions if variable in
    ↪ d['source_variables']]
    if selected:
        reason='pre-registered primary measurement or paired nominal gradient'
    elif variable=='FLUX__amb_e_sat':reason='diagnostic only: 4317-label constant segment
    ↪ and processing origin unresolved'
    elif variable.startswith('FLUX__FETCH'):reason='diagnostic
    ↪ footprint/representativeness; not an independent physical trigger factor'
    elif variable=='FLUX__FC_mass':reason='diagnostic only: species and raw unit
    ↪ interpretation unresolved'
    else:reason='not pre-registered; redundant height/depth/component or ambiguous derived
    ↪ quantity; raw input retained'
    if selected:families=sorted({d['family_id'] for d in selected})
    elif variable.startswith('MET__Ta') or
    ↪ variable=='FLUX__T_SONIC':families=['air_thermal']
    elif variable.startswith('MET__RH') or variable in {'FLUX__amb_e','FLUX__amb_e_sat','F
    ↪ LUX__e','FLUX__e_sat','FLUX__e_probe','FLUX__e_sat_probe','FLUX__H2O_probe','FLUX_
    ↪ _H2O_density','FLUX__VPD'}:families=['air_moisture']
    elif variable.startswith('MET__WS') or variable.startswith('MET__WD') or variable in
    ↪ {'FLUX__Ux','FLUX__Uy','FLUX__Uz'}:families=['mean_wind']
    elif variable=='FLUX__PA':families=['air_pressure']
    elif variable.startswith('SOIL__soilT'):families=['soil_thermal']
    elif variable.startswith('SOIL__VWC'):families=['soil_moisture']
    elif variable.startswith('SOIL__G'):families=['soil_heat_flux']
    elif variable in {'FLUX__TSTAR'}:families=['turbulent_heat']
    elif variable in {'FLUX__TAU'}:families=['turbulent_mechanics']
    else:families=['excluded_diagnostic']
    mappings.append({'variable_id':variable,'family_ids':json.dumps(families),'driver_ids'
    ↪ :json.dumps([d['driver_id'] for d in selected]),
        'ranking_enabled':bool(selected),'selection_status':'included' if selected else
        ↪ 'excluded_from_ranking_preserved_raw',
        'reason':reason,'raw_unit_label':v['raw_unit_label'],'supplied_qc_column':v['qc_ra
        ↪ w_column'],
        'quality_status':'original_qc0_required' if variable in known else
        ↪ 'unknown_qc_not_promoted'})
assert len(mappings)==63
emit_csv(R3/'results/variable_family_map.csv',pd.DataFrame(mappings))

start_export=pd.Timestamp(configuration['export_time_support_source_clock'][0]).tz_convert
↪ ('UTC')
end_export=pd.Timestamp(configuration['export_time_support_source_clock'][1]).tz_convert('
↪ UTC')
selection=(clock['time_utc']>=start_export)&(clock['time_utc']<=end_export)
export=[]
for track in experiment['quality_tracks']:
```

```
        part=clock.loc[selection,['hour_index','time_utc']].copy()
        part['quality_track']=track
        for d in definitions:part[d['driver_id']]=hourly[track][d['driver_id']][selection.to_numpy()]
        export.append(part)
    export_frame=pd.concat(export,ignore_index=True)
    emit_parquet(R3/'exports/hourly_drivers.parquet',export_frame)

    # Check actual anchor sets and the previously frozen median diagnostic.
    target=metadata_frame[metadata_frame['is_target']]
    shifts=[]
    for (track,anchor_id,window), group in target.groupby(['quality_track','anchor_id','window_hours'],sort=False):
        group=group.sort_values('shift_hours')
        firsts=group['first_hour_index'].tolist();lasts=group['last_hour_index'].tolist()
        assert np.all(np.diff(firsts)==1) and np.all(np.diff(lasts)==1)
        shifts.append({'quality_track':track,'anchor_id':anchor_id,'window_hours':int(window),'first_hour_indices':firsts,'last_hour_indices':lasts,'changed_by_one_label_each_shift':True})
    r1=pd.read_csv(ROOT/'track_b/research/q1/results/window_descriptives.csv')
    comparison=[]
    mapping={'MET__Ta_2m':'air_temperature_2m','RADM__Rsd':'shortwave_down','SOIL__soilT_0.1m':'soil_temperature_0p1m','SOIL__VWC_0.1m':'soil_vwc_0p1m','FLUX__LE':'latent_heat_flux','MET__Precip':'precipitation_label'}
    by_id=feature_frame.set_index('sample_id')
    for v,driver_id in mapping.items():
        track='unknown_qc_exploration' if v in unknown else 'original_qc_view'
        for _,old in r1[r1['variable_id'].eq(v)].iterrows():
            q=metadata_frame[metadata_frame['quality_track'].eq(track)&metadata_frame['year'].eq(old['year'])&metadata_frame['day_offset'].eq(0)&metadata_frame['shift_hours'].eq(0)&metadata_frame['window_hours'].eq(old['window_hours'])&metadata_frame['anchor_id'].eq(old['anchor_id'])]
            if q.empty:continue  # Historical usgs anchor intentionally de-duplicated.
            assert len(q)==1
            actual=float(by_id.loc[q.iloc[0]['sample_id'],driver_id+'__median']);expected=float(old['median'])
            ok=(np.isnan(actual) and np.isnan(expected)) or bool(np.isclose(actual,expected,rtol=1e-12,atol=1e-12))
            assert ok,(v,old['year'],old['window_hours'],actual,expected)
            comparison.append({'variable_id':v,'year':int(old['year']),'anchor_id':old['anchor_id'],'window_hours':int(old['window_hours']),'matches_saved_r1_median':True})
    assert all(d['driver_id'] not in strict.columns for d in definitions)
    for d in definitions:
        if d['availability_track']=='unknown_qc_exploration_only':assert np.isnan(hourly['original_qc_view'][d['driver_id']]).all()
    assert all(digest((ROOT/p).read_bytes())==value for p,value in before.items())
    # Mechanical cases test exactly the data behavior relied upon downstream.
    a,s,_,_=extract(np.array([1.,2.,np.nan,3.,4.,5.]),np.arange(6),['longest_increase_fraction','trend_per24h'])
    assert a['longest_increase_fraction']==2/6 and np.isnan(a['trend_per24h']) and s['max_gap_hours']==1
    a,_,_,_=extract(np.array([0.,1.,2.,np.nan,3.,4.]),np.arange(6),['positive_fraction','longest_positive_fraction'])
    assert a['positive_fraction']==4/5 and a['longest_positive_fraction']==2/6
    validation={'schema_version':'q1-r3-feature-build-validation-v1','passed':True,
        'sample_count':len(metadata_frame),'feature_count':len(contract),'driver_count':len(definitions),
        'historical_samples':int((~metadata_frame['is_target']).sum()),'target_samples':int(metadata_frame['is_target'].sum()),
        'quality_tracks':experiment['quality_tracks'],'known_qc_variable_count':len(known),'unknown_qc_variable_count':len(unknown),
        'all_63_variables_mapped':True,'sample_ids_unique':True,'derived_gradients_require_all_sources':True,
```

```
        'known_qc_values_identical_between_tracks':True,'unknown_drivers_empty_in_original_qc_track':True,
        'coverage_filter_applied':False,'feature_support_rows':len(support_frame),
        'nonfinite_features_preserved_as_null':True,'all_float_feature_columns':True,
        'source_inputs_unchanged':True,'input_sha256':before,
        'anchor_shift_checks':shifts,'saved_r1_median_checks':comparison,
        'mechanical_checks':['missing breaks growth','growth counted in adjacent hour increments','trend requires at least 6 valid labels','precip positive fraction denominator is valid labels','positive-chain denominator is nominal labels'],
        'hourly_export_rows':len(export_frame),'hourly_export_first_utc':export_frame['time_utc'].min().isoformat(),
        'hourly_export_last_utc':export_frame['time_utc'].max().isoformat(),
        'rankings_or_event_labels_created':False}
    emit(R3/'results/feature_build_validation.json',json_bytes(validation))

    paths_roles=[('config/features.json','frozen_driver_definitions'),('scripts/build_features.py','offline_feature_builder'),
        ('results/feature_contract.csv','feature_definitions'),('results/variable_family_map.csv','all_input_variables_selection_and_exclusion'),
        ('results/features.parquet','wide_features_by_sample'),('results/sample_metadata.csv','time_support_process_and_anchor_metadata'),
        ('results/feature_support.csv','driver_validity_coverage_and_gap_counts'),('results/feature_extrema.csv','diagnostic_driver_extrema'),
        ('results/feature_build_validation.json','executed_feature_checks'),('exports/hourly_drivers.parquet','2026_regional_hourly_drivers')]
    bundle={'bundle_version':'q1-r3-feature-v1','producer_question':'q1','input_version':configuration['input_version'],
        'evaluation_stratum':'local_observational','event_id':experiment['event_id'],'physical_phase':'background',
        'data_role':'exploration','time_utc':None,'time_uncertainty':None,'available_time':None,
        'forecast_reference_time':None,'issue_time':None,'valid_time':None,
        'space_object':{'id':'Gyirong_regional_background','crs':None,'longitude':85.29,'latitude':28.86,'elevation_m':4140,'coordinate_datum_status':'unknown; metadata provides coordinates but no datum or positional error','status':'metadata assertion; source-area representativeness unverified'},
        'model_id':'not_applicable_feature_extraction','mechanism_id':experiment['mechanism_id'],
        'experiment_id':experiment['experiment_id'],'scenario_id':experiment['scenario_id'],
        'geometry_version':experiment['geometry_version'],'parameter_version':experiment['parameter_version'],
        'variable_semantics':{'driver_configuration':'track_b/research/q1/round3/config/features.json','timestamp_integration_interval':None,
            'quality_view':'original_qc_view plus separate unknown_qc_exploration; original 37 QC columns remain QC0 in both'},
        'files':[{'path':str((R3/p).relative_to(ROOT)),'bytes':(R3/p).stat().st_size,'sha256':digest((R3/p).read_bytes()),'role':role} for p,role in paths_roles],
        'limitations':['Hourly labels do not prove averaging or accumulation intervals',
            'available_time unknown: cannot establish historical operational eligibility',
            'Regional station drivers do not constitute measured source-area forcing',
            'No precipitation accumulation or energy integral was constructed',
            'No coverage cutoff in this bundle; downstream applies preregistered 0.75/0.50/0.90 thresholds',
            'Background windows/weather blocks are not independent disaster labels',
            'Feature extraction does not select a final scientific model or factor ranking',
            'Existing v1 bytes are immutable; changed outputs require a new bundle version and paths']}
    emit(R3/'exports/feature_bundle.json',json_bytes(bundle))
    bundle_hash=digest((R3/'exports/feature_bundle.json').read_bytes())
    text=f'''# Q1 第三轮早交：区域站特征 v1

本接口已实际提取并通过检查；尚未进行因素排序。只交主参谋核验后分发。
```

```
- bundle：`exports/feature_bundle.json`，SHA-256 `{bundle_hash}`。
- 共同基线：`{binding['common_baseline_commit']}`；冻结清单 SHA-256 `{binding['manifest_sha256']}`。
- {len(metadata_frame)}个样本（背景{validation['historical_samples']}、目标{validation['target_samples']}），{len(definitions)}驱动、{len(contract)}排名描述量，完整63原变量选择 / 排除表。
- `results/features.parquet`为`sample_id`加浮点feature_id列；元数据、覆盖支持另表，以sample_id连接。feature_id=`driver_id__statistic`，模型须先按每驱动coverage筛选。
- 每个quality_track有2021—2025×61日×3锚点小时偏移×3窗口背景，以及2026两参考×3偏移×3窗口目标。背景去重为problem_1030；分钟参考相同样本不得多投票。
- 24/72/168h的support_start/end为左闭右开请求区间，实际标签由first/last_hour_index精确确定。process_id是季节首日开始7日历日块代理，不是真实天气事件。
- 原QC轨仅37个原QC0变量；扩展轨只添加原本无QC变量，37列仍QC0。派生量逐小时取原量有效交集，不填补。unknown不升级质量。
- 构建器只要求分布 / 持续性>=2有效标签、趋势>=6；不套75%阈值。增长持续按正相邻小时增量数/名义窗标签数；缺口打断。降水仅标签统计，未做累计。
- `exports/hourly_drivers.parquet`按quality_track长表，UTC时间与24个驱动，2026-07-01 00:00至2026-08-26 11:30（UTC+8）内实际整点标签，共{len(export_frame)}行。时效未知，不代表历史实时可获得。
- 离线命令：`track_b/.venv/bin/python track_b/research/q1/round3/scripts/build_features.py`。再次运行必须产生逐字节相同产物；v1有变化即拒写，须新版本。
- 实际检查：5526唯一样本、24/72/168标签数、±1h支持集合、37QC列跨轨相同、unknown主轨全空、63变量映射、R1固定median对照、缺口持续和趋势机械用例、公共输入哈希不变。详见`results/feature_build_validation.json`。

空间、时区等价、小时起止 / 中心、FLUX处理及真实业务时效仍未核实。因素族只是区域观测候选，不是源区触发证明；论文最终科学采用待队员审视。v1清单不包含自身，所有文件按真实bytes/SHA-256列出。
'''
    emit(R3/'INTERFACE_HANDOFF.md',text.encode())
    print(json.dumps({'passed':True,'samples':len(metadata_frame),'drivers':len(definitions),'features':len(contract),'bundle_sha256':bundle_hash},ensure_ascii=False))


if __name__=='__main__':
    main()
```

程序 2：run_models.py

来源：track_b/research/q1/round3/scripts/run_models.py
SHA256: b15a990c7f357b179f62f318ae86326a93174d9959d2138c0118157f4d523da2

```
#!/usr/bin/env python3
"""Three competing regional pre-reference anomaly routes; AI assisted, no causal labels."""
from pathlib import Path
import os
for key in ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS'):
    os.environ[key]='1'
import json, hashlib, time, warnings
from datetime import datetime, timezone
import numpy as np
import pandas as pd
from scipy.stats import kendalltau

R = Path(__file__).resolve().parents[1]
ROOT = Path(__file__).resolve().parents[5]
CFG = json.loads((R/'config/experiment.json').read_text())
MODELS=CFG['model_ids']
FAMILIES={'air_thermal':'空气热状态','air_moisture':'空气湿状态','mean_wind':'平均风场','air_pressure':'气压','shortwave':'短波辐射','longwave':'长波辐射','soil_thermal':'土壤热状态','soil_moisture':'土壤水分','soil_heat_flux':'土壤热通量','precipitation':'降水标签分布','turbulent_heat':'湍流热交换','turbulent_mechanics':'湍流机械交换'}


def sha(path): return hashlib.sha256(path.read_bytes()).hexdigest()
def dump(obj,path):
    path.write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
def finite_num(x): return None if not np.isfinite(x) else float(x)

def weights(meta, valid=None, draw=None, block=7):
    """Exact year/process/row hierarchy, vectorized without changing estimand."""
    if valid is None:valid=np.ones(len(meta),bool)
    out=np.zeros(len(meta)); idx=np.flatnonzero(valid)
    if not len(idx):return out
    y=meta.year.to_numpy(dtype=int);g=((meta.day_offset.to_numpy()+30)//block).astype(int)
    codes=y[idx]*1000+g[idx]
    unique,inv,counts=np.unique(codes,return_inverse=True,return_counts=True)
    uy=unique//1000;ug=unique%1000
    years,yearinv=np.unique(uy,return_inverse=True)
    if draw is None:
        ym=np.ones(len(years));gm=np.ones(len(unique))
    else:
        ym=np.array([draw.get(int(a),{}).get('count',0) for a in years],float)
        gm=np.array([draw.get(int(a),{}).get('processes',{}).get(int(b),0) for a,b in zip(uy,ug)],float)
    mass=np.bincount(yearinv,weights=gm,minlength=len(years))
    numerator=ym[yearinv]*gm
    denominator=mass[yearinv]*counts
    perrow=np.divide(numerator,denominator,out=np.zeros_like(numerator),where=denominator>0)
    out[idx]=perrow[inv]
    return out/out.sum() if out.sum() else out


def quantile(x,w,q):
    ok=np.isfinite(x)&(w>0)
    if not ok.any():return np.nan
    xx=x[ok];ww=w[ok];ii=np.argsort(xx,kind='stable'); xx=xx[ii];ww=ww[ii]
    return float(xx[np.searchsorted(np.cumsum(ww)/ww.sum(),q,side='left').clip(0,len(xx)-1)])

```

```
53
54 def ecdf(x, val, meta, draw=None, block=7, minimum=3):
55     ok=np.isfinite(x)
56     w=weights(meta,ok,draw,block)
57     if not np.isfinite(val) or len(np.unique(meta.year.to_numpy()[w>0]))<minimum:return np.nan
58     return float(w[x<val].sum()+0.5*w[x==val].sum())
59
60
61 def aggregate(feature_scores, contract):
62     df=pd.DataFrame({'feature_id':contract.feature_id,'driver_id':contract.driver_id,'family_i
       ↪  d':contract.family_id,'value':feature_scores})
63     return df.groupby(['family_id','driver_id']).value.median().groupby('family_id').median().
       ↪  to_dict()
64
65
66 def fit_matrix(X,meta,ids,draw=None,block=7, target=None):
67     """Only training values determine support, centring, scaling and imputation."""
68     w=weights(meta,draw=draw,block=block)
69     keep=[]; med=[]; scales=[]
70     for j in ids:
71         ok=np.isfinite(X[:,j]);obs=float(w[ok].sum())
72         if obs<CFG['covariance_feature_observed_fraction'] or
           ↪  len(np.unique(meta.year.to_numpy()[ok&(w>0)]))<3:continue
73         if target is not None and not np.isfinite(target[j]):continue
74         wj=weights(meta,ok,draw,block)
75         m=quantile(X[:,j],wj,.5); s=1.4826*quantile(abs(X[:,j]-m),wj,.5)
76         if s<=0 or not np.isfinite(s):s=(quantile(X[:,j],wj,.75)-quantile(X[:,j],wj,.25))/1.349
77         if not np.isfinite(s) or s<=0:continue
78         keep.append(j);med.append(m);scales.append(s)
79     keep=np.array(keep,int);med=np.array(med);scales=np.array(scales)
80     if not len(keep):return None
81     xx=X[:,keep]; missing=~np.isfinite(xx);
       ↪  zz=np.clip((np.where(missing,med,xx)-med)/scales,-8,8)
82     mu=w@zz; zz=zz-mu
83     return {'ids':keep,'median':med,'scale':scales,'mean':mu,'z':zz,'weights':w,'imputed_weigh
       ↪  ted_fraction':float(np.sum(w[:,None]*missing)/len(keep))}
84
85
86 def precision_fit(fit,alpha):
87     z=fit['z'];w=fit['weights']; C=(z*w[:,None]).T@z
88     S=(1-alpha)*C+alpha*np.eye(len(fit['ids']))
89     P=np.linalg.inv(S)
90     return P,float(np.linalg.cond(S))
91
92
93 def contributions(fit,P,targets,contract):
94     arr=np.atleast_2d(targets)[:,fit['ids']]
95     zz=np.clip((np.where(np.isfinite(arr),arr,fit['median'])-fit['median'])/fit['scale'],-8,8)
       ↪  -fit['mean']
96     pdiff=zz@P
97     cf=contract.iloc[fit['ids']]
98     result={}
99     for family in sorted(cf.family_id.unique()):
100        loc=np.flatnonzero(cf.family_id.to_numpy()==family)
101        a=pdiff[:,loc]
102        value=np.sum(a*np.linalg.solve(P[np.ix_(loc,loc)],a.T).T,axis=1)/len(loc)
103        assert np.min(value)>-1e-7
104        result[family]=np.maximum(value,0)
105    return result
106
107
108 def b_scores(X,target,meta,contract,alpha,draw=None,block=7):
109    fit=fit_matrix(X,meta,range(X.shape[1]),draw,block,target)
```

```
    if fit is None:return {},{}, {'features':0,'condition_number':None}
    P,condition=precision_fit(fit,alpha)
    values=contributions(fit,P,target,contract)
    # Five grouped out-of-year calibration folds. Whole years cannot overlap supports.
    cal={f:np.full(len(X),np.nan) for f in values}
    folds=[]
    for yr in sorted(meta.year.unique()):
        test=(meta.year.to_numpy()==yr);train=~test
        sub=fit_matrix(X[train],meta.loc[train].reset_index(drop=True),range(X.shape[1]),draw,block,target)
        if sub is None:continue
        pp,_=precision_fit(sub,alpha)
        cv=contributions(sub,pp,X[test],contract)
        for f in cal:
            if f in cv:
                family_cols=sub['ids'][contract.iloc[sub['ids']].family_id.to_numpy()==f]
                observed=np.isfinite(X[test][:,family_cols]).all(axis=1)
                held=cv[f].copy();held[~observed]=np.nan
                cal[f][test]=held
        folds.append({'year':int(yr),'n_train':int(train.sum()),'n_test':int(test.sum())})
    scores={f:ecdf(cal[f],values[f][0],meta,draw,block) for f in values}
    raw={f:float(values[f][0]) for f in values}
    return scores,raw,{'features':len(fit['ids']),'condition_number':condition,'imputed_training_fraction':fit['imputed_weighted_fraction'],'calibration_folds':folds,'calibration_support':{f:{'n_observed_rows':int(np.isfinite(v).sum()),'n_observed_years':int(meta.loc[np.isfinite(v),'year'].nunique()),'n_observed_processes':int(meta.loc[np.isfinite(v),'process_id'].nunique())} for f,v in cal.items()},'active_features':[contract.iloc[j].feature_id for j in fit['ids']],'_fit_for_routes':fit}


def routes(X,target,meta,contract,alpha=.5,k=2,draw=None,block=7,save_detail=False):
    # A: weighted midpoint ECDF, finite even under point-mass backgrounds.
    F=np.array([ecdf(X[:,j],target[j],meta,draw,block) for j in range(X.shape[1])])
    aa=2*abs(F-.5); scoresA=aggregate(aa,contract)
    scoresB,rawB,binfo=b_scores(X,target,meta,contract,alpha,draw,block)
    # C matching excludes all columns of the family under evaluation.
    fit=binfo.pop('_fit_for_routes',None)
    cc=np.full(X.shape[1],np.nan); analogs=[]
    groups=((meta.day_offset.to_numpy()+30)//block).astype(int)
    sample_ids=meta.sample_id.to_numpy();year_values=meta.year.to_numpy()
    positive=weights(meta,draw=draw,block=block)>0
    if fit is not None:
        targetz=np.clip((target[fit['ids']]-fit['median'])/fit['scale'],-8,8)-fit['mean']
        diff=(fit['z']-targetz)**2
        cf=contract.iloc[fit['ids']]
        family_dist={f:diff[:,cf.family_id.to_numpy()==f].mean(axis=1) for f in cf.family_id.unique()}
        for family in contract.family_id.unique():
            outside=[a for f,a in family_dist.items() if f!=family]
            if not outside:continue
            dist=np.sqrt(np.mean(outside,axis=0));selected=[]
            outside_cols=fit['ids'][cf.family_id.to_numpy()!=family]
            observed_matching_fraction=np.isfinite(X[:,outside_cols]).mean(axis=1)
            distance_eligible=observed_matching_fraction>=0.8
            for yr in sorted(meta.year.unique()):
                cand=np.flatnonzero((meta.year.to_numpy()==yr)&positive&distance_eligible)
                nearest=[]
                for gp in np.unique(groups[cand]):
                    ix=cand[groups[cand]==gp];i=int(ix[np.argmin(dist[ix])]);nearest.append((dist[i],i))
                nearest.sort(key=lambda t:(t[0],str(sample_ids[t[1]])))
                selected.extend(i for _,i in nearest[:k])
```

```
            if not selected:continue
            selected=np.array(selected,int);sm=meta.iloc[selected].reset_index(drop=True)
            for j in np.flatnonzero(contract.family_id.to_numpy()==family):
                # Recompute weights over selected finite rows. Each chosen process has one row.
                ff=ecdf(X[selected,j],target[j],sm,draw,block)
                cc[j]=2*abs(ff-.5)
            if save_detail:
                for i in selected:analogs.append({'family_id':family,'selected_sample_id':meta.iloc[i].sample_id,'year':int(meta.iloc[i].year),'process_id':f'{int(meta.iloc[i].year)}-b{int(groups[i])}','distance_excluding_family':float(dist[i]),'observed_outside_family_matching_fraction':float(observed_matching_fraction[i])})
    scoresC=aggregate(cc,contract)
    families=sorted(contract.family_id.unique())
    out={m:{f:s.get(f,np.nan) for f in families} for m,s in [(MODELS[0],scoresA),(MODELS[1],scoresB),(MODELS[2],scoresC)]}
    detail=None
    if save_detail:
        detail=contract[['feature_id','driver_id','family_id','statistic']].copy()
        detail['target_value']=target;detail['background_percentile']=F;detail['signed_direction']=np.sign(F-.5)
        detail['marginal_score']=aa;detail['analog_score']=cc
    return out,{'b_info':binfo,'b_raw_contribution':rawB,'feature_detail':detail,'analogs':analogs}


def ranking(score):
    finite={f:float(x) for f,x in score.items() if np.isfinite(x)}
    # Scores tied at 1e-12 get midranks. Alphabetical labels never select a winner.
    order=sorted(finite,key=lambda f:(-round(finite[f],12),f))
    out={};i=0
    while i<len(order):
        j=i+1
        while j<len(order) and round(finite[order[j]],12)==round(finite[order[i]],12):j+=1
        for f in order[i:j]:out[f]=(i+1+j)/2
        i=j
    return out


def top5_credit(ranks,family):
    if family not in ranks:return 0.0
    r=ranks[family];n=sum(v==r for v in ranks.values());above=r-(n+1)/2
    return float(np.clip((5-above)/n,0,1))


def consensus(result):
    allf=sorted(set().union(*(x.keys() for x in result.values())))
    ranks={m:ranking(s) for m,s in result.items()}
    # Missing route never becomes a good zero rank; report availability and leave mean undefined if <3.
    scores={}
    for f in allf:
        vals=[1-(r[f]-1)/max(len(r)-1,1) for r in ranks.values() if f in r]
        scores[f]=float(np.mean(vals)) if len(vals)==3 else np.nan
    return scores


def select_train(meta,target_meta,exclude=None):
    """Purge actual max-168h support overlap, even when displayed window is 24/72h."""
    keep=np.ones(len(meta),bool);test=np.zeros(len(meta),bool);purge=np.zeros(len(meta),bool)
    if exclude is None:return keep,test,purge
    if exclude[0]=='year':test=meta.year.to_numpy()==exclude[1]
    else:test=meta.process_id.to_numpy()==exclude[1]
```

```
    keep &= ~test
    for _,row in meta.loc[test].iterrows():
        a=pd.Timestamp(row.anchor_time_utc)-pd.Timedelta(hours=168)
        b=pd.Timestamp(row.anchor_time_utc)
        starts=pd.to_datetime(meta.anchor_time_utc,utc=True)-pd.Timedelta(hours=168)
        ends=pd.to_datetime(meta.anchor_time_utc,utc=True)
        overlap=((starts<b)&(ends>a)).to_numpy()
        purge |= overlap&~test
    keep &= ~purge
    return keep,test,purge


def main():
    started=datetime.now(timezone.utc).isoformat(); t0=time.monotonic()
    assert ROOT==Path('<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q1')
    ctr=pd.read_csv(R/'results/feature_contract.csv')
    enabled=ctr.ranking_enabled.astype(str).str.lower().isin(['true','1'])
    ctr=ctr.loc[enabled].reset_index(drop=True)
    wide=pd.read_parquet(R/'results/features.parquet').set_index('sample_id')
    meta=pd.read_csv(R/'results/sample_metadata.csv')
    sup=pd.read_csv(R/'results/feature_support.csv')
    cover=sup.pivot(index='sample_id',columns='driver_id',values='coverage')
    if 'coverage' not in sup:raise ValueError('Missing coverage')
    bank=wide.loc[meta.sample_id,ctr.feature_id].to_numpy(float)
    coverage=cover.loc[meta.sample_id,ctr.driver_id].to_numpy(float)
    ranks=[];details=[];analogs=[];metrics=[];folds=[];stability=[];cv_metrics=[]
    outputs={}; bdiag=[]
    base={'coverage':.75,'block':7,'season':30,'alpha':.5,'k':2}
    variants={'primary':base}
    for name,key,vals in [('support','coverage',[.5,.9]),('process','block',[3,14]),('season',
    ↪  'season',[15]),('shrinkage','alpha',[.1,.9]),('analogs','k',[1,3])]:
        for value in vals:variants[f'{name}_{value}']={**base,key:value}
    def matrices(track,shift,window,par,anchor='problem_1030',exclude=None):
        sel=(meta.quality_track==track)&(meta.shift_hours==shift)&(meta.window_hours==window)
        bg=sel&~meta.is_target.astype(bool)&(meta.day_offset.abs()<=par['season'])
        tgt=sel&meta.is_target.astype(bool)&(meta.anchor_id==anchor)
        assert tgt.sum()==1,(track,shift,window,anchor,tgt.sum())
        bgidx=np.flatnonzero(bg);ti=int(np.flatnonzero(tgt)[0]);bm=meta.loc[bg].reset_index(dr
        ↪  op=True)
        keep,test,purge=select_train(bm,meta.iloc[ti],exclude)
        vals=bank[bgidx].copy();tv=bank[ti].copy()
        vals[coverage[bgidx]<par['coverage']]=np.nan;tv[coverage[ti]<par['coverage']]=np.nan
        # All-null unknown-only drivers stay absent in the accepted-QC track.
        eligible=np.array([len(np.unique(bm.year[np.isfinite(vals[:,j])]))>=3 and
        ↪  np.isfinite(tv[j]) for j in range(vals.shape[1])])
        return vals[keep][:,eligible],tv[eligible],bm.loc[keep].reset_index(drop=True),ctr.loc
        ↪  [eligible].reset_index(drop=True),bm,keep,test,purge,meta.iloc[ti]
    def run_one(track,shift,window,par,variant='primary',anchor='problem_1030',exclude=None,dr
    ↪  aw=None,detail=False):
        X,t,bm,ct,original,keep,test,purge,tm=matrices(track,shift,window,par,anchor,exclude)
        res,info=routes(X,t,bm,ct,par['alpha'],par['k'],draw,par['block'],detail)
        return res,info,(X,t,bm,ct,original,keep,test,purge,tm)
    for track in CFG['quality_tracks']:
        for window in CFG['windows_hours']:
            for variant,par in variants.items():
                shifts=[-1,0,1] if variant=='primary' else [0]
                for shift in shifts:
                    res,info,data=run_one(track,shift,window,par,variant,detail=True)
                    X,t,bm,ct,original,keep,test,purge,tm=data
                    outputs[(track,window,variant,shift)]=res
                    for model,ss in res.items():
                        rr=ranking(ss)
                        for family,score in ss.items():
```

```
                            ranks.append({'quality_track':track,'window_hours':window,'variant
                            ↪  ':variant,'anchor_id':'problem_1030','shift_hours':shift,'mode
                            ↪  l_id':model,'family_id':family,'family_name':FAMILIES.get(fami
                            ↪  ly,family),'score':finite_num(score),'rank':rr.get(family),'to
                            ↪  p5_credit':top5_credit(rr,family),'n_background_windows':len(b
                            ↪  m),'n_background_years':bm.year.nunique(),'n_process_proxies':
                            ↪  len(set(zip(bm.year,(bm.day_offset+30)//par['block']))),'n_fea
                            ↪  tures_available':int((ct.family_id==family).sum()),'evaluation
                            ↪  _stratum':'local_observational','data_role':'exploration'})
                    if info['feature_detail'] is not None:
                        df=info['feature_detail'];df['quality_track']=track;df['window_hours']
                        ↪  =window;df['variant']=variant;df['shift_hours']=shift;details.appe
                        ↪  nd(df)
                    for a in info['analogs']:analogs.append({**a,'quality_track':track,'window
                    ↪  _hours':window,'variant':variant,'shift_hours':shift})
                    bdiag.append({'quality_track':track,'window_hours':window,'variant':varian
                    ↪  t,'shift_hours':shift,**info['b_info'],'b_raw_contribution':info['b_ra
                    ↪  w_contribution']})
                    metrics.append({'quality_track':track,'window_hours':window,'variant':vari
                    ↪  ant,'shift_hours':shift,'background_windows':len(bm),'background_years
                    ↪  ':bm.year.nunique(),'process_groups':len(set(zip(bm.year,(bm.day_offse
                    ↪  t+30)//par['block']))),'feature_columns':len(ct),'b_active_features':i
                    ↪  nfo['b_info']['features'],'b_condition_number':info['b_info'].get('con
                    ↪  dition_number'),'b_training_imputation_fraction':info['b_info'].get('i
                    ↪  mputed_training_fraction'),'independent_accepted_events':0})
            # Explicitly recalculate second anchor; it should select identical labels at each
            ↪  shift.
            for shift in [-1,0,1]:
                second,_,data=run_one(track,shift,window,base,anchor='usgs_reference')
                first=outputs[(track,window,'primary',shift)]
                for model in MODELS:
                    for f in first[model]:
                        a=first[model][f];b=second[model].get(f,np.nan)
                        assert (np.isnan(a) and np.isnan(b)) or
                        ↪  np.isclose(a,b,atol=1e-12),('Anchor mismatch',f)
                for model,ss in second.items():
                    rr=ranking(ss)
                    for family,score in ss.items():
                        ranks.append({'quality_track':track,'window_hours':window,'variant':'p
                        ↪  rimary','anchor_id':'usgs_reference','shift_hours':shift,'model_id
                        ↪  ':model,'family_id':family,'family_name':FAMILIES.get(family,famil
                        ↪  y),'score':finite_num(score),'rank':rr.get(family),'top5_credit':t
                        ↪  op5_credit(rr,family),'evaluation_stratum':'local_observational','
                        ↪  data_role':'exploration'})
            print(json.dumps({'phase':'variants_done','quality_track':track,'window_hours':win
            ↪  dow,'seconds':round(time.monotonic()-t0,1)}),flush=True)
    # Target stability after purged deletion; not supervised forecast validation.
    for track in CFG['quality_tracks']:
        for window in CFG['windows_hours']:
            primary=outputs[(track,window,'primary',0)]
            _,_,bm,_,_,_,_,_,_=matrices(track,0,window,base)
            exclusions=[('year',int(y)) for y in sorted(bm.year.unique())]+[('process',p) for p
            ↪  in sorted(bm.process_id.unique())]
            for typ,value in exclusions:
                res,_,data=run_one(track,0,window,base,exclude=(typ,value))
                _,_,training,ct,original,keep,test,purge,_=data
                fold_id=f'{track}|w{window}|leave_{typ}|{value}'
                # Save exact memberships and the actual fold sizes; all folds fit afresh.
                for i,row in original.iterrows():
```

```
                    folds.append({'fold_id':fold_id,'quality_track':track,'window_hours':windo
                    ↪  w,'sample_id':row.sample_id,'year':int(row.year),'process_id':row.proc
                    ↪  ess_id,'role':'test_background' if test[i] else ('purged_overlap' if
                    ↪  purge[i] else 'training'),'support_start_utc':str(pd.Timestamp(row.anc
                    ↪  hor_time_utc)-pd.Timedelta(hours=168)),'support_end_exclusive_utc':row
                    ↪  .anchor_time_utc})
                for model,ss in res.items():
                    rr=ranking(ss);orig=ranking(primary[model]);a=set(f for f in rr if
                    ↪  top5_credit(rr,f)>0);b=set(f for f in orig if top5_credit(orig,f)>0)
                    cv_metrics.append({'fold_id':fold_id,'kind':'leave_'+typ,'quality_track':t
                    ↪  rack,'window_hours':window,'model_id':model,'n_train':len(training),'n
                    ↪  _test':int(test.sum()),'n_purged':int(purge.sum()),'n_years_train':tra
                    ↪  ining.year.nunique(),'n_process_train':training.process_id.nunique(),'
                    ↪  top5_jaccard':len(a&b)/len(a|b) if a|b else
                    ↪  np.nan,'shared_families':len(set(rr)&set(orig)),'independent_accepted_
                    ↪  events':0})
                    for family in
                    ↪  ss:stability.append({'kind':'leave_'+typ,'repeat_id':str(value),'quali
                    ↪  ty_track':track,'window_hours':window,'model_id':model,'family_id':fam
                    ↪  ily,'rank':rr.get(family),'top5_credit':top5_credit(rr,family),'select
                    ↪  ed_top5':top5_credit(rr,family)>0,'score':finite_num(ss[family])})
            print(json.dumps({'phase':'purged_folds_done','quality_track':track,'window_hours'
            ↪  :window,'folds':len(exclusions),'seconds':round(time.monotonic()-t0,1)}),flush
            ↪  =True)
    # Hierarchical draws: years then weather-process proxies, never individual hours.
    rng=np.random.default_rng(CFG['seed']);draw_specs=[]
    for rep in range(CFG['bootstrap_replicates']):
        sampled=rng.choice(CFG['background_years'],5,replace=True);draw={}
        for yr in sampled:
            yy=int(yr);groups=np.arange(9); gp=rng.choice(groups,len(groups),replace=True)
            if yy not in draw:draw[yy]={'count':0,'processes':{}}
            draw[yy]['count']+=1
            for g in gp:draw[yy]['processes'][int(g)]=draw[yy]['processes'].get(int(g),0)+1
        draw_specs.append({'repeat_id':rep,'sampled_years':sampled.tolist(),'draw':draw})
        for track in CFG['quality_tracks']:
            for window in CFG['windows_hours']:
                res,_,data=run_one(track,0,window,base,draw=draw)
                for model,ss in {**res,'consensus':consensus(res)}.items():
                    rr=ranking(ss)
                    for family in ss:stability.append({'kind':'hierarchical_bootstrap','repeat
                    ↪  _id':str(rep),'quality_track':track,'window_hours':window,'model_id':m
                    ↪  odel,'family_id':family,'rank':rr.get(family),'top5_credit':top5_credi
                    ↪  t(rr,family),'selected_top5':top5_credit(rr,family)>0,'score':finite_n
                    ↪  um(ss[family])})
        if rep%10==0:print(json.dumps({'phase':'bootstrap','replicate':rep,'seconds':round(tim
        ↪  e.monotonic()-t0,1)}),flush=True)
    pd.DataFrame(ranks).to_csv(R/'results/rankings.csv',index=False)
    rd=pd.DataFrame(ranks);top=rd[(rd.variant=='primary')&(rd.shift_hours==0)&(rd.anchor_id=='
    ↪  problem_1030')&(rd.top5_credit>0)]
    top.to_csv(R/'results/top5_candidates.csv',index=False)
    pd.concat(details,ignore_index=True).to_csv(R/'results/feature_anomalies.csv',index=False)
    pd.DataFrame(analogs).to_csv(R/'results/analog_matches.csv',index=False)
    pd.DataFrame(metrics).to_csv(R/'results/comparison_metrics.csv',index=False)
    pd.DataFrame(folds).to_csv(R/'results/fold_membership.csv',index=False)
    sd=pd.DataFrame(stability);sd.to_csv(R/'results/stability_draws.csv',index=False)
    summaries=[]
    for keys,group in
    ↪  sd.groupby(['kind','quality_track','window_hours','model_id','family_id']):
        valid=group['rank'].notna(); g=group.loc[valid]
```

```
        summaries.append(dict(zip(['kind','quality_track','window_hours','model_id','family_id
        ↪ '],keys),n_requested=len(group),n_valid=len(g),top5_frequency=float(g.top5_credit.
        ↪ mean()) if len(g) else
        ↪ np.nan,rank_median=g['rank'].median(),rank_min=g['rank'].min(),rank_max=g['rank'].
        ↪ max()))
    pd.DataFrame(summaries).to_csv(R/'results/ranking_stability.csv',index=False)
    pd.DataFrame(cv_metrics).to_csv(R/'results/fold_metrics.csv',index=False)
    dump(draw_specs,R/'results/bootstrap_draw_specification.json');dump(bdiag,R/'results/covar
    ↪ iance_diagnostics.json')
    # Consensus integrates three routes then three windows, without extra anchor votes.
    recommendations=[];rules=[]
    for track in CFG['quality_tracks']:
        window_consensus={w:consensus(outputs[(track,w,'primary',0)]) for w in
        ↪ CFG['windows_hours']}
        families=sorted(set().union(*(d.keys() for d in window_consensus.values())))
        mean={f:float(np.mean([d[f] for d in window_consensus.values()])) if
        ↪ all(np.isfinite(d.get(f,np.nan)) for d in window_consensus.values()) else np.nan
        ↪ for f in families}
        rr=ranking(mean)
        for f in families:
            freq=pd.DataFrame(summaries)
            v=freq[(freq.kind=='hierarchical_bootstrap')&(freq.quality_track==track)&(freq.mod
            ↪ el_id=='consensus')&(freq.family_id==f)]
            recommendations.append({'quality_track':track,'family_id':f,'family_name':FAMILIES
            ↪ [f],'consensus_score':finite_num(mean[f]),'recommended_rank':rr.get(f),'recomm
            ↪ ended_top5':top5_credit(rr,f)>0,'top5_credit':top5_credit(rr,f),'mean_window_b
            ↪ ootstrap_top5_frequency':v.top5_frequency.mean(),'min_window_bootstrap_valid_d
            ↪ raws':int(v.n_valid.min()) if len(v) else
            ↪ 0,'bootstrap_requested_per_window':100,'frequency_denominator':'valid_draws_on
            ↪ ly; B calibration requires >=4 distinct resampled
            ↪ years','stability_label':'stable_candidate' if len(v) and
            ↪ v.top5_frequency.mean()>=.7 and v.n_valid.min()>=70 else 'conditional_candidat
            ↪ e','evaluation_stratum':'local_observational','interpretation':'regional_pre_r
            ↪ eference_anomaly_candidate_not_causal_driver','available_time':None})
        for rule in ['A_only','B_only','C_only','median_route_rank']:
            sr={}
            for f in families:
                vals=[]
                for w in CFG['windows_hours']:
                    result=outputs[(track,w,'primary',0)];rks={m:ranking(s) for m,s in
                    ↪ result.items()}
                    if rule.endswith('_only'):
                        m=MODELS[['A_only','B_only','C_only'].index(rule)];rk=rks[m]
                        vals.append(1-(rk[f]-1)/max(len(rk)-1,1) if f in rk else np.nan)
                    else:vals.append(float(np.median([1-(rk[f]-1)/max(len(rk)-1,1) for rk in
                    ↪ rks.values() if f in rk])))
                sr[f]=float(np.mean(vals))
            for f,r in ranking(sr).items():rules.append({'quality_track':track,'rule':rule,'fa
            ↪ mily_id':f,'rank':r,'score':sr[f]})
    pd.DataFrame(recommendations).to_csv(R/'results/recommended_factors.csv',index=False)
    pd.DataFrame(rules).to_csv(R/'results/recommendation_rule_sensitivity.csv',index=False)
    # One consolidated cross-window ranking per route, requested three TOP5s.
    combined=[]
    for track in CFG['quality_tracks']:
        for model in MODELS:
            families=set().union(*(outputs[(track,w,'primary',0)][model].keys() for w in
            ↪ CFG['windows_hours']))
            sc={f:np.mean([1-(ranking(outputs[(track,w,'primary',0)][model]).get(f,np.nan)-1)/
            ↪ max(len(ranking(outputs[(track,w,'primary',0)][model]))-1,1) for w in
            ↪ CFG['windows_hours']]) for f in families}
```

```
            for f,r in ranking(sc).items():combined.append({'quality_track':track,'model_id':m
            ↪  odel,'family_id':f,'family_name':FAMILIES[f],'rank':r,'score':float(sc[f]),'to
            ↪  p5_credit':top5_credit(ranking(sc),f),'top5':top5_credit(ranking(sc),f)>0})
    pd.DataFrame(combined).to_csv(R/'results/top5_three_routes.csv',index=False)
    # Rank disagreements and predefined one-factor sensitivity, all families retained.
    differences=[]
    for track in CFG['quality_tracks']:
        for w in CFG['windows_hours']:
            ref=outputs[(track,w,'primary',0)]
            for (qt,ww,variant,shift),res in outputs.items():
                if qt!=track or ww!=w:continue
                for model in MODELS:
                    ra=ranking(ref[model]);rb=ranking(res[model]);common=sorted(set(ra)&set(rb
                    ↪  ))
                    ta={f for f in ra if top5_credit(ra,f)>0};tb={f for f in rb if
                    ↪  top5_credit(rb,f)>0}
                    tau=kendalltau([ra[f] for f in common],[rb[f] for f in common]).statistic
                    ↪  if len(common)>1 else np.nan
                    differences.append({'quality_track':track,'window_hours':w,'variant':varia
                    ↪  nt,'shift_hours':shift,'model_id':model,'top5_jaccard':len(ta&tb)/len(
                    ↪  ta|tb) if ta|tb else
                    ↪  np.nan,'kendall_tau':tau,'common_families':len(common)})
    pd.DataFrame(differences).to_csv(R/'results/sensitivity_metrics.csv',index=False)
    context={k:CFG[k] for k in ['common_baseline_commit','baseline_manifest_sha256','model_ids
    ↪  ','mechanism_id','experiment_id','scenario_id','geometry_version','parameter_version',
    ↪  'evaluation_stratum','data_role','available_time','event_id']}
    context.update(physical_phase='pre_reference_regional_background_comparison',time_utc=None
    ↪  ,time_uncertainty=None,space_object='Gyirong regional station; local-file/geodetic
    ↪  equivalence and source representativeness unverified',forecast_reference_time=None,iss
    ↪  ue_time=None,valid_time=None,source_clock_hypothesis='UTC+8; averaging intervals
    ↪  unknown',independent_accepted_training_events=0,independent_accepted_validation_events
    ↪  =0,interpretation='case anomaly rankings; no hazard discrimination/causality
    ↪  validation')
    dump(context,R/'results/model_context.json')
    for name in ['rankings.csv','top5_candidates.csv','feature_anomalies.csv','analog_matches.
    ↪  csv','comparison_metrics.csv','fold_membership.csv','stability_draws.csv','ranking_sta
    ↪  bility.csv','fold_metrics.csv','recommended_factors.csv','recommendation_rule_sensitiv
    ↪  ity.csv','top5_three_routes.csv','sensitivity_metrics.csv']:
        path=R/'results'/name
        table=pd.read_csv(path)
        table['context_ref']='results/model_context.json'
        for key in ['experiment_id','mechanism_id','scenario_id','geometry_version','parameter
        ↪  _version','evaluation_stratum','data_role']:
            table[key]=context[key]
        table.to_csv(path,index=False)
    result={'passed':True,'started_at_utc':started,'finished_at_utc':datetime.now(timezone.utc
    ↪  ).isoformat(),'duration_seconds':round(time.monotonic()-t0,3),'script_sha256':sha(Path
    ↪  (__file__)),'config_sha256':sha(R/'config/experiment.json'),'ranking_rows':len(ranks),
    ↪  'fold_membership_rows':len(folds),'stability_rows':len(stability),'bootstrap_replicate
    ↪  s':100,'bootstrap_draws_with_fewer_than_three_unique_years':sum(len(d['draw'])<3 for d
    ↪  in draw_specs),'bootstrap_draws_with_fewer_than_four_unique_years_B_crossfit_insuffici
    ↪  ent':sum(len(d['draw'])<4 for d in
    ↪  draw_specs),'checks':{'three_routes_computed':True,'strict_does_not_import_nonzero_qc'
    ↪  :True,'year_process_row_equal_weights':True,'target_not_used_for_fit_or_shrinkage_choi
    ↪  ce':True,'b_calibration_out_of_year_refitted':True,'matched_family_excluded_from_dista
    ↪  nce':True,'purged_folds_use_168h_support':True,'duplicate_minute_anchors_equal':True,'
    ↪  windows_all_retained':True,'unknown_track_separate':True,'no_independent_event_inflati
    ↪  on':True},'check_scope':'Internal implementation checks and calculations; independent
    ↪  reviewer verification separately recorded'}
    dump(result,R/'results/model_run_validation.json');print(json.dumps(result,ensure_ascii=Fa
    ↪  lse),flush=True)
```

```
405 if __name__=='__main__': main()
```

程序 3：summarize_results.py

来源：track_b/research/q1/round3/scripts/summarize_results.py
SHA256: be13ac1f5bdfaf595e76faff3f116e65b06dc75757608db637307b182f705769

```
#!/usr/bin/env python3
"""Produce scientific figures and written comparison from the computed route tables."""
import os
os.environ['MPLBACKEND']='Agg'
os.environ['MPLCONFIGDIR']='<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q1/plot_run_time_v1'
os.makedirs(os.environ['MPLCONFIGDIR'],exist_ok=True)
from pathlib import Path
import json,hashlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
R=Path(__file__).resolve().parents[1]
NAMES={'air_thermal':'Air thermal','air_moisture':'Air moisture','mean_wind':'Mean wind','air_pressure':'Air pressure','shortwave':'Shortwave','longwave':'Longwave','soil_thermal':'Soil thermal','soil_moisture':'Soil moisture','soil_heat_flux':'Soil heat flux','precipitation':'Precip label','turbulent_heat':'Turbulent heat','turbulent_mechanics':'Turbulent mechanics'}
MODELS=['A_marginal_ecdf','B_shrunk_conditional','C_other_family_analog'];TRACKS=['original_qc_view','unknown_qc_exploration']
plt.rcParams.update({'font.family':'DejaVu Sans','font.size':9,'axes.spines.top':False,'axes.spines.right':False,'savefig.dpi':180})
def csv(name):return pd.read_csv(R/'results'/name)
def save(fig,name):
 fig.savefig(R/'figures'/name,bbox_inches='tight');plt.close(fig)
def mdtable(df,cols):
 head='| '+' | '.join(cols)+' |\n| '+' | '.join('---' for _ in cols)+' |\n'
 def show(v):return '未知/不可估计' if pd.isna(v) else (f'{v:.3f}' if isinstance(v,(float,np.floating)) else str(v))
 return head+'\n'.join('| '+' | '.join(show(row[c]) for c in cols)+' |' for _,row in df.iterrows())+'\n'
def main():
 top=csv('top5_three_routes.csv');rec=csv('recommended_factors.csv');st=csv('ranking_stability.csv');sens=csv('sensitivity_metrics.csv');fold=csv('fold_metrics.csv');fa=csv('feature_anomalies.csv')
 context=json.loads((R/'results/model_context.json').read_text())
 context.update(input_version={'common_baseline_commit':context['common_baseline_commit'],'frozen_manifest_sha256':context['baseline_manifest_sha256'],'feature_bundle_version':'q1-r3-feature-v1','feature_bundle_sha256':'4e65566a1a473d7fdd16af10f209c2cb6841282b215e8c9d4981f748d6c1cc8e','advisor_q3_bundle_sha256':'04016b75b0af3be4439952705f39d83493317b2b3b594d9bed49448ea13a1ff3'},evidence_status='inference_and_candidate_from_local_numeric_facts; metadata_equivalence_unverified',variable_semantics={'contract_ref':'results/feature_contract.csv','quality_rule':'original paired QC0 remains strict in both tracks; unknown finite labels separate; 0.75 support primary with 0.50/0.90 sensitivity','units':'feature-specific contract; all scores/ranks dimensionless','integration_interval':None,'availability_status':'unknown'},model_id='multiple_models; see per-row model_id or model_ids list',space_object={'station_id':'Gyirong_regional_background','longitude':85.29,'latitude':28.86,'elevation_m':4140,'crs':None,'positional_uncertainty':None,'status':'metadata assertion; local equivalence and source-area representativeness unverified'})
 (R/'results/model_context.json').write_text(json.dumps(context,ensure_ascii=False,indent=2)+'\n')
 # Per-feature actual support distinguishes five nominal years from observed years.
 meta=csv('sample_metadata.csv');contract=csv('feature_contract.csv')
 bank=pd.read_parquet(R/'results/features.parquet').set_index('sample_id')
```

```
coverage=csv('feature_support.csv').pivot(index='sample_id',columns='driver_id',values='cover
↪ age')
diag=json.loads((R/'results/covariance_diagnostics.json').read_text())
effective=[];family_support=[]
for track in TRACKS:
 for window in [24,72,168]:
  bm=meta[(meta.quality_track==track)&(meta.window_hours==window)&(meta.shift_hours==0)&(~met
  ↪ a.is_target.astype(bool))].reset_index(drop=True)
  tm=meta[(meta.quality_track==track)&(meta.window_hours==window)&(meta.shift_hours==0)&meta.
  ↪ is_target.astype(bool)&(meta.anchor_id=='problem_1030')].iloc[0]
  bd=next(d for d in diag if d['quality_track']==track and d['window_hours']==window and
  ↪ d['variant']=='primary' and d['shift_hours']==0)
  for _,ft in contract.iterrows():
   values=bank.loc[bm.sample_id,ft.feature_id].to_numpy(float);co=coverage.loc[bm.sample_id,f
   ↪ t.driver_id].to_numpy(float)
   ok=np.isfinite(values)&(co>=.75);years=sorted(bm.loc[ok,'year'].unique().tolist())
   effective.append({'quality_track':track,'window_hours':window,'feature_id':ft.feature_id,'
   ↪ driver_id':ft.driver_id,'family_id':ft.family_id,'nominal_background_years':5,'actual_
   ↪ numeric_supported_years':len(years),'years_json':json.dumps(years),'missing_years_json
   ↪ ':json.dumps([y for y in range(2021,2026) if y not in
   ↪ years]),'n_supported_windows':int(ok.sum()),'n_nominal_windows':len(bm),'n_supported_p
   ↪ rocess_proxies':int(bm.loc[ok,'process_id'].nunique()),'per_year_supported_window_coun
   ↪ ts':json.dumps({str(y):int(((bm.year==y)&ok).sum()) for y in
   ↪ range(2021,2026)}),'target_coverage':coverage.loc[tm.sample_id,ft.driver_id],'target_v
   ↪ alue_available':bool(np.isfinite(bank.loc[tm.sample_id,ft.feature_id]) and
   ↪ coverage.loc[tm.sample_id,ft.driver_id]>=.75),'B_active_feature':ft.feature_id in
   ↪ bd.get('active_features',[]),'context_ref':'results/model_context.json'})
  for f in contract.family_id.unique():
   ee=[d for d in effective if d['quality_track']==track and d['window_hours']==window and
   ↪ d['family_id']==f]
   es=[d['actual_numeric_supported_years'] for d in ee if d['target_value_available']]
   cal=bd.get('calibration_support',{}).get(f,{})
   family_support.append({'quality_track':track,'window_hours':window,'family_id':f,'nominal_
   ↪ background_years':5,'minimum_actual_feature_years':min(es) if es else
   ↪ 0,'maximum_actual_feature_years':max(es) if es else
   ↪ 0,'target_available_feature_count':sum(d['target_value_available'] for d in
   ↪ ee),'B_active_feature_count':sum(d['B_active_feature'] for d in
   ↪ ee),'B_observed_calibration_rows':cal.get('n_observed_rows',0),'B_observed_calibration
   ↪ _years':cal.get('n_observed_years',0),'B_observed_calibration_processes':cal.get('n_ob
   ↪ served_processes',0),'context_ref':'results/model_context.json'})
pd.DataFrame(effective).to_csv(R/'results/effective_feature_support.csv',index=False)
pd.DataFrame(family_support).to_csv(R/'results/effective_family_support.csv',index=False)
summary=[]
for track in TRACKS:
 for model in MODELS:
  t=top[(top.quality_track==track)&(top.model_id==model)&top.top5.astype(bool)].sort_values('
  ↪ rank')
  sf=st[(st.quality_track==track)&(st.model_id==model)&(st.kind=='hierarchical_bootstrap')]
  fy=fold[(fold.quality_track==track)&(fold.model_id==model)&(fold.kind=='leave_year')]
  fp=fold[(fold.quality_track==track)&(fold.model_id==model)&(fold.kind=='leave_process')]
  ss=sens[(sens.quality_track==track)&(sens.model_id==model)]
  summary.append({'quality_track':track,'model_id':model,'three_window_top5':'、
  ↪ '.join(t.family_name),'leave_year_mean_jaccard':fy.top5_jaccard.mean(),'leave_process_m
  ↪ ean_jaccard':fp.top5_jaccard.mean(),'leave_process_min_jaccard':fp.top5_jaccard.min(),'
  ↪ anchor_shift_min_jaccard':ss[(ss.variant=='primary')&(ss.shift_hours!=0)].top5_jaccard.
  ↪ min(),'predefined_sensitivity_min_jaccard':ss[ss.variant!='primary'].top5_jaccard.min()
  ↪ ,'bootstrap_min_valid_draws':sf.n_valid.min(),'bootstrap_max_valid_draws':sf.n_valid.ma
  ↪ x(),'bootstrap_requested':100,'independent_accepted_events':0})
comp=pd.DataFrame(summary);comp['context_ref']='results/model_context.json';comp.to_csv(R/'re
↪ sults/comparison_summary.csv',index=False)
quality=[]
for model in MODELS:
```

```
 a=top[(top.quality_track==TRACKS[0])&(top.model_id==model)].set_index('family_id');b=top[(to
 ↪  p.quality_track==TRACKS[1])&(top.model_id==model)].set_index('family_id')
 for f in sorted(set(a.index)|set(b.index)):
  quality.append({'model_id':model,'family_id':f,'rank_original_qc':a.loc[f,'rank'] if f in
  ↪  a.index else None,'rank_unknown_included':b.loc[f,'rank'] if f in b.index else
  ↪  None,'rank_change':b.loc[f,'rank']-a.loc[f,'rank'] if f in a.index and f in b.index else
  ↪  None,'top5_credit_original_qc':a.loc[f,'top5_credit'] if f in a.index else
  ↪  None,'top5_credit_unknown_included':b.loc[f,'top5_credit'] if f in b.index else
  ↪  None,'context_ref':'results/model_context.json'})
pd.DataFrame(quality).to_csv(R/'results/quality_sensitivity.csv',index=False)
# Rank heatmaps include every family, missing remains visibly blank.
fig,axes=plt.subplots(1,2,figsize=(12,6),constrained_layout=True)
for ax,track,title in zip(axes,TRACKS,['Original QC0 view','QC0 + unknown exploration']):
 t=top[top.quality_track==track];families=[f for f in NAMES if f in set(t.family_id)]
 matrix=t.pivot(index='family_id',columns='model_id',values='rank').reindex(index=families,co
 ↪  lumns=MODELS)
 rr=rec[rec.quality_track==track].set_index('family_id');matrix['Consensus']=rr.recommended_r
 ↪  ank.reindex(families)
 im=ax.imshow(matrix.to_numpy(),cmap='viridis_r',vmin=1,vmax=12,aspect='auto')
 ax.set_yticks(range(len(families)),[NAMES[f] for f in families]);ax.set_xticks(range(4),['A
 ↪  marginal','B conditional','C
 ↪  analog','Consensus'],rotation=20,ha='right');ax.set_title(title)
 for i in range(len(families)):
  for j in range(4):
   v=matrix.iloc[i,j];ax.text(j,i,'NA' if pd.isna(v) else
   ↪  f'{v:g}',ha='center',va='center',color='white' if pd.notna(v) and v>=7 else
   ↪  '#102030',fontsize=9)
fig.suptitle('Physical-family ranks across the three fixed windows\nRegional station case
↪  anomalies; tied scores retain midranks; NA is not normal')
fig.colorbar(im,ax=axes,label='Rank (lower = greater relative anomaly)',shrink=.75)
save(fig,'route_rank_comparison.png')
# Conditional bootstrap frequencies and denominator explicitly distinguished.
fig,axes=plt.subplots(1,2,figsize=(13,6),constrained_layout=True)
for ax,track,title in zip(axes,TRACKS,['Original QC0','QC0 + unknown']):
 t=st[(st.kind=='hierarchical_bootstrap')&(st.quality_track==track)]
 families=[f for f in NAMES if f in t.family_id.unique()]
 data=t.groupby(['family_id','model_id']).top5_frequency.mean().unstack().reindex(index=famil
 ↪  ies,columns=MODELS+['consensus'])
 im=ax.imshow(data.to_numpy(),vmin=0,vmax=1,cmap='Blues',aspect='auto')
 ax.set_yticks(range(len(families)),[NAMES[f] for f in
 ↪  families]);ax.set_xticks(range(4),['A','B','C','Consensus']);ax.set_title(title)
 for i in range(len(families)):
  for j,m in enumerate(data.columns):
   v=data.iloc[i,j];rows=t[(t.family_id==families[i])&(t.model_id==m)];n=int(rows.n_valid.min
   ↪  ()) if len(rows) else 0
   ax.text(j,i,'NA' if pd.isna(v) else
   ↪  f'{v:.2f}\nn≥{n}',ha='center',va='center',fontsize=7,color='white' if pd.notna(v) and
   ↪  v>.65 else '#17324a')
fig.suptitle('Top-five fractional inclusion, averaged over 24 / 72 / 168h\nConditional on
↪  estimable draws out of 100; these are stability frequencies, not event probabilities')
fig.colorbar(im,ax=axes,label='Mean valid-draw top-five
↪  credit',shrink=.75);save(fig,'bootstrap_stability.png')
# Actual feature directions: same fixed representative median features.
fixed=['air_temperature_2m','relative_humidity_2m','wind_speed_10m','air_pressure','shortwave
↪  _down','longwave_down','soil_temperature_0p1m','soil_vwc_0p1m','soil_heat_flux_0p1m','pre
↪  cipitation_label','latent_heat_flux','friction_velocity']
t=fa[(fa.variant=='primary')&(fa.shift_hours==0)&(fa.statistic=='median')]
directions=t[t.driver_id.isin(fixed)].copy();directions.to_csv(R/'results/factor_directions.c
↪  sv',index=False)
fig,axes=plt.subplots(1,2,figsize=(12,6),constrained_layout=True)
for ax,track,title in zip(axes,TRACKS,['Original QC0','QC0 + unknown']):
```

```
 data=directions[directions.quality_track==track].pivot(index='driver_id',columns='window_hours',values='background_percentile')
 order=[d for d in fixed if d in data.index];data=data.reindex(order)
 im=ax.imshow(data.to_numpy(),vmin=0,vmax=1,cmap='coolwarm',aspect='auto')
 ax.set_yticks(range(len(data)),[d.replace('_',' ') for d in data.index]);ax.set_xticks(range(len(data.columns)),[f'{w}h' for w in data.columns]);ax.set_title(title)
 for i in range(len(data)):
  for j in range(len(data.columns)):
   v=data.iloc[i,j];ax.text(j,i,'NA' if pd.isna(v) else f'{v:.2f}',ha='center',va='center',fontsize=9)
fig.suptitle('Direction of fixed representative window medians\nYear / process weighted seasonal background ECDF; percentiles are descriptive, not p-values')
fig.colorbar(im,ax=axes,label='Background percentile',shrink=.75);save(fig,'feature_direction_percentiles.png')
# All predefined sensitivities, not a selected favourable subset.
fig,axes=plt.subplots(1,2,figsize=(12,6),constrained_layout=True)
for ax,track,title in zip(axes,TRACKS,['Original QC0','QC0 + unknown']):
 t=sens[sens.quality_track==track].copy();t['comparison']=np.where(t.variant=='primary','anch or '+t.shift_hours.astype(str)+'h',t.variant)
 data=t.groupby(['comparison','model_id']).top5_jaccard.mean().unstack().reindex(columns=MODELS)
 im=ax.imshow(data.to_numpy(),vmin=0,vmax=1,cmap='YlGnBu',aspect='auto');ax.set_yticks(range(len(data)),data.index);ax.set_xticks(range(3),['A','B','C']);ax.set_title(title)
 for i in range(len(data)):
  for j in range(3):ax.text(j,i,f'{data.iloc[i,j]:.2f}',ha='center',va='center',color='white' if data.iloc[i,j]>.65 else '#102030')
fig.suptitle('Top-five set overlap with the preregistered primary setting\nJaccard averaged over fixed windows; boundary ties retained')
fig.colorbar(im,ax=axes,label='Jaccard overlap',shrink=.75);save(fig,'sensitivity_comparison.png')
# Source-label time series: four data classes, last 168h before the fixed primary reference.
hourly=pd.read_parquet(R/'exports/hourly_drivers.parquet')
hourly['time_utc']=pd.to_datetime(hourly.time_utc,utc=True)
reference=pd.Timestamp('2026-08-26T02:30:00Z')
panels=[('air_temperature_2m','Air temperature 2m / degC',TRACKS[0]),('relative_humidity_2m','Relative humidity 2m / %',TRACKS[0]),('wind_speed_10m','Wind speed 10m / m s-1',TRACKS[0]),('shortwave_down','Downward shortwave / W m-2',TRACKS[0]),('soil_temperature_0p1m','Soil temperature 0.1m / degC',TRACKS[0]),('soil_vwc_0p1m','Soil water 0.1m / m3 m-3',TRACKS[0]),('precipitation_label','Precip labels / mm, interval unknown',TRACKS[1]),('latent_heat_flux','Latent heat / W m-2, QC unknown',TRACKS[1])]
fig,axes=plt.subplots(4,2,figsize=(13,9),sharex=True,constrained_layout=True)
import matplotlib.dates as mdates
for ax,(driver,label,track) in zip(axes.flat,panels):
 hh=hourly[(hourly.quality_track==track)&(hourly.time_utc>=reference-pd.Timedelta(hours=168))&(hourly.time_utc<reference)]
 ax.plot(hh.time_utc,hh[driver],lw=1,color='#235d79' if track==TRACKS[0] else '#a65c28')
 ax.set_title(label,fontsize=9,loc='left');ax.grid(alpha=.16);ax.set_xlim(reference-pd.Timedelta(hours=168),reference);ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
for ax in axes[-1]:ax.set_xlabel('UTC hour labels; 2026')
fig.suptitle('Observed regional station context, fixed 168h before the 02:30 UTC reference\nOriginal QC0 in blue; unknown QC in brown; gaps retained; hourly integration boundaries unverified')
save(fig,'event_hourly_context.png')
# Written model comparison with actual computed tables.
text='''# Q1 第三轮模型竞争与推荐

已实际完成三路线及全部预登记敏感性、留年/过程删除、100次年→过程重采样。
结果层级为区域站本次参考时刻前的异常候选；不是灾害因果、风险概率或已验证触发TOP5。
精确启动真值与独立灾害训练/验证事件均未新增。
```

```
## 三套候选 TOP5（跨 24/72/168h 等权名次）

'''
 for track,label in zip(TRACKS,['原 QC0 主轨','QC0 + unknown 探索轨']):
  text+='### '+label+'\n\n'
  for model in MODELS:
   d=top[(top.quality_track==track)&(top.model_id==model)&top.top5.astype(bool)].sort_values('rank')
   text+='**'+model+'**\n\n'+mdtable(d,['family_name','rank','score','top5_credit'])+'\n'
 text+='''## 预登记共识推荐

主报告保留三路线共识：A提供可解释的边际异常底座，B为共线条件异常挑战路线，C为天气过程对照选择挑战路线。没有独立灾害真值，不能以本次目标的异常大小宣称某路线预测获胜。B对组数/缺测和收缩假设敏感，C的对照是匹配代理过程，均保留作挑战证据。

'''
 for track,label in zip(TRACKS,['原 QC0 主轨','unknown 纳入影响']):
  d=rec[(rec.quality_track==track)&rec.recommended_top5.astype(bool)].sort_values('recommended_rank')
  text+='**'+label+'**\n\n'+mdtable(d,['family_name','recommended_rank','consensus_score','mean_window_bootstrap_top5_frequency','min_window_bootstrap_valid_draws','stability_label'])+'\n'
 q=rec[rec.quality_track==TRACKS[0]].set_index('family_id')
 text+=f"第五席需要保留争议：主点估计的平均风场跨窗平均入选频率为{q.loc['mean_wind','mean_window_bootstrap_top5_frequency']:.3f}，候补土壤热通量为{q.loc['soil_heat_flux','mean_window_bootstrap_top5_frequency']:.3f}；两者以每窗44个有效共识draw为条件。按预登记共识仍保留平均风场为第5，但不应将其写成稳定唯一第五致灾因素。\n\n"
 text+='''TOP5边界若并列，候选表保留所有有第五席fractional credit的族，不用字母序制造胜负。主推荐仅为9个已有QC族中的相对异常，unknown轨的新增族不自动得到质量认证；某族在B中因零尺度/不足支持不可估计时，三路共识保持null。

## 实际稳定性与比较分母

'''+mdtable(comp,['quality_track','model_id','leave_year_mean_jaccard','leave_process_mean_jaccard','anchor_shift_min_jaccard','predefined_sensitivity_min_jaccard','bootstrap_min_valid_draws','bootstrap_max_valid_draws'])+'''
`rankings.n_background_years=5`是名义抽取池；每特征实际有效年/过程、缺年与逐年窗口数见effective_feature_support，家族及B校准分母见effective_family_support，尤其unknown通量不能读成全部5年。所有Jaccard是目标排名对训练背景变动的敏感性，非灾害分类准确率。5个背景年与45个7日块代理不等于45个独立灾害。重采样中不足4个唯一年份会使B的留年校准不可估计；其频率仅以有效draw为分母，完整n_requested/n_valid在ranking_stability.csv。推荐表有效draw不足70/100不标稳定。原始100次抽样全部保存，不重抽凑数。

## 方法分歧与限制

A对常值和偏态稳健，但边际差异不排除共线；B可减弱线性共享变化，但收缩、维数、填补和高斯二次距离不是已证实物理过程；C在待评族之外匹配且每年选不同代理过程，仍可能遗漏真实气象背景。比较基于相同因素族、质量轨和窗口，路线间的原始分数不可直接当同一种概率。

方向需逐描述量读取factor_directions和feature_anomalies，不可把整族非负异常分改写成单一升温/增湿方向。名义垂直/土层梯度仅属区域站；降水标签分布不是累计雨量；辐射差值未经本地仪器符号认证；FLUX质量/处理/小时区间unknown；VPD是否依赖已知常值amb_e_sat的上游处理也未证实，不能把它看作独立湿度证据。未取得独立源区配对与位置基准，站点异常不能直接作为源区物理输入。

## 题面要求对照与后续使用

四类气象资料均有变量映射、特征、覆盖和异常；三套TOP5、主推荐、方向、留年/过程/窗口/锚点/质量敏感性均有真实计算。统计关联支线未启动：Q3审核包仍有0个接受的独立启动/暴露单位，不能把区域背景年当无灾标签。Q2/Q3先接收冻结feature_bundle及限制，正式物理输入和历史业务可用性另由主参谋审查。
```

```
'''
 (R/'MODEL_COMPARISON.md').write_text(text)
 paper='''# Q1 论文方法、结果与讨论提纲

## 研究问题与可写主张

研究本次参考时段前，吉隆区域站气象—土壤—通量变量相对近年同季节背景的异常候选因素。明确题面启动对象与参考时间差别，排名为个案关联线索，后续Q2机制验证与Q3外部事件验证各自保留。

## 方法材料

1. 数据：四类原附件/规范化输入、37个原QC变量与26个unknown变量的逐列映射；24个驱动、112个描述量，给出名义层间梯度、趋势与持续链定义，缺口不插值。
2. 背景：2021—2025季节±30日、每天固定时刻、24/72/168h；年→7日过程代理→锚点等权。报告实际305窗/窗口、5年、45代理过程，不能当独立灾害数；±15日与3/14日块对照。
3. A加权ECDF双侧异常；B稳健缩放、Σ=(1−λ)C+λI及Schur条件距离/自由度、留年观测校准；C剔除待评族的等族匹配、每年k个不同过程。写出所有超参数先于计算登记，原始分数不作p值。
4. 因素族层级聚合与跨路线/窗口名次共识，保留边界并列credit；五年留出、真实168h支持清除重叠、100个层级draw、有效分母与两锚点±1h小时集合。

## 结果呈现

'''
 d=rec[(rec.quality_track==TRACKS[0])&rec.recommended_top5.astype(bool)].sort_values('recommended_rank')
 paper+=mdtable(d,['family_name','recommended_rank','consensus_score','mean_window_bootstrap_top5_frequency','min_window_bootstrap_valid_draws'])
 paper+='''
结果图1：三路线/质量轨全因素族名次。图2：有效draw条件下的TOP5频率与分母。图3：预先固定代表变量的方向/背景百分位。图4：全部预定敏感性Jaccard。图5：原参考前168h四类资料实际逐小时标签时序，缺口保留。表格同时呈现三套TOP5、unknown新增因素和B不可估计族，不只报告一致结果。

## 讨论与跨问衔接

解释共识与分歧对应何种统计问题，不用异常度证明机制。区域站坐标/高度为元数据声明，站源转移未校验；约65km指站至口岸，不是站至失稳源区。FLUX处理/QC、小时时段和真实available_time未闭环，unknown结果降低解释层级。相邻小时和同链目录不充独立事件；源区首次启动、传播和到达时间分别引用Q3/Q4。五年背景限制外推与bootstrap可估计性，零尺度不是零风险。

## 可复现与披露

引用EXPERIMENT_PROTOCOL、PROTOCOL_DEVIATIONS、REPRODUCE和来源/输入/产物清单，正文写明AI辅助实现/资料整理、独立AI技术复核范围及队员最终科学采用。当前未建立外部灾害关联/预测效能验证，最终论文须保留此范围或在取得真实独立证据后增加单独研究支线。
'''
 (R/'PAPER_OUTLINE.md').write_text(paper)
 print(json.dumps({'figures_created':5,'comparison_rows':len(comp),'quality_comparison_rows':len(quality),'recommendation_rows':len(rec)}))
if __name__=='__main__':main()
```

程序 4：q1_figures.py

来源：track_b/paper/scripts/q1_figures.py
SHA256: 274a8ec170821f08dc2ae815b3f5cff5075b1a2418ce005c8c25fe29ad65d55d

```
#!/usr/bin/env python3
"""Redraw Q1 paper figures from frozen round-3 outputs; never fit a model.

Run from the repository root:
    track_b/.venv/bin/python track_b/paper/scripts/q1_figures.py
"""
from pathlib import Path
import json
import hashlib

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

ROOT = Path(__file__).resolve().parents[3]
R3 = ROOT / "track_b/research/q1/round3"
OUT = ROOT / "track_b/paper/figures/q1"
QC = "original_qc_view"
MODELS = ["A_marginal_ecdf", "B_shrunk_conditional", "C_other_family_analog"]
FONT_PATHS = [Path.home() / "Library/Fonts/PingFang Regular_0.ttf",
              Path.home() / "Library/Fonts/SimSun.ttf",
              Path("/System/Library/Fonts/Supplemental/Songti.ttc")]
font_path = next(p for p in FONT_PATHS if p.is_file())
font_manager.fontManager.addfont(str(font_path))
plt.rcParams.update({
    "font.family": font_manager.FontProperties(fname=str(font_path)).get_name(),
    "font.size": 8.3, "axes.labelsize": 8.3, "axes.titlesize": 9,
    "xtick.labelsize": 8, "ytick.labelsize": 8,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.unicode_minus": False, "pdf.fonttype": 42, "ps.fonttype": 42,
    "savefig.dpi": 300, "mathtext.fontset": "dejavusans",
})
OUT.mkdir(parents=True, exist_ok=True)
sources = set()


def read_csv(name):
    p = R3 / "results" / name
    sources.add(p)
    return pd.read_csv(p)


def save(fig, name):
    for ext in ["pdf", "png"]:
        meta = {"CreationDate": None, "ModDate": None, "Creator": "Q1 paper plotting"} if ext
        ↪  == "pdf" else None
        fig.savefig(OUT / f"{name}.{ext}", facecolor="white", bbox_inches="tight",
        ↪  pad_inches=.04, metadata=meta)
    plt.close(fig)


def annotated_heatmap(ax, values, rows, cols, title, cmap="Blues"):
    im = ax.imshow(values, vmin=0, vmax=1, cmap=cmap, aspect="auto")
    ax.set_xticks(range(len(cols)), cols)
```

```
    ax.set_yticks(range(len(rows)), rows)
    ax.set_title(title, loc="left", pad=7)
    ax.tick_params(length=0)
    for spine in ax.spines.values():
        spine.set_visible(False)
    for (i, j), val in np.ndenumerate(values):
        ax.text(j, i, f"{val:.2f}", ha="center", va="center", fontsize=7.7,
                color="white" if val > .65 else "#162e3b")
    return im


def state_figure():
    p = R3 / "exports/hourly_drivers.parquet"
    sources.add(p)
    hourly = pd.read_parquet(p)
    reference = pd.Timestamp("2026-08-26T02:30:00Z")
    time_ns = pd.to_datetime(hourly.time_utc, utc=True).astype("datetime64[ns, UTC]").astype("int64")
    subset = hourly[(hourly.quality_track == QC) &
                    (time_ns >= reference.value - 168 * 3600 * 10**9) & (time_ns < reference.value)].copy()
    x = (time_ns.loc[subset.index] - reference.value) / (3600 * 10**9)
    y = subset.soil_vwc_0p1m.to_numpy()
    directions = read_csv("factor_directions.csv")
    ids = ["air_temperature_2m", "relative_humidity_2m", "wind_speed_10m",
           "soil_temperature_0p1m", "soil_vwc_0p1m", "soil_heat_flux_0p1m", "shortwave_down"]
    names = ["2 m 气温", "2 m 相对湿度", "10 m 风速", "0.1 m 土温",
             "0.1 m 含水率", "0.1 m 土热通量", " 向下短波"]
    d = directions[(directions.quality_track == QC) & (directions.statistic == "median")]
    values = d.pivot(index="driver_id", columns="window_hours", values="background_percentile").loc[ids, [24, 72, 168]].to_numpy()
    fig, (ax, hm) = plt.subplots(1, 2, figsize=(6.65, 2.16),
                                 gridspec_kw={"width_ratios": [1.18, .9], "wspace": .54})
    ax.plot(x, y, color="#17657c", lw=1.25)
    ax.axvspan(-24, 0, color="#edf2f3", zorder=-2)
    ax.scatter([x.iloc[0], x.iloc[-1]], [y[0], y[-1]], s=13, color="#b56b35", zorder=3)
    ax.annotate(f"{y[0]:.4f}", (x.iloc[0], y[0]), xytext=(4, -13),
                textcoords="offset points", fontsize=8, color="#8b4f26")
    ax.annotate(f"{y[-1]:.5f}", (x.iloc[-1], y[-1]), xytext=(-5, 8),
                textcoords="offset points", ha="right", fontsize=8, color="#8b4f26")
    ax.set(xlim=(-170, 1), ylim=(.06, .18), xlabel=" 距参考时刻 / h",
           ylabel=r" 体积含水率 / $\mathrm{m^3\,m^{-3}}$")
    ax.set_xticks([-168, -120, -72, -24, 0])
    ax.grid(axis="y", color="#e2e7ea", lw=.5)
    ax.set_title("(a) 土壤水分的实际变化", loc="left", pad=7)
    im = annotated_heatmap(hm, values, names, ["24 h", "72 h", "168 h"],
                           "(b) 窗口中位数的季节背景位置", "YlGnBu")
    cb = fig.colorbar(im, ax=hm, fraction=.06, pad=.035, ticks=[0, .5, 1])
    cb.ax.set_ylabel(" 加权背景百分位", fontsize=8)
    save(fig, "q1_state")
    return {"reference_utc": reference.isoformat(), "n_vwc": int(np.isfinite(y).sum()),
            "vwc_first": float(y[0]), "vwc_last": float(y[-1]),
            "representative_driver_ids": ids, "windows_hours": [24, 72, 168],
            "median_percentiles": values.tolist()}


def stability_figure():
    rec = read_csv("recommended_factors.csv")
    rec = rec[rec.quality_track == QC].sort_values("recommended_rank")
    ids, names = rec.family_id.tolist(), rec.family_name.tolist()
    draws = read_csv("ranking_stability.csv")
    d = draws[(draws.quality_track == QC) & (draws.kind == "hierarchical_bootstrap")]
```

```
    freq = d.groupby(["family_id", "model_id"]).top5_frequency.mean().unstack().loc[ids, MODELS]
    valid = {model: sorted(d[d.model_id == model].n_valid.unique().astype(int).tolist()) for model in MODELS}
    assert valid == {MODELS[0]: [89], MODELS[1]: [44], MODELS[2]: [89]}, valid
    s = read_csv("sensitivity_metrics.csv")
    s = s[s.quality_track == QC]
    groups = [
        (" 参考时刻 ±1 h", (s.variant == "primary") & (s.shift_hours != 0)),
        (" 过程块 3 / 14 d", s.variant.isin(["process_3", "process_14"])),
        (" 季节范围 ±15 d", s.variant == "season_15"),
        (" 支持率 50% / 90%", s.variant.isin(["support_0.5", "support_0.9"])),
        (" 收缩系数 0.1 / 0.9", s.variant.isin(["shrinkage_0.1", "shrinkage_0.9"])),
        (" 每年类比数 1 / 3", s.variant.isin(["analogs_1", "analogs_3"])),
    ]
    sensitivity = np.array([s[mask].groupby("model_id").top5_jaccard.mean().reindex(MODELS).to_numpy()
                            for _, mask in groups])
    fig, (ax, bx) = plt.subplots(1, 2, figsize=(6.65, 2.25),
                                 gridspec_kw={"width_ratios": [.94, .94], "wspace": .77})
    im = annotated_heatmap(ax, freq.to_numpy(), names, ["A (89)", "B (44)", "C (89)"],
                           "(a) 三窗平均 TOP5 入选频率")
    cb = fig.colorbar(im, ax=ax, fraction=.06, pad=.035, ticks=[0, .5, 1])
    im2 = annotated_heatmap(bx, sensitivity, [label for label, _ in groups], ["A", "B", "C"],
                            "(b) 预设变动下的集合一致性", "YlGnBu")
    cb2 = fig.colorbar(im2, ax=bx, fraction=.06, pad=.035, ticks=[0, .5, 1])
    save(fig, "q1_stability")
    return {"family_ids": ids, "models": MODELS, "frequency_mean_over_three_windows": freq.to_numpy().tolist(),
            "valid_bootstrap_draws_per_window": valid, "requested_draws_per_window": 100,
            "sensitivity_categories": [label for label, _ in groups],
            "jaccard_mean_over_three_windows_and_listed_variants": sensitivity.tolist()}


if __name__ == "__main__":
    payload = {"state": state_figure(), "stability": stability_figure(),
               "font": str(font_path), "fit_performed": False}
    payload["source_files"] = [{"path": str(p.relative_to(ROOT)), "sha256": hashlib.sha256(p.read_bytes()).hexdigest()}
                               for p in sorted(sources)]
    (OUT / "q1_figure_values.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2, allow_nan=False) + "\n")
    print("Wrote q1_state.{pdf,png}, q1_stability.{pdf,png}, q1_figure_values.json")
```

程序 5：q1_evidence.py

来源：track_b/paper/scripts/q1_evidence.py
SHA256: 219fb7118b02591288a8c9cb96f4c8ceb886746419b1743e21a1f84a3b516131

```
#!/usr/bin/env python3
"""Register Q1 paper claims from frozen R3 files; fit no model and draw no figure.

Run: track_b/.venv/bin/python -B track_b/paper/scripts/q1_evidence.py
Writes only evidence/q1_claims.json, q1_code_files.json, q1_numeric_review.json.
AI-assisted implementation: OpenAI Codex, 2026-10-24.
"""
from __future__ import annotations

from collections import Counter
import hashlib
import json
from pathlib import Path
import re

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[3]
PAPER = ROOT / 'track_b/paper'
R3 = ROOT / 'track_b/research/q1/round3'
QC = 'original_qc_view'
UNKNOWN = 'unknown_qc_exploration'
MODELS = ['A_marginal_ecdf', 'B_shrunk_conditional', 'C_other_family_analog']
WINDOWS = [24, 72, 168]


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def dump(path, value):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n')


def records(frame):
    return json.loads(frame.to_json(orient='records', force_ascii=False, double_precision=15))


def main():
    checks = Counter()

    def check(condition, category, detail=''):
        if not bool(condition):
            raise AssertionError((category, detail))
        checks[category] += 1

    input_manifest = R3 / 'results/input_manifest.json'
    output_manifest = R3 / 'output_manifest.json'
    manifest_index = {}
    manifests = {}
    for manifest_path, base in [(input_manifest, ROOT), (output_manifest, R3)]:
        manifest = json.loads(manifest_path.read_text())
        for entry in manifest['files']:
            path = (base / entry['path']).resolve()
            check(path.is_relative_to(ROOT), 'manifest_path_within_repository', entry['path'])
```

```
            check(path.stat().st_size == entry['bytes'] and sha(path) == entry['sha256'],
                  'frozen_manifest_file_hash', str(path))
            relative = str(path.relative_to(ROOT))
            if relative in manifest_index:
                check(manifest_index[relative]['sha256'] == entry['sha256'],
                      'overlapping_manifest_entries_agree', relative)
            manifest_index[relative] = {'path':relative, 'bytes':entry['bytes'],
            ↪  'sha256':entry['sha256']}
        manifests[str(manifest_path.relative_to(ROOT))] = {
            'bytes':manifest_path.stat().st_size, 'sha256':sha(manifest_path),
            'file_count':len(manifest['files'])}

    source_snapshots = {}

    def source(path):
        path = Path(path)
        if not path.is_absolute():
            path = ROOT / path
        path = path.resolve()
        relative = str(path.relative_to(ROOT))
        value = {'path':relative, 'sha256':sha(path)}
        if path.is_relative_to(R3) or relative.startswith('track_b/data/') or relative ==
        ↪  'track_b/config/temporal_policy.json':
            check(relative in manifest_index and manifest_index[relative]['sha256'] ==
            ↪  value['sha256'],
                  'claim_source_bound_to_frozen_manifest', relative)
        source_snapshots[relative] = {'bytes':path.stat().st_size, 'sha256':value['sha256']}
        return value

    def csv(name):
        path = R3 / 'results' / name
        source(path)
        return pd.read_csv(path, low_memory=False)

    cfg_path = R3 / 'config/experiment.json'
    features_path = R3 / 'config/features.json'
    cfg = json.loads(cfg_path.read_text())
    feature_cfg = json.loads(features_path.read_text())
    policy_path = ROOT / 'track_b/config/temporal_policy.json'
    policy = json.loads(policy_path.read_text())
    dictionary_path = ROOT / 'track_b/data/processed/temporal/data_dictionary.json'
    dictionary = json.loads(dictionary_path.read_text())['hourly']
    text_path = PAPER / 'sections/q1.tex'
    text = text_path.read_text()
    source(text_path)
    figures_path = PAPER / 'figures/q1/q1_figure_values.json'
    figure_values = json.loads(figures_path.read_text())
    source(figures_path)
    for entry in figure_values['source_files']:
        check(source(entry['path'])['sha256'] == entry['sha256'], 'figure_source_hash',
        ↪  entry['path'])
    check(figure_values['fit_performed'] is False, 'figure_redraw_not_model_fit')

    contract = csv('feature_contract.csv')
    variable_map = csv('variable_family_map.csv')
    meta = csv('sample_metadata.csv')
    routes = csv('top5_three_routes.csv')
    recommended = csv('recommended_factors.csv')
    stability = csv('ranking_stability.csv')
    fold_metrics = csv('fold_metrics.csv')
    memberships = csv('fold_membership.csv')
    directions = csv('factor_directions.csv')
    state = csv('case_state_evidence.csv')
```

```
    sensitivity = csv('sensitivity_metrics.csv')
    effective = csv('effective_family_support.csv')
    diagnostics_path = R3 / 'results/covariance_diagnostics.json'
    diagnostics = json.loads(diagnostics_path.read_text())
    draws_path = R3 / 'results/bootstrap_draw_specification.json'
    draws = json.loads(draws_path.read_text())
    source(draws_path)
    source(diagnostics_path)
    for path in [cfg_path, features_path, policy_path, dictionary_path]:
        source(path)

    claims = []

    def claim(identifier, statement, value, unit, paths, anchor, model=None, scope=None):
        if anchor.startswith('q1-sec/'):
            anchor = 'q1-sec'
        claims.append({'claim_id':identifier, 'statement':statement, 'value':value,
        ↪  'unit':unit,
            'source_files':[source(p) for p in paths], 'model_id':model,
            'scenario_id':cfg['scenario_id'],
            'scope':scope or 'Regional background station, local_observational/exploration;
            ↪  case anomaly description, not causal or hazard-probability validation.',
            'paper_anchor':anchor})

    dictionary_by_id = {item['variable_id']:item for item in dictionary}
    driver_by_id = {item['driver_id']:item for item in feature_cfg['drivers']}
    table_counts = {}
    for kind in ['MET', 'RADM', 'SOIL', 'FLUX']:
        drivers = [d for d in feature_cfg['drivers'] if
        ↪  dictionary_by_id[d['source_variables'][0]]['kind'] == kind]
        known_drivers = {d['driver_id'] for d in drivers if d['availability_track'] ==
        ↪  'both_tracks_original_qc0'}
        all_drivers = {d['driver_id'] for d in drivers}
        known_count = int(contract.driver_id.isin(known_drivers).sum())
        unknown_count = int(contract.driver_id.isin(all_drivers-known_drivers).sum())
        table_counts[kind] = [sum(d['kind']==kind for d in dictionary), len(drivers),
        ↪  known_count, unknown_count]
        claim('q1-counts-'+kind.lower(), f'{kind} 原变量、驱动量、主轨特征及探索增量的实际规模。',
              {'raw_variables':table_counts[kind][0], 'drivers':len(drivers),
               'original_qc_features':known_count, 'unknown_increment_features':unknown_count},
              'counts', [dictionary_path, features_path, R3/'results/feature_contract.csv'],
              ↪  'q1-tab-features')
    totals = np.sum(list(table_counts.values()),axis=0).astype(int).tolist()
    known_families = {d['family_id'] for d in feature_cfg['drivers'] if
    ↪  d['availability_track']=='both_tracks_original_qc0'}
    all_families = {d['family_id'] for d in feature_cfg['drivers']}
    check(totals==[63,24,82,30] and len(known_families)==9 and
    ↪  len(all_families-known_families)==3,
          'feature_table_totals')
    check(len(variable_map)==63, 'all_raw_variables_mapped')
    statistic_counts = contract.statistic.value_counts().astype(int).to_dict()
    claim('q1-feature-schema-total', '完整特征库包含24驱动及112列，主轨82列、探索增量30列；
    ↪  18主轨驱动和6探索驱动。',
          {'raw_variables':63, 'drivers':24, 'original_qc_drivers':18, 'unknown_drivers':6,
           'features':112, 'original_qc_features':82, 'unknown_increment_features':30,
           'main_families':9, 'new_unknown_families':3, 'ordinary_drivers_times_five':21,
           'direction_drivers_times_one':2, 'precipitation_drivers_times_five':1,
           'statistic_counts':statistic_counts}, 'counts',
          [dictionary_path, features_path, R3/'results/feature_contract.csv',
          ↪  R3/'results/variable_family_map.csv'],
          'q1-tab-features')

    station = policy['hourly']['station']
```

```
    reference = pd.Timestamp(cfg['anchor_sources'][0]['source_time']).tz_convert('UTC')
    check(reference.isoformat()=='2026-08-26T02:30:00+00:00', 'reference_clock_conversion')
    claim('q1-station-and-clock', '区域背景站元数据位置及源时钟换算；坐标与海拔是元数据声明。',
          {'latitude':station['latitude'],'longitude':station['longitude'],'elevation_m':stati
          ↪ on['elevation_m'],
           'source_reference':cfg['anchor_sources'][0]['source_time'],'reference_utc':referenc
           ↪ e.isoformat(),
           'source_timezone':cfg['source_timezone'],'geodetic_datum':None,
           'timezone_status':policy['hourly']['timezone_status'],
           'hourly_interval_status':policy['hourly']['timestamp_label_semantics']},
          'degree; m; timezone-aware ISO8601',
          ↪ [policy_path,cfg_path,R3/'evidence/waternet_target.html'],
          'q1-sec/comparison-object', scope='Metadata regional-station coordinates; datum and
          ↪ positional error unspecified. UTC+8 is the documented source-clock assumption;
          ↪ not a source-area measurement or event initiation truth.')

    background = meta[(meta.quality_track==QC)&(meta.shift_hours==0)&(meta.window_hours==24)&~
    ↪ meta.is_target]
    check(len(background)==305 and background.process_id.nunique()==45, 'background_305_45')
    check(len(meta)==5526 and (~meta.is_target).sum()==5490 and meta.is_target.sum()==36,
    ↪ 'sample_bank_5526')
    claim('q1-background-and-sample-counts', '五年各61个同钟点日历背景组成每窗305样本、
    ↪ 45个七日过程代理；完整敏感性样本库5526行。',
          {'background_years':cfg['background_years'],'inclusive_day_offsets':[-30,30],
           'daily_anchors_per_year':61,'background_anchors_per_track_shift_window':305,
           'process_days':7,'processes_per_year':9,'processes_per_track_shift_window':45,
           'window_hours':WINDOWS,'quality_tracks':cfg['quality_tracks'],'anchor_shifts_hours'
           ↪ :[-1,0,1],
           'target_reference_count':2,'samples':5526,'background_samples':5490,'target_samples
           ↪ ':36,
           'formula':'2 * (5 * 61 * 3 * 3 + 2 * 3 * 3)'}, 'counts; hours; days',
          [cfg_path,R3/'results/sample_metadata.csv',R3/'results/feature_build_validation.json
          ↪ '],
          'q1-sec/comparison-object')
    folds = fold_metrics.drop_duplicates('fold_id').copy()
    folds['template'] = folds.fold_id.str.split('|',n=2).str[2]
    check(len(folds)==300 and folds.template.nunique()==50 and len(fold_metrics)==900,
          '300_evaluation_folds_reuse_50_group_templates')
    check(all(g.template.nunique()==50 for _,g in
    ↪ folds.groupby(['quality_track','window_hours'])),
          'same_50_templates_in_six_track_window_combinations')
    check(len(memberships)==91500, 'fold_membership_rows')
    fold_counts = {}
    for kind,g in folds.groupby('kind'):
        fold_counts[kind] = {c:{'minimum':int(g[c].min()),'maximum':int(g[c].max())}
                             for c in ['n_train','n_test','n_purged','n_years_train','n_process
                             ↪ _train']}
    check((folds.n_train+folds.n_test+folds.n_purged).eq(305).all(), 'fold_partition_305')
    for fold_id,g in memberships.groupby('fold_id'):
        train=g[g.role=='training'];test=g[g.role=='test_background']
        a=pd.to_datetime(train.support_start_utc,utc=True).astype('int64').to_numpy()
        b=pd.to_datetime(train.support_end_exclusive_utc,utc=True).astype('int64').to_numpy()
        c=pd.to_datetime(test.support_start_utc,utc=True).astype('int64').to_numpy()
        d=pd.to_datetime(test.support_end_exclusive_utc,utc=True).astype('int64').to_numpy()
        check(not np.any((a[:,None]<d[None,:])&(b[:,None]>c[None,:])),
              'actual_168h_fold_supports_disjoint', fold_id)
    claim('q1-fold-counts-and-isolation', '两质量轨、三窗口复用50种分组形成300个评估折，
    ↪ 所有训练与留出168小时支持无交叠。',
          {'group_templates':50,'leave_year_templates':5,'leave_process_templates':45,
           'evaluation_folds':300,'route_fold_records':900,'membership_records':91500,
           'counts_by_fold_kind':fold_counts,'max_support_hours':168,'overlap_violations':0},
          'counts; hours', [R3/'results/fold_metrics.csv',R3/'results/fold_membership.csv',R3/
          ↪ 'scripts/run_models.py'],
```

```
      'q1-sec/method-and-evaluation')

algorithm_parameters = {
    'coverage_primary':cfg['coverage_primary'],'coverage_sensitivity':cfg['coverage_sensit
    ↪ ivity'],
    'distribution_minimum_valid_labels':2,'trend_minimum_valid_labels':6,
    'ordinary_statistics':cfg['ranking_stats'],'precipitation_statistics':cfg['precip_stat
    ↪ s'],
    'trend':'OLS against true label hour index; slope multiplied by24',
    'growth_run':'longest consecutive accepted +1hour positive increments / nominal label
    ↪ count; missing breaks',
    'quality_rule':'37 supplied-QC variables remain QC0 in both tracks; only26 supplied
    ↪ no-QC variables may use numeric-present values in exploration',
    'wind_direction':'sin/cos of nominal10m degrees; median only',
    'precipitation':'label statistics; no rainfall accumulation or energy integration'}
claim('q1-feature-definitions', '特征、缺测链、质量轨及覆盖规则按登记定义实现。
↪ ',algorithm_parameters,
      'mixed; see each parameter',
      ↪ [cfg_path,features_path,R3/'scripts/build_features.py',policy_path],
      'q1-tab-features')
claim('q1-method-A', 'A 使用加权中点 ECDF 双侧偏离，驱动内和族内依次取中位数。',
      {'weights':'1/(Y * P_y * n_yp), recalculated for finite feature support',
       'ecdf':'sum_i w_i * (I[x_ij<x] +0.5 I[x_ij=x])',
       'feature_score':'2*abs(F_j-0.5)','aggregation':'median statistics within driver,
       ↪ then median drivers within family',
       'minimum_actual_years':3,'interpretation':'relative marginal anomaly, not a
       ↪ p-value'},
      'dimensionless', [R3/'scripts/run_models.py',cfg_path], 'q1-eq-ecdf',model=MODELS[0])
claim('q1-method-B', 'B在训练折内拟合稳健尺度与收缩协方差，
↪ 计算按族维数归一的条件块贡献并留年校准。',
      {'minimum_weighted_observation_fraction':.8,'minimum_actual_years':3,'target_observe
      ↪ d':True,
       'imputation':'training weighted median','primary_scale':'1.4826*weighted_MAD','fall
       ↪ back_scale':'weighted_IQR/1.349; zero scale excluded',
       'transform':'clip((x-median)/scale,-8,8) minus training weighted mean',
       'covariance':'C=sum_i w_i z_i z_i^T; Sigma=(1-lambda)C+lambda*I; P=inverse(Sigma)',
       'lambda':.5,'lambda_sensitivity':[.1,.9],
       'conditional_block':'eta=P*delta; b_g=eta_g^T inverse(P_gg) eta_g / d_g',
       'equivalence':'(full squared Mahalanobis distance minus deleted-family marginal
       ↪ distance using same fitted covariance)/d_g',
       'calibration':'upper midpoint ECDF of out-of-year family contributions; not a
       ↪ two-sided transformation',
       'heldout_support':'all active family columns must actually be observed; each fold
       ↪ refits transforms and active columns',
       'family_contributions_additive':False}, 'dimensionless',
      [R3/'scripts/run_models.py',cfg_path], 'q1-eq-conditional',model=MODELS[1])
claim('q1-method-C', 'C排除待评族，用其他族等权距离选取不同过程类比，
↪ 再用双侧分数及两层中位数聚合。',
      {'matching_columns':'complete-training B fit active columns; all tested-family
      ↪ columns excluded',
       'distance_squared':'mean over other families of mean over their active feature
       ↪ squared standardized differences',
       'minimum_matching_observed_fraction':.8,'processes_per_year':2,'maximum_primary_ana
       ↪ logs':10,
       'selection':'nearest anchor per process, then2 nearest distinct processes per year',
       'feature_score':'2*abs(midpoint weighted ECDF in selected analogs -0.5)',
       'aggregation':'median statistics within driver, then median drivers within family',
       'processes_per_year_sensitivity':[1,3]}, 'dimensionless; counts',
      [R3/'scripts/run_models.py',cfg_path], 'q1-sec/route-C',model=MODELS[2])
claim('q1-consensus-definition', '每路线每窗口先归一中间名次，再等权汇总三路线三窗口；
↪ 缺路不计零。',
      {'fractional_rank':'Q=1-(r-1)/(G-1)','primary_finite_family_count':9,
```

```
        'per_route_cross_window_score':'mean over24/72/168h of Q',
        'consensus':'mean over24/72/168h of mean over A/B/C of Q; all3 routes and all3 windows required',
        'tie_definition':'midrank after score rounding to12 decimal places',
        'top5_credit':'clip((5-number_ranked_above_tie)/tie_size,0,1)',
        'duplicate_minute_anchors_extra_votes':False}, 'dimensionless',
       [R3/'scripts/run_models.py',cfg_path], 'q1-eq-consensus',model='consensus')

primary_rec = recommended[recommended.quality_track==QC].sort_values('recommended_rank')
aliases={'土壤水分':'soil_moisture','空气湿状态':'air_moisture','土壤热状态':'soil_thermal',
         '空气热状态':'air_thermal','平均风':'mean_wind','土热通量':'soil_heat_flux',
         '短波辐射':'shortwave','长波辐射':'longwave','气压':'air_pressure'}

def tex_rows(label):
    block=re.search(r'\\label\{'+re.escape(label)+r'\}.*?\\begin\{tabular\}.*?\n(.*?)\\end\{tabular\}',text,re.S)
    check(block is not None,'tex_table_found',label)
    result={}
    for line in block.group(1).splitlines():
        if '&' not in line:continue
        cells=[re.sub(r'\\textbf\{([^{}]*)\}',r'\1',x).strip() for x in line.split('&')]
        cells[-1]=cells[-1].replace('\\\\','').strip()
        result[cells[0]]=cells[1:]
    return result

count_rows=tex_rows('q1-tab-features')
for kind,values in {**table_counts,'合计':totals}.items():
    check([int(x) for x in count_rows[kind][:4]]==values,'tex_feature_table_transcription',kind)
rank_rows=tex_rows('q1-tab-ranks')
for displayed,family in aliases.items():
    row=primary_rec[primary_rec.family_id==family].iloc[0]
    rr=[float(routes[(routes.quality_track==QC)&(routes.model_id==model)&(routes.family_id==family)].iloc[0]['rank']) for model in MODELS]
    paper_values=rank_rows[displayed]
    check(np.array_equal(np.array(paper_values[:3],float),rr),'tex_rank_transcription',family)
    check(paper_values[3]==f'{row.consensus_score:.3f}' and paper_values[4]==f'{row.mean_window_bootstrap_top5_frequency:.3f}',
          'tex_consensus_frequency_three_decimal_transcription',family)
    claim('q1-table-rank-'+family, f'主质量轨{displayed}三路线跨窗名次、共识及条件入选份额。',
          {'family_id':family,'route_ranks':dict(zip(MODELS,rr)),
           'consensus_score':float(row.consensus_score),'consensus_rank':float(row.recommended_rank),
           'mean_window_top5_credit_frequency':float(row.mean_window_bootstrap_top5_frequency),
           'valid_consensus_draws_per_window':44,'requested_draws_per_window':100,
           'paper_rounded_consensus':paper_values[3],'paper_rounded_frequency':paper_values[4]},
          'rank; dimensionless', [R3/'results/top5_three_routes.csv',R3/'results/recommended_factors.csv',R3/'results/ranking_stability.csv'],
          'q1-tab-ranks',model='consensus')
wind=primary_rec[primary_rec.family_id=='mean_wind'].iloc[0]
ground=primary_rec[primary_rec.family_id=='soil_heat_flux'].iloc[0]
claim('q1-fifth-slot-competition', '风场名义共识第五，土热通量的条件入选份额较高，两种排序摘要刻画不同性质。',
      {'wind_consensus':float(wind.consensus_score),'ground_heat_consensus':float(ground.consensus_score),
       'wind_frequency':float(wind.mean_window_bootstrap_top5_frequency),
       'ground_heat_frequency':float(ground.mean_window_bootstrap_top5_frequency),
       'frequency_difference_percentage_points':float((ground.mean_window_bootstrap_top5_frequency-wind.mean_window_bootstrap_top5_frequency)*100)},
      'dimensionless; percentage points', [R3/'results/recommended_factors.csv'],
```

```
311           'q1-sec/stability-and-fifth-slot',model='consensus')
312
313     boot=stability[(stability.quality_track==QC)&(stability.kind=='hierarchical_bootstrap')]
314     denominators={model:sorted(boot[boot.model_id==model].n_valid.unique().astype(int).tolist(
        ↪  )) for model in MODELS+['consensus']}
315     check(denominators=={MODELS[0]:[89],MODELS[1]:[44],MODELS[2]:[89],'consensus':[44]},'boots
        ↪  trap_89_44_denominators')
316     year_hist=Counter(len(set(d['sampled_years'])) for d in draws)
317     check(year_hist=={2:11,3:45,4:40,5:4},'bootstrap_distinct_year_histogram')
318     claim('q1-bootstrap-denominators', '100次原始层级抽样全部保留；主QC的A/C每窗89次有效，
        ↪  B及共识每窗44次。',
319           {'requested_draws':100,'valid_draws_per_window':denominators,
320            'unique_year_count_histogram':dict(sorted(year_hist.items())),
321            'fewer_than_three_distinct_years':11,'fewer_than_four_distinct_years':56,
322            'frequency_definition':'within each window mean fractional TOP5 credit over
               ↪  finite-rank draws; then equally average3 windows',
323            'independent_events_added':0,'minimum_valid_draws_for_stable_label':70,
324            'all_primary_labels':sorted(primary_rec.stability_label.unique().tolist())},
325           'counts; conditional frequency', [draws_path,R3/'results/ranking_stability.csv',R3/'
              ↪  results/recommended_factors.csv',R3/'scripts/run_models.py'],
326           'q1-fig-stability',model='consensus')
327
328     vwc=state[(state.driver_id=='soil_vwc_0p1m')&(state.quality_track==QC)&(state.window_hours
        ↪  ==168)].iloc[0]
329     claim('q1-soil-water-state-and-direction',
        ↪  '0.1m体积含水率的较高季节背景位置与168小时内下降同时成立。',
330           {key:(int(vwc[key]) if key in ['window_hours','n_valid','n_labels'] else
              ↪  str(vwc[key]) if 'time' in key else float(vwc[key]))
331            for key in ['window_hours','n_valid','n_labels','first_valid_time_utc','last_valid_
               ↪  time_utc','first_valid_value','last_valid_value','endpoint_change','target_medi
               ↪  an','background_median_percentile','target_trend_per24h']},
332           'm3 m-3; OLS slope in m3 m-3 per24h; UTC',
333           [R3/'results/case_state_evidence.csv',R3/'results/factor_directions.csv'],
334           'q1-fig-state',model=MODELS[0],
335           scope='Accepted original-QC regional0.1m soil values,168 hourly labels spanning167h
              ↪  between endpoints. A negative window OLS trend is not proof of monotonic change,
              ↪  source-area pressure or causation.')
336     representative=figure_values['state']['representative_driver_ids']
337     main_directions=directions[(directions.quality_track==QC)&directions.statistic.eq('median'
        ↪  )]
338     for driver in representative:
339         rows=main_directions[main_directions.driver_id==driver].sort_values('window_hours')
340         check(rows.window_hours.tolist()==WINDOWS,'representative_windows',driver)
341         claim('q1-median-direction-'+driver, f'{driver}的固定代表中位数、季节背景中位秩和方向。',
342               records(rows[['window_hours','target_value','background_percentile','signed_dire
                  ↪  ction']]),
343               {'value':driver_by_id[driver]['unit'],'background_percentile':'fraction from0
                  ↪  to1'},
344               [R3/'results/factor_directions.csv',features_path],
                  ↪  'q1-fig-state',model=MODELS[0])
345
346     hourly_path=R3/'exports/hourly_drivers.parquet'
347     source(hourly_path)
348     hourly=pd.read_parquet(hourly_path)
349     clock=pd.to_datetime(hourly.time_utc,utc=True)
350     lower=pd.to_datetime(reference.value-168*3_600_000_000_000,utc=True)
351     sub=hourly[(hourly.quality_track==QC)&(clock>=lower)&(clock<reference)].copy()
352     times=pd.to_datetime(sub.time_utc,utc=True)
353     series=sub.soil_vwc_0p1m.to_numpy(float)
354     percentile_matrix=main_directions.pivot(index='driver_id',columns='window_hours',values='b
        ↪  ackground_percentile').loc[representative,WINDOWS].to_numpy()
355     check(np.isfinite(series).sum()==figure_values['state']['n_vwc']==168,'state_figure168_val
        ↪  ues')
```

```
check(series[0]==figure_values['state']['vwc_first'] and series[-1]==figure_values['state']['vwc_last'],'state_figure_endpoints')
check(np.allclose(percentile_matrix,figure_values['state']['median_percentiles'],rtol=0,atol=1e-12),'state_figure_complete_percentile_matrix')
series_records=[{'time_utc':t.isoformat(),'hours_before_reference':float((t-reference).total_seconds()/3600),
                 'soil_vwc_0p1m':None if not np.isfinite(y) else float(y)} for t,y in zip(times,series)]
claim('q1-figure-state-complete-data', '状态图完整168点时序及7代表量×3窗口背景位置，直接重绘冻结结果。',
      {'state_figure_values':figure_values['state'],'hourly_series':series_records,
       'aggregation':'time series is every accepted original-QC0.1m soil label in [reference-168h,reference); heatmap is each preselected driver median midpoint background ECDF, not family scores or an average over drivers'},
      'UTC; hours; m3 m-3; fraction',
      [hourly_path,R3/'results/factor_directions.csv',figures_path,PAPER/'scripts/q1_figures.py',PAPER/'figures/q1/q1_state.pdf'],
      'q1-fig-state',model=MODELS[0])

sf=figure_values['stability'];ids=sf['family_ids']
frequency=boot.groupby(['family_id','model_id']).top5_frequency.mean().unstack().loc[ids,MODELS].to_numpy()
check(np.allclose(frequency,sf['frequency_mean_over_three_windows'],rtol=0,atol=1e-12),'stability_figure_complete_frequency_matrix')
sm=sensitivity[sensitivity.quality_track==QC]
categories=[('参考时刻 ±1 h',(sm.variant=='primary')&(sm.shift_hours!=0)),
    ('过程块 3 / 14 d',sm.variant.isin(['process_3','process_14'])),
    ('季节范围 ±15 d',sm.variant=='season_15'),
    ('支持率 50% / 90%',sm.variant.isin(['support_0.5','support_0.9'])),
    ('收缩系数 0.1 / 0.9',sm.variant.isin(['shrinkage_0.1','shrinkage_0.9'])),
    ('每年类比数 1 / 3',sm.variant.isin(['analogs_1','analogs_3']))]
jaccard=np.array([sm[mask].groupby('model_id').top5_jaccard.mean().reindex(MODELS).to_numpy() for _,mask in categories])
check([name for name,_ in categories]==sf['sensitivity_categories'],'sensitivity_category_order')
check(np.allclose(jaccard,sf['jaccard_mean_over_three_windows_and_listed_variants'],rtol=0,atol=1e-12),'stability_figure_complete_sensitivity_matrix')
category_support=[{'category':label,'records_per_route':{m:int((sm.loc[mask,'model_id']==m).sum()) for m in MODELS},
                   'variants':sorted(sm.loc[mask,'variant'].unique().tolist()),
                   'shift_hours':sorted(sm.loc[mask,'shift_hours'].unique().astype(int).tolist())} for label,mask in categories]
claim('q1-figure-stability-complete-data', '稳定性图完整9族×3路线频率及6类×3路线Jaccard均值，使用各自有效分母。',
      {'figure_values':sf,'sensitivity_record_support':category_support,
       'left_aggregation':'per family/route equal mean of3 window-specific TOP5 fractional-credit frequencies; A/C denominator89 and B44 per window; not consensus frequency',
       'right_aggregation':'per category/route arithmetic mean TOP5 set Jaccard over all3 windows and all listed variants or hour shifts; deterministic sensitivity, not event prediction accuracy',
       'fit_performed':False}, 'fraction from0 to1; counts',
      [R3/'results/ranking_stability.csv',R3/'results/sensitivity_metrics.csv',R3/'results/recommended_factors.csv',
       figures_path,PAPER/'scripts/q1_figures.py',PAPER/'figures/q1/q1_stability.pdf'],
      'q1-fig-stability',model=MODELS)

unknown_rec=recommended[recommended.quality_track==UNKNOWN].sort_values('recommended_rank')
main_top=primary_rec.loc[primary_rec.recommended_top5,'family_id'].tolist()
unknown_top=unknown_rec.loc[unknown_rec.recommended_top5,'family_id'].tolist()
check(set(main_top)==set(unknown_top) and main_top[:2]==['soil_moisture','air_moisture'] and unknown_top[:2]==['air_moisture','soil_moisture'],
```

```
      'unknown_same_top5_first_two_exchange')
unknown_turb=unknown_rec[unknown_rec.family_id.isin(['turbulent_heat','turbulent_mechanics
↪ '])]
check(unknown_turb.consensus_score.isna().all() and
↪ unknown_turb.recommended_rank.isna().all(), 'unknown_primary_consensus_null')
raw_rank=csv('rankings.csv')
missing_B=raw_rank[(raw_rank.quality_track==UNKNOWN)&(raw_rank.variant=='primary')&
    (raw_rank.anchor_id=='problem_1030')&(raw_rank.shift_hours==0)&
    (raw_rank.model_id==MODELS[1])&raw_rank.family_id.isin(['turbulent_heat','turbulent_me
    ↪ chanics'])]
check(len(missing_B)==6 and missing_B.score.isna().all(),
↪ 'unknown_B_all_three_windows_two_turbulent_families_null')
claim('q1-unknown-track-comparison', '探索轨前五集合相同、前两项互换;
↪ 湍流热和湍流机械的主设置B分数及共识保持空值。',
      {'main_top5':main_top,'unknown_top5':unknown_top,
       'unknown_consensus_rows':records(unknown_rec[['family_id','consensus_score','recomm
       ↪ ended_rank','mean_window_bootstrap_top5_frequency','min_window_bootstrap_valid_
       ↪ draws']]),
       'unknown_B_missing_scores':records(missing_B[['window_hours','family_id','score','r
       ↪ ank']]),
       'actual_primary_family_support':records(effective[effective.quality_track==UNKNOWN]
       ↪ ),
       'bootstrap_denominator_turbulent_A_C':61,'bootstrap_denominator_turbulent_B_consens
       ↪ us':10,
       'unknown_B_valid_draw_rule':'four non2022 years all present and2022 absent;
       ↪ all-five-year draws fail80% LOYO support'},
      'rank; dimensionless; actual-year counts',
      [R3/'results/recommended_factors.csv',R3/'results/rankings.csv',R3/'results/effectiv
      ↪ e_family_support.csv',R3/'results/ranking_stability.csv',draws_path],
      'q1-sec/stability-and-fifth-slot',model='consensus')

code_files=['track_b/research/q1/round3/scripts/build_features.py',
            'track_b/research/q1/round3/scripts/run_models.py',
            'track_b/research/q1/round3/scripts/summarize_results.py',
            'track_b/paper/scripts/q1_figures.py','track_b/paper/scripts/q1_evidence.py']
for path in code_files:
    source(path)
check(len({c['claim_id'] for c in claims})==len(claims), 'unique_claim_ids')
for item in claims:
    check(all(k in item for k in ['claim_id','statement','value','unit','source_files','mo
    ↪ del_id','scenario_id','scope','paper_anchor']),
          'mandatory_claim_fields',item['claim_id'])
for path,entry in source_snapshots.items():
    check(sha(ROOT/path)==entry['sha256'], 'source_unchanged_during_registration',path)
dump(PAPER/'evidence/q1_claims.json',claims)
dump(PAPER/'evidence/q1_code_files.json',code_files)
review={'review_version':'q1-paper-numeric-v1','passed':True,
    'execution':'Actually read frozen data, reconstructed displayed summaries and
    ↪ transcribed TeX tables; no research model fit, plot render, network call or
    ↪ other-question read.',
    'script':'track_b/paper/scripts/q1_evidence.py','script_sha256':sha(Path(__file__)),
    'paper_section_sha256':sha(text_path),'figure_values_sha256':sha(figures_path),
    'manifest_snapshots':manifests,'source_snapshots':source_snapshots,
    'checks':dict(checks),'check_count':sum(checks.values()),'claims_count':len(claims),
    'table_transcription':{'q1-tab-features':'All4 category rows and total counts match
    ↪ frozen dictionary/schema',
                           'q1-tab-ranks':'All9 family rows,27 route ranks and18 rounded
                           ↪ consensus/frequency values match frozen CSVs'},
    'figure_reconstruction':{'q1-fig-state':'168 complete hourly points, exact endpoints
    ↪ and21 background percentile cells',
                             'q1-fig-stability':'27 route-specific frequency cells and18
                             ↪ Jaccard cells; all aggregation record counts retained'},
```

```
        'fold_support_checks':{'distinct_evaluation_folds':300,'group_templates':50,
                               'membership_records':91500,'overlap_violations':0},
        'output_files':[{'path':str(p.relative_to(ROOT)),'bytes':p.stat().st_size,'sha256':sha
        ↪  (p)}
                        for p in [PAPER/'evidence/q1_claims.json',PAPER/'evidence/q1_code_file
                        ↪  s.json']],
        'boundaries':['ECDF positions, normalized ranks and conditional bootstrap frequencies
        ↪  are different quantities; none is a hazard probability or p-value.',
            'A/C use driver-then-family medians; B uses a conditional block distance and upper
            ↪  out-of-year ECDF, not the A/C two-sided transformation.',
            '300 evaluation folds reuse50 grouping templates; overlapping backgrounds and
            ↪  repeated windows are not independent disasters.',
            '44 valid consensus draws per window are conditional on supported sampled years;
            ↪  window means do not create132 independent draws.',
            'The0.1m soil-water median may be high in background rank while its168h temporal
            ↪  slope is negative.',
            'Regional station coordinates/UTC+8 are metadata assertions; original datum, source
            ↪  representativeness and true operational availability remain unspecified.',
            'Unknown-QC increments remain separate; empty B/calibration/consensus scores are
            ↪  not zeros.',
            'Figure verification covers source numbers/aggregation and artifact hashes; visual
            ↪  layout/LaTeX compilation belongs to the separate QA workflow.',
            'QA wrapper, shared preamble, references and volatile preview metadata were not
            ↪  bound as model evidence.']}
    dump(PAPER/'evidence/q1_numeric_review.json',review)
    print(json.dumps({'passed':True,'claims':len(claims),'checks':sum(checks.values()),
        'section_sha256':sha(text_path),'numeric_review_sha256':sha(PAPER/'evidence/q1_numeric
        ↪  _review.json')},ensure_ascii=False))


if __name__=='__main__':
    main()
```

程序 6：共同力学、损伤及储排水推进

来源：track_b/research/q2/round3/scripts/mechanism_engine.py
SHA256: bb6636d2d3f463c894cdc6bc1df9c7fd0f93f9b7a6f9aa6e8233113630afe14e

```
"""Q2R3 plane-slab conditional mechanism engine; SI except kPa and hours.

No event-time fitting; arrays are computational scenarios, not observations.
Time[0] is the specified initial state. Forcing[i] drives [time[i],time[i+1]).
"""
from __future__ import annotations

import numpy as np

MODEL_IDS = {"S": "q2r3_S_v1", "S_T": "q2r3_ST_v1", "S_H": "q2r3_SH_v1", "S_T_H": "q2r3_STH_v1"}
MARGIN_DEFINITION = "(resisting_shear_kPa-driving_shear_kPa)/driving_shear_kPa; positive stable"
G = 9.80665


def stresses(parameters):
    """h is thickness normal to plane, not vertical thickness."""
    theta = np.deg2rad(np.asarray(parameters["theta_deg"], float))
    weight = np.asarray(parameters["density_kg_m3"]) * G * np.asarray(parameters["thickness_normal_m"]) / 1000
    return weight * np.cos(theta), weight * np.sin(theta)


def joint_strength(temperature_c, policy="linear_extension"):
    """Mamot2018 Eq4/5: ice-filled limestone joint, not pure ice.

    Source domain -8<=T<=-.5 degC; effective stress/domain checked by caller.
    linear_extension explicitly retains and flags extrapolation.
    thawed_rock_contact is an uncalibrated challenger above -.5 C.
    """
    t = np.asarray(temperature_c, float)
    c, mu = 53.3 - 73.5 * t, 0.42 - 0.21 * t
    if policy == "thawed_rock_contact":
        c, mu = np.where(t > -0.5, 0., c), np.where(t > -0.5, 0.60, mu)
    elif policy != "linear_extension":
        raise ValueError(policy)
    return c, mu


def resistance(parameters, damage, interface_c, pressure_kpa, policy="linear_extension"):
    sn, drive = stresses(parameters)
    c_joint, mu_joint = joint_strength(interface_c, policy)
    bridge = np.asarray(parameters["bridge_fraction"])
    cohesion = (1-np.asarray(damage)) * (bridge*np.asarray(parameters["bridge_cohesion_kpa"]) + (1-bridge)*c_joint)
    friction = bridge*np.asarray(parameters["rock_mu"]) + (1-bridge)*mu_joint
    eff = sn-np.asarray(pressure_kpa)
    resist = cohesion + np.maximum(eff, 0)*friction
    margin = (resist-drive)/drive
    domain = (np.asarray(interface_c)>=-8) & (np.asarray(interface_c)<=-.5) & (eff>=100) & (eff<=400)
    algebraic_head = (sn-(drive-cohesion)/friction)/(G)  # rho_water=1000: kPa per metre = g
    # With nonnegative effective stress, cohesion > driving cannot be
    # destabilized by pressure alone. Do not export an unreachable root.
    critical_head = np.where(cohesion <= drive, algebraic_head, np.nan)
    root_status = np.where(cohesion > drive, 'no_pressure_only_root',
```

```
                          np.where(cohesion == drive, 'contact_limit_critical',
                                   np.where(algebraic_head <= 0, 'dry_already_critical',
                                   ↪  'positive_pressure_root')))
    return dict(resistance_kpa=resist, driving_kpa=drive, normal_kpa=sn,
                effective_kpa=eff, failure_margin=margin, strength_domain=domain,
                contact_valid=eff>0, critical_head_m=critical_head,
                cohesion_kpa=cohesion, mu=friction,critical_head_status=root_status)


def thermal_transfer(air_c, parameters, dt_hours=1., dz_m=.5, bottom_m=40.):
    """Explicit 1D conduction, surface Dirichlet, bottom no flux.

    Surface min(air + scenario offset, 0) assumes an ice-capped boundary;
    this is not an energy balance or measured source temperature.
    """
    air = np.asarray(air_c, float)
    if not np.all(np.isfinite(air)):
        raise ValueError("missing temperature: cannot silently fill")
    n = len(parameters["initial_temp_c"])
    initial = np.asarray(parameters["initial_temp_c"], float)
    alpha = np.asarray(parameters["diffusivity_m2_s"], float)
    depth = np.asarray(parameters["interface_depth_m"], float)
    z = np.arange(0, bottom_m+dz_m*.5, dz_m)
    if depth.max() >= bottom_m or depth.min() < 0:
        raise ValueError("interface outside thermal domain")
    ratio = alpha*dt_hours*3600/dz_m**2
    substeps = max(1, int(np.ceil(ratio.max()/.45)))
    rr = ratio[:, None]/substeps
    prof = np.repeat(initial[:, None], len(z), axis=1)
    ti = np.empty((len(air), n), float)
    surf = np.minimum(air[:, None]+np.asarray(parameters["station_source_offset_c"])[None, :],
    ↪  0)
    k = np.floor(depth/dz_m).astype(int); frac = depth/dz_m-k; row=np.arange(n)
    ti[0] = initial
    for i in range(len(air)-1):
        for _ in range(substeps):
            old=prof.copy(); old[:,0]=surf[i]
            prof[:,1:-1]=old[:,1:-1]+rr*(old[:,2:]-2*old[:,1:-1]+old[:,:-2])
            prof[:,-1]=old[:,-1]+2*rr[:,0]*(old[:,-2]-old[:,-1])
            prof[:,0]=surf[i]
        ti[i+1]=prof[row,k]*(1-frac)+prof[row,k+1]*frac
    return ti, surf


def reservoir_step(storage_m, input_m_h, tau_h, capacity_m, dt_h):
    """Exact piecewise constant-input reservoir with continuous overflow.

    Integrate until the capacity is hit, then keep capacity storage while
    I-C/tau overflows. This avoids timestep-dependent drain/overflow splits.
    """
    w,inc,tau,cap=np.broadcast_arrays(np.asarray(storage_m,float),np.asarray(input_m_h,float),
    ↪  np.asarray(tau_h,float),np.asarray(capacity_m,float))
    if np.any(tau<=0) or np.any(cap<0) or dt_h<=0 or np.any(inc<0): raise ValueError('invalid
    ↪  reservoir parameters')
    immediate=np.maximum(w-cap,0);w=np.minimum(w,cap)
    equilibrium=inc*tau
    decay=np.exp(-dt_h/tau)
    provisional=w*decay+equilibrium*(1-decay)
    hits=(equilibrium>cap)&(provisional>=cap)
    fraction=np.divide(equilibrium-cap,equilibrium-w,out=np.ones_like(w),where=(equilibrium-w)
    ↪  >0)
    hit_time=-tau*np.log(np.clip(fraction,1e-300,1.))
    active_time=np.where(hits,dt_h-np.clip(hit_time,0,dt_h),0.)
```

```
    storage=np.where(hits,cap,provisional)
    overflow=immediate+np.maximum(inc-cap/tau,0)*active_time
    drainage=np.asarray(storage_m)+inc*dt_h-storage-overflow
    return storage,drainage,overflow


def simulate(air_c, precip_mm_h, parameters, mechanism="S_T_H", policy="linear_extension",
             dt_hours=1., dz_m=.5, thermal=None, drainage_feedback=True, damage_enabled=True):
    """Run all parameter rows together; never removes non-failing scenarios.

    Continuing after first criterion crossing is a counterfactual diagnostic,
    not motion/propagation. Independent-event counts are not defined here.
    """
    if mechanism not in MODEL_IDS: raise ValueError(mechanism)
    air, rain = np.asarray(air_c,float), np.asarray(precip_mm_h,float)
    if len(air)!=len(rain) or not np.all(np.isfinite(rain)) or np.any(rain<0):
        raise ValueError("rain inputs must be finite nonnegative conditional rates")
    tstate, surf = thermal if thermal is not None else
    ↪  thermal_transfer(air,parameters,dt_hours,dz_m)
    nt,n=tstate.shape
    has_t=mechanism in ("S_T","S_T_H"); has_h=mechanism in ("S_H","S_T_H")
    ti=tstate if has_t else
    ↪  np.broadcast_to(np.asarray(parameters['initial_temp_c']),tstate.shape)
    damage=np.empty((nt,n)); storage=np.zeros((nt,n)); pressure=np.zeros((nt,n))
    margin=np.empty((nt,n)); domain=np.empty((nt,n),bool); critical=np.empty((nt,n))
    damage[0]=parameters['initial_damage']
    sn,drive=stresses(parameters)
    capacity_head=np.minimum(np.asarray(parameters['connected_depth_m']),.98*sn/G)
    capacity=capacity_head*np.asarray(parameters['storativity'])
    cumulative_input=np.zeros(n); cumulative_drain=np.zeros(n); cumulative_overflow=np.zeros(n)
    budget_error=np.zeros(n)
    for i in range(nt):
        pressure[i]=G*storage[i]/np.asarray(parameters['storativity']) if has_h else 0
        state=resistance(parameters,damage[i],ti[i],pressure[i],policy)
        margin[i]=state['failure_margin'];domain[i]=state['strength_domain'];critical[i]=state⌋
        ↪  ['critical_head_m']
        if i==nt-1: break
        # Phenomenological stress-ratio damage; no claimed creep-material calibration.
        ratio=drive/np.maximum(state['resistance_kpa'],1e-6)
        rate=np.asarray(parameters['damage_rate_h_inv'])*np.minimum(ratio,10.)**np.asarray(par⌋
        ↪  ameters['damage_exponent']) if damage_enabled else np.zeros(n)
        damage[i+1]=1-(1-damage[i])*np.exp(-rate*dt_hours)
        damage[i+1]=np.minimum(damage[i+1],.999999)
        if has_h:
            source_air=air[i]+np.asarray(parameters['station_source_offset_c'])
            rain_fraction=np.clip((source_air+1)/2,0,1)
            incoming=np.asarray(parameters['rain_multiplier'])*rain[i]/1000*rain_fraction +
            ↪  np.asarray(parameters['melt_mm_deg_day'])*np.maximum(source_air,0)/24000
            # Damage may increase connectivity/drainage; this coupling is a scenario.
            tau=np.asarray(parameters['drainage_hours'])*np.exp(-np.asarray(parameters['draina⌋
            ↪  ge_damage_coupling'])*damage[i]) if drainage_feedback else
            ↪  np.asarray(parameters['drainage_hours'])
            storage[i+1],out,over=reservoir_step(storage[i],incoming,tau,capacity,dt_hours)
            cumulative_input+=incoming*dt_hours;cumulative_drain+=out;cumulative_overflow+=over
            budget_error=np.maximum(budget_error,np.abs(storage[i+1]-cumulative_input+cumulati⌋
            ↪  ve_drain+cumulative_overflow))
    return dict(interface_temp_c=ti,damage=damage,storage_m=storage,pressure_kpa=pressure,
                failure_margin=margin,strength_domain=domain,critical_head_m=critical,
                water_budget_max_abs_m=budget_error,water_input_m=cumulative_input,
                water_drain_m=cumulative_drain,water_overflow_m=cumulative_overflow)


def classify_trajectory(margins, dt_hours=1.):
```

```
    """Distinct missing/left-censored/interval-crossing/never-crossed states.

    A missing value before any crossing prevents a no-start claim. A first
    sample <=0 is already unstable, not an observed onset at time zero.
    """
    x=np.asarray(margins,float)
    if x.ndim!=1 or len(x)==0: raise ValueError('one nonempty trajectory required')
    finite=np.isfinite(x)
    if not finite[0]: return dict(onset_status='missing',onset_elapsed_hours=None,crossing_low
    ↪  er_hours=None,crossing_upper_hours=None)
    if x[0]<=0: return dict(onset_status='left_censored_already_unstable',onset_elapsed_hours=
    ↪  None,crossing_lower_hours=None,crossing_upper_hours=0.)
    below=np.flatnonzero(finite&(x<=0))
    if len(below):
        k=int(below[0])
        if not finite[:k+1].all(): return
        ↪  dict(onset_status='missing_before_possible_crossing',onset_elapsed_hours=None,cros
        ↪  sing_lower_hours=None,crossing_upper_hours=k*dt_hours)
        f=x[k-1]/(x[k-1]-x[k])
        return dict(onset_status='interval_crossing',onset_elapsed_hours=(k-1+f)*dt_hours,cros
        ↪  sing_lower_hours=(k-1)*dt_hours,crossing_upper_hours=k*dt_hours)
    return dict(onset_status='never_crossed' if finite.all() else
    ↪  'missing',onset_elapsed_hours=None,crossing_lower_hours=None,crossing_upper_hours=None)
```

程序 7：因果 erfc 热核与解析热接口

来源：track_b/research/q2/round3/scripts/analytic_thermal_engine.py
SHA256: a113d3f47e97266761fa47b7d6ed7e5a9e830edeeaf2ba3918a70db3c0003607

```
"""Analytical 1D conductive challenger for hourly step boundary conditions.

The mechanical/water/damage equations are the frozen Q2R3 engine.  This module
only supplies the thermal tuple to its public simulate(thermal=...) argument.
No observations, event fitting or probability weights are introduced.
"""
from __future__ import annotations

import math

import numpy as np

from mechanism_engine import simulate


def erfc_array(values):
    """Use the standard-library scalar erfc, avoiding an additional dependency."""
    values = np.asarray(values, float)
    return np.fromiter((math.erfc(float(value)) for value in values.flat),
                       dtype=float, count=values.size).reshape(values.shape)


def step_kernel(nt, depth_m, alpha_m2_s, dt_hours=1.):
    """Semi-infinite Dirichlet step response; K[0]=0 by state0 convention."""
    depth, alpha = np.broadcast_arrays(np.atleast_1d(np.asarray(depth_m, float)),
                                       np.atleast_1d(np.asarray(alpha_m2_s, float)))
    if nt < 1 or dt_hours <= 0 or np.any(depth < 0) or np.any(alpha <= 0):
        raise ValueError("invalid time/depth/diffusivity")
    if not np.all(np.isfinite(depth)) or not np.all(np.isfinite(alpha)):
        raise ValueError("nonfinite thermal parameters")
    kernel = np.zeros((nt, depth.size))
    if nt > 1:
        denominator = 2*np.sqrt(np.arange(1, nt)[:, None] * dt_hours * 3600 * alpha[None, :])
        kernel[1:] = erfc_array(depth[None, :]/denominator)
    return kernel


def finite_step_kernel(nt, depth_m, alpha_m2_s, bottom_m, dt_hours=1., terms=12):
    """Finite0..L Dirichlet/no-flux step response, using a mirror expansion.

    cosh((L-z)q)/cosh(Lq) = sum_n(-1)^n *
        [exp(-(2nL+z)q) + exp(-(2(n+1)L-z)q)], q=sqrt(s/alpha).
    Divide by s for a unit step and invert each exponential to erfc.  The
    complementary paired form below avoids subtracting the main kernel when
    reporting the extremely small production-domain reflection correction.
    """
    depth, alpha = np.broadcast_arrays(np.atleast_1d(np.asarray(depth_m, float)),
                                       np.atleast_1d(np.asarray(alpha_m2_s, float)))
    if bottom_m <= 0 or np.any(depth > bottom_m) or terms < 1:
        raise ValueError("invalid finite thermal domain")
    kernel = step_kernel(nt, depth, alpha, dt_hours)
    correction = np.zeros_like(kernel)
    if nt > 1:
        denominator = 2*np.sqrt(np.arange(1, nt)[:, None] * dt_hours * 3600 * alpha[None, :])
        for n in range(1, terms+1):
            pair = (erfc_array((2*n*bottom_m-depth)[None, :]/denominator)
                    - erfc_array((2*n*bottom_m+depth)[None, :]/denominator))
```

```
            correction[1:] += (1 if n % 2 else -1) * pair
    return kernel+correction, correction


def linear_causal_convolution(increments, kernel):
    """Zero-padded FFT linear convolution, truncated to physical state times.

    nfft >= 2*nt-1 prevents circular wrap-around from future increments.  K0=0
    ensures forcing[i] first influences state[i+1], not state[i].
    """
    increments, kernel = np.asarray(increments, float), np.asarray(kernel, float)
    if increments.shape != kernel.shape or increments.ndim != 2:
        raise ValueError("thermal increments/kernel must have equal(nt,n) shape")
    nt = len(increments)
    nfft = 1 << (2*nt-2).bit_length()
    transformed = np.fft.rfft(increments, n=nfft, axis=0) * np.fft.rfft(kernel, n=nfft, axis=0)
    return np.fft.irfft(transformed, n=nfft, axis=0)[:nt], nfft


def thermal_transfer_analytic(air_c, parameters, dt_hours=1.):
    """Return (interface temperature, imposed surface temperature) at states.

    Boundary B[i]=min(air[i]+offset,0) drives [i,i+1).  Initial interior state
    is uniform Tinit.  dB0=B0-Tinit and dBj=Bj-Bj-1 for j>=1.  Neither the last
    boundary value nor any future perturbation can drive an earlier state.
    Production depths are strictly positive (0.5..20m); if z=0 is requested,
    state[k] uses the left boundary limit B[k-1], with state[0]=Tinit.
    """
    air = np.asarray(air_c, float)
    if air.ndim != 1 or len(air) < 1 or not np.isfinite(air).all():
        raise ValueError("finite nonempty one-dimensional air boundary required")
    initial = np.atleast_1d(np.asarray(parameters["initial_temp_c"], float))
    offset = np.atleast_1d(np.asarray(parameters["station_source_offset_c"], float))
    depth = np.atleast_1d(np.asarray(parameters["interface_depth_m"], float))
    alpha = np.atleast_1d(np.asarray(parameters["diffusivity_m2_s"], float))
    if not all(value.shape == initial.shape for value in [offset, depth, alpha]):
        raise ValueError("thermal parameter arrays must have one shared row shape")
    if not np.isfinite(initial).all() or not np.isfinite(offset).all():
        raise ValueError("nonfinite initial temperature or source offset")
    boundary = np.minimum(air[:, None]+offset[None, :], 0.)
    increments = np.empty_like(boundary)
    increments[0] = boundary[0]-initial
    increments[1:] = np.diff(boundary, axis=0)
    kernel = step_kernel(len(air), depth, alpha, dt_hours)
    response, _ = linear_causal_convolution(increments, kernel)
    temperature = initial[None, :]+response
    # At t0 no interval has elapsed. Restore the exact declared initial state;
    # all other pre-perturbation checks retain actual FFT round-off.
    temperature[0] = initial
    return temperature, boundary


def simulate_analytic(air_c, precip_mm_h, parameters, mechanism="S_T_H",
                      policy="linear_extension", dt_hours=1.,
                      drainage_feedback=True, damage_enabled=True):
    """Public Q3 wrapper: same frozen mechanical engine and criterion/censoring."""
    thermal = thermal_transfer_analytic(air_c, parameters, dt_hours)
    return simulate(air_c, precip_mm_h, parameters, mechanism=mechanism, policy=policy,
                    dt_hours=dt_hours, thermal=thermal,
                    drainage_feedback=drainage_feedback, damage_enabled=damage_enabled)


def reflection_temperature_bound(air_c, parameters, bottom_m=40., dt_hours=1.):
```

```
    """Conservative finite-Neumann versus half-space error, per parameter row.

    Mirror correction is an alternating series of positive decreasing erfc
    differences, hence 0 <= Kfinite-Ksemi <= erfc((2L-z)/(2sqrt(alpha*t))).
    Bound the convolution by this horizon maximum times sum(abs(dB)).  This
    remains a temperature-discretization bound, not a physical/model error.
    """
    air = np.asarray(air_c, float)
    initial = np.atleast_1d(np.asarray(parameters["initial_temp_c"], float))
    offset = np.atleast_1d(np.asarray(parameters["station_source_offset_c"], float))
    depth = np.atleast_1d(np.asarray(parameters["interface_depth_m"], float))
    alpha = np.atleast_1d(np.asarray(parameters["diffusivity_m2_s"], float))
    if bottom_m <= 0 or np.any(depth < 0) or np.any(depth > bottom_m) or dt_hours <= 0:
        raise ValueError("depth outside finite reflection-bound domain")
    boundary = np.minimum(air[:, None]+offset[None, :], 0.)
    increments = np.empty_like(boundary)
    increments[0] = boundary[0]-initial
    increments[1:] = np.diff(boundary, axis=0)
    # Last boundary is unused at the last output state (K0=0).
    total_variation = np.abs(increments[:-1]).sum(axis=0)
    horizon = (len(air)-1)*dt_hours*3600
    first_reflection = erfc_array((2*bottom_m-depth)/(2*np.sqrt(alpha*horizon))) if horizon > 0
    ↪  else np.zeros_like(depth)
    return total_variation*first_reflection, total_variation, first_reflection
```

程序 8：共同 19 维参数设计与情景记录

来源：track_b/research/q2/round3/scripts/run_experiments.py
SHA256: 376fae8826c2be1abb87004b3340c5dd9a420d14087cf21b38d7d943a86ff0b1

```
"""Offline deterministic full Q2R3 conditional competition and early exports."""
from __future__ import annotations
import hashlib, json, platform, sys, time
from pathlib import Path
import numpy as np
import pandas as pd
from mechanism_engine import MODEL_IDS, MARGIN_DEFINITION, G, simulate, thermal_transfer, stresses, resistance, classify_trajectory

ROOT=Path(__file__).resolve().parents[5]
BASE=ROOT/'track_b/research/q2/round3'
CONFIG=BASE/'config/experiment_v1.json'
INPUT_VERSION={'baseline_commit':'904d7257b899b9f71d1ac9e0eab7479e459eab52','baseline_manifest_sha256':'fd755fab38df285a67c986026859be5f635a314aa3f79f2d247a2626246c3a53'}

RANGES={
 'theta_deg':(30,60,'linear','conditional structural plane; not NC surface slope'),
 'thickness_normal_m':(10,60,'linear','conditional; NC inventory 6..154 m is background only'),
 'rock_volume_fraction':(.2,.8,'linear','conditional two-solid mixture; not measured'),
 'bridge_fraction':(.0,.25,'linear','conditional load-bearing area fraction'),
 'bridge_cohesion_kpa':(300,3000,'log','conditional rock-bridge effective cohesion, not measured'),
 'rock_mu':(.45,.85,'linear','conditional friction coefficient'),
 'initial_temp_c':(-6,-.8,'linear','conditional initial joint temperature; within lab thermal domain'),
 'diffusivity_m2_s':(.9e-6,2.1e-6,'linear','external_transfer Weber2025 median +/- MAD endpoints; design interval not statistical CI'),
 'interface_depth_m':(.5,20,'log','conditional depth of thermal path, distinct from slab thickness'),
 'station_source_offset_c':(-18,-6,'linear','conditional combined elevation/exposure/surface transfer offset'),
 'storativity':(.002,.08,'log','conditional water-equivalent storage/head ratio'),
 'drainage_hours':(3,240,'log','conditional reservoir time constant; not fixed by external flow lag'),
 'connected_depth_m':(2,40,'log','conditional maximum connected water head'),
 'rain_multiplier':(.25,3,'log','conditional station-to-source and effective infiltration multiplier'),
 'melt_mm_deg_day':(.5,6,'linear','conditional degree-day coefficient; not an energy balance'),
 'damage_rate_h_inv':(1e-7,2e-4,'log','phenomenological design range; not calibrated ice rheology'),
 'damage_exponent':(2,6,'linear','phenomenological stress-ratio power'),
 'initial_damage':(.05,.7,'linear','conditional cohesion loss fraction'),
 'drainage_damage_coupling':(0,3,'linear','conditional damage-enhanced drainage exp(-k*D)')}


def write_json(path,data):
    path.parent.mkdir(parents=True,exist_ok=True);path.write_text(json.dumps(data,ensure_ascii=False,indent=2,allow_nan=False)+'\n')


def digest(path,role):
    raw=path.read_bytes()
    return {'path':str(path.relative_to(ROOT)),'bytes':len(raw),'sha256':hashlib.sha256(raw).hexdigest(),'role':role}


```

```
def initialize():
    BASE.mkdir(parents=True,exist_ok=True)
    for d in ['results','figures','exports','config']: (BASE/d).mkdir(exist_ok=True)
    if not CONFIG.exists():
        write_json(CONFIG,dict(experiment_id='q2r3-competing-mechanisms-v1',parameter_version='q2r3-design-v1',
          seed=260826,n_parameter_sets=256,models=list(MODEL_IDS),policies=['linear_extension','thawed_rock_contact'],
          start_utc='2026-06-01T00:00:00Z',end_utc='2026-08-27T00:00:00Z',evaluation_start_utc='2026-08-19T00:00:00Z',
          dt_hours=1.,dz_m=.5,thermal_bottom_m=40.,ranges={k:dict(min=v[0],max=v[1],scale=v[2],source=v[3]) for k,v in RANGES.items()},
          initial_water_storage_m=0,design_is_probability_prior=False,weights=None,independent_events=0,
          information_available_time=None,forecast_input_mode='ideal_weather',evaluation_stratum='conditional_scenario',
          crossing_rule='first sampled nonpositive margin after stable finite initial value; linear interpolation within bracket; initial nonpositive left censored',
          source_mapping='Tsource=Tstation+offset; surface=min(Tsource,0); rain as assumed mm/h times multiplier times linear -1..+1 C liquid partition; melt positive source-air degree day'))
    return json.loads(CONFIG.read_text())


def load_forcing(cfg):
    temporal=ROOT/'track_b/data/processed/temporal'
    axis=pd.read_parquet(temporal/'hourly_time_axis.parquet')
    strict=pd.read_parquet(temporal/'hourly_strict_qc0.parquet',columns=['MET__Ta_2m'])
    raw=pd.read_parquet(temporal/'hourly_raw_numeric.parquet',columns=['MET__Precip'])
    choose=(axis.time_utc>=cfg['start_utc'])&(axis.time_utc<=cfg['end_utc'])
    out=pd.DataFrame({'time_utc':axis.loc[choose,'time_utc'].reset_index(drop=True),
       'station_air_temp_c':strict.loc[choose,'MET__Ta_2m'].reset_index(drop=True),
       'precip_assumed_mm_h':raw.loc[choose,'MET__Precip'].reset_index(drop=True).replace(9999.9,np.nan)})
    assert out[['station_air_temp_c','precip_assumed_mm_h']].notna().all().all()
    assert (out.time_utc.diff().dropna()==pd.Timedelta(hours=1)).all()
    assert (out.precip_assumed_mm_h>=0).all()
    out['elapsed_hours']=np.arange(len(out));out['data_role']='scenario';out['forecast_input_mode']='ideal_weather'
    out['temperature_quality']='original_strict_QC0';out['precip_quality']='unknown_QC_and_accumulation; assumed_1h_interval'
    out['available_time']=None
    out.to_csv(BASE/'results/forcing_hourly_v1.csv',index=False)
    write_json(BASE/'results/forcing_support.json',{'rows':len(out),'first_utc':str(out.time_utc.iloc[0]),'last_utc':str(out.time_utc.iloc[-1]),
      'temperature_missing':0,'precip_missing':0,'precip_label_sum_mm_under_hourly_assumption':float(out.precip_assumed_mm_h.sum()),
      'no_interpolation':True,'operational_availability':None,'station_timezone_assumption':'UTC+8 as inherited, local equivalence still unknown',
      'source_space':'Gyirong regional station; source temperature/rain are conditional transfers',
      'used_as':'conditional_scenario ideal_weather retrospective forcing, not validated source observation',
      'excluded_long_gap':'2026-08-27T01:00Z onward not filled or treated as zero'})
    return out


def parameter_design(cfg):
    rng=np.random.default_rng(cfg['seed']);n=cfg['n_parameter_sets']
    u=np.column_stack([(rng.permutation(n)+rng.random(n))/n for _ in RANGES])
    p={}
    for j,(name,(lo,hi,scale,_)) in enumerate(RANGES.items()):
```

```
        p[name]=np.exp(np.log(lo)+(np.log(hi)-np.log(lo))*u[:,j]) if scale=='log' else lo+(hi-lo)*u[:,j]
    p['density_kg_m3']=917*(1-p['rock_volume_fraction'])+2700*p['rock_volume_fraction']
    return p


def take(parameters,index): return {k:np.asarray(v)[index] for k,v in parameters.items()}


def scenario_id(i,mechanism,policy): return f'q2r3-p{i:04d}-{mechanism}-'+('linear' if policy=='linear_extension' else 'thaw')


def run():
    t0=time.perf_counter();cfg=initialize();forcing=load_forcing(cfg);p=parameter_design(cfg)
    n=cfg['n_parameter_sets'];nt=len(forcing);air=forcing.station_air_temp_c.to_numpy();rain=forcing.precip_assumed_mm_h.to_numpy()
    thermal=thermal_transfer(air,p,1.,cfg['dz_m'])
    print('Thermal state complete',flush=True)
    # Representatives fixed by design index before seeing any result.
    reps=np.linspace(0,n-1,8,dtype=int);ev=int(np.flatnonzero(forcing.time_utc>=cfg['evaluation_start_utc'])[0])
    rows=[];tracks=[];params=[];ablation=[];crit=[];releases=[];summary_arrays={};watermax=0
    for policy in cfg['policies']:
      for mech in cfg['models']:
        result=simulate(air,rain,p,mech,policy,thermal=thermal)
        print(f'Completed {policy} {mech}',flush=True)
        summary_arrays[(mech,policy)]=np.min(result['failure_margin'][ev:],axis=0)
        watermax=max(watermax,float(result['water_budget_max_abs_m'].max()))
        for i in range(n):
          sid=scenario_id(i,mech,policy)
          onset=classify_trajectory(result['failure_margin'][:,i]);late=classify_trajectory(result['failure_margin'][ev:,i])
          record=dict(scenario_id=sid,parameter_set_id=f'q2r3-p{i:04d}',model_id=MODEL_IDS[mech],mechanism_id=mech,law_policy=policy,
            experiment_id=cfg['experiment_id'],parameter_version=cfg['parameter_version'],geometry_version='conditional_plane_slab_v1',
            evaluation_stratum='conditional_scenario',data_role='scenario',forecast_input_mode='ideal_weather',
            min_margin_all=float(result['failure_margin'][:,i].min()),min_margin_evaluation=float(result['failure_margin'][ev:,i].min()),
            final_margin=float(result['failure_margin'][-1,i]),initial_margin=float(result['failure_margin'][0,i]),
            final_interface_temp_c=float(result['interface_temp_c'][-1,i]),final_damage=float(result['damage'][-1,i]),
            peak_pressure_kpa=float(result['pressure_kpa'][:,i].max()),strength_domain_fraction=float(result['strength_domain'][:,i].mean()),
            evaluation_onset_status=late['onset_status'],**onset)
          record['crossing_time_utc']=str(forcing.time_utc.iloc[0]+pd.Timedelta(hours=onset['onset_elapsed_hours'])) if onset['onset_elapsed_hours'] is not None else None
          rows.append(record)
          params.append(dict(scenario_id=sid,parameter_set_id=f'q2r3-p{i:04d}',mechanism_id=mech,model_id=MODEL_IDS[mech],law_policy=policy,
            parameter_version=cfg['parameter_version'],geometry_version='conditional_plane_slab_v1',design_role='space_filling_not_probability',prior_weight=None,
            **{k:float(v[i]) for k,v in p.items()}))
        for i in reps:
          sid=scenario_id(i,mech,policy)
          status=classify_trajectory(result['failure_margin'][:,i])['onset_status']
          track=pd.DataFrame({'scenario_id':sid,'model_id':MODEL_IDS[mech],'mechanism_id':mech,'law_policy':policy,
           'time_utc':forcing.time_utc,'elapsed_hours':forcing.elapsed_hours,'failure_margin':result['failure_margin'][:,i],
```

```
136          'margin_definition':MARGIN_DEFINITION,'interface_temp_c':result['interface_temp_c'][:,i],'damage':result['damage'][:,i],
137          'storage_m':result['storage_m'][:,i],'pressure_kpa':result['pressure_kpa'][:,i],
138          'strength_in_lab_domain':result['strength_domain'][:,i],'critical_head_m':result['critical_head_m'][:,i],
139          'data_role':'scenario','forecast_input_mode':'ideal_weather','evaluation_stratum':'conditional_scenario',
140          'parameter_version':cfg['parameter_version'],'geometry_version':'conditional_plane_slab_v1','physical_phase':'initiation',
141          'available_time':None,'onset_status_whole_run':status})
142        # Last forcing contributing to each state, not a claim of real-time availability.
143        track['forcing_cutoff_utc']=forcing.time_utc.shift(1)
144        tracks.append(track)
145        if policy=='linear_extension':
146          plan_area=float([10000,40000,100000][list(reps).index(i)%3]);h=float(p['thickness_normal_m'][i]);theta=float(p['theta_deg'][i]);rho=float(p['density_kg_m3'][i]);rf=float(p['rock_volume_fraction'][i])
147          releases.append(dict(scenario_id=sid,model_id=MODEL_IDS[mech],mechanism_id=mech,geometry_version='conditional_area_requires_q4_polygon_v1',
148            parameter_version=cfg['parameter_version'],source_polygon_or_release_raster=None,release_thickness_or_volume='see normal/vertical thickness and conditional plan-area volume columns',
149            plan_area_m2=plan_area,release_thickness_normal_m=h,release_thickness_vertical_m=h/np.cos(np.deg2rad(theta)),
150            release_volume_m3=plan_area*h/np.cos(np.deg2rad(theta)),phase_fraction_basis='volume',phase_fractions=json.dumps({'rock':rf,'ice':1-rf,'free_water':0}),
151            density=rho,density_unit='kg
               m-3',initial_velocity=json.dumps({'x':0,'y':0,'z':0,'unit':'m
               s-1','crs':'EPSG:32645; zero vector direction immaterial'}),
152            onset_status=status,evaluation_stratum='conditional_scenario',
153            provenance_and_assumptions='area/plane/thickness/composition/zero velocity
               explicitly specified scenarios, not measured release;
               V=plan_area*h_normal/cos(theta); source polygon pending Q4; simulated onset
               is separate from travel time'))
154        if policy=='linear_extension' and mech=='S_T_H':
155          for label,kwargs in [('no_damage',dict(damage_enabled=False)),('no_drainage_damage_feedback',dict(drainage_feedback=False))]:
156            alt=simulate(air,rain,p,mech,policy,thermal=thermal,**kwargs)
157            for i in range(n):ablation.append(dict(parameter_set_id=f'q2r3-p{i:04d}',ablation=label,base_min_margin=float(result['failure_margin'][ev:,i].min()),
158              alternative_min_margin=float(alt['failure_margin'][ev:,i].min()),paired_difference=float((alt['failure_margin'][ev:,i]-result['failure_margin'][ev:,i]).min()),evaluation_stratum='conditional_scenario'))
159          dry=simulate(air,np.zeros_like(rain),p,mech,policy,thermal=thermal)
160          for i in range(n):ablation.append(dict(parameter_set_id=f'q2r3-p{i:04d}',ablation='zero_rain_scenario_melt_retained',base_min_margin=float(result['failure_margin'][ev:,i].min()),
161             alternative_min_margin=float(dry['failure_margin'][ev:,i].min()),paired_difference=float(dry['failure_margin'][ev:,i].min()-result['failure_margin'][ev:,i].min()),evaluation_stratum='conditional_scenario'))
162          for i in reps:
163            for temp in [-6.,-3.,-1.,-.5,0.]:
164              pi=take(p,[i]);state=resistance(pi,np.array([p['initial_damage'][i]]),np.array([temp]),np.array([0.]))
165              head=state['critical_head_m'][0];cap=min(p['connected_depth_m'][i],.98*state['normal_kpa'][0]/G)
166              crit.append(dict(parameter_set_id=f'q2r3-p{i:04d}',temperature_c=temp,normal_kpa=state['normal_kpa'][0],driving_kpa=state['driving_kpa'][0],
167                initial_damage=p['initial_damage'][i],critical_head_m=float(head) if np.isfinite(head) else None,
168                root_status=str(state['critical_head_status'][0]),allowed_head_capacity_m=cap,
```

```
169                 root_reachable_before_capacity=bool(np.isfinite(head) and
                    ↪  0<head<=cap),lab_temperature_domain=-8<=temp<=-.5,
170                 evaluation_stratum='conditional_scenario'))
171     outcomes=pd.DataFrame(rows);outcomes.to_csv(BASE/'results/scenario_outcomes.csv',index=Fal
        ↪  se)
172     pd.DataFrame(params).to_csv(BASE/'results/parameter_ensemble.csv',index=False)
173     pd.concat(tracks,ignore_index=True).to_parquet(BASE/'results/state_trajectories.parquet',i
        ↪  ndex=False,compression='zstd')
174     pd.DataFrame(releases).to_csv(BASE/'results/release_states.csv',index=False)
175     pd.DataFrame(ablation).to_csv(BASE/'results/ablation.csv',index=False)
176     pd.DataFrame(crit).to_csv(BASE/'results/critical_relations.csv',index=False)
177     comparison=[]
178     for (mech,policy),group in outcomes.groupby(['mechanism_id','law_policy'],sort=False):
179       comparison.append(dict(mechanism_id=mech,model_id=MODEL_IDS[mech],law_policy=policy,n_pa
          ↪  rameter_sets=len(group),
180        interval_crossing_n=int((group.onset_status=='interval_crossing').sum()),left_censored_
           ↪  n=int(group.onset_status.str.startswith('left_censored').sum()),
181        never_crossed_n=int((group.onset_status=='never_crossed').sum()),missing_n=int(group.on
           ↪  set_status.str.contains('missing').sum()),
182        median_min_margin=float(group.min_margin_evaluation.median()),p05_min_margin=float(grou
           ↪  p.min_margin_evaluation.quantile(.05)),p95_min_margin=float(group.min_margin_evalua
           ↪  tion.quantile(.95)),
183        wholly_lab_domain_n=int((group.strength_domain_fraction==1).sum()),mean_lab_domain_frac
           ↪  tion=float(group.strength_domain_fraction.mean()),
184        independent_local_training_events=0,independent_local_validation_events=0,evaluation_st
           ↪  ratum='conditional_scenario',design_fraction_is_probability=False))
185     pd.DataFrame(comparison).to_csv(BASE/'results/model_comparison.csv',index=False)
186     sensitivity=[]
187     for (mech,policy),response in summary_arrays.items():
188       # Joint design sensitivity, not confidence or causal parameter importance.
189       x=np.column_stack([p[k] if RANGES[k][2]=='linear' else np.log(p[k]) for k in RANGES])
190       z=(x-x.mean(0))/x.std(0);y=(response-response.mean())/response.std()
191       beta=np.linalg.lstsq(np.column_stack([np.ones(n),z]),y,rcond=None)[0][1:]
192       for j,k in enumerate(RANGES):
193         sensitivity.append(dict(mechanism_id=mech,law_policy=policy,parameter=k,spearman_rho=f
            ↪  loat(pd.Series(p[k]).rank().corr(pd.Series(response).rank())),standardized_linear_
            ↪  coefficient=float(beta[j]),
194            outcome='minimum margin Aug19..Aug27',n_joint_design=n,evaluation_stratum='conditio
               ↪  nal_scenario',interpretation='design-dependent association; no causal/empirical
               ↪  posterior interpretation'))
195     pd.DataFrame(sensitivity).to_csv(BASE/'results/sensitivity.csv',index=False)
196     # Pointwise extrema interaction does not commute: report both paired minima
197     # contrast and representative trajectory contrasts separately.
198     inter=summary_arrays[('S_T_H','linear_extension')]-summary_arrays[('S_T','linear_extension
        ↪  ')]-summary_arrays[('S_H','linear_extension')]+summary_arrays[('S','linear_extension')]
199     pd.DataFrame({'parameter_set_id':[f'q2r3-p{i:04d}' for i in
        ↪  range(n)],'contrast_of_minima_STH_minus_ST_minus_SH_plus_S':inter,
200       'evaluation_stratum':'conditional_scenario'}).to_csv(BASE/'results/interaction.csv',inde
          ↪  x=False)
201     write_json(BASE/'results/run_metadata.json',dict(runtime_seconds=time.perf_counter()-t0,py
        ↪  thon=platform.python_version(),numpy=np.__version__,
202       parameter_sets=n,models=4,law_policies=2,scenario_outcomes=len(rows),representative_stat
          ↪  e_rows=len(reps)*8*nt,
203       time_origin_utc=str(forcing.time_utc.iloc[0]),time_step_hours=1,nt=nt,information_cutoff
          ↪  ='forcing_cutoff_utc per state; real available_time unknown',
204       max_water_budget_error_m=watermax,input_version=INPUT_VERSION,probability_prior=False,in
          ↪  dependent_validation_events=0))
205     contract=dict(contract_version='q2r3-mechanism-contract-v1',input_version=INPUT_VERSION,mo
        ↪  del_ids=MODEL_IDS,
206        parameter_version=cfg['parameter_version'],evaluation_stratum='conditional_scenario',da
           ↪  ta_role='scenario',forecast_input_mode='ideal_weather',
```

```
        time_origin_utc=cfg['start_utc'],time_step_hours=1,forcing_interval='forcing[i] drives [time[i],time[i+1]); state[0] specified initial condition',
        real_available_time=None,margin_definition=MARGIN_DEFINITION,positive_is_stable=True,
        onset_status_values=['left_censored_already_unstable','interval_crossing','never_crossed','missing','missing_before_possible_crossing'],
        onset_rule=cfg['crossing_rule'],no_probability_prior=True,grid_weights=None,
        engine_entry='scripts/mechanism_engine.py: simulate(air_c,precip_mm_h,parameters,mechanism,policy,dt_hours,dz_m)',
        state_columns={'interface_temp_c':'degC','damage':'dimensionless cohesion-loss fraction','storage_m':'water-equivalent m','pressure_kpa':'kPa','failure_margin':'dimensionless'},
        equations=['sigma_n=rho*g*h_normal*cos(theta)/1000 kPa; tau=rho*g*h_normal*sin(theta)/1000 kPa',
         'R=(1-D)*(fb*cb+(1-fb)*cj(T))+max(sigma_n-u,0)*(fb*mub+(1-fb)*muj(T))',
         'cj=53.3-73.5*T kPa; muj=.42-.21*T; external_transfer ice-filled limestone fast-shear law',
         'dT/dt=alpha*Tzz; Tsurface=min(Tstation+offset,0); no-flux at40m',
         'dW/dt=I-W/tau_d; head=W/Sy; u=g*head kPa; capacity=min(connected_depth,.98*sigma_n/g)',
         'Dnext=1-(1-D)*exp(-A*min(tau/max(R,1e-6),10)^n*dt); cap D .999999',
         'tau_d=drainage_hours*exp(-drainage_damage_coupling*D)'],
        applicability={'Mamot2018':{'temperature_c':[-8,-.5],'effective_normal_kpa':[100,400],
          'material':'ice-filled limestone joint','strain_rate_s_inv':'mean (4.8+/-1.4)e-3; not slow creep'},
          'conditional_extensions':['effective-stress substitution','area mixing','damage law','temperature/rain transfer','melt proxy','thaw branch mu=.6']},
        refutation=['deep joint temperature unchanged contradicts rapid conductive weakning at that depth','measured head below conditional critical relation challenges pressure trigger','geometry/material mismatch rejects assigned slab/strength transfer'],
        limitations=['not event-calibrated; independent events=0','linear_extension outside lab range explicitly flagged','thaw branch uncalibrated','post-crossing continuation diagnostic only','representatives are 8 fixed design indices; all 2048 outcomes retain no-start and censoring'])
    write_json(BASE/'results/mechanism_contract.json',contract)
    export_v1()
    print(json.dumps({'outcomes':len(rows),'representative_state_rows':len(reps)*8*nt,'runtime_seconds':time.perf_counter()-t0,'water_budget_max_abs_m':watermax}))


def export_v1():
    common=['EXPERIMENT_PROTOCOL.md','config/experiment_v1.json','scripts/mechanism_engine.py','results/mechanism_contract.json','results/parameter_ensemble.csv','results/forcing_hourly_v1.csv']
    limitations=['All source mechanism and release states conditional scenarios, not local measurements or calibrated event probability.',
      'No probability weights. First-row unstable left-censored, never-crossed and missing retained.','Real weather availability unknown; ideal_weather retrospective forcing.',
      'Mamot strength transferred beyond original material/stress/rate/temperature domain where marked.','Initiation is separate from propagation arrival. Human scientific adoption pending.']
    for name,paths in [('mechanism',common+['results/state_trajectories.parquet','results/scenario_outcomes.csv']),
                       ('release',['results/release_states.csv','results/parameter_ensemble.csv','results/mechanism_contract.json','config/experiment_v1.json'])]:
        target=BASE/f'exports/{name}_bundle.json'
        bundle=dict(bundle_version=f'q2-r3-{name}-v1',producer_question='q2',input_version=INPUT_VERSION,evaluation_stratum='conditional_scenario',
          files=[digest(BASE/p,p) for p in paths],limitations=limitations)
        # Never mutate a published bundle or any byte-bound v1 file on rerun.
        if target.exists():
            assert json.loads(target.read_text())==bundle,'published bundle changed; create v2 instead'
```

```
243         else: write_json(target,bundle)
244
245
246 if __name__=='__main__': run()
```

程序 9：全 2048 解析热竞争与参考配对后处理

来源：track_b/research/q2/round3/scripts/run_analytic_thermal.py
SHA256: 8cb54a4a6db606994c18d9e82ab9dce8269e77b1e5445287f9f5d865f3e3e1d6

```
"""Full2048 analytical conductive challenger with offline paired numerical audit."""
from __future__ import annotations

import hashlib
import io
import json
import math
import platform
import time
from pathlib import Path

import numpy as np
import pandas as pd

from analytic_thermal_engine import (thermal_transfer_analytic, linear_causal_convolution,
                                     finite_step_kernel, reflection_temperature_bound)
from mechanism_engine import MODEL_IDS, MARGIN_DEFINITION, thermal_transfer, simulate,
↪  classify_trajectory
from run_experiments import RANGES, parameter_design, scenario_id

ROOT = Path(__file__).resolve().parents[5]
BASE = ROOT / "track_b/research/q2/round3"
OUT = BASE / "results/analytic_thermal"
ENGINE_SHA = "bb6636d2d3f463c894cdc6bc1df9c7fd0f93f9b7a6f9aa6e8233113630afe14e"
BUNDLE = BASE / "exports/analytic_thermal_bundle.json"
VERSION = "q2-r3-analytic-erfc-thermal-v1"


def digest(path, role):
    return dict(path=path.relative_to(ROOT).as_posix(), bytes=path.stat().st_size,
                sha256=hashlib.sha256(path.read_bytes()).hexdigest(), role=role)


def emit(path, raw):
    raw = raw.encode() if isinstance(raw, str) else raw
    assert path.resolve().is_relative_to(BASE)
    if path.exists():
        assert path.read_bytes() == raw, f"immutable analytical output differs: {path}"
    else:
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(raw)


def emit_json(path, data):
    emit(path, json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False)+"\n")


def emit_csv(path, frame):
    emit(path, frame.to_csv(index=False, lineterminator="\n"))


def bound_bundles():
    records = []
    for name in ["mechanism_bundle.json", "release_bundle.json", "mechanism_bundle_v2.json",
    ↪  "release_bundle_v2.json"]:
        path = BASE / "exports" / name
        obj = json.loads(path.read_text())
```

```
        records.append(digest(path, name))
        for item in obj["files"]:
            member = Path(item["path"])
            member = member if member.is_absolute() else ROOT/member
            assert member.stat().st_size == item["bytes"]
            assert hashlib.sha256(member.read_bytes()).hexdigest() == item["sha256"]
    return records


def numerical_benchmarks():
    p = dict(initial_temp_c=np.array([-5., -3.]), station_source_offset_c=np.array([0., 0.]),
             interface_depth_m=np.array([1., 2.]), diffusivity_m2_s=np.array([1e-6, 2e-6]))
    air = np.full(97, -1.)
    temperature, _ = thermal_transfer_analytic(air, p)
    direct_step = np.repeat(p["initial_temp_c"][None, :], len(air), axis=0)
    for k in range(1, len(air)):
        for j in range(2):
            direct_step[k, j] += (-1-p["initial_temp_c"][j])*math.erfc(p["interface_depth_m"][j]/(2*math.sqrt(p["diffusivity_m2_s"][j]*k*3600)))
    step_error = float(np.max(np.abs(temperature-direct_step)))
    assert step_error < 1e-11
    boundary = np.array([-4., -1., -3., -2., -6., 1., 2., -2., -1., -3., -4., 0., -5.])
    arbitrary, b = thermal_transfer_analytic(boundary, p)
    direct = np.repeat(p["initial_temp_c"][None, :], len(boundary), axis=0)
    delta = np.vstack([b[0]-p["initial_temp_c"], np.diff(b, axis=0)])
    for k in range(1, len(boundary)):
        for j in range(2):
            for i in range(k):
                direct[k, j] += delta[i, j]*math.erfc(p["interface_depth_m"][j]/(2*math.sqrt(p["diffusivity_m2_s"][j]*(k-i)*3600)))
    arbitrary_error = float(np.max(np.abs(arbitrary-direct)))
    assert arbitrary_error < 1e-11
    baseline = -4+np.sin(np.arange(97)/7)
    changed = baseline.copy(); changed[30:] += 4
    before, _ = thermal_transfer_analytic(baseline, p)
    after, _ = thermal_transfer_analytic(changed, p)
    causal_error = float(np.max(np.abs(before[:31]-after[:31])))
    assert causal_error < 1e-11
    assert float(np.max(np.abs(before[31:]-after[31:]))) > .01
    # z=0 interface sampling is the left boundary limit at the stored state.
    boundary_p = {**p, "interface_depth_m": np.array([0., 0.])}
    at_surface, bb = thermal_transfer_analytic(boundary, boundary_p)
    surface_error = float(np.max(np.abs(at_surface[1:]-bb[:-1])))
    assert surface_error < 1e-11
    matrix = []
    for bottom, depth, alpha, hours in [(2., 1., 2e-6, 168), (4., 3., 2e-6, 240), (40., 20., 2.1e-6, 2088)]:
        kernel, correction = finite_step_kernel(hours+1, [depth], [alpha], bottom, terms=12)
        kernel24, _ = finite_step_kernel(hours+1, [depth], [alpha], bottom, terms=24)
        assert np.max(np.abs(kernel-kernel24)) < 1e-13
        target = -5+4*kernel[-1, 0]
        previous_error = None
        for dt, dz in [(1., .125), (.25, .03125)]:
            pp = dict(initial_temp_c=np.array([-5.]), station_source_offset_c=np.array([0.]),
                      interface_depth_m=np.array([depth]), diffusivity_m2_s=np.array([alpha]))
            finite, _ = thermal_transfer(np.full(round(hours/dt)+1, -1.), pp, dt, dz, bottom)
            error = float(abs(finite[-1, 0]-target))
            if previous_error is not None:
                assert error < previous_error and error < .001
            previous_error = error
            matrix.append(dict(bottom_m=bottom, interface_depth_m=depth, alpha_m2_s=alpha,
            duration_hours=hours,
```

```
                                fd_dt_hours=dt, fd_dz_m=dz,
                                finite_mirror_final_temperature_c=float(target),
                                fd_final_temperature_c=float(finite[-1, 0]),
                                abs_fd_minus_mirror_c=error,
                                finite_minus_semi_kernel=float(correction[-1, 0]),
                                evaluation_stratum="numerical_benchmark"))
    return dict(benchmark_version="analytic-erfc-engineering-v1", all_passed=True,
                step_erfc_max_abs_error_c=step_error,
                arbitrary_direct_vs_fft_max_abs_error_c=arbitrary_error,
                future_perturbation_prefix_max_abs_error_c=causal_error,
                surface_left_limit_max_abs_error_c=surface_error,
                causal_tolerance_c=1e-11, future_change_first_input_index=30,
                invariant_state_indices_inclusive=[0, 30],
                zero_padding_rule="nfft=next_power_of_two_at_least(2nt-1)",
                finite_mirror_fd_case_matrix=matrix, independent_observed_events=0),
                pd.DataFrame(matrix)


def main():
    clock = time.perf_counter()
    assert str(ROOT) == "<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q2" and
    Path.cwd().resolve() == ROOT
    engine_path = BASE / "scripts/mechanism_engine.py"
    assert digest(engine_path, "engine")["sha256"] == ENGINE_SHA
    frozen = bound_bundles()
    cfg0 = json.loads((BASE / "config/experiment_v1.json").read_text())
    protocol = dict(experiment_id="q2r3-analytic-thermal-challenger-v1", parameter_points=256,
                    mechanism_groups=cfg0["models"], law_policies=cfg0["policies"],
                    outcomes_expected=2048,
                    fixed_representatives=np.linspace(0, 255, 8, dtype=int).tolist(),
                    boundary="B[i]=min(air[i]+offset,0) on [i,i+1); state0=Tinit; positive
                    source depths0.5..20m",
                    increments="dB0=B0-Tinit; dBj=Bj-Bj-1; K0=0,
                    Kk=erfc(z/(2sqrt(alpha*k*3600)))",
                    convolution="linear causal via rfft/irfft with nfft>=2nt-1, keep firstnt
                    outputs",
                    mechanical_engine_sha256=ENGINE_SHA, state_dt_hours=1.,
                    comparisons=["v2(0.125m) all2048 summaries and onset", "full0.0625m FD
                    all2048 trajectories and onset", "direct analytical/causal and
                    finite-mirror benchmarks"],
                    no_model_or_parameter_tuning_to_event_time=True,
                    design_is_probability_prior=False,
                    independent_observed_events=0,
                    registration_scope="Protocol saved before full256 analytical/FD
                    computation. Implementation-only finite mirror/FD cases were
                    preflighted before this file; they are engineering checks, not
                    independent scientific validation.")
    emit_json(OUT / "protocol.json", protocol)
    benchmark, matrix = numerical_benchmarks()
    emit_json(OUT / "numerical_benchmarks.json", benchmark)
    emit_csv(OUT / "finite_mirror_fd_cases.csv", matrix)
    print("Analytical direct/causal and finite-mirror benchmarks passed", flush=True)
    forcing_path = BASE / "results/refined_v2/forcing_hourly_v2.csv"
    forcing = pd.read_csv(forcing_path, parse_dates=["time_utc"])
    air, rain = forcing.station_air_temp_c.to_numpy(), forcing.precip_assumed_mm_h.to_numpy()
    assert len(air) == 2089 and np.array_equal(forcing.elapsed_hours, np.arange(len(air)))
    p = parameter_design(cfg0)
    assert np.all((p["interface_depth_m"] >= .5) & (p["interface_depth_m"] <= 20))
    assert np.all((p["diffusivity_m2_s"] >= .9e-6) & (p["diffusivity_m2_s"] <= 2.1e-6))
    cfg = {**cfg0, "experiment_id": protocol["experiment_id"], "solver_id":
    "analytic_halfspace_erfc_linear_causal_fft_v1",
           "thermal_dz_m": None, "thermal_domain": "semi_infinite; finite40m Neumann reflection
           explicitly bounded",
```

```
        "scenario_suffix": "analytic_erfc_v1", "state_dt_hours": 1.,
        ↪  "fft_padding_min_samples": 2*len(air)-1,
        "fft_nfft": 1 << (2*len(air)-2).bit_length()}
    cfg.pop("dz_m")
    emit_json(OUT / "config.json", cfg)
    emit_csv(OUT / "forcing_hourly.csv", forcing)
    thermal = thermal_transfer_analytic(air, p)
    bounds, total_variation, reflection = reflection_temperature_bound(air, p, 40.)
    bound_frame = pd.DataFrame(dict(parameter_set_id=[f"q2r3-p{i:04d}" for i in range(256)],
                                    interface_depth_m=p["interface_depth_m"],
                                    ↪  diffusivity_m2_s=p["diffusivity_m2_s"],
                                    used_boundary_total_variation_c=total_variation,
                                    first_reflection_kernel_upper_bound=reflection,
                                    finite40_minus_halfspace_temperature_abs_bound_c=bounds,
                                    evaluation_stratum="numerical_benchmark"))
    emit_csv(OUT / "reflection_bounds.csv", bound_frame)
    print(f"Analytical all256 thermal complete; max finite40 reflection temperature
    ↪  bound={bounds.max():.6g}C", flush=True)
    print("Computing all256 FD reference dz0.0625m", flush=True)
    fd_thermal = thermal_transfer(air, p, 1., .0625, 40.)
    reps = np.asarray(protocol["fixed_representatives"])
    evaluation = int(np.flatnonzero(forcing.time_utc >= cfg["evaluation_start_utc"])[0])
    v2 = pd.read_csv(BASE / "results/refined_v2/scenario_outcomes.csv")
    v2_by_base = v2.set_index("base_scenario_id")
    existing_fd = pd.read_csv(BASE /
    ↪  "results/refined_v2/full_reference_checks.csv").set_index("base_scenario_id")
    records, parameter_records, tracks, pairs, diagnostic, minima = [], [], [], [], [], {}
    watermax = 0.
    model_ids = {mechanism: f"q2r3_{mechanism}_analytic_v1" for mechanism in cfg["models"]}
    for policy in cfg["policies"]:
        for mechanism in cfg["models"]:
            result = simulate(air, rain, p, mechanism, policy, thermal=thermal)
            fd = simulate(air, rain, p, mechanism, policy, thermal=fd_thermal)
            minima[(mechanism, policy)] = result["failure_margin"][evaluation:].min(axis=0)
            watermax = max(watermax, float(result["water_budget_max_abs_m"].max()))
            for i in range(256):
                base_id = scenario_id(i, mechanism, policy)
                sid = base_id+"__analytic_erfc_v1"
                trajectory = result["failure_margin"][:, i]
                onset = classify_trajectory(trajectory)
                fd_onset = classify_trajectory(fd["failure_margin"][:, i])
                old_fd = existing_fd.loc[base_id]
                assert fd_onset["onset_status"] == old_fd.reference_onset_status
                if fd_onset["onset_elapsed_hours"] is not None:
                    np.testing.assert_allclose(fd_onset["onset_elapsed_hours"],
                    ↪  old_fd.reference_onset_elapsed_hours, atol=1e-8, rtol=0)
                late = classify_trajectory(trajectory[evaluation:])
                row = dict(scenario_id=sid, base_scenario_id=base_id,
                ↪  v2_scenario_id=base_id+"__thermal125v2",
                           parameter_set_id=f"q2r3-p{i:04d}", model_id=model_ids[mechanism],
                           ↪  mechanism_id=mechanism,
                           law_policy=policy, experiment_id=cfg["experiment_id"],
                           ↪  parameter_version=cfg["parameter_version"],
                           geometry_version="conditional_plane_slab_v1",
                           ↪  evaluation_stratum="conditional_scenario",
                           data_role="scenario", forecast_input_mode="ideal_weather",
                           min_margin_all=float(trajectory.min()),
                           ↪  min_margin_evaluation=float(trajectory[evaluation:].min()),
                           final_margin=float(trajectory[-1]),
                           ↪  initial_margin=float(trajectory[0]),
                           final_interface_temp_c=float(result["interface_temp_c"][-1, i]),
                           final_damage=float(result["damage"][-1, i]),
                           ↪  peak_pressure_kpa=float(result["pressure_kpa"][:, i].max()),
```

```
                    strength_domain_fraction=float(result["strength_domain"][:,
                    ↪  i].mean()),
                    evaluation_onset_status=late["onset_status"], **onset)
                row["crossing_time_utc"] = (forcing.time_utc.iloc[0]+pd.Timedelta(hours=onset[
                ↪  "onset_elapsed_hours"])).isoformat() if onset["onset_elapsed_hours"] is not
                ↪  None else None
                records.append(row)
                parameter_records.append(dict(scenario_id=sid, base_scenario_id=base_id,
                ↪  parameter_set_id=f"q2r3-p{i:04d}",
                                              model_id=model_ids[mechanism],
                                              ↪  mechanism_id=mechanism, law_policy=policy,
                                              parameter_version=cfg["parameter_version"],
                                              ↪  geometry_version="conditional_plane_slab_v1",
                                              design_role="space_filling_not_probability",
                                              ↪  prior_weight=None,
                                              **{key: float(value[i]) for key, value in
                                              ↪  p.items()}))
                v2row = v2_by_base.loc[base_id]
                adiff = trajectory-fd["failure_margin"][:, i]
                both_fd = onset["onset_elapsed_hours"] is not None and
                ↪  fd_onset["onset_elapsed_hours"] is not None
                both_v2 = onset["onset_elapsed_hours"] is not None and
                ↪  pd.notna(v2row.onset_elapsed_hours)
                branch_mismatch = (result["interface_temp_c"][:, i] > -.5) !=
                ↪  (fd["interface_temp_c"][:, i] > -.5)
                pairs.append(dict(scenario_id=sid, base_scenario_id=base_id,
                ↪  parameter_set_id=f"q2r3-p{i:04d}",
                                  mechanism_id=mechanism, law_policy=policy,
                                  analytic_onset_status=onset["onset_status"], analytic_onset_
                                  ↪  elapsed_hours=onset["onset_elapsed_hours"],
                                  analytic_crossing_lower_hours=onset["crossing_lower_hours"],
                                  ↪  analytic_crossing_upper_hours=onset["crossing_upper_hour
                                  ↪  s"],
                                  fd0625_onset_status=fd_onset["onset_status"], fd0625_onset_e
                                  ↪  lapsed_hours=fd_onset["onset_elapsed_hours"],
                                  fd0625_crossing_lower_hours=fd_onset["crossing_lower_hours"],
                                  ↪  fd0625_crossing_upper_hours=fd_onset["crossing_upper_hou
                                  ↪  rs"],
                                  v2_onset_status=v2row.onset_status,
                                  ↪  v2_onset_elapsed_hours=v2row.onset_elapsed_hours,
                                  abs_analytic_minus_fd_onset_hours=abs(onset["onset_elapsed_h
                                  ↪  ours"]-fd_onset["onset_elapsed_hours"]) if both_fd else
                                  ↪  None,
                                  signed_analytic_minus_fd_onset_hours=onset["onset_elapsed_ho
                                  ↪  urs"]-fd_onset["onset_elapsed_hours"] if both_fd else
                                  ↪  None,
                                  abs_analytic_minus_v2_onset_hours=abs(onset["onset_elapsed_h
                                  ↪  ours"]-v2row.onset_elapsed_hours) if both_v2 else None,
                                  fd_onset_category_changed=onset["onset_status"] !=
                                  ↪  fd_onset["onset_status"],
                                  v2_onset_category_changed=onset["onset_status"] !=
                                  ↪  v2row.onset_status,
                                  max_abs_margin_analytic_minus_fd=float(np.abs(adiff).max()),
                                  final_margin_delta_analytic_minus_fd=float(adiff[-1]),
                                  evaluation_min_margin_delta_analytic_minus_fd=float(trajecto
                                  ↪  ry[evaluation:].min()-fd["failure_margin"][evaluation:,
                                  ↪  i].min()),
                                  final_margin_delta_analytic_minus_v2=float(trajectory[-1]-v2
                                  ↪  row.final_margin),
                                  evaluation_min_margin_delta_analytic_minus_v2=float(trajecto
                                  ↪  ry[evaluation:].min()-v2row.min_margin_evaluation),
```

```
                                max_abs_interface_temperature_analytic_minus_fd_c=float(np.a
                                ↪  bs(result["interface_temp_c"][:,
                                ↪  i]-fd["interface_temp_c"][:, i]).max()),
                                thaw_branch_mismatch_hours=int(branch_mismatch.sum()) if
                                ↪  policy == "thawed_rock_contact" else 0,
                                evaluation_stratum="numerical_benchmark"))
        for i in reps:
            base_id = scenario_id(i, mechanism, policy)
            track = pd.DataFrame(dict(scenario_id=base_id+"__analytic_erfc_v1",
            ↪  base_scenario_id=base_id,
                                      model_id=model_ids[mechanism],
                                      ↪  mechanism_id=mechanism, law_policy=policy,
                                      time_utc=forcing.time_utc,
                                      ↪  elapsed_hours=forcing.elapsed_hours,
                                      failure_margin=result["failure_margin"][:, i],
                                      ↪  margin_definition=MARGIN_DEFINITION,
                                      interface_temp_c=result["interface_temp_c"][:, i],
                                      ↪  damage=result["damage"][:, i],
                                      storage_m=result["storage_m"][:, i],
                                      ↪  pressure_kpa=result["pressure_kpa"][:, i],
                                      strength_in_lab_domain=result["strength_domain"][:,
                                      ↪  i], critical_head_m=result["critical_head_m"][:,
                                      ↪  i],
                                      data_role="scenario",
                                      ↪  forecast_input_mode="ideal_weather",
                                      ↪  evaluation_stratum="conditional_scenario",
                                      physical_phase="initiation",
                                      ↪  parameter_version=cfg["parameter_version"],
                                      ↪  geometry_version="conditional_plane_slab_v1",
                                      available_time=None, onset_status_whole_run=classify
                                      ↪  _trajectory(result["failure_margin"][:,
                                      ↪  i])["onset_status"]))
            track["forcing_cutoff_utc"] = forcing.time_utc.shift(1)
            tracks.append(track)
        if mechanism == "S_T_H":
            for i in [52, 107, 227]:
                diagnostic.append(pd.DataFrame(dict(parameter_set_id=f"q2r3-p{i:04d}",
                ↪  mechanism_id=mechanism,
                                                    law_policy=policy,
                                                    ↪  time_utc=forcing.time_utc, elapsed
                                                    ↪  _hours=forcing.elapsed_hours,
                                                    analytic_margin=result["failure_margin
                                                    ↪  "][:, i],
                                                    ↪  fd0625_margin=fd["failure_margin"]
                                                    ↪  [:, i],
                                                    analytic_temp_c=result["interface_temp
                                                    ↪  _c"][:, i],
                                                    ↪  fd0625_temp_c=fd["interface_temp_c
                                                    ↪  "][:, i])))
        print(f"Analytical+FD paired complete {policy} {mechanism}", flush=True)
outcomes, parameters, pair_frame = pd.DataFrame(records), pd.DataFrame(parameter_records),
↪  pd.DataFrame(pairs)
assert len(outcomes) == 2048
emit_csv(OUT / "scenario_outcomes.csv", outcomes); emit_csv(OUT / "parameter_ensemble.csv",
↪  parameters)
emit_csv(OUT / "paired_v2_fd0625.csv", pair_frame)
emit_csv(OUT / "diagnostic_traces.csv", pd.concat(diagnostic, ignore_index=True))
memory = io.BytesIO()
pd.concat(tracks, ignore_index=True).to_parquet(memory, index=False, compression="zstd")
emit(OUT / "state_trajectories.parquet", memory.getvalue())
competition = []
```

```
    for (mechanism, policy), group in outcomes.groupby(["mechanism_id", "law_policy"], sort=False):
        competition.append(dict(mechanism_id=mechanism, model_id=model_ids[mechanism], law_policy=policy,
                                n_parameter_sets=len(group),
                                interval_crossing_n=int((group.onset_status == "interval_crossing").sum()),
                                left_censored_n=int(group.onset_status.str.startswith("left_censored").sum()),
                                never_crossed_n=int((group.onset_status == "never_crossed").sum()),
                                missing_n=int(group.onset_status.str.contains("missing").sum()),
                                median_min_margin=float(group.min_margin_evaluation.median()),
                                p05_min_margin=float(group.min_margin_evaluation.quantile(.05)),
                                p95_min_margin=float(group.min_margin_evaluation.quantile(.95)),
                                wholly_lab_domain_n=int((group.strength_domain_fraction == 1).sum()),
                                mean_lab_domain_fraction=float(group.strength_domain_fraction.mean()), independent_validation_events=0,
                                design_fraction_is_probability=False,
                                evaluation_stratum="conditional_scenario"))
    emit_csv(OUT / "model_comparison.csv", pd.DataFrame(competition))
    sensitivity = []
    for (mechanism, policy), response in minima.items():
        x = np.column_stack([p[key] if RANGES[key][2] == "linear" else np.log(p[key]) for key in RANGES])
        z = (x-x.mean(0))/x.std(0); y = (response-response.mean())/response.std()
        beta = np.linalg.lstsq(np.column_stack([np.ones(256), z]), y, rcond=None)[0][1:]
        for j, key in enumerate(RANGES):
            sensitivity.append(dict(mechanism_id=mechanism, law_policy=policy, parameter=key,
                                    spearman_rho=float(pd.Series(p[key]).rank().corr(pd.Series(response).rank())),
                                    standardized_linear_coefficient=float(beta[j]),
                                    n_joint_design=256,
                                    evaluation_stratum="conditional_scenario",
                                    interpretation="space-filling design association, no empirical/causal posterior"))
    emit_csv(OUT / "joint_sensitivity.csv", pd.DataFrame(sensitivity))
    interaction = minima[("S_T_H", "linear_extension")]-minima[("S_T", "linear_extension")]-minima[("S_H", "linear_extension")]+minima[("S", "linear_extension")]
    emit_csv(OUT / "interaction.csv", pd.DataFrame(dict(parameter_set_id=[f"q2r3-p{i:04d}" for i in range(256)], contrast_of_minima_STH_minus_ST_minus_SH_plus_S=interaction)))
    contract = json.loads((BASE / "results/mechanism_contract.json").read_text())
    contract.update(contract_version="q2r3-analytic-erfc-contract-v1", model_ids=model_ids,
                    engine_entry="scripts/analytic_thermal_engine.py: simulate_analytic(air_c, precip_mm_h,parameters,mechanism,policy,dt_hours=1)",
                    thermal_entry="thermal_transfer_analytic(air_c,parameters,dt_hours=1) returns frozen-engine-compatible(temperature,boundary)",
                    scenario_suffix="analytic_erfc_v1", base_scenario_join="base_scenario_id joins v1/v2 and release-geometry-v2; release states unchanged, use this model version onset/censoring when combined",
                    thermal_domain="semi_infinite; finite40m Neumann difference bounded using boundary total variation",
                    time0_surface_convention="positive depth: specified uniform interior initial state; if depth0 requested, states k>=1 use B[k-1] left limit",
                    state_trajectory_scope="8 fixed representatives x4 mechanisms x2 policies; all256 parameters/outcomes retained per group",
                    engine_sha256=ENGINE_SHA, source_available_time=None)
```

```
    contract["equations"] = ["T[k]=Tinit+sum_{j=0}^{k-1}dB[j]*erfc(z/(2sqrt(alpha*(k-j)*3600)));
    B=min(air+offset,0)" if str(item).startswith("dT/dt=") else item for item in contract["equations"]]
    emit_json(OUT / "mechanism_contract.json", contract)

    def maximum(column):
        good = pair_frame[column].dropna()
        if not len(good):
            return None
        row = pair_frame.loc[good.idxmax()]
        return dict(value=float(row[column]), base_scenario_id=row.base_scenario_id,
        mechanism_id=row.mechanism_id, law_policy=row.law_policy)

    bound_i = int(bounds.argmax())
    parameter_bound = float(math.erfc(60/(2*math.sqrt(2.1e-6*2088*3600))))
    summary = dict(analytic_version=VERSION, evaluation_stratum="conditional_scenario",
    input_version=json.loads((BASE /
    "exports/mechanism_bundle.json").read_text())["input_version"],
                   frozen_bundles_before_and_after=frozen, engine_sha256=ENGINE_SHA,
                   benchmark_all_passed=True, full_parameter_sets=256,
                   scenario_outcomes=len(outcomes), representative_state_rows=133696,
                   native_state_dt_hours=1.,
                   time_origin_utc=forcing.time_utc.iloc[0].isoformat(),
                   forcing_used_sha256=digest(forcing_path, "forcing")["sha256"],
                   fft_nfft=cfg["fft_nfft"],
                   fft_min_linear_padding=cfg["fft_padding_min_samples"],
                   max_actual_reflection_temperature_bound_c=float(bounds.max()),
                   max_bound_parameter_set_id=f"q2r3-p{bound_i:04d}",
                   max_actual_reflection_kernel_bound=float(reflection.max()),
                   conservative_parameter_box_first_reflection_kernel_bound=parameter_bound,
                   max_boundary_total_variation_c=float(total_variation.max()),
                   box_kernel_times_max_actual_boundary_tv_bound_c=float(parameter_bound*total
                   _variation.max()),
                   finite_bound_scope="max over supplied256 actual parameter+boundary columns,
                   horizon2088h, L40m; box bound uses z20/alpha2.1e-6",
                   analytic_status_counts=outcomes.onset_status.value_counts().to_dict(),
                   finite_analytic_fd_onset_pairs=int(pair_frame.abs_analytic_minus_fd_onset_h
                   ours.notna().sum()),
                   finite_analytic_v2_onset_pairs=int(pair_frame.abs_analytic_minus_v2_onset_h
                   ours.notna().sum()),
                   fd_onset_category_changes=int(pair_frame.fd_onset_category_changed.sum()),
                   v2_onset_category_changes=int(pair_frame.v2_onset_category_changed.sum()),
                   max_analytic_fd_onset_difference=maximum("abs_analytic_minus_fd_onset_hours
                   "),
                   max_analytic_v2_onset_difference=maximum("abs_analytic_minus_v2_onset_hours
                   "),
                   max_analytic_fd_margin_difference=maximum("max_abs_margin_analytic_minus_fd
                   "),
                   max_analytic_fd_temp_difference=maximum("max_abs_interface_temperature_anal
                   ytic_minus_fd_c"),
                   thaw_branch_mismatch_hours=int(pair_frame.thaw_branch_mismatch_hours.sum()),
                   max_water_budget_error_m=watermax,
                   probabilities_assigned=False, independent_observed_events=0,
                   scientific_adoption="pending_team_review",
                   runtime_stack=dict(python=platform.python_version(), numpy=np.__version__,
                   pandas=pd.__version__))
    emit_json(OUT / "summary.json", summary)
    doc = f"""# 解析热传导竞争路线

本路线完成原256参数×4机制×2强度策略=2048情景和8预定代表×8组×2089小时=133696行状态。原引擎SHA
`{ENGINE_SHA}`；只通过`simulate(thermal=...)`替换热传导模块，1h水/损伤状态步长、参数、气象边界、
判据和删失规则不变，不进行事件时刻拟合。
```

方程、因果性与边界

半无限一维热传导，初始均匀Tinit，表面B[i]=min(air[i]+offset,0)于[i,i+1)保持。令ΔB0=B0−Tinit，ΔBj=Bj−Bj−1，K0=0，Kk=erfc[z/(2√(αk・3600))]，则T[k]=Tinit+Σ(j<k)ΔB[j]K[k−j]。使用零填充FFT线性卷积，nt=2089，nfft={cfg['fft_nfft']}≥2nt−1={cfg['fft_padding_min_samples']}，禁止循环卷积或未来边界泄漏。实际深度0.5–20m严格大于0；若调用者给z=0，state[k]取边界B[k−1]左极限，state0保留指定初值。公开可调用`thermal_transfer_analytic`和`simulate_analytic`均在`analytic_thermal_engine.py`。

独立工程检查：阶跃对erfc最大温差{benchmark['step_erfc_max_abs_error_c']:.5g}°C；任意预设分段边界」直接逐项和对FFT误差{benchmark['arbitrary_direct_vs_fft_max_abs_error_c']:.5g}°C；只改t≥30h的未来输入时，state0..30最大差{benchmark['future_perturbation_prefix_max_abs_error_c']:.5g}°C，均在1e−11°C容差内。镜像有限域解与细FD的矩阵见`finite_mirror_fd_cases.csv`，包括可见反射的小域测试和生产域极端测试；不是本地事件科学验证。

40m 无通量边界与半无限解的差

Laplace传递比cosh((L−z)q)/cosh(Lq)=[exp(−zq)+exp(−(2L−z)q)]/[1+exp(−2Lq)]，q=√(s/α)。展开几何级数并对阶跃除s，逆变换得到Σ(n≥0)(−1)^n[erfc((2nL+z)/a)+erfc((2(n+1)L−z)/a)]，a=2√(αt)。与半无限核相减后可成对写作Σ(n≥1)(−1)^(n−1)[erfc((2nL−z)/a)−erfc((2nL+z)/a)]。括号非负且随n递减，因此修正位于0与首项之间，并小于erfc((2L−z)/a)。任意分段边界的温差绝对值上界为这一末时核上界乘Σ|ΔB|；包含初始表面跃变，末行未作用的边界不计。

实际256参数/实际边界的最大温差上界 **{float(bounds.max()):.9g}°C**，参数q2r3-p{bound_i:04d}。参数盒z≤20m、α≤2.1e−6m²/s、t≤2088h、L=40m的首反射核上界{parameter_bound:.9g}，乘实际最大边界总变差{float(total_variation.max()):.9g}°C后上界{float(parameter_bound*total_variation.max()):.9g}°C。修正小于浮点舍入，代码仍独立保存反射核修正及界，未将相减得到0误称数学上等于0。这个界只说明两种理想热边界的差，不约束真实源区边界、热通道、相变、热对流或观测误差。

全量竞争和数值比较

解析对0.0625m有限差分的有限启动时间对{summary['finite_analytic_fd_onset_pairs']}/2048，类别改变{summary['fd_onset_category_changes']}；对已发布0.125m v2的有限时间对{summary['finite_analytic_v2_onset_pairs']}/2048，类别改变{summary['v2_onset_category_changes']}。最大启动差分别{summary['max_analytic_fd_onset_difference']['value']:.9g}h与{summary['max_analytic_v2_onset_difference']['value']:.9g}h，逐情景及原生小时首越阈区间见`paired_v2_fd0625.csv`。解析对0.0625m全期逐时最大裕度差{summary['max_analytic_fd_margin_difference']['value']:.9g}；thaw挑战分支切换不一致合计{summary['thaw_branch_mismatch_hours']}个情景小时。不要因启动类别相同就声称时刻已收敛。

此解析模块可作为所声明线性热传导方程的数值参考，避免热空间网格误差；水压、损伤和强度反馈仍按1h状态离散，不是整套耦合系统解析解。局部温压、物源/结构面、站源映射与损伤材料仍为条件情景或外部迁移。冰填石灰岩快速剪切实验的温/应力域标志不证明当地材料适用；升温穿越−0.5°C的thaw强度切换仍是未校准挑战假设。没有独立事件、概率先验或实测启动真值，后越阈计算只是反事实诊断。

新scenario_id加`__analytic_erfc_v1`，`base_scenario_id`对应v1/v2及已有释放几何v2；释放体积、厚度、组分、几何不改。与本路线组合时采用本模型版本的onset/censor，不能把旧释放表onset当第二个真值。最终科学采用由队员联合审视。

实际离线复跑命令：

```sh
track_b/.venv/bin/python track_b/research/q2/round3/scripts/run_analytic_thermal.py
```

```
协议先于全量求解保存在`results/analytic_thermal/protocol.json`；
    协议明确披露实现前期镜像/FD工程预检，未当独立科学验证。
    入口`exports/analytic_thermal_bundle.json`逐文件绑定，无自包含；所有旧包原字节保留。
"""
    emit(BASE / "ANALYTIC_THERMAL.md", doc)
    paths = [(OUT/name, name) for name in ["protocol.json", "config.json",
    "forcing_hourly.csv", "numerical_benchmarks.json", "finite_mirror_fd_cases.csv",
            "reflection_bounds.csv", "scenario_outcomes.csv", "parameter_ensemble.csv",
            "state_trajectories.parquet", "model_comparison.csv",
            "paired_v2_fd0625.csv", "diagnostic_traces.csv", "joint_sensitivity.csv",
            "interaction.csv", "mechanism_contract.json", "summary.json"]]
    paths += [(BASE/"scripts/analytic_thermal_engine.py", "callable_analytic_thermal_wrapper"),
              (Path(__file__).resolve(), "analytic_reproduction_script"), (engine_path,
              "frozen_mechanical_engine"),
              (BASE/"scripts/run_experiments.py", "frozen_parameter_design_source"),
              (BASE/"ANALYTIC_THERMAL.md", "methods_results_limits")]
    bundle = dict(bundle_version=VERSION, producer_question="q2",
    input_version=summary["input_version"], evaluation_stratum="conditional_scenario",
                  files=[digest(path, role) for path, role in paths],
                  limitations=["All source models conditional; no local event calibration,
                  probabilities or verified initiation time.",
                               "Analytical solution is for linear half-space thermal conduction
                               with the specified hourly step boundary only.",
                               "Finite40m Neumann replacement bounded by actual boundary total
                               variation; thermal advection, enthalpy/melting and
                               unmeasured boundary physics remain excluded.",
                               "Hourly water/damage/strength coupling retains
                               time-discretization uncertainty; analytic thermal does not
                               solve the whole coupled system exactly.",
                               "Initial instability is left-censored; no-start/missing
                               retained. base_scenario_id maps prior mechanism/release
                               versions without rewriting them.",
                               "Mamot transfer/thaw policy/source geometry remain scientific
                               assumptions requiring team review."])
    assert bound_bundles() == frozen
    emit_json(BUNDLE, bundle)
    print(json.dumps(dict(bundle=digest(BUNDLE, "analytic_thermal"),
    max_reflection_bound_c=float(bounds.max()),
                          max_fd_time_difference=summary["max_analytic_fd_onset_difference"],
                          max_v2_time_difference=summary["max_analytic_v2_onset_difference"],
                          fd_category_changes=summary["fd_onset_category_changes"],
                          runtime_seconds=time.perf_counter()-clock)), flush=True)


if __name__ == "__main__":
    main()
```

程序 10：局部雅可比与参数补偿基准

来源：track_b/research/q2/round3/scripts/analyze_identifiability.py
SHA256: 13bf5a0542300cf60b7b6f693fdc3189e055d58b9bbdfc1671dbfd72ce2a3569

```python
"""Prospective local identifiability benchmark, registered here before calculation.

Design: use mechanism_engine.resistance at one fixed, in-domain engineering state.
Four unknowns are normalized by explicitly chosen parameter scales. Differentiate
the prescribed observation operators by central finite differences, standardize
by prescribed measurement noise, and compare SVD/rank for five nested designs.
There are no measured observations, fitted event times or independent events.
Repeated margin measurements mean hypothetical independent instrument repeats
at an unchanged state; their information gain is conditional on that assumption.

Only outputs results/identifiability.csv and identifiability_details.json.
"""
from __future__ import annotations

import csv
import hashlib
import importlib.util
import json
from pathlib import Path
import sys

import numpy as np

sys.dont_write_bytecode = True
HERE = Path(__file__).resolve().parent
ROUND = HERE.parent
ENGINE = HERE / "mechanism_engine.py"
BASELINE = "904d7257b899b9f71d1ac9e0eab7479e459eab52"

PARAMETERS = ["bridge_cohesion_kpa", "initial_damage", "head_m", "interface_temperature_c"]
UNITS = ["kPa", "1", "m", "degC"]
CENTER = np.array([800.0, 0.30, 5.0, -2.5])
SCALES = np.array([500.0, 0.20, 5.0, 1.0])
NOISE = {"failure_margin": 0.05, "interface_temperature_c": 0.10,
         "pressure_kpa": 5.0, "damage": 0.05}
FIXED = {"theta_deg": 55.0, "density_kg_m3": 2400.0,
         "thickness_normal_m": 20.0, "bridge_fraction": 0.20,
         "rock_mu": 0.60}
DESIGNS = [
    ("margin_single", ["failure_margin"]),
    ("margin_24_fixed_condition_repeats", ["failure_margin"] * 24),
    ("margin_plus_temperature", ["failure_margin", "interface_temperature_c"]),
    ("margin_plus_temperature_pressure", ["failure_margin", "interface_temperature_c",
     "pressure_kpa"]),
    ("margin_plus_temperature_pressure_damage", ["failure_margin", "interface_temperature_c",
     "pressure_kpa", "damage"]),
]


def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def load_engine():
    spec = importlib.util.spec_from_file_location("q2_identifiability_engine", ENGINE)
    engine = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(engine)
```

```
    return engine


def main():
    engine_hash = digest(ENGINE)
    engine = load_engine()

    def physical_state(x):
        p = {**FIXED, "bridge_cohesion_kpa": float(x[0])}
        state = engine.resistance(p, float(x[1]), float(x[3]), engine.G * float(x[2]))
        return {k: (bool(v) if np.asarray(v).dtype == bool else
                    str(v) if np.asarray(v).dtype.kind in "US" else float(v))
                for k, v in state.items()}

    def observe(x, names):
        state = physical_state(x)
        values = {"failure_margin": state["failure_margin"],
                  "interface_temperature_c": float(x[3]),
                  "pressure_kpa": float(engine.G * x[2]), "damage": float(x[1])}
        return np.array([values[name] for name in names])

    def jacobian(names, step):
        jac = np.empty((len(names), len(PARAMETERS)))
        noise = np.array([NOISE[name] for name in names])
        for j in range(len(PARAMETERS)):
            delta = np.zeros(len(PARAMETERS)); delta[j] = step * SCALES[j]
            jac[:, j] = (observe(CENTER + delta, names) - observe(CENTER - delta, names)) / (2
            ↪  * step * noise)
        return jac

    base = physical_state(CENTER)
    assert base["strength_domain"] and base["contact_valid"]
    fb, damage, t = FIXED["bridge_fraction"], CENTER[1], CENTER[3]
    cj, muj = engine.joint_strength(t)
    unscaled_cohesion = fb * CENTER[0] + (1 - fb) * float(cj)
    analytic_margin = np.array([
        (1 - damage) * fb,
        -unscaled_cohesion,
        -engine.G * base["mu"],
        (1 - damage) * (1 - fb) * -73.5 + base["effective_kpa"] * (1 - fb) * -0.21,
    ]) / base["driving_kpa"] * SCALES / NOISE["failure_margin"]

    expected_ranks = [1, 1, 2, 3, 4]
    results, rows = [], []
    checks = []
    for (design_id, names), expected in zip(DESIGNS, expected_ranks):
        jac = jacobian(names, 1e-4)
        u, singular, vt = np.linalg.svd(jac, full_matrices=True)
        tol = 1e-10 * singular[0]
        rank = int((singular > tol).sum())
        singular_full = np.zeros(len(PARAMETERS)); singular_full[:len(singular)] = singular
        nonzero = singular[singular > tol]
        condition = float(nonzero[0] / nonzero[-1]) if rank == len(PARAMETERS) else None
        subspace_condition = float(nonzero[0] / nonzero[-1])
        perturbations = []
        for step in [1e-3, 1e-5]:
            other = jacobian(names, step)
            other_sv = np.linalg.svd(other, compute_uv=False)
            other_rank = int((other_sv > other_sv[0] * 1e-10).sum())
            perturbations.append({"normalized_step": step, "rank": other_rank,
                                  "max_abs_jacobian_difference": float(np.max(np.abs(other -
                                  ↪  jac)))})
            assert other_rank == expected
```

```
        assert rank == expected
        error = float(np.max(np.abs(jac[0] - analytic_margin)))
        assert error < 1e-7
        check = {"design_id": design_id, "expected_rank": expected, "actual_rank": rank,
                 "analytic_margin_jacobian_max_abs_error": error,
                 "finite_difference_step_checks": perturbations}
        checks.append(check)
        record = {"design_id": design_id, "observation_names": names,
                  "observation_center_values": observe(CENTER, names).tolist(),
                  "standardized_jacobian": jac.tolist(),
                  "singular_values_returned_by_svd": singular.tolist(),
                  "parameter_space_singular_values_padded": singular_full.tolist(),
                  "rank": rank, "rank_relative_tolerance": 1e-10,
                  "nullity": len(PARAMETERS) - rank,
                  "nullspace_directions_normalized_parameters": vt[rank:].tolist(),
                  "condition_full_parameter_space": condition,
                  "condition_status": "finite" if condition is not None else
                  ↪  "infinite_rank_deficient",
                  "condition_identified_subspace": subspace_condition,
                  "evaluation_stratum": "numerical_benchmark", "independent_events": 0}
        results.append(record)
        meanings = {
            1: "one local mechanical resistance combination only; three-dimensional parameter
            ↪  nullspace",
            2: "interface temperature separately constrained; cohesion-damage-head remain
            ↪  compensating",
            3: "temperature and pressure constrained; surviving product
            ↪  (1-D)*(fb*cb+(1-fb)*cj)",
            4: "four parameters locally distinguishable only given ideal direct damage, known
            ↪  geometry/fb/rock_mu and correct law",
        }
        rows.append({"experiment_id": "q2r3_local_identifiability_v1", "design_id": design_id,
                     "model_id": engine.MODEL_IDS["S_T_H"], "mechanism_id": "S_T_H",
                     "evaluation_stratum": "numerical_benchmark", "independent_events": 0,
                     "data_role": "scenario", "n_observations": len(names),
                     "n_unknown_parameters": len(PARAMETERS), "rank": rank,
                     "nullity": len(PARAMETERS)-rank,
                     "singular_values_noise_and_parameter_scaled":
                     ↪  json.dumps(singular_full.tolist()),
                     "condition_full_parameter_space": "" if condition is None else condition,
                     "condition_status": record["condition_status"],
                     "condition_identified_subspace": subspace_condition,
                     "normalization": "d(y/noise)/d((parameter-center)/scale); all scales and
                     ↪  noise prescribed",
                     "identifiable_combination": meanings[rank],
                     "most_valuable_added_measurements": "independent interface temperature and
                     ↪  hydraulic head; calibrated damage/bridge constraint needed to separate
                     ↪  cohesion-damage product",
                     "limitations": "local Jacobian only; fixed known geometry and fractions;
                     ↪  no local observations or likelihood calibration; unchanged repeats add
                     ↪  precision only under independent noise",
                     "engine_sha256": engine_hash})

    # Exact compensation pairs for this engine's fixed-state algebra. All remain
    # in-domain and retain the same original driving stress and failure margin.
    cases = [("specified_center", CENTER.copy())]
    x = CENTER.copy(); x[1] = 0.50
    x[0] = (base["cohesion_kpa"] / (1-x[1]) - (1-fb)*float(cj)) / fb
    cases.append(("higher_bridge_cohesion_with_more_damage", x))
    x = CENTER.copy(); x[2] = 8.0
    required_cohesion = base["cohesion_kpa"] + engine.G * (x[2]-CENTER[2]) * base["mu"]
    x[0] = (required_cohesion/(1-x[1]) - (1-fb)*float(cj)) / fb
    cases.append(("higher_bridge_cohesion_with_more_head", x))
```

```
    x = CENTER.copy(); x[3] = -2.0
    cj_new, mu_new = engine.joint_strength(x[3])
    mu_mixed_new = fb*FIXED["rock_mu"] + (1-fb)*float(mu_new)
    required_cohesion = base["resistance_kpa"] - base["effective_kpa"] * mu_mixed_new
    x[0] = (required_cohesion/(1-x[1]) - (1-fb)*float(cj_new)) / fb
    cases.append(("higher_bridge_cohesion_with_warmer_joint", x))
    pairs = []
    for label, x in cases:
        state = physical_state(x)
        assert state["strength_domain"] and state["contact_valid"]
        difference = state["failure_margin"] - base["failure_margin"]
        assert abs(difference) < 1e-12
        pairs.append({"case_id": label, "parameters": dict(zip(PARAMETERS, map(float,x))),
                      "state": state, "margin_difference_from_center": difference,
                      "evaluation_stratum": "numerical_benchmark", "independent_events": 0})

    details = {"experiment_id": "q2r3_local_identifiability_v1", "baseline_commit": BASELINE,
               "evaluation_stratum": "numerical_benchmark", "independent_events": 0,
               "data_role": "scenario", "event_id": None,
               "time_utc": None, "time_note": "fixed-state hypothetical experimental design,
               ↪  not an event observation",
               "input_version": {"engine_path": str(ENGINE.relative_to(ROUND.parents[3])),
               ↪  "engine_sha256": engine_hash,
                                 "script_sha256": digest(Path(__file__))},
               "parameter_order": PARAMETERS, "parameter_units": UNITS,
               "parameter_center": CENTER.tolist(), "parameter_scales": SCALES.tolist(),
               "fixed_known_parameters": FIXED, "prescribed_noise_standard_deviations": NOISE,
               "noise_status": "arbitrary prospective design values, independent Gaussian
               ↪  measurement standardization only; no noisy data generated or fitted",
               "finite_difference": {"method": "central", "normalized_step": 1e-4,
               ↪  "check_steps": [1e-3,1e-5]},
               "observation_definitions": {"failure_margin": engine.MARGIN_DEFINITION,
                   "interface_temperature_c": "hypothetical direct interface temperature",
                   "pressure_kpa": "hypothetical direct pressure, u=g*head with water
                   ↪  density1000kg/m3",
                   "damage": "ideal direct model-state damage; actual damage proxy would
                   ↪  require separate calibration"},
               "center_state": base, "designs": results, "exact_compensation_cases": pairs,
               "structural_analysis": {"surviving_combination_given_T_and_u":
               ↪  "C_eff=(1-D)*(fb*cb+(1-fb)*cj(T))",
                   "cohesion_damage_null_relation": "dcb=[fb*cb+(1-fb)*cj(T)]/[fb*(1-D)]*dD",
                   "cohesion_pressure_null_relation_at_fixed_D_and_T":
                   ↪  "dcb=mu_mixed/[fb*(1-D)]*du",
                   "repeated_unchanged_conditions": "24 identical Jacobian rows retain rank1;
                   ↪  singular value increases sqrt(24) only with independent noise and no
                   ↪  shared model discrepancy",
                   "full_rank_scope": "local conditional identifiability of four unknowns only;
                   ↪  not global uniqueness or practical field identifiability; allowing
                   ↪  unknown fb/geometry/constitutive coefficients changes the result"},
               "validation": {"rank_expected_vector": expected_ranks, "all_rank_checks_passed":
               ↪  True,
                   "analytic_margin_jacobian": analytic_margin.tolist(),
                   ↪  "finite_difference_checks": checks,
                   "same_margin_max_abs_error": max(abs(p["margin_difference_from_center"]) for
                   ↪  p in pairs),
                   "all_compensation_cases_in_joint_domain": True},
               "limits": ["No local measured data, inverse fit, independent validation,
               ↪  event-time estimate or coverage claim",
                          "All parameter centers/scales/noise and measurement designs are
                          ↪  prescribed numerical-benchmark choices",
                          "Rank depends on stated normalization/tolerance and local
                          ↪  differentiability away from contact/domain transitions",
```

```
                        "Mamot brittle shear law transferred conditionally; not locally calibrated; added direct damage is idealized",
                        "Parameter compensation shows non-uniqueness, not likelihood or posterior probability"]}
    assert digest(ENGINE) == engine_hash, "engine changed during benchmark; rerun against stable version"
    with (ROUND/"results/identifiability.csv").open("w",newline="") as f:
        writer=csv.DictWriter(f,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(rows)
    (ROUND/"results/identifiability_details.json").write_text(json.dumps(details,indent=2,allow_nan=False)+"\n")
    print(json.dumps({"rank_vector":[r["rank"] for r in results],
                      "full_design_condition_number":results[-1]["condition_full_parameter_space"],
                      "compensation_max_abs_margin_error":details["validation"]["same_margin_max_abs_error"],
                      "center_strength_domain":base["strength_domain"],"independent_events":0}))


if __name__ == "__main__":
    main()
```

程序 11：完整外部实验本构拟合、分组留出与条件不确定性

来源：track_b/research/q2/round3/lab_strength/model/competition.py
SHA256: 5f93736b630b683ad52ba0c7d9fbca0fe9f50b4f88d0e954579a620506d53292

```
"""Fixed, interpretable lab constitutive competition; no local-event inputs.

All domains/held-out groups use nominal experimental classes. Continuous
predictors are the measured temperature and area-corrected normal stress.
"""
from __future__ import annotations

from collections import Counter
import hashlib
import json
from pathlib import Path
import platform
import sys
import time

import numpy as np
import pandas as pd
from scipy.stats import t as student_t

HERE = Path(__file__).resolve().parent
LAB = HERE.parent
OUT = HERE / "results"
DATA = LAB / "audit/results/observations.csv"
T = "temperature_at_failure_c"
S = "normal_stress_at_failure_kpa"
TC = "temperature_target_c"
SC = "normal_stress_target_kpa"
Y = "peak_shear_kpa"
MODELS = (
    "mc_linear", "mc_temperature_quadratic", "mc_stress_quadratic",
    "mc_stress_hinge_400",
)
DOMAINS = ("low_full", "low_eq6_temperature", "warm_low", "warm_all")
WEIGHTS = ("row_ols", "cell_equal")
BOOTSTRAPS = 1000
SEED = 26082418


def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def save_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) +
    ↪  "\n")


def write_csv(name, rows):
    pd.DataFrame(rows).to_csv(OUT / name, index=False, float_format="%.15g",
    ↪  lineterminator="\n")


def data_domains(df):
    low = df[SC] <= 400
    warm = df[TC].between(-4, -.5)
    return {
        "low_full": df.loc[low].copy(),
```

```
        "low_eq6_temperature": df.loc[low & df[TC].between(-8, -.5)].copy(),
        "warm_low": df.loc[low & warm].copy(),
        "warm_all": df.loc[warm].copy(),
    }


def features(model, temperature, stress):
    t, s = np.broadcast_arrays(np.asarray(temperature, float) / 5, np.asarray(stress, float) /
    ↪  400)
    columns = [np.ones_like(t), t, s, t * s]
    if model == "mc_temperature_quadratic":
        columns += [t**2, t**2 * s]
    elif model == "mc_stress_quadratic":
        columns += [s**2, t * s**2]
    elif model == "mc_stress_hinge_400":
        hinge = np.maximum(s - 1, 0)
        columns += [hinge, t * hinge]
    elif model != "mc_linear":
        raise ValueError(model)
    return np.stack(columns, axis=-1)


def weights(df, policy):
    if policy == "row_ols":
        return np.ones(len(df))
    if policy != "cell_equal":
        raise ValueError(policy)
    n = df.groupby([TC, SC])[Y].transform("size").to_numpy(float)
    w = 1 / n
    return w / w.mean()


def solve(x, y, w):
    good = w > 0
    xw = x[good] * np.sqrt(w[good, None])
    yw = y[good] * np.sqrt(w[good])
    beta, _, rank, singular = np.linalg.lstsq(xw, yw, rcond=1e-12)
    condition = float(singular[0] / singular[-1]) if rank == x.shape[1] else None
    return beta, int(rank), condition


def fit(df, model, policy="row_ols"):
    x, y = features(model, df[T], df[S]), df[Y].to_numpy(float)
    w = weights(df, policy)
    beta, rank, condition = solve(x, y, w)
    result = {"beta": beta, "rank": rank, "condition": condition, "p": x.shape[1], "n": len(y)}
    if rank < x.shape[1]:
        return result
    residual = y - x @ beta
    bread = np.linalg.inv(x.T @ (w[:, None] * x))
    leverage = np.sum((x @ bread) * x, axis=1) * w
    # HC3: working independent-test calculation, not a material/event CI.
    adjusted = w * residual / np.maximum(1 - leverage, 1e-12)
    score = x * adjusted[:, None]
    hc3 = bread @ (score.T @ score) @ bread
    labels = df["specimen_label"].to_numpy()
    unique = np.unique(labels)
    blocks = np.array([(x[labels == label] * (w[labels == label] * residual[labels ==
    ↪  label])[:, None]).sum(axis=0) for label in unique])
    factor = len(unique) / (len(unique) - 1) * (len(y) - 1) / (len(y) - x.shape[1])
    cr1 = factor * bread @ (blocks.T @ blocks) @ bread
    result.update({
        "residual": residual, "bread": bread, "hc3": hc3, "label_cr1": cr1,
```

```
        "max_training_leverage": float(leverage.max()), "clusters": len(unique),
    })
    return result


def metrics(df, predicted):
    residual = np.asarray(predicted) - df[Y].to_numpy(float)
    error = df[[TC, SC]].copy()
    error["squared"] = residual**2
    error["absolute"] = np.abs(residual)
    error["bias"] = residual
    cells = error.groupby([TC, SC])[["squared", "absolute", "bias"]].mean()
    return {
        "n": len(df), "rmse_kpa": float(np.sqrt(np.mean(residual**2))),
        "mae_kpa": float(np.mean(np.abs(residual))), "bias_kpa": float(np.mean(residual)),
        "cell_macro_rmse_kpa": float(np.sqrt(cells["squared"].mean())),
        "cell_macro_mae_kpa": float(cells["absolute"].mean()),
        "max_absolute_error_kpa": float(np.max(np.abs(residual))),
    }


def physical_diagnostics(df, model, beta):
    ts = np.linspace(df[TC].min(), df[TC].max(), 81)
    ss = np.linspace(df[SC].min(), df[SC].max(), 81)
    tt, sn = np.meshgrid(ts, ss, indexing="ij")
    tau = features(model, tt, sn) @ beta
    dt = (features(model, tt + 1e-4, sn) @ beta - features(model, tt - 1e-4, sn) @ beta) / 2e-4
    ds = (features(model, tt, sn + .001) @ beta - features(model, tt, sn - .001) @ beta) / .002
    # c(T) is a zero-stress extrapolated intercept, not an observed measurement.
    cohesion = features(model, ts, np.zeros_like(ts)) @ beta
    return {
        "diagnostic_grid": "81x81 nominal rectangular domain; not additional observations",
        "negative_strength_grid_points": int((tau < -1e-8).sum()),
        "negative_cohesion_intercept_points": int((cohesion < -1e-8).sum()),
        "warming_strength_increase_grid_points": int((dt > 1e-8).sum()),
        "negative_stress_derivative_grid_points": int((ds < -1e-8).sum()),
        "min_predicted_strength_kpa": float(tau.min()),
        "min_cohesion_intercept_kpa": float(cohesion.min()),
        "min_d_tau_d_sigma": float(ds.min()), "max_d_tau_d_T_kpa_per_c": float(dt.max()),
    }


def bootstrap(df, model, scheme, rng):
    x, y = features(model, df[T], df[S]), df[Y].to_numpy(float)
    labels = df["specimen_label"].to_numpy()
    distinct = np.unique(labels)
    cells = list(df.groupby([TC, SC], sort=True).indices.values())
    draws, failed = [], 0
    for _ in range(BOOTSTRAPS):
        if scheme == "within_cell_independent_tests":
            indices = np.concatenate([rng.choice(i, len(i), replace=True) for i in cells])
            xb, yb = x[indices], y[indices]
            beta, rank, _ = solve(xb, yb, np.ones(len(indices)))
        elif scheme == "specimen_label_blocks_assumed":
            chosen = Counter(rng.choice(distinct, len(distinct), replace=True))
            w = np.array([chosen.get(label, 0) for label in labels], dtype=float)
            beta, rank, _ = solve(x, y, w)
        else:
            raise ValueError(scheme)
        if rank < x.shape[1] or not np.isfinite(beta).all():
            failed += 1
        else:
            draws.append(beta)
```

```
    return np.asarray(draws), failed


def run():
    started = time.monotonic()
    OUT.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(DATA)
    assert len(df) == 141 and df["observation_id"].is_unique
    assert np.isfinite(df[[T, S, TC, SC, Y]].to_numpy(float)).all()
    sets = data_domains(df)
    assert {key: len(value) for key, value in sets.items()} == {
        "low_full": 121, "low_eq6_temperature": 107, "warm_low": 63, "warm_all": 83,
    }
    config = {
        "models": list(MODELS), "fixed_reference": "author_eq6",
        "basis": "t=T_degC/5, s=sigma_kPa/400; tau_kPa",
        "models_and_terms": {
            "mc_linear": ["1", "t", "s", "t*s"],
            "mc_temperature_quadratic": ["1", "t", "s", "t*s", "t^2", "t^2*s"],
            "mc_stress_quadratic": ["1", "t", "s", "t*s", "s^2", "t*s^2"],
            "mc_stress_hinge_400": ["1", "t", "s", "t*s", "max(s-1,0)", "t*max(s-1,0)"],
        },
        "subset_and_holdout_groups": "nominal experimental classes; measured continuous predictors",
        "design": "four predeclared domains, two weights, complete temperature/stress/specimen-label holdouts",
        "fit": "weighted least squares, rcond1e-12; no penalty/model selection/clipping",
        "bootstrap_repetitions": BOOTSTRAPS, "seed": SEED,
        "bootstrap": ["within_cell_independent_tests", "specimen_label_blocks_assumed"],
        "fixed_reference_cv": "Not included in CV rankings: original article reused these data",
        "independent_local_events": 0,
        "protocol_sha256": sha(LAB / "PROTOCOL.md"),
        "protocol_amendment_sha256": sha(LAB / "PROTOCOL_AMENDMENT.md"),
        "data_sha256": sha(DATA), "code_sha256": sha(__file__),
        "environment": {"python": platform.python_version(), "executable": sys.executable, "numpy": np.__version__, "pandas": pd.__version__},
    }
    # This writes the frozen recipe before any fits in this run.
    save_json(OUT / "run_config.json", config)
    fits, residual_rows, coefficient_rows, covariance_rows, domain_rows = [], [], [], [], []
    cv_predictions, folds, cv_aggregate, boot_summary, boot_curves, draw_store = [], [], [], [], [], {}
    rng = np.random.default_rng(SEED)
    for domain, sub in sets.items():
        sub = sub.reset_index(drop=True)
        domain_rows.append({
            "domain": domain, "n": len(sub), "temperature_classes": sorted(sub[TC].unique().tolist()),
            "stress_classes": sorted(sub[SC].unique().tolist()), "specimen_labels": sorted(sub["specimen_label"].unique().tolist()),
            "measured_temperature_min_c": float(sub[T].min()), "measured_temperature_max_c": float(sub[T].max()),
            "measured_normal_stress_min_kpa": float(sub[S].min()), "measured_normal_stress_max_kpa": float(sub[S].max()),
            "actual_outside_original_eq6_strict_continuous_domain": int((~sub[T].between(-8, -.5) | ~sub[S].between(100, 400)).sum()),
            "actual_outside_eq6_with_explicit_temperature_tolerance_0p1c": int((~sub[T].between(-8.1, -.4) | ~sub[S].between(100, 400)).sum()),
            "single_test_cells": int((sub.groupby([TC, SC]).size() == 1).sum()),
            "test_cells": int(sub.groupby([TC, SC]).ngroups),
        })
```

```
        author_pred = 53.3 - 73.5 * sub[T].to_numpy() + sub[S].to_numpy() * (.42 - .21 *
        ↪  sub[T].to_numpy())
        fits.append({"domain": domain, "model": "author_eq6", "weight":
        ↪  "fixed_published_reference", "parameter_count": 0,
                     "role": "retrospective_published_reference_not_independent_validation",
                     ↪  **metrics(sub, author_pred)})
        for idx, (_, row) in enumerate(sub.iterrows()):
            residual_rows.append({"domain": domain, "model": "author_eq6", "weight":
            ↪  "fixed_published_reference", "observation_id": row.observation_id,
                                  TC: row[TC], SC: row[SC], T: row[T], S: row[S],
                                  ↪  "observed_kpa": row[Y], "predicted_kpa":
                                  ↪  author_pred[idx], "residual_pred_minus_observed_kpa":
                                  ↪  author_pred[idx] - row[Y]})
        for model in MODELS:
            for policy in WEIGHTS:
                fitted = fit(sub, model, policy)
                assert fitted["rank"] == fitted["p"], (domain, model, policy)
                beta = fitted["beta"]
                pred = features(model, sub[T], sub[S]) @ beta
                fits.append({"domain": domain, "model": model, "weight": policy,
                ↪  "parameter_count": fitted["p"], "rank": fitted["rank"],
                            "condition_number_scaled_design": fitted["condition"],
                            ↪  "max_training_leverage": fitted["max_training_leverage"],
                            "role": "external_lab_calibration", **metrics(sub, pred),
                            ↪  **physical_diagnostics(sub, model, beta)})
                for k, value in enumerate(beta):
                    coefficient_rows.append({"domain": domain, "model": model, "weight":
                    ↪  policy, "term": config["models_and_terms"][model][k],
                    ↪  "coefficient_kpa": value,
                                             "hc3_independence_se_kpa": np.sqrt(max(0,
                                             ↪  fitted["hc3"][k, k])),
                                             "assumed_label_cluster_cr1_se_kpa": np.sqrt(max(0,
                                             ↪  fitted["label_cr1"][k, k])), "label_clusters":
                                             ↪  fitted["clusters"]})
                for covariance_name in ("hc3", "label_cr1"):
                    covariance_rows.append({"domain": domain, "model": model, "weight": policy,
                    ↪  "method": covariance_name,
                                            "matrix_kpa_squared":
                                            ↪  fitted[covariance_name].tolist(),
                                            ↪  "conditional_assumptions": "See
                                            ↪  PROTOCOL_AMENDMENT.md; labels not verified
                                            ↪  physical UIDs"})
                for idx, (_, row) in enumerate(sub.iterrows()):
                    residual_rows.append({"domain": domain, "model": model, "weight": policy,
                    ↪  "observation_id": row.observation_id,
                                          TC: row[TC], SC: row[SC], T: row[T], S: row[S],
                                          ↪  "observed_kpa": row[Y], "predicted_kpa":
                                          ↪  pred[idx], "residual_pred_minus_observed_kpa":
                                          ↪  pred[idx] - row[Y]})
                for group_column, scheme in ((TC, "temperature"), (SC, "normal_stress"),
                ↪  ("specimen_label", "specimen_label_assumed")):
                    scheme_rows = []
                    for group in sorted(sub[group_column].unique()):
                        test_mask = sub[group_column] == group
                        train, test = sub.loc[~test_mask], sub.loc[test_mask]
                        trained = fit(train, model, policy)
                        identity = {"domain": domain, "model": model, "weight": policy,
                        ↪  "holdout_scheme": scheme, "heldout_group": str(group)}
                        fold = {
                            **identity, "training_n": len(train), "test_n": len(test), "rank":
                            ↪  trained["rank"], "parameter_count": trained["p"],
                            "condition_number_scaled_design": trained["condition"],
```

```
                        "train_stress_classes":
                        ↪  json.dumps(sorted(train[SC].unique().tolist())),
                        "train_temperature_classes":
                        ↪  json.dumps(sorted(train[TC].unique().tolist())),
                        "train_measured_sigma_min_kpa": train[S].min(),
                        ↪  "train_measured_sigma_max_kpa": train[S].max(),
                        "train_measured_T_min_c": train[T].min(), "train_measured_T_max_c":
                        ↪  train[T].max(),
                        "test_ids_sha256": hashlib.sha256("\n".join(test.observation_id).e⌋
                        ↪  ncode()).hexdigest(),
                        "status": "full_rank" if trained["rank"] == trained["p"] else
                        ↪  "rank_deficient_no_prediction",
                        "high_stress_design_support_absent": bool(train[SC].max() <= 400
                        ↪  and test[SC].max() > 400),
                        "test_group_is_nominal_edge": bool(group_column != "specimen_label"
                        ↪  and group in (sub[group_column].min(),
                        ↪  sub[group_column].max())),
                    }
                    if trained["rank"] != trained["p"]:
                        folds.append(fold)
                        continue
                    xt = features(model, test[T], test[S])
                    predicted = xt @ trained["beta"]
                    leverage = np.sum((xt @ trained["bread"]) * xt, axis=1)
                    se = np.sqrt(np.maximum(np.einsum("ij,jk,ik->i", xt,
                    ↪  trained["label_cr1"], xt), 0))
                    fold.update({**metrics(test, predicted), "max_test_design_leverage":
                    ↪  float(leverage.max()),
                                 "max_conditional_mean_label_cr1_se_kpa": float(se.max()),
                                 "max_training_leverage":
                                 ↪  trained["max_training_leverage"]})
                    folds.append(fold)
                    for idx, (_, row) in enumerate(test.iterrows()):
                        item = {
                            **identity, "observation_id": row.observation_id,
                            ↪  "specimen_label": row.specimen_label,
                            TC: row[TC], SC: row[SC], T: row[T], S: row[S], "observed_kpa":
                            ↪  row[Y],
                            "predicted_kpa": predicted[idx],
                            ↪  "residual_pred_minus_observed_kpa": predicted[idx] -
                            ↪  row[Y],
                            "design_leverage": leverage[idx],
                            ↪  "assumed_label_cr1_conditional_mean_se_kpa": se[idx],
                            "outside_training_measured_sigma_range": bool(row[S] <
                            ↪  train[S].min() or row[S] > train[S].max()),
                            "outside_training_measured_temperature_range": bool(row[T] <
                            ↪  train[T].min() or row[T] > train[T].max()),
                        }
                        scheme_rows.append(item)
                        cv_predictions.append(item)
                group_folds = [r for r in folds if all(r.get(k) == v for k, v in {"domain":
                ↪  domain, "model": model, "weight": policy, "holdout_scheme":
                ↪  scheme}.items())]
                if scheme_rows:
                    pframe = pd.DataFrame(scheme_rows)
                    pseudo = pframe.rename(columns={"observed_kpa": Y})
                    usable_folds = [r for r in group_folds if r["status"] == "full_rank"]
                    cv_aggregate.append({
                        "domain": domain, "model": model, "weight": policy,
                        ↪  "holdout_scheme": scheme,
                        **metrics(pseudo, pframe.predicted_kpa), "folds": len(group_folds),
                        ↪  "usable_folds": len(usable_folds),
```

```
                        "missing_prediction_count": len(sub) - len(pframe),
                        "group_macro_rmse_kpa": float(np.sqrt(np.mean([r["rmse_kpa"]**2 for r in usable_folds]))),
                        "group_macro_mae_kpa": float(np.mean([r["mae_kpa"] for r in usable_folds])),
                    })
            # Bootstrap primary row-OLS fits only; weighted fit is a separate sensitivity.
            ts = np.linspace(sub[TC].min(), sub[TC].max(), 81)
            stresses = sorted(sub[SC].unique())
            grid_t, grid_s = np.meshgrid(ts, stresses, indexing="ij")
            gridx = features(model, grid_t.ravel(), grid_s.ravel())
            primary = fit(sub, model, "row_ols")
            for scheme in ("within_cell_independent_tests", "specimen_label_blocks_assumed"):
                draws, failed = bootstrap(sub, model, scheme, rng)
                assert len(draws) > 0
                key = domain + "__" + model + "__" + scheme
                draw_store[key] = draws
                q = np.quantile(draws, [.025, .5, .975], axis=0)
                boot_summary.append({"domain": domain, "model": model, "scheme": scheme, "attempted_draws": BOOTSTRAPS,
                                     "usable_draws": len(draws), "rank_deficient_or_nonfinite_draws": failed,
                                     "actual_label_count": sub.specimen_label.nunique(), "physical_UID_known": False,
                                     "coefficient_percentiles_2p5_50_97p5_kpa": q.tolist(),
                                     "interpretation": "conditional percentile sensitivity; not guaranteed 95% coverage or local event probability"})
                predictions = draws @ gridx.T
                curves = np.quantile(predictions, [.025, .5, .975], axis=0)
                hc3_se = np.sqrt(np.maximum(np.einsum("ij,jk,ik->i", gridx, primary["hc3"], gridx), 0))
                cr1_se = np.sqrt(np.maximum(np.einsum("ij,jk,ik->i", gridx, primary["label_cr1"], gridx), 0))
                primary_pred = gridx @ primary["beta"]
                hc3_q = student_t.ppf(.975, len(sub) - primary["p"])
                cr1_q = student_t.ppf(.975, primary["clusters"] - 1)
                for i in range(len(gridx)):
                    boot_curves.append({"domain": domain, "model": model, "scheme": scheme, "temperature_c": grid_t.ravel()[i], "normal_stress_kpa": grid_s.ravel()[i],
                                        "primary_mean_kpa": primary_pred[i], "percentile_2p5_mean_kpa": curves[0, i], "percentile_50_mean_kpa": curves[1, i], "percentile_97p5_mean_kpa": curves[2, i],
                                        "hc3_independence_mean_low_kpa": primary_pred[i] - hc3_q * hc3_se[i], "hc3_independence_mean_high_kpa": primary_pred[i] + hc3_q * hc3_se[i],
                                        "assumed_label_cr1_mean_low_kpa": primary_pred[i] - cr1_q * cr1_se[i], "assumed_label_cr1_mean_high_kpa": primary_pred[i] + cr1_q * cr1_se[i]})
    write_csv("fits.csv", fits)
    write_csv("coefficients.csv", coefficient_rows)
    write_csv("calibration_residuals.csv", residual_rows)
    write_csv("holdout_folds.csv", folds)
    write_csv("holdout_predictions.csv", cv_predictions)
    write_csv("holdout_summary.csv", cv_aggregate)
    write_csv("uncertainty_curves.csv", boot_curves)
    save_json(OUT / "covariances.json", covariance_rows)
    save_json(OUT / "domains.json", domain_rows)
    save_json(OUT / "bootstrap_summary.json", boot_summary)
```

```
    np.savez_compressed(OUT / "bootstrap_coefficient_draws.npz", **draw_store)
    audit = {
        "complete": True, "domains": {k: len(v) for k, v in sets.items()},
        "new_fits": len([r for r in fits if r["model"] != "author_eq6"]),
        "fixed_author_references": 4, "holdout_fits": len(folds),
        "holdout_predictions": len(cv_predictions), "bootstrap_attempts": BOOTSTRAPS *
        ↪  len(boot_summary),
        "bootstrap_usable": int(sum(r["usable_draws"] for r in boot_summary)),
        "bootstrap_failed_rank_or_nonfinite": int(sum(r["rank_deficient_or_nonfinite_draws"]
        ↪  for r in boot_summary)),
        "models_not_automatically_selected": True, "local_models_modified": False,
        ↪  "independent_local_events": 0,
        "elapsed_seconds": time.monotonic() - started,
        "scope": "External laboratory experiments; grouping and covariance sensitivity do not
        ↪  establish local applicability",
    }
    save_json(OUT / "run_summary.json", audit)
    print(json.dumps(audit, ensure_ascii=False))


if __name__ == "__main__":
    run()
```

程序 12：本章节三幅矢量图重绘

来源：track_b/paper/scripts/q2_make_figures.py
SHA256: 9c2ee22d79d5d5a289491266b569e4e882c76b5c2312bf1c0c0c943998fa8e34

```
"""Redraw three compact Chinese publication figures from frozen Q2 results.

No model execution, fitting, new scenarios or research-packet writes. PDF and
300dpi PNG outputs plus provenance/support JSON are confined to figures/q2.
"""
from __future__ import annotations

import hashlib
import json
import os
from pathlib import Path
import shutil
import sys
import tempfile

sys.dont_write_bytecode = True
ROOT = Path(__file__).resolve().parents[3]
OUT = ROOT / "track_b/paper/figures/q2"
OUT.mkdir(parents=True, exist_ok=True)
# The plotting cache is temporary and remains inside the owned figure folder.
_cache = tempfile.TemporaryDirectory(prefix=".q2_mpl_", dir=OUT)
os.environ["MPLCONFIGDIR"] = _cache.name
for _prior_cache in [Path.home()/'.cache/matplotlib',Path.home()/'.matplotlib']:
    for _cache_file in _prior_cache.glob('fontlist-v*.json'):
        shutil.copy2(_cache_file,Path(_cache.name)/_cache_file.name)

import matplotlib
matplotlib.use("Agg")
from matplotlib import colors, font_manager, ft2font
from matplotlib.cm import ScalarMappable
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Rectangle
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

RESEARCH = ROOT / "track_b/research/q2/round3"
INPUTS = {
    "mechanism_comparison": RESEARCH / "results/analytic_thermal/model_comparison.csv",
    "critical_head": RESEARCH / "results/critical_relations.csv",
    "joint_sensitivity": RESEARCH / "results/analytic_thermal/joint_sensitivity.csv",
    "laboratory_cv": RESEARCH / "lab_strength/model/results/holdout_summary.csv",
}
FONT = Path("<LOCAL_HOME>/Library/Fonts/PingFang Regular_0.ttf")
COLORS = {"linear_extension": "#325B7B", "thawed_rock_contact": "#B8753B",
          "temperature": "#325B7B", "normal_stress": "#B8753B"}
MECHANISMS = ["S", "S_T", "S_H", "S_T_H"]
MECHANISM_LABELS = [" 结构 S", "S+T", "S+H", "S+T+H"]
REPS = [0,36,72,109,145,182,218,255]
PARAMETER_LABELS = {
    "theta_deg":" 结构面倾角", "initial_temp_c":" 初始界面温度",
    "initial_damage":" 初始损伤", "station_source_offset_c":" 站点—源区温差",
    "thickness_normal_m":" 法向厚度", "interface_depth_m":" 界面传热深度",
    "bridge_fraction":" 岩桥面积比例", "rock_volume_fraction":" 岩石体积分数",
    "bridge_cohesion_kpa":" 岩桥黏聚力", "rock_mu":" 岩面摩擦系数",
    "melt_mm_deg_day":" 融水系数", "storativity":" 储水系数",
    "drainage_hours":" 排水时间", "diffusivity_m2_s":" 热扩散率",
```

```
    "damage_exponent":" 损伤指数", "rain_multiplier":" 降水映射系数",
    "damage_rate_h_inv":" 损伤速率", "connected_depth_m":" 连通水头上限",
    "drainage_damage_coupling":" 损伤—排水耦合",
}


def digest(path, role):
    return {"path":path.relative_to(ROOT).as_posix(),"bytes":path.stat().st_size,
            "sha256":hashlib.sha256(path.read_bytes()).hexdigest(),"role":role}


def configure():
    font_manager.fontManager.addfont(str(FONT))
    # A unique in-memory alias prevents the same family name resolving to the
    # macOS CFF collection, which can produce mismatched CIDFontType2 embedding.
    family='Q2-PingFang-Regular-Static'
    font_manager.fontManager.ttflist.append(font_manager.FontEntry(
        fname=str(FONT),name=family,style='normal',variant='normal',weight='normal',stretch='n
        ↪ ormal'))
    face=ft2font.FT2Font(str(FONT))
    assert all(face.get_char_index(ord(ch)) for ch in " 结构温度水损伤裕阈值")
    matplotlib.rcParams.update({"font.family":family,"font.size":7.5,
        "axes.labelsize":7.5,"axes.titlesize":8.0,"xtick.labelsize":7.0,"ytick.labelsize":7.0,
        "axes.linewidth":.55,"xtick.major.width":.5,"ytick.major.width":.5,
        "xtick.major.size":2.5,"ytick.major.size":2.5,"xtick.major.pad":2.,"ytick.major.pad":2
        ↪ .,
        "axes.unicode_minus":False,"pdf.fonttype":42,"ps.fonttype":42,
        "savefig.dpi":300,"axes.spines.top":False,"axes.spines.right":False,
        "text.color":"#222222","axes.labelcolor":"#222222","axes.edgecolor":"#555555",
        "grid.color":"#DDDDDD","grid.linewidth":.4,"mathtext.fontset":"dejavusans"})
    return family


def save(fig, stem, title):
    fig.savefig(OUT/f"{stem}.pdf",metadata={"Title":title,"Creator":"q2_make_figures.py; frozen
    ↪ analytical results",
        "CreationDate":None,"ModDate":None},dpi=300)
    fig.savefig(OUT/f"{stem}.png",dpi=300)
    plt.close(fig)
    return [digest(OUT/f"{stem}.{ext}","publication_figure" if ext=="pdf" else
    ↪ "300dpi_visual_review") for ext in ["pdf","png"]]


def mechanism_figure(data):
    table=data.copy()
    assert len(table)==8 and (table.n_parameter_sets==256).all()
    assert (table.interval_crossing_n+table.left_censored_n+table.never_crossed_n+table.missin
    ↪ g_n==256).all()
    assert (table.left_censored_n==31).all() and (table.missing_n==0).all()
    rows=table[table.law_policy=="linear_extension"].set_index("mechanism_id")
    assert abs(rows.loc['S_T','median_min_margin']-.48529976742120917)<1e-12
    assert abs(rows.loc['S_T_H','median_min_margin']-.4717932586129435)<1e-12
    fig=plt.figure(figsize=(17/2.54,4.5/2.54))
    a=fig.add_axes([.065,.20,.375,.53]);b=fig.add_axes([.57,.20,.395,.53])
    x=np.arange(4);width=.28
    handles=[]
    for k,policy in enumerate(["linear_extension","thawed_rock_contact"]):
        q=table[table.law_policy==policy].set_index('mechanism_id').loc[MECHANISMS]
        xx=x+(k-.5)*width
        bars=a.bar(xx,q.interval_crossing_n,width=width*.9,color=COLORS[policy],alpha=.95)
        for bar,value in zip(bars,q.interval_crossing_n):
            a.text(bar.get_x()+bar.get_width()/2,value+.7,str(int(value)),ha='center',va='bott
            ↪ om',fontsize=7.0)
```

```
        median=q.median_min_margin.to_numpy()
        low=median-q.p05_min_margin.to_numpy();high=q.p95_min_margin.to_numpy()-median
        b.errorbar(x+(k-.5)*.08,median,yerr=np.vstack([low,high]),fmt='o' if k==0 else 's',
            ms=3.5 if k==0 else 2.8,elinewidth=.65,capsize=2,color=COLORS[policy],
            markerfacecolor=COLORS[policy] if k==0 else 'white',markeredgewidth=.75,zorder=2+k)
        handles.append(Patch(facecolor=COLORS[policy],label='线性冻结关系' if k==0 else '暖端岩接触对照'))
    a.set(xticks=x,xticklabels=MECHANISM_LABELS,ylim=(0,37),yticks=[0,10,20,30],ylabel='后续越阈数 / 256')
    a.set_title('(a) 共同参数下的四机制竞争',loc='left',pad=7)
    b.axhline(0,color='#666666',ls='--',lw=.65,zorder=1)
    b.set(xticks=x,xticklabels=MECHANISM_LABELS,ylim=(-.66,3.1),yticks=[-.5,0,1,2,3],ylabel='评估期最小稳定裕度')
    b.set_title('(b) 中位数与设计分位范围',loc='left',pad=7)
    for j,val in enumerate(rows.loc[MECHANISMS].median_min_margin):
        b.text(j,val+.18,f'{val:.3f}',ha='center',va='bottom',fontsize=6.5,color=COLORS['linear_extension'])
    a.grid(axis='y',alpha=.65);a.set_axisbelow(True);b.grid(axis='y',alpha=.5);b.set_axisbelow(True)
    fig.legend(handles=handles,loc='upper
    center',bbox_to_anchor=(.53,1.0),ncol=2,frameon=False,
        handlelength=1.3,columnspacing=2.3,fontsize=7.4)
    outputs=save(fig,'q2_mechanism_comparison','Q2 四机制共同设计的稳定性比较')
    return dict(figure_id='q2-mechanism-comparison',outputs=outputs,size_cm=[17,4.5],
        input_keys=['mechanism_comparison'],selection='All8 rows of final analytic thermal
        comparison,4 mechanisms x2 policies; no scenario filtering',
        plotted_records=table.to_dict('records'),scientific_layer='conditional_scenario',
        caption_note='解析热参考；每个机制与策略均有256个共同参数设计点，
        初始已不稳定31个保留为左删失。误差线为5%–95%设计分位，非事件概率或置信区间。')


def critical_sensitivity_figure(critical,sensitivity):
    crit=critical[critical.temperature_c.isin([-6.,-3.,-1.,-.5])].copy()
    pids=[f'q2r3-p{i:04d}' for i in REPS]
    assert len(crit)==32 and set(crit.parameter_set_id)==set(pids)
    sens=sensitivity[(sensitivity.mechanism_id=='S_T_H')&(sensitivity.law_policy=='linear_extension')].copy()
    sens['abs_rho']=sens.spearman_rho.abs()
    sens=sens.sort_values(['abs_rho','parameter'],ascending=[False,True]).head(8)
    assert len(sens)==8 and (sens.n_joint_design==256).all()
    fig=plt.figure(figsize=(17/2.54,5.4/2.54))
    a=fig.add_axes([.085,.215,.37,.64]);cbax=fig.add_axes([.47,.215,.012,.64]);b=fig.add_axes([.715,.215,.255,.64])
    cmap=matplotlib.colormaps['YlGnBu'];norm=colors.Normalize(0,60)
    temperatures=[-6.,-3.,-1.,-.5]
    for i,pid in enumerate(pids):
        for j,t in enumerate(temperatures):
            row=crit[(crit.parameter_set_id==pid)&(crit.temperature_c==t)].iloc[0]
            status=row.root_status
            if status=='positive_pressure_root':
                val=float(row.critical_head_m);fill=cmap(norm(val));hatch=None
                label=f'{val:.1f}'+('＊' if not row.root_reachable_before_capacity else '')
                color='white' if val>34 else '#222222'
            elif status=='dry_already_critical':
                fill='#E6E6E6';hatch=None;label='干态越阈';color='#333333'
            elif status=='no_pressure_only_root':
                fill='white';hatch='///';label='无水压根';color='#333333'
            else:raise ValueError(status)
            a.add_patch(Rectangle((j-.5,i-.5),1,1,facecolor=fill,edgecolor='#CCCCCC',lw=.4,hatch=hatch))
            a.text(j,i,label,ha='center',va='center',fontsize=6.5,color=color,
                bbox=dict(facecolor=fill,edgecolor='none',pad=.1) if hatch else None)
    a.set(xlim=(-.5,3.5),ylim=(7.5,-.5),xticks=range(4),xticklabels=['-6','-3','-1','-0.5'],
```

```
          yticks=range(8),yticklabels=[f'p{i:04d}' for i in REPS],xlabel='界面温度 / °C')
    a.tick_params(length=0);a.set_title('(a) 共同初态的临界水头',loc='left',pad=7)
    for spine in a.spines.values():spine.set_visible(False)
    cb=fig.colorbar(ScalarMappable(norm=norm,cmap=cmap),cax=cbax,ticks=[0,20,40,60])
    cb.set_label('临界水头 / m',fontsize=7.0,labelpad=2)
    if cb.solids is not None:cb.solids.set_rasterized(False)
    y=np.arange(len(sens));v=sens.spearman_rho.to_numpy()
    b.barh(y,v,color=[COLORS['linear_extension'] if z<0 else COLORS['thawed_rock_contact'] for z in v],height=.58)
    b.axvline(0,color='#555555',lw=.6)
    b.set(yticks=y,yticklabels=[PARAMETER_LABELS[k] for k in sens.parameter],ylim=(7.7,-.7),
          xlim=(-.66,.43),xticks=[-.6,-.3,0,.3],xlabel=r'Spearman $\rho_s$')
    b.tick_params(axis='y',length=0,pad=3,labelsize=6.9)
    b.grid(axis='x',alpha=.5);b.set_axisbelow(True)
    b.set_title('(b) 耦合模型联合敏感性',loc='left',pad=7)
    for yy,value in zip(y,v):
        b.text(value+.014,yy,f'{value:.2f}',ha='left',va='center',fontsize=6.0,
               color='white' if value<0 else '#222222')
    outputs=save(fig,'q2_critical_head_sensitivity','Q2 临界水头与联合敏感性')
    # NaN encodes no root in the source; serialize as null, never as zero.
    plotted=json.loads(crit.to_json(orient='records'))
    return dict(figure_id='q2-critical-head-sensitivity',outputs=outputs,size_cm=[17,5.4],
        input_keys=['critical_head','joint_sensitivity'],scientific_layer='conditional_scenario',
        selection='Predeclared8 representative parameter indices;4 registered temperature slices within published thermal range; STH linear-policy8largest absolute existing Spearman correlations',
        representative_indices=REPS,critical_records=plotted,sensitivity_records=sens.drop(columns='abs_rho').to_dict('records'),
        root_handling='Negative algebraic roots shown as dry_already_critical, source null roots as no_pressure_only_root, unreachable positive roots retain capacity marker; no root converted to zero',
        caption_note='左图为8个预定代表、给定几何和初始损伤下的条件静力临界水头；
        ＊表示正根超过给定水头容量，灰底为干态已越阈，斜线为无水压单独根。
        温度范围筛选不代表有效应力、材料和速率均在实验域内。
        右图为S+T+H线性分支按绝对Spearman相关排序的前8项设计关联，非因果贡献。')


def lab_cv_figure(table):
    order=['mc_linear','mc_temperature_quadratic','mc_stress_quadratic','mc_stress_hinge_400']
    names=['线性温度 MC','温度二次','应力二次','400 kPa 铰链']
    q=table[(table.domain=='warm_all')&(table.weight=='row_ols')&
            table.holdout_scheme.isin(['temperature','normal_stress'])].copy()
    assert len(q)==8 and (q.n==83).all() and (q.missing_prediction_count==0).all()
    fig=plt.figure(figsize=(17/2.54,4.2/2.54));a=fig.add_axes([.18,.27,.77,.55])
    y=np.arange(4);values={}
    for k,scheme in enumerate(['temperature','normal_stress']):
        d=q[q.holdout_scheme==scheme].set_index('model').loc[order]
        vals=d.group_macro_rmse_kpa.to_numpy();values[scheme]=vals
        a.scatter(vals,y+(k-.5)*.23,color=COLORS[scheme],marker='o' if k==0 else 's',s=17,zorder=3,
            label='整温层留出（5 组）' if k==0 else '整应力类留出（4 组）')
        for yy,val in zip(y+(k-.5)*.23,vals):
            a.text(val+12,yy+(-.15 if k==0 else .15),f'{val:.1f}',va='center',ha='left',fontsize=6.5,color=COLORS[scheme])
    for i in range(4):a.plot([values['temperature'][i],values['normal_stress'][i]],[i-.115,i+.115],color='#BBBBBB',lw=.6,zorder=1)
    a.set(yticks=y,yticklabels=names,ylim=(3.55,-.65),xlim=(0,1170),xticks=[0,200,400,600,800,1000],
          xlabel='留出组等权宏平均 RMSE / kPa')
    a.tick_params(axis='y',length=0,pad=5);a.grid(axis='x',alpha=.6);a.set_axisbelow(True)
    a.spines['left'].set_visible(False)
```

```
    fig.legend(*a.get_legend_handles_labels(),loc='upper center',bbox_to_anchor=(.58,1.0),ncol=2,frameon=False,
        handlelength=1.,columnspacing=2.,fontsize=7.2)
    outputs=save(fig,'q2_lab_grouped_validation','Q2 外部暖温层试验的分组留出比较')
    return dict(figure_id='q2-lab-grouped-validation',outputs=outputs,size_cm=[17,4.2],
        input_keys=['laboratory_cv'],scientific_layer='external_transfer',
        selection='Fixed warm_all domain83tests, row_ols weight, all4candidate model IDs and temperature/normal-stress holdout schemes; group_macro_rmse_kpa, not pooled row RMSE',
        model_order=order,plotted_records=q.to_dict('records'),
        caption_note='外部暖温层的83次试验，名义温度-4至-0.5°C、应力100–800kPa；各模型均为行等权拟合。显示按留出组等权的宏平均RMSE，名义类用于分组、连续实测值用于拟合。外部实验评价不并入本地区独立事件分母。')


def main():
    assert Path.cwd().resolve()==ROOT
    family=configure()
    before={key:digest(path,key) for key,path in INPUTS.items()}
    frames={key:pd.read_csv(path) for key,path in INPUTS.items()}
    figures=[mechanism_figure(frames['mechanism_comparison']),
             critical_sensitivity_figure(frames['critical_head'],frames['joint_sensitivity']),
             lab_cv_figure(frames['laboratory_cv'])]
    for key,path in INPUTS.items():assert digest(path,key)==before[key]
    from pypdf import PdfReader
    pdf_checks=[]
    for fig in figures:
        p=ROOT/fig['outputs'][0]['path'];reader=PdfReader(p)
        assert len(reader.pages)==1
        page=reader.pages[0]
        width,height=float(page.mediabox.width)*2.54/72,float(page.mediabox.height)*2.54/72
        np.testing.assert_allclose([width,height],fig['size_cm'],atol=1e-6)
        fonts=[]
        for ref in page['/Resources']['/Font'].get_object().values():
            font=ref.get_object();embedded=False
            descendants=font.get('/DescendantFonts',[])
            for descendant in descendants:
                descriptor=descendant.get_object().get('/FontDescriptor')
                if descriptor:
                    d=descriptor.get_object();embedded=any(k in d for k in ['/FontFile','/FontFile2','/FontFile3'])
            fonts.append(dict(base_font=str(font.get('/BaseFont')),subtype=str(font.get('/Subtype')),embedded=embedded))
        images=[]
        for name,ref in page['/Resources'].get('/XObject',{}).get_object().items() if '/XObject' in page['/Resources'] else []:
            if ref.get_object().get('/Subtype')=='/Image':images.append(str(name))
        assert not images,'publication plot unexpectedly rasterized'
        assert all(f['embedded'] for f in fonts)
        pdf_checks.append(dict(figure_id=fig['figure_id'],one_page=True,size_cm=[width,height],embedded_fonts=fonts,raster_image_objects=images))
    resolved_font=Path(font_manager.findfont(font_manager.FontProperties(family=family,style='normal',weight='normal')))
    source=dict(schema_version='q2-paper-figures-v1',base_head='eecf07db7873b3c2c63275fa0b4198 67d8bdd3b2',
        script=digest(Path(__file__),'reproducible_publication_plotting_code'),inputs=list(before.values()),
        figures=figures,font=dict(requested_path=str(FONT),requested_sha256=hashlib.sha256(FONT.read_bytes()).hexdigest(),family=family,
            resolved_path=str(resolved_font),resolved_sha256=hashlib.sha256(resolved_font.read_bytes()).hexdigest()),
        software=dict(python=sys.version,matplotlib=matplotlib.__version__,numpy=np.__version__,pandas=pd.__version__),
```

```
263         reproduction_command='track_b/.venv/bin/python
            ↪ track_b/paper/scripts/q2_make_figures.py',
264         models_reexecuted=0,new_scenarios=0,source_inputs_unchanged=True,pdf_checks=pdf_checks,
265         visual_review_status='pending_actual_open; see q2_visual_review.json once reviewed')
266     (OUT/'q2_figure_sources.json').write_text(json.dumps(source,ensure_ascii=False,indent=2,al
        ↪ low_nan=False)+'\n')
267     print(json.dumps(dict(figures=len(figures),pdf_pages_checked=len(pdf_checks),all_sources_u
        ↪ nchanged=True,
268         files=[item['path'] for fig in figures for item in
            ↪ fig['outputs']]),ensure_ascii=False))
269     _cache.cleanup()
270
271
272 if __name__=='__main__':main()
```

程序 13：paper plotting, preview compilation or evidence assembly; no model refit

来源：track_b/paper/scripts/q3_evidence.py
SHA256：e0bc4e0998bb840164507d3496145cbb4752e0b625304ba4050876bc7ce3d0bf

```
#!/usr/bin/env python3
"""Assemble/check Q3 paper evidence from frozen results; never fit or rerun models."""
import argparse
import hashlib
import json
from pathlib import Path
import re

ROOT = Path(__file__).resolve().parents[3]
PAPER = ROOT/'track_b/paper'
EVIDENCE = PAPER/'evidence'
R = ROOT/'track_b/research/q3/round3'


def read(path):
    return json.loads(path.read_text())


def file_record(path):
    return {'path': path.relative_to(ROOT).as_posix(),
            'sha256': hashlib.sha256(path.read_bytes()).hexdigest()}


def source_records(*values):
    """Deduplicate repository-relative references found in nested audit records."""
    found = {}
    def visit(value):
        if isinstance(value, dict):
            if isinstance(value.get('path'), str) and 'sha256' in value:
                path = value['path']
                if path.startswith('track_b/'):
                    record = {'path': path, 'sha256': value['sha256']}
                    if path in found:
                        assert found[path] == record, path
                    found[path] = record
            for item in value.values():
                visit(item)
        elif isinstance(value, list):
            for item in value:
                visit(item)
    for value in values:
        visit(value)
    return sorted(found.values(), key=lambda v: v['path'])


def write(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False)+'\n')


def build():
    a, b, c = [read(EVIDENCE/f'q3_{name}_audit.json') for name in 'abc']
    figures = read(EVIDENCE/'q3_figure_data.json')
    claims = []
    def claim(identifier, statement, value, unit, sources, model, scenario, scope, anchor):
        assert sources, identifier
```

```
        claims.append({'claim_id': identifier, 'statement': statement, 'value': value,
                       'unit': unit, 'source_files': source_records(sources),
                       'model_id': model, 'scenario_id': scenario,
                       'scope': scope, 'paper_anchor': anchor})

    anchors = {'q3-a-design':'q3:sec:models', 'q3-a-fixed-bounds':'q3:eq:bounds',
               'q3-a-history-risk':'q3:eq:bounds',
               'q3-a-extrapolation-support':'q3:sec:models',
               'q3-a-weather-response':'q3:sec:models'}
    for item in a['claims']:
        copy = dict(item)
        copy['paper_anchor'] = anchors[item['claim_id']]
        copy['source_files'] = source_records(item['source_files'])
        claims.append(copy)
    claim('q3-a-clock-and-availability',
          '75 个发布时刻各含 8 条假设延迟轨、1 条理想轨、1 条严格首发轨；严格历史首发时刻未知。',
          a['design'], 'count; UTC; h', a['source_files'],
          'A_analytic_erfc_fixed_history', 'GFS_three_18Z_cycles',
          '10 是候选信息轨数；包含拒用/未知轨，非 10 条完整实际可用 GFS 轨。', 'q3:sec:models')
    claim('q3-a-firstpass-and-domain', '逐小时首次非正、线性插值、
    ↪  完整实际随访域诊断与固定历史风险信息界。',
          [x for x in a['formulas'] if x['formula_id'] in
           {'q3-a-first-crossing','q3-a-fixed-domain','q3-a-event-partition'}],
          'h; kPa; degC; dimensionless', a['formulas'],
          'A_analytic_erfc_fixed_history', 'delivered_2048_design_ensemble',
          '域外或随访不足转为未知；历史已启动分类保持。', 'q3:eq:firstpass')
    claim('q3-local-independent-initiation-labels', '本地首次机械启动的独立验证事件为 0。',
          {'independent_local_first_mechanical_validation_events': 0},
          'physical_event', a['source_files'], 'A_analytic_erfc_fixed_history', 'local_source',
          '外部终末失稳标签不替代本地首次机械启动真值。', 'q3:sec:design')
    claim('q3-b-estimator-and-uncertainty',
          'OLS/局部趋势 SS 使用发布前数据，512 次当前联合估计抽样；SS 不另模拟未来过程噪声。',
          {'formulas': b['model_formulas'], 'sampling': b['sampling_and_abstention']},
          'h; inverse_normalized_deformation_rate', b['source_files'],
          ['B_IV','B_SS'], 'causal_terminal_collapse_forecast',
          '归一逆速度5点速率/16点拟合；Weissmiess为直接给定速率；截断当前逆速度正值。',
          ↪  'q3:eq:deformation')
    claim('q3-b-fixed-cutoff-coverage',
          '在标签下界前 24 小时发布 H72，完整覆盖/相交的机会分母保留全部 13 失稳组。',
          [x for x in b['fixed_lower_minus24h_table_24_48_72'] if x['horizon_hours']==72],
          'count; h', b['source_files'], ['B_IV','B_SS'],
          'label_lower_minus24h_H72_primary_sensor',
          '点误差为中位预测到标签区间距离；其跨组中位数仅基于有限点 7/8 组。', 'q3:tab:coverage')
    claim('q3-b-two-first-alert-definitions',
          '72 小时滚动规则将首个时域内正确告警与首个任意告警分别汇总。',
          [x for x in b['rolling_lead_and_alert_table']
           if x['horizon_hours']==72 and x['evaluation_set']=='hourly'],
          'physical_group; h', b['source_files'], ['B_IV','B_SS'],
          'retrospective_last30days_hourly_p72_ge0.5',
          '13个预先选择失稳病例；提前量中位数的分母为各自有告警组，Brienz组内取最早。',
          ↪  'q3:tab:lead')
    claim('q3-b-publication-scale-sensitivity',
          '同 10 物理组内比较小时发布与最后 48 小时原生发布，改变了观测时期及发布尺度。',
          [x for x in b['paired_same10groups_table_24_48_72'] if x['horizon_hours'] in
          ↪  [48,72]],
          'dimensionless_Brier', b['source_files'], ['B_IV','B_SS'],
          'same10groups_hourly_vs_native_last48',
          '不是只改变采样频率的受控实验；不把不同有效组分数直接排名。', 'q3:sec:results')
    claim('q3-c-exposure-and-group-denominators',
          'C 使用 13 个终末失稳组、1 个真实删失组，确知生存暴露 56381.84 小时。',
          c['dataset_and_exposure'], 'physical_group; h; daily_record',
          c['dataset_and_exposure'], 'nonlinear_rbf_hazard_no_record_age_raw',
          'external_terminal_collapse_LOSO',
```

```
        'Stampa725日属1个物理过程；56526.19小时上界含144.35小时标签不确定性，不全是生存暴露。',
        ↪  'q3:sec:design')
    claim('q3-c-likelihood-and-fit',
        '无记录年龄 RBF 的 10 维基、分段指数风险及区间/删失似然在嵌套留组中拟合。',
        {'formulas':c['formula_tex_suggestions'], 'fitting_protocol': c['fitting_protocol'],
         'publication_scales':c['publication_and_result_scales']},
        'hazard_per_h; h', c['source_files'],
        'nonlinear_rbf_hazard_no_record_age_raw', 'external_terminal_collapse_LOSO',
        '预测固定最后发布前特征；记录年龄支线只作诊断；不是每小时完整 C 回放。', 'q3:eq:hazard')
    claim('q3-c-original10-known-scores',
        '原 C 锚点 RBF raw 的 48/72 小时分数用 10 个已知结局组计算。',
        [x for x in c['original_fixed_anchor_table_all_four_C_branches']
         if x.get('model_id')=='nonlinear_rbf_hazard_no_record_age_raw'],
        'dimensionless_Brier; physical_group', c['source_files'],
        'nonlinear_rbf_hazard_no_record_age_raw', 'original_C_predefined_anchors',
        '与 B/C 共同有效 8 组分数使用不同分母；原锚点回顾选取。', 'q3:sec:results')
    claim('q3-common-anchor-probability-comparison',
        'SS 与无年龄 RBF raw 按同传感器/同锚点/同 H 配对，n 分别为 5/8/8。',
        c['fair_BSS_vs_C_same_issued_known_opportunities'],
        'dimensionless_Brier; physical_group',
        ↪  c['fair_BSS_vs_C_same_issued_known_opportunities'],
        ['B_SS','nonlinear_rbf_hazard_no_record_age_raw'], 'common_anchors_no_age',
        '总14物理组，回顾锚点followup_end-H；两模型都发布且结局已知才进入配对，弃权为空。',
        ↪  'q3:tab:brier')
    claim('q3-conditional-interval-and-label-boundary',
        '模型条件中央95%区间、完整标签包含和标签相交是不同量；共同锚点的标签上界可能正好等于H。',
        c['coverage_and_conditional_interval_interpretation'],
        'h; count', c['source_files'], ['B_SS','nonlinear_rbf_hazard_no_record_age_raw'],
        'common_anchors_no_age',
        '连续条件分布97.5%分位小于H会使该锚点完整包含为0；不同于固定标签下界前24小时协议。',
        ↪  'q3:eq:interval')
    claim('q3-conformal-rank-feasibility',
        'C另留独立校准组为0；乐观上界13组的95%秩14超出样本数。B实际留一12组的秩13同样超出。',
        {'C':c['conformal'], 'B':b['conformal']}, 'physical_group; rank',
        source_records(c['conformal'], b['source_files']),
        ['B_IV','B_SS','nonlinear_rbf_hazard_no_record_age_raw'], 'rank_feasibility_only',
        'C 未计算独立共形分数或有限端点；无界是秩可行性状态，非已验证覆盖。', 'q3:sec:results')
    claim('q3-c-grid-sensitivity',
        '512 与 1024 节点保持同一物理样本；RBF raw 锚点最大概率变化 0.01041154。',
        c['numerical_grid_512_1024'], 'absolute_probability; dimensionless_Brier',
        c['numerical_grid_512_1024'], 'nonlinear_rbf_hazard_no_record_age_raw',
        'same_data_512_vs_1024',
        '48/72小时Brier变化0.00295944/0.00214720；是数值敏感性，不是独立复现样本。',
        ↪  'q3:sec:models')

    propagation_summary = read(R/'results/final_propagation_v4/summary.json')
    propagation_sources = [file_record(R/'results/final_propagation_v4'/name)
                           for name in ['summary.json','conditional_arrivals_49.csv','mapped_r」
                           ↪  uns_7.csv','input_manifest.json']]
    claim('q3-release-clock-and-49-target-records',
        '同base有7个运行、4个设计、49条目标响应；未知启动至整体释放时延使绝对到达及到达概率为空。',
        {'summary':propagation_summary, 'junction_records':figures['release_response']},
        's; UTC; target_record', propagation_sources,
        'Q2_base_mapped_to_Q4_release_response', 'seven_runs_four_designs',
        'p0000/p0072仅给定释放；p0255全历史首跨阈与评价窗左删失分列；不是7个独立灾害。',
        ↪  'q3:eq:arrival')
    claim('q3-release-support-footprints',
        '同体积全源面p0036的120/60米足迹面积为23.0112/12.1608平方千米，
        ↪  支撑距离11.554/10.841千米。',
        figures['footprints'], 'km2; km; m; s', figures['source_files'],
        'frozen_release_response', 'h_ge0.1_saved_frames_requested1800s',
        '距离从实际正释放栅格足迹计算；非河程、质心半径或已验证预警距离。', 'q3:fig:release')
    claim('q3-paper-figure-inputs',
```

```
        '两张论文图均由冻结 R3 结果重绘，发布概率信息界与外部评分使用不同证据对象。',
        {'selection':figures['information_bound_selection'],
         'information_bounds':figures['information_bounds'],
         'paired_scores':figures['paired_scores'], 'figures':figures['figures']},
        'probability; dimensionless_Brier', figures['source_files'],
        ['A_analytic_erfc_fixed_history','B_SS','nonlinear_rbf_hazard_no_record_age_raw'],
        'paper_only_redraw', '图1阴影为条件于原历史风险集的信息界，不是95%置信区间。',
        'q3:fig:information')
    assert all(item['value'] not in [None, []] for item in claims)
    write(EVIDENCE/'q3_claims.json', claims)

    codes = {}
    for item in a['self_authored_code_files']+b['self_authored_code_files']+c['actual_code_files']:
        codes[item['path']] = dict(item)
    for name, role in [
        ('final_propagation_v4.py','join/check frozen seven-run conditional target response'),
        ('join_propagation.py','first conditional propagation join'),
        ('join_propagation_v2.py','compact/base parameter handoff'),
        ('join_propagation_v3.py','full source-mask join')]:
        path = R/'scripts'/name
        codes[path.relative_to(ROOT).as_posix()] = {**file_record(path), 'role':role}
    for path in sorted((PAPER/'scripts').glob('q3_*.py')):
        codes[path.relative_to(ROOT).as_posix()] = {**file_record(path),
            'role':'paper plotting, preview compilation or evidence assembly; no model refit'}
    write(EVIDENCE/'q3_code_files.json', {
        'authorship':'团队自编模型、后处理及论文程序，OpenAI Codex 辅助；采用决定由队员审阅。',
        'rerun_for_paper':'仅绘图、证据复核和 LaTeX 编译；冻结模型不重跑。',
        'files':sorted(codes.values(), key=lambda value:value['path']),
        'third_party_solver_note':'传播求解器归Q4维护；此清单只收Q3映射、核对和后处理，
        不把第三方求解器列为自编。'})
    cited = set()
    for group in re.findall(r'\\cite\{([^}]+)\}', (PAPER/'sections/q3.tex').read_text()):
        cited.update(group.split(','))
    references = a['references']+b['references']+c['primary_references_checked']
    selected_references = [v for v in references if v['key'] in cited]
    assert {v['key'] for v in selected_references} == cited
    write(EVIDENCE/'q3_reference_reading.json', {
        'policy':'仅登记正文实际引用来源；阅读范围按实际 HTML/摘要/原文或元数据区别。',
        'references':selected_references})


def check():
    claims = read(EVIDENCE/'q3_claims.json')
    codes = read(EVIDENCE/'q3_code_files.json')
    figure_data = read(EVIDENCE/'q3_figure_data.json')
    audited_sources = source_records(claims, codes, figure_data)
    for record in audited_sources:
        path = ROOT/record['path']
        assert path.is_file(), record['path']
        assert file_record(path)['sha256']==record['sha256'], record['path']
    required = {'claim_id','statement','value','unit','source_files','model_id','scenario_id',
    'scope','paper_anchor'}
    assert all(required <= value.keys() and value['source_files'] for value in claims)
    text = (PAPER/'sections/q3.tex').read_text()
    assert not [c for c in text if ord(c)<32 and c not in '\n\r']
    labels = re.findall(r'\\label\{([^}]+)\}',text)
    assert len(labels)==len(set(labels)) and all(value.startswith('q3:') for value in labels)
    assert all(value['paper_anchor'] in labels for value in claims)
    cited = {key for group in re.findall(r'\\cite\{([^}]+)\}',text) for key in
    group.split(',')}
    bibkeys =
    set(re.findall(r'\\bibitem\{([^}]+)\}',(EVIDENCE/'q3_references.tex').read_text()))
```

```
    assert cited==bibkeys and all(key.startswith('q3-') for key in cited)
    log = (PAPER/'figures/q3/.qa/q3_preview.log').read_text(errors='replace')
    prohibited = ['Overfull', 'Missing character', 'undefined references', 'LaTeX Warning: Citation',
                  'LaTeX Font Warning', '! LaTeX Error']
    assert not [line for line in log.splitlines() if any(value in line for value in prohibited)]
    aux = (PAPER/'figures/q3/.qa/q3_preview.aux').read_text()
    end = re.search(r'\\newlabel\{q3:preview-end\}\{\{[^}]*\}\{(\d+)\}',aux)
    assert end
    result = {
        'passed':True, 'claims':len(claims), 'hashed_source_and_code_files':len(audited_sources),
        'body_han_characters':sum('\u4e00'<=char<='\u9fff' for char in text),
        'section_body_pages':int(end[1]), 'preview_references_on_separate_page':True,
        'figures':text.count(r'\begin{figure}'), 'tables':text.count(r'\begin{table}'),
        'numbered_equations':text.count(r'\begin{equation}'), 'cited_references':len(cited),
        'labels':labels, 'tex_errors_overfull_missing_glyphs_undefined_citations':0,
        'source_hashes_checked':True, 'no_model_refit':True,
        'compile_inputs':[file_record(path) for path in
                          [PAPER/'preamble.tex', PAPER/'sections/q3.tex',
                          EVIDENCE/'q3_references.tex']],
        'compiled_artifacts':[file_record(path) for path in sorted((PAPER/'figures/q3').glob('*.pdf'))],
        'scope':'仅 Q3 论文层静态核对；不增加独立事件样本，不替代模型科学评价。'}
    write(EVIDENCE/'q3_validation.json', result)
    print(json.dumps({key:result[key] for key in
                      ['passed','claims','hashed_source_and_code_files','body_han_characters',
                       'section_body_pages','figures','tables','cited_references']},ensure_asc
                       ii=False))


if __name__=='__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--check-only',action='store_true')
    args = parser.parse_args()
    if not args.check_only:
        build()
    check()
```

程序 14：paper plotting, preview compilation or evidence assembly; no model refit

来源：track_b/paper/scripts/q3_figures.py
SHA256：1b0c236fc7fd132c838333a96c70b43e1951aa7ad5111513488f58471d624e70

```
#!/usr/bin/env python3
"""Paper-only redraw from frozen Q3 results; no model fitting or simulation."""
import os
from pathlib import Path
import hashlib
import json
import subprocess

ROOT = Path(__file__).resolve().parents[3]
PAPER = ROOT / 'track_b/paper'
R = ROOT / 'track_b/research/q3/round3'
OUT = PAPER / 'figures/q3'
OUT.mkdir(parents=True, exist_ok=True)
os.environ.setdefault('MPLCONFIGDIR', str(OUT / '.mpl-cache'))
os.environ.setdefault('MPL_IGNORE_SYSTEM_FONTS', '1')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.lines import Line2D
from matplotlib.text import Text
import matplotlib.dates as mdates
import numpy as np
import pandas as pd

font_file=Path(subprocess.check_output(['fc-match','-f','%{file}','SimSun'],text=True).strip())
font_manager.fontManager.addfont(str(font_file))
font_name=font_manager.FontProperties(fname=str(font_file)).get_name()
from matplotlib.ft2font import FT2Font
assert FT2Font(str(font_file)).get_char_index(ord('预'))>0,'Install a Chinese font before
↪  rendering'
plt.rcParams.update({'font.family':['DejaVu Sans'],'font.size':9.5,
                     'axes.unicode_minus':False,'pdf.fonttype':42,'ps.fonttype':42,
                     'axes.spines.top':False,'axes.spines.right':False,
                     'axes.labelsize':10,'legend.fontsize':8.5,'xtick.labelsize':9,'ytick.labe」
                     ↪  lsize':9})
BLUE, ORANGE, GRAY = '#286988', '#BD6C42', '#6B747B'
inputs = []


def record(p):
    return
    ↪  {'path':str(p.relative_to(ROOT)),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()}


def load(relative):
    p = R / relative
    if p not in inputs:
        inputs.append(p)
    return pd.read_csv(p)


def save(fig, name):
    for artist in fig.findobj(match=Text):
```

```
        artist.set_fontproperties(font_manager.FontProperties(fname=str(font_file),size=artist.get_fontsize()))
    fig.savefig(OUT/name, bbox_inches='tight', pad_inches=.045)
    plt.close(fig)


def main():
    (PAPER/'evidence').mkdir(parents=True,exist_ok=True)
    fixed = load('results/applicability_sensitivity/risk_set_revision/fixed_history_risk_predictions.csv.gz')
    selected = fixed[(fixed.model_id=='mechanism_policy_mixture') &
                     (fixed.probability_prior=='uniform_design_assumption') &
                     (fixed.forecast_input_mode=='assumed_latency') &
                     (fixed.space_object=='aoi_unweighted_mean') &
                     (fixed.assumed_latency_hours==0) &
                     (fixed.horizon_hours==72)].sort_values('issue_time')
    assert len(selected)==75 and selected.probability_start_within_horizon.isna().all()
    assert selected.probability_start_lower.eq(0).all()
    pairs = load('results/common_anchors_no_age/paired_valid_scores.csv')
    model_c = 'nonlinear_rbf_hazard_no_record_age_raw'
    paired = pairs[(pairs.model_one=='B_SS') & (pairs.model_two==model_c)].sort_values('horizon_hours')
    assert list(paired.horizon_hours)==[24,48,72]
    assert list(paired.same_known_probability_groups)==[5,8,8]
    fig, ax = plt.subplots(1,2,figsize=(6.65,2.25),gridspec_kw={'width_ratios':[1.1,1]})
    fig.subplots_adjust(wspace=.38)
    t = pd.to_datetime(selected.issue_time)
    lo,hi = selected.probability_start_lower.to_numpy()*100,selected.probability_start_upper.to_numpy()*100
    ax[0].fill_between(t,lo,hi,color=BLUE,alpha=.23,label='启动质量信息界')
    ax[0].plot(t,hi,color=BLUE,lw=1.5)
    ax[0].plot(t,lo,color=BLUE,lw=.8)
    ax[0].set_ylim(-1,104)
    ax[0].set_ylabel('条件启动概率界 / %')
    ax[0].set_xlabel('发布时刻（UTC）')
    ax[0].xaxis.set_major_locator(mdates.DayLocator())
    ax[0].xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
    ax[0].set_title('(a) 72 h：固定历史风险集',fontsize=10,pad=7)
    ax[0].legend(loc='upper right',frameon=False)
    x = np.arange(3);width=.31
    ax[1].bar(x-width/2,paired.brier_one,width,color=BLUE,label='状态空间 SS')
    ax[1].bar(x+width/2,paired.brier_two,width,color=ORANGE,label='无年龄 RBF')
    ax[1].set_xticks(x,['24 h\n(n=5)','48 h\n(n=8)','72 h\n(n=8)'])
    ax[1].set_ylabel('同机会 Brier 分数')
    ax[1].set_xlabel('预测时域')
    ax[1].set_ylim(0,.425)
    ax[1].legend(loc='upper left',frameon=False)
    ax[1].set_title('(b) 外部病例分组配对',fontsize=10,pad=7)
    for a in ax:
        a.grid(axis='y',color='#D9DEE2',alpha=.65,lw=.5);a.set_axisbelow(True)
    save(fig,'q3_information_scores.pdf')

    joined = load('results/final_propagation_v4/conditional_arrivals_49.csv')
    footprint = pd.concat([load('results/arrival/propagation_v2/depth_footprint_diagnostics.csv'),
                           load('results/arrival/propagation_v3/depth_footprint_diagnostics.csv'),
                           load('results/arrival/compact_delta_v1/depth_footprint_diagnostics.csv')],ignore_index=True)
    f = footprint[(footprint.scope=='saved_outputs_through_requested_horizon') & (footprint.threshold_depth_m==.1)]
    junction = joined[joined.target_id=='paper_junction'].copy()
    assert len(junction)==len(f)==7
```

```
ids = ['q2map60_q2r3_p0000_S_linear__sf0','q2full_q2r3_p0000_S_linear__geomv2_60m__sf0',
       'q2map60_q2r3_p0036_S_T_linear__sf0','q2full_q2r3_p0036_S_T_linear__geomv2_120m__sf0',
       'q2full_q2r3_p0036_S_T_linear__geomv2_60m__sf0','q2map60_q2r3_p0072_S_H_linear__sf0',
       'q2map60_q2r3_p0255_S_T_H_linear__sf0']
junction = junction.set_index('run_id').loc[ids].reset_index()
f = f.set_index('run_id').loc[ids].reset_index()
names = ['S 紧凑 60 m','S 全源面 60 m','ST 紧凑 60 m','ST 全源面 120 m',
         'ST 全源面 60 m','SH 紧凑 60 m','STH 紧凑 60 m']
compact = [not rid.startswith('q2full_') for rid in ids]
fig,ax = plt.subplots(1,2,figsize=(6.65,2.5),gridspec_kw={'width_ratios':[1.34,1]})
fig.subplots_adjust(wspace=.45)
for j,row in enumerate(junction.itertuples()):
    color=ORANGE if compact[j] else BLUE
    lower=row.diagnostic_bulk_release_arrival_lower_seconds
    upper=row.diagnostic_bulk_release_arrival_upper_seconds
    if np.isfinite(lower) and np.isfinite(upper):
        middle=(lower+upper)/2
        ax[0].errorbar(middle,j,xerr=[[middle-lower],[upper-middle]],fmt='o',ms=4.3,
                       color=color,mfc='white' if row.if_released_propagation else color,capsize=3,lw=1.2)
    else:
        ax[0].text(1060,j,'保存帧未命中',va='center',fontsize=8.5,color=GRAY)
ax[0].set_yticks(range(7),names)
ax[0].invert_yaxis();ax[0].set_ylim(6.6,-.6)
ax[0].set_xlim(0,1810);ax[0].set_xticks([0,600,1200,1800])
ax[0].set_xlabel('距给定整体释放的时延 / s')
ax[0].set_title('(a) 汇流点保存帧首达括号',fontsize=10,pad=8)
ax[0].grid(axis='x',color='#D9DEE2',lw=.5)
offsets=[(5,6),(-40,0),(-12,9),(-34,7),(-40,-3),(-32,10),(8,-13)]
short=['S','S','ST','ST120','ST60','SH','STH']
for j,row in enumerate(f.itertuples()):
    color=ORANGE if compact[j] else BLUE
    dx=row.maximum_affected_cell_centre_distance_from_release_support_m/1000
    area=row.affected_plan_area_m2/1e6
    ax[1].scatter(dx,area,s=30,color=color,marker='o' if compact[j] else 's',zorder=3)
    ax[1].annotate(short[j],(dx,area),xytext=offsets[j],textcoords='offset points',fontsize=8.5,color=color)
ax[1].set_xlabel('距释放栅格支撑的最远距离 / km')
ax[1].set_ylabel('阈值足迹面积 / km$^2$')
ax[1].set_xlim(4.8,12.3);ax[1].set_ylim(0,26)
ax[1].set_title('(b) h≥0.1 m，保存帧 ≤1800 s',fontsize=10,pad=8)
ax[1].legend(handles=[Line2D([0],[0],marker='o',color=ORANGE,lw=0,label='紧凑源'),
                      Line2D([0],[0],marker='s',color=BLUE,lw=0,label='全源面')],loc='upper left',frameon=False)
ax[1].grid(color='#D9DEE2',lw=.5);ax[1].set_axisbelow(True)
save(fig,'q3_release_response.pdf')
data={'description':'All numeric plot inputs are projected from frozen research outputs; no fitting or dynamics rerun.',
      'source_files':[record(p) for p in inputs],
      'information_bound_selection':{'model_id':'mechanism_policy_mixture','probability_prior':'uniform_design_assumption',
          'forecast_input_mode':'assumed_latency','space_object':'aoi_unweighted_mean','assumed_latency_hours':0,'horizon_hours':72},
      'information_bounds':selected[['issue_time','probability_start_lower','probability_start_upper','at_risk_prior_mass','unresolved_future_prior_mass']].to_dict('records'),
      'paired_scores':paired.to_dict('records'),
      'release_response':junction.where(pd.notna(junction),None).to_dict('records'),
      'footprints':f.to_dict('records'),'figures':[record(OUT/n) for n in ['q3_information_scores.pdf','q3_release_response.pdf']]}
# pandas float columns retain NaN after where(None); JSON uses a standard null projection.
```

```
    data=json.loads(json.dumps(data,default=str).replace('NaN','null'))
    (PAPER/'evidence/q3_figure_data.json').write_text(json.dumps(data,ensure_ascii=False,inden
    ↪  t=2,allow_nan=False)+'\n')


if __name__=='__main__':
    main()
```

程序 15：paper plotting, preview compilation or evidence assembly; no model refit

来源：track_b/paper/scripts/q3_preview.py
SHA256：d9c0adae0df2133c3864be93b4c35b2e75a490156ece208cb87f65dc0a096b67

```
#!/usr/bin/env python3
"""Compile the Q3 fragment with the shared read-only preamble, without editing main.tex."""
from pathlib import Path
import shutil
import subprocess

ROOT = Path(__file__).resolve().parents[3]
PAPER = ROOT / 'track_b/paper'
QA = PAPER / 'figures/q3/.qa'


def main():
    QA.mkdir(parents=True, exist_ok=True)
    wrapper = QA / 'q3_preview.tex'
    wrapper.write_text(r'''\documentclass[UTF8,a4paper,zihao=-4,fontset=none]{ctexart}
\input{preamble}
\begin{document}
\input{sections/q3}
\label{q3:preview-end}
\clearpage
\begin{thebibliography}{99}
\input{evidence/q3_references}
\end{thebibliography}
\end{document}
''')
    binary = shutil.which('xelatex')
    if not binary:
        candidate = Path.home()/'.local/texlive/texlive/2026/bin/universal-darwin/xelatex'
        if candidate.is_file():
            binary = str(candidate)
    if not binary:
        raise SystemExit('XeLaTeX is required; install a TeX distribution with ctex and the
        ↪  shared preamble packages.')
    command = [binary, '-interaction=nonstopmode', '-halt-on-error',
               '-output-directory=figures/q3/.qa', '-jobname=q3_preview',
               'figures/q3/.qa/q3_preview.tex']
    for _ in range(2):
        result = subprocess.run(command, cwd=PAPER, capture_output=True, text=True)
        (QA/'q3_compile_stdout.txt').write_text(result.stdout+result.stderr)
        if result.returncode:
            print(result.stdout[-6000:])
            raise SystemExit(result.returncode)
    destination = PAPER/'figures/q3/q3_section_preview.pdf'
    shutil.copyfile(QA/'q3_preview.pdf', destination)
    print(destination.relative_to(ROOT))


if __name__ == '__main__':
    main()
```

程序 16：erfc 线性因果卷积热提供者

来源：track_b/research/q3/round3/inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1/files/scripts/analytic_thermal_engine.py
SHA256：a113d3f47e97266761fa47b7d6ed7e5a9e830edeeaf2ba3918a70db3c0003607

```
"""Analytical 1D conductive challenger for hourly step boundary conditions.

The mechanical/water/damage equations are the frozen Q2R3 engine.  This module
only supplies the thermal tuple to its public simulate(thermal=...) argument.
No observations, event fitting or probability weights are introduced.
"""
from __future__ import annotations

import math

import numpy as np

from mechanism_engine import simulate


def erfc_array(values):
    """Use the standard-library scalar erfc, avoiding an additional dependency."""
    values = np.asarray(values, float)
    return np.fromiter((math.erfc(float(value)) for value in values.flat),
                       dtype=float, count=values.size).reshape(values.shape)


def step_kernel(nt, depth_m, alpha_m2_s, dt_hours=1.):
    """Semi-infinite Dirichlet step response; K[0]=0 by state0 convention."""
    depth, alpha = np.broadcast_arrays(np.atleast_1d(np.asarray(depth_m, float)),
                                       np.atleast_1d(np.asarray(alpha_m2_s, float)))
    if nt < 1 or dt_hours <= 0 or np.any(depth < 0) or np.any(alpha <= 0):
        raise ValueError("invalid time/depth/diffusivity")
    if not np.all(np.isfinite(depth)) or not np.all(np.isfinite(alpha)):
        raise ValueError("nonfinite thermal parameters")
    kernel = np.zeros((nt, depth.size))
    if nt > 1:
        denominator = 2*np.sqrt(np.arange(1, nt)[:, None] * dt_hours * 3600 * alpha[None, :])
        kernel[1:] = erfc_array(depth[None, :]/denominator)
    return kernel


def finite_step_kernel(nt, depth_m, alpha_m2_s, bottom_m, dt_hours=1., terms=12):
    """Finite0..L Dirichlet/no-flux step response, using a mirror expansion.

    cosh((L-z)q)/cosh(Lq) = sum_n(-1)^n *
        [exp(-(2nL+z)q) + exp(-(2(n+1)L-z)q)], q=sqrt(s/alpha).
    Divide by s for a unit step and invert each exponential to erfc.  The
    complementary paired form below avoids subtracting the main kernel when
    reporting the extremely small production-domain reflection correction.
    """
    depth, alpha = np.broadcast_arrays(np.atleast_1d(np.asarray(depth_m, float)),
                                       np.atleast_1d(np.asarray(alpha_m2_s, float)))
    if bottom_m <= 0 or np.any(depth > bottom_m) or terms < 1:
        raise ValueError("invalid finite thermal domain")
    kernel = step_kernel(nt, depth, alpha, dt_hours)
    correction = np.zeros_like(kernel)
    if nt > 1:
        denominator = 2*np.sqrt(np.arange(1, nt)[:, None] * dt_hours * 3600 * alpha[None, :])
        for n in range(1, terms+1):
```

```
            pair = (erfc_array((2*n*bottom_m-depth)[None, :]/denominator)
                    - erfc_array((2*n*bottom_m+depth)[None, :]/denominator))
            correction[1:] += (1 if n % 2 else -1) * pair
    return kernel+correction, correction


def linear_causal_convolution(increments, kernel):
    """Zero-padded FFT linear convolution, truncated to physical state times.

    nfft >= 2*nt-1 prevents circular wrap-around from future increments.  K0=0
    ensures forcing[i] first influences state[i+1], not state[i].
    """
    increments, kernel = np.asarray(increments, float), np.asarray(kernel, float)
    if increments.shape != kernel.shape or increments.ndim != 2:
        raise ValueError("thermal increments/kernel must have equal(nt,n) shape")
    nt = len(increments)
    nfft = 1 << (2*nt-2).bit_length()
    transformed = np.fft.rfft(increments, n=nfft, axis=0) * np.fft.rfft(kernel, n=nfft, axis=0)
    return np.fft.irfft(transformed, n=nfft, axis=0)[:nt], nfft


def thermal_transfer_analytic(air_c, parameters, dt_hours=1.):
    """Return (interface temperature, imposed surface temperature) at states.

    Boundary B[i]=min(air[i]+offset,0) drives [i,i+1).  Initial interior state
    is uniform Tinit.  dB0=B0-Tinit and dBj=Bj-Bj-1 for j>=1.  Neither the last
    boundary value nor any future perturbation can drive an earlier state.
    Production depths are strictly positive (0.5..20m); if z=0 is requested,
    state[k] uses the left boundary limit B[k-1], with state[0]=Tinit.
    """
    air = np.asarray(air_c, float)
    if air.ndim != 1 or len(air) < 1 or not np.isfinite(air).all():
        raise ValueError("finite nonempty one-dimensional air boundary required")
    initial = np.atleast_1d(np.asarray(parameters["initial_temp_c"], float))
    offset = np.atleast_1d(np.asarray(parameters["station_source_offset_c"], float))
    depth = np.atleast_1d(np.asarray(parameters["interface_depth_m"], float))
    alpha = np.atleast_1d(np.asarray(parameters["diffusivity_m2_s"], float))
    if not all(value.shape == initial.shape for value in [offset, depth, alpha]):
        raise ValueError("thermal parameter arrays must have one shared row shape")
    if not np.isfinite(initial).all() or not np.isfinite(offset).all():
        raise ValueError("nonfinite initial temperature or source offset")
    boundary = np.minimum(air[:, None]+offset[None, :], 0.)
    increments = np.empty_like(boundary)
    increments[0] = boundary[0]-initial
    increments[1:] = np.diff(boundary, axis=0)
    kernel = step_kernel(len(air), depth, alpha, dt_hours)
    response, _ = linear_causal_convolution(increments, kernel)
    temperature = initial[None, :]+response
    # At t0 no interval has elapsed. Restore the exact declared initial state;
    # all other pre-perturbation checks retain actual FFT round-off.
    temperature[0] = initial
    return temperature, boundary


def simulate_analytic(air_c, precip_mm_h, parameters, mechanism="S_T_H",
                      policy="linear_extension", dt_hours=1.,
                      drainage_feedback=True, damage_enabled=True):
    """Public Q3 wrapper: same frozen mechanical engine and criterion/censoring."""
    thermal = thermal_transfer_analytic(air_c, parameters, dt_hours)
    return simulate(air_c, precip_mm_h, parameters, mechanism=mechanism, policy=policy,
                    dt_hours=dt_hours, thermal=thermal,
                    drainage_feedback=drainage_feedback, damage_enabled=damage_enabled)

```

```
def reflection_temperature_bound(air_c, parameters, bottom_m=40., dt_hours=1.):
    """Conservative finite-Neumann versus half-space error, per parameter row.

    Mirror correction is an alternating series of positive decreasing erfc
    differences, hence 0 <= Kfinite-Ksemi <= erfc((2L-z)/(2sqrt(alpha*t))).
    Bound the convolution by this horizon maximum times sum(abs(dB)).  This
    remains a temperature-discretization bound, not a physical/model error.
    """
    air = np.asarray(air_c, float)
    initial = np.atleast_1d(np.asarray(parameters["initial_temp_c"], float))
    offset = np.atleast_1d(np.asarray(parameters["station_source_offset_c"], float))
    depth = np.atleast_1d(np.asarray(parameters["interface_depth_m"], float))
    alpha = np.atleast_1d(np.asarray(parameters["diffusivity_m2_s"], float))
    if bottom_m <= 0 or np.any(depth < 0) or np.any(depth > bottom_m) or dt_hours <= 0:
        raise ValueError("depth outside finite reflection-bound domain")
    boundary = np.minimum(air[:, None]+offset[None, :], 0.)
    increments = np.empty_like(boundary)
    increments[0] = boundary[0]-initial
    increments[1:] = np.diff(boundary, axis=0)
    # Last boundary is unused at the last output state (K0=0).
    total_variation = np.abs(increments[:-1]).sum(axis=0)
    horizon = (len(air)-1)*dt_hours*3600
    first_reflection = erfc_array((2*bottom_m-depth)/(2*np.sqrt(alpha*horizon))) if horizon > 0
    ↪  else np.zeros_like(depth)
    return total_variation*first_reflection, total_variation, first_reflection
```

程序 17：结构/损伤/储水与温应力强度关系自编引擎

来源：track_b/research/q3/round3/inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1/files/scripts/mechanism_engine.py
SHA256：bb6636d2d3f463c894cdc6bc1df9c7fd0f93f9b7a6f9aa6e8233113630afe14e

```
"""Q2R3 plane-slab conditional mechanism engine; SI except kPa and hours.

No event-time fitting; arrays are computational scenarios, not observations.
Time[0] is the specified initial state. Forcing[i] drives [time[i],time[i+1]).
"""
from __future__ import annotations

import numpy as np

MODEL_IDS = {"S": "q2r3_S_v1", "S_T": "q2r3_ST_v1", "S_H": "q2r3_SH_v1", "S_T_H": "q2r3_STH_v1"}
MARGIN_DEFINITION = "(resisting_shear_kPa-driving_shear_kPa)/driving_shear_kPa; positive stable"
G = 9.80665


def stresses(parameters):
    """h is thickness normal to plane, not vertical thickness."""
    theta = np.deg2rad(np.asarray(parameters["theta_deg"], float))
    weight = np.asarray(parameters["density_kg_m3"]) * G * np.asarray(parameters["thickness_normal_m"]) / 1000
    return weight * np.cos(theta), weight * np.sin(theta)


def joint_strength(temperature_c, policy="linear_extension"):
    """Mamot2018 Eq4/5: ice-filled limestone joint, not pure ice.

    Source domain -8<=T<=-.5 degC; effective stress/domain checked by caller.
    linear_extension explicitly retains and flags extrapolation.
    thawed_rock_contact is an uncalibrated challenger above -.5 C.
    """
    t = np.asarray(temperature_c, float)
    c, mu = 53.3 - 73.5 * t, 0.42 - 0.21 * t
    if policy == "thawed_rock_contact":
        c, mu = np.where(t > -0.5, 0., c), np.where(t > -0.5, 0.60, mu)
    elif policy != "linear_extension":
        raise ValueError(policy)
    return c, mu


def resistance(parameters, damage, interface_c, pressure_kpa, policy="linear_extension"):
    sn, drive = stresses(parameters)
    c_joint, mu_joint = joint_strength(interface_c, policy)
    bridge = np.asarray(parameters["bridge_fraction"])
    cohesion = (1-np.asarray(damage)) * (bridge*np.asarray(parameters["bridge_cohesion_kpa"]) + (1-bridge)*c_joint)
    friction = bridge*np.asarray(parameters["rock_mu"]) + (1-bridge)*mu_joint
    eff = sn-np.asarray(pressure_kpa)
    resist = cohesion + np.maximum(eff, 0)*friction
    margin = (resist-drive)/drive
    domain = (np.asarray(interface_c)>=-8) & (np.asarray(interface_c)<=-.5) & (eff>=100) & (eff<=400)
    algebraic_head = (sn-(drive-cohesion)/friction)/(G)  # rho_water=1000: kPa per metre = g
    # With nonnegative effective stress, cohesion > driving cannot be
    # destabilized by pressure alone. Do not export an unreachable root.
```

```
    critical_head = np.where(cohesion <= drive, algebraic_head, np.nan)
    root_status = np.where(cohesion > drive, 'no_pressure_only_root',
                           np.where(cohesion == drive, 'contact_limit_critical',
                                    np.where(algebraic_head <= 0, 'dry_already_critical',
                                    ↪  'positive_pressure_root')))
    return dict(resistance_kpa=resist, driving_kpa=drive, normal_kpa=sn,
                effective_kpa=eff, failure_margin=margin, strength_domain=domain,
                contact_valid=eff>0, critical_head_m=critical_head,
                cohesion_kpa=cohesion, mu=friction,critical_head_status=root_status)


def thermal_transfer(air_c, parameters, dt_hours=1., dz_m=.5, bottom_m=40.):
    """Explicit 1D conduction, surface Dirichlet, bottom no flux.

    Surface min(air + scenario offset, 0) assumes an ice-capped boundary;
    this is not an energy balance or measured source temperature.
    """
    air = np.asarray(air_c, float)
    if not np.all(np.isfinite(air)):
        raise ValueError("missing temperature: cannot silently fill")
    n = len(parameters["initial_temp_c"])
    initial = np.asarray(parameters["initial_temp_c"], float)
    alpha = np.asarray(parameters["diffusivity_m2_s"], float)
    depth = np.asarray(parameters["interface_depth_m"], float)
    z = np.arange(0, bottom_m+dz_m*.5, dz_m)
    if depth.max() >= bottom_m or depth.min() < 0:
        raise ValueError("interface outside thermal domain")
    ratio = alpha*dt_hours*3600/dz_m**2
    substeps = max(1, int(np.ceil(ratio.max()/.45)))
    rr = ratio[:, None]/substeps
    prof = np.repeat(initial[:, None], len(z), axis=1)
    ti = np.empty((len(air), n), float)
    surf = np.minimum(air[:, None]+np.asarray(parameters["station_source_offset_c"])[None, :],
    ↪  0)
    k = np.floor(depth/dz_m).astype(int); frac = depth/dz_m-k; row=np.arange(n)
    ti[0] = initial
    for i in range(len(air)-1):
        for _ in range(substeps):
            old=prof.copy(); old[:,0]=surf[i]
            prof[:,1:-1]=old[:,1:-1]+rr*(old[:,2:]-2*old[:,1:-1]+old[:,:-2])
            prof[:,-1]=old[:,-1]+2*rr[:,0]*(old[:,-2]-old[:,-1])
            prof[:,0]=surf[i]
        ti[i+1]=prof[row,k]*(1-frac)+prof[row,k+1]*frac
    return ti, surf


def reservoir_step(storage_m, input_m_h, tau_h, capacity_m, dt_h):
    """Exact piecewise constant-input reservoir with continuous overflow.

    Integrate until the capacity is hit, then keep capacity storage while
    I-C/tau overflows. This avoids timestep-dependent drain/overflow splits.
    """
    w,inc,tau,cap=np.broadcast_arrays(np.asarray(storage_m,float),np.asarray(input_m_h,float),⌋
    ↪  np.asarray(tau_h,float),np.asarray(capacity_m,float))
    if np.any(tau<=0) or np.any(cap<0) or dt_h<=0 or np.any(inc<0): raise ValueError('invalid
    ↪  reservoir parameters')
    immediate=np.maximum(w-cap,0);w=np.minimum(w,cap)
    equilibrium=inc*tau
    decay=np.exp(-dt_h/tau)
    provisional=w*decay+equilibrium*(1-decay)
    hits=(equilibrium>cap)&(provisional>=cap)
    fraction=np.divide(equilibrium-cap,equilibrium-w,out=np.ones_like(w),where=(equilibrium-w)⌋
    ↪  >0)
```

```
    hit_time=-tau*np.log(np.clip(fraction,1e-300,1.))
    active_time=np.where(hits,dt_h-np.clip(hit_time,0,dt_h),0.)
    storage=np.where(hits,cap,provisional)
    overflow=immediate+np.maximum(inc-cap/tau,0)*active_time
    drainage=np.asarray(storage_m)+inc*dt_h-storage-overflow
    return storage,drainage,overflow


def simulate(air_c, precip_mm_h, parameters, mechanism="S_T_H", policy="linear_extension",
             dt_hours=1., dz_m=.5, thermal=None, drainage_feedback=True, damage_enabled=True):
    """Run all parameter rows together; never removes non-failing scenarios.

    Continuing after first criterion crossing is a counterfactual diagnostic,
    not motion/propagation. Independent-event counts are not defined here.
    """
    if mechanism not in MODEL_IDS: raise ValueError(mechanism)
    air, rain = np.asarray(air_c,float), np.asarray(precip_mm_h,float)
    if len(air)!=len(rain) or not np.all(np.isfinite(rain)) or np.any(rain<0):
        raise ValueError("rain inputs must be finite nonnegative conditional rates")
    tstate, surf = thermal if thermal is not None else
    ↪  thermal_transfer(air,parameters,dt_hours,dz_m)
    nt,n=tstate.shape
    has_t=mechanism in ("S_T","S_T_H"); has_h=mechanism in ("S_H","S_T_H")
    ti=tstate if has_t else
    ↪  np.broadcast_to(np.asarray(parameters['initial_temp_c']),tstate.shape)
    damage=np.empty((nt,n)); storage=np.zeros((nt,n)); pressure=np.zeros((nt,n))
    margin=np.empty((nt,n)); domain=np.empty((nt,n),bool); critical=np.empty((nt,n))
    damage[0]=parameters['initial_damage']
    sn,drive=stresses(parameters)
    capacity_head=np.minimum(np.asarray(parameters['connected_depth_m']),.98*sn/G)
    capacity=capacity_head*np.asarray(parameters['storativity'])
    cumulative_input=np.zeros(n); cumulative_drain=np.zeros(n); cumulative_overflow=np.zeros(n)
    budget_error=np.zeros(n)
    for i in range(nt):
        pressure[i]=G*storage[i]/np.asarray(parameters['storativity']) if has_h else 0
        state=resistance(parameters,damage[i],ti[i],pressure[i],policy)
        margin[i]=state['failure_margin'];domain[i]=state['strength_domain'];critical[i]=state⌋
        ↪  ['critical_head_m']
        if i==nt-1: break
        # Phenomenological stress-ratio damage; no claimed creep-material calibration.
        ratio=drive/np.maximum(state['resistance_kpa'],1e-6)
        rate=np.asarray(parameters['damage_rate_h_inv'])*np.minimum(ratio,10.)**np.asarray(par⌋
        ↪  ameters['damage_exponent']) if damage_enabled else np.zeros(n)
        damage[i+1]=1-(1-damage[i])*np.exp(-rate*dt_hours)
        damage[i+1]=np.minimum(damage[i+1],.999999)
        if has_h:
            source_air=air[i]+np.asarray(parameters['station_source_offset_c'])
            rain_fraction=np.clip((source_air+1)/2,0,1)
            incoming=np.asarray(parameters['rain_multiplier'])*rain[i]/1000*rain_fraction +
            ↪  np.asarray(parameters['melt_mm_deg_day'])*np.maximum(source_air,0)/24000
            # Damage may increase connectivity/drainage; this coupling is a scenario.
            tau=np.asarray(parameters['drainage_hours'])*np.exp(-np.asarray(parameters['draina⌋
            ↪  ge_damage_coupling'])*damage[i]) if drainage_feedback else
            ↪  np.asarray(parameters['drainage_hours'])
            storage[i+1],out,over=reservoir_step(storage[i],incoming,tau,capacity,dt_hours)
            cumulative_input+=incoming*dt_hours;cumulative_drain+=out;cumulative_overflow+=over
            budget_error=np.maximum(budget_error,np.abs(storage[i+1]-cumulative_input+cumulati⌋
            ↪  ve_drain+cumulative_overflow))
    return dict(interface_temp_c=ti,damage=damage,storage_m=storage,pressure_kpa=pressure,
                failure_margin=margin,strength_domain=domain,critical_head_m=critical,
                water_budget_max_abs_m=budget_error,water_input_m=cumulative_input,
                water_drain_m=cumulative_drain,water_overflow_m=cumulative_overflow)

```

```

def classify_trajectory(margins, dt_hours=1.):
    """Distinct missing/left-censored/interval-crossing/never-crossed states.

    A missing value before any crossing prevents a no-start claim. A first
    sample <=0 is already unstable, not an observed onset at time zero.
    """
    x=np.asarray(margins,float)
    if x.ndim!=1 or len(x)==0: raise ValueError('one nonempty trajectory required')
    finite=np.isfinite(x)
    if not finite[0]: return dict(onset_status='missing',onset_elapsed_hours=None,crossing_lower_hours=None,crossing_upper_hours=None)
    if x[0]<=0: return dict(onset_status='left_censored_already_unstable',onset_elapsed_hours=None,crossing_lower_hours=None,crossing_upper_hours=0.)
    below=np.flatnonzero(finite&(x<=0))
    if len(below):
        k=int(below[0])
        if not finite[:k+1].all(): return dict(onset_status='missing_before_possible_crossing',onset_elapsed_hours=None,crossing_lower_hours=None,crossing_upper_hours=k*dt_hours)
        f=x[k-1]/(x[k-1]-x[k])
        return dict(onset_status='interval_crossing',onset_elapsed_hours=(k-1+f)*dt_hours,crossing_lower_hours=(k-1)*dt_hours,crossing_upper_hours=k*dt_hours)
    return dict(onset_status='never_crossed' if finite.all() else 'missing',onset_elapsed_hours=None,crossing_lower_hours=None,crossing_upper_hours=None)
```

程序 18：同输入热提供者与同随访天气状态对照

来源：track_b/research/q3/round3/results/mechanistic_replay/analytic_erfc/postprocess.py
SHA256：94205d0a172062b5088fc261688e8c68ea3c4939e695cac5f7ee9683b7694ee2

```
#!/usr/bin/env python3
"""Independent lineage/branch audit of completed analytic and FD exports."""
from pathlib import Path
import hashlib,json
from collections import defaultdict
from datetime import datetime,timezone
import numpy as np
import pandas as pd
import pyarrow.parquet as pq

HERE=Path(__file__).resolve().parent
R=HERE.parents[2];ROOT=R.parents[3]
def sha(path):return hashlib.sha256(path.read_bytes()).hexdigest()
def dump(path,x):path.write_text(json.dumps(x,indent=2,ensure_ascii=False,allow_nan=False)+'\n')
def read_bundle(name):
    path=R/'inputs/q2'/name/'bundle.json';b=json.loads(path.read_text())
    return path,b,{x['role']:ROOT/x['path'] for x in b['files']}

def main():
    oldp,oldb,old=read_bundle('mechanism_bundle_v2_q2-r3-mechanism-thermal-refinement-v2')
    newp,newb,new=read_bundle('analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1')
    a=pd.read_csv(old['parameter_ensemble.csv']).set_index('base_scenario_id').sort_index()
    b=pd.read_csv(new['parameter_ensemble.csv']).set_index('base_scenario_id').sort_index()
    assert a.index.equals(b.index) and a.index.is_unique
    config=json.loads(new['config.json'].read_text());physical=list(config['ranges'])+['density_kg_m3']
    physical_diff={k:float(np.max(abs(a[k].to_numpy(float)-b[k].to_numpy(float)))) for k in physical}
    exact_metadata={k:a[k].equals(b[k]) for k in ['parameter_set_id','mechanism_id','law_policy','geometry_version','parameter_version']}
    oldforcing=next(path for key,path in old.items() if key in ['forcing_hourly_v2.csv','results/forcing_hourly_v1.csv'])
    fa=pd.read_csv(oldforcing);fb=pd.read_csv(new['forcing_hourly.csv'])
    forcing_same=fa.equals(fb)
    common=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','forecast_reference_time',
      'base_scenario_id','parameter_set_id','mechanism_id','law_policy','followup_hours','history_status','forecast_status']
    fields=['final_margin','crossing_point_hours_after_issue','initial_margin_at_issue','future_interface_temperature_change_c','max_observed_pressure_kpa']
    pa=pq.ParquetFile(R/'results/mechanistic_replay/v2/parameter_first_passages.parquet')
    pb=pq.ParquetFile(HERE/'parameter_first_passages.parquet')
    assert pa.metadata.num_rows==pb.metadata.num_rows==1536000
    counts=defaultdict(lambda:dict(rows=0,changed_crossing_status=0,changed_point_nullness=0,maximum_point_time_difference_hours=0.,
      maximum_initial_margin_difference=0.,maximum_final_margin_difference=0.,maximum_thermal_change_difference_c=0.,maximum_pressure_difference_kpa=0.))
    weather_effects=[]
    keys_match=True;total=0
    ideal_for_issue={}
    # Earlier metadata cleanup regrouped FD v2 Parquet into 65536-row blocks;
    # storage row groups are not scientific track boundaries. Stream both in
    # the registered 2048-row physical-design blocks instead.
    xa=pa.iter_batches(batch_size=2048,columns=common+fields)
```

```
    ya=pb.iter_batches(batch_size=2048,columns=common+fields)
    for i,(xb,yb) in enumerate(zip(xa,ya)):
        x=xb.to_pandas();y=yb.to_pandas()
        total+=len(y)
        compare_keys=[k for k in common if k not in ['forecast_status']]
        keys_match &= x[compare_keys].equals(y[compare_keys])
        if not keys_match:raise AssertionError('Version rows must pair on physical/scenario/weather keys')
        for mech,idx in y.groupby('mechanism_id').groups.items():
            ix=np.asarray(idx);g=counts[mech];g['rows']+=len(ix)
            g['changed_crossing_status']+=int(np.sum(x.forecast_status.iloc[ix].to_numpy()!=y.forecast_status.iloc[ix].to_numpy()))
            xp=x.crossing_point_hours_after_issue.iloc[ix].to_numpy(float);yp=y.crossing_point_hours_after_issue.iloc[ix].to_numpy(float)
            g['changed_point_nullness']+=int(np.sum(np.isfinite(xp)!=np.isfinite(yp)))
            good=np.isfinite(xp)&np.isfinite(yp)
            if good.any():g['maximum_point_time_difference_hours']=max(g['maximum_point_time_difference_hours'],float(np.max(abs(xp[good]-yp[good]))))
            for field,label in [('initial_margin_at_issue','maximum_initial_margin_difference'),('final_margin','maximum_final_margin_difference'),
              ('future_interface_temperature_change_c','maximum_thermal_change_difference_c'),('max_observed_pressure_kpa','maximum_pressure_difference_kpa')]:
                g[label]=max(g[label],float(np.max(abs(x[field].iloc[ix].to_numpy(float)-y[field].iloc[ix].to_numpy(float)))))
        mode=y.forecast_input_mode.iloc[0];issue=y.issue_time.iloc[0]
        if mode=='ideal_weather':ideal_for_issue={issue:y}
        if mode=='assumed_latency' and issue in ideal_for_issue:
            oracle=ideal_for_issue[issue]
            # State endpoints are comparable only at the same followup time.
            if y.followup_hours.iloc[0]==oracle.followup_hours.iloc[0] and y.followup_hours.iloc[0]>0:
                for mech,idx in y.groupby('mechanism_id').groups.items():
                    ix=np.asarray(idx)
                    weather_effects.append(dict(issue_time=issue,space_object=y.space_object.iloc[0],assumed_latency_hours=float(y.assumed_latency_hours.iloc[0]),
                      mechanism_id=mech,followup_hours=int(y.followup_hours.iloc[0]),rows=len(ix),
                      maximum_final_margin_weather_vs_ideal_difference=float(np.max(abs(y.final_margin.iloc[ix].to_numpy(float)-oracle.final_margin.iloc[ix].to_numpy(float)))),
                      maximum_interface_temperature_weather_vs_ideal_difference_c=float(np.max(abs(y.future_interface_temperature_change_c.iloc[ix].to_numpy(float)-oracle.future_interface_temperature_change_c.iloc[ix].to_numpy(float)))),
                      maximum_pressure_weather_vs_ideal_difference_kpa=float(np.max(abs(y.max_observed_pressure_kpa.iloc[ix].to_numpy(float)-oracle.max_observed_pressure_kpa.iloc[ix].to_numpy(float))))))
    rows=[dict(mechanism_id=k,**v) for k,v in sorted(counts.items())]
    pd.DataFrame(rows).to_csv(HERE/'paired_v2_analytic_parameter_summary.csv',index=False)
    pd.DataFrame(weather_effects).to_csv(HERE/'equal_followup_weather_state_contrasts.csv',index=False)
    checks=dict(full_2048_same_physical_parameter_values=all(x==0 for x in physical_diff.values()),
      full_2048_same_base_id_and_metadata=all(exact_metadata.values()),same_history_forcing_values=forcing_same,
      all_1536000_rows_exact_weather_base_scenario_pairing=keys_match and total==1536000,
      nonthermal_S_and_SH_branches_unchanged=all(counts[k]['maximum_final_margin_difference']==0 and counts[k]['maximum_initial_margin_difference']==0 and
        counts[k]['maximum_pressure_difference_kpa']==0 and counts[k]['maximum_point_time_difference_hours']==0 and
        counts[k]['changed_crossing_status']==0 and counts[k]['changed_point_nullness']==0 for k in ['S','S_H']))
```

```
    result=dict(checks=checks,passed=all(checks.values()),parameter_rows=2048,paired_forecast_rows=total,
      physical_parameter_maximum_absolute_differences=physical_diff,metadata_identity=exact_metadata,
      input_bundles=[dict(path=str(p.relative_to(ROOT)),sha256=sha(p)) for p in [oldp,newp]],
      forecast_parameter_comparison=rows,equal_followup_weather_state_contrast_rows=len(weather_effects),
      interpretation='Thermal-provider sensitivity under identical physical designs and source forcing. GFS vs ideal endpoint contrasts require equal followup. Nonzero state changes can coexist with unchanged rare discrete first-cross events; no empirical weather-skill claim.',
      audit_source_sha256=sha(Path(__file__)))
    dump(HERE/'lineage_and_parameter_validation.json',result)
    assert result['passed'],result
    summary=json.loads((HERE/'summary.json').read_text());summary['lineage_and_parameter_validation']=result
    summary['checks'].update(checks);summary['passed']=all(summary['checks'].values());dump(HERE/'summary.json',summary)
    validation=json.loads((HERE/'validation.json').read_text());validation['checks'].update(checks)
    validation['lineage_and_parameter_validation']=result;validation['passed']=all(validation['checks'].values());dump(HERE/'validation.json',validation)
    print(json.dumps(result,ensure_ascii=False),flush=True)

if __name__=='__main__':main()
```

程序 19：实际解析热与逐 issue 天气耦合消费端

来源：track_b/research/q3/round3/results/mechanistic_replay/analytic_erfc/replay.py
SHA256：61dfe484844fed5676f3902733ead10416eef89fd34218b1461dd3bcebd816d9

```
#!/usr/bin/env python3
"""Consume the actual Q2 erfc bundle; keep prior FD deliveries immutable."""
from __future__ import annotations
import hashlib
import json
import sys
import types
from collections import Counter
from datetime import datetime, timedelta, timezone
from pathlib import Path
import numpy as np
import pandas as pd

HERE=Path(__file__).resolve().parent
R=HERE.parents[2]
ROOT=R.parents[3]
FD_SOURCE=R/'scripts/mechanistic_replay.py'
BUNDLE=R/'inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1/bundle.json'
BUNDLE_HASH='05e8b885198a3011e316472f562ba92480788cb04c6ab9bebfb3e04dbd3660ca'
FD_INDEX=R/'results/mechanistic_replay/delivery_index.json'
FD_INDEX_HASH='404bba4b1a429d16ac154fd73549fe2e69aff05fd5be6bc54c89a0030e69a4a7'
REGISTRATION=HERE/'ANALYTIC_REPLAY.md'
EXPERIMENT='q3r3-full-mechanistic-weather-replay-analytic-erfc-v1'

def sha(path):return hashlib.sha256(path.read_bytes()).hexdigest()

# Execute the prior consumer read-only, without __pycache__ or CLI execution.
fd=types.ModuleType('frozen_fd_replay_helpers')
fd.__file__=str(FD_SOURCE)
exec(compile(FD_SOURCE.read_text(),str(FD_SOURCE),'exec'),fd.__dict__)
assert fd.ROOT==ROOT and fd.R==R
original_history=fd.continuous_history
original_forecast=fd.forecast

def load_inputs(mechanisms=None):
    assert mechanisms is None,'Registered study retains all four mechanisms'
    assert ROOT==Path('<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q3')
    assert sha(BUNDLE)==BUNDLE_HASH and sha(FD_INDEX)==FD_INDEX_HASH
    assert REGISTRATION.exists()
    index=json.loads(FD_INDEX.read_text())
    # Validate every immutable prior-delivery entry, not only its index hash.
    for item in index['files']:
        path=ROOT/item['path']
        assert path.stat().st_size==item['bytes'] and sha(path)==item['sha256'],str(path)
    bundle=json.loads(BUNDLE.read_text());files={}
    for item in bundle['files']:
        path=ROOT/item['path']
        assert path.is_relative_to(R/'inputs') and path.stat().st_size==item['bytes'] and
        ↪  sha(path)==item['sha256']
        assert item['role'] not in files
        files[item['role']]=path
    assert len(files)==21
    aliases={'scripts/mechanism_engine.py':'frozen_mechanical_engine',
      'config/experiment_v1.json':'config.json','results/parameter_ensemble.csv':'parameter_en」
      ↪  semble.csv',
```

```
      'results/forcing_hourly_v1.csv':'forcing_hourly.csv','results/scenario_outcomes.csv':'sc
      ↪ enario_outcomes.csv',
      'results/state_trajectories.parquet':'state_trajectories.parquet'}
    for canonical,actual in aliases.items():files[canonical]=files[actual]
    mechanical=types.ModuleType('q2_erfc_frozen_mechanical')
    exec(compile(files['frozen_mechanical_engine'].read_text(),str(files['frozen_mechanical_en
    ↪ gine']),'exec'),mechanical.__dict__)
    analytic=types.ModuleType('q2_delivered_analytic_thermal')
    previous=sys.modules.get('mechanism_engine')
    sys.modules['mechanism_engine']=mechanical
    try:exec(compile(files['callable_analytic_thermal_wrapper'].read_text(),str(files['callabl
    ↪ e_analytic_thermal_wrapper']),'exec'),analytic.__dict__)
    finally:
        if previous is None:sys.modules.pop('mechanism_engine',None)
        else:sys.modules['mechanism_engine']=previous
    mechanical.analytic=analytic
    ensemble=pd.read_csv(files['parameter_ensemble.csv'])
    forcing=pd.read_csv(files['forcing_hourly.csv'])
    config=json.loads(files['config.json'].read_text())
    assert len(ensemble)==2048 and ensemble.scenario_id.is_unique and
    ↪ ensemble.base_scenario_id.notna().all()
    assert set(ensemble.mechanism_id)=={'S','S_T','S_H','S_T_H'}
    assert len(forcing)==2089
    clock=[fd.dt(x) for x in forcing.time_utc]
    assert all(y-x==timedelta(hours=1) for x,y in zip(clock[:-1],clock[1:]))
    air=forcing.station_air_temp_c.to_numpy(float);rain=forcing.precip_assumed_mm_h.to_numpy(f
    ↪ loat)
    assert np.isfinite(air).all() and np.isfinite(rain).all() and (rain>=0).all()
    sources=HERE/'execution_source';sources.mkdir(parents=True,exist_ok=True)
    for path,label,suffix in [(Path(__file__),'analytic_replay','.py.txt'),(FD_SOURCE,'frozen_
    ↪ fd_helpers','.py.txt'),
                              (REGISTRATION,'registered_protocol','.md')]:
        snapshot=sources/(label+'_'+sha(path)+suffix)
        if snapshot.exists():assert snapshot.read_bytes()==path.read_bytes()
        else:snapshot.write_bytes(path.read_bytes())
        files['consumer:'+label]=snapshot
    for path in [BUNDLE,FD_INDEX,R/'inputs/weather/gfs_hourly_summary.csv',
                 R/'results/mechanistic_replay/v2/predictions.csv.gz']:
        files['consumer:'+str(path.relative_to(R))]=path
    return mechanical,bundle,files,ensemble,forcing,config,clock,air,rain

class AnalyticPhysics(fd.Physics):
    """Q2 mechanical state plus complete causal thermal boundary history."""
    def __init__(self,q,ensemble,config):
        self.q=q;self.f=ensemble;self.n=len(ensemble)
        self.names=list(config['ranges'])+['density_kg_m3']
        self.p={k:ensemble[k].to_numpy(float) for k in self.names}
        bases=ensemble.drop_duplicates('parameter_set_id').sort_values('parameter_set_id')
        self.baseids=list(bases.parameter_set_id);self.nb=len(bases)
        self.bp={k:bases[k].to_numpy(float) for k in self.names}
        self.baseindex=np.array([self.baseids.index(x) for x in ensemble.parameter_set_id])
        assert self.nb==256 and all(np.array_equal(self.p[k],self.bp[k][self.baseindex]) for k
        ↪ in self.names)
        self.has_t=ensemble.mechanism_id.isin(['S_T','S_T_H']).to_numpy()
        self.has_h=ensemble.mechanism_id.isin(['S_H','S_T_H']).to_numpy()
        self.policies={k:np.flatnonzero(ensemble.law_policy.eq(k)) for k in
        ↪ ['linear_extension','thawed_rock_contact']}
        self.policy_params={k:{a:v[idx] for a,v in self.p.items()} for k,idx in
        ↪ self.policies.items()}
        self.sn,self.drive=q.stresses(self.p)
        self.capacity=np.minimum(self.p['connected_depth_m'],.98*self.sn/q.G)*self.p['storativ
        ↪ ity']
        self.dt=config['dt_hours'];assert self.dt==1.
```

```
        self.bottom=config['thermal_bottom_m'];self.dz=None;self.z=[];self.substeps=None
        self.water_budget_max=0.;self.past_cache={};self.direct_checks=[]
        self.context=None;self.context_air=None;self.cursor=0
        self.kernel=None;self.history_temperature=None;self.history_air=None;self.history_boundary=None
        self.history_increments=None;self.reference_indices=set()
        self.future_causality_error=None

    def set_context(self,temperature,air):
        assert temperature.shape==(len(air)+1,self.nb) or temperature.shape==(len(air),self.nb)
        self.context=temperature;self.context_air=air;self.cursor=0

    def initial(self):
        assert self.context is not None and self.cursor==0
        return self.context[0].copy(),self.p['initial_damage'].copy(),np.zeros(self.n)

    def interface_base(self,profile):return profile

    def thermal_step(self,profile,air):
        assert self.cursor<len(self.context)-1
        assert air==self.context_air[self.cursor],'temperature interval alignment mismatch'
        assert np.max(abs(profile-self.context[self.cursor]))<1e-10
        self.cursor+=1
        return self.context[self.cursor].copy()

    def establish_history(self,air,issues):
        self.history_air=air.copy()
        self.history_temperature,self.history_boundary=self.q.analytic.thermal_transfer_analytic(air,self.bp,self.dt)
        self.history_increments=np.empty_like(self.history_boundary)
        self.history_increments[0]=self.history_boundary[0]-self.bp['initial_temp_c']
        self.history_increments[1:]=np.diff(self.history_boundary,axis=0)
        max_states=max(len(air),max(issues)+73)
        self.kernel=self.q.analytic.step_kernel(max_states,self.bp['interface_depth_m'],self.bp['diffusivity_m2_s'],self.dt)
        self.reference_indices={issues[0],issues[len(issues)//2],issues[-1]}
        self.set_context(self.history_temperature,air)
        # Use the actual complete provider to check FFT future perturbation.
        change=max(1,len(air)-73)
        altered=air.copy();altered[change:]-=10.
        changed,_=self.q.analytic.thermal_transfer_analytic(altered,self.bp,self.dt)
        self.future_causality_error=float(np.max(abs(changed[:change+1]-self.history_temperature[:change+1])))

    def future_temperature(self,j,aa):
        length=len(aa)
        if not length:return self.history_temperature[j:j+1].copy()
        assert j>0
        # Past-only convolution includes increments strictly before issue.
        # The remaining long memory is advanced for all 72 possible hours.
        if j not in self.past_cache:
            nt=j+73;increments=np.zeros((nt,self.nb))
            increments[:j]=self.history_increments[:j]
            response,nfft=self.q.analytic.linear_causal_convolution(increments,self.kernel[:nt])
            assert nfft>=2*nt-1
            self.past_cache[j]=response[j:j+73].copy()
        boundary=np.minimum(aa[:,None]+self.bp['station_source_offset_c'][None,:],0.)
        future_increments=np.zeros((length+1,self.nb))
        future_increments[0]=boundary[0]-self.history_boundary[j-1]
        future_increments[1:length]=np.diff(boundary,axis=0)
        future_response,nfft=self.q.analytic.linear_causal_convolution(future_increments,self.kernel[:length+1])
```

```
        assert nfft>=2*(length+1)-1
        temperature=self.bp['initial_temp_c'][None,:]+self.past_cache[j][:length+1]+future_response
        # K0=0. Preserve exactly the common historical state at issue.
        issue_error=float(np.max(abs(temperature[0]-self.history_temperature[j])))
        assert issue_error<1e-10
        temperature[0]=self.history_temperature[j]
        if j in self.reference_indices:
            # Last endpoint does not drive any returned state. This repeated
            # endpoint is an API shape placeholder, not an imputed interval.
            direct_air=np.r_[self.history_air[:j],aa,aa[-1]]
            direct,_=self.q.analytic.thermal_transfer_analytic(direct_air,self.bp,self.dt)
            error=float(np.max(abs(temperature-direct[j:])))
            self.direct_checks.append(dict(history_intervals=j,future_intervals=length,
              direct_provider_maximum_absolute_error_c=error,issue_state_roundoff_c=issue_error,
              concatenated_boundary_sha256=hashlib.sha256(direct_air.tobytes()).hexdigest()))
            assert error<1e-10
        return temperature

def continuous_history(engine,clock,air,rain,issues):
    indices=[clock.index(x) for x in issues]
    engine.establish_history(air,indices)
    history=original_history(engine,clock,air,rain,issues)
    return history

def forecast(engine,history,j,aa,pp):
    engine.set_context(engine.future_temperature(j,aa),aa)
    return original_forecast(engine,history,j,aa,pp)

def version_comparison(pred,bundle_version):
    old=R/'results/mechanistic_replay/v2/predictions.csv.gz'
    prior=pd.read_csv(old)
    keys=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','mechanism_id','law_policy','probability_prior','horizon_hours']
    values=['model_id','probability_start_within_horizon','conditional_lower_hours','conditional_median_hours','conditional_upper_hours',
      'already_started_prior_mass','at_risk_prior_mass','event_prior_mass','unresolved_future_prior_mass']
    assert not prior.duplicated(keys).any() and not pred.duplicated(keys).any()
    assert set(prior.mechanism_id)==set(pred.mechanism_id)
    pair=prior[keys+values].merge(pred[keys+values],on=keys,how='outer',suffixes=('_v2','_analytic'),validate='one_to_one',indicator=True)
    for name in values[1:]:pair[name+'_difference_analytic_minus_v2']=pair[name+'_analytic']-pair[name+'_v2']
    pair['comparison_scope']='same 2048 physical/mechanism/policy rows; linear thermal numerical provider sensitivity; no forecast skill ranking'
    pair['fragile_prior_changes_with_initial_margin']=pair.probability_prior.eq(fd.PRIORS[2])
    fd.csv_out(HERE/'paired_v2_analytic_weather_predictions.csv.gz',pair.to_dict('records'))
    x=pair.probability_start_within_horizon_difference_analytic_minus_v2.dropna()
    t=pair.conditional_median_hours_difference_analytic_minus_v2.dropna()
    grouped=[]
    for key,g in pair.groupby(['mechanism_id','law_policy','probability_prior'],dropna=False):
        p=g.probability_start_within_horizon_difference_analytic_minus_v2.dropna()
        m=g.conditional_median_hours_difference_analytic_minus_v2.dropna()
        grouped.append(dict(mechanism_id=key[0],law_policy=key[1],probability_prior=key[2],matched_rows=len(g),
          identified_probability_pairs=len(p),maximum_absolute_probability_difference=float(p.abs().max()) if len(p) else None,
          changed_probability_rows_above_1e_12=int((p.abs()>1e-12).sum()),conditional_median_pairs=len(m),
          maximum_absolute_conditional_median_difference_hours=float(m.abs().max()) if len(m) else None))
```

```
    fd.csv_out(HERE/'paired_v2_analytic_summary.csv',grouped)
    return dict(status='matched_same_full_2048_universe',old_input_path=str(old.relative_to(RO
    ↪  OT)),old_input_sha256=sha(old),
      new_bundle_version=bundle_version,matched_rows=int(pair['_merge'].eq('both').sum()),unma
      ↪  tched_rows=int(pair['_merge'].ne('both').sum()),
      identified_probability_pairs=len(x),maximum_absolute_probability_difference=float(x.abs(
      ↪  ).max()) if len(x) else None,
      changed_probability_rows_above_1e_12=int((x.abs()>1e-12).sum()),conditional_median_pairs
      ↪  =len(t),
      maximum_absolute_conditional_median_difference_hours=float(t.abs().max()) if len(t) else
      ↪  None,
      interpretation='same issue/mechanism/law/space/latency/prior/horizon; model IDs versioned
      ↪  and paired by mechanism; no empirical accuracy or coverage claim')

engine_holder=[]
class CapturedPhysics(AnalyticPhysics):
    def __init__(self,*args,**kwargs):
        super().__init__(*args,**kwargs);engine_holder.append(self)

def main():
    assert not (HERE/'summary.json').exists(),'completed outputs are immutable; choose a new
    ↪  version directory rather than overwrite'
    fd.BUNDLE=BUNDLE;fd.OUT=HERE;fd.EXPERIMENT=EXPERIMENT
    fd.load_inputs=load_inputs;fd.Physics=CapturedPhysics
    fd.continuous_history=continuous_history;fd.forecast=forecast;fd.version_comparison=versio
    ↪  n_comparison
    # Source provenance of audit records points to this actual wrapper; the
    # read-only helper source is independently snapshotted and hash-bound.
    fd.__file__=str(Path(__file__))
    fd.run()
    engine=engine_holder[0]
    analytic_checks={'causal_decomposition_matches_actual_provider':bool(engine.direct_checks
    ↪  and all(x['direct_provider_maximum_absolute_error_c']<1e-10 for x in
    ↪  engine.direct_checks)),
      'future_boundary_perturbation_has_no_past_effect_beyond_roundoff':engine.future_causalit
      ↪  y_error<1e-10,
      'prior_FD_delivery_index_and_every_bound_file_unchanged':sha(FD_INDEX)==FD_INDEX_HASH}
    for item in json.loads(FD_INDEX.read_text())['files']:
        path=ROOT/item['path']
        analytic_checks['prior_FD_delivery_index_and_every_bound_file_unchanged'] &=
        ↪  path.stat().st_size==item['bytes'] and sha(path)==item['sha256']
    record=dict(checks=analytic_checks,passed=all(analytic_checks.values()),direct_provider_co
    ↪  mparisons=engine.direct_checks,
      future_perturbation_maximum_preperturbation_temperature_error_c=engine.future_causality_
      ↪  error,
      method='actual delivered erfc kernel and padded linear convolution; past-only plus
      ↪  future-jump decomposition; water/damage warm state retained',
      thermal_memory='full boundary increments from 2026-06-01; interface temperature alone is
      ↪  not a Markov restart state',
      thermal_dt_hours=engine.dt,independent_observed_events=0)
    fd.dump(HERE/'analytic_causality_validation.json',record)
    assert record['passed'],record
    for name in ['summary.json','validation.json']:
        result=json.loads((HERE/name).read_text())
        result['checks'].update(analytic_checks);result['passed']=all(result['checks'].values(
        ↪  ))
        result['analytic_causality_validation']=record
        if name=='summary.json':
            result['thermal_provider']=dict(solver_id='analytic_halfspace_erfc_linear_causal_f
            ↪  ft_v1',spatial_grid=None,
              state_dt_hours=engine.dt,history_memory='full causal boundary
              ↪  increments',finite_bottom_comparator_m=engine.bottom,
```

```
              scope='linear constant diffusivity half-space with hourly step boundary; mechanical/water/damage still hourly discrete')
            result.pop('thermal_discretization',None)
            result['paired_thermal_provider_comparison']=result.pop('paired_grid_version_comparison')
            result['limitations'].extend(['Analytic thermal is a numerical reference only for the registered linear half-space model, not an exact coupled system or measured source state',
              'Prior FD differences are retained; the Q2 historical fine-FD comparison is not a bound or confidence interval for future forecasts'])
            result['actual_command']='track_b/.venv/bin/python '+str(Path(__file__).relative_to(ROOT))
        fd.dump(HERE/name,result)
    fd.event_support_diagnostics(HERE)
    fd.audit_exports(HERE)
    summary=json.loads((HERE/'summary.json').read_text())
    support=json.loads((HERE/'event_support_summary.json').read_text())
    summary['event_support_diagnostics']=support
    fd.dump(HERE/'summary.json',summary)
    output_files=[p for p in HERE.rglob('*') if p.is_file() and p.name not in ['delivery_index.json','run.stdout.log']]
    index=dict(experiment_id=EXPERIMENT,generated_at_utc=fd.stamp(datetime.now(timezone.utc)),
      upstream_bundle_sha256=BUNDLE_HASH,prior_FD_delivery_index_sha256=FD_INDEX_HASH,
      files=[dict(path=str(p.relative_to(ROOT)),bytes=p.stat().st_size,sha256=sha(p)) for p in sorted(output_files)])
    fd.dump(HERE/'delivery_index.json',index)
    print(json.dumps(dict(status='completed',summary_sha256=sha(HERE/'summary.json'),predictions_sha256=sha(HERE/'predictions.csv.gz'),
      delivery_index_sha256=sha(HERE/'delivery_index.json'),event_support_summary=support),ensure_ascii=False),flush=True)

if __name__=='__main__':main()
```

程序 20：Primarypublic data acquisition/provenance andcanonical time/label/measurement audit

来源：track_b/research/q3/round3/scripts/acquire_deformation.py
SHA256：e4d4f2f40e0616e159676f6efa9ab5bcb7d08a8898d9e8ecc42c40019fc9317e

```
#!/usr/bin/env python3
"""Bounded public source acquisition + offline canonical deformation extraction.
Raw original archives stay in the Q3-owned cache. No upstream code is executed.
"""
from __future__ import annotations
import argparse,csv,datetime as dt,hashlib,io,json,re,subprocess,urllib.request,zipfile
from pathlib import Path
import numpy as np
import pandas as pd
R=Path(__file__).resolve().parents[1]
CACHE=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/deformation')
OUT=R/'inputs/deformation'
UTC=dt.timezone.utc
PFTF_COMMIT='63a83e2da5c7a9c37382eef68be1b44963cd0af3'
URLS={
 'PFTF_real_collection.zip':'https://dataserv.ub.tum.de/public.php/dav/files/m1688868/?accept=zip',
 'brienz_tps.xlsx':'https://research-collection.ethz.ch/server/api/core/bitstreams/af4072e8-e0a4-4446-bedf-f90aad4c8783/content',
 'brienz_readme.pdf':'https://research-collection.ethz.ch/server/api/core/bitstreams/fb26c12d-b285-4288-ab55-07c153954101/content',
 'eth_item.json':'https://research-collection.ethz.ch/server/api/core/items/71f80c84-731a-4486-a529-47f3ef0b37dd',
 'brienz_primary_article.html':'https://link.springer.com/article/10.1007/s10346-024-02380-z',
 'veslemannen_primary_article.html':'https://link.springer.com/article/10.1007/s10346-020-0160-9-x',
 'Veslemannen_ESM6.txt':'https://static-content.springer.com/esm/art%3A10.1007%2Fs10346-020-01609-x/MediaObjects/10346_2020_1609_MOESM6_ESM.txt',
}
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def dump(p,x):p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(x,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
def fetch(name,url):
 p=CACHE/name
 if p.exists():return
 start=dt.datetime.now(UTC).isoformat()
 try:
  r=urllib.request.urlopen(urllib.request.Request(url,headers={'User-Agent':'Q3 academic reproduction/1.0'}),timeout=90)
  b=r.read();p.write_bytes(b)
  j={'url':url,'requested_at_utc':start,'retrieved_at_utc':dt.datetime.now(UTC).isoformat(),'status':r.status,'bytes':len(b),'sha256':sha(p),'headers':{k:v for k,v in r.headers.items() if k.lower() not in ('set-cookie','cookie')},'path':str(p)}
 except Exception as e:j={'url':url,'requested_at_utc':start,'error':str(e)}
 dump(CACHE/(name+'.request.json'),j)
def text(b):
 try:return b.decode('utf-8-sig')
 except UnicodeDecodeError:return b.decode('cp1252')
def parse_dates(s,fmt=None):
 return pd.to_datetime(s,format=fmt or 'mixed',errors='coerce',dayfirst=bool(fmt and fmt.startswith('%d')))
def clean_pairs(times,values):
 a=pd.DataFrame({'t':times,'v':pd.to_numeric(values,errors='coerce')}).dropna()
 return a.groupby('t',as_index=False).v.median().sort_values('t')
def main():
```

```
ap=argparse.ArgumentParser();ap.add_argument('--collect',action='store_true');args=ap.parse_args();CACHE.mkdir(parents=True,exist_ok=True);OUT.mkdir(parents=True,exist_ok=True)
if args.collect:
 for n,u in URLS.items():fetch(n,u)
 p=CACHE/'PFTF_63a83e2'
 if not p.exists():
  subprocess.run(['git','init',str(p)],check=True)
  subprocess.run(['git','-C',str(p),'remote','add','origin','https://gitlab.com/inverse-velocity/PFTF.git'],check=True)
  subprocess.run(['git','-C',str(p),'fetch','--depth','1','origin',PFTF_COMMIT],check=True)
  subprocess.run(['git','-C',str(p),'checkout','--detach','FETCH_HEAD'],check=True)
 assert subprocess.check_output(['git','-C',str(p),'rev-parse','HEAD'],text=True).strip()==PFTF_COMMIT
z=zipfile.ZipFile(CACHE/'PFTF_real_collection.zip')
checksum=[]
for line in z.read('m1688868/checksums.sha512').decode().splitlines():
 h,n=line.split(' *',1);member='m1688868/'+n.replace('\\','/');ok=hashlib.sha512(z.read(member)).hexdigest()==h;checksum.append({'member':member,'sha512':h,'matches':ok});assert ok
dump(OUT/'publisher_checksum_verification.json',checksum)
metas=[];series=[];exclusions=[]
# Deterministic, preregistered channel choices; no outcome-based best-sensor selection.
choices={'Agoyama':'cm','Arvigo':'POT5_3','Galterengraben':'TJM1','Grabengufer_inclinometer':'inclinometer_north','Hogarth':'J1','Kagemori':'1_mm','La_Saxe':'B4','Nevis-Bluff':'1_hor_m','Ohto':'mm','Preonzo_extensometers':'ext1_mm','Ryan_Call_slope2':'cm','Weissmiess_radar':'mean'}
unitmap={'Agoyama':'cm','Arvigo':'mm','Galterengraben':'mm','Grabengufer_inclinometer':'degree','Hogarth':'foot','Kagemori':'mm','La_Saxe':'m','Nevis-Bluff':'m','Ohto':'mm','Preonzo_extensometers':'mm','Ryan_Call_slope2':'cm','Weissmiess_radar':'mm'}
for member in z.namelist():
 if '/data/' not in member or not member.endswith('.csv'):continue
 stem=Path(member).stem;s=text(z.read(member));lines=s.splitlines();meta={'file':member,'sha256':hashlib.sha256(z.read(member)).hexdigest(),'metadata_lines_1_7':lines[:7],'read_scope':'full CSV; one preregistered channel used; pre-label only; max trailing 180 days retained'};metas.append(meta)
 if stem not in choices:
  exclusions.append({'file':member,'reason':{'Vaiont':'failure header 1963-02-01 conflicts with series until 1963-10-08; not silently corrected','Grabengufer_gnss':'header metres inconsistent with latitude/longitude-looking e,n values; inclino chosen instead','Preonzo_reflectors':'same Preonzo physical event; extensometer chosen a priori','Weissmiess_camera':'velocity mPixel/day; radar velocity chosen instead','Veslemannen':'linked ESM6 acquired separately; see supplemental record'}.get(stem,'not selected')});continue
 d=pd.read_csv(io.StringIO('\n'.join(lines[7:])))
 failure=dt.datetime.fromisoformat(lines[5].split(': ',1)[1].strip());date_only=len(lines[5].split(': ',1)[1].strip())==10
 group=lines[0].split(': ',1)[1].strip().replace(' ','_');col=choices[stem];kind='velocity' if stem=='Weissmiess_radar' else 'displacement';explicit_utc=d.columns[0]=='UTC Time'
 if stem=='La_Saxe':
  d=d[d.TargetName==col];a=clean_pairs(parse_dates(d.Epoch,'%d/%m/%Y %H.%M.%S'),d.SlopeDistance)
 else:a=clean_pairs(parse_dates(d.iloc[:,0]),d[col])
 lo=failure if date_only else failure-dt.timedelta(seconds=30);hi=failure+dt.timedelta(days=1) if date_only else failure+dt.timedelta(seconds=30)
 # Drop the whole coarse-labelled failure day to avoid using post-failure measurements.
 a=a[(a.t<lo)&(a.t>=lo-dt.timedelta(days=180))]
 if len(a)<10:exclusions.append({'file':member,'reason':'less than 10 valid pre-label measurements'});continue
 add_series(series,a,stem+'__'+col,group,unitmap[stem],kind,lo,hi,lines[5],member,explicit_utc,'curator_failure_day_interval' if date_only else 'curator_minute_label_resolution_interval',meta)
# Direct journal supplement: raw file units not labelled; primary paper describes LOS mm.
if (CACHE/'Veslemannen_ESM6.txt').exists():
```

```
 d=pd.read_csv(CACHE/'Veslemannen_ESM6.txt',sep='\t');a=clean_pairs(parse_dates(d.Time,'%d.%m.%y %H:%M'),d.Pts_01);failure=dt.datetime(2019,9,5,19);lo=failure-dt.timedelta(minutes=1);hi=failure+dt.timedelta(minutes=1);a=a[(a.t<lo)&(a.t>=lo-dt.timedelta(days=180))]
 add_series(series,a,'Veslemannen__Pts_01','Veslemannen','mm','displacement',lo,hi,'2019-09-05 19:00 curator; primary paper 20:58 local; alignment not independently proven','Veslemannen_ESM6.txt',False,'curator_minute_label_with_clock_alignment_unknown',{'metadata_lines_1_7':['LOS cumulative displacement; primary DOI10.1007/s10346-020-01609-x; mm units primary description/figure magnitude; source no timezone'],'read_scope':'Time + Pts_01; whole file read; trailing 180d pre-label; coherence caveat final hours'})
# Brienz fixed reflectors 715 and 719; all same physical event. Extract raw distance, no derived averaged velocity.
brienz=extract_brienz()
for col in ('CH','CV'):
 a=clean_pairs([dt.datetime.strptime(x['A'],'%d.%m.%Y %H:%M:%S') for x in brienz if col in x],[x[col] for x in brienz if col in x]);lo=dt.datetime(2023,6,15,23,26);hi=dt.datetime(2023,6,15,23,41);a=a[a.t<lo]
 add_series(series,a,'Brienz__'+('715' if col=='CH' else '719'),'Brienz_Insel','mm','displacement',lo,hi,'main collapse interval23:26–23:41 local on 2023-06-15 from Loew et al DOI10.1007/s10346-024-02380-z','brienz_tps.xlsx Ark1 '+col,False,'documented_main_collapse_interval',{'metadata_lines_1_7':['raw corrected slope distance [mm]; datetime timezone unspecified in workbook; local wall clock alignment assumed only for external benchmark'],'read_scope':'Ark1 A/CH/CV; Jan1–June15 2023, prior to label; full sheet streamed'})
obs=[];registry=[]
for e in series:
 a=e.pop('observations');obs.extend(a);registry.append(e)
pd.DataFrame(obs).to_csv(OUT/'observations.csv',index=False,float_format='%.12g')
pd.DataFrame(registry).to_csv(OUT/'site_registry.csv',index=False,float_format='%.12g')
dump(OUT/'metadata_audit.json',{'sources':metas,'exclusions':exclusions,'selection':'fixed named channels recorded in code; not performance selected; multi-sensor Brienz grouped; original archives retained','external_failure_phase':'published terminal/main catastrophic collapse, not first onset of any slow mechanical motion','timestamp_policy':'naive source clocks retained; only explicit UTC columns receive UTC, label-clock equivalence remains curator assumption; no fabricated UTC epoch','label_resolution':'minute brackets are display precision only, unknown physical/proxy bias separate; day labels entire source day','post_failure_rule':'strict before label lower; raw later measurements retained only in archive','max_trailing_days':180})
sources=[]
for name,url in URLS.items():
 p=CACHE/name
 if not p.exists():continue
 log=next((q for q in (CACHE/(name+'.request.json'),CACHE/(name.replace('.txt','')+'.request.json')) if q.exists()),None);j=json.loads(log.read_text()) if log else {}
```

```
 sources.append({'canonical_url_or_product_id':url,'product_version':'PFTF real archive DOI10.14459/2023mp1688868' if name.startswith('PFTF_real') else ('Loew2024 DOI10.3929/ethz-b-000670494' if name in ('brienz_tps.xlsx','brienz_readme.pdf','eth_item.json') else ('Loew2024 primary article' if name=='brienz_primary_article.html' else 'Kristensen2021 primary/ESM6')),'retrieved_at_utc':j.get('retrieved_at_utc'),'actual_read_scope':{'PFTF_real_collection.zip':'all17data-file metadata and publisher checksums; selected 12 channels; raw whole archive','brienz_tps.xlsx':'Ark1 A,CH,CV values and header; full worksheet stream, selected2023period','brienz_readme.pdf':'full README,2pages textual','eth_item.json':'full metadata and license','Veslemannen_ESM6.txt':'Time/Pts_01 series; whole text','brienz_primary_article.html':'monitoring methods and main collapse23:26–23:41 local, sensor-loss times','veslemannen_primary_article.html':'GBInSAR methods/phase wrapping and terminal collapse20:58 local, no clock equivalence assertion'}.get(name),'bytes':p.stat().st_size,'sha256':sha(p),'cache_path':str(p),'license':'CC-BY4.0 curator collection' if name.startswith('PFTF_real') else ('CC-BY-NC4.0 ETH item' if name in ('brienz_tps.xlsx','brienz_readme.pdf','eth_item.json') else 'journal CC-BY4.0 article; ESM source attributed'),'data_role':'external_transfer','available_time_or_unknown':None,'request_log':str(log) if log else None})
repo=CACHE/'PFTF_63a83e2'
for n in ('README.md','LICENSE','displ_data.Rda'):
 p=repo/n
 if p.exists():sources.append({'canonical_url_or_product_id':'https://gitlab.com/inverse-velocity/PFTF/-/blob/'+PFTF_COMMIT+'/'+n,'product_version':PFTF_COMMIT,'retrieved_at_utc':dt.datetime.fromtimestamp(p.stat().st_mtime,UTC).isoformat(),'retrieval_clock_note':'filesystem receipt metadata, not historical release availability','actual_read_scope':('full README methods/instructions/demo/license claim read' if n=='README.md' else ('license title and first25lines inspected; full bytes hashed; not full legal-text analysis' if n=='LICENSE' else 'full bytes hashed only; artificial demo from README, not decoded or fitted')),'bytes':p.stat().st_size,'sha256':sha(p),'cache_path':str(p),'data_role':'method_review_only','available_time_or_unknown':None})
for n,u,scope in [('data_landing.html','https://mediatum.ub.tum.de/1688868','full landing; data download link and CC-BY4.0 license inspected'),('datacite.json','https://api.datacite.org/dois/10.14459/2023mp1688868','full metadata, DOI title/description/canonicalURL'),('eth_bitstreams.json','https://research-collection.ethz.ch/server/api/core/bundles/60d7e8a5-8251-492e-81e5-f3f8a38f9442/bitstreams','full original-bundle list, only TPS and README fetched'),('tum_download_landing.html','https://dataserv.ub.tum.de/index.php/s/m1688868','public-share metadata and advertised public ZIP link read')]:
 q=CACHE/n
 if q.exists():sources.append({'canonical_url_or_product_id':u,'product_version':'retrieved20-10-24','retrieved_at_utc':dt.datetime.fromtimestamp(q.stat().st_mtime,UTC).isoformat(),'retrieval_clock_note':'filesystem write/receipt time from this acquisition; no historical availability inference','actual_read_scope':scope,'bytes':q.stat().st_size,'sha256':sha(q),'cache_path':str(q),'data_role':'source_and_license_audit','available_time_or_unknown':None})
dump(R/'results/deformation_sources.json',{'sources':sources,'acquisition_failures':[{'url':'https://mediatum.ub.tum.de/1688868','status':'several timeout attempts; unbounded initial DOI request eventually successful; switched official dataserv link'},{'url':'https://www.research-collection.ethz.ch/server/api/core/items/71f80c84-731a-4486-a529-47f3ef0b37dd/bundles','status':'HTTP429; lawful non-www official host succeeded'}],'PFTF_demo':'artificial Rda explicitly excluded from empirical denominator','PFTF_license_conflict':'README GPL3-or-later; LICENSE AGPL3 text, no upstream code copied/executed','historical_availability':'retrieval now does not establish original real-time sensor data availability','group_count':len(set(x['event_group'] for x in registry)),'sensor_count':len(registry),'observation_count':len(obs)})
print(json.dumps({'groups':len(set(x['event_group'] for x in registry)),'sensors':len(registry),'observations':len(obs)}))
def add_series(out,a,sid,group,unit,kind,lo,hi,label,source,utc,labkind,meta):
a=a.sort_values('t');origin=a.t.iloc[0].to_pydatetime();t=np.array([(x.to_pydatetime()-origin).total_seconds()/3600 for x in a.t]);v=a.v.to_numpy(float);rawv=v.copy()
if kind=='velocity':v=np.r_[0,np.cumsum((v[1:]+v[:-1])/2*np.diff(t)/24)]
```

```
# All displacements remain signed original/causal-integrated, orientation handled with past-only information by models.
observations=[{'site_id':sid,'event_group':group,'time_relative_hours':float(x),'time_utc':stamp.to_pydatetime().replace(tzinfo=UTC).isoformat() if utc else None,'source_timestamp':stamp.isoformat(),'displacement':float(y),'unit':unit,'raw_velocity_per_day':float(rv) if kind=='velocity' else None,'observed':True,'variable_kind':kind,'value_derivation':'causal_trapezoid_velocity_integration' if kind=='velocity' else 'raw_source_scalar'} for x,y,rv,stamp in zip(t,v,rawv,a.t)]
low=(lo-origin).total_seconds()/3600;high=(hi-origin).total_seconds()/3600
out.append({'site_id':sid,'event_group':group,'time_origin_source_clock':origin.isoformat(),'time_origin_utc':origin.replace(tzinfo=UTC).isoformat() if utc else None,'start_hours':0,'end_hours':float(t[-1]),'last_observation_hours':float(t[-1]),'followup_end_hours':high,'failure_time_hours':(low+high)/2,'failure_lower_hours':low,'failure_upper_hours':high,'failure_time_utc':((lo+(hi-lo)/2).replace(tzinfo=UTC).isoformat() if utc else None),'event_observed':True,'censoring_verified':False,'risk_status':'interval_labelled_failure','label_kind':labkind,'label_source':'https://doi.org/10.14459/2023mp1688868 / '+source if not sid.startswith('Brienz') else 'https://doi.org/10.1007/s10346-024-02380-z','label_original':label,'label_semantics':'published terminal/main catastrophic failure; not first mechanical movement initiation','unit':unit,'raw_velocity_unit':('mm/day' if kind=='velocity' else None),'derived_displacement_unit':unit,'primary_evaluation_eligible':group!='Veslemannen','evaluation_exclusion_reason':('curator19:00 vs primary20:58local clock alignment unresolved; minute bracket is not physical uncertainty' if group=='Veslemannen' else None),'variable_kind':kind,'median_cadence_hours':float(np.median(np.diff(t))),'n_observations':len(a),'data_role':'external_transfer','clock_alignment':'explicit UTC observation header; curator label same-clock assumption' if utc else 'source naive; UTC unknown; curator same-clock assumption','sensor_selection':'fixed before fitting; no best-sensor outcome selection','observations':observations})
def extract_brienz():
 p=CACHE/'brienz_selected_raw.json'
 if p.exists():return json.loads(p.read_text())
 import xml.etree.ElementTree as E
 z=zipfile.ZipFile(CACHE/'brienz_tps.xlsx');ns='{http://schemas.openxmlformats.org/spreadsheetml/2006/main}';ss=[''.join(x.itertext()) for x in E.fromstring(z.read('xl/sharedStrings.xml'))];out=[]
 for _,x in E.iterparse(z.open('xl/worksheets/sheet1.xml'),events=('end',)):
  if x.tag!=ns+'row':continue
  row={}
  for c in x:
   val=c.find(ns+'v');col=''.join(k for k in c.get('r','') if k.isalpha())
   if val is not None and col in ('A','CH','CV'):row[col]=ss[int(val.text)] if c.get('t')=='s' else val.text
  try:d=dt.datetime.strptime(row.get('A',''),'%d.%m.%Y %H:%M:%S')
  except ValueError:d=None
  if d and dt.datetime(2023,1,1)<=d<=dt.datetime(2023,6,16):out.append(row)
  x.clear()
 dump(p,out);return out
if __name__=='__main__':main()
```

程序 21：GFS 实际字段获取、原生累计解析与区域摘要

来源：track_b/research/q3/round3/scripts/acquire_weather.py
SHA256: b451ea7e17d2384dc3812f521260391c3709fc7962de2d1dc5ef7aac7da340d6

```
#!/usr/bin/env python3
"""Acquire bounded NOAA GFS fields, then offline derive hourly regional weather.

AI assistance: OpenAI Codex, 2026-10-24. No claim of operational availability.
Original project environment is read only; this script uses its existing packages.
"""
from __future__ import annotations
import argparse, concurrent.futures, csv, hashlib, json, re, time
from datetime import datetime, timedelta, timezone
from pathlib import Path
from urllib.request import Request, urlopen
import eccodes
import numpy as np

R = Path(__file__).resolve().parents[1]
ROOT = R.parents[3]
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/weather')
OUT = R / 'inputs/weather'
RES = R / 'results/weather'
CYCLES = ['2026082318', '2026082418', '2026082518']
MAX_LEAD = 96
BASE = 'https://noaa-gfs-bdp-pds.s3.amazonaws.com/'

def now(): return datetime.now(timezone.utc).isoformat().replace('+00:00', 'Z')
def stamp(x): return x.isoformat().replace('+00:00', 'Z')
def sha(p): return hashlib.sha256(p.read_bytes()).hexdigest()
def dump(p, obj):
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
def writecsv(p, rows):
    p.parent.mkdir(parents=True, exist_ok=True)
    with p.open('w', newline='') as f:
        w=csv.DictWriter(f, fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
def product(cycle, lead):
    return f'gfs.{cycle[:8]}/{cycle[8:]}/atmos/gfs.t{cycle[8:]}z.pgrb2.0p25.f{lead:03d}'

def fetch(url, path, byte_range=None):
    log = path.with_suffix(path.suffix+'.http.json')
    if path.exists() and log.exists():
        record=json.loads(log.read_text())
        expected_range=list(byte_range) if byte_range else None
        expected_status=206 if byte_range else 200
        consistent=(record['canonical_url_or_product_id']==url and
            record['byte_range']==expected_range and record['cache_path']==str(path) and
            record['http_status']==expected_status and record['bytes']==path.stat().st_size and
            sha(path)==record['sha256'])
        if byte_range:
            h={k.lower():v for k,v in record['response_headers'].items()}
            consistent=consistent and record['bytes']==byte_range[1]-byte_range[0]+1 and
            ↪  h.get('content-range','').startswith(f'bytes {byte_range[0]}-{byte_range[1]}/')
        if not consistent:raise ValueError(f'Cache provenance/hash mismatch {path}')
        return record
    if path.exists() or log.exists(): raise ValueError(f'Incomplete immutable cache {path}')
    headers={'User-Agent':'MathorCup-Q3-research/round3'}
    if byte_range: headers['Range']=f'bytes={byte_range[0]}-{byte_range[1]}'
    error=None
    for attempt in range(3):
```

```
        try:
            begin=now()
            with urlopen(Request(url, headers=headers), timeout=60) as response:
                status=response.status; h=dict(response.headers.items())
                if byte_range:
                    expected=byte_range[1]-byte_range[0]+1
                    cr=response.headers.get('Content-Range','')
                    if status!=206 or not cr.startswith(f'bytes {byte_range[0]}-{byte_range[1]}/'):
                        raise ValueError(f'Range not honored {status} {cr}')
                    raw=response.read(expected+1)
                    if len(raw)!=expected:raise ValueError('range length mismatch')
                    if raw[:4]!=b'GRIB' or raw[-4:]!=b'7777' or int.from_bytes(raw[8:16],'big')!=expected:
                        raise ValueError('not one complete GRIB2 message')
                else:
                    if status!=200:raise ValueError(f'index status {status}')
                    raw=response.read(1000001)
                    if len(raw)>1000000:raise ValueError('index size out of scope')
            end=now();path.parent.mkdir(parents=True,exist_ok=True);path.write_bytes(raw)
            record={'canonical_url_or_product_id':url,'request_started_at_utc':begin,'retrieved_at_utc':end,
                    'http_status':status,'response_headers':h,'bytes':len(raw),'sha256':sha(path),
                    'byte_range':list(byte_range) if byte_range else None,'cache_path':str(path),
                    'actual_read_scope':'one complete global GRIB2 field; derive only 85–86E,28–29N' if byte_range else 'field names and byte offsets only',
                    'available_time':None,'data_role':'conditional_scenario',
                    'product_version':{'exact_key':url.removeprefix(BASE),'etag':h.get('ETag') or h.get('Etag')}}
            dump(log,record);return record
        except Exception as exc:
            error=exc
            if attempt<2:time.sleep(1+attempt)
    raise RuntimeError(f'{url}: {error}')

def acquire_one(cycle,lead):
    key=product(cycle,lead);d=CACHE/cycle/f'f{lead:03d}';d.mkdir(parents=True,exist_ok=True)
    idx=d/'inventory.idx';records=[fetch(BASE+key+'.idx',idx)]
    lines=idx.read_text().splitlines();offsets=[int(x.split(':')[1]) for x in lines]
    selected=[]
    temp=[i for i,l in enumerate(lines) if ':TMP:2 m above ground:' in l]
    if len(temp)!=1:raise ValueError(f'TMP unique {cycle}/{lead}')
    selected.append(('temperature_2m',temp[0]))
    if lead:
        # Two APCP messages can share 0-N text at initial bucket. Last is the
        # explicit full-run accumulation (record 597 in initial audited sample).
        p=[i for i,l in enumerate(lines) if ':APCP:surface:' in l and
           (f':0-{lead} hour acc fcst:' in l or
            (lead%24==0 and f':0-{lead//24} day acc fcst:' in l))]
        if not p:raise ValueError(f'No full-run accumulation {cycle}/{lead}')
        selected.append(('precipitation_cumulative',p[-1]))
    for name,i in selected:
        record=fetch(BASE+key,d/(name+'.grib2'),(offsets[i],offsets[i+1]-1))
        record=dict(record,index_line=lines[i],index_sha256=sha(idx),variable=name)
        records.append(record)
    return records

def decode(path,cycle,lead):
    with path.open('rb') as f:g=eccodes.codes_grib_new_from_file(f)
    try:
```

```
        fields={k:eccodes.codes_get(g,k) for k in ['shortName','units','typeOfLevel','level','dataDate','dataTime',
          'validityDate','validityTime','startStep','endStep','stepUnits','stepType','Ni','Nj',
          'latitudeOfFirstGridPointInDegrees','longitudeOfFirstGridPointInDegrees',
          'iDirectionIncrementInDegrees','jDirectionIncrementInDegrees','jScansPositively',
          'centre','generatingProcessIdentifier']}
        ref=datetime.strptime(cycle,'%Y%m%d%H').replace(tzinfo=timezone.utc)
        valid=ref+timedelta(hours=lead)
        assert fields['dataDate']==int(cycle[:8]) and fields['dataTime']==int(cycle[8:])*100
        assert fields['endStep']==lead and fields['stepUnits']==1
        assert fields['validityDate']==int(valid.strftime('%Y%m%d')) and fields['validityTime']==int(valid.strftime('%H%M'))
        lat=fields['latitudeOfFirstGridPointInDegrees']+np.arange(fields['Nj'])*fields['jDirectionIncrementInDegrees']*(1 if fields['jScansPositively'] else -1)
        lon=fields['longitudeOfFirstGridPointInDegrees']+np.arange(fields['Ni'])*fields['iDirectionIncrementInDegrees']
        yi=np.flatnonzero((lat>=28)&(lat<=29));xi=np.flatnonzero((lon>=85)&(lon<=86))
        vals=eccodes.codes_get_values(g).reshape(fields['Nj'],fields['Ni'])[np.ix_(yi,xi)]
        assert vals.shape==(5,5) and np.isfinite(vals).all()
        return lat[yi],lon[xi],vals,fields
    finally:eccodes.codes_release(g)

def derive():
    protocol=R/'EXPERIMENT_PROTOCOL.md'
    if not protocol.exists():raise RuntimeError('Experiment protocol not registered')
    grid=[];means=[];metadata=[];negative=[];sources=[]
    proxy_path=ROOT/'track_b/data/external/event_remote/runs/20261024T034202Z/raw/usgs_event.json'
    proxy_lon,proxy_lat,_=json.loads(proxy_path.read_text())['geometry']['coordinates']
    sources.append({'canonical_url_or_product_id':'repo-input:'+str(proxy_path.relative_to(ROOT)),
        'product_version':'frozen-round3-baseline-file-sha256','retrieved_at_utc':None,
        'actual_read_scope':'original us7000tbwb catalogue geometry only; not accepted physical source geometry',
        'bytes':proxy_path.stat().st_size,'sha256':sha(proxy_path),'cache_path':str(proxy_path),
        'data_role':'proxy_spatial_reference','available_time':None})
    for cycle in CYCLES:
        ref=datetime.strptime(cycle,'%Y%m%d%H').replace(tzinfo=timezone.utc)
        prev=np.zeros((5,5));origin_cum=None
        for lead in range(MAX_LEAD+1):
            d=CACHE/cycle/f'f{lead:03d}'
            sources.append(json.loads((d/'inventory.idx.http.json').read_text()))
            lat,lon,temp,tmeta=decode(d/'temperature_2m.grib2',cycle,lead)
            assert tmeta['shortName']=='2t' and tmeta['typeOfLevel']=='heightAboveGround' and tmeta['level']==2
            t_record=json.loads((d/'temperature_2m.grib2.http.json').read_text())
            sources.append(dict(t_record,variable='temperature_2m'))
            if lead:
                _,_,cum,pmeta=decode(d/'precipitation_cumulative.grib2',cycle,lead)
                assert pmeta['startStep']==0 and pmeta['stepType']=='accum'
                assert pmeta['shortName']=='tp' and pmeta['typeOfLevel']=='surface' and pmeta['level']==0
                assert pmeta['units']=='kg m**-2' and tmeta['units']=='K'
                raw_increment=cum-prev
                # Quantisation can cause tiny negative differences; preserve raw
                # value in CSV and mark it, never silently sum overlapping totals.
                negative.extend([{'cycle':cycle,'lead':lead,'raw_increment_mm':float(v)} for v in raw_increment[raw_increment<0]])
                inc=raw_increment.copy();inc[inc<0]=np.nan
                p_record=json.loads((d/'precipitation_cumulative.grib2.http.json').read_text())
                th={k.lower():v for k,v in t_record['response_headers'].items()}
                ph={k.lower():v for k,v in p_record['response_headers'].items()}
```

```
                assert th.get('etag') and th['etag']==ph.get('etag'), 'T/APCP object versions differ'
                sources.append(dict(p_record,variable='precipitation_cumulative'))
                prev=cum
            else:
                cum=np.zeros((5,5));inc=np.full((5,5),np.nan);raw_increment=inc;pmeta=None
            valid=ref+timedelta(hours=lead)
            common={'forecast_reference_time':stamp(ref),'valid_time':stamp(valid),'lead_hours':lead,
                'issue_time':None,
                'precipitation_interval_start':stamp(valid-timedelta(hours=1)) if lead else None,
                'precipitation_interval_end':stamp(valid) if lead else None,'available_time':None,
                'available_time_status':'unknown_historical_first_public_availability',
                'source_product_version':product(cycle,lead),'data_role':'conditional_scenario'}
            for j,la in enumerate(lat):
                for i,lo in enumerate(lon):
                    grid.append(dict(common,latitude=float(la),longitude=float(lo),temperature_2m_K=float(temp[j,i]),
                       precipitation_increment_mm=float(inc[j,i]) if np.isfinite(inc[j,i]) else None,
                       precipitation_increment_raw_mm=float(raw_increment[j,i]) if np.isfinite(raw_increment[j,i]) else None,
                       precipitation_cumulative_mm=float(cum[j,i]) if lead else 0.0,
                       precipitation_qc='negative_difference_missing' if lead and not np.isfinite(inc[j,i]) else 'full_run_difference' if lead else 'no_preceding_forecast_interval'))
            # AOI mean is an unweighted diagnostic, not observed source climate.
            # Proxy-point nearest grid: original USGS centre 28.271N,85.515E;
            # use immutable original GeoJSON centre when source map is required.
            j=int(np.argmin(abs(lat-proxy_lat)));i=int(np.argmin(abs(lon-proxy_lon)))
            for space,tt,pp,cc in
            [('aoi_unweighted_mean',float(temp.mean()),float(np.nanmean(inc)) if lead else None,float(cum.mean())),
                ('proxy_nearest_grid_28.25N_85.5E',float(temp[j,i]),float(inc[j,i]) if lead and np.isfinite(inc[j,i]) else None,float(cum[j,i]))]:
                means.append(dict(common,space_object=space,temperature_2m_K=tt,temperature_2m_C=tt-273.15,
                    precipitation_increment_mm=pp,precipitation_cumulative_mm=cc))
            metadata.append({'cycle':cycle,'lead':lead,'temperature':tmeta,'precipitation':pmeta})
    groups={}
    for row in grid:
        groups.setdefault((row['forecast_reference_time'],row['latitude'],row['longitude']),[]).append(row)
    check_groups=[]
    for key,rows in groups.items():
        ordered=sorted(rows,key=lambda x:x['lead_hours'])
        check_groups.append(list(x['lead_hours'] for x in ordered)==list(range(MAX_LEAD+1)) and
            abs(sum(x['precipitation_increment_raw_mm'] for x in ordered[1:])-ordered[-1]['precipitation_cumulative_mm'])<1e-8 and
            all(x['precipitation_interval_start']==ordered[n-1]['valid_time'] for n,x in enumerate(ordered) if n))
    checks={'expected_grid_rows':len(grid)==len(CYCLES)*(MAX_LEAD+1)*25,
        'expected_summary_rows':len(means)==len(CYCLES)*(MAX_LEAD+1)*2,
        'unique_time_grid_keys':len({(x['forecast_reference_time'],x['lead_hours'],x['latitude'],x['longitude']) for x in grid})==len(grid),
        'complete_leads_intervals_and_precipitation_telescope':all(check_groups),
        'finite_plausible_temperature':all(np.isfinite(x['temperature_2m_K']) and 150<x['temperature_2m_K']<350 for x in grid),
```

```
        'nonnegative_cumulative_precipitation':all(x['precipitation_cumulative_mm']>=0 for x in grid),
        'f000_interval_and_increment_missing':all(x['precipitation_increment_mm'] is None and x['precipitation_interval_start'] is None for x in grid if x['lead_hours']==0),
        'unknown_historical_availability_preserved':all(x['available_time'] is None and x['issue_time'] is None for x in grid)}
    if not all(checks.values()):raise ValueError(f'Weather validation failed {checks}')
    dump(RES/'validation.json',{'checks':checks,'passed':all(checks.values()),'grid_point_cycle_series_count':len(groups),
        'independent_disaster_events_evaluated':0,'source_metadata_assertions':'variable/level/reference/valid/interval/units/T-APCP_ETag checked on decoding'})
    writecsv(OUT/'gfs_hourly_source_grid.csv',grid)
    writecsv(OUT/'gfs_hourly_summary.csv',means)
    dump(RES/'decoded_metadata.json',metadata)
    dump(RES/'precipitation_difference_qc.json',{'negative_difference_cells':len(negative),'negative_values':negative,
       'rule':'negative raw values preserved, usable increments missing; no silent clipping or overlapping sum'})
    local_summary,local_sources=local_case()
    sources.extend(local_sources)
    documentation=OUT/'docs/sources.json'
    if documentation.exists():
        for item in json.loads(documentation.read_text()):
            item['cache_path']=str(ROOT/item['path'])
            sources.append(item)
    dump(R/'results/weather_sources.json',{'schema_version':'q3-weather-v1','sources':sources,
       'script_sha256':sha(Path(__file__)),'protocol_sha256':sha(protocol)})
    summary={'schema_version':'q3-weather-v1','cycles':CYCLES,'forecast_hours':list(range(MAX_LEAD+1)),
      'original_grid_cells':25,'grid_rows':len(grid),'summary_rows':len(means),'negative_increment_cells':len(negative),
      'first_valid_time':min(x['valid_time'] for x in grid),'last_valid_time':max(x['valid_time'] for x in grid),
      'historically_available_records':0,'available_time':None,'historically_available_status':'not_established',
      'assumed_latency_hours':[0,3,6,9],
      'raw_cached_grib_bytes':sum(x['bytes'] for x in sources if x.get('byte_range')),
      'all_source_hashes_rechecked':all(sha(Path(x['cache_path']))==x['sha256'] for x in sources),
      'local_observational_case':local_summary,
      'outputs':[{ 'path':str(p.relative_to(ROOT)),'bytes':p.stat().st_size,'sha256':sha(p)} for p in [OUT/'gfs_hourly_source_grid.csv',OUT/'gfs_hourly_summary.csv']],
      'limitations':['0.25 degree forecast grid does not resolve source slope/valley weather',
        'Weather sequences are model forecasts, not observations; no local meteorological skill calibration',
        'Source target exact location remains uncertain; AOI and proxy nearest point are exploratory',
        'Historical first availability unknown; Last-Modified not substituted for it',
        'Hourly increments are differences of independently packed full-run accumulations; negative rounding differences are missing',
        'f000 has no precipitation interval; cumulative zero there is mathematical empty-interval boundary']}
    dump(RES/'summary.json',summary);print(json.dumps(summary,ensure_ascii=False))

def local_case():
    """Keep actual regional station observations distinct from model forecasts."""
    import pandas as pd
    d=ROOT/'track_b/data/processed/temporal'
    inputs=['hourly_time_axis.parquet','hourly_raw_numeric.parquet','hourly_strict_qc0.parquet',
            'hourly_raw_qc.parquet','data_dictionary.json']
    axis=pd.read_parquet(d/inputs[0],columns=['hour_index','time_utc'])
```

```
    selected=axis[(axis.time_utc>='2026-08-22')&(axis.time_utc<'2026-08-30')]
    ix=selected.hour_index
    strict=pd.read_parquet(d/'hourly_strict_qc0.parquet',columns=['MET__Ta_2m']).iloc[ix]
    raw=pd.read_parquet(d/'hourly_raw_numeric.parquet',columns=['MET__Ta_2m','MET__Precip']).iloc[ix]
    qc=pd.read_parquet(d/'hourly_raw_qc.parquet',columns=['MET__Ta_2m']).iloc[ix]
    rows=[]
    for k,(_,x) in enumerate(selected.iterrows()):
        t=strict.iloc[k,0];r=raw.iloc[k,1]
        tval=float(t) if pd.notna(t) else None
        rain=float(r) if pd.notna(r) and np.isfinite(r) and r!=9999.9 else None
        rows.append({'time_utc':x.time_utc.isoformat().replace('+00:00','Z'),
          'station_id':'Gyirong_regional_background',
          'temperature_2m_C_qc0':tval,'temperature_2m_K_qc0':tval+273.15 if tval is not None else None,
          'temperature_qc_raw':qc.iloc[k,0],
          'precipitation_header_mm_raw':rain,'precipitation_interval_start':None,'precipitation_interval_end':None,
          'precipitation_qc_status':'no_QC_column; finite_non_sentinel_only',
          'timezone_status':'UTC+8_assumed_from_same_title_metadata_mirror; file_equivalence_not_hash_verified',
          'timestamp_label_semantics':'unverified_instant_start_end_or_centre',
          'available_time':None,'data_role':'local_observational_ideal_input_only'})
    writecsv(OUT/'local_weather_case.csv',rows)
    records=[{'canonical_url_or_product_id':'repo-input:track_b/data/processed/temporal/'+p,
       'product_version':'frozen-round3-baseline-file-sha256',
       'retrieved_at_utc':None,'actual_read_scope':'2026-08-22 through 29 UTC; temp2m,precip only; dictionary and time axis',
       'bytes':(d/p).stat().st_size,'sha256':sha(d/p),'cache_path':str(d/p),
       'data_role':'local_observational_ideal_input_only','available_time':None} for p in inputs]
    return {'rows':len(rows),'temperature_qc0_present':sum(x['temperature_2m_C_qc0'] is not None for x in rows),
       'precipitation_finite_non_sentinel_present':sum(x['precipitation_header_mm_raw'] is not None for x in rows),
       'precipitation_interval_verified':False,'first_valid_temp':next((x['time_utc'] for x in rows if x['temperature_2m_C_qc0'] is not None),None),
       'last_valid_temp':next((x['time_utc'] for x in reversed(rows) if x['temperature_2m_C_qc0'] is not None),None)},records

def main():
    a=argparse.ArgumentParser();a.add_argument('--collect',action='store_true');a.add_argument('--workers',type=int,default=8);args=a.parse_args()
    assert ROOT==Path('<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q3')
    if args.collect:
        jobs=[(c,h) for c in CYCLES for h in range(MAX_LEAD+1)]
        with concurrent.futures.ThreadPoolExecutor(max_workers=args.workers) as ex:
            for n,records in enumerate(ex.map(lambda t:acquire_one(*t),jobs),1):
                if n%12==0:print(f'acquired {n}/{len(jobs)} forecast hours',flush=True)
    derive()
if __name__=='__main__':main()
```

程序 22：固定历史风险适用性质量划分

来源：track_b/research/q3/round3/scripts/applicability_fixed_risk.py
SHA256: 4d99af9d88d1155b58222f93520fdbe4a424a951057913d5d7ffa5eb015e2b28

```
#!/usr/bin/env python3
"""Fixed-history-risk primary applicability contrast; no physics rerun."""
from datetime import datetime,timezone
from pathlib import Path
import json
import numpy as np
import pandas as pd
import pyarrow.parquet as pq
from applicability_sensitivity import R,O,A,PRIORS,summarize,rec,sha,dump
N=O/'risk_set_revision';POLICY='fixed_history_risk_out_of_domain_to_unknown';DIAG='revoke_out_of_domain_history_classification_diagnostic'

def fixed_summary(g,H,prior):
    r=summarize(g,H,prior,'original_design')
    margin=g.initial_margin_at_issue.to_numpy(float);risk=g.known_at_risk_under_conditional_history.to_numpy(bool)
    point=g.crossing_point_hours_after_issue.to_numpy(float);length=g.followup_hours.to_numpy(int);valid=g.strength_domain_fraction_observed.to_numpy(float)>=1-1e-12
    if prior==PRIORS[0]:w=np.ones(len(g))
    elif prior==PRIORS[1]:w=1/g.groupby('mechanism_id').mechanism_id.transform('size').to_numpy(float)
    else:w=np.exp(-np.clip(margin,0,10))
    w=w/w.sum();original_event=risk&np.isfinite(point)&(point<=H)
    original_no=risk&((np.isfinite(point)&(point>H))|(~np.isfinite(point)&(length>=H)))
    event=original_event&valid;no=original_no&valid;unknown=risk&~(event|no)
    rm=float(w[risk].sum());em=float(w[event].sum());nm=float(w[no].sum());um=float(w[unknown].sum())
    lo=em/rm if rm else None;hi=(em+um)/rm if rm else None;p=lo if rm and um<1e-12 else None;q=[None,None,None]
    if p is not None and em>0:
        x=point[event];ww=w[event];order=np.argsort(x);x=x[order];cw=np.cumsum(ww[order])/ww.sum();q=[float(x[min(np.searchsorted(cw,t),len(x)-1)]) for t in [.025,.5,.975]]
    r.update(applicability_policy=POLICY,probability_start_within_horizon=p,probability_no_start_within_horizon=1-p if p is not None else None,
      probability_start_lower=lo,probability_start_upper=hi,probability_no_start_lower=1-hi if hi is not None else None,probability_no_start_upper=1-lo if lo is not None else None,
      conditional_lower_hours=q[0],conditional_median_hours=q[1],conditional_upper_hours=q[2],
      event_prior_mass=em,no_start_prior_mass=nm,unresolved_future_prior_mass=um,positive_weight_event_count=int(event.sum()),
      positive_weight_unknown_count=int(unknown.sum()),event_physical_design_count=int(g.loc[event,'parameter_set_id'].nunique()),
      event_mass_transferred_to_unknown=float(w[original_event&~valid].sum()),no_start_mass_transferred_to_unknown=float(w[original_no&~valid].sum()),
      already_started_mass_transferred_to_unknown=0.,risk_set_definition='fixed_original_conditional_history',
      identification_status='no_original_risk' if not rm else 'identified_finite_discrete_design_only' if p is not None else 'positive_unknown_in_fixed_original_risk')
    return r

def main():
    N.mkdir(parents=True,exist_ok=True);assert (N/'PROTOCOL.md').exists()
    paths=[A/'parameter_first_passages.parquet',O/'predictions.csv.gz',O/'conservative_mass_transfers.csv.gz',N/'PROTOCOL.md',Path(__file__),R/'scripts/applicability_sensitivity.py']
```

```
dump(N/'input_registration.json',dict(registered_at_utc=datetime.now(timezone.utc).isoformat(),physics_rerun=False,inputs=[rec(p) for p in paths],primary_risk_set='original_conditional_history'))
rows=[];tracks=0
for b in pq.ParquetFile(paths[0]).iter_batches(batch_size=2048):
    f=b.to_pandas();assert len(f)==2048
    common=f.iloc[0][['issue_time','forecast_input_mode','space_object','assumed_latency_hours','forecast_reference_time','forcing_status','followup_hours']].to_dict()
    groups=[('mechanism_policy_mixture','all_delivered_mechanisms','mixture_of_both_law_policies',f)]
    groups.extend((m,mech,law,g) for (m,mech,law),g in f.groupby(['model_id','mechanism_id','law_policy'],sort=True))
    for model,mech,law,g in groups:
      for prior in PRIORS:
       for H in [24,48,72]:rows.append(dict(common,model_id=model,mechanism_id=mech,law_policy=law,**fixed_summary(g,H,prior)))
    tracks+=1
    if tracks%250==0:print(json.dumps({'tracks':tracks,'physics_rerun':False}),flush=True)
fixed=pd.DataFrame(rows);fixed.to_csv(N/'fixed_history_risk_predictions.csv.gz',index=False,compression={'method':'gzip','mtime':0})
old=pd.read_csv(paths[1],low_memory=False);old.loc[old.applicability_policy.eq('out_of_domain_mass_to_unknown'),'applicability_policy']=DIAG
old['risk_set_definition']=np.where(old.applicability_policy.eq(DIAG),'out_of_domain_history_classification_revoked','original_conditional_history')
combined=pd.concat([old,fixed],ignore_index=True);combined.to_csv(N/'five_policy_predictions.csv.gz',index=False,compression={'method':'gzip','mtime':0})
keys=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','model_id','mechanism_id','law_policy','horizon_hours','probability_prior']
origin=old[old.applicability_policy.eq('original_design')];pair=fixed.merge(origin,on=keys,suffixes=('_fixed','_original'),validate='one_to_one')
trans=pd.read_csv(paths[2]);trans['fixed_risk_unknown_mass']=trans.original_future_unknown_mass+trans.event_mass_transferred_to_unknown+trans.no_start_mass_transferred_to_unknown
trans['fixed_risk_known_event_mass']=trans.original_event_mass-trans.event_mass_transferred_to_unknown
trans['fixed_risk_known_no_start_mass']=trans.original_no_start_mass-trans.no_start_mass_transferred_to_unknown
trans['fixed_risk_already_started_mass']=trans.original_already_started_mass;trans['fixed_risk_already_started_mass_transferred_to_unknown']=0.
trans.to_csv(N/'fixed_and_revoked_history_mass_comparison.csv.gz',index=False,compression={'method':'gzip','mtime':0})
mixture=fixed[fixed.model_id=='mechanism_policy_mixture'];mp=mixture.merge(trans,on=[k for k in keys if k not in ['model_id','mechanism_id','law_policy']],validate='one_to_one')
checks={'750_tracks_and_60750_new_fixed_risk_rows':tracks==750 and len(fixed)==60750,'five_named_policies_303750_rows':len(combined)==303750 and combined.applicability_policy.nunique()==5,
   'original_history_risk_mass_unchanged':bool(np.allclose(pair.at_risk_prior_mass_fixed,pair.at_risk_prior_mass_original)),
   'original_already_started_mass_unchanged':bool(np.allclose(pair.already_started_prior_mass_fixed,pair.already_started_prior_mass_original)),
   'no_already_started_mass_transferred':fixed.already_started_mass_transferred_to_unknown.eq(0).all(),
   'fixed_risk_four_way_mass_preserved':bool(np.allclose(fixed[['already_started_prior_mass','event_prior_mass','no_start_prior_mass','unresolved_future_prior_mass']].sum(axis=1),1)),
   'mixture_unknown_matches_explicit_category_transfers':bool(np.allclose(mp.unresolved_future_prior_mass,mp.fixed_risk_unknown_mass)),
   'positive_unknown_never_precise_probability':fixed.loc[fixed.unresolved_future_prior_mass>1e-12,'probability_start_within_horizon'].isna().all(),
   'old_diagnostic_preserved_under_separate_name':len(old[old.applicability_policy.eq(DIAG)])==60750,
```

```
71        'zero_event_support_no_conditional_interval':fixed.loc[fixed.event_physical_design_coun⌋
          ↪  t.eq(0),'conditional_lower_hours'].isna().all()}
72      combined.groupby(['applicability_policy','probability_prior','horizon_hours']).agg(rows=('⌋
        ↪  model_id','size'),identified_probability_rows=('probability_start_within_horizon','cou⌋
        ↪  nt'),
73        maximum_identified_probability=('probability_start_within_horizon','max'),minimum_upper_⌋
          ↪  bound=('probability_start_upper','min'),maximum_upper_bound=('probability_start_uppe⌋
          ↪  r','max')).reset_index().to_csv(N/'comparison_summary.csv',index=False)
74      summary=dict(generated_at_utc=datetime.now(timezone.utc).isoformat(),physics_rerun=False,p⌋
        ↪  rimary_policy=POLICY,secondary_history_revocation_diagnostic=DIAG,
75        fixed_risk_rows=len(fixed),five_policy_rows=len(combined),fixed_precise_probability_rows⌋
          ↪  =int(fixed.probability_start_within_horizon.notna().sum()),
76        fixed_lower_bound_min=float(fixed.probability_start_lower.min()),fixed_upper_bound_min=f⌋
          ↪  loat(fixed.probability_start_upper.min()),fixed_upper_bound_max=float(fixed.probabil⌋
          ↪  ity_start_upper.max()),
77        checks={k:bool(v) for k,v in checks.items()},passed=all(checks.values()),local_truth_eve⌋
          ↪  nts=0,empirical_coverage=None)
78      dump(N/'summary.json',summary);dump(N/'aggregation_validation.json',dict(checks=summary['c⌋
        ↪  hecks'],passed=summary['passed']));assert summary['passed'],summary
79      print(json.dumps(summary),flush=True)
80  if __name__=='__main__':main()
```

程序 23：原先验与适用性分组汇总

来源：track_b/research/q3/round3/scripts/applicability_sensitivity.py
SHA256: 9c5fa15bc8a514bd90c6a335812b159bf15a91cad80176b6a4e8ccb82abff357

```
#!/usr/bin/env python3
"""Post-hoc experimental-domain reweighting of frozen scenarios; no physics run."""
from datetime import datetime,timezone
from pathlib import Path
import hashlib,json,os
import numpy as np
import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq
R=Path(__file__).resolve().parents[1];ROOT=R.parents[3]
O=R/'results/applicability_sensitivity';A=R/'results/mechanistic_replay/analytic_erfc'
Q=R/'inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1/files'
PRIORS=['uniform_design_assumption','equal_mechanism_then_design_assumption','fragile_issue_in
    itial_margin_assumption']
POLICIES=['original_design','all_observed_states_in_experiment_domain','observed_domain_fracti
    on_weight','out_of_domain_mass_to_unknown']
def sha(p):
    h=hashlib.sha256()
    with p.open('rb') as f:
        for b in iter(lambda:f.read(4194304),b''):h.update(b)
    return h.hexdigest()
def dump(p,obj):p.write_text(json.dumps(obj,indent=2,ensure_ascii=False,allow_nan=False)+'\n')
def rec(p):return dict(path=str(p.relative_to(ROOT)),bytes=p.stat().st_size,sha256=sha(p))

def summarize(g,H,prior,policy):
    margin=g.initial_margin_at_issue.to_numpy(float);original_risk=g.known_at_risk_under_condi
        tional_history.to_numpy(bool);risk=original_risk.copy()
    point=g.crossing_point_hours_after_issue.to_numpy(float);length=g.followup_hours.to_numpy(
        int)
    domain=g.strength_domain_fraction_observed.to_numpy(float)
    if prior==PRIORS[0]:base=np.ones(len(g))
    elif prior==PRIORS[1]:base=1/g.groupby('mechanism_id').mechanism_id.transform('size').to_n
        umpy(float)
    else:base=np.exp(-np.clip(margin,0,10))
    base=base/base.sum()
    valid=domain>=1-1e-12
    factor=np.ones(len(g)) if policy in [POLICIES[0],POLICIES[3]] else valid.astype(float) if
        policy==POLICIES[1] else domain
    raw=base*factor;retained=float(raw.sum());w=raw/retained if retained>0 else
        np.zeros(len(g))
    original_event=original_risk&np.isfinite(point)&(point<=H)
    original_no=original_risk&((np.isfinite(point)&(point>H))|(~np.isfinite(point)&(length>=H)
        ))
    original_unknown=original_risk&~(original_event|original_no)
    if policy==POLICIES[3]:
        risk=(original_risk&valid)|~valid
        event=original_event&valid;no=original_no&valid
    else:event=original_event;no=original_no
    unknown=risk&~(event|no);positive=w>0
    rm=float(w[risk].sum());em=float(w[event].sum());nm=float(w[no].sum());um=float(w[unknown]
        .sum())
    lo=em/rm if rm>0 else None;hi=(em+um)/rm if rm>0 else None
    identified=rm>0 and um<1e-12;prob=lo if identified else None;qt=[None,None,None]
    if identified and em>0:
        use=event&positive;x=point[use];ew=w[use];order=np.argsort(x);x=x[order];cw=np.cumsum(
            ew[order])/ew.sum()
```

```
        qt=[float(x[min(np.searchsorted(cw,q),len(x)-1)]) for q in [.025,.5,.975]]
    _,inv=np.unique(g.parameter_set_id.to_numpy(),return_inverse=True)
    design_risk_mass=np.bincount(inv,weights=w*risk);positive_designs=int((design_risk_mass>0).sum())
    ess=float(rm*rm/np.square(design_risk_mass).sum()) if rm>0 else None
    status='no_positive_total_weight' if retained==0 else 'no_positive_at_risk_weight' if rm==0 else 'identified_finite_discrete_design_only' if identified else 'unresolved_future_positive_weight'
    return dict(horizon_hours=H,probability_prior=prior,applicability_policy=policy,
      probability_start_within_horizon=prob,probability_no_start_within_horizon=1-prob if prob is not None else None,
      probability_start_lower=lo,probability_start_upper=hi,probability_no_start_lower=1-hi if hi is not None else None,probability_no_start_upper=1-lo if lo is not None else None,
      conditional_lower_hours=qt[0],conditional_median_hours=qt[1],conditional_upper_hours=qt[2],conditional_interval_level=.95,
      already_started_prior_mass=float(w[~risk].sum()),at_risk_prior_mass=rm,event_prior_mass=em,no_start_prior_mass=nm,unresolved_future_prior_mass=um,
      original_history_already_started_prior_mass=float(base[~original_risk].sum()),
      out_of_domain_originally_already_started_mass=float(base[~original_risk&~valid].sum()),
      out_of_domain_prior_mass=float(base[~valid].sum()),
      original_prior_mass_retained=retained,original_prior_mass_zero_weighted=float(base[factor==0].sum()),
      event_original_prior_mass_removed=float((base[original_event]*(1-factor[original_event])).sum()),
      unknown_original_prior_mass_removed=float((base[original_unknown]*(1-factor[original_unknown])).sum()),
      already_started_original_prior_mass_removed=float((base[~original_risk]*(1-factor[~original_risk])).sum()),
      original_scenario_count=len(g),positive_weight_scenario_count=int(positive.sum()),zero_weight_scenario_count=int((~positive).sum()),
      original_at_risk_count=int(original_risk.sum()),positive_weight_at_risk_count=int((risk&positive).sum()),
      raw_event_scenario_count=int(original_event.sum()),positive_weight_event_count=int((event&positive).sum()),
      positive_weight_unknown_count=int((unknown&positive).sum()),original_unknown_count=int(original_unknown.sum()),
      positive_weight_at_risk_physical_design_count=positive_designs,at_risk_physical_design_ESS=ess,
      event_physical_design_count=int(g.loc[event&positive,'parameter_set_id'].nunique()),identification_status=status,
      conditional_interval_interpretation='posthoc_assumed_design_given_event; no interval without event support',
      data_role='scenario',evaluation_stratum='retrospective_model_applicability_sensitivity',available_time=None,
      domain_rule_uses_future_scenario=True,operational_filter_approved=False,local_material_domain_certified=False,
      independent_observed_events=0,empirical_coverage=None)

def main():
    O.mkdir(parents=True,exist_ok=True)
    assert (O/'PROTOCOL.md').exists()
    inputs=[A/'parameter_first_passages.parquet',A/'predictions.csv.gz',Q/'results/analytic_thermal/parameter_ensemble.csv',
      Q/'results/analytic_thermal/scenario_outcomes.csv',Q/'scripts/mechanism_engine.py',O/'PROTOCOL.md',Path(__file__)]
    assert sha(inputs[0])=='fd557599f6fa54ad21bd7a44dd19aa74f5578ce73533db1ad8aeef19de2688d5'
    assert sha(inputs[1])=='a3dc1602e9d4ea7c80173286f5e78e5ef6cc15b2e00a0b0fb5d33f5ff3fa122d'
    registration=dict(registered_at_utc=datetime.now(timezone.utc).isoformat(),base_research_commit='bb9258d791a6e9735c28528ff4408c7434c3d149',
       analysis_is_posthoc=True,physics_rerun=False,inputs=[rec(p) for p in inputs],policies=POLICIES,original_assumed_priors=PRIORS)
    dump(O/'input_registration.json',registration)
```

```
params=pd.read_csv(inputs[2]);p=params[params.parameter_set_id.eq('q2r3-p0149')].copy()
p['normal_stress_kpa_independent']=p.density_kg_m3*9.80665*p.thickness_normal_m*np.cos(np.deg2rad(p.theta_deg))/1000
p.to_csv(O/'p0149_parameter_audit.csv',index=False)
outcomes=pd.read_csv(inputs[3]);outcomes[outcomes.parameter_set_id.eq('q2r3-p0149')].to_csv(O/'p0149_full_history_outcomes.csv',index=False)
predictions=[];crossings=[];partitions=[];offset=0;tracks=0;writer=None
try:
  for batch in pq.ParquetFile(inputs[0]).iter_batches(batch_size=2048):
    f=batch.to_pandas();assert len(f)==2048 and f.base_scenario_id.nunique()==2048 and f.issue_time.nunique()==1
    assert f.followup_hours.nunique()==1 and f.forcing_status.nunique()==1
    domain=f.strength_domain_fraction_observed.to_numpy(float);assert np.isfinite(domain).all() and ((domain>=0)&(domain<=1)).all()
    audit=f[['issue_time','base_scenario_id','parameter_set_id','mechanism_id','law_policy','history_status','forecast_status','followup_hours','strength_domain_fraction_observed']].copy()
    audit['source_zero_based_row_number']=np.arange(offset,offset+len(f));audit['strict_observed_domain_factor']=domain>=1-1e-12
    audit['fraction_domain_factor']=domain
    tab=pa.Table.from_pandas(audit,preserve_index=False)
    if writer is None:writer=pq.ParquetWriter(O/'parameter_weight_audit.parquet',tab.schema,compression='zstd')
    writer.write_table(tab);offset+=len(f);tracks+=1
    crossed=f[f.crossing_point_hours_after_issue.notna()].copy()
    if len(crossed):crossings.append(crossed)
    groups=[('mechanism_policy_mixture','all_delivered_mechanisms','mixture_of_both_law_policies',f)]
    groups.extend((m,mech,law,g) for (m,mech,law),g in f.groupby(['model_id','mechanism_id','law_policy'],sort=True))
    common=f.iloc[0][['issue_time','forecast_input_mode','space_object','assumed_latency_hours','forecast_reference_time','forcing_status','followup_hours']].to_dict()
    for model,mech,law,g in groups:
      partitions.append(dict(common,model_id=model,mechanism_id=mech,law_policy=law,rows=len(g),
        fully_observed_in_domain=int((g.strength_domain_fraction_observed>=1-1e-12).sum()),
        no_observed_state_in_domain=int((g.strength_domain_fraction_observed==0).sum()),
        partially_in_domain=int(((g.strength_domain_fraction_observed>0)&(g.strength_domain_fraction_observed<1-1e-12)).sum())))
      for policy in POLICIES:
       for prior in PRIORS:
        for H in [24,48,72]:predictions.append(dict(common,model_id=model,mechanism_id=mech,law_policy=law,**summarize(g,H,prior,policy)))
    if tracks%150==0:print(json.dumps({'tracks':tracks,'physics_rerun':False}),flush=True)
finally:
    if writer:writer.close()
cf=pd.concat(crossings,ignore_index=True);cf.to_csv(O/'crossing_support_audit.csv',index=False)
pd.DataFrame(partitions).to_csv(O/'domain_partition_by_track.csv',index=False)
pred=pd.DataFrame(predictions);pred.to_csv(O/'predictions.csv.gz',index=False,compression={'method':'gzip','mtime':0})
keys=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','model_id','mechanism_id','law_policy','horizon_hours','probability_prior']
original=pd.read_csv(inputs[1]);same=pred[pred.applicability_policy.eq('original_design')].merge(original,on=keys,suffixes=('_new','_old'),validate='one_to_one')
fields=['probability_start_within_horizon','probability_no_start_within_horizon','probability_start_lower','probability_start_upper','conditional_lower_hours','conditional_median_hours','conditional_upper_hours','already_started_prior_mass','at_risk_prior_mass','event_prior_mass','no_start_prior_mass','unresolved_future_prior_mass']
deltas={};nullsame={}
for k in fields:
```

```
        x=same[k+'_new'];y=same[k+'_old'];nullsame[k]=bool(x.isna().equals(y.isna()));d=(x-y).abs().dropna();deltas[k]=float(d.max()) if len(d) else 0.
    checks={'full_750_tracks_1536000_rows':tracks==750 and offset==1536000,
      'all_342_crossing_rows_have_zero_observed_domain_fraction':len(cf)==342 and cf.strength_domain_fraction_observed.eq(0).all(),
      'all_crossings_same_single_S_parameter_design':cf.parameter_set_id.nunique()==1 and cf.parameter_set_id.iloc[0]=='q2r3-p0149' and cf.mechanism_id.eq('S').all(),
      'original_60750_distributions_reproduced':len(same)==60750 and all(nullsame.values()) and max(deltas.values())<1e-12,
      'strict_and_fraction_zero_positive_weight_events':pred.loc[pred.applicability_policy.ne('original_design'),'positive_weight_event_count'].eq(0).all(),
      'unknown_positive_weight_never_exact_probability':pred.loc[pred.unresolved_future_prior_mass>1e-12,'probability_start_within_horizon'].isna().all(),
      'empty_positive_risk_has_null_probability':pred.loc[pred.at_risk_prior_mass.eq(0),'probability_start_within_horizon'].isna().all(),
      'zero_event_support_has_no_conditional_time_interval':pred.loc[pred.event_physical_design_count.eq(0),'conditional_lower_hours'].isna().all(),
      'no_historical_availability_or_local_validation_inferred':pred.available_time.isna().all() and pred.empirical_coverage.isna().all()}
    agg=pred.groupby(['applicability_policy','probability_prior','horizon_hours']).agg(rows=('model_id','size'),
      identified_probability_rows=('probability_start_within_horizon','count'),maximum_identified_probability=('probability_start_within_horizon','max'),
      maximum_probability_upper_bound=('probability_start_upper','max'),minimum_positive_weight_risk_designs=('positive_weight_at_risk_physical_design_count','min'),
      zero_risk_rows=('at_risk_prior_mass',lambda x:int(x.eq(0).sum())),unresolved_rows=('probability_start_within_horizon',lambda x:int(x.isna().sum())),
      maximum_event_support=('event_physical_design_count','max')).reset_index();agg.to_csv(O/'comparison_summary.csv',index=False)
    summary=dict(generated_at_utc=datetime.now(timezone.utc).isoformat(),physics_rerun=False,prediction_rows=len(pred),parameter_rows=offset,
      original_distribution_rows_verified=len(same),original_maximum_numeric_difference=deltas,original_null_patterns_identical=nullsame,
      crossing_rows=len(cf),crossing_unique_physical_designs=int(cf.parameter_set_id.nunique()),crossing_domain_fraction_all_zero=bool(cf.strength_domain_fraction_observed.eq(0).all()),
      p0149_normal_stress_kpa=float(p.normal_stress_kpa_independent.iloc[0]),
      checks={k:bool(v) for k,v in checks.items()},passed=all(checks.values()),
      interpretation='Current nonzero finite-design future-onset mass is wholly supported by trajectories outside implemented shear-law experiment domain. Retrospective reweighting removes that support; no safety, calibrated probability, material certification or operational eligibility is established.')
    dump(O/'summary.json',summary);dump(O/'validation.json',dict(checks=summary['checks'],passed=summary['passed'],local_truth_events=0,physics_rerun=False))
    assert summary['passed'],checks;print(json.dumps(summary),flush=True)

if __name__=='__main__':main()
```

程序 24：self-written common sensor/issue/H B-vs-C opportunity construction and group-paired scoring

来源：track_b/research/q3/round3/scripts/common_anchor_comparison.py
SHA256: 32d1d0efa170ff57e347f735f90d780b6d931f75ffa71ccd0e98b1e8b2942083

```
#!/usr/bin/env python3
"""Recompute B at C's exactly matched, predeclared sensor/issue anchors."""
from datetime import datetime,timezone
import argparse
import hashlib,json,math,os
from pathlib import Path
from types import SimpleNamespace
os.environ.setdefault('MPL_IGNORE_SYSTEM_FONTS','1')
os.environ.setdefault('MPLCONFIGDIR','<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/plot_runtime')
import numpy as np
import pandas as pd
import engine_b as B
R=Path(__file__).resolve().parents[1]

def main():
    cli=argparse.ArgumentParser();cli.add_argument('--c-predictions',default='results/engine_c/anchor_predictions.csv');cli.add_argument('--outdir',default='results/common_anchors');args=cli.parse_args()
    protocol=R/'COMMON_ANCHOR_PROTOCOL.md';assert protocol.exists()
    anchor_path=R/args.c_predictions;c=pd.read_csv(anchor_path)
    c=c[c.horizon_role.eq('common_hours') & c.horizon_hours.isin([24,48,72])].copy()
    if c.duplicated(['event_group','site_id','model_id','horizon_hours']).any():raise ValueError('Nonunique C anchor')
    obs=pd.read_csv(R/'inputs/deformation/observations.csv');reg=pd.read_csv(R/'inputs/deformation/site_registry.csv')
    so=R/'inputs/survival/stampa_observations.csv';sr=R/'inputs/survival/stampa_site_registry.csv'
    obs=pd.concat([obs,pd.read_csv(so)],ignore_index=True);reg=pd.concat([reg,pd.read_csv(sr)],ignore_index=True)
    # A case can have no issuable anchor in the selected observation window.
    # Retain it in the all-case opportunity denominator without inventing a
    # prediction or assuming the file start was the deployment date.
    c['status']='ok';c['anchor_publication_status']='issuable_in_selected_record';missing=[]
    for group in sorted(c.event_group.unique()):
      base=c[c.event_group.eq(group)].iloc[0];source=reg[reg.site_id.eq(base.site_id)].iloc[0]
      for model in c.model_id.unique():
       for H in [24,48,72]:
        if ((c.event_group==group)&(c.model_id==model)&(c.horizon_hours==H)).any():continue
        r=base.to_dict();end=float(source.followup_end_hours);issue=end-H
        event=str(source.event_observed).lower()=='true';lower=float(source.failure_lower_hours) if event else math.inf;upper=float(source.failure_upper_hours) if event else math.inf
        r.update(model_id=model,horizon_hours=H,issue_relative_hours=issue,information_cutoff_relative_hours=np.nan,observation_age_hours=np.nan,
          probability_start_within_horizon=np.nan,probability_no_start_within_horizon=np.nan,conditional_lower95_offset_hours=np.nan,
          conditional_median_offset_hours=np.nan,conditional_upper95_offset_hours=np.nan,hazard_per_hour=np.nan,
          failure_lower_offset_hours=lower-issue if event else np.nan,failure_upper_offset_hours=upper-issue if event else np.nan,
          outcome_known=True,event_within_horizon=int(event and upper<=issue+H),status='no_issuable_anchor_in_selected_record',anchor_publication_status='no_issuable_anchor_in_selected_record')
        missing.append(r)
```

```
    if missing:c=pd.concat([c,pd.DataFrame(missing)],ignore_index=True)
    anchors=c.drop_duplicates(['event_group','site_id','horizon_hours']);rows=[]
    for x in anchors.itertuples():
        source=reg[reg.site_id.eq(x.site_id)].iloc[0];s=obs[obs.site_id.eq(x.site_id)].sort_values('time_relative_hours')
        t=s.time_relative_hours.to_numpy(float);d=s.displacement.to_numpy(float)
        raw=pd.to_numeric(s.get('raw_velocity_per_day',pd.Series(np.nan,index=s.index)),errors='coerce').to_numpy(float)
        v=B.causal_vel(t,d,raw);r=source.to_dict()
        if str(r.get('event_observed')).lower()!='true':r['failure_lower_hours']=math.inf;r['failure_upper_hours']=math.inf;r['label_kind']='verified_right_censor_within_monitored_record'
        r['primary_evaluation_eligible']=True;r=SimpleNamespace(**r)
        for model in ['B_IV','B_SS']:
            pred=next(y for y in B.one_prediction(t,v,float(x.issue_relative_hours),model,x.site_id,r,'C_matched_anchor',16,.01) if y['horizon_hours']==x.horizon_hours)
            issue=float(x.issue_relative_hours);lo=pred['conditional_lower_hours']-issue;hi=pred['conditional_upper_hours']-issue
            rows.append(dict(engine='B',model_id=model,event_group=x.event_group,site_id=x.site_id,issue_relative_hours=issue,horizon_hours=x.horizon_hours,
                information_cutoff_relative_hours=pred['last_input_relative_hours'],probability_start_within_horizon=pred['probability_start_within_horizon'],
                probability_no_start_within_horizon=pred['probability_no_start_within_horizon'],conditional_lower_offset_hours=lo,conditional_upper_offset_hours=hi,
                conditional_median_offset_hours=pred['conditional_median_hours']-issue,status=pred['status'],outcome_known=x.outcome_known,event_within_horizon=x.event_within_horizon,
                failure_lower_offset_hours=x.failure_lower_offset_hours,failure_upper_offset_hours=x.failure_upper_offset_hours,right_censored=x.right_censored,
                conditional_interval_level=.95,interval_semantics='model_internal_given_failure_within_H;
                same_C_anchor',anchor_publication_status=x.anchor_publication_status,evaluation_stratum='external_transfer'))
    for x in c.itertuples():rows.append(dict(engine='C',model_id=x.model_id,event_group=x.event_group,site_id=x.site_id,issue_relative_hours=x.issue_relative_hours,horizon_hours=x.horizon_hours,
                information_cutoff_relative_hours=x.information_cutoff_relative_hours,probability_start_within_horizon=x.probability_start_within_horizon,
                probability_no_start_within_horizon=x.probability_no_start_within_horizon,conditional_lower_offset_hours=x.conditional_lower95_offset_hours,
                conditional_upper_offset_hours=x.conditional_upper95_offset_hours,conditional_median_offset_hours=x.conditional_median_offset_hours,status=x.status,anchor_publication_status=x.anchor_publication_status,
                outcome_known=x.outcome_known,event_within_horizon=x.event_within_horizon,failure_lower_offset_hours=x.failure_lower_offset_hours,
                failure_upper_offset_hours=x.failure_upper_offset_hours,right_censored=x.right_censored,conditional_interval_level=.95,interval_semantics=x.interval_semantics,evaluation_stratum='external_transfer'))
    f=pd.DataFrame(rows);known=f.outcome_known.astype(str).str.lower().eq('true');y=pd.to_numeric(f.event_within_horizon,errors='coerce');p=f.probability_start_within_horizon
    f['brier_known']=(p-y).pow(2).where(known);f['brier_lower']=(p-y).pow(2).where(known,np.minimum(p.pow(2),(1-p).pow(2)))
    f['brier_upper']=(p-y).pow(2).where(known,np.maximum(p.pow(2),(1-p).pow(2)))
    positive=known &
    y.eq(1);lo=f.conditional_lower_offset_hours;hi=f.conditional_upper_offset_hours
    f['interval_issued']=lo.notna() & hi.notna();f['coverage_full_label']=((lo<=f.failure_lower_offset_hours)&(hi>=f.failure_upper_offset_hours)).astype(float).where(positive)
    f['coverage_intersects_label']=((lo<=f.failure_upper_offset_hours)&(hi>=f.failure_lower_offset_hours)).astype(float).where(positive)
    f['conditional_width_hours']=hi-lo;f['alert_at_fixed_half']=p.ge(.5).where(p.notna())
    summary=[]
    for (model,H),g in f.groupby(['model_id','horizon_hours'],sort=True):
```

```
    k=g.outcome_known.astype(str).str.lower().eq('true');yy=pd.to_numeric(g.event_within_horizon,errors='coerce');pos=k&yy.eq(1);neg=k&yy.eq(0);pp=g.probability_start_within_horizon
    summary.append(dict(model_id=model,horizon_hours=H,physical_groups=g.event_group.nunique(),known_outcome_groups=int(k.sum()),unknown_outcome_groups=int((~k).sum()),
      probability_scored_groups=int((k&pp.notna()).sum()),abstain_groups=int(pp.isna().sum()),known_event_groups=int(pos.sum()),known_no_event_groups=int(neg.sum()),
      issuable_anchor_groups=int(g.anchor_publication_status.eq('issuable_in_selected_record').sum()),no_issuable_anchor_groups=int(g.anchor_publication_status.ne('issuable_in_selected_record').sum()),
      brier_known_group_equal=g.brier_known.mean(),probability_available_groups=int(pp.notna().sum()),brier_bound_denominator=int(g.brier_lower.notna().sum()),
      brier_lower_mean_conditional_on_probability_issued=g.brier_lower.mean(),brier_upper_mean_conditional_on_probability_issued=g.brier_upper.mean(),
      brier_bound_scope='issued_probabilities_only; unknown_labels_bounded; abstentions_unscored',
      coverage_full_including_no_interval=g.coverage_full_label.mean(),coverage_intersects_including_no_interval=g.coverage_intersects_label.mean(),
      issued_interval_event_groups=int((pos&g.interval_issued).sum()),conditional_interval_width_hours=g.loc[g.interval_issued,'conditional_width_hours'].mean(),
      coverage_full_conditional_on_interval=g.loc[pos&g.interval_issued,'coverage_full_label'].mean(),coverage_intersects_conditional_on_interval=g.loc[pos&g.interval_issued,'coverage_intersects_label'].mean(),
      true_alert_groups=int((pos&pp.ge(.5)).sum()),missed_alert_groups=int((pos&pp.lt(.5)).sum()),event_abstain_groups=int((pos&pp.isna()).sum()),
      false_alert_groups=int((neg&pp.ge(.5)).sum()),noevent_abstain_groups=int((neg&pp.isna()).sum()),evaluation_stratum='external_transfer',
      protocol='exact_same_C_anchor_and_lexical_sensor; retrospective_case_conditioned'))
out=(R/args.outdir).resolve();assert out.is_relative_to(R/'results');out.mkdir(parents=True,exist_ok=True);f.to_csv(out/'predictions.csv',index=False);pd.DataFrame(summary).to_csv(out/'evaluation.csv',index=False)
pairwise=[]
from itertools import combinations
for H,part in f.groupby('horizon_hours'):
  pivot=part.pivot(index='event_group',columns='model_id',values='probability_start_within_horizon')
  truth=part.drop_duplicates('event_group').set_index('event_group');yy=pd.to_numeric(truth.event_within_horizon,errors='coerce');kk=truth.outcome_known.astype(str).str.lower().eq('true')
  for one,two in combinations(sorted(pivot.columns),2):
    use=kk&pivot[one].notna()&pivot[two].notna();n=int(use.sum())
    pairwise.append(dict(model_one=one,model_two=two,horizon_hours=H,total_physical_cases=len(pivot),same_known_probability_groups=n,
      positive_groups=int((yy[use]==1).sum()),negative_groups=int((yy[use]==0).sum()),
      brier_one=float(((pivot.loc[use,one]-yy[use])**2).mean()) if n else None,brier_two=float(((pivot.loc[use,two]-yy[use])**2).mean()) if n else None,
      interpretation='same valid opportunity subset; abstentions and nonissuable case remain in separate total-case denominators'))
pd.DataFrame(pairwise).to_csv(out/'paired_valid_scores.csv',index=False)
checks=dict(same_anchors=f.groupby(['model_id','horizon_hours']).size().nunique()==1,
            no_future_input=bool(((f.information_cutoff_relative_hours<=f.issue_relative_hours+1e-7)|f.information_cutoff_relative_hours.isna()).all()),
            probability_mass_normalized=bool(np.allclose((f.probability_start_within_horizon+f.probability_no_start_within_horizon).dropna(),1)),
            unique_model_group_horizon=not f.duplicated(['model_id','event_group','horizon_hours']).any())
report=dict(generated_at_utc=datetime.now(timezone.utc).isoformat(),protocol_sha256=hashlib.sha256(protocol.read_bytes()).hexdigest(),C_input_path=str(anchor_path.relative_to(R)),C_input_sha256=hashlib.sha256(anchor_path.read_bytes()).hexdigest(),B_code_sha256=hashlib.sha256(Path(B.__file__).read_bytes()).hexdigest(),rows=len(f),physical_groups=f.event_group.nunique(),
```

```
105                 checks={k:bool(v) for k,v in
                    ↪  checks.items()},passed=all(checks.values()),interpretation='Exactly matched
                    ↪  external terminal-collapse anchors; no local first-motion coverage')
106     (out/'validation.json').write_text(json.dumps(report,indent=2)+'\n');assert
        ↪  report['passed'],report;print(json.dumps(report))
107
108 if __name__=='__main__':main()
```

程序 25：OLS/SS models, causalvelocity, MonteCarlo mixture, replay/evaluation andStampa challenge

来源：track_b/research/q3/round3/scripts/engine_b.py
SHA256：84fb535ef46fbdd9e6a1ee8878567e5e158244782b355ec2bfdcd8607db60977

```
#!/usr/bin/env python3
"""Independent past-only inverse velocity and Kalman inverse-velocity models.
Not a port/reproduction of the PFTF R-tool. Model-internal uncertainty only.
"""
from __future__ import annotations
import hashlib,json,math,gzip
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
R=Path(__file__).resolve().parents[1];O=R/'results/engine_b';I=R/'inputs/deformation'
SEED=20261024;NS=512
Z=np.random.default_rng(SEED).standard_normal((NS*8,2))
def causal_vel(t,d,raw=None):
 if raw is not None and np.isfinite(raw).sum()>5:return np.abs(raw)/24
 v=np.full(len(t),np.nan)
 for i in range(4,len(t)):
  x=t[i-4:i+1];y=d[i-4:i+1];x=x-x.mean();den=x@x
  if den>0:v[i]=abs(float(x@(y-y.mean())/den))
 return v

def psd(c):
 c=(c+c.T)/2;v,a=np.linalg.eigh(c);return (a*np.maximum(v,1e-15))@a.T

def fit_roots(t,v,issue,model,w=16,qratio=.01):
 good=(t<=issue)&np.isfinite(v)&(v>0);x=t[good][-w:]-issue;y=1/v[good][-w:]
 if len(y)<6:return np.full(NS,np.inf),'insufficient_positive_velocities',None
 # Normalize inverse velocity scale for numerically stable fitting/filtering.
 scale=max(float(np.median(np.abs(y))),1e-30);y=y/scale
 X=np.c_[np.ones(len(x)),x];beta=np.linalg.lstsq(X,y,rcond=None)[0];res=y-X@beta
 var=max(float(res@res/max(len(x)-2,1)),1e-8)
 cov=psd(var*np.linalg.pinv(X.T@X))
 if model=='B_SS':
  noise=max(float(np.median(np.diff(y)**2)/2),1e-8)
  # Start filter from first three points only; never initialize with later fitted beta.
  x0=x[:3];X0=np.c_[np.ones(3),x0-x0[-1]];b0=np.linalg.lstsq(X0,y[:3],rcond=None)[0]
  m=b0.copy();P=np.diag([noise*10,noise/max(np.median(np.diff(x))**2,1e-6)])
  last=x[2]
  for xx,yy in zip(x[3:],y[3:]):
   delta=xx-last;F=np.array([[1.,delta],[0,1.]]);Q=noise*qratio*np.array([[delta**3/3,delta**2/2],[delta**2/2,delta]])
   m=F@m;P=F@P@F.T+Q;S=P[0,0]+noise;K=P[:,0]/S;m=m+K*(yy-m[0]);P=(np.eye(2)-np.outer(K,[1.,0.]))@P;P=psd(P);last=xx
  delta=-last;F=np.array([[1.,delta],[0,1.]]);m=F@m;P=F@P@F.T+noise*qratio*np.array([[delta**3/3,delta**2/2],[delta**2/2,delta]]) if delta>0 else P
  beta=m;cov=psd(P)
 eigvec=np.linalg.eigh(cov);L=eigvec[1]@np.diag(np.sqrt(np.maximum(eigvec[0],0)));pool=beta+Z@L.T
 if beta[0]<=0:return np.full(NS,np.inf),'fitted_nonpositive_inverse_velocity_abstain',None
 draw=pool[pool[:,0]>0][:NS]
 if len(draw)<NS:return np.full(NS,np.inf),'positive_state_sampling_tail_abstain',None
 roots=np.full(NS,np.inf);ok=draw[:,1]<0;roots[ok]=-draw[ok,0]/draw[ok,1];roots[roots<=0]=np.inf
```

```
 return roots,'ok',{'n_fit':len(y),'positive_state_pool_acceptance':float(np.mean(pool[:,0]>0)),'last_input_relative_hours':float(t[good][-1]),'window_n':w,'inverse_velocity_scale':scale,'q_ratio':qratio}

def one_prediction(t,v,issue,model,site,reg,grid,w=16,qratio=.01):
 roots,status,fit=fit_roots(t,v,issue,model,w,qratio);past=t[t<=issue];age=float(issue-past[-1]) if len(past) else np.nan;pastdif=np.diff(past[-33:]);cad=float(np.median(pastdif[pastdif>0])) if np.any(pastdif>0) else np.inf
 if len(past)<12:status='not_yet_monitored' if not len(past) else 'insufficient_history'
 if age>2*cad:status='stale_observation_abstain'
 lo=float(reg.failure_lower_hours);hi=float(reg.failure_upper_hours);out=[]
 for H in (24,48,72):
  finite=roots[(roots>0)&(roots<=H)];p=len(finite)/NS
  low,med,up=(np.quantile(finite,[.025,.5,.975])+issue) if len(finite) else (np.nan,np.nan,np.nan)
  target=1 if hi<=issue+H else (0 if lo>issue+H else np.nan)
  if status!='ok':p=np.nan;low=med=up=np.nan
  out.append({'engine':'B','model_id':model,'mechanism_id':'alpha2_inverse_velocity_local_trend','experiment_id':'fixed_past_only_w'+str(w)+'_q'+str(qratio),'scenario_id':'external_'+str(reg.event_group),'geometry_version':None,'parameter_version':'B_fixed_v1','event_group':reg.event_group,'site_id':site,'issue_time_relative_hours':float(issue),'issue_time_utc':None,'information_cutoff_relative_hours':float(issue),'horizon_hours':H,'issue_grid':grid,'probability_start_within_horizon':p,'probability_no_start_within_horizon':1-p,'conditional_lower_hours':low,'conditional_median_hours':med,'conditional_upper_hours':up,'conditional_interval_level':.95,'interval_semantics':'conditional_on_linear_zero_within_horizon_model_internal','failure_lower_hours':lo,'failure_upper_hours':hi,'failure_label_kind':reg.label_kind,'target_within_horizon':target,'brier':(p-target)**2 if np.isfinite(target) else np.nan,'coverage_full_label':float(low<=lo and up>=hi) if np.isfinite(low) and target==1 else np.nan,'coverage_intersects_label':float(low<=hi and up>=lo) if np.isfinite(low) and target==1 else np.nan,'coverage_full_label_all_known_events':float(np.isfinite(low) and low<=lo and up>=hi) if target==1 else np.nan,'coverage_intersects_label_all_known_events':float(np.isfinite(low) and low<=hi and up>=lo) if target==1 else np.nan,'conditional_width_hours':up-low,'point_error_to_label_interval_hours':(0 if lo<=med<=hi else min(abs(med-lo),abs(med-hi))) if np.isfinite(med) and target==1 else np.nan,'observation_age_hours':age,'last_input_relative_hours':fit['last_input_relative_hours'] if fit else (float(past[-1]) if len(past) else np.nan),'n_past_observations':len(past),'positive_state_pool_acceptance':fit['positive_state_pool_acceptance'] if fit else np.nan,'window_n':w,'q_ratio':qratio,'status':status,'data_role':'validation_online_past_observations','evaluation_stratum':'external_transfer','forecast_input_mode':None,'forecast_input_mode_reason':'no_weather_input_for_B','sensor_availability_role':'retrospective_sensor_replay_available_time_unknown','input_version':'canonical_deformation_v1','primary_evaluation_eligible':bool(reg.primary_evaluation_eligible),'external_failure_phase':'published_terminal_or_main_collapse','p_prior_note':'joint Gaussian estimation distribution conditional on positive current inverse velocity; not calibrated general hazard probability'})
 return out

def main():
 assert (R/'EXPERIMENT_PROTOCOL.md').exists();assert (R/'DEFORMATION_EVIDENCE.md').exists();O.mkdir(parents=True,exist_ok=True)
 obs=pd.read_csv(I/'observations.csv');regs=pd.read_csv(I/'site_registry.csv');pred=[];sens=[];fold=[];checks=[]
 # Freeze all protocol-defined groups/sensors; no learned shared hyperparameters, calibration separate.
 for reg in regs.itertuples():
  d=obs[obs.site_id==reg.site_id].sort_values('time_relative_hours');t=d.time_relative_hours.to_numpy(float);y=d.displacement.to_numpy(float);v=causal_vel(t,y,d.raw_velocity_per_day.to_numpy(float));L=float(reg.failure_lower_hours)
```

```
 start=max(t[11],L-30*24);issues=np.arange(math.ceil(start),math.floor(L)+1,dtype=float);issues=issues[issues<L]
 native=t[(t>=max(t[11],L-48))&(t<L)]
 for model in ('B_IV','B_SS'):
  for issue in issues:pred.extend(one_prediction(t,v,float(issue),model,reg.site_id,reg,'hourly',16))
  for issue in native:pred.extend(one_prediction(t,v,float(issue),model,reg.site_id,reg,'native_last48h',16))
  # Fixed retrospective lead anchors; same issue used for sensitivities, never pick by accuracy.
  for lead in (24,48,72):
   issue=L-lead
   for w in (8,16,32):
    for q in ((.001,.01,.1) if model=='B_SS' and w==16 else (.01,)):
     rr=one_prediction(t,v,float(issue),model,reg.site_id,reg,'fixed_label_lower_minus_'+str(lead)+'h',w,q)
     for row in rr:row['anchor_lead_hours']=lead
     sens.extend(rr)
 fold.append({'fold_id':'LOSO_'+str(reg.event_group),'test_group':reg.event_group,'test_sensor':reg.site_id,'training_groups':'|'.join(sorted(set(regs[regs.primary_evaluation_eligible].event_group)-{reg.event_group})),'model_parameter_training':'none_shared; fixed_structures; test_sensor_past_only_online_filter','calibration_groups':'other_primary_evaluation_eligible_physical_groups_only; no finite95 calibration performed','event_observed':bool(reg.event_observed),'failure_label_kind':reg.label_kind})
 # Direct truncation check for future perturbation; roots use only t<=issue.
 issue=float(issues[len(issues)//2]) if len(issues) else float(t[11]);v2=v.copy();v2[t>issue]=1e12
 for model in ('B_IV','B_SS'):
  a=fit_roots(t,v,issue,model)[0];b=fit_roots(t,v2,issue,model)[0];checks.append({'test':'future_velocity_rejection','site_id':reg.site_id,'model_id':model,'passed':bool(np.array_equal(np.isinf(a),np.isinf(b)) and np.allclose(a,b,rtol=1e-9,atol=1e-8)),'max_finite_root_difference_hours':float(np.max(np.abs(a[np.isfinite(a)&np.isfinite(b)]-b[np.isfinite(a)&np.isfinite(b)]))) if np.any(np.isfinite(a)&np.isfinite(b)) else 0.,'comparison_tolerance':'rtol1e-9 atol1e-8; infinity masks exactly equal'})
 yy=y.copy();yy[t>issue]+=1e8;vv=causal_vel(t,yy,d.raw_velocity_per_day.to_numpy(float));checks.append({'test':'future_displacement_preprocessing_rejection','site_id':reg.site_id,'passed':bool(np.allclose(vv[t<=issue],v[t<=issue],equal_nan=True))})
 print(reg.site_id,len(issues),len(native),flush=True)
p=pd.DataFrame(pred);s=pd.DataFrame(sens);raw=p.to_csv(index=False,float_format='%.10g').encode();packed=gzip.compress(raw,mtime=0);digest=hashlib.sha256(packed).hexdigest();cache=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/deformation/outputs');cache.mkdir(parents=True,exist_ok=True);cp=cache/('engine_b_predictions_'+digest+'.csv.gz');cp.write_bytes(packed) if not cp.exists() else None;assert hashlib.sha256(cp.read_bytes()).hexdigest()==digest;dump(O/'prediction_cache.json',{'cache_path':str(cp),'bytes':len(packed),'sha256':digest,'uncompressed_csv_sha256':hashlib.sha256(raw).hexdigest(),'rows':len(p),'columns':list(p.columns),'role':'all hourly and native rolling predictions, including stale/abstention and no-event mass','format':'CSV gzip, UTF8; deterministic gzip mtime0; immutable content address','reproduce':'track_b/.venv/bin/python track_b/research/q3/round3/scripts/engine_b.py'});s.to_csv(O/'sensitivity_predictions.csv',index=False,float_format='%.10g');pd.DataFrame(fold).to_csv(O/'fold_membership.csv',index=False)
# Equal sensor within event, equal event across corpus. Anchors unlike hourly duplicates.
evaluations=[]
for name,d in [('hourly',p[p.issue_grid=='hourly']),('native_last48h',p[p.issue_grid=='native_last48h']),('fixed_anchors',s[(s.window_n==16)&(s.q_ratio==.01)])]:
 for (model,H),a in d.groupby(['model_id','horizon_hours']):
  for group,g in a.groupby('event_group'):
```

```
    evaluations.append({'evaluation_set':name,'model_id':model,'horizon_hours':H,'event_group':group,'independent_event_count':1,'sensor_count':g.site_id.nunique(),'publication_count':len(g),'ok_publications':int((g.status=='ok').sum()),'known_binary_publications':int(g.target_within_horizon.notna().sum()),'conditional_event_publications_with_interval':int(g.coverage_full_label.notna().sum()),'known_event_publications_including_no_interval':int(g.coverage_full_label_all_known_events.notna().sum()),'coverage_full_label_all_known_events':g.groupby('site_id').coverage_full_label_all_known_events.mean().mean(),'coverage_intersects_label_all_known_events':g.groupby('site_id').coverage_intersects_label_all_known_events.mean().mean(),'brier_sensor_equal':g.groupby('site_id').brier.mean().mean(),'coverage_full_label_sensor_equal':g.groupby('site_id').coverage_full_label.mean().mean(),'coverage_intersects_label_sensor_equal':g.groupby('site_id').coverage_intersects_label.mean().mean(),'conditional_width_hours_sensor_equal':g.groupby('site_id').conditional_width_hours.mean().mean(),'interval_point_error_hours_sensor_equal':g.groupby('site_id').point_error_to_label_interval_hours.mean().mean(),'coverage_semantics':'publication diagnostic within independent group; event counts not publications'})
e=pd.DataFrame(evaluations);e['primary_evaluation_eligible']=e.event_group!='Veslemannen';e.to_csv(O/'evaluation.csv',index=False,float_format='%.10g')
summary=e[e.primary_evaluation_eligible].groupby(['evaluation_set','model_id','horizon_hours']).agg(independent_groups=('event_group','nunique'),publications=('publication_count','sum'),ok_publications=('ok_publications','sum'),known_binary_group_denominator=('known_binary_publications',lambda
x:int((x>0).sum())),coverage_opportunity_group_denominator=('known_event_publications_including_no_interval',lambda
x:int((x>0).sum())),interval_issued_event_group_denominator=('conditional_event_publications_with_interval',lambda
x:int((x>0).sum())),brier_group_equal=('brier_sensor_equal','mean'),coverage_full_label_group_equal=('coverage_full_label_sensor_equal','mean'),coverage_full_label_all_event_opportunities=('coverage_full_label_all_known_events','mean'),coverage_intersects_all_event_opportunities=('coverage_intersects_label_all_known_events','mean'),coverage_intersects_label_group_equal=('coverage_intersects_label_sensor_equal','mean'),mean_conditional_width_hours=('conditional_width_hours_sensor_equal','mean'),mean_interval_point_error_hours=('interval_point_error_hours_sensor_equal','mean')).reset_index();summary.to_csv(O/'model_comparison.csv',index=False,float_format='%.10g')
# Finite-sample split-conformal feasibility using independent groups, not publication rows.
ng=len(set(regs[regs.primary_evaluation_eligible].event_group));conf=[]
for group in sorted(set(regs[regs.primary_evaluation_eligible].event_group)):
 ncal=ng-1;rank=math.ceil((ncal+1)*.95);conf.append({'test_group':group,'n_calibration_independent_groups':ncal,'rank95':rank,'finite_95_supported':rank<=ncal,'radius95_hours':None,'radius_status':'infinite_rank_exceeds_calibration_count' if rank>ncal else 'not_computed','exchangeability':'not_established_across_heterogeneous_sites','interpretation':'no finite95distribution_free guarantee'})
pd.DataFrame(conf).to_csv(O/'conformal_feasibility.csv',index=False)
tests_t=np.arange(20.,dtype=float);tests_v=1/(50-tests_t);roots=fit_roots(tests_t,tests_v,19.,'B_IV')[0];checks.append({'test':'numerical_alpha2_root_31h','passed':bool(abs(float(np.median(roots[np.isfinite(roots)]))-31)<.05),'evaluation_stratum':'numerical_benchmark'});roots=fit_roots(tests_t,np.ones_like(tests_t),19.,'B_IV')[0];checks.append({'test':'constant_velocity_no_start72h','passed':bool(not
np.any((roots>0)&(roots<=72))),'evaluation_stratum':'numerical_benchmark'})
```

```
probs=p[p.status=='ok'];checks.extend([{'test':'probability_normalization','passed':bool(np.allclose(probs.probability_start_within_horizon+probs.probability_no_start_within_horizon,1))},{'test':'conditional_interval_inside_horizon','passed':bool(((probs.conditional_lower_hours>=probs.issue_time_relative_hours)|(probs.conditional_lower_hours.isna())).all() and ((probs.conditional_upper_hours<=probs.issue_time_relative_hours+probs.horizon_hours)|(probs.conditional_upper_hours.isna())).all())},{'test':'past_input_cutoff','passed':bool(((p.last_input_relative_hours<=p.issue_time_relative_hours+1e-7)|p.last_input_relative_hours.isna()).all())},{'test':'no_overlap_test_calibration_groups','passed':all(x['test_group'] not in x['training_groups'].split('|') for x in fold)},{'test':'no_synthetic_in_empirical_groups','passed':not any('demo' in x.lower() for x in set(regs.event_group))}]);dump(O/'validation.json',{'checks':checks,'passed':all(x['passed'] for x in checks),'independent_physical_groups':ng,'sensors':len(regs),'main_rows':len(p),'sensitivity_rows':len(s),'seed':SEED,'draws_per_parameter_state_distribution':NS,'sampling_constraint':'positive current inverse velocity; nonpositive fitted mean abstains; nonnegative trend retained as no-future-zero mass','inputs':{str(q.relative_to(R)):hashlib.sha256(q.read_bytes()).hexdigest() for q in [I/'observations.csv',I/'site_registry.csv',R/'EXPERIMENT_PROTOCOL.md',I/'engine_b_preregistered.md',I/'engine_b_domain_amendment.json']},'dependencies':{'numpy':np.__version__,'pandas':pd.__version__,'matplotlib':matplotlib.__version__}})
if (O/'predictions.csv').exists():(O/'predictions.csv').unlink() # prior build large CSV now sealed in content-addressed gzip
plot_results(p,s,regs,obs)
supplementary(p,s,regs)
run_censored_challenge()
print(summary.to_string(index=False));print('checks',all(x['passed'] for x in checks))
def dump(p,x):p.write_text(json.dumps(x,indent=2,ensure_ascii=False,allow_nan=False)+'\n')
def plot_results(p,s,regs,obs):
 plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False})
 groups=['Galterengraben','Grabengufer','Preonzo','Brienz_Insel'];fig,axes=plt.subplots(2,2,figsize=(11,7.4),constrained_layout=True)
 for ax,grp in zip(axes.ravel(),groups):
  g=p[(p.event_group==grp)&(p.horizon_hours==72)&(p.issue_grid=='hourly')];site=sorted(set(g.site_id))[0] if len(g) else '';g=g[g.site_id==site]
  for model,c in [('B_IV','#266e9e'),('B_SS','#c46628')]:
   a=g[(g.model_id==model)&(g.issue_time_relative_hours>=g.failure_lower_hours-168)];x=a.issue_time_relative_hours-a.failure_lower_hours;med=a.conditional_median_hours-a.failure_lower_hours;low=a.conditional_lower_hours-a.failure_lower_hours;up=a.conditional_upper_hours-a.failure_lower_hours;ax.plot(x,med,label=model,color=c,lw=1.3);ax.fill_between(x,low,up,color=c,alpha=.13)
  ax.axhline(0,color='black',lw=.8,ls='--');ax.set_title(grp+' (one physical event)');ax.set_xlabel('Issue hours before label lower');ax.set_ylabel('Forecast offset from label lower (h)');ax.set_ylim(-180,80)
 axes[0,0].legend(frameon=False);fig.suptitle('External terminal-collapse replay: conditional 95% model intervals\nNot local first-motion validation; missing segments retain abstention/no in-horizon roots',fontsize=12);fig.savefig(O/'rolling_comparison.png',dpi=160);plt.close(fig)
fig,ax=plt.subplots(figsize=(11,5.3),constrained_layout=True)
a=s[(s.horizon_hours==72)&(s.window_n==16)&(s.q_ratio==.01)&(s.anchor_lead_hours==24)&s.primary_evaluation_eligible]
groups=sorted(set(a.event_group));x=np.arange(len(groups));
for model,delta,c in [('B_IV',-.17,'#266e9e'),('B_SS',.17,'#c46628')]:
 v=[a[(a.event_group==grp)&(a.model_id==model)].probability_start_within_horizon.mean() for grp in groups];ax.bar(x+delta,v,.32,color=c,label=model)
 for xx,vv in zip(x+delta,v):
  if not np.isfinite(vv):ax.plot([xx],[.015],marker='x',color='#9d2d2d');ax.text(xx,.045,'NA',ha='center',fontsize=8,color='#9d2d2d')
ax.set_xticks(x,groups,rotation=42,ha='right');ax.set_ylim(0,1);ax.set_ylabel('p(collapse zero within 72h)');ax.set_title('Fixed retrospective issue = label lower minus 24h; 13 physical groups\nNA = stale / insufficient / not monitored; Veslemannen clock-uncertain excluded');ax.legend(frameon=False);fig.savefig(O/'event_probabilities.png',dpi=160);plt.close(fig)

```

```
def load_predictions():
 j=json.loads((O/'prediction_cache.json').read_text());p=Path(j['cache_path']);assert hashlib.sha256(p.read_bytes()).hexdigest()==j['sha256'];raw=gzip.decompress(p.read_bytes());assert hashlib.sha256(raw).hexdigest()==j['uncompressed_csv_sha256'];import io
 return pd.read_csv(io.BytesIO(raw))
def supplementary(p,s,regs):
 opportunities=[];alerts=[]
 for name,d in [('hourly',p[p.issue_grid=='hourly']),('native_last48h',p[p.issue_grid=='native_last48h']),('fixed_anchors',s[(s.window_n==16)&(s.q_ratio==.01)])]:
  d=d[d.primary_evaluation_eligible]
  for (model,H,grp),a in d.groupby(['model_id','horizon_hours','event_group']):
   known=a.target_within_horizon.notna();scored=a.brier.notna();positive=a.target_within_horizon.eq(1);negative=a.target_within_horizon.eq(0);warning=a.probability_start_within_horizon.ge(.5);L=float(a.failure_lower_hours.iloc[0])
   opportunities.append({'evaluation_set':name,'model_id':model,'horizon_hours':H,'event_group':grp,'independent_event_count':1,'opportunities':len(a),'known_binary_opportunities':int(known.sum()),'probability_scored_opportunities':int(scored.sum()),'abstain_or_no_prediction_opportunities':int(a.probability_start_within_horizon.isna().sum()),'known_positive_opportunities':int(positive.sum()),'positive_with_conditional_interval':int(a.coverage_full_label.notna().sum()),'primary_evaluation_eligible':True})
   alerts.append({'evaluation_set':name,'model_id':model,'horizon_hours':H,'event_group':grp,'threshold':.5,'threshold_status':'fixed descriptive convention, not optimized','known_negative_opportunities':int(negative.sum()),'false_alarm_opportunities':int((negative&warning).sum()),'known_positive_opportunities':int(positive.sum()),'detected_positive_opportunities':int((positive&warning).sum()),'missed_positive_including_abstentions':int((positive&~warning).sum()),'any_alarm_lead_to_label_lower_hours':L-a.loc[warning,'issue_time_relative_hours'].min() if warning.any() else np.nan,'first_correct_in_horizon_alarm_lead_to_label_lower_hours':L-a.loc[positive&warning,'issue_time_relative_hours'].min() if (positive&warning).any() else np.nan,'late_after_failure_hours':None,'late_reason':'replay stops before label lower; no post-failure publication','independent_event_count':1,'false_alarm_interpretation':'within failure-selected monitoring windows only; not general stable-slope false-alarm rate'})
 paired=[]
 for name,d in [('hourly',p[p.issue_grid=='hourly']),('native_last48h',p[p.issue_grid=='native_last48h']),('fixed_anchors',s[(s.window_n==16)&(s.q_ratio==.01)])]:
  d=d[d.primary_evaluation_eligible];keys=['site_id','event_group','issue_grid','issue_time_relative_hours','horizon_hours'];aa=d[d.model_id=='B_IV'][keys+['brier']];bb=d[d.model_id=='B_SS'][keys+['brier']];m=aa.merge(bb,on=keys,suffixes=('_IV','_SS'),validate='one_to_one');m=m.dropna(subset=['brier_IV','brier_SS'])
  for (H,grp),gg in m.groupby(['horizon_hours','event_group']):paired.append({'evaluation_set':name,'horizon_hours':H,'event_group':grp,'paired_publications':len(gg),'paired_brier_IV':gg.groupby('site_id').brier_IV.mean().mean(),'paired_brier_SS':gg.groupby('site_id').brier_SS.mean().mean(),'independent_group_count':1})
 pc=pd.DataFrame(paired);pc.to_csv(O/'paired_model_by_group.csv',index=False,float_format='%.10g');pc.groupby(['evaluation_set','horizon_hours']).agg(paired_group_denominator=('event_group','nunique'),paired_publications=('paired_publications','sum'),paired_brier_IV=('paired_brier_IV','mean'),paired_brier_SS=('paired_brier_SS','mean')).reset_index().to_csv(O/'paired_model_comparison.csv',index=False,float_format='%.10g')
 op=pd.DataFrame(opportunities);op.to_csv(O/'opportunity_denominators.csv',index=False);pd.DataFrame(alerts).to_csv(O/'alert_metrics.csv',index=False,float_format='%.10g')
 q=op.groupby(['evaluation_set','model_id','horizon_hours']).agg(probability_scored_group_denominator=('probability_scored_opportunities',lambda
 x:int((x>0).sum())),probability_scored_opportunities=('probability_scored_opportunities','sum'),abstain_or_no_prediction_opportunities=('abstain_or_no_prediction_opportunities','sum')).reset_index();m=pd.read_csv(O/'model_comparison.csv');m=m.drop(columns=[c for c in
 q.columns if c in m.columns and c not in ['evaluation_set','model_id','horizon_hours']]);
 m.merge(q,on=['evaluation_set','model_id','horizon_hours'],how='left').to_csv(O/'model_comparison.csv',index=False,float_format='%.10g')
```

```
w=s[s.primary_evaluation_eligible].groupby(['model_id','horizon_hours','window_n','q_ratio','anchor_lead_hours','event_group']).agg(brier=('brier','mean'),issued_probability=('probability_start_within_horizon',lambda x:int(x.notna().sum())),opportunities=('probability_start_within_horizon','size'),full_event_coverage_including_abstentions=('coverage_full_label_all_known_events','mean'),interval_width_hours=('conditional_width_hours','mean')).reset_index();w.to_csv(O/'window_noise_comparison.csv',index=False,float_format='%.10g')
# One fixed primary sensor per physical group, selected lexicographically without outcome scores.
primary=regs[regs.primary_evaluation_eligible].sort_values('site_id').drop_duplicates('event_group').set_index('event_group').site_id.to_dict();an=s[(s.window_n==16)&(s.q_ratio==.01)&s.primary_evaluation_eligible].copy();an=an[an.apply(lambda row:row.site_id==primary[row.event_group],axis=1)];an.to_csv(O/'anchor_primary_sensor_predictions.csv',index=False,float_format='%.10g');ac=[]
for (model,H,lead),g in an.groupby(['model_id','horizon_hours','anchor_lead_hours']):
 ev=g.target_within_horizon.eq(1);known=g.target_within_horizon.notna();sc=g.brier.notna();has=g.conditional_lower_hours.notna();ac.append({'model_id':model,'horizon_hours':H,'anchor_lead_hours':lead,'independent_group_opportunities':g.event_group.nunique(),'known_event_groups':int(ev.sum()),'known_binary_groups':int(known.sum()),'probability_scored_groups':int(sc.sum()),'abstain_groups':int(g.probability_start_within_horizon.isna().sum()),'brier_scored_groups':g.brier.mean(),'known_events_with_interval':int((ev&has).sum()),'full_label_covered_events_including_abstentions':int(g.coverage_full_label_all_known_events.fillna(0).sum()),'label_intersected_events_including_abstentions':int(g.coverage_intersects_label_all_known_events.fillna(0).sum()),'full_label_coverage_all_known_events':g.coverage_full_label_all_known_events.mean(),'label_intersection_all_known_events':g.coverage_intersects_label_all_known_events.mean(),'mean_conditional_interval_width_hours':g.loc[ev&has,'conditional_width_hours'].mean(),'sensor_policy':'lexicographic fixed primary per physical group; Brienz715; no performance selection','evaluation_role':'retrospective fixed lead from label lower; not prospective station enrollment'})
pd.DataFrame(ac).to_csv(O/'anchor_event_counts.csv',index=False,float_format='%.10g')
# Audit all returned horizons without treating repeated publications as independent failures.
vp=p[p.status=='ok'].pivot_table(index=['model_id','site_id','issue_grid','issue_time_relative_hours'],columns='horizon_hours',values='probability_start_within_horizon');monotonic=bool(((vp[24]<=vp[48]+1e-12)&(vp[48]<=vp[72]+1e-12)).all())
extra_checks=[]
for model in ('B_IV','B_SS'):
 nt=np.arange(20.,dtype=float);nv=1/(50-nt);a=fit_roots(nt,nv,19.,model)[0];b=fit_roots(nt,nv*100,19.,model)[0];extra_checks.append({'test':'positive_unit_scale_invariant_zero_time_'+model,'passed':bool(np.allclose(a,b,rtol=1e-9,atol=1e-8)),'evaluation_stratum':'numerical_benchmark'})
vj=json.loads((O/'validation.json').read_text());vj['checks']=[x for x in vj['checks'] if x['test']!='horizon_probability_monotonicity' and not x['test'].startswith('positive_unit_scale_invariant')];vj['checks'].extend(extra_checks);vj['checks'].append({'test':'horizon_probability_monotonicity','passed':monotonic,'publication_groups_checked':len(vp)});vj['passed']=all(x['passed'] for x in vj['checks']);vj['prediction_cache']=json.loads((O/'prediction_cache.json').read_text());dump(O/'validation.json',vj)
p.assign(conditional_draw_count=lambda x:(x.probability_start_within_horizon*NS).round()).groupby(['model_id','horizon_hours','issue_grid']).conditional_draw_count.agg(['count','min','median',lambda x:int(((x>0)&(x<20)).sum())]).rename(columns={'<lambda_0>':'positive_conditional_draws_lt_20'}).to_csv(O/'conditional_mc_quality.csv')
# Diagnostic: how far is the last velocity point? Preserve rather than silently ignore sparse windows.
p[p.status=='ok'].assign(last_velocity_age_hours=lambda x:x.issue_time_relative_hours-x.last_input_relative_hours).groupby('site_id').last_velocity_age_hours.agg(['count','median','max']).to_csv(O/'velocity_observation_age.csv')
def run_censored_challenge():
 from types import SimpleNamespace
 ip=R/'inputs/survival/stampa_observations.csv';rp=R/'inputs/survival/stampa_site_registry.csv'
 if not (ip.exists() and rp.exists()):dump(O/'censored_challenge.json',{'status':'not_executed_missing_stable_C_packet'});return
```

```
a=pd.read_csv(ip);rr=pd.read_csv(rp).iloc[0];assert not bool(rr.event_observed) and bool(rr.censoring_verified)
t=a.time_relative_hours.to_numpy(float);v=causal_vel(t,a.displacement.to_numpy(float));end=float(rr.followup_end_hours);reg=SimpleNamespace(failure_lower_hours=float('inf'),failure_upper_hours=float('inf'),label_kind='right_censored_terminal_collapse',event_group=rr.event_group,primary_evaluation_eligible=True);pred=[]
for model in ('B_IV','B_SS'):
 for grid,issues in [('hourly_last30d',np.arange(max(math.ceil(t[11]),math.ceil(end-720)),math.floor(end)+1)),('native_full_censored_exposure',t[11:])]:
  for issue in issues:
   rows=one_prediction(t,v,float(issue),model,rr.site_id,reg,grid)
   for row in rows:
    row['failure_lower_hours']=None;row['failure_upper_hours']=None;row['target_within_horizon']=0 if issue+row['horizon_hours']<=end else np.nan;row['brier']=row['probability_start_within_horizon']**2 if np.isfinite(row['target_within_horizon']) else np.nan;row['evaluation_stratum']='external_transfer';row['unit_note']='original source displacement unit unknown; zero-crossing duration invariant under positive unit scale';row['label_semantics']='right-censored before documented later collapse; no failure time inferred';row['censor_time_hours']=end
   pred.extend(rows)
p=pd.DataFrame(pred);raw=p.to_csv(index=False,float_format='%.10g').encode();packed=gzip.compress(raw,mtime=0);h=hashlib.sha256(packed).hexdigest();path=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/deformation/outputs')/('engine_b_stampa_'+h+'.csv.gz');path.parent.mkdir(parents=True,exist_ok=True)
if not path.exists():path.write_bytes(packed)
dump(O/'censored_challenge.json',{'status':'executed','physical_sites':1,'observed_failure_events':0,'verified_right_censored_sites':1,'calendar_monitoring_days':725,'exposure_duration_hours':end,'source':'C-owned stable Stampa packet, official author MWCD data and primary article; sources/survival_stampa_article.html + results/survival_sources.json','data_inputs':[{'path':str(q.relative_to(R)),'sha256':hashlib.sha256(q.read_bytes()).hexdigest(),'bytes':q.stat().st_size} for q in [ip,rp]],'full_prediction_cache':{'cache_path':str(path),'sha256':h,'bytes':len(packed),'rows':len(p),'uncompressed_csv_sha256':hashlib.sha256(raw).hexdigest()},'unit_note':'unknown source displacement unit; time-to-zero scale invariant; no physical speed threshold claimed','censoring_rule':'binarynegative only if full future horizon ends within verified followup; after-followup outcome unknown'})
p['false_alarm_evaluable']=np.where(p.target_within_horizon.eq(0),p.probability_start_within_horizon.ge(.5).astype(float),np.nan)
p.groupby(['issue_grid','model_id','horizon_hours']).agg(publications=('site_id','size'),probability_predictions=('probability_start_within_horizon','count'),known_negative_opportunities=('target_within_horizon','count'),probability_scored_negative_opportunities=('brier','count'),brier=('brier','mean'),mean_p_start=('probability_start_within_horizon','mean'),false_alarm_count_p_ge_half=('false_alarm_evaluable','sum')).reset_index().to_csv(O/'censored_evaluation.csv',index=False,float_format='%.10g')
if __name__=='__main__':
 import sys
 if '--summarize-only' in sys.argv:
  p=load_predictions();s=pd.read_csv(O/'sensitivity_predictions.csv');regs=pd.read_csv(I/'site_registry.csv');supplementary(p,s,regs);plot_results(p,s,regs,pd.read_csv(I/'observations.csv'));run_censored_challenge()
 else:main()

```

程序 26：self-written dimensionless causal features, piecewise exponential/RBF likelihood, group-heldout tuning, rate scaling and predictions; 512/1024 run entry

来源：track_b/research/q3/round3/scripts/engine_c.py
SHA256: fc46b252ef66f6e489b71e48ca201963d829c7eab3297db536f31054b65380fd

```
#!/usr/bin/env python3
"""Causal landmark survival competition on independently grouped real series.

No fabricated failures/nonfailures.  Poisson piecewise-exponential likelihood
contains the actual at-risk duration and one event at most per physical group.
All preprocessing, regularization selection and cross-fit rate calibration use
the outer training groups only.  Forecast covariates are frozen at issue time.
"""
from __future__ import annotations

import argparse
import csv
import gzip
import hashlib
import io
import json
import math
from pathlib import Path
import sys
from types import SimpleNamespace
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd

ROUND = Path(__file__).resolve().parents[1]
ROOT = ROUND.parents[3]
OUT = ROUND / "results/engine_c"
FEATURES = ["log_relative_velocity", "log_recent_velocity_ratio", "log_monitoring_age_hours"]
MODELS = ["linear_dynamic_hazard", "nonlinear_rbf_hazard"]
PENALTIES = [0.1, 1.0, 10.0]
SEED = 20261024
MAX_NODES = 512
ANCHORS_ONLY = False
EXCLUDE_RECORD_AGE = False
EXECUTED_SOURCE_BYTES = Path(__file__).read_bytes()
EXECUTED_SOURCE_SHA256 = hashlib.sha256(EXECUTED_SOURCE_BYTES).hexdigest()


def minimize_newton(objective, init):
    beta = init.copy()
    success, message = False, "iteration_limit"
    for iteration in range(80):
        loss, grad, hess = objective(beta)
        if np.max(np.abs(grad)) < 1e-7:
            success, message = True, "gradient_converged"
            break
        try:
            step = np.linalg.solve(hess, grad)
        except np.linalg.LinAlgError:
            step = np.linalg.lstsq(hess + np.eye(len(beta)) * 1e-8, grad, rcond=None)[0]
        scale = 1.
        while scale > 1e-8:
```

```
            trial = beta - scale * step
            if objective(trial)[0] <= loss - 1e-4 * scale * float(grad @ step):
                break
            scale *= .5
        beta = trial
        if np.max(np.abs(scale * step)) < 1e-7:
            success, message = True, "step_converged"
            break
    return SimpleNamespace(x=beta, success=success, message=message, iterations=iteration + 1)


def golden_scalar(objective, low=-6., high=6.):
    ratio = (math.sqrt(5.) - 1.) / 2
    a, b = high - ratio * (high - low), low + ratio * (high - low)
    fa, fb = objective(a), objective(b)
    for _ in range(55):
        if fa < fb:
            high, b, fb = b, a, fa
            a = high - ratio * (high - low); fa = objective(a)
        else:
            low, a, fa = a, b, fb
            b = low + ratio * (high - low); fb = objective(b)
    return (low + high) / 2


def prepare_stampa():
    """Canonicalize the fixed public Stampa pre-collapse monitoring record."""
    sources = json.loads((ROUND / "results/survival_sources.json").read_text())
    source = next(r for r in sources if "/data/stampa.csv" in r["canonical_url"] and
    ↪  r.get("status") == 200)
    raw = Path(source["path"])
    if hashlib.sha256(raw.read_bytes()).hexdigest() !=
    ↪  "c11d1589f6ffe7971577844481792837bb58826fd2fc33d04a483e222c774ba5":
        raise RuntimeError("fixed Stampa data hash mismatch")
    data = pd.read_csv(raw)
    dates = pd.to_datetime(data.time)
    t = (dates - dates.iloc[0]).dt.total_seconds() / 3600
    dest = ROUND / "inputs/survival"
    dest.mkdir(parents=True, exist_ok=True)
    # One fixed target sensor.  Base is another part of the same physical chain,
    # not a second independent unfailed site.
    observations = pd.DataFrame({"site_id": "stampa_block4a", "event_group":
    ↪  "Stampa_rocksection4a",
                                 "time_relative_hours": t, "time_utc": "", "displacement":
                                 ↪  data.block4a,
                                 "unit": "source_displacement_unit_not_explicit_in_CSV_dimensi⌋
                                 ↪  onless_features_only",
                                 "observed": data.block4a.notna(), "source_date": data.time})
    observations.to_csv(dest / "stampa_observations.csv", index=False)
    pd.DataFrame([{"site_id": "stampa_block4a", "event_group": "Stampa_rocksection4a",
    ↪  "start_hours": float(t.min()),
                   "end_hours": float(t.max()), "followup_end_hours": float(t.max()),
                   "failure_time_hours": None, "failure_lower_hours": None,
                   ↪  "failure_upper_hours": None,
                   "event_observed": False, "censoring_verified": True,
                   "risk_status": "external_catastrophic_collapse_at_risk_already_slow_moving",
                   "physical_phase":
                   ↪  "catastrophic_collapse_external_not_first_mechanical_motion",
                   "label_source": "https://www.nature.com/articles/s41598-026-48268-x",
                   "label_semantics": "right_censored_at_last_record_2022-11-27;
                   ↪  article_documents_two_stage_collapse_July2023;
                   ↪  no_exact_failure_time_assigned",
```

```
                    "time_origin": str(dates.iloc[0].date()), "time_zone":
                    ↪  "unknown_calendar_dates_only"}]).to_csv(dest /
                    ↪  "stampa_site_registry.csv", index=False)
    jsave(OUT / "stampa_audit.json", {
        "physical_groups": 1, "selected_sensor": "Block4a",
        ↪  "same_chain_sensor_not_independent": "Base",
        "daily_rows": len(data), "finite_block4a_rows": int(data.block4a.notna().sum()),
        "first_date": str(dates.min().date()), "last_date": str(dates.max().date()),
        "relative_span_hours": float(t.max()), "source_csv_sha256": source["sha256"],
        "catastrophic_failure_not_within_record": "article documents collapse in July2023,
        ↪  after November2022 record end",
        "first_mechanical_initiation": "already moving; label not supplied; external collapse
        ↪  phase only",
        "data_licence": "repository MIT software licence does not independently establish
        ↪  third_party_measurement_data_licence",
        "weather_columns": "retained only in raw cache; not survival predictors; soil_moisture
        ↪  and infiltration are HBV-model outputs",
    })


def jsave(path, obj):
    path.write_text(json.dumps(obj, indent=2, ensure_ascii=False, allow_nan=False) + "\n")


def truth(value):
    return str(value).strip().lower() in {"true", "1", "1.0", "yes"}


def finite(value):
    try:
        v = float(value)
        return v if np.isfinite(v) else None
    except (TypeError, ValueError):
        return None


def causal_features(t, d, j):
    """Only indices <= j.  Dimensionless velocities avoid sensor-unit mixing."""
    if j < 5:
        return None
    # Fixed finite past window avoids quadratic memory/runtime on high-frequency
    # records.  No future-fitted smoothing or whole-series scale is used.
    tt, dd = t[max(0, j - 63):j + 1], d[max(0, j - 63):j + 1]
    velocity = np.diff(dd) / np.diff(tt)
    recent = abs(float(np.polyfit(tt[-6:] - tt[-1], dd[-6:], 1)[0]))
    baseline = float(np.median(np.abs(velocity)))
    previous = float(np.median(np.abs(velocity[-11:-5]))) if len(velocity) >= 11 else baseline
    eps = max(baseline * 1e-6, 1e-12)
    values = np.array([
        np.clip(np.log((recent + eps) / (baseline + eps)), -12, 12),
        np.clip(np.log((recent + eps) / (previous + eps)), -12, 12),
        np.log1p(max(t[j] - t[0], 0)),
    ])
    return values[:2] if EXCLUDE_RECORD_AGE else values


def load_data(obs_path, reg_path, administrative_fraction=None, supplement_obs=None,
↪  supplement_reg=None):
    obs, registry = pd.read_csv(obs_path), pd.read_csv(reg_path)
    if supplement_obs is not None:
        obs = pd.concat([obs, pd.read_csv(supplement_obs)], ignore_index=True)
        registry = pd.concat([registry, pd.read_csv(supplement_reg)], ignore_index=True)
    required = {"site_id", "event_group", "time_relative_hours", "displacement"}
```

```
    if not required <= set(obs):
        raise ValueError(f"observations missing {required - set(obs)}")
    selected, audit, intervals = {}, [], []
    for group, rows in registry.groupby("event_group", sort=True):
        # Fixed lexical sensor, independent of event timing or model performance.
        candidates = sorted(rows.site_id.astype(str).unique())
        site = candidates[0]
        r = rows[rows.site_id.astype(str) == site].iloc[0]
        s = obs[obs.site_id.astype(str) == site].copy()
        if "observed" in s:
            s = s[s.observed.map(truth)]
        s["time_relative_hours"] = pd.to_numeric(s.time_relative_hours, errors="coerce")
        s["displacement"] = pd.to_numeric(s.displacement, errors="coerce")
        s = s.dropna(subset=["time_relative_hours", "displacement"])
        s = s.sort_values("time_relative_hours").drop_duplicates("time_relative_hours")
        t, d = s.time_relative_hours.to_numpy(float), s.displacement.to_numpy(float)
        a = {"event_group": str(group), "selected_site_id": site, "candidate_sensors":
        ↪  candidates,
             "selection": "lexicographic_sensor_predefined_not_best_forecast", "observations":
             ↪  len(t)}
        if str(group) == "Veslemannen" or ("primary_evaluation_eligible" in r and not
        ↪  truth(r.get("primary_evaluation_eligible")) and
        ↪  pd.notna(r.get("primary_evaluation_eligible"))):
            a.update(status="excluded",
            ↪  reason="unresolved_curator_clock_alignment_primary_evaluation_ineligible")
            audit.append(a)
            continue
        if len(t) < 7:
            a.update(status="excluded", reason="fewer_than_7_finite_unique_observations")
            audit.append(a)
            continue
        fail = finite(r.get("failure_time_hours"))
        lower = finite(r.get("failure_lower_hours"))
        upper = finite(r.get("failure_upper_hours"))
        lower = lower if lower is not None else fail
        upper = upper if upper is not None else fail
        event = truth(r.get("event_observed", False))
        registry_end = finite(r.get("end_hours"))
        followup = finite(r.get("followup_end_hours"))
        end = followup if followup is not None else (registry_end if registry_end is not None
        ↪  else t[-1])
        # 'event_observed' alone cannot silently extend monitored follow-up.
        if event and (lower is None or upper is None):
            a.update(status="excluded", reason="event_true_without_failure_time")
            audit.append(a)
            continue
        if lower is not None and lower <= t[0]:
            a.update(status="excluded",
            ↪  reason="left_censored_or_already_unstable_at_first_observation")
            audit.append(a)
            continue
        event_in_followup = bool(event and lower is not None and upper is not None and upper <=
        ↪  end + 1e-8)
        original_event_in_followup = event_in_followup
        if event_in_followup:
            end = upper
        label = str(r.get("label_semantics", "unspecified"))
        status = str(r.get("risk_status", ""))
        if status in {"left_censored", "already_unstable", "outcome_unknown", "not_at_risk"}:
            a.update(status="excluded", reason=status, label_semantics=label)
            audit.append(a)
            continue
        if not event_in_followup and not truth(r.get("censoring_verified", False)):
```

```
            a.update(status="excluded", reason="no_verified_event_or_censor_followup", label_semantics=label)
            audit.append(a)
            continue
        if administrative_fraction is not None:
            # Preregistered retrospective censoring view of real observations.
            # The record end is for experiment selection only, never a feature.
            administrative_end = float(t[0] + administrative_fraction * (t[-1] - t[0]))
            keep = t <= administrative_end
            t, d = t[keep], d[keep]
            if len(t) < 7:
                a.update(status="excluded", reason="fewer_than_7_observations_after_administrative_censor")
                audit.append(a)
                continue
            end, event_in_followup, event, fail = administrative_end, False, False, None
            lower, upper = None, None
            r = r.copy()
            r["censoring_verified"] = True
            label += ";administrative_censoring_simulation_not_new_nonfailure"
        if event_in_followup:
            # Observations inside an uncertain failure-label interval must not
            # become confidently pre-event predictor observations.
            keep = t <= lower
            t, d = t[keep], d[keep]
            if len(t) < 7:
                a.update(status="excluded", reason="insufficient_unambiguously_prefailure_observations")
                audit.append(a)
                continue
        t0 = t[5]
        if end <= t0:
            a.update(status="excluded", reason="no_exposure_after_causal_feature_warmup")
            audit.append(a)
            continue
        # Interval left endpoints use the last genuinely observed state.
        risk_stop = lower if event_in_followup else end
        edges = sorted(set([float(v) for v in t[5:] if v < risk_stop] + [float(risk_stop)]))
        original_edges_count = len(edges)
        if len(edges) > MAX_NODES + 1:
            edge_indices = np.unique(np.linspace(0, len(edges) - 1, MAX_NODES + 1).round().astype(int))
            edges = [edges[k] for k in edge_indices]
        rows_c = []
        for left, right in zip(edges[:-1], edges[1:]):
            j = int(np.searchsorted(t, left, side="right") - 1)
            x = causal_features(t, d, j)
            if x is None or right <= left:
                continue
            record = {"event_group": str(group), "site_id": site, "start_hours": left,
                      "end_hours": right, "exposure_hours": right - left,
                      "event": int(event_in_followup and upper == lower and abs(right - end) < 1e-7),
                      "event_kind": "exact" if event_in_followup and upper == lower and abs(right - end) < 1e-7 else "survival",
                      "last_observation_hours": float(t[j]),
                      "observation_age_hours": left - float(t[j])}
            record.update(zip(FEATURES, x))
            rows_c.append(record)
        if event_in_followup and upper > lower:
            j = int(np.searchsorted(t, lower, side="right") - 1)
            x = causal_features(t, d, j)
            record = {"event_group": str(group), "site_id": site, "start_hours": float(lower),
```

```
                    "end_hours": float(upper), "exposure_hours": float(upper - lower),
                    "event": 1, "event_kind": "interval", "last_observation_hours":
                    ↪  float(t[j]),
                    "observation_age_hours": float(lower - t[j])}
            record.update(zip(FEATURES, x)); rows_c.append(record)
        if not rows_c:
            a.update(status="excluded", reason="empty_at_risk_intervals")
            audit.append(a)
            continue
        selected[str(group)] = dict(site=site, t=t, d=d, end=float(end),
        ↪  event=event_in_followup,
                                    fail=fail, lower=lower, upper=upper,
                                    ↪  issue_end=float(risk_stop),
                                    original_event=original_event_in_followup,
                                    time_origin_utc=r.get("time_origin_utc") if
                                    ↪  pd.notna(r.get("time_origin_utc")) else None,
                                    start=float(t0), label_semantics=label)
        intervals.extend(rows_c)
        a.update(status="accepted", feature_start_hours=float(t0),
        ↪  followup_end_hours=float(end),
                 event_observed_in_followup=event_in_followup,
                 censoring_verified=truth(r.get("censoring_verified", False)),
                 sensor_last_observation_hours=float(t[-1]),
                 monitored_model_exposure_hours=float(end - t0), label_semantics=label)
        a.update(failure_lower_hours=lower, failure_upper_hours=upper,
                 guaranteed_survival_exposure_hours=float(risk_stop - t0),
                 interval_label_likelihood_width_hours=float(upper - lower) if
                 ↪  event_in_followup else 0.)
        a.update(original_risk_edge_count=original_edges_count,
        ↪  retained_risk_edge_count=len(edges),
                 max_survival_intervals=MAX_NODES,
                 ↪  original_event_in_followup=original_event_in_followup)
        audit.append(a)
    return selected, pd.DataFrame(intervals), audit


class Hazard:
    def __init__(self, model, penalty):
        self.model, self.penalty = model, penalty

    def basis(self, x):
        z = np.clip((x - self.center) / self.scale, -6, 6)
        parts = [np.ones((len(z), 1)), z]
        if self.model == "nonlinear_rbf_hazard":
            parts.extend([np.exp(-0.5 * (z - k) ** 2) for k in (-1., 0., 1.)])
            parts.extend([(z[:, 0] * z[:, 1])[:, None]])
        return np.column_stack(parts)

    def fit(self, frame):
        x = frame[FEATURES].to_numpy(float)
        self.center = np.median(x, axis=0)
        self.scale = np.maximum(np.quantile(x, .75, axis=0) - np.quantile(x, .25, axis=0), .25)
        a = self.basis(x)
        exposure, y = frame.exposure_hours.to_numpy(float), frame.event.to_numpy(float)
        is_interval = frame.event_kind.to_numpy() == "interval"
        init = np.zeros(a.shape[1])
        init[0] = math.log((sum(y) + .5) / (sum(exposure) + 1.0))
        # Fixed weak intercept prior prevents all-censored degenerate estimates.
        prior_intercept = init[0]
        def objective(beta):
            eta = np.clip(a @ beta, -35, 20)
            expectation = exposure * np.exp(eta)
            reg = np.r_[.001, np.repeat(self.penalty, len(beta) - 1)]
```

```
            delta = beta.copy(); delta[0] -= prior_intercept
            losses = expectation - y * eta
            gradient_eta = expectation - y
            hessian_eta = expectation.copy()
            mu = expectation[is_interval]
            losses[is_interval] = -np.log(-np.expm1(-mu))
            derivative, curvature = np.zeros(len(mu)), np.zeros(len(mu))
            small, middle = mu < 1e-4, (mu >= 1e-4) & (mu < 50.)
            derivative[small] = -1 + mu[small] / 2 - mu[small] ** 2 / 12
            curvature[small] = mu[small] / 2 - mu[small] ** 2 / 6
            ratio = mu[middle] / np.expm1(mu[middle])
            derivative[middle] = -ratio
            curvature[middle] = -ratio + ratio ** 2 * np.exp(mu[middle])
            gradient_eta[is_interval], hessian_eta[is_interval] = derivative,
            ↪  np.maximum(curvature, 0.)
            loss = np.sum(losses) + .5 * np.sum(reg * delta ** 2)
            grad = a.T @ gradient_eta + reg * delta
            hess = a.T @ (hessian_eta[:, None] * a) + np.diag(reg)
            return loss, grad, hess
        fit = minimize_newton(objective, init)
        self.beta, self.success = fit.x, bool(fit.success)
        self.message = str(fit.message)
        return self

    def predict(self, x):
        return np.exp(np.clip(self.basis(np.asarray(x)) @ self.beta, -35, 20))


def score(model, frame):
    hazard = model.predict(frame[FEATURES].to_numpy(float))
    tmp = frame[["event_group", "event", "exposure_hours"]].copy()
    tmp["nll"] = likelihood_terms(hazard, frame)
    # Each physical group contributes equally to hyperparameter ranking.
    return float(tmp.groupby("event_group").nll.sum().mean())


def likelihood_terms(hazard, frame):
    mu = hazard * frame.exposure_hours.to_numpy(float)
    terms = mu - frame.event.to_numpy(float) * np.log(hazard)
    is_interval = frame.event_kind.to_numpy() == "interval"
    terms[is_interval] = -np.log(-np.expm1(-mu[is_interval]))
    return terms


def tune(frame, model):
    groups = sorted(frame.event_group.unique())
    if len(groups) < 4:
        return 1.0, [], "fixed_pre_registered_penalty_insufficient_inner_groups"
    table = []
    for penalty in PENALTIES:
        losses = []
        for held in groups:
            fit = Hazard(model, penalty).fit(frame[frame.event_group != held])
            losses.append(score(fit, frame[frame.event_group == held]))
        table.append({"penalty": penalty, "mean_inner_group_nll": float(np.mean(losses))})
    best = min(table, key=lambda row: (row["mean_inner_group_nll"],
    ↪  -row["penalty"]))["penalty"]
    return best, table, "inner_leave_one_physical_group_out"


def crossfit_scale(frame, model, penalty):
    groups = sorted(frame.event_group.unique())
    if len(groups) < 3:
```

```
        return 1., None, "no_calibration_insufficient_training_groups"
    expected, events, oof = 0., 0., []
    for held in groups:
        fit = Hazard(model, penalty).fit(frame[frame.event_group != held])
        target = frame[frame.event_group == held]
        hazard = fit.predict(target[FEATURES].to_numpy(float))
        expected += float(np.sum(hazard * target.exposure_hours))
        events += float(target.event.sum())
        oof.append((hazard, target))
    # Out-of-training-group predictions, still not an independent calibration set.
    def objective(gamma):
        scale = math.exp(float(gamma))
        return sum(float(likelihood_terms(h * scale, f).sum()) for h, f in oof) + .05 *
        ↪  float(gamma) ** 2
    gamma = golden_scalar(objective)
    return math.exp(gamma), {"events": events, "expected_events_before_scaling": expected,
    ↪  "optimizer_success": True},
    ↪  "training_side_crossfit_likelihood_rate_scale_no_independent_coverage_guarantee"


def issue_grid(s):
    # Entire observed risk span.  Native rows retain short/high frequency scales.
    native = s["t"][(s["t"] >= s["start"]) & (s["t"] < s["issue_end"])]
    hourly = np.arange(math.ceil(s["start"]), math.ceil(s["issue_end"]), 1.)
    values = sorted(set(float(x) for x in np.r_[native, hourly] if x < s["issue_end"]))
    native_set = set(float(x) for x in native)
    if ANCHORS_ONLY:
        keep = set()
        step = float(np.median(np.diff(s["t"])))
        for h in [24., 48., 72., step, step * 3, step * 7]:
            candidates = [v for v in values if v <= s["end"] - h]
            if candidates:
                keep.add(candidates[-1])
        values = sorted(keep)
    return [(v, "native_and_hourly" if v in native_set and abs(v - round(v)) < 1e-9 else
    ↪  "native" if v in native_set else "hourly") for v in values]


def conditional_quantile(hazard, horizon, q):
    p = -math.expm1(-hazard * horizon)
    return -math.log1p(-q * p) / hazard


def censor_km(frame):
    follow = frame.groupby("event_group").agg(start=("start_hours", "min"), end=("end_hours",
    ↪  "max"), event=("event", "max"))
    durations = (follow.end - follow.start).to_numpy(float)
    censored = (1 - follow.event).to_numpy(int)
    table, survival = [], 1.
    for tm in sorted(set(durations)):
        at_risk = int(np.sum(durations >= tm))
        dc = int(np.sum((durations == tm) & (censored == 1)))
        survival *= 1 - dc / at_risk
        table.append((float(tm), float(survival)))
    def g(value, left=False):
        valid = [surv for tm, surv in table if tm < value or (not left and tm <= value)]
        return valid[-1] if valid else 1.
    return g


def main():
    global OUT, MAX_NODES, ANCHORS_ONLY, EXCLUDE_RECORD_AGE, FEATURES
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--observations", type=Path, default=ROUND / "inputs/deformation/observations.csv")
    parser.add_argument("--registry", type=Path, default=ROUND / "inputs/deformation/site_registry.csv")
    parser.add_argument("--supplement-observations", type=Path, default=ROUND / "inputs/survival/stampa_observations.csv")
    parser.add_argument("--supplement-registry", type=Path, default=ROUND / "inputs/survival/stampa_site_registry.csv")
    parser.add_argument("--administrative-censor-fraction", type=float, choices=[.5, .75])
    parser.add_argument("--prepare-stampa", action="store_true", help="offline fixed-source canonicalization only")
    parser.add_argument("--max-nodes", type=int, choices=[512, 1024], default=512)
    parser.add_argument("--anchors-only", action="store_true", help="fixed anchors only for preregistered 1024-grid sensitivity")
    parser.add_argument("--exclude-record-age", action="store_true", help="recommended feature-domain correction: no curated file-age clock")
    args = parser.parse_args()
    if ROOT != Path("<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q3"):
        raise RuntimeError("wrong execution root")
    if not (ROUND / "EXPERIMENT_PROTOCOL.md").exists():
        raise RuntimeError("experiment must be registered before computation")
    if args.prepare_stampa:
        OUT.mkdir(parents=True, exist_ok=True)
        prepare_stampa()
        return
    MAX_NODES, ANCHORS_ONLY = args.max_nodes, args.anchors_only
    EXCLUDE_RECORD_AGE = args.exclude_record_age
    if EXCLUDE_RECORD_AGE:
        if not (ROUND / "results/engine_c/feature_domain_amendment.json").exists():
            raise RuntimeError("record-age correction must be preregistered")
        FEATURES = FEATURES[:2]
        OUT = OUT / "no_record_age"
    elif ANCHORS_ONLY and MAX_NODES == 512:
        OUT = OUT / "record_age_anchor_reference"
    if args.administrative_censor_fraction is not None:
        if not (ROUND / "EXPERIMENT_AMENDMENT_CENSOR.md").exists():
            raise RuntimeError("administrative censor amendment must be registered")
        OUT = OUT / f"administrative_censor_{args.administrative_censor_fraction:g}"
    if MAX_NODES != 512:
        OUT = OUT / f"nodes_{MAX_NODES}"
    OUT.mkdir(parents=True, exist_ok=True)
    use_supplement = args.supplement_observations.exists() and args.supplement_registry.exists()
    selected, intervals, audit = load_data(args.observations, args.registry, args.administrative_censor_fraction,
                                           args.supplement_observations if use_supplement else None,
                                           args.supplement_registry if use_supplement else None)
    jsave(OUT / "exposure_audit.json", audit)
    if intervals.empty or len(selected) < 2:
        jsave(OUT / "execution_status.json", {"status": "no_supervised_competition", "reason": "fewer_than_2_verified_groups", "audited_groups": len(audit), "accepted_groups": len(selected)})
        return
    intervals.to_csv(OUT / "risk_intervals.csv", index=False)
    cache = Path("<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q3/survival")
    cache.mkdir(parents=True, exist_ok=True)
    run_fingerprint = hashlib.sha256((hashlib.sha256(args.observations.read_bytes()).hexdigest() +
                      hashlib.sha256(args.registry.read_bytes()).hexdigest() +
                      hashlib.sha256(Path(__file__).read_bytes()).hexdigest() +
                      str(args)).encode()).hexdigest()
```

```
    prediction_path = cache / f"engine_c_{run_fingerprint[:16]}.partial.csv.gz"
    folds, fits, checks = [], [], []
    prediction_count = 0
    with prediction_path.open("wb") as binary, gzip.GzipFile(filename="", mode="wb",
    ↪  fileobj=binary, mtime=0) as compressed, io.TextIOWrapper(compressed, newline="") as
    ↪  stream:
        writer = None
        for held, s in selected.items():
            train = intervals[intervals.event_group != held]
            test = intervals[intervals.event_group == held]
            train_groups = sorted(train.event_group.unique())
            for group in selected:
                folds.append({"fold_id": f"C_LOSO_{held}", "event_group": group, "site_id":
                ↪  selected[group]["site"], "role": "validation" if group == held else
                ↪  "training_hyperparameter_crossfit_calibration"})
            g = censor_km(train)
            median_step = float(np.median(np.diff(s["t"])))
            horizons = [(float(h), "common_hours") for h in (24, 48, 72)]
            horizons += [(median_step * n, f"native_{n}_median_steps") for n in (1, 3, 7)]
            for model_name in MODELS:
                penalty, tuning, tuning_status = tune(train, model_name)
                fit = Hazard(model_name, penalty).fit(train)
                scale, cal_counts, cal_status = crossfit_scale(train, model_name, penalty)
                n_cal = len(train_groups)
                conformal_rank = math.ceil((n_cal + 1) * .95)
                fits.append({"fold_id": f"C_LOSO_{held}", "held_group": held, "model_id":
                ↪  model_name, "training_groups": train_groups,
                             "feature_specification": "no_record_age_main" if
                             ↪  EXCLUDE_RECORD_AGE else
                             ↪  "record_age_curation_sensitive_challenge",
                             "selected_penalty": penalty, "tuning": tuning, "tuning_status":
                             ↪  tuning_status,
                             "center": fit.center.tolist(), "scale": fit.scale.tolist(),
                             ↪  "coefficients": fit.beta.tolist(),
                             "optimizer_success": fit.success, "optimizer_message":
                             ↪  fit.message,
                             "heldout_survival_nll_raw": score(fit, test),
                             "training_failure_groups":
                             ↪  int(train.groupby("event_group").event.max().sum()),
                             "test_failure_groups": int(test.event.sum()),
                             "test_exposure_hours": float(test.exposure_hours.sum()),
                             "rate_calibration_scale": scale, "calibration_counts": cal_counts,
                             ↪  "calibration_status": cal_status,
                             "conformal_calibration_groups_upper_bound": n_cal,
                             ↪  "conformal_95_rank": conformal_rank,
                             "conformal_95_status": "unbounded_insufficient_groups" if
                             ↪  conformal_rank > n_cal else
                             ↪  "not_run_no_independent_calibration_set"})
                for issue, grid_role in issue_grid(s):
                    j = int(np.searchsorted(s["t"], issue, side="right") - 1)
                    x = causal_features(s["t"], s["d"], j)
                    raw = float(fit.predict(x[None, :])[0])
                    for calibrated in (False, True):
                        hazard = raw * scale if calibrated else raw
                        for horizon, horizon_role in horizons:
                            p = -math.expm1(-hazard * horizon)
                            lo, med, hi = [conditional_quantile(hazard, horizon, q) for q in
                            ↪  (.025, .5, .975)]
                            failure_in_horizon = bool(s["event"] and s["upper"] <= issue +
                            ↪  horizon)
                            known_negative = bool(issue + horizon < s["lower"]) if s["event"]
                            ↪  else bool(issue + horizon <= s["end"])
```

```
                known = failure_in_horizon or known_negative
                y = int(failure_in_horizon) if known else None
                event_lower_offset = s["lower"] - issue if s["event"] else None
                event_upper_offset = s["upper"] - issue if s["event"] else None
                event_offset = event_lower_offset if s["event"] and s["lower"] ==
                ↪  s["upper"] else None
                cutoff = float(s["t"][j])
                origin_utc = s.get("time_origin_utc")
                if origin_utc:
                    origin = datetime.fromisoformat(str(origin_utc).replace("Z",
                    ↪  "+00:00"))
                    issue_utc = (origin + timedelta(hours=issue)).astimezone(timez⌋
                    ↪  one.utc).isoformat()
                    cutoff_utc = (origin + timedelta(hours=cutoff)).astimezone(tim⌋
                    ↪  ezone.utc).isoformat()
                else:
                    issue_utc, cutoff_utc = "", ""
                elapsed = issue - s["start"]
                at_issue = g(elapsed, left=True)
                denominator = g(event_upper_offset + elapsed, left=True) if
                ↪  failure_in_horizon else g(elapsed + horizon)
                ipcw = at_issue / denominator if known and denominator > 1e-12 else
                ↪  None
                row = {"engine": "C", "model_id": model_name + ("_no_record_age" if
                ↪  EXCLUDE_RECORD_AGE else "") + ("_crossfit_calibrated" if
                ↪  calibrated else "_raw"),
                       "feature_specification": "no_record_age_main" if
                       ↪  EXCLUDE_RECORD_AGE else
                       ↪  "record_age_curation_sensitive_challenge",
                       "mechanism_id": "empirical_causal_deformation_hazard",
                       "experiment_id": "C_LOSO_survival_competition",
                       "scenario_id": "C_real_" + held + ("_administrative_" +
                       ↪  str(args.administrative_censor_fraction) if
                       ↪  args.administrative_censor_fraction is not None else
                       ↪  ""),
                       "geometry_version":
                       ↪  "external_sensor_series_no_spatial_geometry",
                       "parameter_version": f"C_LOSO_{held}_penalty_{penalty:g}",
                       "fold_id": f"C_LOSO_{held}", "event_group": held, "site_id":
                       ↪  s["site"],
                       "issue_time_utc": issue_utc, "information_cutoff_utc":
                       ↪  cutoff_utc, "issue_relative_hours": issue,
                       "information_cutoff_relative_hours": cutoff,
                       ↪  "observation_age_hours": issue - cutoff,
                       "issue_grid_role": grid_role, "horizon_hours": horizon,
                       ↪  "horizon_role": horizon_role,
                       "hazard_per_hour": hazard,
                       ↪  "probability_start_within_horizon": p,
                       "probability_no_start_within_horizon": 1 - p,
                       "conditional_median_offset_hours": med,
                       ↪  "conditional_lower95_offset_hours": lo,
                       "conditional_upper95_offset_hours": hi,
                       ↪  "conditional_interval_level": .95,
                       "interval_semantics": "model_internal_given_failure_within_⌋
                       ↪  horizon_frozen_covariates",
                       "event_within_horizon": y, "outcome_known": known,
                       "failure_offset_hours": event_offset,
                       "failure_lower_offset_hours": event_lower_offset,
                       ↪  "failure_upper_offset_hours": event_upper_offset,
                       "followup_remaining_hours": s["end"] - issue,
                       "right_censored": not s["event"],
                       ↪  "ipcw_weight_training_censor_km": ipcw,
```

```
                                    "brier_known_outcome": (p - y) ** 2 if known else None,
                                    "logloss_known_outcome": -(y * math.log(max(p, 1e-15)) + (1 - y) * math.log(max(1 - p, 1e-15))) if known else None,
                                    "brier_lower_bound": (p - y) ** 2 if known else min(p * p, (1 - p) ** 2),
                                    "brier_upper_bound": (p - y) ** 2 if known else max(p * p, (1 - p) ** 2),
                                    "conditional_interval_covers": lo <= event_lower_offset and event_upper_offset <= hi if failure_in_horizon else None,
                                    "conditional_interval_overlaps": lo <= event_upper_offset and event_lower_offset <= hi if failure_in_horizon else None,
                                    "conditional_interval_width_hours": hi - lo,
                                    "evaluation_stratum": "conditional_scenario" if args.administrative_censor_fraction is not None else "external_transfer",
                                    "administrative_censor_fraction": args.administrative_censor_fraction,
                                    "data_role": "site_heldout_validation",
                                    "sampling_scope": "conditional_on_source_case_selection_not_population_risk",
                                    "observation_available_time": "unknown_retrospective_external_benchmark",
                                    "label_semantics": s["label_semantics"],
                                    "input_version": "stage4-r3-real-deformation-canonical"}
                                if writer is None:
                                    writer = csv.DictWriter(stream, fieldnames=list(row))
                                    writer.writeheader()
                                writer.writerow(row)
                                prediction_count += 1
                                if cutoff > issue + 1e-9 or not (-1e-12 <= p <= 1 + 1e-12):
                                    checks.append({"issue": issue, "cutoff": cutoff, "p": p})
            print(f"completed held group {held}: train={len(train_groups)} groups", flush=True)
    pd.DataFrame(folds).to_csv(OUT / "fold_membership.csv", index=False)
    jsave(OUT / "fit_details.json", fits)
    summarize(prediction_path, selected, intervals, prediction_count, checks, args)
    digest = hashlib.sha256(prediction_path.read_bytes()).hexdigest()
    immutable_path = cache / f"engine_c_predictions_{digest}.csv.gz"
    if immutable_path.exists():
        if hashlib.sha256(immutable_path.read_bytes()).hexdigest() != digest:
            raise RuntimeError("cache hash collision or corruption")
        prediction_path.unlink()
    else:
        prediction_path.rename(immutable_path)
    jsave(OUT / "prediction_cache.json", {"path": str(immutable_path), "bytes": immutable_path.stat().st_size,
           "sha256": digest, "role": "all_raw_C_predictions_gzip_CSV_no_rows_dropped", "rows": prediction_count,
           "max_nodes": MAX_NODES, "anchors_only": ANCHORS_ONLY})
    plot_and_grid_comparison()
    feature_domain_comparison()


def feature_domain_comparison():
    base = ROUND / "results/engine_c"
    no_age = base / "no_record_age/anchor_evaluation.csv"
    with_age = base / "record_age_anchor_reference/anchor_evaluation.csv"
    if no_age.exists() and with_age.exists():
        a, b = pd.read_csv(with_age), pd.read_csv(no_age)
        b["model_id"] = b.model_id.str.replace("_no_record_age", "", regex=False)
        merged = a.merge(b, on=["model_id", "horizon_hours"], suffixes=("_record_age", "_no_record_age"))
```

```
        for metric in ["brier", "logloss", "conditional_coverage", "conditional_overlap", "interval_width_hours"]:
            merged[metric + "_noage_minus_record_age"] = merged[metric + "_no_record_age"] - merged[metric + "_record_age"]
        merged["record_age_role"] = "curation_sensitive_challenge"
        merged["no_record_age_role"] = "recommended_feature_domain_for_external_survival_benchmark"
        merged.to_csv(base / "record_age_ablation.csv", index=False)


def plot_and_grid_comparison():
    table = OUT / "anchor_evaluation.csv"
    if table.exists():
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        df = pd.read_csv(table)
        fig, axes = plt.subplots(1, 2, figsize=(11.2, 4.4), layout="constrained")
        colors = {"linear_dynamic_hazard": "#007f7b", "nonlinear_rbf_hazard": "#d26a30"}
        for model, group in df.groupby("model_id"):
            base = model.replace("_raw", "").replace("_crossfit_calibrated", "").replace("_no_record_age", "")
            label = ("Linear" if base.startswith("linear") else "Nonlinear RBF") + (" + cross-fit scale" if "calibrated" in model else " raw")
            g = group.sort_values("horizon_hours")
            style = "--" if "calibrated" in model else "-"
            axes[0].plot(g.horizon_hours, g.brier, style, marker="o", color=colors[base], label=label)
            axes[1].plot(g.horizon_hours, g.conditional_coverage, style, marker="o", color=colors[base])
            axes[1].fill_between(g.horizon_hours, g.conditional_coverage, g.conditional_overlap, color=colors[base], alpha=.08)
        axes[0].set_ylabel("Brier score, observable outcomes")
        axes[1].set_ylabel("Full-label inclusion / overlap bound")
        for ax in axes:
            ax.set_xlabel("Forecast horizon (hours)")
            ax.set_xticks([24, 48, 72]); ax.grid(alpha=.2)
        axes[1].set_ylim(-.03, 1.03)
        axes[0].legend(fontsize=8)
        fig.suptitle("C: held-out physical sites; retrospective fixed anchors\nExternal terminal-collapse target; no 95% coverage guarantee", fontsize=11)
        fig.savefig(OUT / "comparison.png", dpi=160)
        plt.close(fig)
    if MAX_NODES == 1024:
        parent = OUT.parent
        coarse, fine = parent / "fit_details.json", OUT / "fit_details.json"
        if coarse.exists() and fine.exists():
            a, b = pd.DataFrame(json.loads(coarse.read_text())), pd.DataFrame(json.loads(fine.read_text()))
            merged = a[["held_group", "model_id", "heldout_survival_nll_raw"]].merge(
                b[["held_group", "model_id", "heldout_survival_nll_raw"]], on=["held_group", "model_id"], suffixes=("_512", "_1024"))
            merged["nll_1024_minus_512"] = merged.heldout_survival_nll_raw_1024 - merged.heldout_survival_nll_raw_512
            merged.to_csv(OUT / "grid_likelihood_sensitivity.csv", index=False)
        if table.exists() and (parent / "anchor_evaluation.csv").exists():
            a, b = pd.read_csv(parent / "anchor_evaluation.csv"), pd.read_csv(table)
            cols = ["model_id", "horizon_hours", "brier", "conditional_coverage", "conditional_overlap"]
            merged = a[cols].merge(b[cols], on=["model_id", "horizon_hours"], suffixes=("_512", "_1024"))
            for field in ["brier", "conditional_coverage", "conditional_overlap"]:
```

```
                merged[field + "_1024_minus_512"] = merged[field + "_1024"] - merged[field + "_512"]
            merged.to_csv(OUT / "grid_anchor_sensitivity.csv", index=False)


def summarize(path, selected, intervals, count, checks, args):
    # Chunked aggregation retains all raw forecasts without expanding memory.
    grouped, anchors = {}, {}
    group_cols = ["model_id", "event_group", "horizon_role", "horizon_hours"]
    for chunk in pd.read_csv(path, chunksize=100000):
        for key, df in chunk[chunk.horizon_role == "common_hours"].groupby(group_cols, sort=False):
            target = selected[key[1]]["end"] - float(key[3])
            eligible = df[df.issue_relative_hours <= target]
            if len(eligible):
                row = eligible.loc[eligible.issue_relative_hours.idxmax()].to_dict()
                if key not in anchors or row["issue_relative_hours"] > anchors[key]["issue_relative_hours"]:
                    anchors[key] = row
        for key, df in chunk.groupby(group_cols, dropna=False, sort=False):
            rec = grouped.setdefault(key, {"prediction_rows": 0, "known_outcomes": 0, "event_positive_rows": 0,
                                           "brier_sum": 0., "logloss_sum": 0., "brier_lower_sum": 0., "brier_upper_sum": 0.,
                                           "conditional_coverage_count": 0., "conditional_coverage_rows": 0,
                                           "conditional_overlap_count": 0.,
                                           "width_sum": 0., "ipcw_sum": 0., "ipcw_brier_sum": 0., "ipcw_rows": 0})
            known = df.brier_known_outcome.notna()
            coverage = df.conditional_interval_covers.map(lambda v: truth(v) if pd.notna(v) else np.nan)
            overlap = df.conditional_interval_overlaps.map(lambda v: truth(v) if pd.notna(v) else np.nan)
            weights = df.ipcw_weight_training_censor_km
            rec["prediction_rows"] += len(df)
            rec["known_outcomes"] += int(known.sum())
            rec["event_positive_rows"] += int((df.event_within_horizon == 1).sum())
            rec["brier_sum"] += float(df.brier_known_outcome.sum())
            rec["logloss_sum"] += float(df.logloss_known_outcome.sum())
            rec["brier_lower_sum"] += float(df.brier_lower_bound.sum())
            rec["brier_upper_sum"] += float(df.brier_upper_bound.sum())
            rec["conditional_coverage_count"] += float(coverage.sum())
            rec["conditional_overlap_count"] += float(overlap.sum())
            rec["conditional_coverage_rows"] += int(coverage.notna().sum())
            rec["width_sum"] += float(df.conditional_interval_width_hours.sum())
            rec["ipcw_rows"] += int((weights.notna() & known).sum())
            rec["ipcw_sum"] += float(weights[known].sum())
            rec["ipcw_brier_sum"] += float((weights * df.brier_known_outcome).sum())
    rows = []
    for key, r in grouped.items():
        n, nk = r["prediction_rows"], r["known_outcomes"]
        rows.append(dict(zip(group_cols, key)) | r | {
            "brier_observed_outcomes": r["brier_sum"] / nk if nk else None,
            "logloss_observed_outcomes": r["logloss_sum"] / nk if nk else None,
            "brier_all_outcome_lower_bound": r["brier_lower_sum"] / n,
            "brier_all_outcome_upper_bound": r["brier_upper_sum"] / n,
            "conditional_model_interval_coverage_rolling": r["conditional_coverage_count"] / r["conditional_coverage_rows"] if r["conditional_coverage_rows"] else None,
            "conditional_model_interval_overlap_rolling": r["conditional_overlap_count"] / r["conditional_coverage_rows"] if r["conditional_coverage_rows"] else None,
            "conditional_model_interval_mean_width_hours": r["width_sum"] / n,
            "ipcw_brier_mean_over_all_issues": r["ipcw_brier_sum"] / n,
```

```
            "ipcw_assumption":
            ↪  "independent_censoring_transportable_from_training_sites_not_verified",
            "independent_site_denominator": 1,
            "observed_failure_denominator": int(selected[key[1]]["event"]),
            "right_censor_denominator": int(not selected[key[1]]["event"]),
        })
    evaluation = pd.DataFrame(rows)
    evaluation.to_csv(OUT / "evaluation_by_site.csv", index=False)
    aggregate = evaluation.groupby(["model_id", "horizon_role", "horizon_hours"],
    ↪  dropna=False).agg(
        independent_sites=("event_group", "nunique"),
        ↪  observed_failure_sites=("observed_failure_denominator", "sum"),
        right_censored_sites=("right_censor_denominator", "sum"),
        ↪  rolling_prediction_rows=("prediction_rows", "sum"),
        equal_site_brier_known=("brier_observed_outcomes", "mean"),
        ↪  equal_site_logloss_known=("logloss_observed_outcomes", "mean"),
        equal_site_brier_lower_bound=("brier_all_outcome_lower_bound", "mean"),
        equal_site_brier_upper_bound=("brier_all_outcome_upper_bound", "mean"),
        equal_site_conditional_coverage=("conditional_model_interval_coverage_rolling",
        ↪  "mean"),
        equal_site_conditional_overlap=("conditional_model_interval_overlap_rolling", "mean"),
        mean_conditional_width_hours=("conditional_model_interval_mean_width_hours", "mean"))
    aggregate.to_csv(OUT / "evaluation.csv")
    anchor = pd.DataFrame(anchors.values())
    if len(anchor):
        anchor["anchor_semantics"] = "latest_issue_at_or_before_followup_end_minus_horizon_ret
        ↪  rospective_case_conditioned"
        anchor.to_csv(OUT / "anchor_predictions.csv", index=False)
        anchor["failure_group"] = anchor.event_group.map(lambda k: int(selected[k]["event"]))
        anchor["censor_group"] = 1 - anchor.failure_group
        anchor["coverage_numeric"] = anchor.conditional_interval_covers.map(lambda v:
        ↪  float(truth(v)) if pd.notna(v) else np.nan)
        anchor["overlap_numeric"] = anchor.conditional_interval_overlaps.map(lambda v:
        ↪  float(truth(v)) if pd.notna(v) else np.nan)
        anchor.groupby(["model_id", "horizon_hours"]).agg(
            independent_sites=("event_group", "nunique"), failure_sites=("failure_group",
            ↪  "sum"),
            censored_sites=("censor_group", "sum"), known_outcomes=("brier_known_outcome",
            ↪  "count"),
            brier=("brier_known_outcome", "mean"), logloss=("logloss_known_outcome", "mean"),
            conditional_coverage=("coverage_numeric", "mean"),
            ↪  conditional_coverage_sites=("coverage_numeric", "count"),
            conditional_overlap=("overlap_numeric", "mean"),
            interval_width_hours=("conditional_interval_width_hours", "mean")).to_csv(OUT /
            ↪  "anchor_evaluation.csv")
    input_paths = [args.observations, args.registry, ROUND / "EXPERIMENT_PROTOCOL.md",
    ↪  Path(__file__)]
    if args.supplement_observations.exists() and args.supplement_registry.exists():
        input_paths += [args.supplement_observations, args.supplement_registry]
    if args.administrative_censor_fraction is not None:
        input_paths += [ROUND / "EXPERIMENT_AMENDMENT_CENSOR.md"]
    inputs = [{"path": str(p.resolve()), "bytes": len(EXECUTED_SOURCE_BYTES) if p ==
    ↪  Path(__file__) else p.stat().st_size,
               "sha256": EXECUTED_SOURCE_SHA256 if p == Path(__file__) else
               ↪  hashlib.sha256(p.read_bytes()).hexdigest()}
              for p in input_paths]
    summary = {"status": "completed", "seed": SEED, "real_physical_groups": len(selected),
               "observed_failure_groups": sum(s["event"] for s in selected.values()),
               "right_censored_groups": sum(not s["event"] for s in selected.values()),
               "administrative_censor_fraction": args.administrative_censor_fraction,
               "numerical_grid_max_nodes": MAX_NODES, "anchors_only": ANCHORS_ONLY,
               "feature_specification": "no_record_age_main" if EXCLUDE_RECORD_AGE else
               ↪  "record_age_curation_sensitive_challenge",
```

```
                "executed_source_sha256": EXECUTED_SOURCE_SHA256,
                "evaluation_stratum": "conditional_scenario" if
                ↪  args.administrative_censor_fraction is not None else "external_transfer",
                "censored_groups_are_new_real_nonfailure_sites": False,
                "monitored_model_exposure_hours": float(intervals.exposure_hours.sum()),
                "guaranteed_survival_exposure_hours": float(intervals[intervals.event_kind !=
                ↪  "interval"].exposure_hours.sum()),
                "interval_label_width_hours": float(intervals[intervals.event_kind ==
                ↪  "interval"].exposure_hours.sum()),
                "risk_intervals": len(intervals), "raw_prediction_rows": count,
                "model_branches": 4, "future_cutoff_or_probability_violations": checks,
                "inputs": inputs, "utc": "source UTC absent retained native relative clocks",
                "limitations": ["Case-selection conditional probability, not operational
                ↪  population risk",
                                "Rolling rows are dependent; physical group count is the
                                ↪  independent denominator",
                                "Frozen-covariate finite-horizon model; no measured main-region
                                ↪  initiation truth",
                                "Model-internal conditional 95% interval is not a validated 95%
                                ↪  coverage guarantee",
                                "Crossfit scale calibration has no independent calibration-set
                                ↪  guarantee",
                                "Censor KM assumes transferable independent censoring; actual
                                ↪  assumption unverified",
                                "For interval event labels IPCW censor KM uses upper endpoint as
                                ↪  event-removal bound; report score bounds alongside it",
                                "Sensor preprocessing may have used retrospective smoothing in
                                ↪  original data",
                                "No event label or verified censor follow-up means excluded from
                                ↪  likelihood"]}
    jsave(OUT / "summary.json", summary)
    jsave(OUT / "validation_checks.json", {
        "future_information_cutoff_rejected": len(checks) == 0,
        "physical_groups_disjoint_per_outer_fold": all(held not in
        ↪  set(intervals[intervals.event_group != held].event_group) for held in selected),
        "at_most_one_event_per_physical_group":
        ↪  bool((intervals.groupby("event_group").event.sum() <= 1).all()),
        "risk_intervals_positive": bool((intervals.exposure_hours > 0).all()),
        "no_event_after_risk_end": all(float(intervals[intervals.event_group ==
        ↪  group].end_hours.max()) <= s["end"] + 1e-8 for group, s in selected.items()),
        "no_synthetic_rows_added": True,
        "all_sources_real_observations_or_labels": True,
        "probability_normalization_and_bounds": len(checks) == 0,
        "conformal_guarantee_claimed": False,
        "rolling_issues_independent_sample_claimed": False,
    })
    print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

程序 27：join/check frozen seven-run conditional target response

来源：track_b/research/q3/round3/scripts/final_propagation_v4.py
SHA256: 519feb9c39279fa0f1b77e92e42b15a3ea7231757afb56ef7f2ec269e2777a71

```
#!/usr/bin/env python3
"""Join seven already executed conditional cases; seal a portable unique index."""
import argparse
import csv
from datetime import datetime, timezone
import hashlib
from pathlib import Path
import subprocess
import sys

sys.dont_write_bytecode = True
import seal_fullmask_v3 as full

R, ROOT = full.R, full.ROOT
read, write, sha, record = full.read, full.write, full.sha, full.record
BASE = 'c6200b95db074775cc1cb24b163e3caef29f5cd2'
START = '0d5326d6e3428df4d714d8101c41c7c925eca0ef'
O = R / 'results/final_propagation_v4'
D = R / 'results/arrival/compact_delta_v1'
BUNDLE = R / 'exports/final_propagation_join_bundle_v4.json'
PACKETS = [
    ('compact_v2', R/'inputs/q4/bundle_q4-r3-propagation-v2/bundle.json',
    ↪  '9fec7f2083ab43f81b19da2c1cca3cef9b2ad0b5db36441495162412d1e8dda4',39),
    ('fullmask_v3',full.Q,full.QPIN,66),
    ('compact_delta_v1',R/'inputs/q4/bundle_q4-r3-compact-delta-v1/bundle.json','044a9a08ce09e」
    ↪  8b2b5470a6a4351942fb08054e52e9728e2bb4d51a2295841ac',48),
]
DELTA_PRODUCER = '28c67deb49d694e992c005365cf19c73d1db31a89126f64dbe8b6973adba61cb'
SOURCES = [
    ('compact_v2','compact_geometry_v1',R/'results/arrival/propagation_v2',14),
    ('fullmask_v3','fullmask_geometry_v2',full.P,21),
    ('compact_delta_v1','compact_geometry_v1',D,14),
]


def rows(path):
    with path.open(newline='') as stream:
        return list(csv.DictReader(stream))


def csv_write(path, data):
    names = list(dict.fromkeys(k for row in data for k in row))
    with path.open('w',newline='') as stream:
        writer = csv.DictWriter(stream,fieldnames=names)
        writer.writeheader()
        writer.writerows(data)


def flag(value):
    return str(value).lower() == 'true'


def joint_rows():
    combined, inventory, source_records = [], [], []
    for name, geometry, directory, count in SOURCES:
        source = directory/'conditional_arrivals.csv'
        data = rows(source)
```

```
        assert len(data) == count
        assert all('join_source' not in row and 'source_geometry_family' not in row for row in data)
        source_records.append(dict(record(source),join_source=name,source_rows=count))
        combined.extend(dict(row,join_source=name,source_geometry_family=geometry) for row in data)
        byrun = {}
        for row in data:
            byrun.setdefault(row['run_id'],[]).append(row)
        for run_id, records in byrun.items():
            assert len(records) == 7 and len({x['target_id'] for x in records}) == 7
            keys = ['base_scenario_id','mechanism_id','original_upstream_model_id','analytic_onset_model_id','source_first_passage_status',
                    'source_evaluation_onset_status','original_evaluation_window_onset_status','analytic_evaluation_window_onset_status',
                    'source_history_risk_at_evaluation_start','evaluation_window_first_unstable_is_new_start','if_released_propagation',
                    'conditional_onset_lower_utc','conditional_onset_upper_utc','conditional_onset_interpolated_point_utc']
            for key in keys:
                assert len({x.get(key,'') for x in records}) == 1, (run_id,key)
            inventory.append(dict(join_source=name,source_geometry_family=geometry,run_id=run_id,
                                  **{key:records[0].get(key,'') for key in keys},target_records=7,
                                  independent_local_validation_events=0,probability_weight='',qualified_for_prediction=False,
                                  include_in_arrival_envelope=False,initiation_to_release_delay_seconds=''))
    assert len(combined) == 49 and len(inventory) == 7
    assert len({x['run_id'] for x in inventory}) == 7
    assert len({x['base_scenario_id'] for x in inventory}) == 4
    assert {x['mechanism_id'] for x in inventory} == {'S','S_T','S_H','S_T_H'}
    for row in combined:
        for key,value in row.items():
            if key.startswith(('absolute_arrival_','qualified_arrival_')) or key in ['arrival_probability','probability_weight','initiation_to_release_delay_seconds']:
                assert value == '', (row['run_id'],key,value)
        assert not flag(row['include_in_arrival_envelope']) and not flag(row['scientifically_qualified_for_prediction'])
        assert not flag(row['mass_conservation_qualified'])
        assert int(row['independent_local_validation_events']) == 0
        if row['source_first_passage_status'] == 'no_cross_observed_complete':
            assert flag(row['if_released_propagation']) and row['conditional_onset_lower_utc'] == ''
        if row['base_scenario_id'] == 'q2r3-p0255-S_T_H-linear':
            assert row['source_first_passage_status'] == 'point_crossing_linear_between_samples'
            assert row['source_evaluation_onset_status'] == 'left_censored_already_unstable'
            assert not flag(row['evaluation_window_first_unstable_is_new_start'])
    return combined,inventory,source_records


def snapshot():
    raw = subprocess.check_output(['git','ls-tree','-rz',BASE,'--',str(R.relative_to(ROOT))],cwd=ROOT)
    result = []
    for part in raw.split(b'\0'):
        if not part:
            continue
        head,path = part.decode().split('\t',1)
        _,kind,oid = head.split()
```

```
        p = ROOT/path
        data = p.read_bytes()
        assert kind == 'blob' and hashlib.sha1(b'blob
        ↪  '+str(len(data)).encode()+b'\0'+data).hexdigest() == oid,path
        result.append(dict(record(p),git_blob_oid=oid))
    assert len(result) == 740
    return dict(source_commit=BASE,original_increment_base=START,files=result,
                scope='All prior Q3 round3 bytes including the newly frozen fullmask-v3 packet;
                ↪  no producer Git object is required for verification')


def main():
    cli = argparse.ArgumentParser(description=__doc__)
    cli.add_argument('--verify-only',action='store_true')
    args = cli.parse_args()
    packets,approved = [],{}
    for name,path,pin,count in PACKETS:
        assert sha(path) == pin
        packet = read(path)
        assert len(packet['files']) == count
        packets.append((name,path,pin,packet))
        for f in packet['files']:
            if Path(f['path']).is_absolute():
                value = (f['bytes'],f['sha256'])
                assert f['path'] not in approved or approved[f['path']] == value
                approved[f['path']] = value
    assert len(approved) == 23
    for _,_,_,packet in packets:
        for f in packet['files']:
            full.target(f,approved)
    assert sha(PACKETS[-1][1].parent/'upstream_bundle.json') == DELTA_PRODUCER
    full_run = subprocess.run([sys.executable,str(R/'scripts/seal_fullmask_v3.py'),'--verify-o
    ↪  nly'],cwd=ROOT,capture_output=True,text=True)
    assert full_run.returncode == 0,full_run.stdout+full_run.stderr
    for _,_,directory,_ in SOURCES:
        validation = read(directory/'validation.json')
        assert validation['passed']
        for name in ['input_index','output_index']:
            index = directory/(name+'.json')
            assert sha(index) == validation[name+'_sha256']
            for f in read(index)['files']:
                full.target(f,approved)
    review = read(D/'review/final_review.json')
    assert review['passed']
    for f in review['inputs']:
        full.canonical(f,approved)
    qa = read(D/'figure_review.json')
    assert qa['passed'] and qa['figures']
    for f in qa['figures']:
        assert f['actually_opened'] and not Path(f['path']).is_absolute() and
        ↪  sha(ROOT/f['path']) == f['sha256']
    combined,inventory,source_records = joint_rows()
    if not args.verify_only:
        O.mkdir(parents=True,exist_ok=True)
        now = datetime.now(timezone.utc).isoformat()
        write(O/'frozen_prior_files.json',snapshot())
        (O/'fullmask_v3_verification.log').write_text(full_run.stdout+full_run.stderr)
        csv_write(O/'conditional_arrivals_49.csv',combined)
        csv_write(O/'mapped_runs_7.csv',inventory)
        write(O/'summary.json',dict(generated_at_utc=now,mapped_runs=7,mechanical_designs=4,ru
        ↪  n_target_records=49,
              mechanisms=['S','S_T','S_H','S_T_H'],source_tables=source_records,independent_lo
              ↪  cal_validation_events=0,
```

```
      probability_weights=None,qualified_arrival_rows=0,absolute_arrival_rows=0,arriva
      ↪ l_probability_rows=0,
      selected_cases_are_2048_weighted_ensemble=False,initiation_to_release_lag='unkno
      ↪ wn',empirical_coverage=None,
      scientific_adoption='pending_team_review',physical_solver_rerun=False))
merged = {}
registration_count = 0

def add(f,origin):
    nonlocal registration_count
    item = full.canonical(f,approved)
    key = item['path']
    if key not in merged:
        merged[key] = {k:item[k] for k in ['path','bytes','sha256']}
        merged[key].update(roles=[],registrations=[])
    entry = merged[key]
    assert (entry['bytes'],entry['sha256']) == (item['bytes'],item['sha256'])
    for role in item.get('roles',[item.get('role',origin.get('role','registered
    ↪ dependency'))]):
        if role not in entry['roles']:
            entry['roles'].append(role)
    entry['registrations'].append(origin)
    registration_count += 1

for name,path,pin,packet in packets:
    for i,f in enumerate(packet['files']):
        add(f,dict(bundle=str(path.relative_to(ROOT)),bundle_sha256=pin,row_index_0bas
        ↪ ed=i))
    add(record(path),dict(source=name,role='pinned consumer packet'))
indices=[full.O/'input_manifest.json',D/'input_index.json',D/'review/final_review.json
↪ ',
         SOURCES[0][2]/'input_index.json']
for index in indices:
    data = read(index)
    for i,f in enumerate(data.get('files',data.get('inputs',[]))):
        add(f,dict(index=str(index.relative_to(ROOT)),index_sha256=sha(index),row_inde
        ↪ x_0based=i))
for p in [D/'validation.json',D/'input_index.json',D/'output_index.json',D/'review/fin
↪ al_review.json',
          full.O/'packet_validation.json',full.O/'output_manifest.json',R/'exports/ful
          ↪ lmask_propagation_join_bundle_v3.json',
          SOURCES[0][2]/'validation.json',SOURCES[0][2]/'output_index.json',Path(__fil
          ↪ e__).resolve(),
          PACKETS[-1][1].parent/'upstream_bundle.json',*[directory/'conditional_arriva
          ↪ ls.csv' for _,_,directory,_ in SOURCES]]:
    add(record(p),dict(source='final integration',role='exact frozen or reviewed
    ↪ source'))
inputs=list(merged.values())
write(O/'input_manifest.json',dict(generated_at_utc=now,files=inputs,unique_file_count
↪ =len(inputs),registration_count=registration_count,
      duplicate_registration_count=registration_count-len(inputs),approved_unique_cach
      ↪ e_files=23,
      normalization_scope='Repository-relative paths; exact cache allowlist only. Merge
      ↪ canonical path with identical bytes/SHA; preserve all roles and index/bundle
      ↪ row provenance. Never extract historical source_path/upstream_path as a new
      ↪ read.',
      source_base_commit=BASE,original_increment_base=START,physics_rerun=False))
bundle_files=[p for p in sorted(D.rglob('*')) if p.is_file() and '__pycache__' not in
↪ p.parts and p.suffix!='.log']
bundle_files += [O/'conditional_arrivals_49.csv',O/'mapped_runs_7.csv',O/'summary.json
↪ ',O/'input_manifest.json',O/'frozen_prior_files.json',R/'FINAL_PROPAGATION_HANDOFF
↪ .md',R/'exports/fullmask_propagation_join_bundle_v3.json']
```

```
    write(BUNDLE,dict(bundle_version='q3-r3-final-propagation-v4',producer_question='q3',created_at_utc=now,
          files=[dict(record(p),role=str(p.relative_to(R))) for p in bundle_files],source_base_commit=BASE,original_increment_base=START,
          mapped_runs=7,mechanical_designs=4,run_target_records=49,local_truth_events_added=0,empirical_coverage=None,
          physics_rerun=False,accepted_arrival_envelope_runs=[],arrival_probability=None,absolute_arrival=None,
          limitations=['Selected mapped runs are not independent events or the2048-weighted forecast ensemble',
                       'Unknown initiation-to-bulk-release delay; qualified arrivals and probabilities remain null',
                       'Global mechanical crossing and evaluation-window left censoring are separate states',
                       'No forced join of general Q4 scenarios without a Q2 parent identity']))
    exclude={O/'output_manifest.json',O/'packet_validation.json'}
    directories=[D,O,PACKETS[-1][1].parent]
    paths=[p for folder in directories for p in folder.rglob('*') if p.is_file() and p not in exclude and '__pycache__' not in p.parts]
    paths += [BUNDLE,R/'FINAL_PROPAGATION_HANDOFF.md',R/'scripts/join_compact_delta_v1.py',Path(__file__).resolve()]
    paths += [full.target(f,approved) for f in read(full.O/'output_manifest.json')['files']]
    paths += [full.O/'output_manifest.json',full.O/'packet_validation.json']
    write(O/'output_manifest.json',dict(generated_at_utc=now,original_increment_base=START,frozen_fullmask_commit=BASE,
          files=[record(p) for p in sorted(set(paths))],scope='Complete additions since original increment base, including immutable fullmask outputs plus compact delta and final metadata',
          excluded={'output_manifest.json':'self reference','packet_validation.json':'avoid cycle','__pycache__':'transient bytecode'}))
    checks=dict(all_153_approved_registrations_verified=True,seven_runs_four_designs_49_rows=True,all_source_columns_retained=True,
          no_qualified_arrival_absolute_or_probability=True,delta_consumer_checks=read(D/'validation.json')['passed'],delta_independent_review=review['passed'],
          fullmask_packet_offline_verification=True,prior_740_tracked_files_preserved=True,unique_portable_master_inputs=True,delta_final_figure_opened=True)
    write(O/'packet_validation.json',dict(generated_at_utc=now,passed=all(checks.values()),checks=checks,input_manifest_sha256=sha(O/'input_manifest.json'),
          output_manifest_sha256=sha(O/'output_manifest.json'),frozen_prior_manifest_sha256=sha(O/'frozen_prior_files.json'),bundle_sha256=sha(BUNDLE),
          source_base_commit=BASE,original_increment_base=START,physics_rerun=False))
packet = read(O/'packet_validation.json')
assert packet['passed'] and all(packet['checks'].values())
for key,p in [('input_manifest_sha256',O/'input_manifest.json'),('output_manifest_sha256',O/'output_manifest.json'),('frozen_prior_manifest_sha256',O/'frozen_prior_files.json'),('bundle_sha256',BUNDLE)]:
    assert sha(p) == packet[key]
for path in [O/'input_manifest.json',O/'output_manifest.json',O/'frozen_prior_files.json',BUNDLE]:
    full.check_unique(read(path)['files'],approved)
projected=rows(O/'conditional_arrivals_49.csv')
assert len(projected) == len(combined)
for source,actual in zip(combined,projected):
    assert all(actual[key] == value for key,value in source.items())
assert rows(O/'mapped_runs_7.csv') == [{key:str(value) for key,value in row.items()} for row in inventory]
master=read(O/'input_manifest.json')
assert master['unique_file_count'] == len(master['files'])
assert master['registration_count'] == sum(len(f['registrations']) for f in master['files'])
```

```
    print(full.json.dumps(dict(passed=True,mode='verify_only' if args.verify_only else 'seal_final_increment',mapped_runs=7,mechanical_designs=4,target_rows=49,
                unique_inputs=len(master['files']),input_manifest_sha256=sha(O/'input_manifest.json'),output_manifest_sha256=sha(O/'output_manifest.json'),bundle_sha256=sha(BUNDLE))))


if __name__ == '__main__':
    main()
```

程序 28：self-written final integration of already computed paired scores

来源：track_b/research/q3/round3/scripts/integrate_results.py
SHA256：bc6404976b94e5b25678471b3d9cf2a3075e006a6c5fd5206bee8eed171fb9f8

```
#!/usr/bin/env python3
"""Project stable, completed engine outputs into clearly stratified common tables."""
from datetime import datetime,timezone
import hashlib,json
from pathlib import Path
import numpy as np
import pandas as pd
R=Path(__file__).resolve().parents[1]
VERSION=json.dumps({'baseline_commit':'904d7257b899b9f71d1ac9e0eab7479e459eab52','manifest_sha256':'fd755fab38df285a67c986026859be5f635a314aa3f79f2d247a2626246c3a53'})

def main():
    b=R/'results/engine_b';c=R/'results/engine_c';a=R/'results/mechanistic_replay'
    for p in [b/'validation.json',a/'validation.json',a/'v2/validation.json',a/'analytic_erfc/validation.json',c/'no_record_age/validation_checks.json',R/'results/common_anchors_no_age/validation.json']:
        doc=json.loads(p.read_text())
        if 'passed' in doc:assert doc['passed'],str(p)
        else:
            positive=['future_information_cutoff_rejected','physical_groups_disjoint_per_outer_fold','at_most_one_event_per_physical_group','risk_intervals_positive','no_event_after_risk_end','no_synthetic_rows_added','all_sources_real_observations_or_labels','probability_normalization_and_bounds']
            assert all(doc[k] for k in positive) and not doc['conformal_guarantee_claimed'] and not doc['rolling_issues_independent_sample_claimed'],str(p)
    acsv=a/'analytic_erfc/predictions.csv.gz';af=pd.read_csv(acsv)
    af=af[af.probability_prior.eq('uniform_design_assumption')].copy()
    af=af.rename(columns={'conditional_lower_hours':'conditional_lower_offset_hours','conditional_median_hours':'conditional_median_offset_hours','conditional_upper_hours':'conditional_upper_offset_hours'})
    af['prediction_projection']='all_75_issues_all_weather_tracks_uniform_assumed_prior;other_priors_in_engine_raw'
    af['physical_phase']='first_mechanical_initiation_at_source';af['event_id']='B20260826_Chhochen'
    af['engine_input_version']=af.input_version
    af['model_version']=af.input_bundle_version
    af['input_version']=[json.dumps(dict(json.loads(VERSION),mechanism_bundle=v,mechanism_bundle_sha256=h,weather_sha256=w)) for v,h,w in zip(af.input_bundle_version,af.input_bundle_sha256,af.input_weather_sha256)]
    af['space_object']=[json.dumps({'weather_support':s,'crs':'EPSG:4326','gfs_resolution_degrees':.25,'source_geometry':'Q4 candidate; see results/spatial_context.json','positional_uncertainty':None}) for s in af.space_object]
    af['variable_semantics']='time_offsets_hours; Q2_dimensionless_margin; GFS2m_C_and_APCP_mm_intervals; regional_to_source_mapping_conditional'
    af['time_uncertainty']=json.dumps({'conditional_model_interval_level':.95,'physical_time_error':None,'discrete_prior_support_may_be_one; not empirical coverage':True})
    clock=pd.to_datetime(af.issue_time,utc=True,format='mixed')
    af['valid_time']=(clock+pd.to_timedelta(af.horizon_hours,unit='h')).astype(str)
    af['time_utc']=(clock+pd.to_timedelta(af.conditional_median_offset_hours,unit='h')).where(af.conditional_median_offset_hours.notna()).astype('string')
    af['evidence_status']='inference'
    common_path=R/'results/common_anchors_no_age/predictions.csv';cf=pd.read_csv(common_path)
    cf['prediction_projection']='exact_same_sensor_issue_horizon_anchor;all_rolling_records_retained_in_engine_cache'
    cf['physical_phase']='external_terminal_or_main_collapse';cf['event_id']='external:'+cf.event_group
```

```
cf['issue_time']=None;cf['available_time']=None;cf['data_role']='validation';cf['validation_role']='external_site_or_event_heldout_benchmark'
cf['forecast_input_mode']=None;cf['input_version']=VERSION
cf['interval_semantics']='model_internal_given_terminal_collapse_within_H; no distribution_free95 guarantee'
cf['parameter_version']=np.where(cf.engine.eq('B'),'B_fixed_w16_q0.01','C_no_record_age_LOSO_nested_selected')
cf['model_input_role']='real_published_deformation; source preprocessing/curation and historical availability uncertified'
cf['mechanism_id']=np.where(cf.engine.eq('B'),'deformation_acceleration','statistical_dynamic_hazard')
cf['experiment_id']='q3r3-exact-matched-external-anchors-no-record-age'
cf['scenario_id']=[f'{g}:anchor_relative_hours={t}:H={h}' for g,t,h in zip(cf.event_group,cf.issue_relative_hours,cf.horizon_hours)]
cf['geometry_version']='external_sensor_native_support_not_local_source_geometry'
cf['space_object']=[json.dumps({'site_id':s,'physical_event_group':g,'crs':None,'resolution':None,'positional_uncertainty':None}) for s,g in zip(cf.site_id,cf.event_group)]
cf['time_utc']=None;cf['valid_time']=None;cf['forecast_reference_time']=None
cf['valid_time_relative_hours']=cf.issue_relative_hours+cf.horizon_hours
cf['time_uncertainty']=[json.dumps({'label_lower_offset_hours':float(l) if pd.notna(l) else None,'label_upper_offset_hours':float(u) if pd.notna(u) else None,'clock':'native_relative; no UTC fabricated','physical_proxy_error':None}) for l,u in zip(cf.failure_lower_offset_hours,cf.failure_upper_offset_hours)]
cf['variable_semantics']='native_source_units preserved in inputs/deformation/site_registry.csv; B_time_zero_scale_invariant; C_dimensionless_deformation; past_only_Q3_processing'
cf['evidence_status']='inference'
pred=pd.concat([af,cf],ignore_index=True);pred.to_csv(R/'results/predictions.csv',index=False)
evaluation=[]
for p,label in [(R/'results/common_anchors_no_age/evaluation.csv','primary_exact_matched_anchors_no_record_age'),
                (b/'model_comparison.csv','B_original_registered_publication_sets'),
                (b/'paired_model_comparison.csv','B_same_valid_opportunity_pairing'),
                (c/'no_record_age/anchor_evaluation.csv','C_no_record_age_primary_anchors'),
                (c/'record_age_anchor_reference/anchor_evaluation.csv','C_record_age_sensitive_challenge'),
                (c/'evaluation.csv','C_record_age_sensitive_full_rolling_challenge')]:
    f=pd.read_csv(p);f['comparison_id']=label;f['evaluation_stratum']='external_transfer';f['source_table']=str(p.relative_to(R));evaluation.append(f)
pd.concat(evaluation,ignore_index=True).to_csv(R/'results/evaluation.csv',index=False)
paired=pd.read_csv(R/'results/common_anchors_no_age/paired_valid_scores.csv')
paired['comparison_id']='primary_exact_same_anchor_and_probability_issued_subset'
paired.to_csv(R/'results/paired_model_comparison.csv',index=False)
folds=[]
for p,label in [(b/'fold_membership.csv','B_fixed_structure_online_prefix'),(c/'no_record_age/fold_membership.csv','C_no_record_age_primary'),(c/'fold_membership.csv','C_record_age_challenge')]:
    f=pd.read_csv(p);f['pipeline']=label;folds.append(f)
pd.concat(folds,ignore_index=True).to_csv(R/'results/fold_membership.csv',index=False)
intervals=pd.read_csv(R/'results/common_anchors_no_age/evaluation.csv')
intervals['interval_type']='model_internal_given_start_within_H';intervals['conditional_interval_level']=.95;intervals['guaranteed_empirical95']=False
conf=pd.read_csv(b/'conformal_feasibility.csv').rename(columns={'n_calibration_independent_groups':'upper_bound_other_failure_groups'})
conf['independent_calibration_groups']=0;conf['calibration_fit_executed']=False
conf['interval_type']='conformal_independent_group_support_audit';conf['conditional_interval_level']=.95
conf['model_id']='B_conformal_feasibility';conf['status']='no_finite_distribution_free_interval; feasibility_only_not_calibration_fit';conf['guaranteed_empirical95']=False
cconf=pd.DataFrame([dict(model_id='C_conformal_feasibility',interval_type='conformal_independent_group_support_audit',conditional_interval_level=.95,
```

```
          upper_bound_other_groups=13,independent_calibration_groups=0,calibration_fit_executed=False,
          finite_sample_rank=int(np.ceil((13+1)*.95)),status='rank14_exceeds13; feasibility_only_not_calibration_fit',
          exchangeability_established=False,guaranteed_empirical95=False)])
    pd.concat([intervals,conf,cconf],ignore_index=True).to_csv(R/'results/interval_comparison.csv',index=False)
    prior=[]
    for p,label in [(a/'prior_sensitivity.csv.gz','Q2_coarse_thermal_v1'),(a/'v2/prior_sensitivity.csv.gz','Q2_refined_thermal_v2'),
                    (a/'analytic_erfc/prior_sensitivity.csv.gz','Q2_analytic_erfc_reference'),(b/'window_noise_comparison.csv','B_window_noise_sensitivity')]:
        f=pd.read_csv(p);f['sensitivity_family']=label;f['source_table']=str(p.relative_to(R));prior.append(f)
    pd.concat(prior,ignore_index=True).to_csv(R/'results/prior_sensitivity.csv',index=False)
    probability=pred.dropna(subset=['probability_start_within_horizon','probability_no_start_within_horizon'])
    checks={'probability_normalization':bool(np.allclose(probability.probability_start_within_horizon+probability.probability_no_start_within_horizon,1)),
            'no_local_truth_coverage_inferred':bool(pred.loc[pred.engine.eq('A_weather_coupled'),'empirical_coverage'].isna().all()),
            'primary_C_has_no_record_age':json.loads((c/'no_record_age/summary.json').read_text())['feature_specification']=='no_record_age_main',
            'frozen_event_v1_retained':hashlib.sha256((R/'exports/event_bundle.json').read_bytes()).hexdigest()=='075b149214b5c9a8827af45af221ee6ee6b6428090e6d9337afb1943698d2faf'}
    contract=json.loads((R.parents[3]/'track_b/config/stage4_contract.json').read_text())
    checks['shared_contract_columns_present']=set(contract['shared_record_fields'])<=set(pred.columns)
    checks['data_roles_explicit']=pred.data_role.isin(['scenario','validation','calibration','exploration']).all().item()
    # The C CLI and registered summary explicitly bind feature choice; fit tables
    # may omit feature labels, so verify the run's own feature manifest as well.
    cs=json.loads((c/'no_record_age/summary.json').read_text())
    checks['all_engine_validations_loaded']=True
    summary=dict(generated_at_utc=datetime.now(timezone.utc).isoformat(),prediction_projection_rows=len(pred),A_weather_rows=len(af),matched_B_C_rows=len(cf),
       primary_A_version='analytic_erfc_linear_thermal_reference; water_damage_hourly_discrete',primary_C_summary=cs,local_first_mechanical_truth_accepted=False,local_empirical_coverage=None,
       full_raw_predictions='See per-engine prediction_cache.json or predictions.csv.gz; root projection keeps primary prior and fixed common anchors, not all raw repeated rows',
       evaluation_tables_compared_without_pooling_strata=True,checks=checks,passed=all(checks.values()))
    (R/'results/integration_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2)+'\n')
    assert summary['passed'],checks;print(json.dumps({k:v for k,v in summary.items() if k!='primary_C_summary'}))

if __name__=='__main__':main()
```

程序 29：first conditional propagation join

来源：track_b/research/q3/round3/scripts/join_propagation.py
SHA256: b2d0b8885ebe6c9aad0d0561a24486e8d68cf92d6a7c39ef22e5037a2e630a4d

```
#!/usr/bin/env python3
"""Strict delivered-role pairing; bulk-release travel never becomes onset truth."""
import argparse
from datetime import datetime,timezone
import hashlib,json,math
from pathlib import Path
import pandas as pd
R=Path(__file__).resolve().parents[1];ROOT=R.parents[3]

def sha(p):
    h=hashlib.sha256()
    with p.open('rb') as f:
        for block in iter(lambda:f.read(4*1024*1024),b''):h.update(block)
    return h.hexdigest()
def present(x):return pd.notna(x) and str(x).strip()!=''

def main():
    cli=argparse.ArgumentParser();cli.add_argument('--bundle',required=True);cli.add_argument('--expected-sha256',required=True)
    cli.add_argument('--onsets',default='results/engine_a/analytic_erfc/first_passages.csv');cli.add_argument('--outdir',default='results/arrival/propagation_v1')
    args=cli.parse_args();bp=ROOT/args.bundle;assert bp.resolve().is_relative_to(R/'inputs/q4') and sha(bp)==args.expected_sha256
    bundle=json.loads(bp.read_text());files=[]
    for f in bundle['files']:
        p=Path(f['path']);p=p if p.is_absolute() else ROOT/p
        # Read external cache objects only when named in this delivered bundle.
        assert p.stat().st_size==f['bytes'] and sha(p)==f['sha256'],str(p)
        files.append((f['role'],p))
    targets=[p for role,p in files if role=='target_arrivals'];assert len(targets)==1
    f=pd.read_csv(targets[0]);onset=pd.read_csv(R/args.onsets);rows=[];rejections=[]
    out=(R/args.outdir).resolve();assert out.is_relative_to(R/'results');out.mkdir(parents=True,exist_ok=True)
    context=f.copy();context['absolute_arrival_utc']=None;context['context_role']='conditional_bulk_release_context; not mechanical-onset pairing'
    context.to_csv(out/'propagation_context.csv',index=False)
    for _,raw in f.iterrows():
        r=raw.to_dict()
        for source,target in [('upstream_model_id','parent_model_id'),('upstream_parameter_version','release_parameter_version'),
                              ('upstream_geometry_version','parent_geometry_version'),('initiation_to_release_delay_seconds','mechanical_to_release_delay_seconds')]:
            if not present(r.get(target)) and present(r.get(source)):r[target]=r[source]
        required=['base_scenario_id','parent_model_id','mechanism_id','release_parameter_version','parent_geometry_version']
        missing=[k for k in required if not present(r.get(k))]
        if missing:
            rejections.append(dict(run_id=r.get('run_id'),scenario_id=r['scenario_id'],target_id=r['target_id'],matching_onset_rows=0,
              reason='missing_explicit_parent_lineage',missing_fields=json.dumps(missing)));continue
        s=onset[onset.base_scenario_id.eq(r['base_scenario_id'])]
        for left,right in [('model_id','parent_model_id'),('mechanism_id','mechanism_id'),('parameter_version','release_parameter_version'),('geometry_version','parent_geometry_version')]:s=s[s[left].eq(r[right])]
```

```
        if len(s)!=1:
            rejections.append(dict(run_id=r.get('run_id'),scenario_id=r['scenario_id'],target_id=r['target_id'],matching_onset_rows=len(s),reason='unresolved_or_nonunique_exact_base_model_parameter_geometry_lineage'));continue
        s=s.iloc[0];lag=pd.to_numeric(r.get('mechanical_to_release_delay_seconds'),errors='coerce')
        lo=pd.to_numeric(r.get('arrival_bracket_lower_seconds'),errors='coerce');hi=pd.to_numeric(r.get('arrival_bracket_upper_seconds'),errors='coerce')
        if any(pd.notna(x) and (not math.isfinite(float(x)) or x<0) for x in [lag,lo,hi]) or (pd.notna(lo) and pd.notna(hi) and lo>hi):
            rejections.append(dict(run_id=r.get('run_id'),scenario_id=r['scenario_id'],target_id=r['target_id'],matching_onset_rows=1,reason='invalid_delay_or_arrival_bracket'));continue
        lower=pd.to_datetime(s.crossing_lower_utc,utc=True,errors='coerce');upper=pd.to_datetime(s.crossing_upper_utc,utc=True,errors='coerce')
        known=all(pd.notna(x) for x in [lag,lo,hi,lower,upper]) and present(r.get('arrival_definition'))
        # Preserve saved-output arrival brackets instead of an exact hit point.
        al=lower+pd.Timedelta(seconds=float(lag)+float(lo)) if known else None
        au=upper+pd.Timedelta(seconds=float(lag)+float(hi)) if known else None
        rows.append(dict(run_id=r.get('run_id'),scenario_id=r['scenario_id'],base_scenario_id=r['base_scenario_id'],source_scenario_id=s.scenario_id,
            mechanism_model_id=s.model_id,propagation_model_id=r['model_id'],target_id=r['target_id'],geometry_version=r['geometry_version'],parameter_version=r['parameter_version'],
            source_first_passage_status=s.status,mechanical_to_release_delay_seconds=float(lag) if pd.notna(lag) else None,
            bulk_release_arrival_lower_seconds=float(lo) if pd.notna(lo) else None,bulk_release_arrival_upper_seconds=float(hi) if pd.notna(hi) else None,
            arrival_lower_utc=al.isoformat() if al is not None else None,arrival_upper_utc=au.isoformat() if au is not None else None,
            arrival_time_utc=None,arrival_definition=r.get('arrival_definition'),physical_phase='arrival_at_defined_target',
            include_in_arrival_envelope=r.get('include_in_arrival_envelope'),numerical_qualification=r.get('numerical_qualification'),
            arrival_status='conditional_matched_interval_with_explicit_lag' if known else 'bulk_release_relative_only_lag_or_onset_unknown',
            evaluation_stratum='conditional_scenario',event_timing_validated=False,arrival_is_initiation_truth=False))
    pd.DataFrame(rows,columns=list(rows[0]) if rows else ['run_id','scenario_id','target_id','arrival_lower_utc','arrival_upper_utc','arrival_time_utc']).to_csv(out/'conditional_arrivals.csv',index=False)
    pd.DataFrame(rejections).to_csv(out/'rejected_pairings.csv',index=False)
    report=dict(generated_at_utc=datetime.now(timezone.utc).isoformat(),status='strict_delivered_role_pairing_executed',bundle_version=bundle['bundle_version'],bundle_sha256=sha(bp),
       delivered_files_verified=len(files),target_rows=len(f),unique_pairs=len(rows),rejected_pairs=len(rejections),absolute_interval_pairs=sum(r['arrival_lower_utc'] is not None for r in rows),
       checks={'every_target_retained_as_pair_or_rejection':len(rows)+len(rejections)==len(f),'no_absolute_point_arrival_inferred':all(r['arrival_time_utc'] is None for r in rows)},
       limitations=['Bulk release is a different physical origin from first mechanical initiation; lag not assumed zero',
         'No approximate scenario string matching or Cartesian sampling','Saved-output first hit is an interval bracket, not exact arrival',
         'Q4 partial envelope flag does not certify numerical accuracy, probability or event-time validation'])
    report['passed']=all(report['checks'].values());(out/'status.json').write_text(json.dumps(report,ensure_ascii=False,indent=2)+'\n')
    assert report['passed'];print(json.dumps(report,ensure_ascii=False))

```

```
75 if __name__=='__main__':main()
```

程序 30：compact/base parameter handoff

来源：track_b/research/q3/round3/scripts/join_propagation_v2.py
SHA256: 87171f5ce437abaa91e688e07194219f5146cbe6e8fbaeed2ae316d4bdf3bc5b

```
#!/usr/bin/env python3
"""Audit an explicitly mapped Q4 packet without rerunning propagation physics.

Passing adapter QA does not qualify the numerical model or create arrival truth.
Only the two registered model replacements below are accepted. No fuzzy joins.
"""
import argparse
import csv
from datetime import datetime, timezone
import hashlib
import json
import math
from pathlib import Path
import sys

import numpy as np

R = Path(__file__).resolve().parents[1]
ROOT = R.parents[3]
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official')
PIN = '9fec7f2083ab43f81b19da2c1cca3cef9b2ad0b5db36441495162412d1e8dda4'
DEFAULT_BUNDLE = R / 'inputs/q4/bundle_q4-r3-propagation-v2/bundle.json'
DEFAULT_OUT = R / 'results/arrival/propagation_v2'
MODEL_REPLACEMENTS = {
    'q2r3_S_v1': ('S', 'q2r3_S_analytic_v1'),
    'q2r3_ST_v1': ('S_T', 'q2r3_S_T_analytic_v1'),
}
PARAMETER_VERSION = 'q2r3-design-v1'
MAPPING_VERSION = 'q4-r3-q2-mapped-release-v1'
PHYSICAL_FIELDS = [
    'theta_deg', 'thickness_normal_m', 'rock_volume_fraction', 'bridge_fraction',
    'bridge_cohesion_kpa', 'rock_mu', 'initial_temp_c', 'diffusivity_m2_s',
    'interface_depth_m', 'station_source_offset_c', 'storativity', 'drainage_hours',
    'connected_depth_m', 'rain_multiplier', 'melt_mm_deg_day', 'damage_rate_h_inv',
    'damage_exponent', 'initial_damage', 'drainage_damage_coupling', 'density_kg_m3',
]


def sha(path):
    h = hashlib.sha256()
    with Path(path).open('rb') as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):
            h.update(block)
    return h.hexdigest()


def read_json(path):
    return json.loads(Path(path).read_text())


def rows(path):
    with Path(path).open(newline='') as stream:
        return list(csv.DictReader(stream))


def write_json(path, data):
```

```
    Path(path).write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False) + '\n')


def write_csv(path, data, columns=None):
    if columns is None:
        columns = list(dict.fromkeys(k for row in data for k in row))
    with Path(path).open('w', newline='') as stream:
        writer = csv.DictWriter(stream, fieldnames=columns)
        writer.writeheader()
        writer.writerows(data)


def present(value):
    return value is not None and str(value).strip() != ''


def flag(value):
    return value is True or str(value).lower() == 'true'


def number(value):
    return float(value) if present(value) else None


def close(a, b, rtol=1e-10, atol=1e-10):
    return math.isclose(float(a), float(b), rel_tol=rtol, abs_tol=atol)


def unique(data, **keys):
    found = [x for x in data if all(x.get(k) == v for k, v in keys.items())]
    if len(found) != 1:
        raise ValueError(f'Expected one exact row for {keys}, got {len(found)}')
    return found[0]


def resolve(path):
    p = Path(path)
    return p if p.is_absolute() else ROOT / p


def relative(path):
    p = Path(path)
    return str(p.relative_to(ROOT)) if p.is_relative_to(ROOT) else str(p)


def file_record(path, role):
    p = Path(path)
    return dict(path=relative(p), bytes=p.stat().st_size, sha256=sha(p), role=role)


def support_distance(x, y, rectangles):
    """Euclidean point distance to a union of actual release raster cell boxes."""
    answer = np.full(np.asarray(x).shape, np.inf, dtype=float)
    for xmin, xmax, ymin, ymax in rectangles:
        dx = np.maximum(np.maximum(xmin - x, x - xmax), 0)
        dy = np.maximum(np.maximum(ymin - y, y - ymax), 0)
        answer = np.minimum(answer, np.hypot(dx, dy))
    return answer


def saved_first_hit(time, depth, mask, horizon):
    eligible = np.isfinite(time) & (time <= horizon)
```

```
    at = np.flatnonzero(eligible)
    if not mask.any():
        return dict(status='no_registered_cell_centres_in_buffer', hit=False,
                    lower=None, upper=None, last_saved=None, cells=0)
    hits = np.any(depth[:, mask] >= 0.1, axis=1)
    found = at[hits[at]]
    first = int(found[0]) if len(found) else None
    return dict(status=('already_inundated_at_initial_saved_state' if first == 0 else
                        'arrived_at_saved_output' if first is not None else
                        'not_arrived_at_saved_outputs_through_horizon'),
                hit=first is not None,
                lower=float(time[first - 1]) if first is not None and first > 0 else None,
                upper=float(time[first]) if first is not None else None,
                last_saved=float(time[at[-1]]) if len(at) else None,
                cells=int(mask.sum()))


def main():
    cli = argparse.ArgumentParser(description=__doc__)
    cli.add_argument('--bundle', default=str(DEFAULT_BUNDLE))
    cli.add_argument('--expected-sha256', default=PIN)
    cli.add_argument('--outdir', default=str(DEFAULT_OUT))
    args = cli.parse_args()
    bp, out = Path(args.bundle).resolve(), Path(args.outdir).resolve()
    if not bp.is_relative_to(R / 'inputs/q4') or out != DEFAULT_OUT:
        raise ValueError('Only the imported Q4 packet and owned propagation_v2 output are
        ↪  allowed')
    if args.expected_sha256 != PIN or sha(bp) != PIN:
        raise ValueError('Unregistered Q4 packet SHA256')
    if not (out / 'PROTOCOL.md').exists():
        raise ValueError('Read/retain the registered protocol before running')
    out.mkdir(parents=True, exist_ok=True)
    checks, inputs, role_files = [], [file_record(bp, 'pinned delivered packet')], {}

    def check(name, passed, detail=None):
        entry = dict(name=name, passed=bool(passed))
        if detail is not None:
            entry['detail'] = detail
        checks.append(entry)
        if not passed:
            raise ValueError(f'Failed adapter check: {name}; {detail}')

    bundle = read_json(bp)
    for f in bundle['files']:
        p = resolve(f['path']).resolve()
        allowed = p.is_relative_to(bp.parent) or (
            f.get('storage') == 'immutable_external_cache' and p.is_relative_to(CACHE))
        check('delivered_path_scope:' + relative(p), allowed)
        check('delivered_hash:' + relative(p),
              p.stat().st_size == f['bytes'] and sha(p) == f['sha256'])
        role_files.setdefault(f['role'], []).append(p)
        inputs.append(file_record(p, f['role']))
    check('exact_delivered_file_count', len(bundle['files']) == 39)

    def role(name):
        pp = role_files.get(name, [])
        check('unique_role:' + name, len(pp) == 1)
        return pp[0]

    export = role('model_runs').parent
    run_rows, target_rows = rows(role('model_runs')), rows(role('target_arrivals'))
    metrics = rows(role('comparison_metrics'))
    source_release = export / 'provenance/upstream_q2/release_states.csv'
```

```
original_bundle = read_json(export / 'provenance/upstream_q2/bundle.json')
original_upstream = read_json(export / 'provenance/upstream_q2/upstream_bundle.json')
release_rows = rows(source_release)
check('compact_original_release_hash_from_upstream',
      sha(source_release) == unique(original_upstream['files'],
      ↪  role='results/release_states.csv')['sha256'])
check('compact_received_and_original_bundle_agree',
      unique(original_bundle['files'], role='results/release_states.csv')['sha256'] ==
      ↪  sha(source_release))
check('only_two_registered_runs', set(x['run_id'] for x in run_rows) ==
↪  set(bundle['selected_run_ids'])
      and len(run_rows) == 2)
check('14_rows_7_representative_targets', len(target_rows) == 14 and
      len(set(x['target_id'] for x in target_rows)) == 7)

oldroot = R / 'inputs/q2/mechanism_bundle_q2-r3-mechanism-v1'
newroot = R / 'inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1'
pp = oldroot / 'files/results/parameter_ensemble.csv'
ap = newroot / 'files/results/analytic_thermal/parameter_ensemble.csv'
cp = newroot / 'files/results/analytic_thermal/mechanism_contract.json'
fp = R / 'results/engine_a/analytic_erfc/first_passages.csv'
gp = R / 'inputs/q2/release_bundle_v2_q2-r3-release-v2-spatial-copdem-v1/files/results/rel⌋
↪  ease_states_geometry_v2.csv'
oldscript = R / 'scripts/join_propagation.py'
for p, description in [(pp, 'original shared physical parameters'), (ap, 'analytic shared
↪  physical parameters'),
                       (cp, 'analytic explicit release-join contract'), (fp, 'frozen
                       ↪  analytic first passages'),
                       (gp, 'different broad-source release-v2 comparison'), (oldscript,
                       ↪  'previous adapter source')]:
    inputs.append(file_record(p, description))
for rootp, required in [(oldroot, [pp]), (newroot, [ap, cp])]:
    imported = read_json(rootp / 'bundle.json')
    inputs.append(file_record(rootp / 'bundle.json', 'existing Q3 imported Q2 manifest'))
    for p in required:
        match = [f for f in imported['files'] if resolve(f['path']).resolve() ==
        ↪  p.resolve()]
        check('existing_import_hash:' + relative(p), len(match) == 1 and
              sha(p) == match[0]['sha256'] and p.stat().st_size == match[0]['bytes'])
check('same_original_parameter_ensemble_as_compact_release',
      sha(pp) == unique(original_upstream['files'],
      ↪  role='results/parameter_ensemble.csv')['sha256'])
contract = read_json(cp)
check('analytic_contract_allows_release_join',
      contract['parameter_version'] == PARAMETER_VERSION and
      contract['base_scenario_join'] == 'base_scenario_id joins v1/v2 and
      ↪  release-geometry-v2; release states unchanged, use this model version
      ↪  onset/censoring when combined')
original_parameters, analytic_parameters = rows(pp), rows(ap)
onsets, broad_sources = rows(fp), rows(gp)
lineage_audit, parameter_audit, release_audit, geometry_comparison, legacy_comparison = [],
↪  [], [], [], []
joined, target_checks, footprints, arrays = [], [], [], []
resolved_by_run = {}
# Raster readers are used only after every approved external object was hashed.
import rasterio
from pyproj import Transformer
target_points = {x['target_id']: (float(x['x_m']), float(x['y_m']), x['geometry_version'])
                 for x in rows(export / 'geometry/spatial_targets_v2.csv')}
geometries = read_json(export / 'geometry/computational_geometries.geojson')
lonlat_to_metric = Transformer.from_crs('EPSG:4326', 'EPSG:32645', always_xy=True)
for f in geometries['features']:
    if f['geometry']['type'] == 'Point':
```

```
            p = f['properties']
            tx, ty = lonlat_to_metric.transform(*f['geometry']['coordinates'])
            target_points[p['space_id']] = (tx, ty, p['geometry_version'])

    for run in run_rows:
        rid, base = run['run_id'], run['base_scenario_id']
        lineage = unique(bundle['lineage_by_run'], run_id=rid)
        case_path = export / f'provenance/{rid}/case.json'
        case = read_json(case_path)
        check('paper_lonlat_to_metric_matches_executed_case:' + rid,
              all(t['id'] in target_points and np.allclose(target_points[t['id']][:2], t['xy'],
                  rtol=0, atol=1e-6) for t in case['targets']))
        release = unique(release_rows, scenario_id=base)
        old = unique(original_parameters, scenario_id=base)
        check('registered_model_replacement:' + rid, release['model_id'] in MODEL_REPLACEMENTS)
        mechanism, new_model = MODEL_REPLACEMENTS[release['model_id']]
        check('contract_model_replacement:' + rid, contract['model_ids'][mechanism] ==
        ↪  new_model)
        new = unique(analytic_parameters, base_scenario_id=base, model_id=new_model)
        onset = unique(onsets, base_scenario_id=base, model_id=new_model,
                       mechanism_id=mechanism, parameter_version=PARAMETER_VERSION,
                       geometry_version=old['geometry_version'])
        keys = ['base_scenario_id', 'upstream_model_id', 'upstream_parameter_version',
                'mechanism_id', 'parameter_version', 'release_mapping_version',
                ↪  'geometry_version']
        check('bundle_lineage_matches_model_run:' + rid, all(lineage[k] == run[k] for k in
        ↪  keys))
        check('explicit_original_release_lineage:' + rid,
              release['model_id'] == old['model_id'] == lineage['upstream_model_id'] ==
              ↪  case['upstream_model_id'] and
              release['mechanism_id'] == old['mechanism_id'] == lineage['mechanism_id'] ==
              ↪  mechanism and
              release['parameter_version'] == old['parameter_version'] ==
              ↪  new['parameter_version'] ==
              case['upstream_parameter_version'] == lineage['upstream_parameter_version'] ==
              ↪  PARAMETER_VERSION and
              case['scenario_id'] == lineage['scenario_id'] == base and
              case['geometry_version'] == lineage['geometry_version'] ==
              ↪  lineage['release_mapping_version'] == MAPPING_VERSION)
        check('mechanism_geometry_preserved_across_model_replacement:' + rid,
              old['geometry_version'] == new['geometry_version'] == onset['geometry_version'])
        check('same_nonphysical_design_identity:' + rid,
              all(old[k] == new[k] for k in ['parameter_set_id', 'mechanism_id', 'law_policy',
                                             'parameter_version', 'geometry_version',
                                             ↪  'design_role', 'prior_weight']))
        for key in PHYSICAL_FIELDS:
            same = close(old[key], new[key], rtol=1e-13, atol=1e-13)
            parameter_audit.append(dict(run_id=rid, base_scenario_id=base, parameter=key,
                                        original_value=old[key], analytic_value=new[key],
                                        ↪  identical=same))
            check(f'physical_parameter_preserved:{rid}:{key}', same)

        fractions = json.loads(release['phase_fractions'])
        iv = json.loads(release['initial_velocity'])
        area, normal, vertical, volume, density = map(float, [release['plan_area_m2'],
            release['release_thickness_normal_m'], release['release_thickness_vertical_m'],
            release['release_volume_m3'], release['density']])
        expected_vertical = normal / math.cos(math.radians(float(old['theta_deg'])))
        check('release_area_thickness_volume_and_density:' + rid,
              close(area, case['source_plan_area_m2']) and close(normal,
              ↪  old['thickness_normal_m']) and
              close(vertical, expected_vertical) and close(vertical,
              ↪  case['release_thickness_m']) and
```

```
283               close(volume, area * vertical) and close(density, case['bulk_density_kg_m3']) and
284               close(density, old['density_kg_m3']) and close(density, fractions['ice'] * 917 +
                  ↪  fractions['rock'] * 2700))
285           check('material_volume_composition_not_solver_phase:' + rid,
286               release['phase_fraction_basis'] == case['passive_material_fraction_basis'] ==
                  ↪  'volume' and
287               fractions == case['passive_material_volume_fractions'] and
288               close(sum(fractions.values()), 1) and case['phase_volume_fractions'] == [1])
289           check('initial_zero_velocity:' + rid, all(iv[k] == 0 for k in ['x', 'y', 'z']) and
290               case['initial_velocity_east_north_m_s'] == [0, 0])
291           check('original_onset_kept_in_propagation_record:' + rid,
292               release['onset_status'] == lineage['upstream_onset_status'] ==
                  ↪  case['upstream_onset_status'])
293           never = onset['status'] == 'no_cross_observed_complete'
294           check('never_cross_is_only_if_released:' + rid,
295               not never or (flag(run['if_released_propagation']) and
                  ↪  case['if_released_propagation'] and
296                             not present(onset['crossing_lower_utc']) and not
                                ↪  present(onset['crossing_upper_utc'])))
297           check('unknown_release_lag_preserved:' + rid,
298               case['initiation_to_release_delay_seconds'] is None and
299               lineage['initiation_to_release_delay_seconds'] is None and
300               not present(run['initiation_to_release_delay_seconds']))
301           check('quality_does_not_upgrade:' + rid, not flag(run['include_in_arrival_envelope'])
              ↪  and
302               not flag(run['mass_conservation_qualified']) and not
                  ↪  flag(run['scientifically_qualified_for_prediction']) and
303               run['numerical_qualification'] ==
                  ↪  'unqualified_compact_source_underresolution_and_budget_discrepancy')
304
305           release_path = Path(case['release']).resolve()
306           check('release_array_is_explicitly_approved:' + rid,
307               release_path in role_files['immutable native release thickness'] and
308               sha(release_path) == case['release_sha256'] ==
                  ↪  case['source_release_reference']['sha256'])
309           with rasterio.open(release_path) as ds:
310               rel = ds.read(1).astype(float)
311               transform = ds.transform
312               check('release_crs_and_affine:' + rid, str(ds.crs) == 'EPSG:32645' and
313                     transform.a == 60 and transform.e == -60 and transform.b == transform.d == 0)
314           positive = rel > 0
315           grid_volume = float(rel[positive].sum() * abs(transform.a * transform.e))
316           effective_area = grid_volume / vertical
317           check('raster_integrated_release_volume:' + rid,
318               close(grid_volume, case['release_grid_volume_m3'], rtol=2e-7) and
319               close(grid_volume, volume, rtol=2e-7) and close(effective_area, area, rtol=2e-7))
320           m = unique(metrics, run_id=rid)
321           check('underresolution_support_cells_match:' + rid, int(positive.sum()) ==
              ↪  int(run['source_support_cells']) and
322               close(area / 3600, run['source_equivalent_full_cells']))
323           release_audit.append(dict(run_id=rid, base_scenario_id=base,
324               original_release_geometry_version=release['geometry_version'],
                  ↪  mapped_geometry_version=case['geometry_version'],
325               original_plan_area_m2=area, normal_thickness_m=normal,
                  ↪  theta_deg=float(old['theta_deg']),
326               original_vertical_thickness_m=vertical,
                  ↪  executed_vertical_thickness_m=case['release_thickness_m'],
327               original_volume_m3=volume, supplied_raster_volume_m3=grid_volume,
328               supplied_volume_relative_difference=grid_volume / volume - 1,
329               raster_equivalent_area_m2=effective_area,
                  ↪  positive_support_cell_count=int(positive.sum()),
330               support_footprint_area_m2=int(positive.sum()) * 3600, equivalent_full_cells=area /
                  ↪  3600,
```

```
            density_kg_m3=density, material_fraction_basis='volume',
            ↪ material_fractions=json.dumps(fractions, sort_keys=True),
            native_solver_phase_fraction='[1] equivalent_bulk', initial_velocity='[0,0] m/s
            ↪ east,north',
            release_raster_sha256=sha(release_path),
            ↪ declared_source_geometry_sha256=case['source_geometry_sha256'],
            exact_compact_polygon_delivered=False, independent_mapping_table_delivered=False,
            exact_compact_polygon_area_independently_verified=False,
            ↪ case_to_original_release_audit_passed=True,
            ascii_budget_relative_residual=float(m['ascii_raster_source_adjusted_budget_relati⌋
            ↪ ve_residual']),
            native_internal_budget_relative_residual=float(m['native_internal_summary_mass_bud⌋
            ↪ get_relative_residual']),
            numerical_qualification=run['numerical_qualification']))
        for b in broad_sources:
            if b['base_scenario_id'] == base:
                geometry_comparison.append(dict(run_id=rid, base_scenario_id=base,
                    executed_mapping_version=MAPPING_VERSION, executed_plan_area_m2=area,
                    executed_volume_m3=volume, different_release_scenario_id=b['scenario_id'],
                    different_geometry_version=b['geometry_version'],
                    ↪ different_mapping_version=b['release_mapping_version'],
                    different_grid_resolution_m=b['grid_resolution_m'],
                    ↪ different_area_m2=b['plan_area_m2'],
                    different_volume_m3=b['release_volume_m3'], interchangeable=False,
                    reason='broad_paper_mask_changes_release_area_and_volume;
                    ↪ not_compact_mapping_alias'))
        la = dict(run_id=rid, base_scenario_id=base, mechanism_id=mechanism,
            original_model_id=old['model_id'], replacement_model_id=new_model,
            replacement_scenario_id=onset['scenario_id'], parameter_version=PARAMETER_VERSION,
            physical_parameter_fields_compared=len(PHYSICAL_FIELDS),
            ↪ all_physical_parameters_preserved=True,
            replacement_contract_version=contract['contract_version'],
            replacement_reason='explicit analytic-erfc thermal model version; same physical
            ↪ parameter design and release state',
            mechanism_geometry_version=old['geometry_version'],
            ↪ original_release_geometry_version=release['geometry_version'],
            mapped_release_geometry_version=MAPPING_VERSION,
            ↪ upstream_onset_status=release['onset_status'],
            analytic_onset_status=onset['status'],
            ↪ conditional_onset_lower_utc=onset['crossing_lower_utc'] or None,
            conditional_onset_upper_utc=onset['crossing_upper_utc'] or None,
            conditional_onset_interpolated_point_utc=onset['crossing_point_utc'] or None,
            forecast_input_mode=onset['forecast_input_mode'],
            ↪ if_released_propagation=flag(run['if_released_propagation']),
            initiation_to_release_delay_seconds=None,
            ↪ lineage_adapter_status='accepted_explicit_versioned_mapping',
            arrival_is_initiation_truth=False)
        lineage_audit.append(la)
        resolved_by_run[rid] = la

        field_rec = unique(bundle['files'], role='actual_official_process_fields', run_id=rid)
        check('field_provenance_binding:' + rid,
              sha(case_path) == field_rec['case_sha256'] and
              sha(export / f'provenance/{rid}/execution.json') == field_rec['execution_sha256']
              ↪ and
              sha(export / f'provenance/{rid}/run.log') == field_rec['log_sha256'])
        with np.load(field_rec['path'], allow_pickle=False) as z:
            time = z['time_s'].copy()
            depth = z['depth_frames'].copy()
            tr = z['transform'].copy()
            cumulative = z['depth_max_native_cumulative'].copy()
        check('saved_depth_time_grid:' + rid, depth.shape[0] == len(time) and
```

```
      depth.shape[1:] == rel.shape and np.all(np.isfinite(depth)) and np.min(depth) >= 0 and
      np.all(np.diff(time) > 0) and np.array_equal(tr, np.array(tuple(transform))))
check('actual_native_end_time:' + rid, close(time[-1], field_rec['time_end_seconds']) and
      close(time[-1], run['actual_end_time_seconds']))
yy, xx = np.indices(rel.shape)
xx = transform.c + (xx + 0.5) * transform.a
yy = transform.f + (yy + 0.5) * transform.e
rectangles = [(float(xx[j, i] - 30), float(xx[j, i] + 30),
               float(yy[j, i] - 30), float(yy[j, i] + 30))
              for j, i in zip(*np.nonzero(positive))]
distance = support_distance(xx, yy, rectangles)
horizon = float(run['requested_horizon_seconds'])
upto = time <= horizon
requested_max = np.max(depth[upto], axis=0)
actual_max = np.max(depth, axis=0)
check('saved_footprint_reproduces_delivered_area:' + rid,
      close(np.count_nonzero(actual_max >= 0.1) * 3600, m['impact_area_m2']))
for scope, hmax, end in [('saved_outputs_through_requested_horizon', requested_max, float(time[upto][-1])),
                         ('all_actual_saved_outputs', actual_max, float(time[-1]))]:
    for threshold in [0.1, 1.0, 5.0]:
        wet = hmax >= threshold
        footprints.append(dict(run_id=rid, base_scenario_id=base, scope=scope,
            requested_horizon_seconds=horizon, last_included_saved_output_seconds=end,
            threshold_depth_m=threshold, affected_cell_count=int(wet.sum()),
            affected_plan_area_m2=int(wet.sum()) * 3600,
            maximum_affected_cell_centre_distance_from_release_support_m=float(distance[wet].max()) if wet.any() else None,
            distance_definition='max over affected cell centres of Euclidean distance to union of positive supplied-release cell footprints',
            source_geometry='actual supplied compact release raster support, not broad candidate polygon',
            metric_crs='EPSG:32645', grid_resolution_m=60,
            scientifically_qualified_for_prediction=False,
            mass_conservation_qualified=False, validated_warning_distance_m=None,
            footprint_is_validated_hazard_zone=False, arrival_probability=None,
            source_fields_sha256=field_rec['sha256'],
            release_raster_sha256=sha(release_path)))
arrays.append((base, requested_max, cumulative, positive, xx, yy))
for target in [x for x in target_rows if x['run_id'] == rid]:
    tid = target['target_id']
    for key in ['scenario_id', 'base_scenario_id', 'upstream_model_id', 'upstream_parameter_version',
                'geometry_version', 'release_mapping_version', 'parameter_version']:
        check(f'target_exact_lineage:{rid}:{tid}:{key}', target[key] == run[key])
    check('target_unknown_lag:' + rid + ':' + tid, not present(target['initiation_to_release_delay_seconds']))
    check('target_quality_exclusion_retained:' + rid + ':' + tid,
          not flag(target['include_in_arrival_envelope']) and
          not flag(target['scientifically_qualified_for_prediction']) and
          not flag(target['mass_conservation_qualified']))
    check('target_explicit_geometry:' + rid + ':' + tid, tid in target_points and
          target_points[tid][2] == target['target_geometry_version'])
    tx, ty, _ = target_points[tid]
    mask = np.hypot(xx - tx, yy - ty) <= 100
    actual = saved_first_hit(time, depth, mask, float(time[-1]))
    formal = saved_first_hit(time, depth, mask, horizon)
    hit_matches = actual['hit'] == flag(target['observed_hit'])
    bracket_matches = ((actual['lower'] is None and not present(target['arrival_bracket_lower_seconds'])) or
```

```
            (actual['lower'] is not None and close(actual['lower'],
            ↪  target['arrival_bracket_lower_seconds']))) and (
            (actual['upper'] is None and not
            ↪  present(target['arrival_bracket_upper_seconds'])) or
            (actual['upper'] is not None and close(actual['upper'],
            ↪  target['arrival_bracket_upper_seconds'])))
        check('independent_actual_field_arrival:' + rid + ':' + tid, hit_matches and
        ↪  bracket_matches and
              field_rec['sha256'] == target['source_fields_sha256'] and
              ↪  flag(target['target_field_hash_matches']))
        target_checks.append(dict(run_id=rid, target_id=tid,
        ↪  target_buffer_cell_count=actual['cells'],
            recomputed_actual_status=actual['status'],
            ↪  actual_first_hit_lower_seconds=actual['lower'],
            actual_first_hit_upper_seconds=actual['upper'],
            ↪  actual_last_saved_seconds=actual['last_saved'],
            requested_horizon_status=formal['status'],
            ↪  requested_first_hit_lower_seconds=formal['lower'],
            requested_first_hit_upper_seconds=formal['upper'],
            ↪  requested_last_saved_seconds=formal['last_saved'],
            original_actual_hit_agrees=hit_matches,
            ↪  original_bracket_agrees=bracket_matches,
            bound_source_field_hash_agrees=True,
            target_point_distance_to_release_raster_support_m=float(support_distance(np.ar⌋
            ↪  ray(tx), np.array(ty), rectangles))))
        joined.append(dict(run_id=rid, base_scenario_id=base,
        ↪  source_scenario_id=onset['scenario_id'],
            original_upstream_model_id=old['model_id'], analytic_onset_model_id=new_model,
            propagation_model_id=run['model_id'], mechanism_id=mechanism,
            ↪  parameter_version=PARAMETER_VERSION,
            release_mapping_version=MAPPING_VERSION, target_id=tid,
            ↪  entity_id=target['entity_id'] or None,
            target_kind=target['target_kind'] or None,
            ↪  target_geometry_version=target['target_geometry_version'],
            source_first_passage_status=onset['status'],
            ↪  original_upstream_onset_status=release['onset_status'],
            conditional_onset_lower_utc=onset['crossing_lower_utc'] or None,
            conditional_onset_upper_utc=onset['crossing_upper_utc'] or None,
            if_released_propagation=flag(run['if_released_propagation']),
            initiation_to_release_delay_seconds=None, time_origin='bulk_release',
            raw_conditional_arrival_status=target['arrival_status'],
            diagnostic_bulk_release_arrival_lower_seconds=actual['lower'],
            diagnostic_bulk_release_arrival_upper_seconds=actual['upper'],
            diagnostic_arrival_within_requested_horizon=formal['hit'],
            requested_horizon_seconds=horizon,
            ↪  last_saved_within_requested_horizon_seconds=formal['last_saved'],
            actual_last_saved_seconds=actual['last_saved'],
            arrival_definition=target['arrival_definition'],
            absolute_arrival_lower_utc=None, absolute_arrival_upper_utc=None,
            ↪  absolute_arrival_time_utc=None,
            qualified_arrival_lower_seconds=None, qualified_arrival_upper_seconds=None,
            arrival_probability=None, probability_weight=None,
            ↪  include_in_arrival_envelope=False,
            numerical_qualification=run['numerical_qualification'],
            ↪  mass_conservation_qualified=False,
            scientifically_qualified_for_prediction=False, event_timing_validated=False,
            arrival_is_initiation_truth=False,
            ↪  physical_phase='conditional_bulk_release_to_target_arrival',
            evaluation_stratum='conditional_scenario',
            ↪  independent_local_validation_events=0,
            adapter_lineage_status='accepted_explicit_versioned_mapping',
            arrival_status='unqualified_diagnostic_only;unknown_initiation_to_release_lag',
```

```
                source_fields_sha256=field_rec['sha256']))
            legacy_comparison.append(dict(run_id=rid, target_id=tid, same_v2_input=True,
                previous_adapter_result_on_same_headers='reject_missing_explicit_parent_lineage',
                previous_missing_fields='mechanism_id,parent_geometry_version',
                previous_exact_model_would_match_new_analytic=False,
                previous_implicit_geometry_conflation_allowed=False,
                v2_result='accepted_explicit_versioned_mapping_unqualified_diagnostic',
                fix='mechanism from verified per-run lineage; separate geometry roles; registered analytic model replacement',
                physical_qualification_changed=False,
                previous_adapter_source_sha256=sha(oldscript)))

    check('all_targets_retained_without_rejection', len(joined) == len(target_rows) == 14)
    check('one_pair_per_run_target', len({(x['run_id'], x['target_id']) for x in joined}) == 14)
    check('no_probability_or_absolute_arrival_created', all(
        all(x[k] is None for k in ['absolute_arrival_lower_utc', 'absolute_arrival_upper_utc',
            'absolute_arrival_time_utc', 'arrival_probability', 'probability_weight',
            'qualified_arrival_lower_seconds', 'qualified_arrival_upper_seconds']) for x in joined))
    check('all_excluded_from_qualified_arrival_envelope', all(not x['include_in_arrival_envelope'] for x in joined))
    check('p0000_no_start_retained', all(x['if_released_propagation'] and
        x['source_first_passage_status'] == 'no_cross_observed_complete' for x in joined
        if x['base_scenario_id'] == 'q2r3-p0000-S-linear'))
    check('p0036_actual_analytic_crossing_used', resolved_by_run[
        'q2map60_q2r3_p0036_S_T_linear__sf0']['analytic_onset_status'] == 'point_crossing_linear_between_samples')
    write_csv(out / 'lineage_audit.csv', lineage_audit)
    write_csv(out / 'physical_parameter_audit.csv', parameter_audit)
    write_csv(out / 'release_mapping_audit.csv', release_audit)
    write_csv(out / 'different_geometry_comparison.csv', geometry_comparison)
    write_csv(out / 'conditional_arrivals.csv', joined)
    write_csv(out / 'saved_field_target_checks.csv', target_checks)
    write_csv(out / 'depth_footprint_diagnostics.csv', footprints)
    write_csv(out / 'previous_adapter_comparison.csv', legacy_comparison)
    write_csv(out / 'rejected_pairings.csv', [], ['run_id', 'target_id', 'reason'])
    schema = dict(version='q3-q4-explicit-mapped-adapter-v2',
        field_resolution={
            'original_upstream_model_id': 'target.upstream_model_id = model_run.upstream_model_id = bundle.lineage_by_run.upstream_model_id = Q2 compact release.model_id',
            'mechanism_id': 'verified bundle.lineage_by_run.mechanism_id; absent target header is not a missing scientific quantity',
            'parameter_version': 'target.upstream_parameter_version = Q2 original release and original/analytic parameter ensemble version',
            'mechanism_geometry_version': 'original/analytic parameter ensemble and analytic first_passages.geometry_version',
            'original_release_geometry_version': 'original compact release.geometry_version; conceptual conditional area only',
            'mapped_release_geometry_version': 'executed case.geometry_version = bundle/run/target.release_mapping_version',
            'target_geometry_version': 'target.target_geometry_version checked against delivered representative point geometry',
            'initiation_to_release_delay_seconds': 'delivered null; same physical quantity as old mechanical_to_release_delay_seconds alias',
        }, model_replacements=MODEL_REPLACEMENTS, matching='exact keys; no approximate strings or cross-product',
        missing_compact_vector_not_reconstructed=True, independent_mapping_table_missing=True,
        adapter_pass_is_physical_qualification=False, rejected_pair_count=0)
    write_json(out / 'adapter_schema.json', schema)
```

```
    # Standalone scientific diagnostic figure, using actual stored fields.
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(1, 2, figsize=(12, 5), constrained_layout=True)
    for ax, (base, hmax, cumulative, positive, xx, yy) in zip(axes, arrays):
        masked = np.ma.masked_where(hmax < 0.1, hmax)
        image = ax.pcolormesh(xx / 1000, yy / 1000, masked, cmap='viridis', shading='nearest',
        ↪  vmin=0, vmax=60)
        ax.contour(xx / 1000, yy / 1000, positive.astype(float), levels=[0.5], colors=['red'],
        ↪  linewidths=1.2)
        for tid, (tx, ty, _) in target_points.items():
            ax.scatter(tx / 1000, ty / 1000, s=15, marker='x', color='black')
            if tid in ['paper_junction', 'paper_port_symbol']:
                ax.annotate(tid.replace('paper_', ''), (tx / 1000, ty / 1000), xytext=(3, 3),
                ↪  textcoords='offset points', fontsize=7)
        ax.set_aspect('equal')
        ax.set_xlabel('Easting (km; EPSG:32645)')
        ax.set_ylabel('Northing (km; EPSG:32645)')
        ax.set_title(base + '\nUNQUALIFIED; saved outputs <=1800s', fontsize=9)
    fig.colorbar(image, ax=axes, label='Saved-output maximum depth (m; shown >=0.1m)',
    ↪  shrink=0.8)
    fig.suptitle('Conditional diagnostics; red = release raster support; black x = target
    ↪  representatives\nNo absolute arrival time, probability or validated warning distance',
    ↪  fontsize=11)
    fig.savefig(out / 'conditional_footprints.png', dpi=180)
    plt.close(fig)

    summary = dict(status='mapped_adapter_executed_unqualified_diagnostics_only',
        bundle_version=bundle['bundle_version'], bundle_sha256=PIN,
        delivered_files_verified=39, physical_model_runs=2,
        ↪  independent_local_validation_events=0,
        representative_target_count=7, osm_representative_count=5, osm_entity_count=3,
        paper_candidate_point_count=2, run_target_rows=14, accepted_lineage_rows=14,
        ↪  rejected_lineage_rows=0,
        raw_saved_target_hits=sum(x['actual_first_hit_upper_seconds'] is not None for x in
        ↪  target_checks),
        requested_horizon_saved_target_hits=sum(x['requested_first_hit_upper_seconds'] is not
        ↪  None for x in target_checks),
        qualified_arrival_rows=0, qualified_arrival_envelope_rows=0, absolute_arrival_rows=0,
        arrival_probability_rows=0, source_geometry_version=MAPPING_VERSION,
        compact_polygon_independently_audited=False, mapping_table_delivered=False,
        compact_raster_volume_audited=True,
        ↪  parameter_fields_checked_per_run=len(PHYSICAL_FIELDS),
        numerical_qualification='unqualified_compact_source_underresolution_and_budget_discrep
        ↪  ancy',
        adapter_checks_passed=True, predictive_qualification=False,
        scientific_adoption='pending_team_review',
        no_physical_solver_rerun=True, no_existing_frozen_result_modified=True)
    write_json(out / 'status.json', summary)
    write_json(out / 'summary.json', summary)
    text = f'''# Mapped propagation adapter v2 results

All **39 delivered file hashes** verified. Exact lineage succeeds for **14 run-target rows = 2
↪  physical runs x 7 representative points**, with no rejected pairing. Five OSM points
↪  describe three entities; two paper points remain candidate objects. No independent local
↪  validation event is added.

```

```
The original compact Q2 release table is embedded in the packet and matches the upstream frozen SHA256. Each run preserves all {len(PHYSICAL_FIELDS)} shared physical parameters through the explicitly documented analytic-erfc thermal-model replacement. The original conceptual area geometry, mechanism slab geometry, Q4 mapped compact raster, and later broad-source release-v2 geometry are distinct. The broad-source table has a different area and volume and is not used for this adapter. A compact vector polygon and independent mapping table were not delivered; those geometry details remain unaudited. The actual supplied compact release raster, volume, grid alignment, support cells, composition and density were audited.

p0000 remains `no_cross_observed_complete`, with no onset timestamp, and all of its propagation records remain **if released**. p0036 uses the actual analytic `point_crossing_linear_between_samples`, bracket **2026-07-08 15:00–16:00 UTC** and interpolation **15:34:17.258933921 UTC**. These are conditional ideal-weather retrospective mechanical first-crossing results, not bulk-release truth or operational verified predictions.

The saved-field check finds **{summary['raw_saved_target_hits']} raw target hit**: p0036 at the paper junction, bracket **1360.5–1380.1 seconds after assumed bulk release**, at h>=0.1m within the registered 100m representative-point buffer. It remains **unqualified**. The other 13 rows have no threshold hit at the stored outputs; this does not establish continuous-time non-arrival or safety. Requested 1800s and actual final 1800.1/1801.7s are retained separately; last outputs within the requested horizon are recorded explicitly.

Both runs retain the compact-source underresolution and unresolved mass-budget qualification failure. Native internal residuals are **+21.372917% / +12.716325%**; raster-source-adjusted residuals are **+21.153363% / +12.613015%**. These are different representations and are not averaged or relabelled as conserved. All qualified travel intervals, absolute arrival times, probabilities and arrival-envelope inclusion remain null/false. Unknown mechanical-initiation-to-bulk-release lag is never set to zero.

`depth_footprint_diagnostics.csv` and `conditional_footprints.png` use actual hashed fields, without a physics rerun. The 0.1/1/5m thresholds are descriptive. Area is affected grid-cell area; distance is maximum affected cell-centre distance to the union of **positive actual release raster cell footprints**, not to a centroid, the broad candidate source, or a cost path. At 60m the 8/16-cell source support exceeds the exact partial-cell compact polygon; exact polygon distance is unavailable. No footprint or distance is a validated hazard zone or warning distance.

`previous_adapter_comparison.csv` evaluates the previous adapter's header/exact-model rule against these same v2 rows. Its old required aliases would reject the absent target mechanism/parent-geometry columns. The new adapter resolves explicit delivered lineage and a registered model replacement while preserving distinct geometry semantics. QA success checks the adapter and provenance; physical qualification stays unchanged.

Reproduce from the Q3 worktree:

```sh
OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 track_b/.venv/bin/python track_b/research/q3/round3/scripts/join_propagation_v2.py
```
'''
    (out / 'RESULTS.md').write_text(text)
    inputs.append(file_record(out / 'PROTOCOL.md', 'precalculation adapter protocol'))
    inputs.append(file_record(Path(__file__), 'executed adapter source'))
    for entry in inputs:
        p = resolve(entry['path'])
        check('input_unchanged_during_adapter:' + entry['path'],
              p.stat().st_size == entry['bytes'] and sha(p) == entry['sha256'])
    # No self-reference: input index excludes outputs; output index excludes itself and validation.
    write_json(out / 'input_index.json', dict(files=inputs, file_list_excludes_self=True,
```

```
        external_read_scope='exact39 delivered-role files only; q4/official allowed objects only',
        no_active_other_question_worktree_reads=True))
    write_json(out / 'output_index.json', dict(files=[file_record(p, 'incremental adapter output')
        for p in sorted(out.iterdir()) if p.is_file() and p.name not in ['output_index.json', 'validation.json']],
        file_list_excludes_self=True, validation_excluded_to_avoid_cycle=True))
    visual_path = out / 'figure_review.json'
    visual = read_json(visual_path) if visual_path.exists() else None
    if visual is not None:
        check('external_visual_review_matches_figure', visual.get('passed') is True and
              visual.get('figure_sha256') == sha(out / 'conditional_footprints.png'))
    report = dict(generated_at_utc=datetime.now(timezone.utc).isoformat(), passed=all(x['passed'] for x in checks),
                  adapter_pass_is_predictive_qualification=False, check_count=len(checks), checks=checks,
                  input_index_sha256=sha(out / 'input_index.json'), output_index_sha256=sha(out / 'output_index.json'),
                  figure_visual_qa='passed_external_visual_inspection' if visual is not None else
                    'pending_external_visual_inspection; not claimed by this script')
    write_json(out / 'validation.json', report)
    print(json.dumps(summary, ensure_ascii=False))
    print(f"Adapter QA: {len(checks)}/{len(checks)} passed; output {out}")


if __name__ == '__main__':
    main()
```

程序 31：full source-mask join

来源：track_b/research/q3/round3/scripts/join_propagation_v3.py
SHA256: cf80cd39517fae3ec850d6f711157a8c1ea6f2767b3da2c257e6ef1babae2b4e

```
#!/usr/bin/env python3
"""Audit full-source Q2 geometry-v2/Q4 propagation-v3 without rerunning physics.

Passing adapter QA does not qualify the numerical model or create arrival truth.
Only the two registered model replacements below are accepted. No fuzzy joins.
"""
import argparse
import csv
from datetime import datetime, timezone
import hashlib
import json
import math
from pathlib import Path
import sys

import numpy as np

R = Path(__file__).resolve().parents[1]
ROOT = R.parents[3]
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official')
PIN = '8457a9db624c33c0d0988ad8f4d8c8513ff21ac858ccff830bae577f77f0b683'
PRODUCER_PIN = '150c72537ee162d29cc2e20e3e31183df1915f64999234789e1875d368c09b9c'
DEFAULT_BUNDLE = R / 'inputs/q4/bundle_q4-r3-propagation-v3/bundle.json'
DEFAULT_OUT = R / 'results/arrival/propagation_v3'
MODEL_REPLACEMENTS = {
    'q2r3_S_v1': ('S', 'q2r3_S_analytic_v1'),
    'q2r3_ST_v1': ('S_T', 'q2r3_S_T_analytic_v1'),
}
PARAMETER_VERSION = 'q2r3-design-v1'
MAPPING_VERSION = 'q2-r3-release-v2-spatial-copdem-v1'
GEOMETRY_VERSION = 'q4-r3-spatial-copdem-v1'
PHYSICAL_FIELDS = [
    'theta_deg', 'thickness_normal_m', 'rock_volume_fraction', 'bridge_fraction',
    'bridge_cohesion_kpa', 'rock_mu', 'initial_temp_c', 'diffusivity_m2_s',
    'interface_depth_m', 'station_source_offset_c', 'storativity', 'drainage_hours',
    'connected_depth_m', 'rain_multiplier', 'melt_mm_deg_day', 'damage_rate_h_inv',
    'damage_exponent', 'initial_damage', 'drainage_damage_coupling', 'density_kg_m3',
]


def sha(path):
    h = hashlib.sha256()
    with Path(path).open('rb') as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):
            h.update(block)
    return h.hexdigest()


def read_json(path):
    return json.loads(Path(path).read_text())


def rows(path):
    with Path(path).open(newline='') as stream:
        return list(csv.DictReader(stream))


```

```
def write_json(path, data):
    Path(path).write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False) +
    ↪  '\n')


def write_csv(path, data, columns=None):
    if columns is None:
        columns = list(dict.fromkeys(k for row in data for k in row))
    with Path(path).open('w', newline='') as stream:
        writer = csv.DictWriter(stream, fieldnames=columns)
        writer.writeheader()
        writer.writerows(data)


def present(value):
    return value is not None and str(value).strip() != ''


def flag(value):
    return value is True or str(value).lower() == 'true'


def number(value):
    return float(value) if present(value) else None


def close(a, b, rtol=1e-10, atol=1e-10):
    return math.isclose(float(a), float(b), rel_tol=rtol, abs_tol=atol)


def unique(data, **keys):
    found = [x for x in data if all(x.get(k) == v for k, v in keys.items())]
    if len(found) != 1:
        raise ValueError(f'Expected one exact row for {keys}, got {len(found)}')
    return found[0]


def resolve(path):
    p = Path(path)
    return p if p.is_absolute() else ROOT / p


def relative(path):
    p = Path(path)
    return str(p.relative_to(ROOT)) if p.is_relative_to(ROOT) else str(p)


def file_record(path, role):
    p = Path(path)
    return dict(path=relative(p), bytes=p.stat().st_size, sha256=sha(p), role=role)


def support_distance(x, y, rectangles):
    """Euclidean point distance to a union of actual release raster cell boxes."""
    answer = np.full(np.asarray(x).shape, np.inf, dtype=float)
    for xmin, xmax, ymin, ymax in rectangles:
        dx = np.maximum(np.maximum(xmin - x, x - xmax), 0)
        dy = np.maximum(np.maximum(ymin - y, y - ymax), 0)
        answer = np.minimum(answer, np.hypot(dx, dy))
    return answer


def saved_first_hit(time, depth, mask, horizon):
```

```
    eligible = np.isfinite(time) & (time <= horizon)
    at = np.flatnonzero(eligible)
    if not mask.any():
        return dict(status='no_registered_cell_centres_in_buffer', hit=False,
                    lower=None, upper=None, last_saved=None, cells=0)
    hits = np.any(depth[:, mask] >= 0.1, axis=1)
    found = at[hits[at]]
    first = int(found[0]) if len(found) else None
    return dict(status=('already_inundated_at_initial_saved_state' if first == 0 else
                        'arrived_at_saved_output' if first is not None else
                        'not_arrived_at_saved_outputs_through_horizon'),
                hit=first is not None,
                lower=float(time[first - 1]) if first is not None and first > 0 else None,
                upper=float(time[first]) if first is not None else None,
                last_saved=float(time[at[-1]]) if len(at) else None,
                cells=int(mask.sum()))


def main():
    cli = argparse.ArgumentParser(description=__doc__)
    cli.add_argument('--bundle', default=str(DEFAULT_BUNDLE))
    cli.add_argument('--expected-sha256', default=PIN)
    cli.add_argument('--outdir', default=str(DEFAULT_OUT))
    args = cli.parse_args()
    bp, out = Path(args.bundle).resolve(), Path(args.outdir).resolve()
    if bp != DEFAULT_BUNDLE or out != DEFAULT_OUT or args.expected_sha256 != PIN or sha(bp) !=
    ↪  PIN:
        raise ValueError('Unregistered packet or output scope')
    if not (out / 'PROTOCOL.md').exists():
        raise ValueError('Retain the registered protocol before computation')
    checks, input_by_path, roles = [], {}, {}

    def check(name, passed, detail=None):
        record = dict(name=name, passed=bool(passed))
        if detail is not None:
            record['detail'] = detail
        checks.append(record)
        if not passed:
            raise ValueError(f'Adapter check failed: {name}; {detail}')

    def add_input(path, role):
        p = Path(path).resolve()
        entry = file_record(p, role)
        key = str(p)
        if key in input_by_path:
            current = input_by_path[key]
            check('repeated_input_binding:' + relative(p),
                  entry['sha256'] == current['sha256'] and entry['bytes'] == current['bytes'])
            if role not in current['roles']:
                current['roles'].append(role)
        else:
            entry['roles'] = [role]
            del entry['role']
            input_by_path[key] = entry
        return p

    add_input(bp, 'pinned imported Q4 packet')
    producer_path = bp.parent / 'upstream_bundle.json'
    check('producer_packet_pin', sha(producer_path) == PRODUCER_PIN)
    add_input(producer_path, 'pinned producer packet snapshot; no producer path access')
    bundle = read_json(bp)
    check('advisor_review_binds_producer', bundle['advisor_review']['upstream_bundle_sha256']
    ↪  == PRODUCER_PIN)
```

```
    check('66_unique_delivered_role_objects', len(bundle['files']) == len({f['path'] for f in bundle['files']}) == 66)
    for f in bundle['files']:
        p = resolve(f['path']).resolve()
        allowed = p.is_relative_to(bp.parent) or (p.is_relative_to(CACHE) and
                  f.get('storage') == 'immutable_external_cache')
        check('approved_delivered_path:' + relative(p), allowed)
        check('delivered_hash:' + relative(p), p.stat().st_size == f['bytes'] and sha(p) == f['sha256'])
        roles.setdefault(f['role'], []).append(p)
        add_input(p, f['role'])

    def one_role(name):
        found = roles.get(name, [])
        check('one_role:' + name, len(found) == 1)
        return found[0]

    run_path, target_path, metric_path = map(one_role, ['model_runs', 'target_arrivals', 'comparison_metrics'])
    export = run_path.parent
    runs, targets, metrics = rows(run_path), rows(target_path), rows(metric_path)
    check('three_selected_runs', len(runs) == 3 and
          set(x['run_id'] for x in runs) == set(bundle['selected_run_ids']))
    check('21_unique_run_target_rows', len(targets) == len({(x['run_id'], x['target_id']) for x in targets}) == 21 and
          len({x['target_id'] for x in targets}) == 7)
    check('two_base_scenarios_no_statistical_weights', len({x['base_scenario_id'] for x in runs}) == 2 and
          bundle['probability_weights'] is None and
          bundle['independent_local_validation_events'] == 0)
    decision = read_json(export / 'quality/scientific_acceptance_decision_05.json')
    check('empty_accepted_envelope_list', decision['accepted_arrival_envelope_run_ids'] ==
          bundle['accepted_arrival_envelope_run_ids'] == [])
    check('decision05_false_qualification', all(decision[k] is False and bundle[k] is False
          for k in ['scientifically_qualified_for_prediction', 'mass_conservation_qualified',
          'regional_accuracy_qualified']))
    protocol = read_json(export / 'provenance/fullmask_registration/q2_fullmask_integration_protocol.json')
    check('fullmask_preregistered_selection', protocol['before_selected_fullmask_propagation'] is True and
          set(protocol['selected_scenario_ids']) == {x['scenario_id'] for x in runs} and
          protocol['release_mapping_version'] == MAPPING_VERSION and
          protocol['geometry_version'] == GEOMETRY_VERSION)

    fullroot = R / 'inputs/q2/release_bundle_v2_q2-r3-release-v2-spatial-copdem-v1'
    originalroot = R / 'inputs/q2/mechanism_bundle_q2-r3-mechanism-v1'
    analyticroot = R / 'inputs/q2/analytic_thermal_bundle_q2-r3-analytic-erfc-thermal-v1'
    full_manifest_path = add_input(fullroot / 'bundle.json', 'frozen Q3-imported Q2 geometry-v2 manifest')
    full_manifest = read_json(full_manifest_path)
    fullpath = fullroot / 'files/results/release_states_geometry_v2.csv'
    copied_fullpath = export / 'provenance/upstream_q2_fullmask/release_states_geometry_v2.csv'
    copied_manifest_path = export / 'provenance/upstream_q2_fullmask/bundle.json'
    copied_manifest = read_json(copied_manifest_path)
    check('fullmask_received_bundle_matches_registration', sha(copied_manifest_path) == protocol['upstream_bundle_sha256'])
    check('copied_and_Q3_frozen_release_tables_identical', sha(fullpath) == sha(copied_fullpath) ==
          unique(full_manifest['files'], role='release_states_geometry_v2')['sha256'] ==
          unique(copied_manifest['files'], role='release_states_geometry_v2')['sha256'])
    add_input(fullpath, 'frozen Q3 fullmask release table byte-identical to Q4 copied source')
    pp = originalroot / 'files/results/parameter_ensemble.csv'
    ap = analyticroot / 'files/results/analytic_thermal/parameter_ensemble.csv'
```

```
    cp = analyticroot / 'files/results/analytic_thermal/mechanism_contract.json'
    fp = R / 'results/engine_a/analytic_erfc/first_passages.csv'
    for manifestroot, required in [(originalroot, [pp]), (analyticroot, [ap, cp])]:
        manifest = read_json(add_input(manifestroot / 'bundle.json', 'frozen imported mechanism manifest'))
        for p in required:
            entries = [f for f in manifest['files'] if resolve(f['path']).resolve() == p.resolve()]
            check('existing_import_hash:' + relative(p), len(entries) == 1 and
                  p.stat().st_size == entries[0]['bytes'] and sha(p) == entries[0]['sha256'])
            add_input(p, 'frozen original/analytic parameters and explicit join contract')
    add_input(fp, 'frozen actual analytic-erfc first-passage results')
    contract = read_json(cp)
    check('analytic_contract_preserves_release_and_base_join', contract['parameter_version'] ==
    ↪  PARAMETER_VERSION and
          contract['base_scenario_join'] == 'base_scenario_id joins v1/v2 and release-geometry-v2; release states unchanged, use this model version
          ↪  onset/censoring when combined')
    fullrows, oldpars, newpars, onsets = rows(fullpath), rows(pp), rows(ap), rows(fp)
    compact_audit_path = add_input(R /
    ↪  'results/arrival/propagation_v2/release_mapping_audit.csv', 'frozen compact-source v2 descriptive comparison only')
    ↪  descriptive comparison only')
    compact_arrival_path = add_input(R /
    ↪  'results/arrival/propagation_v2/conditional_arrivals.csv', 'frozen compact-source target diagnostics; different source experiment')
    ↪  target diagnostics; different source experiment')
    compact_audits, compact_targets = rows(compact_audit_path), rows(compact_arrival_path)
    lineage_audit, parameter_audit, volume_audit, joined, target_audit, footprints, comparison,
    ↪  arrays = ([] for _ in range(8))

    import rasterio
    from pyproj import Transformer
    projection = Transformer.from_crs('EPSG:4326', 'EPSG:32645', always_xy=True)
    points = {x['target_id']: (float(x['x_m']), float(x['y_m']), x['geometry_version'])
              for x in rows(export / 'geometry/spatial_targets_v2.csv')}
    geom = read_json(export / 'geometry/computational_geometries.geojson')
    check('GeoJSON_coordinate_semantics', geom['crs_note'] == 'GeoJSON CRS84 longitude/latitude; distances in EPSG32645')
    ↪  longitude/latitude; distances in EPSG32645')
    for f in geom['features']:
        if f['geometry']['type'] == 'Point':
            tx, ty = projection.transform(*f['geometry']['coordinates'])
            points[f['properties']['space_id']] = (tx, ty, f['properties']['geometry_version'])

    for run in runs:
        rid, scenario, base = run['run_id'], run['scenario_id'], run['base_scenario_id']
        line = unique(bundle['lineage_by_run'], run_id=rid)
        case_path = export / f'provenance/{rid}/case.json'
        case = read_json(case_path)
        release = unique(fullrows, scenario_id=scenario, base_scenario_id=base)
        registered = unique(protocol['records'], scenario_id=scenario, base_scenario_id=base)
        embedded = case['upstream_fullmask_record']
        old = unique(oldpars, scenario_id=base)
        check('registered_model_replacement:' + rid, old['model_id'] in MODEL_REPLACEMENTS)
        mechanism, analytic_model = MODEL_REPLACEMENTS[old['model_id']]
        check('analytic_model_contract:' + rid, contract['model_ids'][mechanism] ==
        ↪  analytic_model)
        new = unique(newpars, base_scenario_id=base, model_id=analytic_model)
        onset = unique(onsets, base_scenario_id=base, model_id=analytic_model,
        ↪  mechanism_id=mechanism,
                       parameter_version=PARAMETER_VERSION,
                       ↪  geometry_version=old['geometry_version'])
        check('full_release_keys_exact:' + rid, all(
            release[k] == registered[k] == embedded[k] for k in ['scenario_id',
            ↪  'base_scenario_id',
```

```
            'model_id', 'mechanism_id', 'parameter_version', 'geometry_version',
            ↪ 'release_mapping_version',
            'parameter_set_id', 'grid_resolution_m']))
        check('bundle_run_lineage_exact:' + rid, all(run[k] == line[k] for k in [
            'scenario_id', 'base_scenario_id', 'upstream_model_id',
            ↪ 'upstream_parameter_version',
            'mechanism_id', 'parameter_version', 'geometry_version',
            ↪ 'release_mapping_version']))
        check('case_frozen_release_lineage:' + rid,
              case['run_id'] == rid and case['scenario_id'] == scenario and
              ↪ case['base_scenario_id'] == base and
              release['model_id'] == old['model_id'] == case['upstream_model_id'] ==
              ↪ line['upstream_model_id'] and
              release['mechanism_id'] == mechanism == line['mechanism_id'] and
              release['parameter_version'] == old['parameter_version'] ==
              ↪ new['parameter_version'] ==
              case['upstream_parameter_version'] == line['upstream_parameter_version'] ==
              ↪ PARAMETER_VERSION and
              case['geometry_version'] == line['geometry_version'] ==
              ↪ release['geometry_version'] == GEOMETRY_VERSION and
              case['release_mapping_version'] == line['release_mapping_version'] ==
              ↪ release['release_mapping_version'] == MAPPING_VERSION)
        check('mechanism_geometry_separate_from_source:' + rid,
              old['geometry_version'] == new['geometry_version'] == onset['geometry_version']
              ↪ == 'conditional_plane_slab_v1')
        check('design_identity_preserved:' + rid, all(old[k] == new[k] for k in [
            'parameter_set_id', 'mechanism_id', 'law_policy', 'parameter_version',
            ↪ 'geometry_version', 'design_role', 'prior_weight']))
        for k in PHYSICAL_FIELDS:
            same = close(old[k], new[k], rtol=1e-13, atol=1e-13)
            parameter_audit.append(dict(run_id=rid, base_scenario_id=base, parameter=k,
                original_value=old[k], analytic_value=new[k], unchanged=same))
            check('physical_parameter_preserved:' + rid + ':' + k, same)
        for k in ['plan_area_m2', 'mechanism_thickness_normal_m', 'mechanism_plane_theta_deg',
                  'release_thickness_vertical_m', 'release_thickness_normal_equivalent_m',
                  ↪ 'release_volume_m3',
                  'integrated_volume_m3', 'density', 'release_mass_kg']:
            check('case_and_registration_release_value:' + rid + ':' + k,
                  close(release[k], registered[k]) and close(release[k], embedded[k]))
        area = float(release['plan_area_m2'])
        vertical = float(release['release_thickness_vertical_m'])
        normal = float(release['mechanism_thickness_normal_m'])
        angle = float(release['mechanism_plane_theta_deg'])
        density, volume = float(release['density']), float(release['release_volume_m3'])
        resolution = float(release['grid_resolution_m'])
        reference_area = float(case['reference_source_grid_area_m2'])
        unnormalised_vertical = normal / math.cos(math.radians(angle))
        norm_factor = reference_area / area
        check('reference_area_and_grid_normalization:' + rid, close(reference_area, 925200) and
              close(normal, old['thickness_normal_m']) and close(angle, old['theta_deg']) and
              close(volume, reference_area * unnormalised_vertical) and close(vertical,
              ↪ unnormalised_vertical * norm_factor) and
              close(vertical, case['release_thickness_m']) and close(area,
              ↪ case['source_plan_area_m2']) and
              close(norm_factor, case['grid_thickness_normalization_factor']) and close(volume,
              ↪ area * vertical))
        fractions = json.loads(release['phase_fractions'])
        initial_velocity = json.loads(release['initial_velocity'])
        check('material_composition_and_bulk_phase:' + rid,
              release['phase_fraction_basis'] == case['passive_material_fraction_basis'] ==
              ↪ 'volume' and
              fractions == case['passive_material_volume_fractions'] and
              ↪ close(sum(fractions.values()), 1) and
```

```
      close(density, fractions['ice'] * 917 + fractions['rock'] * 2700) and
      close(density, case['bulk_density_kg_m3']) and close(density,
      ↪  old['density_kg_m3']) and
      case['phase_volume_fractions'] == [1])
check('zero_initial_velocity:' + rid, all(initial_velocity[k] == 0 for k in ['x', 'y',
↪  'z']) and
      case['initial_velocity_east_north_m_s'] == [0, 0])
check('original_no_cross_is_if_released:' + rid, release['onset_status'] ==
↪  case['upstream_onset_status'] ==
      line['upstream_onset_status'] and (release['onset_status'] != 'never_crossed' or
          (case['if_released_propagation'] and flag(run['if_released_propagation']) and
           onset['status'] == 'no_cross_observed_complete' and
           not present(onset['crossing_lower_utc']) and not
           ↪  present(onset['crossing_upper_utc']))))
check('unknown_release_lag:' + rid, case['initiation_to_release_delay_seconds'] is None
↪  and
      line['initiation_to_release_delay_seconds'] is None and not
      ↪  present(run['initiation_to_release_delay_seconds']))
check('qualification_remains_false:' + rid, all(not flag(run[k]) for k in [
    'scientifically_qualified_for_prediction', 'mass_conservation_qualified',
    ↪  'regional_accuracy_qualified',
    'include_in_arrival_envelope', 'accepted_arrival_envelope_run_membership']))
check('paper_target_coordinates_match_case:' + rid,
      all(t['id'] in points and np.allclose(points[t['id']][:2], t['xy'], rtol=0,
      ↪  atol=1e-6) for t in case['targets']))

actual_release_path = Path(case['release']).resolve()
check('actual_release_path_is_explicit_role:' + rid, actual_release_path in
↪  roles['immutable native release thickness'])
check('actual_release_hash_is_registered:' + rid, sha(actual_release_path) ==
      case['release_sha256'] == registered['release_raster_sha256'] ==
      ↪  embedded['release_raster_sha256'])
expected_tiff_role = f"release_thickness_{old['parameter_set_id']}_{int(resolution)}m"
expected_entry = unique(full_manifest['files'], role=expected_tiff_role)
frozen_tiff = resolve(expected_entry['path']).resolve()
check('frozen_Q2_tiff_pin:' + rid, frozen_tiff.is_relative_to(fullroot) and
      sha(frozen_tiff) == expected_entry['sha256'] == sha(actual_release_path) and
      frozen_tiff.stat().st_size == expected_entry['bytes'])
add_input(frozen_tiff, 'frozen Q3-imported fullmask release TIFF')
with rasterio.open(actual_release_path) as ds:
    supplied, transform, crs = ds.read(1).astype(float), ds.transform, str(ds.crs)
with rasterio.open(frozen_tiff) as ds:
    check('frozen_Q2_release_array_and_grid_identical:' + rid,
          np.array_equal(supplied, ds.read(1)) and transform == ds.transform and
          ↪  str(ds.crs) == crs)
check('release_grid_metric_axis_aligned:' + rid, crs == 'EPSG:32645' and
      transform.a == resolution and transform.e == -resolution and transform.b ==
      ↪  transform.d == 0)
positive, cell_area = supplied > 0, resolution ** 2
actual_volume = float(supplied[positive].sum() * cell_area)
check('actual_fullmask_volume_not_legacy:' + rid,
      close(actual_volume, volume, rtol=1e-12) and close(actual_volume,
      ↪  case['release_grid_volume_m3'], rtol=1e-12) and
      close(actual_volume, registered['actual_grid_volume_m3'], rtol=1e-12) and
      close(actual_volume, embedded['actual_grid_volume_m3'], rtol=1e-12) and
      not close(actual_volume, case['expected_reference_grid_volume_m3']))
check('actual_source_support:' + rid, int(positive.sum()) ==
↪  int(case['source_mask_cells']) and
      int(positive.sum()) == (257 if resolution == 60 else 67) and
      close(int(positive.sum()) * cell_area, area) and close(area,
      ↪  case['source_grid_area_m2']))
field_entry = unique(bundle['files'], role='actual_official_process_fields',
↪  run_id=rid)
```

```
        check('field_case_execution_log_binding:' + rid, sha(case_path) ==
        ↪  field_entry['case_sha256'] and
              sha(export / f'provenance/{rid}/execution.json') ==
              ↪  field_entry['execution_sha256'] and
              sha(export / f'provenance/{rid}/run.log') == field_entry['log_sha256'])
        parser = export / 'provenance/postprocessing' /
        ↪  Path(field_entry['parser_source_snapshot_path']).name
        check('copied_parser_hash_binding:' + rid, parser in roles['actual parser source
        ↪  snapshot bound to selected process fields'] and
              sha(parser) == field_entry['parser_sha256_at_start'])
        with np.load(field_entry['path'], allow_pickle=False) as z:
            time, depth, tr, stored_volume = [z[k].copy() for k in ['time_s', 'depth_frames',
            ↪  'transform', 'volume_m3']]
        check('actual_saved_depth_time_grid:' + rid, len(time) == depth.shape[0] ==
        ↪  len(stored_volume) and
              depth.shape[1:] == supplied.shape and np.all(np.isfinite(depth)) and depth.min()
              ↪  >= 0 and
              np.all(np.diff(time) > 0) and np.array_equal(tr, np.array(tuple(transform))) and
              close(time[0], 0) and close(time[-1], field_entry['time_end_seconds']) and
              ↪  close(time[-1], run['actual_end_time_seconds']))
        native_rows = []
        with (export / f'provenance/{rid}/native_internal_summary.txt').open() as stream:
            for row in csv.DictReader(stream, delimiter='\t'):
                if row['nout'].isdigit():
                    native_rows.append(row)
        check('native_summary_time_binding:' + rid, native_rows[0]['nout'] == '0' and
              close(native_rows[0]['tsum'], 0) and close(native_rows[-1]['tsum'], time[-1]))
        native_initial = float(native_rows[0]['volume'])
        native_final = float(native_rows[-1]['volume'])
        raw_validation = json.loads(run['raw_validation_json'])
        native_residual = native_final / native_initial - 1
        metric = unique(metrics, run_id=rid)
        initial_grid_volume = float(depth[0].astype(float).sum() * cell_area)
        max_initial_depth_difference = float(np.max(np.abs(depth[0] - supplied)))
        check('native_initial_and_saved_TIFF_rounding_audit:' + rid,
              max_initial_depth_difference <= 0.00051 and
              close(initial_grid_volume, stored_volume[0], rtol=1e-12) and
              close(native_initial,
              ↪  raw_validation['native_internal_summary_initial_phase_volumes_m3'][0]) and
              abs(initial_grid_volume - volume) <= 0.00051 * int(positive.sum()) * cell_area
              ↪  and
              close(max_initial_depth_difference,
              ↪  raw_validation['initial_depth_vs_supplied_max_difference_m'], rtol=1e-8))
        check('native_budget_kept_separate_and_reproduced:' + rid,
              close(native_final,
              ↪  raw_validation['native_internal_summary_final_phase_volumes_m3'][0]) and
              close(native_residual,
              ↪  metric['native_internal_summary_mass_budget_relative_residual'], rtol=1e-8)
              ↪  and
              all(x == 0 for x in raw_validation['native_signed_entrainment_phase_volumes_m3'])
              ↪  and
              all(x == 0 for x in
              ↪  raw_validation['native_edge_store_outflow_phase_volumes_m3']))
        saved_residual = float(stored_volume[-1] / stored_volume[0] - 1)
        check('saved_raster_budget_reproduced:' + rid,
              close(saved_residual,
              ↪  metric['ascii_raster_source_adjusted_budget_relative_residual'], rtol=1e-8))
        volume_audit.append(dict(run_id=rid, scenario_id=scenario, base_scenario_id=base,
        ↪  grid_resolution_m=resolution,
            source_geometry_version=GEOMETRY_VERSION, release_mapping_version=MAPPING_VERSION,
            mechanism_normal_thickness_m=normal, mechanism_plane_theta_deg=angle,
            source_exact_polygon_area_m2=case['source_exact_area_m2'],
            ↪  reference_60m_area_m2=reference_area,
```

```
            actual_grid_area_m2=area, source_support_cells=int(positive.sum()),
            grid_thickness_normalization_factor=norm_factor,
            release_vertical_thickness_m=vertical,
            reference_unnormalised_vertical_thickness_m=unnormalised_vertical,
            Q2_frozen_release_volume_m3=volume, actual_release_TIFF_volume_m3=actual_volume,
            executed_case_release_grid_volume_m3=case['release_grid_volume_m3'],
            saved_initial_depth_integral_m3=initial_grid_volume,
            NPZ_initial_volume_m3=float(stored_volume[0]),
            native_internal_initial_volume_m3=native_initial,
            native_internal_final_volume_m3=native_final,
            saved_initial_relative_difference=initial_grid_volume / volume - 1,
            saved_initial_max_depth_difference_m=max_initial_depth_difference,
            native_ASCII_depth_rounding_audit_bound_m=0.00051,
            legacy_expected_reference_grid_volume_m3=case['expected_reference_grid_volume_m3'],
            legacy_reference_used_as_actual_or_budget_denominator=False,
            density_kg_m3=density, passive_material_fractions=json.dumps(fractions,
            sort_keys=True),
            solver_phase='one equivalent_bulk phase; composition passive',
            raster_budget_relative_residual=saved_residual,
            native_internal_budget_relative_residual=native_residual,
            budget_representations_separate=True, mass_conservation_qualified=False,
            scientifically_qualified_for_prediction=False,
            actual_release_raster_sha256=sha(actual_release_path),
            source_geometry_sha256=case['source_geometry_sha256'],
            source_fields_sha256=field_entry['sha256']))
        lineage_audit.append(dict(run_id=rid, source_release_scenario_id=scenario,
        base_scenario_id=base,
            mechanism_id=mechanism, original_model_id=old['model_id'],
            analytic_model_id=analytic_model,
            analytic_scenario_id=onset['scenario_id'], parameter_version=PARAMETER_VERSION,
            mechanism_geometry_version=old['geometry_version'],
            release_geometry_version=GEOMETRY_VERSION,
            release_mapping_version=MAPPING_VERSION, grid_resolution_m=resolution,
            physical_parameters_compared=len(PHYSICAL_FIELDS),
            physical_parameters_unchanged=True,
            old_onset_status=release['onset_status'], analytic_onset_status=onset['status'],
            conditional_analytic_crossing_lower_utc=onset['crossing_lower_utc'] or None,
            conditional_analytic_crossing_upper_utc=onset['crossing_upper_utc'] or None,
            conditional_analytic_interpolated_point_utc=onset['crossing_point_utc'] or None,
            if_released_propagation=case['if_released_propagation'],
            initiation_to_release_delay_seconds=None,
            adapter_lineage_status='accepted_exact_fullmask_lineage_and_registered_analytic_re
            placement',
            evaluation_stratum='conditional_scenario',
            scientific_acceptance_decision_version=decision['version']))

        yy, xx = np.indices(supplied.shape)
        xx = transform.c + (xx + 0.5) * transform.a
        yy = transform.f + (yy + 0.5) * transform.e
        half = resolution / 2
        rectangles = [(float(xx[j, i] - half), float(xx[j, i] + half),
                       float(yy[j, i] - half), float(yy[j, i] + half)) for j, i in
                       zip(*np.nonzero(positive))]
        distance = support_distance(xx, yy, rectangles)
        horizon = float(run['requested_horizon_seconds'])
        permitted = time <= horizon
        requested_max, all_max = depth[permitted].max(axis=0), depth.max(axis=0)
        check('saved_impact_area_reproduced:' + rid,
              close(np.count_nonzero(all_max >= 0.1) * cell_area, metric['impact_area_m2']))
        for scope, hmax, last in [('saved_outputs_through_requested_horizon', requested_max,
        float(time[permitted][-1])),
                                  ('all_actual_saved_outputs', all_max, float(time[-1]))]:
```

```
            for threshold in [0.1, 1, 5]:
                wet = hmax >= threshold
                footprints.append(dict(run_id=rid, scenario_id=scenario, scope=scope,
                    requested_horizon_seconds=horizon, last_included_saved_seconds=last,
                    grid_resolution_m=resolution,
                    threshold_depth_m=threshold, affected_cell_count=int(wet.sum()),
                    affected_plan_area_m2=float(wet.sum() * cell_area),
                    maximum_affected_cell_centre_distance_from_release_support_m=float(distanc
                    e[wet].max()) if wet.any() else None,
                    distance_definition='Euclidean cell-centre to union of positive actual
                    per-run release raster cell footprints',
                    metric_crs='EPSG:32645', source_geometry_version=GEOMETRY_VERSION,
                    source_geometry_is_observed_release=False,
                    footprint_is_validated_hazard_zone=False,
                    validated_warning_distance_m=None,
                    scientifically_qualified_for_prediction=False,
                    mass_conservation_qualified=False, arrival_probability=None,
                    source_fields_sha256=field_entry['sha256']))
        arrays.append((scenario, requested_max, positive, xx, yy))
        for target in [x for x in targets if x['run_id'] == rid]:
            tid = target['target_id']
            check('target_exact_fullmask_lineage:' + rid + ':' + tid, all(target[k] == run[k]
            for k in [
                'scenario_id', 'base_scenario_id', 'upstream_model_id',
                'upstream_parameter_version',
                'geometry_version', 'release_mapping_version', 'parameter_version']))
            check('target_qualification_false:' + rid + ':' + tid, all(not flag(target[k]) for
            k in [
                'include_in_arrival_envelope', 'scientifically_qualified_for_prediction',
                'mass_conservation_qualified',
                'regional_accuracy_qualified', 'accepted_arrival_envelope_run_membership']) and
                not present(target['probability_weight']) and not
                present(target['initiation_to_release_delay_seconds']))
            check('target_geometry_binding:' + rid + ':' + tid,
                  tid in points and points[tid][2] == target['target_geometry_version'])
            tx, ty, _ = points[tid]
            mask = np.hypot(xx - tx, yy - ty) <= 100
            actual, formal = saved_first_hit(time, depth, mask, float(time[-1])),
            saved_first_hit(time, depth, mask, horizon)
            agrees = actual['hit'] == flag(target['observed_hit']) and all(
                (actual[key] is None and not present(target[column])) or
                (actual[key] is not None and present(target[column]) and close(actual[key],
                target[column]))
                for key, column in [('lower', 'arrival_bracket_lower_seconds'), ('upper',
                'arrival_bracket_upper_seconds')])
            check('independent_field_target_reproduction:' + rid + ':' + tid,
                  agrees and target['source_fields_sha256'] == field_entry['sha256'] and
                  flag(target['target_field_hash_matches']))
            target_audit.append(dict(run_id=rid, target_id=tid, grid_resolution_m=resolution,
                target_buffer_cell_count=int(mask.sum()),
                actual_last_saved_seconds=actual['last_saved'],
                recomputed_actual_status=actual['status'],
                actual_first_hit_lower_seconds=actual['lower'],
                actual_first_hit_upper_seconds=actual['upper'],
                requested_horizon_status=formal['status'],
                requested_last_saved_seconds=formal['last_saved'],
                requested_first_hit_lower_seconds=formal['lower'],
                requested_first_hit_upper_seconds=formal['upper'],
                original_actual_hit_and_bracket_agree=agrees,
                source_field_binding_agrees=True,
                target_point_to_release_raster_support_distance_m=float(
                    support_distance(np.array(tx), np.array(ty), rectangles))))
```

```
            joined.append(dict(run_id=rid, scenario_id=scenario, base_scenario_id=base,
            ↪  event_id=bundle['common_context']['event_id'],
                source_analytic_scenario_id=onset['scenario_id'],
                ↪  original_upstream_model_id=old['model_id'],
                analytic_onset_model_id=analytic_model, propagation_model_id=run['model_id'],
                ↪  mechanism_id=mechanism,
                parameter_version=PARAMETER_VERSION,
                ↪  mechanism_geometry_version=old['geometry_version'],
                release_geometry_version=GEOMETRY_VERSION,
                ↪  release_mapping_version=MAPPING_VERSION, grid_resolution_m=resolution,
                target_id=tid, entity_id=target['entity_id'] or None,
                ↪  target_kind=target['target_kind'] or None,
                target_geometry_version=target['target_geometry_version'],
                ↪  source_first_passage_status=onset['status'],
                original_upstream_onset_status=release['onset_status'],
                ↪  conditional_onset_lower_utc=onset['crossing_lower_utc'] or None,
                conditional_onset_upper_utc=onset['crossing_upper_utc'] or None,
                ↪  forecast_input_mode=onset['forecast_input_mode'],
                if_released_propagation=case['if_released_propagation'],
                ↪  initiation_to_release_delay_seconds=None,
                time_origin='bulk_release',
                ↪  physical_phase='conditional_bulk_release_to_target_arrival',
                raw_conditional_arrival_status=target['arrival_status'],
                ↪  diagnostic_bulk_release_arrival_lower_seconds=actual['lower'],
                diagnostic_bulk_release_arrival_upper_seconds=actual['upper'],
                ↪  diagnostic_arrival_within_requested_horizon=formal['hit'],
                requested_horizon_seconds=horizon,
                ↪  last_saved_within_requested_horizon_seconds=formal['last_saved'],
                actual_last_saved_seconds=actual['last_saved'],
                ↪  arrival_definition=target['arrival_definition'],
                diagnostic_observation_available=flag(target['diagnostic_observation_available
                ↪  ']),
                conditional_diagnostic_range_eligible=flag(target['conditional_diagnostic_rang
                ↪  e_eligible']),
                diagnostic_eligibility_is_qualification=False, absolute_arrival_lower_utc=None,
                absolute_arrival_upper_utc=None, absolute_arrival_time_utc=None,
                ↪  qualified_arrival_lower_seconds=None,
                qualified_arrival_upper_seconds=None, arrival_probability=None,
                ↪  probability_weight=None,
                include_in_arrival_envelope=False,
                ↪  accepted_arrival_envelope_run_membership=False,
                numerical_qualification=run['numerical_qualification'],
                ↪  mass_conservation_qualified=False,
                regional_accuracy_qualified=False,
                ↪  scientifically_qualified_for_prediction=False,
                scientific_acceptance_decision_version=decision['version'],
                arrival_envelope_semantics_version=bundle['arrival_envelope_semantics_version'
                ↪  ],
                event_timing_validated=False, arrival_is_initiation_truth=False,
                ↪  evaluation_stratum='conditional_scenario',
                independent_local_validation_events=0,
                ↪  adapter_lineage_status='accepted_exact_fullmask_lineage',
                arrival_status='unqualified_conditional_diagnostic;unknown_initiation_to_bulk_
                ↪  release_lag',
                source_fields_sha256=field_entry['sha256']))
        compact = unique(compact_audits, base_scenario_id=base)
        oldhit = unique(compact_targets, base_scenario_id=base, target_id='paper_junction')
        currenthit = unique(target_audit, run_id=rid, target_id='paper_junction')
        comparison.append(dict(run_id=rid, base_scenario_id=base,
        ↪  fullmask_grid_resolution_m=resolution,
            fullmask_source_area_m2=area, fullmask_actual_volume_m3=actual_volume,
            compact_run_id=compact['run_id'], compact_grid_resolution_m=60,
```

```
            compact_source_plan_area_m2=compact['original_plan_area_m2'],
            compact_volume_m3=compact['original_volume_m3'],
            source_volume_ratio=actual_volume / float(compact['original_volume_m3']),
            compact_junction_diagnostic_lower_seconds=oldhit['diagnostic_bulk_release_arrival_lower_seconds'],
            compact_junction_diagnostic_upper_seconds=oldhit['diagnostic_bulk_release_arrival_upper_seconds'],
            fullmask_junction_diagnostic_lower_seconds=currenthit['actual_first_hit_lower_seconds'],
            fullmask_junction_diagnostic_upper_seconds=currenthit['actual_first_hit_upper_seconds'],
            same_mechanical_parameters=True, changed_computational_source=True,
            changed_grid_resolution=resolution != 60,
            causal_geometry_effect_or_model_gain_validated=False,
            both_results_scientifically_unqualified=True,
            comparison_scope='descriptive different-source experiment; not validation, probability or qualified model gain'))

    check('all21_lineage_pairs_retained', len(joined) == 21)
    check('3raw_junction_hits18_saved_nonhits', sum(x['actual_first_hit_upper_seconds'] is not None for x in target_audit) == 3 and
          all(x['target_id'] == 'paper_junction' for x in target_audit if x['actual_first_hit_upper_seconds'] is not None))
    check('all_absolute_qualified_probability_fields_null', all(all(x[k] is None for k in [
        'absolute_arrival_lower_utc', 'absolute_arrival_upper_utc', 'absolute_arrival_time_utc',
        'qualified_arrival_lower_seconds', 'qualified_arrival_upper_seconds', 'arrival_probability', 'probability_weight']) for x in joined))
    check('all_no_accepted_envelope_membership', all(not x['include_in_arrival_envelope'] and
          not x['accepted_arrival_envelope_run_membership'] for x in joined))
    check('p36_actual_July8_analytic_crossing', all(x['analytic_onset_status'] == 'point_crossing_linear_between_samples' and
          x['conditional_analytic_crossing_lower_utc'] == '2026-07-08T15:00:00+00:00' and
          x['conditional_analytic_crossing_upper_utc'] == '2026-07-08T16:00:00+00:00'
          for x in lineage_audit if x['base_scenario_id'] == 'q2r3-p0036-S_T-linear'))
    write_csv(out / 'lineage_audit.csv', lineage_audit)
    write_csv(out / 'physical_parameter_audit.csv', parameter_audit)
    write_csv(out / 'release_mapping_audit.csv', volume_audit)
    write_csv(out / 'conditional_arrivals.csv', joined)
    write_csv(out / 'saved_field_target_checks.csv', target_audit)
    write_csv(out / 'depth_footprint_diagnostics.csv', footprints)
    write_csv(out / 'compact_v2_comparison.csv', comparison)
    write_csv(out / 'rejected_pairings.csv', [], ['run_id', 'target_id', 'reason'])
    write_json(out / 'adapter_schema.json', dict(version='q3-q4-fullmask-adapter-v3',
        exact_keys=['full_release_scenario_id', 'base_scenario_id', 'model_id', 'mechanism_id',
                    'parameter_version', 'release_mapping_version', 'source_geometry_version',
                    'grid_resolution_m'],
        mechanism_model_replacement=MODEL_REPLACEMENTS, physical_fields=PHYSICAL_FIELDS,
        source_volume_authority='Q2 frozen geometry-v2 row + actual supplied release TIFF integral + case actual grid volume + audited native/saved initial state',
        legacy_reference_field_is_actual_volume=False,
        source_geometry_is_mechanism_geometry=False,
        material_fractions_are_solver_phase_fractions=False,
        selected_runs_are_probability_ensemble=False,
        accepted_arrival_envelope_run_ids=[], absolute_release_lag_unknown=True,
        semantic_adapter_pass_is_predictive_qualification=False))

    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    fig, axes = plt.subplots(1, 3, figsize=(16, 5), constrained_layout=True)
    for ax, (scenario, hmax, positive, xx, yy) in zip(axes, arrays):
```

```
        image = ax.pcolormesh(xx / 1000, yy / 1000, np.ma.masked_where(hmax < 0.1, hmax),
                              cmap='viridis', shading='nearest', vmin=0, vmax=120)
        ax.contour(xx / 1000, yy / 1000, positive.astype(float), levels=[0.5], colors=['red'], linewidths=1)
        for tid, (tx, ty, _) in points.items():
            ax.scatter(tx / 1000, ty / 1000, marker='x', color='black', s=12)
            if tid in ['paper_junction', 'paper_port_symbol']:
                ax.annotate(tid.replace('paper_', ''), (tx / 1000, ty / 1000), xytext=(3, 3), textcoords='offset points', fontsize=7)
        ax.set_aspect('equal')
        ax.set_title(scenario.replace('__geomv2_', '\n') + '\nUNQUALIFIED; saved outputs <=1800s', fontsize=9)
        ax.set_xlabel('Easting (km; EPSG:32645)')
        ax.set_ylabel('Northing (km; EPSG:32645)')
    fig.colorbar(image, ax=axes, label='Saved-output max depth (m; shown >=0.1m)', shrink=0.85)
    fig.suptitle('Full-source conditional diagnostics; red = actual release raster support; black x = target representatives\nNo absolute arrival, arrival probability or validated warning distance', fontsize=11)
    fig.savefig(out / 'conditional_footprints.png', dpi=180)
    plt.close(fig)
    summary = dict(status='fullmask_v3_adapter_executed_unqualified_diagnostics_only', bundle_version=bundle['bundle_version'],
        bundle_sha256=PIN, producer_bundle_sha256=PRODUCER_PIN, delivered_files_verified=66,
        selected_physical_runs=3, unique_base_parameter_scenarios=2, representative_targets=7,
        osm_representative_points=5, osm_candidate_entities=3, paper_candidate_points=2,
        run_target_rows=21, accepted_lineage_pairs=21, rejected_lineage_pairs=0,
        raw_saved_junction_hits=3, raw_saved_target_nonhits=18, independent_local_validation_events=0,
        qualified_arrival_rows=0, qualified_arrival_envelope_rows=0, absolute_arrival_rows=0,
        probability_rows=0, probability_weights=None, selected_runs_are_2048_probability_ensemble=False,
        physical_parameters_compared_per_run=len(PHYSICAL_FIELDS), release_mapping_version=MAPPING_VERSION,
        geometry_version=GEOMETRY_VERSION, legacy_reference_volume_used_as_actual=False,
        native_saved_initial_volume_audited=True, mass_conservation_qualified=False,
        regional_accuracy_qualified=False, scientifically_qualified_for_prediction=False,
        accepted_arrival_envelope_run_ids=[], scientific_acceptance_decision_version=decision['version'],
        no_physical_solver_rerun=True, no_existing_frozen_file_modified=True)
    write_json(out / 'summary.json', summary)
    write_json(out / 'status.json', summary)
    (out / 'RESULTS.md').write_text('''# Full-source propagation v3 adapter results

All 66 delivered role objects were verified by exact path, bytes and SHA256. The imported packet and producer snapshot are separately pinned. Exact frozen Q2 geometry-v2 and analytic-erfc joins retain all **21 rows: 3 runs x 7 representative points**, representing only **2 selected base parameter scenarios**. Five OSM points describe three candidate entities; the two paper points are candidate objects. No independent validation event or statistical weight is created.

The source is the full paper-candidate computational mask, distinct from the compact v2 experiment. At 60m the source has **257 cells / 925,200m²**; at 120m **67 cells / 964,800m²**. The declared exact polygon area (927,822.8616m²) is a different area definition. p0000-60 releases **51,755,728.59264966m³**; p0036 at either resolution releases **31,184,139.1284297m³**. At 120m the vertical thickness is **32.3218689142m**, normalized from 33.7052952101m by 925200/964800. The mechanical normal thickness and all 20 shared physical parameters remain unchanged. Actual TIFF arrays, hashes, grid transforms and integrals match the frozen Q3-imported Q2-v2 rasters. The inherited **13,878,000m³ expected-reference field is stale metadata**, separately audited and never used as actual release volume or a budget denominator.
```

```
The explicit analytic model replacement retains p0000's **no_cross_observed_complete / if-released** status with no onset timestamp. p0036 retains the actual conditional July 8 15:00–16:00 UTC analytic crossing bracket (interpolation 15:34:17.258933921 UTC). These ideal-weather retrospective mechanical first-crossing times are not observed bulk release or operational forecasts.

Actual saved native depth and native times reproduce the three paper-junction threshold brackets:

| Selected run | Bulk-release-relative diagnostic interval (s) |
| --- | --- |
| p0000 S, 60m | (161.1, 180.2] |
| p0036 ST, 120m | (240.6, 260.1] |
| p0036 ST, 60m | (221.0, 240.7] |

Detection is h>=0.1m at registered cell centres within a 100m representative-point buffer, with previous-below/first-above saved-output brackets. The other 18 records have no threshold hit at saved outputs; this does not establish continuous-time safety. Requested 1800s and actual slightly later final output times are retained separately.

All three runs remain **scientifically, regionally and mass-conservation unqualified** under decision05's empty accepted-envelope list. Execution diagnostic eligibility does not confer acceptance. Saved-raster source-adjusted residuals are **+0.947210%, -0.397350%, +1.663189%**; native internal residuals are separately **+0.947784%, -0.388377%, +1.665052%**. No percentage observed here is designated a physical pass threshold. The depth-rounding bound in the initial-input audit is an I/O consistency check, not conservation or prediction certification. All absolute arrivals, qualified intervals, probabilities and weights remain null, because acceptance is absent and the initiation-to-bulk-release delay remains unknown.

The footprint table and three-panel figure read only actual stored fields. Depth thresholds 0.1/1/5m are descriptive; area is affected cell area. Distance is affected cell-centre Euclidean distance to the actual per-grid positive-release raster cell footprints. It is not a centroid/path length, exact observed polygon runout, validated hazard zone or warning distance. The compact-v2 comparison retains the different source assumption and grid difference; it is not a validated model gain or a weighted 2048-member forecast.

Reproduce from the Q3 worktree:

```sh
OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 VECLIB_MAXIMUM_THREADS=1 track_b/.venv/bin/python track_b/research/q3/round3/scripts/join_propagation_v3.py
```

`review/` is independently owned and is excluded from this script's output index. Root seals that review and these results together. No old script, result, bundle, or portable manifest is edited by this adapter.
''')
    add_input(out / 'PROTOCOL.md', 'registered precomputation protocol')
    add_input(Path(__file__), 'actually executed fullmask adapter source')
    for entry in input_by_path.values():
        p = resolve(entry['path'])
        check('input_unchanged_during_adapter:' + entry['path'],
              p.stat().st_size == entry['bytes'] and sha(p) == entry['sha256'])
    figure_review = out / 'figure_review.json'
    if figure_review.exists():
        visual = read_json(figure_review)
        check('actual_figure_review_binding', visual.get('passed') is True and
              len(visual.get('figures', [])) == 1 and visual['figures'][0]['actually_opened'] is True and
              resolve(visual['figures'][0]['path']) == out / 'conditional_footprints.png' and
              visual['figures'][0]['sha256'] == sha(out / 'conditional_footprints.png'))
```

```
    input_records = sorted(input_by_path.values(), key=lambda x: x['path'])
    check('input_index_has_no_repeated_canonical_paths', len(input_records) == len({x['path'] for x in input_records}))
    write_json(out / 'input_index.json', dict(files=input_records, file_list_excludes_self=True,
        exact_role_objects_verified=66, roles_merged_for_repeated_canonical_paths=True,
        path_resolution='repository entries Q3 ROOT-relative; external entries exact approved cache path',
        source_and_upstream_metadata_paths_are_not_read_permissions=True,
        no_active_other_question_worktree_reads=True))
    output_records = [file_record(p, 'new fullmask adapter output') for p in sorted(out.iterdir())
        if p.is_file() and p.name not in ['output_index.json', 'validation.json']]
    write_json(out / 'output_index.json', dict(files=output_records, file_list_excludes_self=True,
        validation_excluded_to_avoid_cycle=True, independent_review_directory_excluded=True,
        path_resolution='Q3 ROOT-relative repository paths; no recursive output traversal'))
    validation = dict(passed=all(x['passed'] for x in checks), generated_at_utc=datetime.now(timezone.utc).isoformat(),
        check_count=len(checks), checks=checks, adapter_pass_is_predictive_qualification=False,
        input_index_sha256=sha(out / 'input_index.json'), output_index_sha256=sha(out / 'output_index.json'),
        figure_visual_qa='passed_external_visual_inspection' if figure_review.exists() else 'pending_external_visual_inspection')
    write_json(out / 'validation.json', validation)
    print(json.dumps(summary, ensure_ascii=False))
    print(f'Adapter QA: {len(checks)}/{len(checks)} passed; inputs={len(input_records)}, outputs={len(output_records)}')


if __name__ == '__main__':
    main()

```

程序 32：机械状态更新、首过阈、权重、信息界与导出通用逻辑

来源：track_b/research/q3/round3/scripts/mechanistic_replay.py
SHA256: 4610d63dc9140ae0c61d011bd768f134c617b008a8501b352444ba8c7edbfa75

```
#!/usr/bin/env python3
"""Warm-restart the delivered Q2 physics under per-issue actual GFS scenarios.

No local truth or historical public-availability claim. See the preregistration.
Only existing NumPy/pandas/pyarrow are used; no networking or installations.
"""
from __future__ import annotations
import argparse, csv, gzip, hashlib, io, json, types, warnings
from collections import Counter
from datetime import datetime, timedelta, timezone
from pathlib import Path
import numpy as np
import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq

warnings.filterwarnings('ignore',message="The 'generic' unit for NumPy timedelta is deprecated",category=DeprecationWarning)
R=Path(__file__).resolve().parents[1];ROOT=R.parents[3]
OUT=R/'results/mechanistic_replay'
BUNDLE=R/'inputs/q2/mechanism_bundle_q2-r3-mechanism-v1/bundle.json'
EXPERIMENT='q3r3-full-mechanistic-weather-replay-v1'
PRIORS=['uniform_design_assumption','equal_mechanism_then_design_assumption','fragile_issue_initial_margin_assumption']
HORIZONS=[24,48,72]
START=datetime(2026,8,23,tzinfo=timezone.utc);END=datetime(2026,8,26,2,tzinfo=timezone.utc)

def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
def stamp(t):return t.isoformat().replace('+00:00','Z')
def dt(s):return datetime.fromisoformat(str(s).replace('Z','+00:00'))
def num(s):return float(s) if s not in ['',None] else np.nan
def clean(x):
    if isinstance(x,dict):return {k:clean(v) for k,v in x.items()}
    if isinstance(x,(list,tuple)):return [clean(v) for v in x]
    if isinstance(x,np.generic):x=x.item()
    if isinstance(x,float) and not np.isfinite(x):return None
    return x
def dump(p,obj):p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(clean(obj),indent=2,ensure_ascii=False,allow_nan=False)+'\n')
def csv_read(p):return list(csv.DictReader(p.open()))
def csv_out(p,rows):
    frame=pd.DataFrame(rows)
    if p.suffix=='.gz':frame.to_csv(p,index=False,compression={'method':'gzip','mtime':0})
    else:frame.to_csv(p,index=False)

def load_inputs(mechanisms=None):
    assert ROOT==Path('<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q3')
    assert (R/'MECHANISTIC_REPLAY.md').exists(),'preregister before computing'
    b=json.loads(BUNDLE.read_text());files={}
    for item in b['files']:
        p=ROOT/item['path']
        assert p.is_relative_to(R/'inputs') and p.stat().st_size==item['bytes'] and sha(p)==item['sha256']
        files[item['role']]=p
    aliases={
      'scripts/mechanism_engine.py':['frozen_local_mechanism_engine'],
      'config/experiment_v1.json':['config.json'],
```

```
      'results/parameter_ensemble.csv':['parameter_ensemble.csv'],
      'results/forcing_hourly_v1.csv':['forcing_hourly_v2.csv'],
      'results/scenario_outcomes.csv':['scenario_outcomes.csv'],
      'results/state_trajectories.parquet':['state_trajectories.parquet']}
    for canonical,alternatives in aliases.items():
        if canonical not in files:
            available=[files[k] for k in alternatives if k in files]
            assert len(available)==1,f'No unambiguous delivered role for {canonical}'
            files[canonical]=available[0]
    # Execute read-only source without generating __pycache__ in delivered input.
    module=types.ModuleType('q2_delivered_physics')
    exec(compile(files['scripts/mechanism_engine.py'].read_text(),str(files['scripts/mechanism_engine.py']),'exec'),module.__dict__)
    ensemble=pd.read_csv(files['results/parameter_ensemble.csv'])
    forcing=pd.read_csv(files['results/forcing_hourly_v1.csv'])
    config=json.loads(files['config/experiment_v1.json'].read_text())
    assert len(ensemble)==2048 and ensemble.scenario_id.is_unique
    if 'base_scenario_id' not in ensemble:ensemble['base_scenario_id']=ensemble.scenario_id
    if mechanisms:
        ensemble=ensemble[ensemble.mechanism_id.isin(mechanisms)].reset_index(drop=True)
        assert len(ensemble)>0 and set(ensemble.mechanism_id)==set(mechanisms)
    assert len(forcing)==2089
    clock=[dt(x) for x in forcing.time_utc]
    assert all((y-x)==timedelta(hours=1) for x,y in zip(clock[:-1],clock[1:]))
    air=forcing.station_air_temp_c.to_numpy(float);rain=forcing.precip_assumed_mm_h.to_numpy(float)
    assert np.isfinite(air).all() and np.isfinite(rain).all() and (rain>=0).all()
    # Preserve exact executed code and the preregistration used by this output
    # version. Later documentation additions cannot rewrite that input history.
    sources=OUT/'execution_source';sources.mkdir(parents=True,exist_ok=True)
    raw=Path(__file__).read_bytes()
    code_snapshot=sources/f'mechanistic_replay_{hashlib.sha256(raw).hexdigest()}.py.txt'
    if code_snapshot.exists():assert code_snapshot.read_bytes()==raw
    else:code_snapshot.write_bytes(raw)
    registration=sources/f"registered_protocol_{sha(R/'MECHANISTIC_REPLAY.md')}.md"
    if not registration.exists():registration.write_bytes((R/'MECHANISTIC_REPLAY.md').read_bytes())
    for p in [R/'inputs/weather/gfs_hourly_summary.csv',registration,code_snapshot,BUNDLE]:
        files['consumer:'+str(p.relative_to(R))]=p
    return module,b,files,ensemble,forcing,config,clock,air,rain

class Physics:
    """Exact Q2 update equations with complete restart state, not a new model."""
    def __init__(self,q,ensemble,config):
        self.q=q;self.f=ensemble;self.n=len(ensemble)
        self.names=list(config['ranges'])+['density_kg_m3']
        self.p={k:ensemble[k].to_numpy(float) for k in self.names}
        bases=ensemble.drop_duplicates('parameter_set_id').sort_values('parameter_set_id')
        self.baseids=list(bases.parameter_set_id);self.nb=len(bases)
        self.bp={k:bases[k].to_numpy(float) for k in self.names}
        self.baseindex=np.array([self.baseids.index(x) for x in ensemble.parameter_set_id])
        # Physical parameters must agree for every mechanism/policy clone.
        assert self.nb==256 and all(np.array_equal(self.p[k],self.bp[k][self.baseindex]) for k in self.names)
        self.has_t=ensemble.mechanism_id.isin(['S_T','S_T_H']).to_numpy()
        self.has_h=ensemble.mechanism_id.isin(['S_H','S_T_H']).to_numpy()
        self.policies={k:np.flatnonzero(ensemble.law_policy.eq(k)) for k in ['linear_extension','thawed_rock_contact']}
        self.policy_params={k:{a:v[idx] for a,v in self.p.items()} for k,idx in self.policies.items()}
        self.sn,self.drive=q.stresses(self.p)
        self.capacity=np.minimum(self.p['connected_depth_m'],.98*self.sn/q.G)*self.p['storativity']
```

```
        self.dz=config['dz_m'];self.bottom=config['thermal_bottom_m'];self.dt=config['dt_hours']
        assert self.dt==1.
        self.z=np.arange(0,self.bottom+self.dz*.5,self.dz)
        ratio=self.bp['diffusivity_m2_s']*self.dt*3600/self.dz**2
        self.substeps=max(1,int(np.ceil(ratio.max()/.45)));self.rr=ratio[:,None]/self.substeps
        self.k=np.floor(self.bp['interface_depth_m']/self.dz).astype(int)
        self.frac=self.bp['interface_depth_m']/self.dz-self.k;self.row=np.arange(self.nb)
        assert self.bp['interface_depth_m'].max()<self.bottom
        self.water_budget_max=0.

    def initial(self):
        return np.repeat(self.bp['initial_temp_c'][:,None],len(self.z),axis=1),self.p['initial_damage'].copy(),np.zeros(self.n)

    def thermal_step(self,profile,air):
        profile=profile.copy()
        surf=np.minimum(air+self.bp['station_source_offset_c'],0.)
        for _ in range(self.substeps):
            old=profile.copy();old[:,0]=surf
            profile[:,1:-1]=old[:,1:-1]+self.rr*(old[:,2:]-2*old[:,1:-1]+old[:,:-2])
            profile[:,-1]=old[:,-1]+2*self.rr[:,0]*(old[:,-2]-old[:,-1])
            profile[:,0]=surf
        return profile

    def interface_base(self,profile):
        return profile[self.row,self.k]*(1-self.frac)+profile[self.row,self.k+1]*self.frac

    def observe(self,profile,damage,storage):
        ib=self.interface_base(profile)
        ti=np.where(self.has_t,ib[self.baseindex],self.p['initial_temp_c'])
        pressure=np.where(self.has_h,self.q.G*storage/self.p['storativity'],0.)
        margin=np.empty(self.n);resist=np.empty(self.n);domain=np.empty(self.n,bool)
        for policy,idx in self.policies.items():
            out=self.q.resistance(self.policy_params[policy],damage[idx],ti[idx],pressure[idx],policy)
            margin[idx]=out['failure_margin'];resist[idx]=out['resistance_kpa'];domain[idx]=out['strength_domain']
        return margin,resist,domain,ti,pressure

    def advance(self,profile,damage,storage,air,rain,resist):
        assert np.isfinite(air) and np.isfinite(rain) and rain>=0
        ratio=self.drive/np.maximum(resist,1e-6)
        rate=self.p['damage_rate_h_inv']*np.minimum(ratio,10.)**self.p['damage_exponent']
        damage_next=np.minimum(1-(1-damage)*np.exp(-rate*self.dt),.999999)
        source_air=air+self.p['station_source_offset_c']
        rain_fraction=np.clip((source_air+1)/2,0,1)
        incoming=self.p['rain_multiplier']*rain/1000*rain_fraction+self.p['melt_mm_deg_day']*np.maximum(source_air,0)/24000
        incoming=np.where(self.has_h,incoming,0.)
        tau=self.p['drainage_hours']*np.exp(-self.p['drainage_damage_coupling']*damage)
        storage_next,drain,over=self.q.reservoir_step(storage,incoming,tau,self.capacity,self.dt)
        self.water_budget_max=max(self.water_budget_max,float(np.max(np.abs(storage+incoming*self.dt-storage_next-drain-over))))
        return self.thermal_step(profile,air),damage_next,storage_next

def continuous_history(engine,clock,air,rain,issues):
    nt=len(clock);n=engine.n
    margins=np.empty((nt,n));damage_hist=np.empty((nt,n));storage_hist=np.empty((nt,n));ibase=np.empty((nt,engine.nb))
    first=np.full(n,np.nan);left=np.zeros(n,bool);minmargin=np.full(n,np.inf);peakp=np.zeros(n)
    prof,dmg,store=engine.initial();snapshots={}
```

```
    ix={x:i for i,x in enumerate(clock)};issue_idx=[ix[x] for x in issues]
    wanted=set(issue_idx+[min(i+72,nt-1) for i in issue_idx])
    initial_m=None;previous=None
    for j in range(nt):
        margin,resist,domain,ti,pressure=engine.observe(prof,dmg,store)
        margins[j]=margin;damage_hist[j]=dmg;storage_hist[j]=store;ibase[j]=engine.interface_base(prof)
        minmargin=np.minimum(minmargin,margin);peakp=np.maximum(peakp,pressure)
        if j==0:
            initial_m=margin.copy();left=margin<=0;first[left]=0
        else:
            hit=np.isnan(first)&(margin<=0)
            first[hit]=(j-1)+previous[hit]/(previous[hit]-margin[hit])
        if j in wanted:snapshots[j]=prof.copy()
        if j<nt-1:prof,dmg,store=engine.advance(prof,dmg,store,air[j],rain[j],resist)
        previous=margin
    return dict(margin=margins,damage=damage_hist,storage=storage_hist,interface_base=ibase,profiles=snapshots,
      first_elapsed=first,left=left,initial_margin=initial_m,min_margin=minmargin,peak_pressure=peakp,issue_indices=issue_idx,clock_index=ix)

def history_validation(history,engine,ensemble,files):
    q2=pd.read_csv(files['results/scenario_outcomes.csv']).set_index('scenario_id').loc[ensemble.scenario_id]
    expected_status=np.where(history['left'],'left_censored_already_unstable',np.where(np.isfinite(history['first_elapsed']),'interval_crossing','never_crossed'))
    diffs={
      'final_margin':float(np.max(abs(history['margin'][-1]-q2.final_margin.to_numpy(float)))),
      'minimum_margin':float(np.max(abs(history['min_margin']-q2.min_margin_all.to_numpy(float)))),
      'final_damage':float(np.max(abs(history['damage'][-1]-q2.final_damage.to_numpy(float)))),
      'peak_pressure':float(np.max(abs(history['peak_pressure']-q2.peak_pressure_kpa.to_numpy(float))))}
    finalti=np.where(engine.has_t,history['interface_base'][-1,engine.baseindex],engine.p['initial_temp_c'])
    diffs['final_interface_temperature']=float(np.max(abs(finalti-q2.final_interface_temp_c.to_numpy(float))))
    target=q2.onset_elapsed_hours.to_numpy(float);mask=np.isfinite(target)
    diffs['onset_elapsed_hours']=float(np.max(abs(history['first_elapsed'][mask]-target[mask])) if mask.any() else 0.
    statuses_match=np.array_equal(expected_status,q2.onset_status.to_numpy())
    representative=pd.read_parquet(files['results/state_trajectories.parquet'])
    representative=representative[representative.scenario_id.isin(ensemble.scenario_id)].copy()
    lookup={s:i for i,s in enumerate(ensemble.scenario_id)}
    columns={'failure_margin':'margin','damage':'damage','storage_m':'storage'}
    repdiff={}
    ti=pd.to_datetime(representative.time_utc,utc=True)
    origin=pd.Timestamp('2026-06-01T00:00Z')
    rows=np.rint((ti-origin).dt.total_seconds().to_numpy()/3600).astype(int)
    cols=np.array([lookup[s] for s in representative.scenario_id])
    for raw,key in columns.items():
        if raw in representative:repdiff[raw]=float(np.max(abs(history[key][rows,cols]-representative[raw].to_numpy(float))))
    if 'interface_temp_c' in representative:
        a=np.where(engine.has_t[cols],history['interface_base'][rows,engine.baseindex[cols]],engine.p['initial_temp_c'][cols])
        repdiff['interface_temp_c']=float(np.max(abs(a-representative.interface_temp_c.to_numpy(float))))
    if 'pressure_kpa' in representative:
        a=np.where(engine.has_h[cols],engine.q.G*history['storage'][rows,cols]/engine.p['storativity'][cols],0)
        repdiff['pressure_kpa']=float(np.max(abs(a-representative.pressure_kpa.to_numpy(float))))
```

```
    result=dict(full_design_rows=len(q2),status_match=statuses_match,maximum_absolute_differences=diffs,
      representative_rows=len(representative),representative_scenarios=int(representative.scenario_id.nunique()),representative_maximum_absolute_differences=repdiff,
      passed=statuses_match and all(v<1e-8 for v in [*diffs.values(),*repdiff.values()]))
    if not result['passed']:raise AssertionError(result)
    return result

def weather_lookup():
    rows=csv_read(R/'inputs/weather/gfs_hourly_summary.csv');lookup={};refs={}
    for row in rows:
        space=row['space_object'];ref=dt(row['forecast_reference_time']);valid=dt(row['valid_time'])
        lookup[(space,ref,valid)]=row;refs.setdefault(space,set()).add(ref)
    return lookup,{k:sorted(v) for k,v in refs.items()}

def get_forcing(mode,space,latency,issue,j,air,rain,clock,weather,refs):
    if mode=='historically_available':return np.array([]),np.array([]),None,'rejected_historical_first_availability_unknown'
    if mode=='ideal_weather':
        length=min(72,len(clock)-1-j)
        return air[j:j+length],rain[j:j+length],None,'complete_72h' if length==72 else 'right_censored_at_original_forcing_end'
    eligible=[r for r in refs[space] if r+timedelta(hours=latency)<=issue]
    if not eligible:return np.array([]),np.array([]),None,'rejected_no_eligible_acquired_initialization'
    ref=max(eligible);aa=[];pp=[];reason='complete_72h'
    for s in range(72):
        start=issue+timedelta(hours=s);end=start+timedelta(hours=1)
        trow=weather.get((space,ref,start));prow=weather.get((space,ref,end))
        if trow is None or prow is None:
            reason='right_censored_at_GFS_forecast_tail';break
        av=num(trow['temperature_2m_C']);pv=num(prow['precipitation_increment_mm'])
        valid_boundary=bool(prow['precipitation_interval_start'] and prow['precipitation_interval_end'] and
            dt(prow['precipitation_interval_start'])==start and dt(prow['precipitation_interval_end'])==end)
        if not (np.isfinite(av) and np.isfinite(pv) and pv>=0 and valid_boundary):
            reason='missing_or_invalid_future_forcing_interval';break
        aa.append(av);pp.append(pv)
    return np.array(aa),np.array(pp),ref,reason

def forecast(engine,history,j,aa,pp):
    length=len(aa);prof=history['profiles'][j].copy();dmg=history['damage'][j].copy();storage=history['storage'][j].copy()
    margins=np.empty((length+1,engine.n));domain=np.empty((length+1,engine.n),bool)
    pressure_max=np.zeros(engine.n);ti_start=None
    for k in range(length+1):
        margin,resist,flag,ti,pressure=engine.observe(prof,dmg,storage)
        margins[k]=margin;domain[k]=flag;pressure_max=np.maximum(pressure_max,pressure)
        if k==0:ti_start=ti.copy()
        if k<length:prof,dmg,storage=engine.advance(prof,dmg,storage,aa[k],pp[k],resist)
    past=np.isfinite(history['first_elapsed'])&(history['first_elapsed']<=j)
    known_at_risk=~past
    point=np.full(engine.n,np.nan);lower=np.full(engine.n,np.nan);upper=np.full(engine.n,np.nan)
    found=(margins<=0).any(axis=0)&known_at_risk
    hit=np.argmax(margins<=0,axis=0)
    for k in np.unique(hit[found]):
        assert k>0,'at-risk scenario cannot start forecast already unstable'
        ix=found&(hit==k);lower[ix]=k-1;upper[ix]=k
        point[ix]=(k-1)+margins[k-1,ix]/(margins[k-1,ix]-margins[k,ix])
```

```
    return dict(margin=margins,domain_fraction=domain.mean(axis=0),point=point,lower=lower,upper=upper,
      past=past,at_risk=known_at_risk,followup=length,final_margin=margins[-1],max_pressure=pressure_max,
      thermal_change=ti-ti_start,profile_final=prof,damage_final=dmg,storage_final=storage)

def assumed_weights(frame,initial_margin,scheme):
    if scheme==PRIORS[0]:w=np.ones(len(frame))
    elif scheme==PRIORS[1]:
        counts=frame.groupby('mechanism_id',dropna=False).mechanism_id.transform('size').to_numpy(float);w=1/counts
    elif scheme==PRIORS[2]:w=np.exp(-np.clip(initial_margin,0,10))
    else:raise ValueError(scheme)
    assert np.isfinite(w).all() and (w>0).all()
    return w/w.sum()

def distribution(frame,result,initial_margin,indices,H,scheme):
    part=frame.iloc[indices];w=assumed_weights(part,initial_margin[indices],scheme)
    risk=result['at_risk'][indices];point=result['point'][indices];length=result['followup']
    event=risk&np.isfinite(point)&(point<=H)
    no=risk&((np.isfinite(point)&(point>H)) | (~np.isfinite(point)&(length>=H)))
    unknown=risk&~(event|no)
    rm=float(w[risk].sum());em=float(w[event].sum());nm=float(w[no].sum());um=float(w[unknown].sum())
    lower=em/rm if rm>0 else None;upper=(em+um)/rm if rm>0 else None
    p=lower if rm>0 and um<1e-12 else None
    qs=[None,None,None]
    if p is not None and p>0:
        vals=point[event];ww=w[event];order=np.argsort(vals);vals=vals[order];ww=ww[order];cw=np.cumsum(ww)/ww.sum()
        qs=[float(vals[min(int(np.searchsorted(cw,q,side='left')),len(vals)-1)]) for q in [.025,.5,.975]]
    return dict(probability_start_within_horizon=p,probability_no_start_within_horizon=1-p if p is not None else None,
        probability_start_lower=lower,probability_start_upper=upper,probability_no_start_lower=1-upper if upper is not None else None,
        probability_no_start_upper=1-lower if lower is not None else None,
        conditional_lower_hours=qs[0],conditional_median_hours=qs[1],conditional_upper_hours=qs[2],conditional_interval_level=.95,
        conditional_interval_interpretation='assumed_prior; given_start_within_horizon_and_retrospective_at_risk',
        already_started_prior_mass=float(w[~risk].sum()),at_risk_prior_mass=rm,event_prior_mass=em,no_start_prior_mass=nm,
        unresolved_future_prior_mass=um,scenario_count=len(indices),at_risk_scenarios=int(risk.sum()),
        observed_event_scenarios=int(event.sum()),known_no_start_scenarios=int(no.sum()),right_censored_or_unknown_scenarios=int(unknown.sum()),
        probability_prior=scheme,prior_is_analyst_assumption=True,design_is_probability_prior=False,
        strength_domain_fraction=float(result['domain_fraction'][indices].mean()))

PARQUET_FIELDS=[
 ('issue_time',pa.string()),('forecast_input_mode',pa.string()),('space_object',pa.string()),('assumed_latency_hours',pa.float64()),
 ('forecast_reference_time',pa.string()),('assumed_available_time',pa.string()),('available_time',pa.string()),
 ('scenario_id',pa.string()),('base_scenario_id',pa.string()),('parameter_set_id',pa.string()),('model_id',pa.string()),('mechanism_id',pa.string()),
 ('law_policy',pa.string()),('geometry_version',pa.string()),('parameter_version',pa.string()),
 ('history_status',pa.string()),('forecast_status',pa.string()),('known_at_risk_under_conditional_history',pa.bool_()),
 ('initial_margin_at_issue',pa.float64()),('historical_first_cross_elapsed_hours',pa.float64()),
```

```
 ('crossing_lower_hours_after_issue',pa.float64()),('crossing_upper_hours_after_issue',pa.floa
    t64()),('crossing_point_hours_after_issue',pa.float64()),
 ('followup_hours',pa.int32()),('forcing_status',pa.string()),('final_margin',pa.float64()),
 ('future_interface_temperature_change_c',pa.float64()),('max_observed_pressure_kpa',pa.float6
    4()),
 ('strength_domain_fraction_observed',pa.float64()),('evaluation_stratum',pa.string())]

def write_parameter_run(writer,frame,result,initial_margin,history,common):
    n=len(frame);length=result['followup'];past=result['past'];point=result['point']
    status=np.full(n,'no_cross_within_complete_72h',dtype=object)
    if length<72:status[:]='right_censored_or_unknown_future'
    if common['forcing_status'].startswith('rejected'):status[:]=common['forcing_status']
    status[np.isfinite(point)]='point_first_crossing_between_samples'
    status[past]='already_started_by_retrospective_history'
    hist_status=np.where(history['left'],'initial_history_left_censored',np.where(past,'first_
        crossed_before_or_at_issue','no_prior_cross_in_conditional_history'))
    data={k:frame[k].tolist() for k in ['scenario_id','base_scenario_id','parameter_set_id','m
        odel_id','mechanism_id','law_policy','geometry_version','parameter_version']}
    for k in ['issue_time','forecast_input_mode','space_object','assumed_latency_hours','forec
        ast_reference_time','assumed_available_time','available_time','forcing_status']:
        data[k]=[common[k]]*n
    data.update(history_status=hist_status,forecast_status=status,known_at_risk_under_conditio
        nal_history=result['at_risk'],
      initial_margin_at_issue=initial_margin,historical_first_cross_elapsed_hours=np.where(pas
        t&~history['left'],history['first_elapsed'],np.nan),
      crossing_lower_hours_after_issue=result['lower'],crossing_upper_hours_after_issue=result
        ['upper'],crossing_point_hours_after_issue=point,
      followup_hours=np.full(n,length),final_margin=result['final_margin'],future_interface_te
        mperature_change_c=result['thermal_change'],
      max_observed_pressure_kpa=result['max_pressure'],strength_domain_fraction_observed=resul
        t['domain_fraction'],evaluation_stratum=['conditional_scenario']*n)
    table=pa.Table.from_arrays([pa.array(data[k],type=t,from_pandas=True) for k,t in
        PARQUET_FIELDS],schema=pa.schema(PARQUET_FIELDS))
    writer.write_table(table)
    return Counter(status)

def redact_history_metadata(directory):
    """Remove future oracle times from an earlier audit export, not predictions.

    The original forecast uses only first_elapsed<=issue; this corrects a
    metadata column which otherwise disclosed full-forcing future audit times.
    """
    directory=directory.resolve();assert
        directory.is_relative_to(R/'results/mechanistic_replay')
    summary=json.loads((directory/'summary.json').read_text())
    origin=pd.Timestamp(summary.get('history_time_origin_utc','2026-06-01T00:00:00Z'))
    outcome_inputs=[ROOT/x['path'] for x in summary['inputs'] if
        Path(x['path']).name=='scenario_outcomes.csv']
    assert len(outcome_inputs)==1
    left_ids={x['scenario_id'] for x in csv_read(outcome_inputs[0]) if
        x['onset_status']=='left_censored_already_unstable'}
    records=[]
    for name in ['parameter_first_passages.parquet','history_states_at_issue.parquet']:
        path=directory/name;original_sha=sha(path);reader=pq.ParquetFile(path)
        column='historical_first_cross_elapsed_hours'
        tmp=directory/(name+'.causal.tmp');count=0;left_count=0
        with pq.ParquetWriter(tmp,reader.schema_arrow,compression='zstd',use_dictionary=True)
            as writer:
            for batch in reader.iter_batches(batch_size=65536):
                table=pa.Table.from_batches([batch]);values=table[column].to_numpy(zero_copy_o
                    nly=False)
                issue=pd.to_datetime(table['issue_time'].to_pylist(),utc=True)
```

```
                elapsed=np.asarray((issue-origin).total_seconds()/3600)
                bad=np.isfinite(values)&(values>elapsed+1e-12);count+=int(bad.sum())
                is_left=np.isin(np.array(table['scenario_id'].to_pylist()),list(left_ids))
                left_bad=is_left&np.isfinite(values);left_count+=int(left_bad.sum())
                values=values.copy();values[bad|is_left]=np.nan
                idx=table.schema.get_field_index(column)
                table=table.set_column(idx,table.schema.field(idx),pa.array(values,type=pa.flo⌋
                ↪  at64(),from_pandas=True))
                writer.write_table(table)
        tmp.replace(path)
        records.append(dict(path=str(path.relative_to(ROOT)),future_oracle_values_redacted=cou⌋
        ↪  nt,
            left_censored_pseudo_point_values_redacted=left_count,
            original_sha256=original_sha,sha256=sha(path),bytes=path.stat().st_size))
    source=directory/'execution_source'/f'metadata_redaction_{sha(Path(__file__))}.py.txt'
    if not source.exists():source.write_bytes(Path(__file__).read_bytes())
    rec=dict(method='null any first-cross audit time later than issue or left-censored initial
    ↪  pseudo-point; forecast math and ensemble predictions unchanged',
        physics_predictions_changed=False,files=records,source_path=str(source.relative_to(ROO⌋
        ↪  T)),source_sha256=sha(source))
    summary['history_metadata_redaction']=rec;summary['checks']['history_metadata_export_conta⌋
    ↪  ins_no_future_oracle_onset']=True
    summary['inputs'].append(dict(path=str(source.relative_to(ROOT)),bytes=source.stat().st_si⌋
    ↪  ze,sha256=sha(source)))
    summary['passed']=all(summary['checks'].values());dump(directory/'summary.json',summary)
    validation=json.loads((directory/'validation.json').read_text())
    validation['checks']['history_metadata_export_contains_no_future_oracle_onset']=True
    validation['history_metadata_redaction']=rec;validation['passed']=all(validation['checks']⌋
    ↪  .values());dump(directory/'validation.json',validation)
    dump(directory/'causal_metadata_redaction.json',rec);print(json.dumps(rec),flush=True)

def event_support_diagnostics(directory):
    """Count physical-design and time-point support, not repeated scenarios."""
    directory=directory.resolve();assert
    ↪  directory.is_relative_to(R/'results/mechanistic_replay')
    file=directory/'parameter_first_passages.parquet'
    columns=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','scenar⌋
    ↪  io_id','parameter_set_id',
        'model_id','mechanism_id','law_policy','initial_margin_at_issue','crossing_point_hours⌋
        ↪  _after_issue']
    events={};resolved_rows=0
    def runkey(x):
        value=x['assumed_latency_hours'];lat=float(value) if value not in ['',None] and
        ↪  pd.notna(value) else None
        return x['issue_time'],x['forecast_input_mode'],x['space_object'],lat
    for batch in pq.ParquetFile(file).iter_batches(batch_size=65536,columns=columns):
        table=pa.Table.from_batches([batch]);point=table['crossing_point_hours_after_issue'].t⌋
        ↪  o_numpy(zero_copy_only=False)
        mask=np.isfinite(point)&(point>0)
        if mask.any():
            for item in table.filter(pa.array(mask)).to_pylist():events.setdefault(runkey(item⌋
            ↪  ),[]).append(item);resolved_rows+=1
    def ess(items,key,fragile=False):
        weights={}
        for item in items:
            value=np.exp(-np.clip(item['initial_margin_at_issue'],0,10)) if fragile else 1.
            ident=item['parameter_set_id'] if key=='design' else
            ↪  round(item['crossing_point_hours_after_issue'],9)
            weights[ident]=weights.get(ident,0.)+float(value)
        w=np.array(list(weights.values()))
        return float(w.sum()**2/np.square(w).sum()) if len(w) else None
    rows=[];support_all=set();count_checks=[]
```

```
    with gzip.open(directory/'predictions.csv.gz','rt') as source:
      for item in csv.DictReader(source):
        if item['probability_prior']!=PRIORS[0]:continue
        H=int(item['horizon_hours']);hit=events.get(runkey(item),[])
        if item['model_id']!='mechanism_policy_mixture':
            hit=[x for x in hit if x['model_id']==item['model_id'] and
            ↪  x['law_policy']==item['law_policy']]
        hit=[x for x in hit if x['crossing_point_hours_after_issue']<=H]
        identifiers=sorted({x['parameter_set_id'] for x in
        ↪  hit});support_all.update(identifiers)
        times={round(x['crossing_point_hours_after_issue'],9) for x in hit}
        count_checks.append(len(hit)==int(item['observed_event_scenarios']))
        if len(identifiers)==1:status='degenerate_discrete_support_not_empirical_uncertainty'
        elif len(identifiers)==0:status='no_resolved_event_support'
        else:status='finite_design_support_not_empirical_uncertainty'
        lo=num(item['conditional_lower_hours']);hi=num(item['conditional_upper_hours'])
        rows.append({k:item[k] for k in ['issue_time','forecast_input_mode','space_object','as
        ↪  sumed_latency_hours','model_id','mechanism_id','law_policy','horizon_hours']}|
          dict(event_scenario_rows=len(hit),distinct_parameter_set_ids=len(identifiers),suppor
          ↪  t_parameter_ids=';'.join(identifiers),
            distinct_probability_time_support_points=len(times),effective_design_support_unifo
            ↪  rm=ess(hit,'design'),
            effective_design_support_equal_mechanism=ess(hit,'design'),effective_design_suppor
            ↪  t_fragile=ess(hit,'design',True),
            effective_time_support_uniform=ess(hit,'time'),effective_time_support_fragile=ess(
            ↪  hit,'time',True),
            unresolved_future_prior_mass=num(item['unresolved_future_prior_mass']),
            conditional_interval_computed=bool(item['conditional_median_hours']),
            conditional_interval_width_hours=hi-lo if np.isfinite(hi) and np.isfinite(lo) else
            ↪  None,
            support_status=status,independent_observed_event_count=0))
    assert all(count_checks),'Event support does not match generated forecast event counts'
    output=directory/'event_support_diagnostics.csv';csv_out(output,rows)
    counter=Counter(x['support_status'] for x in rows)
    complete=[x for x in rows if x['conditional_interval_computed']]
    summary=dict(diagnostic_rows=len(rows),resolved_parameter_event_rows_read=resolved_rows,
      distinct_support_parameter_ids=sorted(support_all),status_counts=dict(counter),
      nominal_conditional_interval_rows_uniform_prior=len(complete),
      nominal_intervals_with_one_physical_design=sum(x['distinct_parameter_set_ids']==1 for x
      ↪  in complete),
      maximum_conditional_interval_width_hours=max((x['conditional_interval_width_hours'] for x
      ↪  in complete),default=None),
      event_counts_match_prediction_rows=all(count_checks),independent_observed_events=0,
      explanation='ESS aggregates duplicated law/model scenario weights by physical
      ↪  parameter_set_id or time point. One design/point is discrete degeneracy, not precise
      ↪  empirical uncertainty; equal-mechanism equals uniform here only because delivered
      ↪  design is balanced.',
      inputs=[dict(path=str(p.relative_to(ROOT)),sha256=sha(p),bytes=p.stat().st_size) for p in
      ↪  [file,directory/'predictions.csv.gz']],
      output=dict(path=str(output.relative_to(ROOT)),sha256=sha(output),bytes=output.stat().st
      ↪  _size))
    dump(directory/'event_support_summary.json',summary)
    print(json.dumps(summary),flush=True)

def audit_exports(directory):
    directory=directory.resolve();assert
    ↪  directory.is_relative_to(R/'results/mechanistic_replay')
    summary=json.loads((directory/'summary.json').read_text());origin=pd.Timestamp(summary.get
    ↪  ('history_time_origin_utc','2026-06-01T00:00:00Z'))
    totals=Counter()
    cols=['issue_time','historical_first_cross_elapsed_hours','history_status','known_at_risk_
    ↪  under_conditional_history',
```

```
          'initial_margin_at_issue','crossing_point_hours_after_issue','followup_hours']
    p=directory/'parameter_first_passages.parquet'
    for batch in pq.ParquetFile(p).iter_batches(batch_size=65536,columns=cols):
        x=pa.Table.from_batches([batch]);t=pd.to_datetime(x['issue_time'].to_pylist(),utc=True)
        elapsed=np.asarray((t-origin).total_seconds()/3600);past=x['historical_first_cross_ela
        ↪ psed_hours'].to_numpy(zero_copy_only=False)
        point=x['crossing_point_hours_after_issue'].to_numpy(zero_copy_only=False)
        left=np.array(x['history_status'].to_pylist())=='initial_history_left_censored'
        risk=x['known_at_risk_under_conditional_history'].to_numpy(zero_copy_only=False)
        margin=x['initial_margin_at_issue'].to_numpy(zero_copy_only=False);follow=x['followup_
        ↪ hours'].to_numpy(zero_copy_only=False)
        totals['rows']+=len(x);totals['future_history_time_violations']+=int(np.sum(np.isfinit
        ↪ e(past)&(past>elapsed+1e-12)))
        totals['left_censored_exact_pseudo_time_violations']+=int(np.sum(left&np.isfinite(past
        ↪ )))
        totals['already_started_future_point_violations']+=int(np.sum(~risk&np.isfinite(point)
        ↪ ))
        totals['at_risk_nonpositive_initial_margin']+=int(np.sum(risk&(margin<=0)))
        totals['crossing_outside_followup']+=int(np.sum(np.isfinite(point)&((point<=0)|(point>
        ↪ follow))))
    retained=all((ROOT/x['path']).stat().st_size==x['bytes'] and
    ↪ sha(ROOT/x['path'])==x['sha256'] for x in summary['inputs'])
    checks={'exported_parameter_rows_complete':totals['rows']==summary['parameter_prediction_r
    ↪ ows'],
        'no_future_history_oracle_time_exported':totals['future_history_time_violations']==0,
        'no_precise_historical_time_assigned_to_initial_left_censor':totals['left_censored_exa
        ↪ ct_pseudo_time_violations']==0,
        'past_started_scenarios_have_no_future_first_cross':totals['already_started_future_poi
        ↪ nt_violations']==0,
        'at_risk_initial_state_stable':totals['at_risk_nonpositive_initial_margin']==0,
        'future_crossing_inside_actual_followup':totals['crossing_outside_followup']==0,
        'recorded_runtime_input_hashes_retained':retained}
    result=dict(checks=checks,passed=all(checks.values()),counts=dict(totals),
        file_path=str(p.relative_to(ROOT)),file_sha256=sha(p),independent_observed_events=0,
        executed_source_sha256=sha(Path(__file__)))
    dump(directory/'postprocess_audit.json',result)
    if not result['passed']:raise AssertionError(result)
    print(json.dumps(result),flush=True)

def version_comparison(pred,bundle_version):
    old_path=R/'results/mechanistic_replay/predictions.csv.gz'
    if OUT==old_path.parent or not old_path.exists():return None
    prior=pd.read_csv(old_path)
    old_universe=set(prior.loc[prior.model_id.ne('mechanism_policy_mixture'),'mechanism_id'])
    new_universe=set(pred.loc[pred.model_id.ne('mechanism_policy_mixture'),'mechanism_id'])
    if old_universe!=new_universe:
        return dict(status='not_comparable_different_mechanism_universes',old_universe=sorted(
        ↪ old_universe),new_universe=sorted(new_universe))
    keys=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','mechanism
    ↪ _id','law_policy','probability_prior','horizon_hours']
    values=['model_id','probability_start_within_horizon','conditional_lower_hours','condition
    ↪ al_median_hours','conditional_upper_hours',
            'already_started_prior_mass','at_risk_prior_mass','event_prior_mass','unresolved_f
            ↪ uture_prior_mass']
    assert not prior.duplicated(keys).any() and not pred.duplicated(keys).any()
    pair=prior[keys+values].merge(pred[keys+values],on=keys,how='outer',suffixes=('_v1','_v2')
    ↪ ,validate='one_to_one',indicator=True)
    for value in ['probability_start_within_horizon','conditional_median_hours','at_risk_prior
    ↪ _mass','already_started_prior_mass']:
        pair[value+'_difference_v2_minus_v1']=pair[value+'_v2']-pair[value+'_v1']
    pair['comparison_scope']='same physical design universe; numerical-grid sensitivity, not
    ↪ forecast skill'
```

```
    pair['fragile_prior_changes_with_initial_margin']=pair.probability_prior.eq(PRIORS[2])
    pair.to_csv(OUT/'paired_v1_v2_weather_predictions.csv.gz',index=False,compression={'method
    ↪  ':'gzip','mtime':0})
    x=pair.probability_start_within_horizon_difference_v2_minus_v1.dropna()
    t=pair.conditional_median_hours_difference_v2_minus_v1.dropna()
    return dict(status='matched_same_universe',old_input_path=str(old_path.relative_to(ROOT)),
    ↪  old_input_sha256=sha(old_path),
       old_model_universe=sorted(old_universe),new_model_universe=sorted(new_universe),new_bun
       ↪  dle_version=bundle_version,
       matched_rows=int(pair['_merge'].eq('both').sum()),unmatched_rows=int(pair['_merge'].ne(
       ↪  'both').sum()),
       identified_probability_pairs=len(x),maximum_absolute_probability_difference=float(x.abs
       ↪  ().max()) if len(x) else None,
       conditional_median_pairs=len(t),maximum_absolute_conditional_median_difference_hours=fl
       ↪  oat(t.abs().max()) if len(t) else None,
       interpretation='same issue/mechanism/law/space/latency/prior/horizon; risk-set changes
       ↪  and fragile-prior reweighting are retained, no winner ranking')

def run(bundle=None,output=None,mechanisms=None,experiment=None):
    global BUNDLE,OUT,EXPERIMENT
    if bundle is not None:BUNDLE=bundle.resolve()
    if output is not None:OUT=output.resolve()
    if experiment is not None:EXPERIMENT=experiment
    assert BUNDLE.is_relative_to(R/'inputs') and
    ↪  OUT.is_relative_to(R/'results/mechanistic_replay')
    started=datetime.now(timezone.utc);q,b,files,frame,forcing,config,clock,air,rain=load_inpu
    ↪  ts(mechanisms)
    OUT.mkdir(parents=True,exist_ok=True);engine=Physics(q,frame,config)
    issues=[START+timedelta(hours=i) for i in range(int((END-START).total_seconds()/3600)+1)]
    print(f'continuous Q2 history: {len(clock)} hours, {len(frame)} design rows, {engine.nb}
    ↪  full thermal profiles',flush=True)
    history=continuous_history(engine,clock,air,rain,issues)
    hv=history_validation(history,engine,frame,files);dump(OUT/'history_equivalence.json',hv)
    print(f'history full2048 and representative validation passed: {hv}',flush=True)
    weather,refs=weather_lookup()
    groups=[('mechanism_policy_mixture','all_delivered_mechanisms','mixture_of_both_law_polici
    ↪  es',np.arange(len(frame)))]
    groups.extend((str(m),str(mech),str(policy),np.flatnonzero((frame.model_id.eq(m)&frame.law
    ↪  _policy.eq(policy)).to_numpy()))
        for (m,mech,policy),_ in
        ↪  frame.groupby(['model_id','mechanism_id','law_policy'],sort=True))
    predictions=[];ranges=[];runlog=[];statuses=Counter();restart_errors=[];s_invariance=[];fu
    ↪  ture_boundaries=[]
    schema=pa.schema(PARQUET_FIELDS)
    writer=pq.ParquetWriter(OUT/'parameter_first_passages.parquet',schema,compression='zstd',u
    ↪  se_dictionary=True)
    history_rows=[];n_runs=0;unique_physics_runs=0;reused_physics_runs=0
    try:
      for issue,j in zip(issues,history['issue_indices']):
        initial=history['margin'][j]
        deterministic_inputs_cache={}
        for n,s in enumerate(frame.scenario_id):
            history_rows.append(dict(issue_time=stamp(issue),scenario_id=s,base_scenario_id=fr
            ↪  ame.base_scenario_id.iloc[n],model_id=frame.model_id.iloc[n],mechanism_id=fram
            ↪  e.mechanism_id.iloc[n],
                law_policy=frame.law_policy.iloc[n],failure_margin=initial[n],damage=history['
                ↪  damage'][j,n],storage_m=history['storage'][j,n],
                interface_temp_c=(history['interface_base'][j,engine.baseindex[n]] if
                ↪  engine.has_t[n] else engine.p['initial_temp_c'][n]),
                historical_first_cross_elapsed_hours=history['first_elapsed'][n] if
                ↪  history['first_elapsed'][n]<=j and not history['left'][n] else np.nan,
                historically_available=False,evaluation_stratum='conditional_scenario'))
```

```
        modes=[('historically_available','not_eligible',None),('ideal_weather','Q2_regional_st
        ↪  ation_forcing',None)]
        modes.extend(('assumed_latency',space,lat) for space in sorted(refs) for lat in
        ↪  [0,3,6,9])
        for mode,space,latency in modes:
            aa,pp,ref,forcing_status=get_forcing(mode,space,latency,issue,j,air,rain,clock,wea
            ↪  ther,refs)
            # Different latency assumptions often select exactly the same
            # initialization. Reuse that identical deterministic computation,
            # retaining every requested metadata/weight/censoring result row.
            cache_key=(aa.tobytes(),pp.tobytes())
            reused=cache_key in deterministic_inputs_cache
            if reused:
                result=deterministic_inputs_cache[cache_key];reused_physics_runs+=1
            else:
                result=forecast(engine,history,j,aa,pp);deterministic_inputs_cache[cache_key]=
                ↪  result;unique_physics_runs+=1
            if mode=='ideal_weather':
                length=result['followup'];full=history['margin'][j:j+length+1]
                error=float(np.max(abs(result['margin']-full)))
                error=max(error,float(np.max(abs(result['profile_final']-history['profiles'][j
                ↪  +length]))),
                    float(np.max(abs(result['damage_final']-history['damage'][j+length]))),flo
                    ↪  at(np.max(abs(result['storage_final']-history['storage'][j+length]))))
                restart_errors.append(error)
            if mode=='assumed_latency':
                s_idx=np.flatnonzero(frame.mechanism_id.eq('S').to_numpy());length=min(result[
                ↪  'followup'],len(clock)-1-j)
                if len(s_idx):s_invariance.append(float(np.max(abs(result['margin'][:length+1,
                ↪  s_idx]-history['margin'][j:j+length+1,s_idx]))))
                if ref is not
                ↪  None:future_boundaries.append(ref+timedelta(hours=latency)<=issue)
            common=dict(issue_time=stamp(issue),information_cutoff_utc=stamp(issue),forecast_i
            ↪  nput_mode=mode,space_object=space,
                event_id='B20260826_Chhochen',physical_phase='first_mechanical_initiation_at_s
                ↪  ource',
                assumed_latency_hours=latency,forecast_reference_time=stamp(ref) if ref else
                ↪  None,
                assumed_available_time=stamp(ref+timedelta(hours=latency)) if ref else
                ↪  None,available_time=None,
                forcing_status=forcing_status,followup_hours=result['followup'],historically_o
                ↪  perationally_certified=False,
                identical_deterministic_input_computation_reused=reused,
                evaluation_stratum='conditional_scenario',data_role='scenario',history_state_r
                ↪  ole='retrospective_Q2_forcing_state_unknown_real_availability',
                future_state_role='per_issue_GFS_recomputed_Q2_physics' if
                ↪  mode=='assumed_latency' else 'retrospective_Q2_forcing_oracle' if
                ↪  mode=='ideal_weather' else 'rejected_unknown_historical_availability',
                experiment_id=EXPERIMENT,input_bundle_version=b['bundle_version'],input_bundle
                ↪  _sha256=sha(BUNDLE),
                input_weather_sha256=sha(R/'inputs/weather/gfs_hourly_summary.csv'),input_vers
                ↪  ion=f"stage4-r3-v1; {b['bundle_version']}; weather-q3-v1")
            statuses.update(write_parameter_run(writer,frame,result,initial,history,common))
            runlog.append(dict(common,design_rows=len(frame),at_risk_rows=int(result['at_risk'
            ↪  ].sum()),forecast_crossing_rows=int(np.isfinite(result['point']).sum())))
            for model,mechanism,policy,indices in groups:
              for H in HORIZONS:
                point=result['point'][indices];risk=result['at_risk'][indices];event=risk&np.i
                ↪  sfinite(point)&(point<=H)
                unresolved=risk&~np.isfinite(point)&(result['followup']<H)
                rr=dict(common,model_id=model,mechanism_id=mechanism,law_policy=policy,horizon
                ↪  _hours=H,scenario_count=len(indices),
```

```
                    at_risk_scenario_count=int(risk.sum()),already_started_scenario_count=int(~risk).sum()),
                    point_crossing_count=int(event.sum()),right_censored_or_unknown_count=int(unresolved.sum()),
                    earliest_resolved_point_hours=float(np.min(point[event])) if event.any() else None,
                    latest_resolved_point_hours=float(np.max(point[event])) if event.any() else None,
                    range_definition='resolved_point_subset_only; unknown/right-censored masses separately retained',
                    role='parameter_design_range_no_probability')
                ranges.append(rr)
                for prior in PRIORS:
                    distribution_row=distribution(frame,result,initial,indices,H,prior)
                    predictions.append(dict(common,**distribution_row,engine='A_weather_coupled',model_id=model,mechanism_id=mechanism,law_policy=policy,
                        scenario_id='delivered_design_ensemble',geometry_version=str(frame.geometry_version.iloc[indices[0]]),parameter_version=str(frame.parameter_version.iloc[indices[0]]),
                        horizon_hours=H,truth_time_utc=None,empirical_coverage=None,independent_observed_events=0))
            n_runs+=1
        if len(runlog)%100==0:print(f'completed {len(runlog)} per-issue weather/rejection tracks',flush=True)
    finally:writer.close()
    csv_out(OUT/'predictions.csv.gz',predictions);csv_out(OUT/'scenario_ranges.csv.gz',ranges);csv_out(OUT/'run_log.csv',runlog)
    pd.DataFrame(history_rows).to_parquet(OUT/'history_states_at_issue.parquet',index=False,compression='zstd')
    pred=pd.DataFrame(predictions)
    aggregate=pred.groupby(['forecast_input_mode','space_object','assumed_latency_hours','model_id','law_policy','probability_prior','horizon_hours'],dropna=False).agg(
       issues=('issue_time','size'),identified_probabilities=('probability_start_within_horizon','count'),
       minimum_probability=('probability_start_within_horizon','min'),mean_probability=('probability_start_within_horizon','mean'),
       maximum_probability=('probability_start_within_horizon','max'),conditional_interval_count=('conditional_median_hours','count'),
       minimum_followup_hours=('followup_hours','min'),maximum_followup_hours=('followup_hours','max'),
       mean_strength_domain_fraction=('strength_domain_fraction','mean')).reset_index()
    aggregate.to_csv(OUT/'model_weather_comparison.csv',index=False)
    pred[pred.model_id.eq('mechanism_policy_mixture')].to_csv(OUT/'prior_sensitivity.csv.gz',index=False,compression={'method':'gzip','mtime':0})
    keys=['issue_time','model_id','mechanism_id','law_policy','probability_prior','horizon_hours']
    ideal=pred[pred.forecast_input_mode.eq('ideal_weather')][keys+['probability_start_within_horizon','conditional_median_hours']]
    comp=pred[pred.forecast_input_mode.eq('assumed_latency')].merge(ideal,on=keys,suffixes=('','_ideal'),how='left')
    comp['probability_difference_vs_ideal']=comp.probability_start_within_horizon-comp.probability_start_within_horizon_ideal
    comp['conditional_median_difference_hours_vs_ideal']=comp.conditional_median_hours-comp.conditional_median_hours_ideal
    comp.to_csv(OUT/'matched_weather_contrasts.csv.gz',index=False,compression={'method':'gzip','mtime':0})
    paired_versions=version_comparison(pred,b['bundle_version'])
    if paired_versions is not None:dump(OUT/'version_comparison.json',paired_versions)
    finite=pred.dropna(subset=['probability_start_within_horizon'])
    prior_keys=['issue_time','forecast_input_mode','space_object','assumed_latency_hours','model_id','law_policy','horizon_hours']
```

```
    a=pred[pred.probability_prior.eq(PRIORS[0])].set_index(prior_keys).probability_start_withi⌋
    ↪  n_horizon
    c=pred[pred.probability_prior.eq(PRIORS[1])].set_index(prior_keys).probability_start_withi⌋
    ↪  n_horizon
    equality=bool(np.allclose(a.to_numpy(float),c.to_numpy(float),equal_nan=True))
    monotone=True
    for _,g in pred.groupby(['issue_time','forecast_input_mode','space_object','assumed_latenc⌋
    ↪  y_hours','model_id','law_policy','probability_prior'],dropna=False):
        g=g.sort_values('horizon_hours')
        for column in ['probability_start_lower','probability_start_upper']:
            x=g[column].dropna().to_numpy(float)
            monotone=monotone and bool(np.all(np.diff(x)>=-1e-12))
    checks={
      'all_input_bundle_hashes_verified':True,'full_Q2_history_matches_delivered_outcomes_and_⌋
      ↪  representatives':hv['passed'],
      'all_75_ideal_full_state_restarts_match_continuous_history':max(restart_errors)<1e-8,
      'structural_only_weather_invariance_when_included':max(s_invariance,default=0.)<1e-8,
      'water_budget_closure':engine.water_budget_max<1e-10,
      'assumed_forecast_reference_available_before_issue':all(future_boundaries),
      'probability_normalized_when_identified':bool(np.allclose(finite.probability_start_withi⌋
      ↪  n_horizon+finite.probability_no_start_within_horizon,1)),
      'probability_bounds_valid':bool(((pred.probability_start_lower.dropna()>=-1e-12)&(pred.p⌋
      ↪  robability_start_lower.dropna()<=1+1e-12)).all() and
          ((pred.probability_start_upper.dropna()>=-1e-12)&(pred.probability_start_upper.dropn⌋
          ↪  a()<=1+1e-12)).all()),
      'probability_lower_not_above_upper':bool((pred.probability_start_lower<=pred.probability⌋
      ↪  _start_upper+1e-12).all()),
      'prior_mass_preserved':bool(np.allclose(pred.already_started_prior_mass+pred.event_prior⌋
      ↪  _mass+pred.no_start_prior_mass+pred.unresolved_future_prior_mass,1)),
      'balanced_uniform_and_equal_mechanism_priors_agree':equality,'horizon_probability_bounds⌋
      ↪  _monotone':monotone,
      'historical_unknown_track_rejected':bool(pred.loc[pred.forecast_input_mode.eq('historica⌋
      ↪  lly_available'),'probability_start_within_horizon'].isna().all()),
      'no_false_local_truth_or_empirical_coverage':bool(pred.truth_time_utc.isna().all() and
      ↪  pred.empirical_coverage.isna().all() and not
      ↪  pred.historically_operationally_certified.any()),
      'unknown_future_not_reported_as_exact_probability':bool(pred.loc[pred.unresolved_future_⌋
      ↪  prior_mass>1e-12,'probability_start_within_horizon'].isna().all())}
    summary=dict(experiment_id=EXPERIMENT,started_at_utc=stamp(started),completed_at_utc=stamp⌋
    ↪  (datetime.now(timezone.utc)),
      issue_count=len(issues),parameter_design_rows=len(frame),distinct_physical_designs=engin⌋
      ↪  e.nb,weather_tracks_per_issue=10,
      history_time_origin_utc=stamp(clock[0]),
      bundle_version=b['bundle_version'],included_mechanisms=sorted(frame.mechanism_id.unique(⌋
      ↪  )),
      thermal_discretization=dict(dz_m=engine.dz,bottom_m=engine.bottom,grid_points=len(engine⌋
      ↪  .z),substeps_per_hour=engine.substeps),
      per_issue_track_runs=n_runs,parameter_prediction_rows=n_runs*len(frame),ensemble_predict⌋
      ↪  ion_rows=len(pred),scenario_range_rows=len(ranges),
      unique_full_ensemble_physics_computations=unique_physics_runs,reused_identical_input_tra⌋
      ↪  cks=reused_physics_runs,
      forecast_status_counts=dict(statuses),forecast_forcing_track_counts=dict(Counter(x['forc⌋
      ↪  ing_status'] for x in runlog)),
      historical_initial_left_censored=int(history['left'].sum()),historical_total_first_cross⌋
      ↪  _or_left=int(np.isfinite(history['first_elapsed']).sum()),
      maximum_restart_absolute_error=max(restart_errors),maximum_S_weather_invariance_error=ma⌋
      ↪  x(s_invariance,default=0.),
      S_weather_invariance_actually_checked=bool(s_invariance),maximum_step_water_budget_error⌋
      ↪  _m=engine.water_budget_max,
      identified_probability_rows=len(finite),nonzero_identified_probability_rows=int((finite.⌋
      ↪  probability_start_within_horizon>0).sum()),
      maximum_identified_probability=float(finite.probability_start_within_horizon.max()),
```

```
      conditional_interval_rows=int(pred.conditional_median_hours.notna().sum()),independent_local_observed_validation_events=0,empirical_coverage=None,
      paired_grid_version_comparison=paired_versions,
      inputs=[dict(path=str(p.relative_to(ROOT)),bytes=p.stat().st_size,sha256=sha(p)) for p in dict.fromkeys(files.values())],
      checks=checks,passed=all(checks.values()),limitations=[
        'All source states and forecasts are conditional Q2 scenarios; local first mechanical initiation truth remains unknown',
        'Retrospective regional-station history has unknown real availability and assumed hourly precipitation intervals',
        'Direct GFS-to-regional-station driver replacement retains uncalibrated source offsets and multipliers; no local bias correction',
        'No verified historical first public weather availability; latency assumptions are not business release facts',
        'Three priors are analyst sensitivity assumptions, not probabilities implied by the space-filling design',
        'Q2 rock/ice strength transfer, thermal boundary, damage and reservoir limitations retained',
        'Exact consumer/Q2 numerical agreement is distinct from heat-equation grid convergence and scientific model adoption',
        'Conditional 95 percent quantiles describe assumed scenario distributions; empirical coverage denominator is zero',
        'Short followup and missing future forcing remain unresolved, never imputed as no start'])
    dump(OUT/'summary.json',summary);dump(OUT/'validation.json',dict(checks=checks,passed=all(checks.values()),history_validation=hv,
        maximum_restart_absolute_error=max(restart_errors),maximum_water_budget_error_m=engine.water_budget_max,independent_observed_events=0))
    if not all(checks.values()):raise AssertionError(checks)
    print(json.dumps(clean(summary),ensure_ascii=False),flush=True)

if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--bundle');parser.add_argument('--output-dir');parser.add_argument('--mechanisms');parser.add_argument('--experiment-id');parser.add_argument('--redact-history-metadata');parser.add_argument('--support-diagnostics');parser.add_argument('--audit-exports')
    args=parser.parse_args()
    if args.audit_exports:audit_exports(ROOT/args.audit_exports)
    elif args.support_diagnostics:event_support_diagnostics(ROOT/args.support_diagnostics)
    elif args.redact_history_metadata:redact_history_metadata(ROOT/args.redact_history_metadata)
    else:run(ROOT/args.bundle if args.bundle else None,ROOT/args.output_dir if args.output_dir else None,
        args.mechanisms.split(',') if args.mechanisms else None,args.experiment_id)
```

程序 33：reduced_solver.py

来源：track_b/research/q4/round3/scripts/reduced_solver.py
SHA256：0cb0eb23bb4c752fb6688c4dc721d660b5288de0537ac1e6854fce56614af054

```
"""Independent conservative shallow-water baseline. Not r.avaflow or D-Claw.

Array row velocity points south; exported north velocity has the opposite sign.
Volume is conserved in transport. Bed pressure sources are side-specific Audusse
hydrostatic reconstruction; Coulomb/Voellmy friction contracts momentum only.
"""
from __future__ import annotations
import hashlib, json, time, sys
from pathlib import Path
import numpy as np
try:
    from numba import njit
except ImportError:
    def njit(*a, **kw):
        def deco(f): return f
        return deco

G = 9.81
DRY = 1e-9

@njit(cache=True)
def face_flux(hl, ul, vl, zl, hr, ur, vr, zr, gravity):
    zface = max(zl, zr)
    left = max(0.0, hl + zl - zface)
    right = max(0.0, hr + zr - zface)
    speed = max(abs(ul) + np.sqrt(gravity * left), abs(ur) + np.sqrt(gravity * right))
    f0 = .5 * (left * ul + right * ur - speed * (right-left))
    f1 = .5 * (left*ul*ul + .5*gravity*left*left + right*ur*ur + .5*gravity*right*right -
    ↪  speed*(right*ur-left*ul))
    f2 = .5 * (left*ul*vl + right*ur*vr - speed*(right*vr-left*vl))
    return f0, f1, f2, .5*gravity*(hl*hl-left*left), .5*gravity*(hr*hr-right*right)

@njit(cache=True)
def speeds(h, mx, my):
    ny,nx = h.shape
    ux=np.zeros_like(h);uy=np.zeros_like(h)
    for i in range(ny):
        for j in range(nx):
            if h[i,j]>DRY:
                ux[i,j]=mx[i,j]/h[i,j];uy[i,j]=my[i,j]/h[i,j]
    return ux,uy

@njit(cache=True)
def timestep(h, mx, my, dx, dy, cfl, gravity, one_d):
    ny,nx=h.shape
    max_rate=0.0
    for i in range(ny):
        for j in range(nx):
            if h[i,j]>DRY:
                u=abs(mx[i,j]/h[i,j]);v=abs(my[i,j]/h[i,j]);c=np.sqrt(gravity*h[i,j])
                rate=(u+c)/dx
                if not one_d: rate+=(v+c)/dy
                max_rate=max(max_rate,rate)
    return cfl/max_rate if max_rate>0 else 30.0

@njit(cache=True)
def advance(h,mx,my,z,cosine,dx,dy,dt,mu,xi,gravity,closed,one_d):
```

```
    ny,nx=h.shape
    ux,uy=speeds(h,mx,my)
    fx=np.zeros((3,ny,nx+1));fy=np.zeros((3,ny+1,nx))
    cxL=np.zeros((ny,nx+1));cxR=np.zeros((ny,nx+1))
    cyL=np.zeros((ny+1,nx));cyR=np.zeros((ny+1,nx))
    for i in range(ny):
        for j in range(nx+1):
            il=max(0,j-1);ir=min(nx-1,j)
            ul=ux[i,il];ur=ux[i,ir]
            if closed and j==0: ul=-ur
            if closed and j==nx: ur=-ul
            a,b,c,dl,dr=face_flux(h[i,il],ul,uy[i,il],z[i,il],h[i,ir],ur,uy[i,ir],z[i,ir],grav
            ↪ ity)
            fx[0,i,j]=a;fx[1,i,j]=b;fx[2,i,j]=c;cxL[i,j]=dl;cxR[i,j]=dr
    if not one_d:
        for i in range(ny+1):
            for j in range(nx):
                il=max(0,i-1);ir=min(ny-1,i)
                ul=uy[il,j];ur=uy[ir,j]
                if closed and i==0: ul=-ur
                if closed and i==ny: ur=-ul
                a,b,c,dl,dr=face_flux(h[il,j],ul,ux[il,j],z[il,j],h[ir,j],ur,ux[ir,j],z[ir,j],
                ↪ gravity)
                fy[0,i,j]=a;fy[1,i,j]=c;fy[2,i,j]=b;cyL[i,j]=dl;cyR[i,j]=dr
    hn=np.empty_like(h);xn=np.empty_like(h);yn=np.empty_like(h)
    clipping=0.0;minimum=1e99
    for i in range(ny):
        for j in range(nx):
            depth=h[i,j]-dt/dx*(fx[0,i,j+1]-fx[0,i,j])
            xmom=mx[i,j]-dt/dx*(fx[1,i,j+1]+cxL[i,j+1]-fx[1,i,j]-cxR[i,j])
            ymom=my[i,j]-dt/dx*(fx[2,i,j+1]-fx[2,i,j])
            if not one_d:
                depth-=dt/dy*(fy[0,i+1,j]-fy[0,i,j])
                xmom-=dt/dy*(fy[1,i+1,j]-fy[1,i,j])
                ymom-=dt/dy*(fy[2,i+1,j]+cyL[i+1,j]-fy[2,i,j]-cyR[i,j])
            minimum=min(minimum,depth)
            if depth<0:
                clipping-=depth*dx*dy
                depth=0
            if depth>DRY:
                speed=np.sqrt(xmom*xmom+ymom*ymom)/depth
                rhs=max(0.0,speed-dt*mu*gravity*cosine[i,j])
                a=dt*gravity/(xi*depth) if xi>0 else 0.0
                new_speed=2*rhs/(1+np.sqrt(1+4*a*rhs))
                factor=new_speed/speed if speed>0 else 0.0
                xmom*=factor;ymom*=factor
            else: xmom=0.0;ymom=0.0
            hn[i,j]=depth;xn[i,j]=xmom;yn[i,j]=ymom
    outward=0.0;inward=0.0
    for i in range(ny):
        for v in (-fx[0,i,0]*dy*dt,fx[0,i,nx]*dy*dt):
            if v>=0: outward+=v
            else: inward-=v
    if not one_d:
        for j in range(nx):
            for v in (-fy[0,0,j]*dx*dt,fy[0,ny,j]*dx*dt):
                if v>=0: outward+=v
                else: inward-=v
    return hn,xn,yn,outward,inward,clipping,minimum

def run(h,z,dx,dy,duration,mu=0.15,xi=1000,cfl=.35,closed=False,
        targets=None,interval=30,one_d=False,initial_mx=None,initial_my=None,
        save_snapshots=True,progress=False):
```

```
    """Conservative transport to requested horizon; no kinetic early stopping."""
    h=np.array(h,dtype=np.float64,copy=True);z=np.asarray(z,dtype=np.float64)
    mx=np.zeros_like(h) if initial_mx is None else np.array(initial_mx,copy=True)
    my=np.zeros_like(h) if initial_my is None else np.array(initial_my,copy=True)
    if one_d: gradx=np.gradient(z[0],dx)[None,:];grady=np.zeros_like(z)
    else: grady,gradx=np.gradient(z,dy,dx)
    cosine=1/np.sqrt(1+gradx**2+grady**2)
    initial=float(h.sum()*dx*dy);outflow=inflow=clipping=0.0
    maxh=h.copy();maxv=np.zeros_like(h);time_now=0.0;steps=0
    min_raw=float(h.min());last_output=0.0;next_output=interval
    times=[0.0];snapshots=[h.astype('float32')] if save_snapshots else []
    records=[];targets=targets or {};arrivals={k:None for k in targets}
    last_target_dry={k:0.0 for k in targets};brackets={k:None for k in targets}
    wallclock=time.perf_counter();status='completed';reason=None
    records.append({'elapsed_seconds':0.,'domain_volume_m3':initial,'outflow_m3':0.,'inflow_m3
    ↪  ':0.,'maxdepth_m':float(h.max()),'maxspeed_m_s':0.})
    while time_now<duration-1e-9:
        dt=min(timestep(h,mx,my,dx,dy,cfl,G,one_d),duration-time_now,next_output-time_now)
        if dt<=1e-12:
            next_output=min(duration,next_output+interval);continue
        previous_time=time_now
        h,mx,my,out,inc,clip,minimum=advance(h,mx,my,z,cosine,dx,dy,dt,mu,xi,G,closed,one_d)
        time_now+=dt;steps+=1;outflow+=out;inflow+=inc;clipping+=clip;min_raw=min(min_raw,floa
        ↪  t(minimum))
        if minimum< -1e-8 or not np.isfinite(h).all() or not np.isfinite(mx).all() or not
        ↪  np.isfinite(my).all():
            status='numerical_failure';reason='nonfinite state or negative raw depth below
            ↪  -1e-8';break
        u,v=speeds(h,mx,my);speed=np.hypot(u,v)
        maxh=np.maximum(maxh,h);maxv=np.maximum(maxv,speed)
        for k,mask in targets.items():
            if arrivals[k] is None:
                if mask.any() and float(h[mask].max())>=.1:
                    arrivals[k]=time_now;brackets[k]=[previous_time,time_now]
                else:last_target_dry[k]=time_now
        if time_now>=next_output-1e-8 or time_now>=duration-1e-8:
            times.append(time_now)
            if save_snapshots:snapshots.append(h.astype('float32'))
            records.append({'elapsed_seconds':time_now,'domain_volume_m3':float(h.sum()*dx*dy)
            ↪  ,'outflow_m3':outflow,'inflow_m3':inflow,'maxdepth_m':float(h.max()),'maxspeed
            ↪  _m_s':float(speed[h>=.1].max()) if np.any(h>=.1) else 0.})
            next_output=min(duration,next_output+interval)
            if progress and len(records)%10==0: print(json.dumps({'t':time_now,'step':steps,'m
            ↪  axh':float(h.max()),'outflow':outflow}),flush=True)
        if steps>300000: status='numerical_failure';reason='300000 step safety limit';break
    u,v=speeds(h,mx,my);speed=np.hypot(u,v)
    final=float(h.sum()*dx*dy)
    summary={'execution_status':status,'failure_reason':reason,'duration_requested_seconds':du
    ↪  ration,'duration_actual_seconds':time_now,'steps':steps,'wall_seconds':time.perf_count
    ↪  er()-wallclock,
      'initial_volume_m3':initial,'final_domain_volume_m3':final,'outflow_volume_m3':outflow,'
      ↪  inflow_volume_m3':inflow,'depth_clipping_added_volume_m3':clipping,
      'mass_balance_residual_m3':final+outflow-inflow-clipping-initial,'relative_mass_balance_
      ↪  error':(final+outflow-inflow-clipping-initial)/initial if initial else 0.,
      'minimum_preclip_depth_m':min_raw,'minimum_final_depth_m':float(h.min()),'max_depth_m':f
      ↪  loat(maxh.max()),'maximum_speed_any_wet_m_s':float(maxv.max()),
      'max_speed_final_h_ge_01_m_s':float(speed[h>=.1].max()) if np.any(h>=.1) else
      ↪  0.,'final_stationary_volume_v_lt_01_m3':float(h[(speed<.1)].sum()*dx*dy),'footprint_
      ↪  area_hmax_ge_01_m2':int((maxh>=.1).sum())*dx*dy,
```

```
      'target_arrivals':{k:{'travel_time_seconds':arrivals[k],'arrival_status':('numerical_failure' if status=='numerical_failure' else 'arrived' if arrivals[k] is not None else 'missing' if not mask.any() else 'out_of_domain' if outflow>initial*.999 else 'not_reached_within_horizon'), 'arrival_time_bracket_seconds':brackets[k],'target_cells':int(mask.sum()),'definition':'first solver step with h >= 0.1 m in raster target mask; previous step below threshold','time_origin':'seconds after scenario bulk release','initiation_to_release_delay_seconds':None} for k,mask in targets.items()}}
    fields={'h_final':h.astype('float32'),'maxdepth':maxh.astype('float32'),'maxspeed':maxv.astype('float32'),'u_east_final':u.astype('float32'),'v_north_final':(-v).astype('float32'),'times':np.array(times),'snapshots':np.stack(snapshots) if snapshots else np.empty((0,))}
    return summary,fields,records

def file_record(path,role):
    p=Path(path);return {'path':str(p),'bytes':p.stat().st_size,'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),'role':role}

def write_json(path,obj):
    def scalar(value):
        if isinstance(value,np.generic):return value.item()
        if isinstance(value,np.ndarray):return value.tolist()
        raise TypeError(type(value).__name__)
    p=Path(path);p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=False,default=scalar)+'\n')
```

程序 34：reduced_region.py

来源：track_b/research/q4/round3/scripts/reduced_region.py
SHA256: 37dd019f086240de32300749d29cc426fc32dd2c3c2fa231275aab12686b5e77

```
"""Regional conditional scenarios with immutable common spatial inputs."""
from __future__ import annotations
from pathlib import Path
import argparse,csv,datetime,json,platform,sys,hashlib
import numpy as np
from reduced_solver import run,write_json,file_record

R=Path(__file__).resolve().parents[1]
C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/reduced')
TARGET=np.array([340981.541,3129214.005])

def scenarios():
    rows=[('ref',15,.15,1000),('h5',5,.15,1000),('h30',30,.15,1000),('mu08',15,.08,1000),('mu2 5',15,.25,1000),('xi500',15,.15,500),('xi2000',15,.15,2000),('h136',136,.15,1000)]
    return [dict(name=n,thickness_m=h,mu=mu,xi=xi) for n,h,mu,xi in rows]

def terrain(res):
    p=R/f'results/spatial_inputs_{res}m.npz';d=np.load(p)
    return {k:d[k] for k in d.files},file_record(p,'frozen common terrain and source binary mask')

def path_data(path_id,dx=60):
    rows=[r for r in csv.DictReader((R/'results/spatial_paths.csv').open()) if r['path_id']==path_id]
    station=np.array([float(r['distance_m']) for r in rows]);xx=np.array([float(r['x_m']) for r in rows]);yy=np.array([float(r['y_m']) for r in rows]);zz=np.array([float(r['elevation_m']) for r in rows])
    n=int(np.ceil(station[-1]/dx));dx=station[-1]/n;ss=(np.arange(n)+.5)*dx
    x=np.interp(ss,station,xx);y=np.interp(ss,station,yy);z=np.interp(ss,station,zz)
    d,_=terrain(60);transform=d['transform'];rr=np.floor((transform[5]-y)/60).astype(int);cc=np.floor((x-transform[2])/60).astype(int)
    in_source=d['source_mask'][np.clip(rr,0,d['z'].shape[0]-1),np.clip(cc,0,d['z'].shape[1]-1)].astype(bool)
    # Only the contiguous source reach beginning at the source representative point.
    first_dry=np.flatnonzero(~in_source)
    end=int(first_dry[0]) if len(first_dry) else len(in_source)
    if end<1:raise ValueError('path does not begin in the common binary source mask')
    release=np.arange(n)<end
    source_area=float(d['source_mask'].sum()*60**2);width=source_area/(release.sum()*dx)
    return {'s':ss,'x':x,'y':y,'z':z[None,:],'release':release[None,:],'dx':dx,'width':width,'source_area':source_area,'target':(np.hypot(x-TARGET[0],y-TARGET[1])<=100)[None,:],'source_reach_length_m':end*dx,'path_length_m':station[-1]}

def output_run(identifier,model,params,summary,fields,series,inputs,extra):
    out=C/identifier;out.mkdir(parents=True,exist_ok=True)
    arrays=out/'fields.npz';np.savez_compressed(arrays,**fields)
    record={'run_id':identifier,'model_id':model,'mechanism_id':'homogeneous_coulomb_voellmy','experiment_id':'regional_ofat','scenario_id':f"q4-r3-release-h{params['thickness_m']}-v0","geometry_version":'q4-r3-spatial-copdem-v1','parameter_version':'q4-r3-reduced-v1','evaluation_stratum':'conditional_scenario','parameters':params,'density_kg_m3':2000,'density_role':'scenario bulk density, not observed; no role in volume dynamics','initial_velocity':{'east_m_s':0,'north_m_s':0,'crs':'EPSG:32645'},'time_origin':'bulk release','initiation_to_release_delay_seconds':None,'inputs':inputs,'summary':summary,'fields':file_record(arrays,'full gridded maxdepth/maxspeed/final state and 30s process snapshots'), 'execution':{'python':sys.version,'platform':platform.platform()},**extra}
```

```
    record['summary']['initial_mass_kg']=summary['initial_volume_m3']*2000;record['summary']['final_mass_kg']=summary['final_domain_volume_m3']*2000
    write_json(R/f'results/reduced_{identifier}.json',record)
    p=R/f'results/reduced_{identifier}_timeseries.csv'
    with p.open('w') as f:
        w=csv.DictWriter(f,fieldnames=list(series[0]));w.writeheader();w.writerows(series)
    print(json.dumps({'finished':identifier,'status':summary['execution_status'],'wall_seconds':summary['wall_seconds'],'maxdepth':summary['max_depth_m'],'arrivals':summary['target_arrivals'],'mass_error':summary['relative_mass_balance_error']}),flush=True)

def main():
    parser=argparse.ArgumentParser();parser.add_argument('--mode',choices=['2d','1d','all'],default='all');parser.add_argument('--select',default='all');args=parser.parse_args()
    selected=lambda name:args.select=='all' or name in args.select.split(',')
    params=scenarios()
    if args.mode in ['2d','all']:
        jobs=[(s,60,.35) for s in params]+[(params[0],120,.35),(params[0],60,.175)]
        for par,res,cfl in jobs:
            name=f"2d_{par['name']}_{res}m"+('_cflhalf' if cfl<.35 else '')
            if not selected(name):continue
            d,irec=terrain(res);dx=float(d['dx']);mask=d['source_mask'].astype(bool);source60,_=terrain(60);area=float(source60['source_mask'].sum()*60**2);scale=area/(mask.sum()*dx**2)
            h=mask.astype(float)*par['thickness_m']*scale
            x,y=np.meshgrid(d['x'],d['y']);target=(np.hypot(x-TARGET[0],y-TARGET[1])<=100)
            # Diagnostic receiving river, without implying source-to-river proof.
            channel=(d['channel_distance_m']<=100)&~mask
            s,f,t=run(h,d['z'],dx,dx,1800,mu=par['mu'],xi=par['xi'],cfl=cfl,targets={'paper_port_symbol':target,'candidate_channel':channel},progress=True)
            f.update(x=d['x'],y=d['y'],z=d['z'],source_mask=mask,target_mask=target)
            bedstep=np.maximum(np.abs(np.gradient(d['z'],axis=0)),np.abs(np.gradient(d['z'],axis=1)))
            extra={'raster_shape':list(h.shape),'resolution_m':dx,'source_area_m2_common':area,'native_source_area_m2':int(mask.sum())*dx**2,'source_depth_scale':scale,'source_initial_volume_integral_m3':float(h.sum()*dx**2),'target_definition':'cell centres within 100 m of paper_port_symbol; h>=0.1m at solver step','source_bedstep_gt_thickness_fraction':float((bedstep[mask]>h[mask]).mean()),'no_entrainment':True,'deposition_interpretation':'h_final is residual material, not all stationary deposit'}
            extra['source_bedstep_diagnostic_definition']='legacy key: max absolute central row/col elevation difference (one-cell slope proxy); not maximum adjacent face jump'
            output_run(name,'independent_2d_hydrostatic_fv',{**par,'resolution_m':res,'cfl':cfl},s,f,t,[irec],extra)
    if args.mode in ['1d','all']:
        for path_id in ['P1','P2','P3']:
            for par in params:
                name=f"1d_{path_id}_{par['name']}"
                if not selected(name):continue
                d=path_data(path_id);h=d['release'].astype(float)*par['thickness_m']
                s,f,t=run(h,d['z'],d['dx'],d['width'],1800,mu=par['mu'],xi=par['xi'],one_d=True,targets={'paper_port_symbol':d['target']},progress=False)
                # Longitudinal momentum is mapped onto local UTM tangents for export.
                along=f['u_east_final'].copy();tx=np.gradient(d['x'],d['s']);ty=np.gradient(d['y'],d['s']);norm=np.hypot(tx,ty);tx/=norm;ty/=norm
                f['u_path_final']=along;f['u_east_final']=along*tx[None,:];f['v_north_final']=along*ty[None,:]
                f.update(station=d['s'],x=d['x'],y=d['y'],z=d['z'],target_mask=d['target'])
```

```
                extra={k:d[k] for k in ['source_area','source_reach_length_m','path_length_m','width']};extra.update(path_id=path_id,effective_width_role='constant equivalent width chosen from common raster source area / contiguous path source length; not measured valley width',distance_definition='horizontal UTM45N centreline station, raw DEM elevations interpolated; no monotonic forcing',target_definition='path cell centre within 100 m of paper_port_symbol with h>=0.1 m',lateral_spreading='not represented',no_entrainment=True)
                output_run(name,'independent_1d_path_fv',{**par,'resolution_m':d['dx'],'cfl':.35},s,f,t,[file_record(R/'results/spatial_paths.csv','common terrain-controlled path ensemble'),file_record(R/'results/spatial_inputs_60m.npz','common source area for mass matching')],extra)

if __name__=='__main__':main()
```

程序 35：reduced_benchmarks.py

来源：track_b/research/q4/round3/scripts/reduced_benchmarks.py
SHA256：58c77910c029b60619ffe53000f2bab9c91afd0994c45ef77298f8e0c6b6076a

```
"""Executed numerical checks, independent of any regional event validation."""
from pathlib import Path
import json, datetime, csv
import numpy as np
from reduced_solver import run, write_json,advance,timestep

R=Path(__file__).resolve().parents[1]

def main():
    results={'evaluation_stratum':'numerical_benchmark','created_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'cases':[]}
    # Still water with genuinely dry high terrain.
    x,y=np.meshgrid(np.arange(100)*10,np.arange(80)*10)
    z=.8*np.exp(-((x-500)**2+(y-400)**2)/(200**2))+.03*np.sin(x/200)
    h=np.maximum(.5-z,0)
    s,f,_=run(h,z,10,10,300,mu=0,xi=0,closed=True,interval=300)
    case={'case_id':'lake_at_rest_wet_dry','summary':s,'max_h_change_m':float(np.abs(f['h_final']-h).max()),'max_speed_m_s':float(f['maxspeed'].max()),'tolerance_h_m':1e-6,'tolerance_speed_m_s':1e-7}
    case['pass']=case['max_h_change_m']<case['tolerance_h_m'] and case['max_speed_m_s']<case['tolerance_speed_m_s'];results['cases'].append(case)
    # Analytic Ritter dam-break (flat, initially dry downstream, no friction).
    ritter=[]
    for dx,cfl in [(5.,.35),(2.5,.35),(1.25,.35),(2.5,.175)]:
        xx=np.arange(-500+dx/2,500,dx);h=(xx<0).astype(float)[None,:];z=np.zeros_like(h)
        s,f,_=run(h,z,dx,1,20,mu=0,xi=0,cfl=cfl,one_d=True,closed=True,interval=20)
        c=np.sqrt(9.81);exact=np.where(xx<=-20*c,1.,np.where(xx<40*c,4/(9*9.81)*(c-xx/(40))**2,0.))
        err=float(np.mean(np.abs(f['h_final'][0]-exact)))
        ritter.append({'dx_m':dx,'cfl':cfl,'l1_depth_error_m':err,'summary':s})
    refinement=[x['l1_depth_error_m'] for x in ritter[:3]]
    results['cases'].append({'case_id':'ritter_dry_bed_grid_and_cfl','runs':ritter,'grid_error_strictly_decreases':bool(refinement[2]<refinement[1]<refinement[0]),'cfl_halving_dept
    h_l1_difference_m':float(abs(ritter[1]['l1_depth_error_m']-ritter[3]['l1_depth_error_m
    ']))),'pass':bool(refinement[2]<refinement[1]<refinement[0])})
    # Radial release tests cross-direction symmetry and dry fronts.
    x,y=np.meshgrid((np.arange(100)+.5)*5,(np.arange(100)+.5)*5)
    h=(((x-250)**2+(y-250)**2)<50**2).astype(float);z=np.zeros_like(h)
    s,f,_=run(h,z,5,5,30,mu=0,xi=0,closed=True,interval=30)
    sym=float(np.max(np.abs(f['h_final']-f['h_final'].T)))
    results['cases'].append({'case_id':'radial_dry_front','summary':s,'transpose_symmetry_max_h_m':sym,'pass':abs(s['relative_mass_balance_error'])<1e-11 and
    s['minimum_preclip_depth_m']>=-1e-8 and sym<1e-6})
    # A moving finite plug crosses an open right boundary; actual signed flux is booked.
    xx=np.arange(.5,600,1.);h=((xx>200)&(xx<400)).astype(float)[None,:];z=np.zeros_like(h)
    s,f,_=run(h,z,1,1,70,mu=0,xi=0,one_d=True,initial_mx=h*10,interval=70)
    results['cases'].append({'case_id':'open_boundary_volume_ledger','summary':s,'pass':s['outflow_volume_m3']>0 and abs(s['relative_mass_balance_error'])<1e-10})
    # A uniform incline demonstrates both consistency and steep-cell limitations.
    incline=[]
    for dx in [20.,10.,5.]:
        xx=(np.arange(400)+.5)*dx;h=np.ones((1,400))*10;z=(-.2*xx)[None,:]
        s,f,_=run(h,z,dx,1,5,mu=.15,xi=1000,one_d=True,interval=5)
        actual=float(np.mean(f['u_east_final'][0,180:220]));A=9.81*(.2-.15/np.sqrt(1+.2**2));B=9.81/(1000*10);exact=np.sqrt(A/B)*np.tanh(np.sqrt(A*B)*5)
```

```
        incline.append({'dx_m':dx,'actual_centre_speed_m_s':actual,'cartesian_graph_constant_depth_exact_m_s':float(exact),'absolute_error_m_s':float(abs(actual-exact)),'bed_step_over_depth':.2*dx/10,'summary':s})
    results['cases'].append({'case_id':'uniform_incline_grid_response','runs':incline,'pass':bool(incline[2]['absolute_error_m_s']<incline[0]['absolute_error_m_s']),'interpretation':'Cartesian graph acceleration g*tan(theta), vertical depth. This is not terrain-normal avalanche momentum.'})
    # Separately declared constant-depth rigid parcel equation, using actual surface station.
    theta=np.arctan(.2);h=15.;mu=.15;xi=1000.;A=9.81*(np.sin(theta)-mu*np.cos(theta));B=9.81/(xi*h);T=120.
    analytic_v=float(np.sqrt(A/B)*np.tanh(np.sqrt(A*B)*T));analytic_s=float(np.log(np.cosh(np.sqrt(A*B)*T))/B)
    def parcel(dt):
        v=s=0.
        for _ in range(round(T/dt)):
            # RK4 v, with s integrated by the same stages.
            k1=A-B*v*v;k2=A-B*(v+dt*k1/2)**2;k3=A-B*(v+dt*k2/2)**2;k4=A-B*(v+dt*k3)**2
            s+=dt*(v+2*(v+dt*k1/2)+2*(v+dt*k2/2)+(v+dt*k3))/6;v+=dt*(k1+2*k2+2*k3+k4)/6
        return v,s
    rigid=[]
    for dt in [1.,.5,.25]:
        v,s=parcel(dt);rigid.append({'dt_seconds':dt,'v_m_s':v,'s_m':s,'v_absolute_error':abs(v-analytic_v),'s_absolute_error':abs(s-analytic_s)})
    results['cases'].append({'case_id':'rigid_constant_slope_voellmy_analytic','analytic_v_m_s':analytic_v,'analytic_s_m':analytic_s,'runs':rigid,'pass':rigid[-1]['s_absolute_error']<1e-4,'interpretation':'additional ODE benchmark only; no volume transport and not an event prediction nor the 1D FV model'})
    results['all_declared_checks_pass']=all(c['pass'] for c in results['cases'])
    results['pass_scope']='Declared algebraic properties and direction of grid response only; not a quantitative regional accuracy acceptance or event validation.'
    results['regional_event_accuracy_accepted']=False
    results['incline_accuracy_warning']='At h10m, slope.2, dx20m, the centre speed error is about 75%; refinement reduces this but 60/120m regional grids are not demonstrated accurate.'
    results['jit_evidence']={'advance_compiled_signatures':[str(x) for x in getattr(advance,'signatures',[])],'timestep_compiled_signatures':[str(x) for x in getattr(timestep,'signatures',[])],'all_jit_functions_have_signatures':bool(getattr(advance,'signatures',[]) and getattr(timestep,'signatures',[]))}
    write_json(R/'results/reduced_numerical_validation.json',results)
    print(json.dumps({'all_pass':bool(results['all_declared_checks_pass']),'cases':[(c['case_id'],bool(c['pass'])) for c in results['cases']]}),flush=True)

if __name__=='__main__':main()
```

程序 36：reduced_validate.py

来源：track_b/research/q4/round3/scripts/reduced_validate.py
SHA256: 52b7c745d0174646b1b4dee9715aacf3a2c5ea105c6bb1b84503d6f6d41281e4

```
"""Delivery provenance and physical-ledger checks; freezes only reduced packet."""
from pathlib import Path
import json,csv,hashlib,datetime,py_compile
import numpy as np
from reduced_solver import file_record,write_json

R=Path(__file__).resolve().parents[1]

def main():
    runs=[json.loads(p.read_text()) for p in sorted((R/'results').glob('reduced_[12]d_*.json'))]
    checks={};cache_files={};max_mass=0.;max_vector_error=0.
    checks['expected_34_runs']=len(runs)==34
    checks['all_completed_1800s']=all(r['summary']['execution_status']=='completed' and abs(r['summary']['duration_actual_seconds']-1800)<1e-8 for r in runs)
    checks['all_common_source_volumes']=all(abs(r['summary']['initial_volume_m3']-925200*r['parameters']['thickness_m'])/(925200*r['parameters']['thickness_m'])<1e-12 for r in runs)
    checks['all_primary_null_not_reached_preserved']=all(r['summary']['target_arrivals']['paper_port_symbol']['travel_time_seconds'] is None and r['summary']['target_arrivals']['paper_port_symbol']['arrival_status']=='not_reached_within_horizon' for r in runs)
    files_ok=True;state_ok=True;vector_ok=True
    for r in runs:
        s=r['summary'];max_mass=max(max_mass,abs(s['relative_mass_balance_error']))
        for field_key in ['fields','raw_solver_fields']:
            if field_key not in r:continue
            rec=r[field_key];p=Path(rec['path']);actual=file_record(p,rec['role']);files_ok &= actual['bytes']==rec['bytes'] and actual['sha256']==rec['sha256'];cache_files[str(p)]=actual
        d=np.load(r['fields']['path']);state_ok &= np.isfinite(d['h_final']).all() and np.isfinite(d['maxspeed']).all() and d['h_final'].min()>=0 and np.all(d['maxdepth']+1e-5>=d['h_final']) and len(d['times'])==61 and d['times'][-1]==1800
        if r['model_id'].startswith('independent_1d'):
            speed=np.hypot(d['u_east_final'],d['v_north_final']);expected=np.abs(d['u_path_final']);err=float(np.max(np.abs(speed-expected)));max_vector_error=max(max_vector_error,err);vector_ok &= err<1e-4
    checks.update(cache_field_hashes_match=bool(files_ok),finite_nonnegative_states_and61_saved_times=bool(state_ok),one_d_utm_vector_norm_matches_path_speed=bool(vector_ok),maximum_relative_mass_error_lt_1e_minus12=max_mass<1e-12,all_negative_raw_depths_above_failure_threshold=all(r['summary']['minimum_preclip_depth_m']>=-1e-8 for r in runs))
    bench=json.loads((R/'results/reduced_numerical_validation.json').read_text())
    checks['actual_numba_compilation_signatures_present']=bench['jit_evidence']['all_jit_functions_have_signatures']
    checks['regional_accuracy_false_explicit']=bench['regional_thin_flow_accuracy_qualified'] is False
    rows=list(csv.DictReader((R/'results/reduced_delivery_metrics.csv').open()))
    checks['all_reduced_runs_excluded_from_arrival_envelope']=len(rows)==34 and all(x['include_in_arrival_envelope']=='False' for x in rows)
    targets=list(csv.DictReader((R/'results/reduced_osm_target_arrivals_v2.csv').open()))
    checks['all_170_additional_target_rows']=len(targets)==170 and len(set(x['target_id'] for x in targets))==5 and all(sum(x['run_id']==r['run_id'] for x in targets)==5 for r in runs)
    review=json.loads((R/'results/reduced_visual_review.json').read_text());checks['five_figures_actually_reviewed_and_hashes_match']=len(review['figures'])==5 and all(file_record(x['path'],'')['sha256']==x['sha256'] for x in review['figures'])
```

```
    for p in (R/'scripts').glob('reduced_*.py'):py_compile.compile(str(p),doraise=True)
    checks['source_scripts_compile']=True
    validation={'validated_at_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'checks':checks,'all_pass':all(checks.values()),'max_relative_mass_error':max_mass,'max_1d_velocity_vector_norm_difference_m_s':max_vector_error,'scope':'Data integrity, conservation and declared exclusions only; regional thin-flow timing remains unqualified.','scientific_adoption':'pending_team_review'}
    write_json(R/'results/reduced_delivery_validation.json',validation)
    if not validation['all_pass']:raise RuntimeError(checks)
    repo_files=[]
    for folder,pattern in [('scripts','reduced_*.py'),('config','reduced_*'),('results','reduced_*'),('figures','reduced_*.png')]:
        for p in sorted((R/folder).glob(pattern)):
            if not p.is_file() or p.name=='reduced_delivery_index.json':continue
            rec=file_record(p,'reduced-model '+folder+' artifact');rec['path']=str(p.relative_to(Path.cwd()));repo_files.append(rec)
    repo_files.append({**file_record(R/'REDUCED_MODELS.md','model equations, actual results, rejection evidence and reproduction'),'path':str((R/'REDUCED_MODELS.md').relative_to(Path.cwd()))})
    protocol=json.loads((R/'config/reduced_protocol.json').read_text())
    for source in protocol['sources']:
        if source.get('path') and Path(source['path']).exists():cache_files[source['path']]=file_record(source['path'],'bounded primary-source download; actual read scope recorded in protocol')
    index={'bundle_version':'q4-r3-reduced-delivery-v1','producer_question':'q4','input_version':'904d7257b899b9f71d1ac9e0eab7479e459eab52','evaluation_stratum':'conditional_scenario_and_numerical_benchmark_separately_labelled','runs':34,'files':repo_files,'cache_files':list(cache_files.values()),'exclusions':['this index, to avoid self-reference','Numba cache/venv and ephemeral logs; environment lock/version metadata retained','other research agents outputs'],'limitations':['both reduced routes excluded from arrival envelope because thin-flow HR accuracy is unqualified','no calibration or independent event validation','1D equivalent-width swept area is not a planar hazard footprint','all 120 new 1D-OSM pairs have no target-cell spatial support; retain missing'],'all_scoped_files_stable_at_delivery':True}
    write_json(R/'results/reduced_delivery_index.json',index)
    print(json.dumps({'all_pass':validation['all_pass'],'files':len(repo_files),'cache_files':len(cache_files),'index':file_record(R/'results/reduced_delivery_index.json','delivery index')}))

if __name__=='__main__':main()
```

程序 37：spatial_build.py

来源：track_b/research/q4/round3/scripts/spatial_build.py
SHA256: f8ff904d1d9818c81ae8a9f07dd24e9b075bd2411e7ba773fd1b31a54b349c01

```
from pathlib import Path
import json,hashlib,datetime,csv,math,argparse
import numpy as np,rasterio
from rasterio.warp import reproject,Resampling
from rasterio.transform import from_origin
from rasterio.features import rasterize
from pyproj import Transformer
from shapely.geometry import shape,mapping,LineString,Point
from shapely.ops import transform
from scipy.sparse import coo_matrix
from scipy.sparse.csgraph import dijkstra
from scipy.ndimage import distance_transform_edt
import matplotlib;matplotlib.use('Agg')
import matplotlib.pyplot as plt
R=Path(__file__).resolve().parents[1];W=R.parents[4];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/spatial');C.mkdir(parents=True,exist_ok=True)
parser=argparse.ArgumentParser();parser.add_argument('--output-root');args=parser.parse_args()
if args.output_root:
 newroot=Path(args.output_root).resolve()
 if not newroot.is_relative_to(C):raise SystemExit('Replay root must be under assigned spatial cache')
 if newroot.exists():raise SystemExit('Replay root must be new, preserving immutable versions')
 R=newroot
 for folder in ['results','exports','figures']: (R/folder).mkdir(parents=True,exist_ok=True)
elif (R/'exports/geometry_bundle.json').exists():
 raise SystemExit('Frozen v1 exists; use --output-root <new spatial cache subdirectory> to replay without overwriting')

# W explicit because no dependence on associated chat cwd.
W=Path('<LOCAL_HOME>/Documents/MathorCup/worktrees/track-b-q4')
old=json.loads((W/'track_b/research/q4/round2/results/geometry_candidates.geojson').read_text());tr=Transformer.from_crs(4326,32645,always_xy=True);back=Transformer.from_crs(32645,4326,always_xy=True)
g={f['id']:transform(tr.transform,shape(f['geometry'])) for f in old['features']}
src=g['paper_source_outline'];target=g['paper_port_symbol'];channel=LineString(list(g['paper_chhochen_blue_trace'].coords)+list(g['paper_lende_blue_trace'].coords)[1:]);allbounds=np.array([gg.bounds for gg in
g.values()]);xmin,ymin=np.floor((allbounds[:,:2].min(axis=0)-1500)/120)*120;xmax,ymax=np.ceil((allbounds[:,2:].max(axis=0)+1500)/120)*120
cop=C/'Copernicus_DSM_COG_10_N28_00_E085_00_DEM.tif';
supplied=W/'track_b/data/processed/spatial/dem_utm45_30m.tif'
primary=cop if cop.exists() else supplied
version='q4-r3-spatial-copdem-v1' if primary==cop else 'q4-r3-spatial-supplied-v1'
def rec(p,role):
 b=p.read_bytes();return {'path':str(p.relative_to(W)) if p.is_relative_to(W) else str(p),'bytes':len(b),'sha256':hashlib.sha256(b).hexdigest(),'role':role}
records=[];grids={};feats=[];paths=[]
for res in [60,120]:
 width=int(round((xmax-xmin)/res));height=int(round((ymax-ymin)/res));aff=from_origin(xmin,ymax,res,res)
 z=np.full((height,width),-9999,dtype='float32')
 with rasterio.open(primary) as ds:reproject(rasterio.band(ds,1),z,src_transform=ds.transform,src_crs=ds.crs,dst_transform=aff,dst_crs='EPSG:32645',dst_nodata=-9999,resampling=Resampling.bilinear)
 profile={'driver':'GTiff','dtype':'float32','width':width,'height':height,'count':1,'crs':'EPSG:32645','transform':aff,'nodata':-9999,'compress':'deflate'}
```

```
dst=C/f'{version}_dem_{res}m.tif'
if not dst.exists():
 with rasterio.open(dst,'w',**profile) as ds:ds.write(z,1)
else:
 with rasterio.open(dst) as ds:np.testing.assert_allclose(ds.read(1),z)
mask=rasterize([(src,1)],out_shape=z.shape,transform=aff,fill=0,dtype='uint8')
channelmask=rasterize([(channel.buffer(res*.7),1)],out_shape=z.shape,transform=aff,fill=0,dtype='uint8');dist=distance_transform_edt(channelmask==0,sampling=res)
sr,sc=rasterio.transform.rowcol(aff,src.centroid.x,src.centroid.y);tgtr,tgtc=rasterio.transform.rowcol(aff,target.x,target.y)
arrays=R/f'results/spatial_inputs_{res}m.npz';np.savez_compressed(arrays,z=z,source_mask=mask,channel_distance_m=dist,x=xmin+(np.arange(width)+.5)*res,y=ymax-(np.arange(height)+.5)*res,dx=np.float64(res),source_rc=np.array([sr,sc]),target_rc=np.array([tgtr,tgtc]),transform=np.array(tuple(aff)),geometry_version=np.array(version))
records.extend([rec(dst,f'primary_dem_{res}m'),rec(arrays,f'model_arrays_{res}m')])
grids[str(res)]={'shape':[height,width],'resolution_m':res,'crs':'EPSG:32645','affine':list(aff)[:6],'bounds':[xmin,ymin,xmax,ymax],'valid_cells':int(np.sum(z!=-9999)),'source_cells':int(mask.sum()),'source_raster_area_m2':int(mask.sum())*res*res,'source_centroid_utm':[src.centroid.x,src.centroid.y],'source_rc':[int(sr),int(sc)],'target_paper_utm':[target.x,target.y],'target_rc':[int(tgtr),int(tgtc)],'dem_path':str(dst),'arrays_path':str(arrays.relative_to(W)) if arrays.is_relative_to(W) else str(arrays)}
if res==60:
 rr,cc=np.indices(z.shape);valid=z!=-9999;node=np.arange(z.size).reshape(z.shape)
 for vi,(pid,uw,cw) in enumerate([('P1',12,0),('P2',30,1),('P3',60,2)]):
  a=[];b=[];cost=[]
  for dr,dc in [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]:
   r0=slice(max(0,-dr),min(height,height-dr));c0=slice(max(0,-dc),min(width,width-dc));r1=slice(max(0,dr),min(height,height+dr));c1=slice(max(0,dc),min(width,width+dc));ok=valid[r0,c0]&valid[r1,c1];dz=z[r1,c1]-z[r0,c0];step=res*np.hypot(dr,dc)
   a.append(node[r0,c0][ok]);b.append(node[r1,c1][ok]);cost.append((step+uw*np.maximum(dz,0)+cw*step*np.minimum((dist[r0,c0]+dist[r1,c1])/600,5))[ok])
  # Reverse graph computes predecessor chain from the target to all possible sources.
  graph=coo_matrix((np.concatenate(cost),(np.concatenate(b),np.concatenate(a))),shape=(z.size,z.size)).tocsr();dd,prev=dijkstra(graph,directed=True,indices=tgtr*width+tgtc,return_predecessors=True)
  v=sr*width+sc;ns=[v]
  for j in range(z.size):
   if v==tgtr*width+tgtc:break
   v=int(prev[v]);
   if v<0:raise RuntimeError('route disconnected')
   ns.append(v)
  pts=[];ss=0;ascent=0
  for j,v in enumerate(ns):
   r,c=divmod(v,width);x=xmin+(c+.5)*res;y=ymax-(r+.5)*res
   if j:
    ss+=np.hypot(x-pts[-1][0],y-pts[-1][1]);ascent+=max(0,float(z[r,c])-pts[-1][2])
   pts.append((x,y,float(z[r,c])));paths.append({'path_id':pid,'node_index':j,'x_m':x,'y_m':y,'elevation_m':float(z[r,c]),'distance_m':ss,'row':r,'col':c,'geometry_version':version,'data_role':'conditional_scenario'})
  feats.append({'type':'Feature','id':pid,'properties':{'object_kind':'terrain_cost_path_candidate','geometry_version':version,'uphill_weight':uw,'corridor_weight':cw,'length_m':ss,'cumulative_ascent_m':ascent,'observed_path':False,'source':'Copernicus GLO-30 + paper candidates' if primary==cop else 'supplied DEM unknown product + paper candidates'},'geometry':mapping(transform(back.transform,LineString([(p[0],p[1]) for p in pts])))})
# Keep original candidate geometry IDs while explicitly separating object kinds.
for f in old['features']:
 q=dict(f);q['properties']=dict(f['properties'],geometry_version=version,role='conditional_computational_geometry',accepted_event_geometry=False);feats.append(q)
geo=R/'results/computational_geometries.geojson';geo.write_text(json.dumps({'type':'FeatureCollection','name':version,'crs_note':'GeoJSON CRS84 longitude/latitude; distances in EPSG32645','features':feats},indent=2)+'\n')
```

```
pathcsv=R/'results/spatial_paths.csv'
with pathcsv.open('w',newline='') as f:w=csv.DictWriter(f,fieldnames=list(paths[0]));w.writeheader();w.writerows(paths)
records.extend([rec(geo,'computational_geometry_candidates'),rec(pathcsv,'paths_60m_csv')])
# Native source metadata and derived terrain statistics are separate.
with rasterio.open(primary) as ds:origin={'source_path':str(primary),'sha256':hashlib.sha256(primary.read_bytes()).hexdigest(),'native_crs':str(ds.crs),'native_resolution':list(ds.res),'native_shape':list(ds.shape),'native_bounds':list(ds.bounds),'native_nodata':ds.nodata}
terrain={'geometry_version':version,'input_version':{'baseline_commit':'904d7257b899b9f71d1ac9e0eab7479e459eab52','baseline_manifest_sha256':'fd755fab38df285a67c986026859be5f635a314aa3f79f2d247a2626246c3a53'},'primary':origin,'vertical_datum':'EGM2008 orthometric height' if primary==cop else None,'observation_epoch':'WorldDEM TanDEM-X principal acquisition 2011-2015; edited release 2021, not 2026 event topography' if primary==cop else None,'resampling':'bilinear, no vertical transformation, no hydrologic burn/fill','grids':grids,'files':records,'status':'conditional_scenario; source/target still paper candidates','unknowns':['original paper datum and positional accuracy','actual failure polygon','actual bridge/facility identity at paper point','statistical independence of supplied DEM from Copernicus']}
terrainpath=R/'results/terrain_bundle.json';terrainpath.write_text(json.dumps(terrain,indent=2)+'\n');records.append(rec(terrainpath,'terrain_bundle'))
with np.load(R/'results/spatial_inputs_60m.npz') as zz:
 z=zz['z'];x=zz['x'];y=zz['y'];mask=zz['source_mask']
fig,(ax,ax2)=plt.subplots(1,2,figsize=(13,6));im=ax.imshow(z,extent=[xmin,xmax,ymin,ymax],cmap='terrain',origin='upper');ax.contour(x,y,mask,levels=[.5],colors='red',linewidths=1.2)
for f in feats[:3]:
 p=transform(tr.transform,shape(f['geometry']));ax.plot(*p.xy,label=f['id']);pp=[p for p in paths if p['path_id']==f['id']];ax2.plot([p['distance_m']/1000 for p in pp],[p['elevation_m'] for p in pp],label=f['id'])
ax.scatter([target.x],[target.y],marker='*',s=100,c='black',label='paper port candidate');ax.set_title('Conditional terrain paths; no observed connector');ax.set_xlabel('UTM45N easting (m)');ax.set_ylabel('northing (m)');ax.legend(fontsize=8);fig.colorbar(im,ax=ax,label='height (m)');ax2.set_xlabel('conditional path distance (km)');ax2.set_ylabel('elevation (m)');ax2.set_title('Computed profiles; not observed travel distances');ax2.legend();fig.tight_layout();fp=R/'figures/spatial_terrain_paths.png';fig.savefig(fp,dpi=160);plt.close(fig)
records.append(rec(fp,'terrain_path_figure'))
bundle={'bundle_version':version,'producer_question':'q4','input_version':terrain['input_version'],'evaluation_stratum':'conditional_scenario','files':records,'limitations':terrain['unknowns']+['DEM route is not observed event trajectory; no bridge/facility equivalence accepted','No release thickness or initial speed inferred from NC; caller must supply scenarios'],'source_centroid_utm':[src.centroid.x,src.centroid.y],'target_paper_utm':[target.x,target.y],'grids':grids}
(R/'exports/geometry_bundle.json').write_text(json.dumps(bundle,indent=2)+'\n')
print(json.dumps({'version':version,'grids':grids,'route_stats':[f['properties'] for f in feats[:3]],'bundle':str(R/'exports/geometry_bundle.json')},indent=2))
```

程序 38：map_q2_releases.py

来源：track_b/research/q4/round3/scripts/map_q2_releases.py
SHA256: 55bccbfbeaaf12633e0da851041f382ea5b58791984d6df8432cbcb1bfc2e64e

```
#!/usr/bin/env python3
"""Instantiate Q2 plan areas without substituting the full Q4 source area."""
from pathlib import Path
from datetime import datetime, timezone
import csv
import hashlib
import json
import math

import numpy as np
import rasterio
from affine import Affine
from scipy.optimize import brentq
from pyproj import Transformer
from shapely.geometry import Point, box, mapping, shape
from shapely.ops import transform as transform_geometry

ROOT = Path(__file__).resolve().parents[5]
R = ROOT/'track_b/research/q4/round3'
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/integrated_inputs/q4-r
    3-q2-mapped-release-v1')


def sha(p):
    return hashlib.sha256(p.read_bytes()).hexdigest()


def save(p, d):
    p.write_text(json.dumps(d, ensure_ascii=False, indent=2, allow_nan=False)+'\n')


def rec(p, role):
    return {'path': str(p.relative_to(ROOT)) if p.is_relative_to(ROOT) else str(p),
            'bytes': p.stat().st_size, 'sha256': sha(p), 'role': role}


def main():
    protocol = json.loads((R/'config/q2_release_mapping_protocol.json').read_text())
    bundle_path = ROOT/protocol['q2_bundle_path']
    assert sha(bundle_path) == protocol['q2_bundle_sha256']
    bundle = json.loads(bundle_path.read_text())
    state_entry = next(f for f in bundle['files'] if f['role'] == 'results/release_states.csv')
    state_path = ROOT/state_entry['path']
    assert sha(state_path) == state_entry['sha256']
    states = {row['scenario_id']: row for row in csv.DictReader(state_path.open())}
    geom_path = R/'results/computational_geometries.geojson'
    gj = json.loads(geom_path.read_text())
    # The frozen input is RFC7946-style CRS84 longitude/latitude, with crs_note;
    # never assign a metric CRS to degree coordinates.
    source_ll = shape(next(f['geometry'] for f in gj['features'] if f['id'] ==
    'paper_source_outline'))
    assert -180 <= source_ll.bounds[0] <= 180 and -90 <= source_ll.bounds[1] <= 90
    source = transform_geometry(Transformer.from_crs(4326, 32645, always_xy=True).transform,
    source_ll)
    centre = source.centroid
    a = np.load(R/'results/spatial_inputs_60m.npz')
    z, dx = a['z'], float(a['dx'])
```

```
    transform = Affine(*a['transform'][:6])
    CACHE.mkdir(parents=True, exist_ok=True)
    features, records = [], []
    for ident in protocol['selected_scenario_ids']:
        row = states[ident]
        area = float(row['plan_area_m2'])
        height = float(row['release_thickness_vertical_m'])
        volume = float(row['release_volume_m3'])
        assert 0 < area < source.area and math.isclose(area*height, volume, rel_tol=1e-12)
        fractions = json.loads(row['phase_fractions'])
        assert row['phase_fraction_basis'] == 'volume' and
        ↪  math.isclose(sum(fractions.values()), 1, abs_tol=1e-12)
        density = float(row['density'])
        assert math.isclose(density,
        ↪  2700*fractions['rock']+917*fractions['ice']+1000*fractions['free_water'],
        ↪  rel_tol=1e-12)
        radius = brentq(lambda radius: source.intersection(centre.buffer(radius,
        ↪  quad_segs=64)).area-area,
                        0.01, 10000, xtol=1e-9)
        polygon = source.intersection(centre.buffer(radius, quad_segs=64))
        fraction = np.zeros(z.shape, dtype=np.float64)
        # Domain is small; only polygon bounding-box cells need exact intersections.
        west, south, east, north = polygon.bounds
        r0, c0 = rasterio.transform.rowcol(transform, west, north)
        r1, c1 = rasterio.transform.rowcol(transform, east, south)
        for i in range(max(0, r0-1), min(z.shape[0], r1+2)):
            for j in range(max(0, c0-1), min(z.shape[1], c1+2)):
                x, y = transform*(j, i)
                fraction[i, j] = polygon.intersection(box(x, y-dx, x+dx, y)).area/(dx*dx)
        depth = (height*fraction).astype('float32')
        actual = float(depth.astype('float64').sum()*dx*dx)
        assert math.isclose(float(fraction.sum()*dx*dx), area, rel_tol=1e-10)
        assert math.isclose(actual, volume, rel_tol=1e-6)
        tag = ident.replace('-', '_')
        path = CACHE/f'{tag}_release_60m.tif'
        with rasterio.open(path, 'w', driver='GTiff', height=z.shape[0], width=z.shape[1],
                           count=1, dtype='float32', crs='EPSG:32645', transform=transform,
                           nodata=-9999, compress='deflate') as ds:
            ds.write(depth, 1)
        npz = R/'results'/f'q2_mapped_{tag}_60m.npz'
        np.savez_compressed(npz, release_depth=depth,
        ↪  source_fraction=fraction.astype('float32'),
                            transform=np.array(transform), x=a['x'], y=a['y'])
        metadata = {
            'scenario_id': ident, 'upstream_model_id': row['model_id'], 'mechanism_id':
            ↪  row['mechanism_id'],
            'upstream_parameter_version': row['parameter_version'], 'geometry_version':
            ↪  'q4-r3-q2-mapped-release-v1',
            'source_plan_area_m2': area, 'actual_polygon_area_m2': polygon.area,
            'thickness_normal_m': float(row['release_thickness_normal_m']),
            ↪  'thickness_vertical_m': height,
            'upstream_release_volume_m3': volume, 'grid_integrated_release_volume_m3': actual,
            'grid_volume_relative_error': (actual-volume)/volume, 'grid_resolution_m': dx,
            'fractional_source_cells': int((fraction > 0).sum()),
            ↪  'equivalent_full_source_cells': area/(dx*dx),
            'fractional_area_method': 'Exact intersection area, mean vertical cell thickness;
            ↪  underresolved small sources explicitly retained',
            'phase_fraction_basis': 'volume', 'phase_fractions': fractions,
            ↪  'bulk_density_kg_m3': density,
            'initial_velocity_east_north_m_s': [0, 0], 'initial_velocity_crs': 'EPSG:32645',
            'upstream_onset_status': row['onset_status'], 'if_released_propagation':
            ↪  row['onset_status'] == 'never_crossed',
            'not_an_initiation_prediction': True, 'initiation_to_release_delay_seconds': None,
```

```
            'evaluation_stratum': 'conditional_scenario', 'data_role': 'scenario', 'time_utc':
            ↪  None,
            'time_uncertainty': None, 'available_time': None,
            'source_mapping_assumption': 'Circular compact subsource centred on paper candidate
            ↪  centroid, clipped to that polygon, area solved exactly; no observed release
            ↪  geometry claim',
            'release_raster': rec(path, 'native_solver_mapped_release'), 'release_arrays':
            ↪  rec(npz, 'mapped_release_arrays')}
        features.append({'type': 'Feature', 'id': ident, 'geometry': mapping(polygon),
        ↪  'properties': metadata})
        records.append(metadata)
    out = R/'results/q2_mapped_releases.geojson'
    save(out, {'type': 'FeatureCollection', 'name': 'q4-r3-q2-mapped-release-v1',
               'crs': {'type': 'name', 'properties': {'name': 'EPSG:32645'}}, 'features':
               ↪  features})
    save(R/'results/q2_release_mapping.json', {
        'created_at_utc': datetime.now(timezone.utc).isoformat(), 'protocol':
        ↪  rec(R/'config/q2_release_mapping_protocol.json', 'preregistration'),
        'upstream_bundle': rec(bundle_path, 'advisor_delivered_q2_bundle'), 'source_geometry':
        ↪  rec(geom_path, 'frozen_candidate_envelope'),
        'mapped_geometries': rec(out, 'mapped_compact_sources'), 'releases': records,
        'no_probability_weights': True, 'geometry_truth_accepted': False})
    print(json.dumps({'mapped': len(records), 'volumes_m3':
    ↪  [row['grid_integrated_release_volume_m3'] for row in records],
                      'if_released': [row['scenario_id'] for row in records if
                      ↪  row['if_released_propagation']]}))


if __name__ == '__main__':
    main()
```

程序 39：official_q2_v2_cases.py

来源：track_b/research/q4/round3/scripts/official_q2_v2_cases.py
SHA256：0b8593db9610bb4aabcf3590396876fef21ff773c056001ae7ff5b38f7658821

```
#!/usr/bin/env python3
"""Add fixed full-mask Q2 releases without replacing compact-source v1."""
from pathlib import Path
import json,hashlib,shutil,datetime
import rasterio,numpy as np
P=Path(__file__).resolve().parents[1];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official')
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
pp=P/'config/q2_fullmask_integration_protocol.json';d=json.loads(pp.read_text());items=[]
(P/'config/official_q2_v2_protocol.json').write_text(json.dumps({'registered_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'before_native_runs':True,'upstream_protocol_path':str(pp),'upstream_protocol_sha256':sha(pp),'selection':d['selected_scenario_ids'],'model':'fixed official native single homogeneous equivalent, composition passive','postrelease_parameters':{'mu_basal':.15,'xi':1000},'source_and_volume':'fullmask v2 distinct from compact v1; scenario suffix and base key retained'},indent=2)+'\n')
for record in d['records']:
 dx=int(record['grid_resolution_m']);ref=json.loads((C/f'runs/copdem_{dx}_single_ref/case.json').read_text());src=Path(record['release_raster_path']);assert sha(src)==record['release_raster_sha256'];f=C/'inputs/q2fullmask'/record['scenario_id'];f.mkdir(parents=True,exist_ok=True);target=f/'release.tif';shutil.copy2(src,target)
 with rasterio.open(target) as ds:vol=float(ds.read(1).astype(float).sum()*dx*dx)
 assert abs(vol/record['actual_grid_volume_m3']-1)<1e-9
 rid='q2full_'+record['scenario_id'].replace('-','_');folder=C/'runs'/rid;folder.mkdir(parents=True,exist_ok=True);ref.update({'run_id':rid,'scenario_id':record['scenario_id'],'base_scenario_id':record['base_scenario_id'],'geometry_version':record['geometry_version'],'release_mapping_version':record['release_mapping_version'],'upstream_model_id':record['model_id'],'upstream_parameter_version':record['parameter_version'],'upstream_onset_status':record['onset_status'],'if_released_propagation':record['if_released_propagation'],'not_an_initiation_prediction':True,'release':str(target),'release_sha256':sha(target),'release_grid_volume_m3':vol,'release_thickness_m':float(record['release_thickness_vertical_m']),'bulk_density_kg_m3':float(record['density']),'source_plan_area_m2':float(record['plan_area_m2']),'passive_material_volume_fractions':json.loads(record['phase_fractions']),'passive_material_fraction_basis':'volume','phase_volume_fractions':[1],'initial_velocity_east_north_m_s':[0,0],'initiation_to_release_delay_seconds':None,'evidence_role':'conditional_cross_question_scenario','upstream_fullmask_record':record,'source_geometry_sha256':sha(pp),'extras':{}})
 (folder/'case.json').write_text(json.dumps(ref,indent=2)+'\n');items.append({'run_id':rid,'case':str(folder/'case.json')})
(P/'config/official_cases_q2fullmask.json').write_text(json.dumps({'cases':items},indent=2)+'\n');print(len(items))
```

程序 40：extend_computational_domain.py

来源：track_b/research/q4/round3/scripts/extend_computational_domain.py
SHA256: d93074dc940ab3008751787f7ba0978aca7b84e52a9ddfb9e11caa56e6e4cc3f

```python
#!/usr/bin/env python3
"""Extend same-product borders while preserving the original core for comparison."""
from pathlib import Path
from datetime import datetime, timezone
import hashlib
import json
import numpy as np
import rasterio
from affine import Affine
from rasterio.warp import reproject, Resampling

ROOT = Path(__file__).resolve().parents[5]
R = ROOT/'track_b/research/q4/round3'
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/integrated_inputs/domain_extension_v1')


def sha(p):
    return hashlib.sha256(p.read_bytes()).hexdigest()


def rec(p, role):
    return {'path': str(p), 'bytes': p.stat().st_size, 'sha256': sha(p), 'role': role}


def main():
    protocol = json.loads((R/'config/domain_extension_protocol.json').read_text())
    terrain = json.loads((R/'results/terrain_bundle.json').read_text())
    p = int(protocol['pad_cells_each_side'])
    base = np.load(R/'results/spatial_inputs_120m.npz')
    old = base['z']
    dx = float(base['dx'])
    tr = Affine(*base['transform'][:6]) * Affine.translation(-p, -p)
    z = np.full((old.shape[0]+2*p, old.shape[1]+2*p), np.nan, dtype='float32')
    raw = Path(terrain['primary']['source_path'])
    assert sha(raw) == terrain['primary']['sha256']
    with rasterio.open(raw) as src:
        reproject(rasterio.band(src, 1), z, src_transform=src.transform, src_crs=src.crs,
                  dst_transform=tr, dst_crs='EPSG:32645', resampling=Resampling.bilinear,
                  dst_nodata=np.nan)
    assert np.isfinite(z).all()
    old_slice = (slice(p, p+old.shape[0]), slice(p, p+old.shape[1]))
    difference = z[old_slice].astype('float64')-old
    comparison = {'mean_m': float(difference.mean()), 'rmse_m': float(np.sqrt(np.mean(difference**2))),
                  'max_abs_m': float(np.max(np.abs(difference)))}
    # Explicit same-source core preservation removes a resampling-domain confound.
    z[old_slice] = old
    h = np.pad(base['source_mask'].astype('float64')*(925200/964800)*15, p).astype('float32')
    CACHE.mkdir(parents=True, exist_ok=True)
    files = []
    for name, data in [('dem_120m.tif', z), ('release_h15_120m.tif', h)]:
        path = CACHE/name
        with rasterio.open(path, 'w', driver='GTiff', width=z.shape[1], height=z.shape[0],
                           count=1, dtype='float32', crs='EPSG:32645', transform=tr,
                           nodata=-9999, compress='deflate') as ds:
            ds.write(data, 1)
```

```
        files.append(rec(path, 'extended_dem' if name.startswith('dem') else 'same_reference_release'))
    result = {'version': protocol['extended_geometry_version'], 'created_at_utc': datetime.now(timezone.utc).isoformat(),
              'protocol_sha256': sha(R/'config/domain_extension_protocol.json'),
              'base_grid_path': 'results/spatial_inputs_120m.npz', 'base_grid_sha256': sha(R/'results/spatial_inputs_120m.npz'),
              'source_product': rec(raw, 'same_native_Copernicus_tile'), 'shape': list(z.shape),
              'affine': list(tr)[:6], 'resolution_m': dx, 'pad_cells': p,
              'same_source_resampling_difference_before_core_preservation': comparison,
              'core_dem_exactly_preserved': bool(np.array_equal(z[old_slice], old)),
              'release_grid_volume_m3': float(h.astype('float64').sum()*dx*dx),
              'files': files,
              'target_policy': 'Copy unchanged actual targets directly from the original DClaw reference case.',
              'evaluation_stratum': 'conditional_scenario'}
    (R/'results/extended_domain_bundle.json').write_text(json.dumps(result, ensure_ascii=False, indent=2)+'\n')
    print(json.dumps({'shape': list(z.shape), 'release_volume_m3': result['release_grid_volume_m3'],
                      'core_preserved': result['core_dem_exactly_preserved'], 'resampling_diff': comparison}))


if __name__ == '__main__':
    main()
```

程序 41：official_composite_cases.py

来源：track_b/research/q4/round3/scripts/official_composite_cases.py
SHA256：f8dacb4d99542bfa29a80181c66cabe521ea407a391ff71828fe703a799b27ff

```
#!/usr/bin/env python3
"""Two physically admissible fixed-volume rock-ice composite+water challenges."""
import json,hashlib,datetime
from pathlib import Path
P=Path(__file__).resolve().parents[1];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official');src=C.parents[1]/'runtime/sources/r.avaflow.40G/r.avaflow.main/main.c';lines=src.read_text().splitlines();rho_comp=(.6*2700+.1*917)/.7
protocol={'version':'q4-r3-official-composite-water-v1','registered_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'before_execution':True,'fixed_source_sha256':hashlib.sha256(src.read_bytes()).hexdigest(),'source_excerpts':[{'range':str((a,b)),'text':'\n'.join(f'{i+1}: {lines[i]}' for i in range(a-1,b))} for a,b in [(706,763),(790,804),(1160,1168),(1200,1269)]],'rejected_canonical_pure_RWI':{'densities_kg_m3':[2700,917,1000],'rho_water_over_ice':1000/917,'one_minus_gamma_ice_water':1-1000/917,'reason':'Default native pairdrag coefficient contains(1-rho3/rho2), negative withP2ice/P3water; no unjustified negative-drag run. ReorderingP2water/P3ice doesnot preserve native PHI3-based fluidbuoyancy closure.'},'adopted_challenge':'P1 frictional rock-ice composite with volume-weighted density, P2empty, P3water. No pureice independentvelocity,no melting,no strength change','composite_total_volume_fractions':{'rock':.6,'ice':.1,'water':.3},'rho_composite_kg_m3':rho_comp,'matched_control':'sameP1volume.7 andwater.3, P1allrockdensity2700','held_common':['DEM120m','sourcevolume13.878Mm3','sourceh15scaled','mu.15','xi1000','initialspeed0','couplingdefault','csurface0'],'not_held_common':'Totalmass differs atfixedV becausecomponentdensity changes; not a fixedmass isolatediceeffect','phase_density_parser':'per-case densities_kg_m3 required; nohardcodedproxy mass','end_seconds':1800}
(P/'config/official_composite_protocol.json').write_text(json.dumps(protocol,indent=2)+'\n');reference=json.loads((C/'runs/copdem_120_multi_ref__sf0/case.json').read_text());cases=[]
for rid,rhos,composition in [('composite_RI_water_120__sf0',[rho_comp,1850,1000],{'rock':.6,'ice':.1,'free_water':.3}),('composite_rock_water_120__sf0',[2700,1850,1000],{'rock':.7,'ice':0,'free_water':.3})]:
 case=dict(reference);case.update({'run_id':rid,'scenario_id':rid,'densities_kg_m3':rhos,'phase_volume_fractions':[.7,0,.3],'phase_material_names':['rock_ice_composite' if composition['ice'] else 'rock','empty','water'],'material_volume_fractions':composition,'material_fraction_basis':'volume','cmelt':0,'ice_role':'densityclosureonly; noice-rock relativevelocity,nophasechange,nostrengthchange','expected_initial_mass_kg':case['release_grid_volume_m3']*(.7*rhos[0]+.3*rhos[2]),'evidence_role':'conditional_component_closure_challenge'})
 f=C/'runs'/rid;f.mkdir(parents=True,exist_ok=True);cp=f/'case.json';cp.write_text(json.dumps(case,indent=2)+'\n');cases.append({'run_id':rid,'case':str(cp)})
(P/'config/official_cases_composite.json').write_text(json.dumps({'cases':cases},indent=2)+'\n');print(rho_comp)
```

程序 42：official_entrainment_case.py

来源：track_b/research/q4/round3/scripts/official_entrainment_case.py
SHA256：110482d57e8641fc266a793f2c2cc2b54a23d5a44c4ccf526477706ab5c3af88

```
#!/usr/bin/env python3
"""Register and prepare one bounded native entrainment comparison; no tuning."""
from pathlib import Path
import json,hashlib,datetime,csv
import numpy as np,rasterio
from rasterio.features import rasterize
from shapely.geometry import LineString
P=Path(__file__).resolve().parents[1];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official');RT=C.parents[1]/'runtime'
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
reference=C/'runs/copdem_120_single_ref__sf0/case.json';case=json.loads(reference.read_text());rid='copdem_120_single_entrain1m_P1buf250__sf0';source=RT/'sources/r.avaflow.40G/r.avaflow.main/main.c';lines=source.read_text().splitlines();pathfile=P/'results/spatial_paths.csv';pathrows=[r for r in csv.DictReader(pathfile.open()) if r['path_id']=='P1'];geometry=LineString([(float(r['x_m']),float(r['y_m'])) for r in pathrows]).buffer(250)
with rasterio.open(case['dem']) as ds:profile=ds.profile.copy();shape=ds.shape;tr=ds.transform
with rasterio.open(case['release']) as ds:h=ds.read(1)
mask=rasterize([(geometry,1)],out_shape=shape,transform=tr,fill=0,all_touched=False).astype(bool);mask[h>0]=False;mask[:3]=False;mask[-3:]=False;mask[:,:3]=False;mask[:,-3:]=False
protocol={'version':'q4-r3-native-bounded-entrainment-v1','registered_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'before_execution':True,'historical_registered_case_count':38,'target_applicable_required_count':30,'diagnostic_only_count':1,'material_excluded_not_run_count':7,'covers_decision03_counts_without_modifying_it':True,'reference_run_id':case['run_id'],'new_run_id':rid,'scenario_source':'analystdefined bounded sensitivity, not measured erodiblethickness or calibratedtargettime; independentpreregisteredP1path, notnative2Dwetfootprint','path_file':str(pathfile.resolve()),'path_sha256':sha(pathfile),'path_id':'P1','buffer_metres':250,'bed_max_thickness_m':1,'mask_rules':['cellcentrewithinP1buffer250m','excludeinitialsourceh>0','excludeouter3cellrings'],'max_available_bed_volume_m3':float(mask.sum()*120**2),'bed_density_kg_m3':2105,'centrainment':1,'entrainment':'-7,0','cstopping':0,'fixed_source_sha256':sha(source),'source_excerpts':[{'line_start':a,'line_end':b,'text':'\n'.join(f'{i}: {lines[i-1]}' for i in range(a,b+1))} for a,b in [(7205,7215),(7241,7251),(7280,7285),(8663,8670)]],'implemented_rate':'delta_h=dt*10^(-abs(CENTR))*rho*h*horizontal_speed*betav, capremainingbed; CENTRnegative ->betav1; notkineticenergy','coefficient_units':'m2/kg inferredfromimplementationdimensions, notofficialmeasuredparameter','native_entrained_signed_volume':'nout1 zeroindexed23:26 equalsintegralbasechange, negativeerosion; Madded=-rho*VE','source_adjusted_budget':'Mactivecore+Medgestore-Minitial-Madded; entrainmentgrowthnotmasserror','same_as_reference':['DEM','releasevolume','density','mu.15','xi1000','csurface0','initialvelocity0','1800shorizon','20snominaloutput'],'stationary_residual_definition':'finite finalstate h>=.1m and maximum horizontal speed among wetphases<.1m/s; notpermanentdeposit'}
(P/'config/official_entrainment_protocol.json').write_text(json.dumps(protocol,indent=2)+'\n')
inp=C/'inputs/entrainment120';inp.mkdir(parents=True,exist_ok=True);bed=inp/'erodible_bed_max_1m_P1buf250m.tif';profile.update(dtype='float32',count=1,nodata=-9999,compress='deflate')
with rasterio.open(bed,'w',**profile) as ds:ds.write(mask.astype('float32'),1)
```

```
case.update({'run_id':rid,'scenario_id':rid,'reference_comparison_run_id':case['run_id'],'comparison_reference_run_id':case['run_id'],'centrainment':1,'erodible_bed':str(bed),'erodible_bed_sha256':sha(bed),'max_available_bed_volume_m3':protocol['max_available_bed_volume_m3'],'erodible_bed_density_kg_m3':2105,'extras':{'entrainment':'-7,0'},'evidence_role':'bounded_entrainment_conditional_sensitivity','bed_scenario_source':protocol['scenario_source']});folder=C/'runs'/rid;folder.mkdir(exist_ok=True);cp=folder/'case.json';cp.write_text(json.dumps(case,indent=2)+'\n');item={'run_id':rid,'case':str(cp)};(P/'config/official_cases_entrainment.json').write_text(json.dumps({'protocol':protocol['version'],'cases':[item]},indent=2)+'\n');(P/'results/official_entrainment_preparation.json').write_text(json.dumps({'prepared_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'case':item,'case_sha256':sha(cp),'protocol_sha256':sha(P/'config/official_entrainment_protocol.json'),'bed_path':str(bed),'bed_sha256':sha(bed),'bed_cells':int(mask.sum()),'bed_volume_m3':protocol['max_available_bed_volume_m3'],'release_volume_unchanged_m3':case['release_grid_volume_m3'],'same_reference_case_sha256':sha(reference)},indent=2)+'\n');print(json.dumps({'run_id':rid,'bed_cells':int(mask.sum()),'bed_volume_m3':protocol['max_available_bed_volume_m3']}))
```

程序 43：official_container.py

来源：track_b/research/q4/round3/scripts/official_container.py
SHA256: 7a480b0137c893a1fe5e9cde4b869b01ec6ef50055696df5217146d679971ad3

```
#!/usr/bin/env python3
"""Run inside immutable published Linux image. Does not compile or modify /opt."""
import os,json,sys,subprocess,shutil,math,importlib.util
from pathlib import Path
import numpy as np
from osgeo import gdal
CACHE=Path('/work')
def translatepath(p):return os.environ.get('OFFICIAL_WORK_ROOT','/work')+'/'+str(p).split('/q4/official/',1)[1]
def run_rava(case):
 folder=Path.cwd();dem=translatepath(case['dem']);rel=translatepath(case['release']);dx=case['resolution_m'];out=folder/'run_grass.sh'
 cmd=['r.avaflow.40G','-ae' if case.get('additional_output',True) else '-e',f'prefix={case["run_id"]}',f'cellsize={dx}','elevation=elev','ctopo=0',f'csurface={case.get("csurface",0)}',f'centrainment={case.get("centrainment",0)}','cstopping=0','cmelt=0','gravity=9.81','limiter=1',f'time={case["output_interval_seconds"]},{case["end_seconds"]}',f'cfl={case["extras"].get("cfl",.5)},0.01','thresholds=0.1,10000,10000,0.1,0.000001']
 delta=math.degrees(math.atan(case['mu_basal']))
 if case['model']=='ravaflow_single':cmd+=['phases=1','hrelease=release',f'density={case.get("bulk_density_kg_m3",2105)}',f'friction={case["extras"].get("internal_friction_degrees",35)},{delta},{-case["xi"]}','viscosity=-7']
 else:
  fractions=case['phase_volume_fractions'];delta2=delta
  cmd+=['phases=3','clayers=0','hrelease1=release1','hrelease2=release2','hrelease3=release3','density='+','.join(str(v) for v in case.get('densities_kg_m3',[2700,1850,1000])),f'friction=35,20,0,{delta},{delta2},0,{-case["xi"]},{-case["xi"]},{-case["xi"]}','viscosity=-7,-7,-7','transformation=0,0,0',f'drag={case["extras"].get("drag","1,3,1,0.1,1,1")}',f'virtualmass={case["extras"].get("virtualmass","10,0.12,1")}']
 import shlex
 lines=['#!/bin/bash','set -euo pipefail','command -v r.avaflow.main','sha256sum "$(command -v r.avaflow.main)"',f'r.in.gdal --overwrite input={shlex.quote(dem)} output=elev',f'r.in.gdal --overwrite input={shlex.quote(rel)} output=release','g.region -s raster=elev']
 if case.get('erodible_bed'):
  lines.extend([f'r.in.gdal --overwrite input={shlex.quote(translatepath(case["erodible_bed"]))} output=erodible_bed','r.univar -g map=erodible_bed','r.info -r map=erodible_bed']);cmd+=['hentrmax1=erodible_bed','entrainment='+case['extras']['entrainment']]
 if case['model']!='ravaflow_single':
  for i,x in enumerate(fractions,1):lines.append(f'r.mapcalc --overwrite expression="release{i}=release*{x}"')
 lines.append(shlex.join(cmd));out.write_text('\n'.join(lines)+'\n')
 subprocess.run(['grass','--tmp-location',dem,'--exec','bash',str(out)],check=True)
def dclaw(case):
 # Import official unmodified gully parameter baseline, then explicitly override all scenario-dependent fields.
 sys.path.insert(0,'/opt/clawpack/dclaw/examples/gully');import setrun as original
 d=original.setrun();c=d.clawdata;dx=case['resolution_m']
 dem=gdal.Open(translatepath(case['dem']));z=dem.ReadAsArray();tr=dem.GetGeoTransform();h=gdal.Open(translatepath(case['release'])).ReadAsArray();nx=dem.RasterXSize;ny=dem.RasterYSize
 x=tr[0]+(np.arange(nx)+.5)*dx;y=tr[3]-(np.arange(ny)+.5)*dx
 from clawpack.geoclaw import topotools
 # Boundary nodal padding ensures topography interpolation covers all domain faces.
 for name,arr in [('basal_topo.tt3',z),('thickness.tt3',h)]:
```

```
 topo=topotools.Topography();topo.x=np.r_[x[0]-dx,x,x[-1]+dx];topo.y=np.r_[y[-1]-dx,y[::-1],y[0]+dx];topo.Z=np.pad(arr[::-1],1,mode='edge');topo.write(name,topo_type=3,Z_format='%15.7e')
c.lower=[float(tr[0]),float(tr[3]-ny*dx)];c.upper=[float(tr[0]+nx*dx),float(tr[3])];c.num_cells=[nx,ny];c.output_style=1;c.num_output_times=int(case['end_seconds']/case['output_interval_seconds']);c.tfinal=case['end_seconds'];c.output_t0=True;c.output_format='binary64';c.output_q_components='all';c.output_aux_components='none';c.verbosity=0;c.dt_variable=True;c.dt_initial=.01;c.cfl_desired=.4;c.cfl_max=.49;c.steps_max=100000;c.order=1;c.transverse_waves=0;c.bc_lower=['extrap','extrap'];c.bc_upper=['extrap','extrap'];c.checkpt_style=0
a=d.amrdata;a.amr_levels_max=1;a.refinement_ratios_x=[1];a.refinement_ratios_y=[1];a.refinement_ratios_t=[1];a.verbosity_regrid=0;a.max1d=max(nx,ny)+4;d.regiondata.regions=[];d.flowgrades_data.flowgrades=[];d.fgmax_data.fgmax_grids=[];d.fgout_data.fgout_grids=[];d.gauged_data.gauges=[]
d.geo_data.coordinate_system=1;d.geo_data.sea_level=-9999;d.geo_data.dry_tolerance=1e-3;d.geo_data.manning_coefficient=0.;d.geo_data.friction_forcing=True
dc=d.dclaw_data;dc.rho_s=2700.;dc.rho_f=1000.;dc.m0=case['extras']['m0'];dc.mref=.63;dc.m_crit=.64;dc.kref=case['extras']['kref'];dc.phi=math.degrees(math.atan(case['mu_basal']));dc.mu=.005;dc.entrainment=0;dc.segregation=0;dc.bed_normal=0;dc.src2method=2;dc.alphamethod=1
d.pinitdclaw_data.init_ptype=case['extras']['init_ptype'];d.topo_data.topofiles=[[3,'basal_topo.tt3']];d.qinitdclaw_data.qinitfiles=[[3,1,'thickness.tt3']];d.write()
shutil.copy2('/opt/clawpack/dclaw/examples/gully/xdclaw','xdclaw');subprocess.run(['./xdclaw'],check=True)
if __name__=='__main__':
 case=json.loads(Path(sys.argv[1]).read_text());os.chdir(Path(sys.argv[1]).parent)
 if case['model'].startswith('ravaflow'):run_rava(case)
 else:dclaw(case)
```

程序 44：official_launch_container.py

来源：track_b/research/q4/round3/scripts/official_launch_container.py
SHA256：c501f6e594eb503b4f3af0de962fa628869b30c5fb913cbf271e3d42110e42a9

```
#!/usr/bin/env python3
"""Execute authorised shared container, distinct writable /work/q4/official only."""
from pathlib import Path
import argparse,json,subprocess,datetime,time,hashlib
P=Path(__file__).resolve().parents[1];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official');DEST='/work/q4/official'
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def main():
 a=argparse.ArgumentParser();a.add_argument('--container',default='mathorcup-r3-toolchain-build');a.add_argument('--cases',required=True);a.add_argument('--only',default='');a.add_argument('--retry',action='store_true');args=a.parse_args()
 info=json.loads(subprocess.check_output(['docker','inspect',args.container]))[0];cid=info['Id'];cases=json.loads(Path(args.cases).read_text())['cases']
 context={'container_name':args.container,'container_id':cid,'base_image_id':info['Image'],'published_solver_image_id':None,'fixed_source_manifest':'<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/runtime/source_versions.json','source_versions_sha256':sha(C.parents[1]/'runtime/source_versions.json'),'scope':'authorised existing local compiled container; /opt sources untouched','inspected_utc':datetime.datetime.now(datetime.timezone.utc).isoformat()}
 (P/'sources/official_container_context.json').write_text(json.dumps(context,indent=2)+'\n')
 subprocess.run(['docker','exec',args.container,'mkdir','-p',DEST+'/inputs',DEST+'/runs',DEST+'/scripts'],check=True)
 subprocess.run(['docker','cp',str(C/'inputs')+'/.',args.container+':'+DEST+'/inputs/'],check=True)
 subprocess.run(['docker','cp',str(P/'scripts/official_container.py'),args.container+':'+DEST+'/scripts/official_container.py'],check=True)
 for item in cases:
  rid=item['run_id']
  if args.only and args.only not in rid:continue
  cp=Path(item['case']);f=cp.parent;meta=f/'execution.json'
  if meta.exists() and not args.retry and json.loads(meta.read_text())['returncode']==0:print('skip',rid,flush=True);continue
  subprocess.run(['docker','exec',args.container,'mkdir','-p',DEST+'/runs/'+rid],check=True);subprocess.run(['docker','cp',str(cp),args.container+':'+DEST+'/runs/'+rid+'/case.json'],check=True)
  cmd=['docker','exec','-e','CLAW=/opt/clawpack','-e','PYTHONPATH=/opt/clawpack','-e','GRASS_ADDON_BASE=/opt/grass-addons','-e','MPLBACKEND=Agg','-e','OMP_NUM_THREADS=2','-e','OFFICIAL_WORK_ROOT='+DEST,'-w',DEST+'/runs/'+rid,args.container,'python3',DEST+'/scripts/official_container.py',DEST+'/runs/'+rid+'/case.json']
  start=time.time();log=f/'run.log'
  with log.open('w') as out:r=subprocess.run(cmd,stdout=out,stderr=subprocess.STDOUT)
  copycmd=['docker','cp',args.container+':'+DEST+'/runs/'+rid+'/.',str(f)];cpres=subprocess.run(copycmd,stdout=subprocess.PIPE,stderr=subprocess.PIPE,text=True)
  d={'run_id':rid,'image_id':None,'container_id':cid,'base_image_id':info['Image'],'command':cmd,'copy_output_command':copycmd,'copy_returncode':cpres.returncode,'copy_stderr':cpres.stderr,'case_sha256':sha(cp),'started_utc':datetime.datetime.fromtimestamp(start,datetime.timezone.utc).isoformat(),'elapsed_seconds':time.time()-start,'returncode':r.returncode,'log_path':str(log),'log_sha256':sha(log),'shared_solver_source_unchanged':True};meta.write_text(json.dumps(d,indent=2)+'\n');print(rid,r.returncode,round(d['elapsed_seconds'],2),flush=True)
if __name__=='__main__':main()
```

程序 45：official_analyse.py

来源：track_b/research/q4/round3/scripts/official_analyse.py
SHA256: 5f8f127f8af0ec6f63ca78bb1fd1d9fd02b35cf348b799ec5ff3d98d3530d1d5

```
#!/usr/bin/env python3
"""Parse actual official outputs, never substitute solver exit status for science checks."""
from pathlib import Path
import hashlib
PARSER_SOURCE_BYTES=Path(__file__).read_bytes();PARSER_SHA256=hashlib.sha256(PARSER_SOURCE_BYTES).hexdigest();FIELDS_SCHEMA='v4_phase_depth_stationary'
import numpy as np,rasterio,json,csv,re,hashlib,datetime
import matplotlib;matplotlib.use('Agg')
import matplotlib.pyplot as plt
from official_catalogue import applicability
P=Path(__file__).resolve().parents[1];C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official')
PARSER_ARCHIVE=C/'postprocessing_sources'/('official_analyse_'+PARSER_SHA256+'.py');PARSER_ARCHIVE.parent.mkdir(exist_ok=True)
if not PARSER_ARCHIVE.exists():PARSER_ARCHIVE.write_bytes(PARSER_SOURCE_BYTES)
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def native_ravaflow_times(summary_text,log_text):
 """Only explicit native nout/tsum rows; never infer t from frame index."""
 times={}
 for text in [summary_text,log_text]:
  for line in text.splitlines():
   z=line.split()
   if len(z)>8 and z[0].isdigit():
    try:
     t=float(z[4])
     if np.isfinite(t):times[int(z[0])]=t
    except ValueError:pass
 return times
def solver_identity(folder,case,ex):
 """Describe the binary actually selected, separately from archived official source."""
 log=(folder/'run.log').read_text(errors='replace');build=ex.get('solver_build',case.get('solver_build','official_original'));resourcefix=build=='explicit_resourcefix_build';binary=None;path=None;evidence=None
 if case['model'].startswith('ravaflow'):
  matches=re.findall(r'(?m)^([a-f0-9]{64})\s+(/\S*/r\.avaflow\.main)\s*$',log)
  if matches:binary,path=matches[-1];evidence='sha256sum_in_this_execution_log'
 else:
  p=folder/'xdclaw'
  if p.exists():binary=sha(p);path=str(p);evidence='copied_executed_xdclaw_in_this_run'
 if binary is None:
  snapshot=P/'sources/official_source_identity.json'
  if snapshot.exists():
   verified=json.loads(snapshot.read_text());target='r.avaflow.main' if case['model'].startswith('ravaflow') else 'xdclaw'
   for line in verified.get('binary_hash_stdout','').splitlines():
    z=line.split()
    if len(z)==2 and z[1].endswith('/'+target):binary,path=z;evidence='same_shared_container_binary_snapshot_'+verified['checked_utc']
 command=ex.get('command',[]);addon=[x for x in command if isinstance(x,str) and x.endswith(':/resourcefix:ro')]
```

```
 return {'solver_build':build,'actual_binary_sha256':binary,'actual_binary_path':path,'binary_identity_evidence':evidence,'execution_image_id':ex.get('image_id'),'base_image_id':ex.get('base_image_id',ex.get('image_id')),'container_id':ex.get('container_id'),'readonly_add on_mount':addon[0] if addon else None,'official_source_archive_preserved':True,'executed_solver_source_modified':resourcefix,'source_modification_scope':'45 resource cleanup Segment_release to Segment_close calls only' if resourcefix else 'official fixed native source','physical_equations_modified':False}
def raster(p):
 with rasterio.open(p) as d:a=d.read(1).astype(float);nodata=d.nodata;tr=d.transform
 if nodata is not None:a[a==nodata]=0
 return a,tr
def frame_dclaw(folder,suffix):
 text=(folder/f'fort.t{suffix}').read_text().splitlines();vals=[x.split()[0] for x in text if x.strip()];t=float(vals[0]);ng=int(vals[2]);ghost=int(vals[5]);nq=int(vals[1])
 qlines=[x for x in (folder/f'fort.q{suffix}').read_text().splitlines() if x.strip()]
 if vals[6].lower()=='ascii':
  cursor=0;patches=[]
  for j in range(ng):
   hh=[x.split()[0] for x in qlines[cursor:cursor+8]];cursor+=8;nx=int(hh[2]);ny=int(hh[3]);q=np.array([[float(y.replace('D','E')) for y in x.split()] for x in qlines[cursor:cursor+nx*ny]]).reshape(ny,nx,nq);cursor+=nx*ny;patches.append((q,float(hh[4]),float(hh[5]),float(hh[6]),float(hh[7])))
  return t,patches
 head=[x.split()[0] for x in qlines]
 raw=np.fromfile(folder/f'fort.b{suffix}',dtype='<f8');off=0;patches=[]
 for j in range(ng):
  hh=head[j*8:(j+1)*8];nx=int(hh[2]);ny=int(hh[3]);x0=float(hh[4]);y0=float(hh[5]);dx=float(hh[6]);dy=float(hh[7]);n=nq*(nx+2*ghost)*(ny+2*ghost)
  q=raw[off:off+n].reshape(ny+2*ghost,nx+2*ghost,nq)[ghost:-ghost,ghost:-ghost,:];off+=n;patches.append((q,x0,y0,dx,dy))
 if off!=len(raw):raise ValueError('DClaw binary byte layout mismatch')
 return t,patches

def main():
 runs=[];metrics=[];arrivals=[];hist=[];validation=[];fields=[]
 for folder in sorted((C/'runs').glob('*')):
  if not (folder/'case.json').exists() or not (folder/'execution.json').exists():continue
  case=json.loads((folder/'case.json').read_text());ex=json.loads((folder/'execution.json').read_text())
  if ex.get('log_sha256')!=sha(folder/'run.log'):continue # Active retry log must not inherit previous attempt metadata.
  identity=solver_identity(folder,case,ex);identity['applicability_decision']=applicability().get(case['run_id'],{}).get('decision','required')
  if ex.get('quality_gate'):
   runs.append({'run_id':case['run_id'],'scenario_id':case['scenario_id'],'model':case['model'],'resolution_m':case['resolution_m'],'status':'quarantined_configuration','role':'excluded_configuration','quality_gate':ex['quality_gate'],'image_id':ex.get('image_id')});validation.append({'run_id':case['run_id'],'quality_gate':ex['quality_gate'],'excluded_from_scientific_comparison':True,'reason':'single phase SURFACE2 source bounds risk, original logs/partialfields retained'});continue
  runid=case['run_id'];dx=case['resolution_m'];frames=[];speedframes=[];phaseframes=[];pbframes=[];mframes=[];boundary_flux_rates=[];horizontal_speedframes=[];initial_effective_bed=None;stationary_criterion_speed_final=None;base_change_final=None;source=raster(Path(case['release']))[0]
  status='missing';check={'run_id':runid,'exit_code':ex['returncode'],'case_sha256':sha(folder/'case.json'),'execution_sha256':sha(folder/'execution.json'),'log_sha256':sha(folder/'run.log'),'warnings':[]}
```

```
check['case_matches_execution_record']=ex.get('case_sha256')==check['case_sha256'];check['dem_sha256']=sha(case['dem']);check['release_sha256']=sha(case['release']);check['input_hashes_match_case']=check['dem_sha256']==case.get('dem_sha256') and check['release_sha256']==case.get('release_sha256');check['phase_fraction_basis']='volume';check['initial_phase_volume_fractions']=case['phase_volume_fractions'];check['phase_fractions_sum']=float(sum(case['phase_volume_fractions']))
if case['model'].startswith('ravaflow'):
 ascii_dir=folder/f'{runid}_results'/f'{runid}_ascii';files=folder/f'{runid}_results'/f'{runid}_files';nout=files/f'{runid}_nout1.txt';summ=files/f'{runid}_summary.txt'
 if nout.exists():
  vals=nout.read_text().split();check['solver_success_field']=int(vals[1]);check['nout']=int(vals[0]);check['native_global_cumulative_speed_scalar_m_s']=float(vals[15]);check['reported_outflow_volume_m3']=[float(x) for x in vals[26:29]];check['native_signed_entrainment_phase_volumes_m3']=[float(x) for x in vals[23:26]]
  status='completed' if int(vals[1])==1 and ex['returncode']==0 else 'numerical_failure'
 elif ex['returncode']!=0:status='numerical_failure'
 log=(folder/'run.log').read_text(errors='replace');check['log_contains_ERROR']='ERROR:' in log;check['log_contains_numerical_failure']='NUMERICAL FAILURE' in log;check['pipeline_returncode']=ex['returncode'];check['program_sigsegv_minus11_reported']='return code -11' in log or 'Segmentation fault' in log
 if check['log_contains_ERROR'] or check['log_contains_numerical_failure']:status='numerical_failure'
 # A SIGSEGV before fclose can leave summary unflushed; stdout contains actual tsum.
 times=native_ravaflow_times(summ.read_text() if summ.exists() else '',log)
 check['native_output_time_resolution_seconds']=.1;check['frame_index_times_interval_used']=False;check['frames_without_native_time']=[]
 for f in sorted(ascii_dir.glob(f'{runid}_hflow[0-9][0-9][0-9][0-9].asc')):
  ix=int(f.stem[-4:])
  if ix not in times:check['frames_without_native_time'].append(ix);continue
  h,tr=raster(f);t=times[ix];frames.append((t,h));s=[];ph=[]
  if case['model']=='ravaflow_single':
   vf=ascii_dir/f'{runid}_vflow{ix:04d}.asc';s=raster(vf)[0] if vf.exists() else np.full_like(h,np.nan);ph=[h]
  else:
   for k in range(1,4):
    ph.append(raster(ascii_dir/f'{runid}_hflow{k}{ix:04d}.asc')[0]);vf=ascii_dir/f'{runid}_vflow{k}{ix:04d}.asc';s.append(raster(vf)[0] if vf.exists() else np.full_like(h,np.nan))
   s=np.maximum.reduce(s)
  speedframes.append(s);phaseframes.append(ph)
  hspeed=[];residual_speed=[]
  for phaseindex,tag in enumerate([''] if case['model']=='ravaflow_single' else ['1','2','3']):
   xf=ascii_dir/f'{runid}_vflowx{tag}{ix:04d}.asc';yf=ascii_dir/f'{runid}_vflowy{tag}{ix:04d}.asc'
   if xf.exists() and yf.exists():
    raw_norm=np.hypot(raster(xf)[0],raster(yf)[0]);hspeed.append(np.where(ph[phaseindex]>=.1,raw_norm/np.maximum(ph[phaseindex],1e-30),0));residual_speed.append(np.where(ph[phaseindex]>=1e-6,raw_norm/np.maximum(ph[phaseindex],1e-30),0))
   else:hspeed.append(np.full_like(h,np.nan));residual_speed.append(np.full_like(h,np.nan))
  horizontal_speedframes.append(np.maximum.reduce(hspeed))
  stationary_criterion_speed_final=np.maximum.reduce(residual_speed)
 check['native_output_time_evidence_complete']=not check['frames_without_native_time']
 if check['frames_without_native_time']:status='missing'
 bc=ascii_dir/f'{runid}_basechange_fin.asc'
 if bc.exists():base_change_final=raster(bc)[0]
 pc=files/f'{runid}_paramcomm.txt'
 if pc.exists():
  params=pc.read_text();check['native_paramcomm_path']=str(pc);check['native_paramcomm_sha256']=sha(pc)
```

```
    if case.get('erodible_bed'):check['native_entrainment_parameter_lines']=[line for line in params.splitlines() if 'Entrainment and deposition control' in line or 'Maximum height of entrainment MIXTURE' in line or 'Log 10 of entrainment coefficient' in line];check['native_entrainment_input_slot_verified']='Maximum height of entrainment MIXTURE or PHASE 1 map\terodible_bed' in params and 'Entrainment and deposition control (0=no, 1-4=type of approach)\t1' in params and 'Log 10 of entrainment coefficient\t-7.00' in params
   else:
    ts=sorted(folder.glob('fort.t[0-9][0-9][0-9][0-9]'));lastt=0
    for tf in ts:
     t,patches=frame_dclaw(folder,tf.name[-4:]);lastt=t
     if len(patches)!=1:raise ValueError('Regional configured AMR1 expected one patch')
     q,x0,y0,gdx,gdy=patches[0];q=q[::-1].copy();h=q[:,:,0];tr=rasterio.transform.from_origin(x0,y0+q.shape[0]*gdy,gdx,gdy);frames.append((t,h));v=np.hypot(q[:,:,1],q[:,:,2])/np.maximum(h,1e-30);v[h<.1]=0;speedframes.append(v);horizontal_speedframes.append(v);initial_effective_bed=q[:,:,7]-h if t==0 else initial_effective_bed;phaseframes.append([q[:,:,3],h-q[:,:,3]]);pbframes.append(q[:,:,4]);mframes.append(np.where(h>.1,q[:,:,3]/np.maximum(h,1e-30),np.nan));rho_local=1000+1700*np.where(h>1e-12,q[:,:,3]/np.maximum(h,1e-30),0);boundary_flux_rates.append(float(dx*(np.sum(rho_local[:,-1]*q[:,-1,1])-np.sum(rho_local[:,0]*q[:,0,1])+np.sum(rho_local[0,:]*q[0,:,2])-np.sum(rho_local[-1,:]*q[-1,:,2]))))
    status='completed' if ex['returncode']==0 and lastt>=case['end_seconds']-.01 else ('numerical_failure' if ex['returncode']!=0 else 'missing')
    check['actual_last_time_seconds']=lastt
    stationary_criterion_speed_final=horizontal_speedframes[-1] if horizontal_speedframes else None
   row={'run_id':runid,'scenario_id':case['scenario_id'],'model':case['model'],'resolution_m':dx,'thickness_m':case['release_thickness_m'],'mu_basal':case['mu_basal'],'xi':case['xi'],'status':status,'frames':len(frames),'elapsed_seconds':ex['elapsed_seconds'],'image_id':ex['image_id'],'csurface':case.get('csurface',2) if case['model'].startswith('ravaflow') else None,'official_protocol_version':case.get('official_protocol_version','q4-r3-official-v1'),'role':case.get('evidence_role','conditional_scenario'),**identity};runs.append(row);check.update(identity)
   if not frames:
    validation.append(check)
    for target in case['targets']:arrivals.append({'run_id':runid,'target':target['id'],'status':'numerical_failure' if status=='numerical_failure' else 'missing','arrival_seconds':None,'evaluation_end_time_seconds':None,'source_fields_path':None,'source_fields_sha256':None,'execution_sha256':check['execution_sha256'],'log_sha256':check['log_sha256'],'case_sha256':check['case_sha256'],'arrival_definition':'first saved output at any cell centre within100m, h>=0.1m','initiation_to_release_delay_seconds':None,**identity})
    continue
   if case['model'].startswith('ravaflow'):
    raw_ascii_transform=list(tr)
    with rasterio.open(case['dem']) as executed_dem:
     if executed_dem.shape!=frames[0][1].shape:raise ValueError('Output-vs-executed DEM shape differs; cannot silently remap')
     tr=executed_dem.transform
    check['raw_ascii_header_transform']=raw_ascii_transform;check['evaluated_computational_grid_transform']=list(tr);check['coordinate_rule']='Known regional execution DEM affine; rawASCII usesxllcenter/yllcenter forboundaries, halfcell audit sources/official_ascii_georeference_review.json; rawbytes unchanged'
```

```
hs=np.stack([h for t,h in frames]);times=np.array([t for t,h in frames]);vs=np.stack(speedframes);maxh=hs.max(0);velocity_available=bool(np.isfinite(vs).any());maxv=np.where(hs>=.1,vs,0).max(0) if velocity_available else np.full_like(maxh,np.nan);horizontal=np.stack(horizontal_speedframes);horizontal_available=bool(np.isfinite(horizontal).any());max_horizontal=np.where(hs>=.1,horizontal,0).max(0) if horizontal_available else np.full_like(maxh,np.nan);total=hs.sum((1,2))*dx*dx;phasevol=np.array([[ph.sum()*dx*dx for ph in phases] for phases in phaseframes]);rho=np.array([case.get('bulk_density_kg_m3',2105)] if case['model']=='ravaflow_single' else (case.get('densities_kg_m3',[2700,1850,1000]) if case['model']=='ravaflow_multi' else [2700,1000]));mass=phasevol@rho
boundary_mask=np.zeros(maxh.shape,bool);boundary_mask[[0,-1],:]=True;boundary_mask[:,[0,-1]]=True;boundary_hit=np.any(hs[:,boundary_mask]>=.1,axis=1);first_boundary_time=float(times[np.flatnonzero(boundary_hit)[0]]) if boundary_hit.any() else None
yy,xx=np.indices(maxh.shape);xs=tr.c+(xx+.5)*dx;ys=tr.f-(yy+.5)*dx
for target in case['targets']:
 dist=(xs-target['xy'][0])**2+(ys-target['xy'][1])**2;mask=dist<=target['buffer_m']**2
 if not mask.any():ast='out_of_domain';arr=None
 else:
  hit=np.any(hs[:,mask]>=.1,axis=1);arr=float(times[np.flatnonzero(hit)[0]]) if hit.any() else None;ast='arrival' if arr is not None else ('not_arrived_by_horizon' if status=='completed' else ('missing' if status=='missing' else 'numerical_failure'))
  if check.get('frames_without_native_time'):arr=None;ast='missing'
 arrivals.append({'run_id':runid,'target':target['id'],'status':ast,'arrival_seconds':arr,'earliest_valuation_end_time_seconds':float(times[-1]),'requested_horizon_seconds':case['end_seconds'],'first_boundary_threshold_time_seconds':first_boundary_time,'domain_affected_before_arrival':first_boundary_time is not None and (arr is None or first_boundary_time<arr),'previous_below_threshold_time_seconds':float(times[np.flatnonzero(hit)[0]-1]) if arr is not None and np.flatnonzero(hit)[0]>0 else None,'arrival_definition':'first saved output at any cell centre within100m, h>=0.1m','max_target_depth_m':float(maxh[mask].max()) if mask.any() else None,'initiation_to_release_delay_seconds':None})
edge=np.zeros(maxh.shape,bool);edge[[0,-1],:]=True;edge[:,[0,-1]]=True
chk={'finite':bool(np.isfinite(hs).all() and (not velocity_available or np.isfinite(vs).all())),'velocity_fields_available':velocity_available,'first_boundary_threshold_time_seconds':first_boundary_time,'minimum_depth_m':float(hs.min()),'nonnegative_to_1e-8':bool(hs.min()>=-1e-8),'boundary_depth_ge01':bool(np.any(maxh[edge]>=.1)),'first_output_volume_m3':float(total[0]),'requested_release_volume_m3':case['release_grid_volume_m3'],'initial_volume_relative_difference':float((total[0]-case['release_grid_volume_m3'])/case['release_grid_volume_m3']),'initial_depth_normalized_L1_difference':float(np.sum(np.abs(hs[0]-source))/np.sum(source)) if hs[0].shape==source.shape else None,'initial_supplied_support_cells':int((source>1e-6).sum()),'initial_native_support_cells':int((hs[0]>1e-6).sum()),'initial_depth_vs_supplied_mae_m':float(np.mean(np.abs(hs[0]-source))) if hs[0].shape==source.shape else None,'initial_depth_vs_supplied_max_difference_m':float(np.max(np.abs(hs[0]-source))) if hs[0].shape==source.shape else None,'end_volume_change_fraction':float(total[-1]/total[0]-1),'end_total_mass_change_fraction':float(mass[-1]/mass[0]-1),'conservation_scope':'DClaw volume may evolve through dilation; total solid+fluid mass diagnostic; open boundary loss is not residual correction','actual_last_time_seconds':float(times[-1])};check.update(chk);check['state_field_horizon_complete']=bool(times[-1]>=case['end_seconds']-.01);check['native_integration_success_flag']=check.get('solver_success_field');check['pipeline_status']='completed' if ex['returncode']==0 else 'nonzero_program_or_postprocessing_failure'
if case['model'].startswith('ravaflow'):
```

```
  core=np.zeros(maxh.shape,bool);core[3:-3,3:-3]=True;corevol=np.array([ph[core].sum()*dx*dx for ph in phaseframes[-1]]);outvol=np.array(check.get('reported_outflow_volume_m3',[0,0,0])[:len(rho)]);signed_entr=np.array(check.get('native_signed_entrainment_phase_volumes_m3',[0,0,0])[:len(rho)]);added_mass=float(-signed_entr@rho);check['native_active_interior_final_phase_volumes_m3']=corevol.tolist();check['native_edge_store_outflow_phase_volumes_m3']=outvol.tolist();check['native_entrained_added_mass_kg']=added_mass;check['native_mass_budget_relative_residual']=float((corevol@rho+outvol@rho-mass[0]-added_mass)/mass[0]);check['native_budget_definition']='Mfinal active interior excluding3edge rings + Mofficial edgestore - Minitial - Mentrained; Mentrained=-signednativeVE*density; no phase transformation; notdoublecountingedge rasters';check['native_mass_budget_estimate_representation']='activecore andinitialphasevolumes fromsavedASCII, roundedheight output; separate nativeinternal unscaledsummary budget whenavailable'
  native_summary_volumes=[]
  if summ.exists():
   check['native_summary_path']=str(summ);check['native_summary_sha256']=sha(summ)
   for line in summ.read_text().splitlines():
    z=line.split()
    if z and z[0].isdigit():
     try:
      vv=[float(z[7])] if case['model']=='ravaflow_single' else [float(x) for x in z[11:14]]
      if len(vv)==len(rho):native_summary_volumes.append((int(z[0]),float(z[4]),np.array(vv)))
     except (ValueError,IndexError):pass
  if native_summary_volumes and native_summary_volumes[0][0]==0 and abs(native_summary_volumes[-1][1]-times[-1])<.051:
   nv0=native_summary_volumes[0][2];nvf=native_summary_volumes[-1][2];check['native_internal_summary_initial_phase_volumes_m3']=nv0.tolist();check['native_internal_summary_final_phase_volumes_m3']=nvf.tolist();check['native_internal_summary_mass_budget_relative_residual']=float((nvf@rho+outvol@rho-nv0@rho-added_mass)/(nv0@rho));check['native_internal_summary_volume_units']='m3 fromunscaled *_summary.txt, notstdout volumes/1000; sourceC4808-4820/8323-8347; excludes cstopped/dry states per7619'
  if case.get('erodible_bed'):
   check['erodible_bed_sha256']=sha(case['erodible_bed']);check['erodible_bed_hash_matches_case']=check['erodible_bed_sha256']==case['erodible_bed_sha256'];check['available_bed_volume_m3']=case['max_available_bed_volume_m3'];check['native_entrained_added_volume_m3']=float(-signed_entr.sum());check['remaining_bed_capacity_m3']=float(case['max_available_bed_volume_m3']+signed_entr.sum());check['entrainment_within_prescribed_volume_capacity']=bool(-signed_entr.sum()<=case['max_available_bed_volume_m3']+.5)
   bed=raster(case['erodible_bed'])[0];check['native_bed_import_all_domain_valid']=f'n={bed.size}' in log and 'null_cells=0' in log;check['native_bed_import_min0_max1']='min=0' in log and 'max=1' in log;check['input_bed_grid_volume_m3']=float(bed.sum()*dx*dx)
   if base_change_final is not None:check['basechange_final_min_m']=float(base_change_final.min());check['basechange_final_max_m']=float(base_change_final.max());check['basechange_grid_signed_volume_m3']=float(base_change_final.sum()*dx*dx);check['basechange_outside_erodible_mask_max_abs_m']=float(np.abs(base_change_final[bed==0]).max());check['basechange_cellwise_within_bed_capacity']=bool(np.all(base_change_final>=-bed-.00101));check['remaining_bed_from_saved_basechange_m3']=float(np.maximum(bed+base_change_final,0).sum()*dx*dx)
 if initial_effective_bed is not None:
  supplied_bed=raster(Path(case['dem']))[0];delta_bed=initial_effective_bed-supplied_bed;check['initial_effective_bed_vs_input_mae_m']=float(np.mean(np.abs(delta_bed)));check['initial_effective_bed_vs_input_rmse_m']=float(np.sqrt(np.mean(delta_bed**2)));check['initial_effective_bed_vs_input_max_abs_m']=float(np.max(np.abs(delta_bed)))
 if pbframes:
```

```
 pb=np.stack(pbframes);m=np.stack(mframes);wet=hs>=.1;overburden=(1000+1700*np.nan_to_num(m))*9.81*hs;check['pressure_all_finite']=bool(np.isfinite(pb).all());check['wet_pressure_min_Pa']=float(pb[wet].min());check['wet_pressure_max_Pa']=float(pb[wet].max());check['wet_solid_fraction_min']=float(np.nanmin(m));check['wet_solid_fraction_max']=float(np.nanmax(m));check['wet_max_pressure_over_total_overburden']=float((pb/np.maximum(overburden,1e-30))[wet].max());check['wet_min_effective_stress_Pa']=float((overburden-pb)[wet].min());integrated=float(np.trapezoid(boundary_flux_rates,times));check['estimated_boundary_mass_out_kg']=integrated;check['estimated_boundary_adjusted_mass_residual_fraction']=float((mass[-1]+integrated-mass[0])/mass[0]);check['boundary_estimate_scope']='trapezoidal integration of saved boundary-cell rho*hu/rho*hv; diagnostic approximation, not native timestep flux budget';check['boundary_flux_sampling_sensitivity_residual_fractions']={str(step*case['output_interval_seconds']):float((mass[-1]+np.trapezoid(np.array(boundary_flux_rates)[list(range(0,len(times),step))+([] if (len(times)-1)%step==0 else [len(times)-1])],times[list(range(0,len(times),step))+([] if (len(times)-1)%step==0 else [len(times)-1])])-mass[0])/mass[0]) for step in [1,2,3]};check['native_mass_conservation_claim']=False
phase_stack=np.array(phaseframes);phase_sum_difference=phase_stack.sum(axis=1)-hs;check['phase_depth_sum_vs_total_max_abs_m']=float(np.max(np.abs(phase_sum_difference)));check['phase_depth_sum_vs_total_normalized_L1']=float(np.sum(np.abs(phase_sum_difference))/np.sum(hs));check['phase_depth_all_finite']=bool(np.isfinite(phase_stack).all());check['phase_depth_min_m']=float(phase_stack.min())
empty_phases=[k for k,name in enumerate(case.get('phase_material_names',[])) if name=='empty']
if empty_phases:check['prescribed_empty_phase_max_abs_depth_m']={str(k+1):float(np.abs(phase_stack[:,k]).max()) for k in empty_phases};check['prescribed_empty_phase_max_abs_volume_m3']={str(k+1):float(np.abs(phasevol[:,k]).max()) for k in empty_phases}
validation.append(check)
native_h=None;native_v=None
if case['model'].startswith('ravaflow'):
 nf=ascii_dir/f'{runid}_hflow_max.asc'
 if nf.exists():native_h=raster(nf)[0]
 vfiles=[ascii_dir/f'{runid}_vflow_max.asc'] if case['model']=='ravaflow_single' else [ascii_dir/f'{runid}_vflow{k}_max.asc' for k in range(1,4)]
 if all(f.exists() for f in vfiles):native_v=np.maximum.reduce([raster(f)[0] for f in vfiles])
metrics.append({'run_id':runid,'native_cumulative_max_depth_m':float(native_h.max()) if native_h is not None else None,'native_cumulative_max_speed_m_s':float(native_v.max()) if native_v is not None else None,'model':case['model'],'resolution_m':dx,'status':status,'max_depth_over_saved_outputs_m':float(maxh.max()),'max_speed_over_saved_outputs_m_s':float(maxv.max()) if velocity_available else None,'speed_definition':'native terrain-corrected scalar' if case['model'].startswith('ravaflow') else 'horizontalE/N hypot(hu,hv)/h withbed_normal0',
'max_horizontal_speed_over_saved_outputs_m_s':float(max_horizontal.max()) if horizontal_available else None,'native_global_cumulative_speed_scalar_m_s':check.get('native_global_cumulative_speed_scalar_m_s'),'peak_sampling_definition':'saved outputs, nominal20s with actual native times','state_field_horizon_complete':check['state_field_horizon_complete'],'pipeline_returncode':ex['returncode'],'csurface':case.get('csurface',2) if case['model'].startswith('ravaflow') else None,'impact_area_m2':int((maxh>=.1).sum())*dx*dx,'end_volume_m3':float(total[-1]),'end_mass_kg':float(mass[-1]),'mass_change_fraction':chk['end_total_mass_change_fraction'],'native_mass_budget_relative_residual':check.get('native_mass_budget_relative_residual'),'native_internal_summary_mass_budget_relative_residual':check.get('native_internal_summary_mass_budget_relative_residual'),'estimated_boundary_adjusted_mass_residual_fraction':check.get('estimated_boundary_adjusted_mass_residual_fraction'),'boundary_touched':chk['boundary_depth_ge01'],'last_time_s':float(times[-1])})
```

```
 for k,t in enumerate(times):hist.append({'run_id':runid,'time_s':float(t),'volume_m3':float(total[k]),'mass_kg':float(mass[k]),'max_depth_m':float(hs[k].max()),'max_speed_m_s':float(np.where(hs[k]>=.1,vs[k],0).max()) if velocity_available else None,'max_horizontal_speed_m_s':float(np.where(hs[k]>=.1,horizontal[k],0).max()) if horizontal_available else None,'phase_volumes_m3':json.dumps(phasevol[k].tolist()),'mean_wet_pressure_Pa':float(np.mean(pbframes[k][hs[k]>=.1])) if pbframes and np.any(hs[k]>=.1) else None,'mean_solid_volume_fraction':float(np.nanmean(mframes[k])) if mframes else None})
 output=folder/('summary_fields_'+FIELDS_SCHEMA+'_'+PARSER_SHA256[:12]+'.npz')
 if not output.exists():np.savez_compressed(output,time_s=times,depth_max=maxh,speed_max=maxv,depth_max_native_cumulative=np.array([]) if native_h is None else native_h,speed_max_native_cumulative=np.array([]) if native_v is None else native_v,depth_final=hs[-1],depth_frames=hs.astype('float32'),phase_depth_frames=np.array(phaseframes,dtype='float32'),phase_material_names=np.array(case.get('phase_material_names',['equivalent_bulk'] if case['model']=='ravaflow_single' else ['solid','fluid'] if case['model']=='dclaw' else ['native_phase_1','native_phase_2','native_phase_3'])),speed_frames=vs.astype('float32'),horizontal_speed_frames=horizontal.astype('float32'),stationary_criterion_speed_final=np.array([]) if stationary_criterion_speed_final is None else stationary_criterion_speed_final,base_change_final_m=np.array([]) if base_change_final is None else base_change_final,initial_effective_bed=np.array([]) if initial_effective_bed is None else initial_effective_bed,volume_m3=total,mass_kg=mass,phase_volume_m3=phasevol,transform=np.array(tr),boundary_mass_flux_kg_s=np.array(boundary_flux_rates),pressure_frames=np.array(pbframes,dtype='float32'),solid_fraction_frames=np.array(mframes,dtype='float32'))
 fieldhash=sha(output);exec_hash=sha(folder/'execution.json');loghash=sha(folder/'run.log');c
 heck.update({'source_fields_path':str(output),'source_fields_sha256':fieldhash,'executio
 n_sha256':exec_hash,'log_sha256':loghash,'parser_sha256_at_start':PARSER_SHA256,'parser_
 source_snapshot_path':str(PARSER_ARCHIVE),'fields_schema':FIELDS_SCHEMA})
 for linked in runs+metrics+arrivals+hist:
  if linked.get('run_id')==runid:linked.update({'source_fields_path':str(output),'source_fiel
  ds_sha256':fieldhash,'execution_sha256':exec_hash,'log_sha256':loghash,'case_sha256':sh
  a(folder/'case.json'),'parser_sha256_at_start':PARSER_SHA256,'parser_source_snapshot_pa
  th':str(PARSER_ARCHIVE),'fields_schema':FIELDS_SCHEMA,**identity})
 fields.append({'run_id':runid,'path':str(output),'bytes':output.stat().st_size,'sha256':fiel
 dhash,'role':'actual_official_process_fields','case_sha256':sha(folder/'case.json'),'exe
 cution_sha256':exec_hash,'log_sha256':loghash,'image_id':ex['image_id'],'time_end_second
 s':float(times[-1]),'native_time_evidence':'flushed summary or native stdout tsum; no
 frame index time inference','parser_sha256_at_start':PARSER_SHA256,'parser_source_snapsh
 ot_path':str(PARSER_ARCHIVE),'fields_schema':FIELDS_SCHEMA,**identity})
def writecsv(name,rows):
 if not rows:return
 names=list(dict.fromkeys(k for d in rows for k in d))
 with (P/'results'/name).open('w',newline='') as
  f:w=csv.DictWriter(f,fieldnames=names);w.writeheader();w.writerows(rows)
writecsv('official_runs.csv',runs);writecsv('official_metrics.csv',metrics);writecsv('officia
 l_arrivals.csv',arrivals);writecsv('official_histories.csv',hist)
(P/'results/official_validation.json').write_text(json.dumps({'analysed_utc':datetime.datetim
 e.now(datetime.timezone.utc).isoformat(),'checks':validation,'independent_local_event_val
 idation_count':0,'official_source_archive_preserved':True,'executed_solver_modification_i
 dentity':'reported per run; resourcefix binaries explicitly
 modified'},indent=2)+'\n');(P/'results/official_fields.json').write_text(json.dumps({'fie
 lds':fields},indent=2)+'\n')
# Common axes/maps for baseline competitors, actual file hashes later recorded by root visual
 review.
baselines=[f for f in fields if f['run_id'] in
 ['copdem_120_single_ref__sf0','copdem_120_multi_ref__sf0','copdem_120_dclaw_ref']]
if baselines:
 fig,ax=plt.subplots(1,len(baselines),figsize=(5*len(baselines),4),layout='constrained');ax=n
  p.atleast_1d(ax)
 for axis,f in zip(ax,baselines):
```

```
    d=np.load(f['path']);h=d['depth_max'];im=axis.imshow(np.where(h>=.1,np.log10(np.maximum(h,1e-30)),np.nan),origin='upper',vmin=-1,vmax=2,cmap='magma');axis.set_title(f['run_id'].replace('copdem_120_',''));axis.set_xlabel('Grid column, east');axis.set_ylabel('Grid row, south')
  fig.colorbar(im,ax=ax.tolist(),label='log10(max depth / m)');fig.suptitle('Official solvers: conditional source and terrain; 120 m');fig.savefig(P/'figures/official_max_depth.png',dpi=160);plt.close(fig)
 if hist and baselines:
  fig,ax=plt.subplots(1,2,figsize=(11,4),layout='constrained')
  for f in baselines:
   hh=[x for x in hist if x['run_id']==f['run_id']];ax[0].plot([x['time_s'] for x in hh],[x['max_horizontal_speed_m_s'] for x in hh],label=f['run_id'].replace('copdem_120_',''));m0=hh[0]['mass_kg'];ax[1].plot([x['time_s'] for x in hh],[x['mass_kg']/m0 for x in hh],label=f['run_id'].replace('copdem_120_',''))
  ax[0].set(xlabel='Seconds after sizeable release',ylabel='Peak wet horizontal speed (m/s)');ax[1].set(xlabel='Seconds after sizeable release',ylabel='Domain mass / initial mass');ax[0].legend();ax[1].legend();fig.suptitle('Saved native states; domain mass is not a boundary-corrected budget');fig.savefig(P/'figures/official_process_histories.png',dpi=160);plt.close(fig)
 print(json.dumps({'runs':len(runs),'completed':sum(x['status']=='completed' for x in runs),'fields':len(fields)}))
if __name__=='__main__':main()
```

程序 46：official_targets.py

来源：track_b/research/q4/round3/scripts/official_targets.py
SHA256: c54aada40316dd3eedca552ce6048f59749475c958b9466df32ecaa53a6e3311

```
#!/usr/bin/env python3
"""All preregistered independent OSM candidate centroids, postprocess existing fields."""
from pathlib import Path
import csv,json,hashlib,datetime
import numpy as np
P=Path(__file__).resolve().parents[1]
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
def main():
 targetfile=P/'results/spatial_targets_v2.csv';protocol=P/'config/additional_target_protocol_v2.json'
 if not targetfile.exists():return
 targets=list(csv.DictReader(targetfile.open()));rows=[]
 fields=json.loads((P/'results/official_fields.json').read_text())['fields'];runs={d['run_id']:d for d in csv.DictReader((P/'results/official_runs.csv').open())}
 for item in fields:
  rid=item['run_id']
  if runs[rid].get('role')=='numerical_benchmark':continue
  data=np.load(item['path']);h=data['depth_frames'];times=data['time_s'];a=data['transform'];dx=a[0];dy=-a[4];yy,xx=np.indices(h.shape[1:]);xs=a[2]+(xx+.5)*dx;ys=a[5]-(yy+.5)*dy
  for target in targets:
   mask=(xs-float(target['x_m']))**2+(ys-float(target['y_m']))**2<=100**2;hit=np.any(h[:,mask]>=.1,axis=1) if mask.any() else np.zeros(len(times),bool);idx=np.flatnonzero(hit);status='out_of_domain' if not mask.any() else ('arrival' if len(idx) else ('not_arrived_by_horizon' if runs[rid]['status']=='completed' else 'numerical_failure'));first=int(idx[0]) if len(idx) else None
   rows.append({'run_id':rid,'target_id':target['target_id'],'entity_id':target['entity_id'],'target_kind':target['target_kind'],'geometry_version':target['geometry_version'],'evaluation_role':'postprocess_existing_solver_fields','arrival_definition':'all preregistered representative centroids,100m buffer,first saved output h>=0.1m; not exact structure intersection','status':status,'arrival_time_seconds':float(times[first]) if first is not None else None,'evaluation_end_time_seconds':float(times[-1]),'previous_below_threshold_time_seconds':float(times[first-1]) if first is not None and first>0 else None,'initiation_to_release_delay_seconds':None,'max_depth_m':float(h[:,mask].max()) if mask.any() else None,'target_observation_time_utc':target['observed_time_utc'] or None,'target_position_uncertainty_m':target['position_uncertainty_m'] or None,'source_fields_path':item['path'],'source_fields_sha256':item['sha256'],'execution_sha256':item['execution_sha256'],'log_sha256':item['log_sha256'],'case_sha256':item['case_sha256'],'solver_build':item['solver_build'],'actual_binary_sha256':item['actual_binary_sha256']})
 with (P/'results/official_target_arrivals_v2.csv').open('w',newline='') as f:
  if rows:w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
 (P/'results/official_target_postprocess.json').write_text(json.dumps({'utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'targets_table_sha256':sha(targetfile),'protocol_sha256':sha(protocol),'all_preregistered_targets':len(targets),'rows':len(rows),'no_solver_rerun':True,'no_primary_target_replacement':True},indent=2)+'\n')
 print(len(rows))
if __name__=='__main__':main()
```

程序 47：integrate_comparison.py

来源：track_b/research/q4/round3/scripts/integrate_comparison.py
SHA256: e7e475ffbaf4fc6216cf4c1ce7a50abcf5c069408507d5ed7ebd45cba9b98eeb

```
"""Integrate actual Q4 records without rerunning solvers or changing frozen packets.

Default writes provisional root tables. --freeze requires settled current plans,
stable source hashes, and a NEW version directory; failure records remain visible.
"""
from __future__ import annotations
import argparse, collections, csv, hashlib, io, itertools, json, math, re, shutil
from datetime import datetime, timezone
from pathlib import Path

ROOT=Path(__file__).resolve().parents[5]
R=ROOT/'track_b/research/q4/round3'
C=Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official')
INPUT_VERSION={'baseline_commit':'904d7257b899b9f71d1ac9e0eab7479e459eab52',
               'baseline_manifest_sha256':'fd755fab38df285a67c986026859be5f635a314aa3f79f2d247
               ↪ a2626246c3a53'}
ACTIVE=['official_cases_copdem_v2.json','official_cases_q2mapped_v2.json',
        'official_cases_q2fullmask_v2.json','official_cases_extended.json',
        'official_cases_physics_v2.json','official_cases_output_flag.json',
        'official_cases_resourcefix.json','official_cases_composite.json']
FULLMASK_PARTIAL_IDS={'q2full_q2r3_p0000_S_linear__geomv2_60m__sf0',
    'q2full_q2r3_p0036_S_T_linear__geomv2_60m__sf0',
    'q2full_q2r3_p0036_S_T_linear__geomv2_120m__sf0'}
COMPACT_DELTA_IDS={'q2map60_q2r3_p0072_S_H_linear__sf0',
    'q2map60_q2r3_p0255_S_T_H_linear__sf0'}

def utc():return datetime.now(timezone.utc).isoformat()
def digest(data):return hashlib.sha256(data).hexdigest()
def file_digest(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as f:
        for block in iter(lambda:f.read(1024*1024),b''):h.update(block)
    return h.hexdigest()
def number(x):
    try:
        v=float(x)
        return v if math.isfinite(v) else None
    except (TypeError,ValueError):return None
def truth(x):
    if isinstance(x,bool):return x
    if x is None or x=='':return None
    return str(x).lower() in ['true','1','yes']
def canonical(x):return json.dumps(x,ensure_ascii=False,sort_keys=True,allow_nan=False)
def write_json(path,obj):
    tmp=path.with_name(path.name+'.tmp');tmp.write_text(json.dumps(obj,ensure_ascii=False,inde
    ↪ nt=2,allow_nan=False)+'\n');tmp.replace(path)
def dump_csv(path,rows):
    names=list(dict.fromkeys(k for row in rows for k in row))
    tmp=path.with_name(path.name+'.tmp')
    with tmp.open('w',newline='') as f:
        w=csv.DictWriter(f,fieldnames=names);w.writeheader()
        for row in rows:w.writerow({k:canonical(v) if isinstance(v,(dict,list)) else v for k,v
        ↪ in row.items()})
    tmp.replace(path)

class Inputs:
    def __init__(self):self.bytes={};self.records={};self.warnings=[]
```

```
    def read(self,path,role,required=True):
        path=Path(path)
        if not path.exists():
            if required:raise FileNotFoundError(path)
            return None
        key=str(path.resolve())
        if key not in self.bytes:
            b=path.read_bytes();self.bytes[key]=b
            self.records[key]={'path':str(path.relative_to(ROOT)) if path.is_relative_to(ROOT)
            ↪  else str(path),
                 'bytes':len(b),'sha256':digest(b),'role':role,'read_at_utc':utc()}
        return self.bytes[key]
    def js(self,path,role,required=True):
        b=self.read(path,role,required);return json.loads(b) if b is not None else None
    def csv(self,path,role,required=True):
        b=self.read(path,role,required);return
        ↪  list(csv.DictReader(io.StringIO(b.decode('utf-8-sig')))) if b is not None else []
    def check_bound(self,rec):
        if not rec:return False
        p=Path(rec['path']);p=p if p.is_absolute() else ROOT/p
        if not p.exists():return False
        key=str(p.resolve())
        if key not in self.records:
            self.records[key]={'path':str(p.relative_to(ROOT)) if p.is_relative_to(ROOT) else
            ↪  str(p),'bytes':p.stat().st_size,'sha256':file_digest(p),'role':rec.get('role',
            ↪  'indexed process field'),'read_at_utc':utc(),'hash_operation_scope':'complete
            ↪  byte stream hashed in 1MiB blocks; later array reads for independent
            ↪  target/final residual checks are recorded separately'}
        actual=self.records[key]
        return actual['bytes']==rec['bytes'] and actual['sha256']==rec['sha256']
    def note_array_read(self,path,purpose):
        key=str(Path(path).resolve())
        if key not in self.records:raise RuntimeError('Array input must first be byte-hash
        ↪  bound: '+key)
        operations=self.records[key].setdefault('array_read_operations',[])
        if purpose not in operations:operations.append(purpose)
    def changed(self):
        changed=[]
        for key,rec in self.records.items():
            p=Path(key)
            if not p.exists() or file_digest(p)!=rec['sha256']:changed.append(str(p))
        return changed

def stratum(role):
    return 'numerical_benchmark' if 'benchmark' in str(role) else 'conditional_scenario'

def common_context(runs,arrivals):
    return {'event_id':'B20260826_Chhochen',
        'physical_phase':'bulk_release_propagation_and_target_arrival',
        'time_utc':None,'time_uncertainty':None,'available_time':None,
        'forecast_reference_time':None,'issue_time':None,'valid_time':None,
        'null_time_reasons':{'time_utc':'Absolute UTC of modeled bulk release is unknown;
        ↪  scenario-relative travel times do not supply it',
            'time_uncertainty':'No independently audited absolute release-time uncertainty is
            ↪  available',
            'available_time':'Scenario evidence has no established historical forecast
            ↪  information-availability cutoff',
            'forecast_reference_time':'Not applicable: these are conditional propagation
            ↪  scenarios',
            'issue_time':'Not applicable: this package is not an issued forecast',
            'valid_time':'Not applicable: no absolute forecast-valid timestamp is
            ↪  established'},
        'time_origin':'bulk_release','initiation_to_release_delay_seconds':None,
```

```
        'space_object':{'source_geometry_versions':sorted({x['geometry_version'] for x in runs if x.get('geometry_version')}),
            'target_geometry_versions':sorted({x['target_geometry_version'] for x in arrivals if x.get('target_geometry_version')}),
            'crs':'EPSG:32645','crs_code':32645,'axis_order':['easting','northing'],
            'coordinate_unit':'m','position_uncertainty_m':None,
            'position_uncertainty_reason':'Candidate source/target geometry lacks independently quantified positional error',
            'target_detection_geometry':'registered cell centres inside 100m buffers; entity_id distinguishes five OSM representatives from three candidate objects'},
        'variable_semantics':{'time':{'unit':'s','reference':'relative to modeled bulk release',
                'detection_interval':'previous saved output below threshold to first above; not numerical solver-step timing'},
            'depth':{'unit':'m','definition':'model flow depth; native representation and initial depth/terrain differences retained per run'},
            'horizontal_speed':{'unit':'m/s','definition':'horizontal east/north magnitude or explicitly marked along-path equivalent'},
            'native_scalar_speed':{'unit':'m/s','definition':'ravaflow terrain-corrected scalar, retained separately from horizontal speed'},
            'release_volume':{'unit':'m3','definition':'actual release-grid integral, checked against the explicitly delivered parent mapping and native t0 representation where present',
                'legacy_reference_field':'expected_reference_grid_volume_m3 is an inherited standard h15 baseline reference, not the expected release volume for every thickness or Q2-mapped scenario'},
            'stationary_residual':{'depth_threshold_m':.1,'horizontal_speed_threshold_m_s':.1,
                'definition':'last saved-state h>=0.1m and horizontal speed<0.1m/s; multiphase uses maximum over phases with h_phase>=1e-6m',
                'interpretation':'finite-time stationary residual, not permanent or observed deposit; missing conservative phase velocity remains unavailable'},
            'quality_control':['input/case/execution/log/field byte-hash binding','finite and nonnegative saved states',
                'actual native saved times and requested horizon distinguished','arrival threshold h>=0.1m within registered 100m buffers',
                'sampling definitions, boundary evidence and native mass-budget limitations retained'],
            'qualification':'QC gates do not establish predictive or mass-conservation qualification'},
        'data_role':'scenario','evidence_status':'candidate','row_evaluation_stratum_is_authoritative':True,
        'scenario_lineage_semantics':'Explicit upstream Q2 model/parameter/base scenario/mapping are retained where delivered; missing upstream fields mean unknown Q2 parent and are never inferred from a generic conditional scenario',
        'scientific_acceptance_decision_version':next((x.get('scientific_acceptance_decision_version') for x in runs if x.get('scientific_acceptance_decision_version')),None),
        'arrival_envelope_semantics_version':next((x.get('arrival_envelope_semantics_version') for x in runs if x.get('arrival_envelope_semantics_version')),None),
        'scientifically_qualified_for_prediction':False,'regional_accuracy_qualified':False,'mass_conservation_qualified':False,
        'input_version':dict(INPUT_VERSION),'human_scientific_adoption':'pending_team_review',
        'statistical_weights':None,'independent_local_event_validation_count':0}

def attach_quality_contract(ins,runs,metrics,arrivals):
    audit=ins.js(R/'results/dclaw_mass_audit.json','independent raw-state DClaw mass audit',False)
    ins.read(R/'sources/dclaw_mass_source_audit.json','fixed-source DClaw mass audit provenance',False)
    audit_by={x['run_id']:x for x in (audit or {}).get('runs',[])}
    by={x['run_id']:x for x in runs}
    for row in runs:
        row['scientifically_qualified_for_prediction']=False
```

```
        row['mass_conservation_qualified']=False
        row['mass_conservation_qualification_status']='unresolved_native_flux_source_qfix_ledger' if row.get('model_id')=='dclaw' else 'not_demonstrated_as_regional_physical_qualification'
        row['passes_execution_and_time_gates']=bool(row.get('execution_status')=='completed' and row.get('state_horizon_complete') and row.get('fields_hash_matches_index'))
    for row in arrivals:
        run=by[row['run_id']]
        row['passes_execution_time_boundary_gates']=bool(run['passes_execution_and_time_gates'] and row.get('source_snapshot_stable') and not row.get('domain_affected_before_arrival') and row.get('arrival_status') in ['arrived','not_arrived_by_horizon'] and row.get('formal_requested_horizon_status') in ['arrived_by_requested_horizon','not_arrived_at_saved_outputs_through_requested_horizon'])
        row['conditional_diagnostic_range_eligible']=bool(row.get('include_in_arrival_envelope'))
        row['scientifically_qualified_for_prediction']=False
        row['mass_conservation_qualified']=False
        row['mass_conservation_qualification_status']=run['mass_conservation_qualification_status']
        row['arrival_envelope_gate_scope']='conditional diagnostic eligibility only; execution/time/field/boundary gates do not imply numerical, mass-conservation or predictive qualification'
    for row in runs:
        row['conditional_diagnostic_range_eligible']=any(x['conditional_diagnostic_range_eligible'] for x in arrivals if x['run_id']==row['run_id'])
    for row in metrics:
        run=by[row['run_id']]
        for key in ['scientifically_qualified_for_prediction','mass_conservation_qualified','mass_conservation_qualification_status','conditional_diagnostic_range_eligible']:
            row[key]=run[key]
        if row['run_id'] in audit_by:
            evidence=audit_by[row['run_id']]
            row['raw_state_mass_audit_reference']='results/dclaw_mass_audit.json'
            row['raw_state_mass_audit_final_adjusted_residual_fraction']=evidence['estimated_boundary_adjusted_final_residual_fraction']
            row['raw_state_mass_audit_causal_attribution']=(audit or {}).get('causal_attribution')
            row['raw_state_mass_audit_selected_saved_states']=[{k:x.get(k) for k in ['time_s','mass_change_fraction','boundary_h_max_m','boundary_abs_hu_hv_max_m2_s']} for x in evidence['rows'] if x['time_s'] in [20.,180.]]
    return audit

def apply_q2_compact_quality(ins,runs,metrics,arrivals):
    review=ins.js(R/'config/q2_compact_quality_review_04.json','registered compact-source numerical qualification review',False)
    if not review:return None
    evidence_path=ROOT/review['evidence']['path'];evidence=ins.js(evidence_path,'independently crosschecked Q2 compact ASCII/native budget evidence')
    if digest(ins.bytes[str(evidence_path.resolve())])!=review['evidence']['sha256']:raise RuntimeError('Compact-source quality evidence hash mismatch')
    evidence_by={x['run_id']:x for x in evidence['runs']};by={x['run_id']:x for x in runs}
    for i,rid in enumerate(review['run_ids']):
        if rid not in by:continue
        run=by[rid];item=evidence_by[rid];bound=item['bound_fields']
        binding_ok=ins.check_bound(bound) and bound.get('execution_sha256')==run.get('execution_sha256') and bound.get('log_sha256')==run['raw_execution_json'].get('log_sha256')
        run.update(include_in_arrival_envelope=False,conditional_diagnostic_range_eligible=False,
            scientifically_qualified_for_prediction=False,mass_conservation_qualified=False,
            mass_conservation_qualification_status='unresolved_compact_source_budget_discrepancy',
```

```
            numerical_qualification='unqualified_compact_source_underresolution_and_budget_dis
            ↪ crepancy',
            source_quality_review_version=review['version'],source_quality_evidence_binding_ma
            ↪ tches=binding_ok,
            source_support_cells=review['source_support_cells'][i],source_equivalent_full_cell
            ↪ s=review['source_equivalent_full_cells'][i],
            source_quality_reason=review['quality_reason'])
    for row in metrics:
        if row['run_id'] not in evidence_by:continue
        run=by[row['run_id']];item=evidence_by[row['run_id']]
        for key in ['numerical_qualification','source_quality_review_version','source_quality_
        ↪ evidence_binding_matches','source_support_cells','source_equivalent_full_cells','s
        ↪ ource_quality_reason','mass_conservation_qualification_status']:
            row[key]=run[key]
        row.update(include_in_arrival_envelope=False,conditional_diagnostic_range_eligible=Fal
        ↪ se,
            scientifically_qualified_for_prediction=False,mass_conservation_qualified=False,
            ascii_raster_source_adjusted_budget_relative_residual=item['ascii_raster_source_ad
            ↪ justed_budget_relative_residual'],
            native_internal_summary_mass_budget_relative_residual=item['native_internal_summar
            ↪ y_mass_budget_relative_residual'],
            budget_representations_are_separate=True)
    for row in arrivals:
        if row['run_id'] not in evidence_by:continue
        run=by[row['run_id']]
        for key in ['numerical_qualification','source_quality_review_version','source_quality_
        ↪ reason','source_support_cells','source_equivalent_full_cells','mass_conservation_q
        ↪ ualification_status']:
            row[key]=run[key]
        row.update(include_in_arrival_envelope=False,conditional_diagnostic_range_eligible=Fal
        ↪ se,
            scientifically_qualified_for_prediction=False,mass_conservation_qualified=False)
    return review

def load_scientific_acceptance(ins):
    path=R/'config/scientific_acceptance_decision_05.json'
    decision=ins.js(path,'explicit scientific acceptance and output semantics decision05')
    if digest(ins.bytes[str(path.resolve())])!='1619328370807cbe78be707918993fa83031621e39b469
    ↪ c4408dd689282b3674':
        raise RuntimeError('Adopted decision05 bytes changed; a new acceptance decision must be
        ↪ explicitly adopted')
    rec=decision['evidence'];p=ROOT/rec['path'];evidence=ins.js(p,'scientific qualification
    ↪ checkpoint evidence05; not an acceptance threshold')
    if len(ins.bytes[str(p.resolve())])!=rec['bytes'] or
    ↪ digest(ins.bytes[str(p.resolve())])!=rec['sha256']:
        raise RuntimeError('Scientific acceptance evidence05 bytes mismatch')
    for item in evidence.get('fixed_evidence',[]):
        if not ins.check_bound({**item,'role':'fixed qualification counterevidence bound by
        ↪ decision05'}):
            raise RuntimeError('Decision05 fixed evidence mismatch: '+item['path'])
        p=ROOT/item['path']
        if p.stat().st_size<=256*1024:ins.read(p,'fixed qualification counterevidence text
        ↪ snapshot')
    for row in evidence.get('metrics_at_checkpoint',[]):
        p=Path(row['source_fields_path'])
        if not p.exists() or not ins.check_bound({'path':str(p),'bytes':p.stat().st_size,'sha2
        ↪ 56':row['source_fields_sha256'],'role':'immutable process fields referenced at
        ↪ qualification checkpoint05'}):
            raise RuntimeError('Decision05 checkpoint process field mismatch: '+str(p))
    return decision

def apply_scientific_acceptance(decision,runs,metrics,arrivals):
```

```
    """Scientific acceptance is opt-in; raw diagnostic gates never grant it."""
    by={x['run_id']:x for x in runs};accepted=set(decision['accepted_arrival_envelope_run_ids'])
    semantics='q4-r3-arrival-envelope-accepted-runs-v05'
    review04_ids={x['run_id'] for x in runs if x.get('source_quality_review_version')=='q4-r3-q2compact-quality-review04'}
    diagnostic_scope=decision['conditional_diagnostic_range_eligible_scope']
    for run in runs:
        regional=run.get('evaluation_stratum')=='conditional_scenario'
        material=regional and (run.get('producer')=='reduced' or run.get('applicability_class')=='target_applicable_required')
        run['passes_target_material_applicability_gate']=bool(material)
        run['scientific_acceptance_decision_version']=decision['version']
        run['arrival_envelope_semantics_version']=semantics
        run['arrival_envelope_gate_scope']='explicit scientific accepted-run membership plus execution/time/field/material/boundary gates; diagnostic gate alone grants no accepted envelope'
        run['conditional_diagnostic_range_eligible_scope']=diagnostic_scope
        run['human_scientific_adoption']=decision['human_scientific_adoption']
        run['accepted_arrival_envelope_run_membership']=run['run_id'] in accepted
        if regional:
            run['regional_accuracy_qualified']=bool(decision['regional_accuracy_qualified'])
            run['regional_numerical_accuracy_qualified']=run['regional_accuracy_qualified']
            run['scientifically_qualified_for_prediction']=bool(decision['scientifically_qualified_for_prediction'])
            run['mass_conservation_qualified']=bool(decision['mass_conservation_qualified'])
        run['include_in_arrival_envelope']=False
        run['parameter_comparison_scope']='exploratory conditional/numerical diagnostic; not physical validation or accepted predictive model gain'
    for row in arrivals:
        run=by[row['run_id']];regional=run.get('evaluation_stratum')=='conditional_scenario'
        process_bound=bool(run.get('fields_hash_matches_index') and run.get('field_reference'))
        target_bound=(row.get('target_field_hash_matches') is True) if run.get('producer')=='official' else row.get('arrival_status') not in ['missing','outside_domain']
        target_inside=row.get('arrival_status') not in ['missing','outside_domain','out_of_domain'] and not (row.get('independent_saved_field_arrival_check') or {}).get('outside',False)
        observation=bool(process_bound and target_bound and target_inside and (row.get('observed_hit') or row.get('raw_source_row_json') or row.get('independent_saved_field_arrival_check') or row.get('arrival_status') in ['arrived','not_arrived_by_horizon']))
        row['diagnostic_observation_available']=observation
        row['diagnostic_observation_available_scope']=decision['diagnostic_observation_available_scope']
        row['passes_execution_time_boundary_gates']=bool(row.get('passes_execution_time_boundary_gates') and run.get('actual_native_time_matches_npz') is not False and run.get('parsed_source_snapshot_hash_matches') is not False)
        row['passes_target_material_applicability_gate']=run['passes_target_material_applicability_gate']
        row['passes_execution_time_field_material_boundary_diagnostic_gates']=bool(row['passes_execution_time_boundary_gates'] and target_bound and run['passes_target_material_applicability_gate'])
        row['conditional_diagnostic_range_eligible']=bool(regional and row['passes_execution_time_field_material_boundary_diagnostic_gates'] and row['run_id'] not in review04_ids)
        row['conditional_diagnostic_exclusion_reason']='stricter compact-source quality review04' if row['run_id'] in review04_ids else None if row['conditional_diagnostic_range_eligible'] else 'not a completed time/field/material/boundary-qualified regional diagnostic'
```

```
        for key in ['scientific_acceptance_decision_version','arrival_envelope_semantics_version','arrival_envelope_gate_scope','conditional_diagnostic_range_eligible_scope','human_scientific_adoption','accepted_arrival_envelope_run_membership']:
            row[key]=run[key]
        if regional:
            for key in ['regional_accuracy_qualified','regional_numerical_accuracy_qualified','scientifically_qualified_for_prediction','mass_conservation_qualified']:row[key]=run[key]
        row['include_in_arrival_envelope']=bool(regional and row['run_id'] in accepted
            and row['conditional_diagnostic_range_eligible'] and
            run.get('regional_accuracy_qualified')
            and run.get('mass_conservation_qualified') and
            run.get('scientifically_qualified_for_prediction'))
    for run in runs:
        own=[x for x in arrivals if x['run_id']==run['run_id']]
        run['diagnostic_observation_available']=bool(any(x['diagnostic_observation_available']
        for x in own) or run.get('field_reference') and
        run.get('fields_hash_matches_index'))
        run['passes_execution_time_boundary_gates']=any(x['passes_execution_time_boundary_gates'] for x in own)
        run['conditional_diagnostic_range_eligible']=any(x['conditional_diagnostic_range_eligible'] for x in own)
        run['include_in_arrival_envelope']=any(x['include_in_arrival_envelope'] for x in own)
    for row in metrics:
        run=by[row['run_id']]
        for key in ['include_in_arrival_envelope','diagnostic_observation_available','passes_execution_time_boundary_gates','passes_target_material_applicability_gate','conditional_diagnostic_range_eligible','scientific_acceptance_decision_version','arrival_envelope_semantics_version','arrival_envelope_gate_scope','conditional_diagnostic_range_eligible_scope','human_scientific_adoption','accepted_arrival_envelope_run_membership']:
            row[key]=run[key]
        if run.get('evaluation_stratum')=='conditional_scenario':
            for key in ['regional_accuracy_qualified','regional_numerical_accuracy_qualified','scientifically_qualified_for_prediction','mass_conservation_qualified']:row[key]=run[key]

def bind_fullmask_volume_annotation(ins,runs,metrics):
    """Carry the immutable post-v3 metadata clarification into later rolling/final tables."""
    path=R/'exports/propagation_handoff_v3_annotation.json'
    note=ins.js(path,'immutable clarification of inherited baseline volume versus actual Q2 fullmask release',False)
    if note is None:return None
    if digest(ins.bytes[str(path.resolve())])!='1b4db70b8a19b92860267b14912ed435a89624e8db1d8a9df4c84c541ab87f39':
        raise RuntimeError('Adopted fullmask volume metadata annotation bytes changed')
    source=note['source_bundle'];ins.read(ROOT/source['path'],'unchanged frozen fullmask partial bundle underlying metadata clarification')
    if not ins.check_bound(source):raise RuntimeError('Fullmask annotation source bundle changed')
    by={x['run_id']:x for x in runs}
    keys=['legacy_expected_reference_grid_volume_m3','legacy_field_role','release_grid_volume_m3',
        'q2_frozen_integrated_volume_m3','native_saved_t0_depth_integral_m3_float64']
    for rec in note['runs']:
        if rec['run_id'] not in by:continue
        run=by[rec['run_id']]
        if digest(ins.read(C/'runs'/run['run_id']/'case.json','actual case bound to fullmask volume metadata clarification'))!=rec['original_case']['sha256']:
            raise RuntimeError('Fullmask annotation no longer matches actual native case: '+run['run_id'])
        run.update({k:rec[k] for k in keys})
```

```
        run['legacy_reference_volume_is_release_acceptance_target']=False
        run['release_volume_annotation_version']=note['annotation_version']
        for row in metrics:
            if row['run_id']==run['run_id']:
                for key in keys+['legacy_reference_volume_is_release_acceptance_target','relea
                ↪  se_volume_annotation_version']:row[key]=run[key]
    return {'path':str(path.relative_to(ROOT)),'bytes':len(ins.bytes[str(path.resolve())]),
        'sha256':digest(ins.bytes[str(path.resolve())]),'annotation_version':note['annotation_
        ↪  version'],
        'scope':'input identity and metadata interpretation only; no scientific qualification
        ↪  granted'}

def bind_q2_upstream_catalogues(ins):
    """Bind delivered Q2 material/release provenance without inventing Q4 runs."""
    base=R/'inputs/q2';catalogues=[]
    for path in sorted(base.glob('*/bundle.json')):
        bundle=ins.js(path,'delivered Q2 source/material catalogue descriptor; not additional
        ↪  Q4 model cases')
        ins.js(path.parent/'upstream_bundle.json','original Q2 producer descriptor and delivery
        ↪  lineage')
        for rec in bundle['files']:
            source=(ROOT/rec['path']).resolve()
            if not source.is_relative_to(base.resolve()) or not
            ↪  ins.check_bound({**rec,'role':'delivered Q2 catalogue member: '+rec['role']}):
                raise RuntimeError('Delivered Q2 catalogue member path/bytes/hash mismatch:
                ↪  '+rec['path'])
            if rec['bytes']<=256*1024:
                try:source.read_bytes().decode('utf-8')
                except UnicodeDecodeError:pass
                else:ins.read(source,'delivered Q2 source/material metadata and parameter
                ↪  contract snapshot')
        catalogues.append({'bundle_version':bundle['bundle_version'],'path':str(path.relative_
        ↪  to(ROOT)),
            'bytes':len(ins.bytes[str(path.resolve())]),'sha256':digest(ins.bytes[str(path.res
            ↪  olve())]),
            'member_count':len(bundle['files']),'member_byte_hashes_verified':True})
    return {'catalogues':catalogues,'adds_new_Q4_model_case_count':0,'adds_independent_validat
    ↪  ion_events':0,
        'scope':'delivered source/material/parameter lineage only; generic Q4 scenarios without
        ↪  explicit Q2 parent remain unknown'}

def bind_official_final_receipt(ins):
    path=R/'results/official_final_delivery.json'
    receipt=ins.js(path,'owner final exactready receipt; closed native attempts and stable
    ↪  actual output/control interfaces',False)
    if receipt is None:return None
    if digest(ins.bytes[str(path.resolve())])!='9c510981ad94b93d84ff444b305bffe99e71936e2ab6c9
    ↪  6a71d66e23349f8cd8':
        raise RuntimeError('Authorized final official delivery receipt bytes changed')
    if receipt['required_closed']!=30 or receipt['applicable_required']!=30 or
    ↪  receipt['active_native_containers']!=0 or receipt['continuous_workers']:
        raise RuntimeError('Official final delivery receipt is not the authorized
        ↪  settled30-required scope')
    for rec in receipt['stable_artifacts']:
        if not ins.check_bound({**rec,'role':'owner-final stable official artifact'}):
            raise RuntimeError('Owner-final official artifact no longer matches receipt:
            ↪  '+rec['path'])
        source=ROOT/rec['path']
        if rec['bytes']<=256*1024:
            try:source.read_bytes().decode('utf-8')
            except UnicodeDecodeError:pass
            else:ins.read(source,'owner-final official evidence/control/replay/diagnostic
            ↪  metadata snapshot')
```

```
    return {'path':str(path.relative_to(ROOT)),'bytes':len(ins.bytes[str(path.resolve())]),
        'sha256':digest(ins.bytes[str(path.resolve())]),'verified_utc':receipt['verified_utc'],
        'required_closed':30,'generic_diagnostic_closed':1,'material_excluded_not_run':7,
        'stable_artifacts_byte_hash_verified':True,'physical_accuracy_or_prediction_qualificat
        ↪ ion':False}

def stationary_residual_metrics(ins,runs,metrics):
    """Last-state kinematic residual, never inferred from cumulative maxima."""
    import numpy as np
    by={x['run_id']:x for x in runs}
    definition='last saved state with h>=0.1m and finite horizontal speed<0.1m/s; stationary
    ↪ residual, not measured deposit'
    for row in metrics:
        run=by[row['run_id']];field=run.get('field_reference')
        row.update(stationary_residual_definition=definition,stationary_residual_evaluation_ti
        ↪ me_seconds=run.get('actual_end_time_seconds'),
            stationary_residual_area_m2=None,stationary_residual_equivalent_width_area_m2=None,
            stationary_residual_volume_m3=None,stationary_residual_max_depth_m=None)
        if run.get('execution_status')!='completed' or not field or not
        ↪ run.get('fields_hash_matches_index'):
            row['stationary_residual_status']='not_evaluated_without_completed_bound_saved_sta
            ↪ te';continue
        path=Path(field['path']);path=path if path.is_absolute() else ROOT/path
        ins.note_array_read(path,'NumPy last-state depth and available horizontal velocity
        ↪ inspected for stationary residual; finite/missing velocity guards applied; 1D width
        ↪ or 2D affine geometry used when evaluable')
        with np.load(path,allow_pickle=False) as a:
            if run['producer']=='reduced':
                h=a['h_final'];speed=np.hypot(a['u_east_final'],a['v_north_final'])
                dx=number(run['parameters'].get('resolution_m'))
                is1d=run['model_id'].startswith('independent_1d')
                cell_area=dx*number(run['raw_record_json'].get('width')) if is1d else dx*dx
            else:
                h=a['depth_final'];is1d=False
                if 'stationary_criterion_speed_final' in a.files:
                    speed=a['stationary_criterion_speed_final']
                    row['stationary_residual_velocity_basis']='dedicated native final
                    ↪ horizontal criterion; multiphase uses maximum speed among phases with
                    ↪ h_phase>=1e-6m'
                elif run.get('model_id')=='ravaflow_multi' or
                ↪ run.get('comparison_model_subgroup') in
                ↪ ['composite_solid_water','generic_three_material_proxy']:
                    row['stationary_residual_status']='conservative_wet_phase_final_velocity_u
                    ↪ navailable';continue
                else:
                    horizontal=a['horizontal_speed_frames'] if 'horizontal_speed_frames' in
                    ↪ a.files else None
                    if horizontal is None or horizontal.size==0:
                        row['stationary_residual_status']='horizontal_velocity_unavailable';co
                        ↪ ntinue
                    speed=horizontal[-1]
                    row['stationary_residual_velocity_basis']='native single-phase or DClaw
                    ↪ final horizontal velocity'
                tr=a['transform'];cell_area=abs(float(tr[0]*tr[4]-tr[1]*tr[3]))
            if h.shape!=speed.shape or not np.isfinite(h).all() or not
            ↪ np.isfinite(speed[h>=.1]).all():
                row['stationary_residual_status']='missing_or_nonfinite_last_state';continue
            mask=(h>=.1)&np.isfinite(speed)&(speed<.1)
            area=float(mask.sum()*cell_area)
            row['stationary_residual_status']='saved_state_kinematic_statistic_only'
            row['stationary_residual_equivalent_width_area_m2' if is1d else
            ↪ 'stationary_residual_area_m2']=area
```

```
            row['stationary_residual_volume_m3']=float(h[mask].sum()*cell_area)
            row['stationary_residual_max_depth_m']=float(h[mask].max()) if mask.any() else 0.
            row['stationary_residual_speed_definition']='horizontal E/N magnitude' if not is1d
            ↪  else 'mapped along-path speed magnitude; equivalent-width geometry'
            row['stationary_residual_source_fields_sha256']=field['sha256']

def enrich_native_semantics_and_budget(ins,runs,metrics):
    """Keep max-of-sum depth, phase maxima, native times and source budgets distinct."""
    import numpy as np
    by={x['run_id']:x for x in runs}
    for row in metrics:
        run=by[row['run_id']]
        if run.get('producer')!='official':continue
        case=run['raw_case_json'];check=run['raw_validation_json'];raw=run['raw_metrics_json']
        ↪  ;rid=run['run_id']
        ismulti=case.get('model')=='ravaflow_multi';israva=case.get('model','').startswith('ra
        ↪  vaflow')
        row['speed_peak_comparable_across_models']=False
        row['horizontal_speed_definition']='max over wet phases of horizontal
        ↪  hypot(momentum_east,momentum_north)/h_phase; not mixture velocity' if ismulti else
        ↪  'single equivalent-mixture horizontal hypot(momentum_east,momentum_north)/h'
        row['horizontal_speed_phase_wet_depth_cutoff_m']=.1 if israva else None
        row['horizontal_speed_total_wet_depth_cutoff_m']=.1
        row['speed_definition']='maximum native terrain-corrected phase scalar; not mixture or
        ↪  horizontal speed' if ismulti else 'native terrain-corrected single-phase scalar;
        ↪  separate horizontal magnitude' if israva else 'single-mixture horizontal magnitude
        ↪  with bed_normal=0'
        row['native_cumulative_depth_definition']='max over cells of native ASCII max_t(sum
        ↪  phase depths); not sum of separately located phase maxima' if ismulti else 'max
        ↪  over cells of native ASCII cumulative total depth' if israva else 'not available as
        ↪  native cumulative field; saved-output maximum kept separate'
        row['native_cumulative_scalar_raster_speed_peak_m_s']=number(raw.get('native_cumulativ
        ↪  e_max_speed_m_s'))
        row['native_nout_scalar_speed_diagnostic_m_s']=number(check.get('native_global_cumulat
        ↪  ive_speed_scalar_m_s'))
        row['ascii_raster_source_adjusted_budget_relative_residual']=number(check.get('native_
        ↪  mass_budget_relative_residual'))
        row['native_internal_summary_mass_budget_relative_residual']=number(check.get('native_
        ↪  internal_summary_mass_budget_relative_residual'))
        row['budget_representations_are_separate']=True
        row['source_adjusted_budget_definition']=check.get('native_budget_definition')
        row['native_signed_entrainment_phase_volumes_m3']=check.get('native_signed_entrainment
        ↪  _phase_volumes_m3')
        row['native_entrained_added_volume_m3']=check.get('native_entrained_added_volume_m3')
        row['native_entrained_added_mass_kg']=check.get('native_entrained_added_mass_kg')
        row['remaining_bed_capacity_m3']=check.get('remaining_bed_capacity_m3')
        row['remaining_bed_from_saved_basechange_m3']=check.get('remaining_bed_from_saved_base
        ↪  change_m3')
        row['basechange_cellwise_within_bed_capacity']=check.get('basechange_cellwise_within_b
        ↪  ed_capacity')
        row['native_entrainment_input_slot_verified']=check.get('native_entrainment_input_slot
        ↪  _verified')
        row['entrainment_within_prescribed_volume_capacity']=check.get('entrainment_within_pre
        ↪  scribed_volume_capacity')
        if ismulti and run.get('comparison_model_subgroup')=='composite_solid_water':
            row['material_mapping_scope']='frictional rock-ice composite solid plus separate
            ↪  free water; no independent ice velocity, melting or ice-strength evolution'
            row['material_mapping_admissibility_is_not_mass_qualification']=True
            run['material_mapping_admissibility_is_not_mass_qualification']=True
        parser_path=check.get('parser_source_snapshot_path');parser_sha=check.get('parser_sha2
        ↪  56_at_start')
        if parser_path and parser_sha:
```

```
            p=Path(parser_path)
            run['parsed_source_snapshot_hash_matches']=ins.check_bound({'path':str(p),'bytes': p.stat().st_size if p.exists() else 0,'sha256':parser_sha,'role':'actual hash-bound postprocessing source snapshot'})
            if run['parsed_source_snapshot_hash_matches'] and p.stat().st_size<=256*1024:ins.read(p,'actual hash-bound postprocessing source text snapshot')
        else:run['parsed_source_snapshot_hash_matches']=None
        if run.get('execution_status')!='completed' or not run.get('field_reference'):continue
        field=run['field_reference'];p=Path(field['path'])
        ins.note_array_read(p,'NumPy actual native time and native cumulative total-depth array checked against parsed metrics; phase-source mass ledger recomputed from saved accounting')
        with np.load(p,allow_pickle=False) as data:
            times=data['time_s'];run['actual_native_time_matches_npz']=abs(float(times[-1])-(run.get('actual_end_time_seconds') or -1))<.01
            if israva and data['depth_max_native_cumulative'].size:
                actual=float(data['depth_max_native_cumulative'].max());reported=number(row.get('native_cumulative_max_depth_m'))
                row['native_cumulative_depth_matches_hashed_npz']=reported is not None and abs(actual-reported)<1e-8
            if israva and 'phase_volume_m3' in data.files:
                rho=np.array([case.get('bulk_density_kg_m3',2105)] if case['model']=='ravaflow_single' else case.get('densities_kg_m3',[2700,1850,1000]),dtype=float)
                initial=float(data['phase_volume_m3'][0]@rho)
                core=np.array(check.get('native_active_interior_final_phase_volumes_m3',[]),dtype=float);edge=np.array(check.get('native_edge_store_outflow_phase_volume_m3',[]),dtype=float)
                signed=np.array(check.get('native_signed_entrainment_phase_volumes_m3',[])[:len(rho)],dtype=float)
                if len(core)==len(rho)==len(edge)==len(signed) and initial>0:
                    source=float(-signed@rho);balance=float((core@rho+edge@rho-initial-source)/initial)
                    row['independently_recomputed_ascii_source_adjusted_residual']=balance
                    row['independently_recomputed_entrained_source_mass_kg']=source
                    expected=number(row.get('ascii_raster_source_adjusted_budget_relative_residual'))
                    row['parsed_source_adjusted_budget_matches_recomputation']=expected is not None and abs(balance-expected)<1e-10
        if israva:
            files=C/'runs'/rid/f'{rid}_results'/f'{rid}_files';nout=files/f'{rid}_nout1.txt'
            if nout.exists():
                vals=ins.read(nout,'actual raw native nout diagnostic, separately named from cumulative total-depth raster').decode().split()
                if len(vals)>3:
                    row['raw_nout_hflow_maxmax_m']=number(vals[3])
                    row['raw_nout_hflow_maxmax_definition']='native internal max_t(sum of separate per-phase spatial maxima) under native domain/representation; phase maxima may occur at different locations; not cumulative total-depth raster maximum' if ismulti else 'native single-phase internal global depth diagnostic; kept separate from cumulative raster'

def bind_existing_replays(ins):
    replay=ins.js(R/'results/replay_execution_comparison.json','actual reproduction of existing cases; not new model cases or independent events',False)
    if replay:
        for row in replay.get('replayed_cases',[]):
            for key in ['original_fields','replayed_fields']:
                field=row.get(key)
                if not field:continue
                p=Path(field['path'])
```

```
                if not p.exists() or not
                ↪ ins.check_bound({**field,'bytes':p.stat().st_size,'role':'existing-case
                ↪ replay field evidence'}):
                    raise RuntimeError('Existing-case replay field hash mismatch: '+str(p))
    return replay

def bind_native_controls(ins,explicit_indexes=None):
    """Bind acquired effective native controls, never infer a past runner hash."""
    indexes=[Path(x) if Path(x).is_absolute() else R/Path(x) for x in explicit_indexes or []]
    if not indexes:indexes=sorted((R/'results').glob('official*control*index*.json'))
    records=[];issues=[];owner_runs=[]
    def walk(obj):
        if isinstance(obj,dict):
            if all(k in obj for k in ['path','bytes','sha256']):yield obj
            for value in obj.values():yield from walk(value)
        elif isinstance(obj,list):
            for value in obj:yield from walk(value)
    for path in indexes:
        packet=ins.js(path,'acquired actual native input/control provenance index')
        owner_runs+=packet.get('runs',[])
        for entry in walk(packet):
            p=Path(entry['path']);p=p if p.is_absolute() else ROOT/p
            role=entry.get('role','actual acquired native input/control bytes')
            if not ins.check_bound({**entry,'path':str(p),'role':role}):
                issues.append({'path':str(p),'issue':'missing_or_hash_mismatch','expected':ent
                ↪ ry});continue
            if p.stat().st_size<=256*1024:
                b=p.read_bytes()
                try:b.decode('utf-8')
                except UnicodeDecodeError:pass
                else:ins.read(p,'small acquired actual native control text snapshot: '+role)
            records.append({'path':str(p),'bytes':entry['bytes'],'sha256':entry['sha256'],'rol
            ↪ e':role})
    unique={x['path']:x for x in records}
    return {'status':'indexed_actual_generated_controls' if indexes and unique and not issues
    ↪ else 'index_missing' if not indexes else 'control_dependency_mismatch',
        'index_paths':[str(x.relative_to(ROOT)) if x.is_relative_to(ROOT) else str(x) for x in
        ↪ indexes],
        'unique_file_count':len(unique),'issues':issues,
        'owner_indexed_run_count':len(owner_runs),
        'historical_adapter_identity_status_counts':dict(collections.Counter(x.get('historical
        ↪ _adapter_identity_status','unreported') for x in owner_runs)),
        'control_role_counts':dict(collections.Counter(x['role'] for x in unique.values())),
        'provenance_scope':'Actual acquired generated *.data/native parameter/run scripts and
        ↪ raster/topo inputs; execution-time provenance only where explicitly recorded by
        ↪ runtime owner',
        'unknown_historical_runner_hashes_are_not_reconstructed':True,
        'current_or_repository_adapter_hash_is_not_a_substitute_for_unrecorded_historical_runn
        ↪ er_hash':True}

def bind_scheduling_context(ins):
    files=[]
    for p in sorted(set((R/'config').glob('official_scheduling_amendment_*.json'))|set((R/'res
    ↪ ults').glob('official_scheduling_cpu*receipt*.json'))):
        ins.js(p,'actual scheduling/CPU quota context; not a controlled runtime performance
        ↪ benchmark')
        files.append(str(p.relative_to(ROOT)))
    for p in
    ↪ [R/'results/official_result_matrix.csv',R/'results/official_final_diagnostics.json']:
        if p.exists():ins.read(p,'actual cohort diagnostics; no new model cases or independent
        ↪ validation events')
```

```
    return {'source_paths':files,'wall_time_interpretation':'actual elapsed wall time under mixed concurrency and CPU quota regimes; no controlled model runtime/performance comparison',
        'controlled_performance_benchmark':False,'resource_controls_change_physical_equations':False}

def build_reduced(ins):
    runs=[];metrics=[];arrivals=[]
    packet=ins.js(R/'results/reduced_delivery_index.json','immutable adopted reduced delivery index')
    if digest(ins.bytes[str((R/'results/reduced_delivery_index.json').resolve())])!='cd82c52cc772ed7702a6f4452f4bfbbe5c9a092bbbeeed46d356d09709a4a7dc':raise RuntimeError('Adopted reduced index bytes changed; do not silently replace frozen input')
    for entry in packet['files']:
        if not ins.check_bound(entry):raise RuntimeError('Frozen reduced artifact mismatch: '+entry['path'])
    raw=ins.csv(R/'results/reduced_delivery_metrics.csv','frozen reduced regional delivery')
    for row in raw:
        rid=row['run_id'];rec=ins.js(R/f'results/reduced_{rid}.json','frozen reduced raw run')
        summary=rec['summary'];is1d=rec['model_id'].startswith('independent_1d');s=row['scenario_id']
        base={'run_id':rid,'producer':'reduced','model_id':row['model_id'],'scenario_id':s,
          'base_scenario_id':s,'parameter_version':row['parameter_version'],'geometry_version':row['geometry_version'],
          'upstream_model_id':None,'upstream_parameter_version':None,'upstream_onset_status':None,
          'release_mapping_version':'common_reference','mechanism_id':rec['mechanism_id'],
          'experiment_id':'path_and_material_sensitivity' if is1d else 'regional_ofat_grid_timestep','path_id':rec.get('path_id'),
          'evaluation_stratum':'conditional_scenario','plan_scope':'current_registered',
          'execution_attempted':True,'native_solver_started':True,'execution_status':summary['execution_status'],
          'raw_execution_status':summary['execution_status'],'requested_horizon_seconds':summary['duration_requested_seconds'],
          'actual_end_time_seconds':summary['duration_actual_seconds'],'wall_seconds':summary['wall_seconds'],
          'parameters':rec['parameters'],'initial_velocity_east_north_m_s':[0,0],
          'initiation_to_release_delay_seconds':None,'if_released_propagation':True,
          'time_origin':'bulk_release','state_horizon_complete':summary['duration_actual_seconds']>=summary['duration_requested_seconds']-1e-8,
          'pipeline_returncode':0,'finite_state':True,'nonnegative_state':summary['minimum_pre_clip_depth_m']>=-1e-8,
          'fields_hash_matches_index':ins.check_bound(rec['fields']),'field_reference':rec['fields'],
          'numerical_qualification':'unqualified_thin_flow_HR_acceleration_bias',
          'include_in_scientific_parameter_comparison':True,'include_in_arrival_envelope':False,
          'event_timing_validated':False,'independent_event_validation_count':0,
          'raw_record_json':rec,'source_table_row_json':row}
        runs.append(base)
        initial=summary['initial_volume_m3'];inflow=summary['inflow_volume_m3']
        metrics.append({'run_id':rid,'model_id':base['model_id'],'scenario_id':s,'evaluation_stratum':'conditional_scenario',
          'initial_volume_m3':initial,'final_domain_volume_m3':summary['final_domain_volume_m3'],
          'initial_mass_kg':summary.get('initial_mass_kg'),'final_domain_mass_kg':summary.get('final_mass_kg'),
          'outflow_volume_m3':summary['outflow_volume_m3'],'inflow_volume_m3':inflow,
          'boundary_inflow_fraction_of_initial':inflow/initial if initial else None,
          'implicit_boundary_supply':inflow>initial*1e-6,
          'native_mass_budget_relative_residual':summary['relative_mass_balance_error'],
          'mass_budget_definition':'step-level conservative volume ledger; incoming flux and clipping booked; constant prescribed density',
```

```
        'max_depth_m':summary['max_depth_m'],'speed_peak_m_s':summary['maximum_speed_any_wet_m_s'],
        'speed_definition':'horizontal along-path magnitude' if is1d else 'horizontal E/N magnitude',
        'speed_wet_depth_cutoff_m':1e-9,'peak_sampling_definition':'every solver step',
        'impact_area_m2':None if is1d else summary['footprint_area_hmax_ge_01_m2'],
        'equivalent_width_swept_area_m2':summary['footprint_area_hmax_ge_01_m2'] if is1d else None,
        'impact_area_definition':'not comparable to 2D area' if is1d else 'planar grid area where step cumulative hmax>=.1m',
        'numerical_qualification':base['numerical_qualification'],'include_in_arrival_envelope':False})
    for target,a in summary['target_arrivals'].items():
        travel=a['travel_time_seconds'];bracket=a.get('arrival_time_bracket_seconds')
        arrivals.append({'run_id':rid,'scenario_id':s,'base_scenario_id':s,'model_id':base['model_id'],
          'target_id':target,'target_geometry_version':row['geometry_version'],'geometry_version':row['geometry_version'],
          'parameter_version':row['parameter_version'],'raw_status':a['arrival_status'],
          'arrival_status':'arrived' if travel is not None else 'not_arrived_by_horizon' if 'not_reached' in a['arrival_status'] else a['arrival_status'],
          'observed_hit':travel is not None,'observed_hit_time_seconds':travel,
          'travel_time_seconds':travel,'arrival_bracket_lower_seconds':bracket[0] if bracket else None,
          'arrival_bracket_upper_seconds':bracket[1] if bracket else None,
          'arrival_definition':a['definition'],'evaluation_end_time_seconds':base['actual_end_time_seconds'],
          'requested_horizon_seconds':base['requested_horizon_seconds'],'include_in_arrival_envelope':False,
          'domain_affected_before_arrival':None,'numerical_qualification':base['numerical_qualification'],
          'evaluation_stratum':'conditional_scenario','initiation_to_release_delay_seconds':None,'time_origin':'bulk_release',
          'if_released_propagation':True,'probability_weight':None,'event_timing_validated':False})
for row in ins.csv(R/'results/reduced_osm_target_arrivals_v2.csv','frozen reduced five-object postprocessing'):
    runs_by_id={x['run_id']:x for x in runs};base=runs_by_id[row['run_id']]
    arrivals.append({'run_id':row['run_id'],'scenario_id':row['scenario_id'],'base_scenario_id':base['base_scenario_id'],
      'model_id':row['model_id'],'target_id':row['target_id'],'target_kind':row['target_kind'],
      'target_geometry_version':row['geometry_version'],'geometry_version':base['geometry_version'],
      'parameter_version':row['parameter_version'],'raw_status':row['arrival_status'],
      'arrival_status':'not_arrived_by_horizon' if 'not_reached' in row['arrival_status'] else row['arrival_status'],
      'observed_hit':number(row['travel_time_seconds']) is not None,'observed_hit_time_seconds':number(row['travel_time_seconds']),
      'travel_time_seconds':number(row['travel_time_seconds']),
      'arrival_bracket_lower_seconds':number(row['arrival_lower_seconds']),'arrival_bracket_upper_seconds':number(row['arrival_upper_seconds']),
      'arrival_definition':row['definition'],'evaluation_end_time_seconds':number(row['actual_horizon_seconds']),
      'requested_horizon_seconds':1800,'include_in_arrival_envelope':False,
      'domain_affected_before_arrival':None,'numerical_qualification':base['numerical_qualification'],
      'evaluation_stratum':'conditional_scenario','initiation_to_release_delay_seconds':None,'time_origin':'bulk_release',
      'if_released_propagation':True,'probability_weight':None,'event_timing_validated':False,'raw_source_row_json':row})
return runs,metrics,arrivals
```

```
592
593 def collect_official(ins,active_configs):
594     active=set();active_sources={};planned={}
595     applicability=ins.js(R/'config/model_applicability_decision_03.json','registered material
    ↪  applicability and necessary-case decision',False) or {}
596     classes=applicability.get('applicability_classes',{})
597     diagnostic_ids=set(classes.get('generic_material_proxy_diagnostic_only',{}).get('run_ids',
    ↪  []))
598     excluded_ids=set(classes.get('target_material_proxy_inapplicable_not_run',{}).get('run_ids
    ↪  ',[]))
599     replacement=applicability.get('replacement',{})
600     retry_budget=ins.js(R/'config/execution_budget_amendment_02.json','registered legacy retry
    ↪  process and cancellation scope',False) or {}
601     cancellation_index=ins.js(R/'results/official_superseded_cancellations.json','independent
    ↪  stopped-retry cancellation receipts',False) or {}
602     cancellation_by_id={x['run_id']:x for x in cancellation_index.get('records',[])}
603     upstream={}
604     for path in sorted((R/'inputs/q2').glob('**/release_states*.csv')):
605         for state in ins.csv(path,'delivered Q2 upstream state and mechanism keys'):
606             upstream[state['scenario_id']]=state
607     # Only this explicitly authorized supplemental family is discovered dynamically;
608     # historical generic/surface2 preparation packets remain outside the active plan.
609     supplemental=sorted(set((R/'config').glob('official_cases_entrain*.json')))
610     for protocol in sorted((R/'config').glob('official_entrain*protocol*.json')):
611         ins.js(protocol,'preregistered bounded native entrainment scenario protocol')
612     for name in list(active_configs)+supplemental:
613         path=Path(name);path=path if path.is_absolute() else R/'config'/path
614         packet=ins.js(path,'current registered official plan',False)
615         if packet:
616             for item in packet.get('cases',[]):
617                 active.add(item['run_id']);planned[item['run_id']]=item
618                 active_sources.setdefault(item['run_id'],[]).append(str(path))
619     if replacement.get('new_run_id'):
620         rid=replacement['new_run_id'];active.add(rid)
621         planned[rid]={'run_id':rid,'case':str(C/'runs'/rid/'case.json'),'model':'ravaflow_mult
        ↪  i','evidence_role':'conditional_scenario'}
622         active_sources.setdefault(rid,[]).append(str(R/'config/model_applicability_decision_03
        ↪  .json'))
623     tables={x['run_id']:x for x in ins.csv(R/'results/official_runs.csv','official parsed run
    ↪  snapshot')}
624     metrics={x['run_id']:x for x in ins.csv(R/'results/official_metrics.csv','official parsed
    ↪  metric snapshot')}
625     validation=ins.js(R/'results/official_validation.json','official numerical evidence
    ↪  snapshot')
626     checks={x['run_id']:x for x in validation['checks']}
627     fields={x['run_id']:x for x in ins.js(R/'results/official_fields.json','official process
    ↪  field index')['fields']}
628     raw_arrivals=ins.csv(R/'results/official_arrivals.csv','official primary target snapshot')
629     history=ins.csv(R/'results/official_histories.csv','official saved-time mass/volume
    ↪  history',False)
630     initial_history={}
631     for row in history:
632         rid=row['run_id']
633         if rid not in initial_history or number(row['time_s'])<number(initial_history[rid]['ti
        ↪  me_s']):initial_history[rid]=row
634     extra=ins.csv(R/'results/official_target_arrivals_v2.csv','official OSM target
    ↪  snapshot',False)
635     cases={};executions={}
636     for folder in sorted((C/'runs').glob('*')):
637         if not (folder/'case.json').exists():continue
638         case=ins.js(folder/'case.json','official case
        ↪  inventory');rid=case['run_id'];cases[rid]=case
```

```
        ex=ins.js(folder/'execution.json','official actual execution/classification',False)
        if ex is not None:executions[rid]=ex
    for rid in active-cases.keys():
        item=planned[rid];path=Path(item.get('case',C/'runs'/rid/'case.json'))
        case=ins.js(path,'registered case outside prepared cache',False)
        cases[rid]=case or {'run_id':rid,'model':item.get('model','unprepared_unknown'),
            'evidence_role':item.get('evidence_role','conditional_scenario'),
            'unprepared_plan_entry':item}
    runs=[];outmetrics=[]
    for rid,case in sorted(cases.items()):
        raw=tables.get(rid,{});m=metrics.get(rid,{});check=checks.get(rid,{});ex=executions.ge
        ↪ t(rid,{});original=ex.get('original_execution_record') or {}
        gate=ex.get('quality_gate') or check.get('quality_gate') or raw.get('quality_gate')
        csurface=case.get('csurface',2) if case['model'].startswith('ravaflow') else None
        quarantine=bool(gate) or case['model']=='ravaflow_single' and csurface==2
        scope='current_registered' if rid in active else 'superseded_configuration' if
        ↪ quarantine or csurface==2 else 'historical_preparation'
        log_exists=(C/'runs'/rid/'run.log').exists()
        attempted=bool(ex.get('command') or ex.get('started_utc') or original.get('command') or
        ↪ original.get('started_utc') or log_exists or 'cancel' in str(ex.get('status','')))
        evaluation_stratum=stratum(case.get('evidence_role'))
        if rid in excluded_ids:applicability_class='target_material_proxy_inapplicable_not_run'
        elif rid in diagnostic_ids:applicability_class='generic_material_proxy_diagnostic_only'
        elif scope=='current_registered' and evaluation_stratum=='conditional_scenario':applic
        ↪ ability_class='target_applicable_required' if applicability else 'pending_review'
        else:applicability_class='numerical_benchmark' if
        ↪ evaluation_stratum=='numerical_benchmark' else 'outside_current_required_scope'
        required=applicability_class=='target_applicable_required'
        completion_required=required or rid in diagnostic_ids and attempted or not
        ↪ applicability and scope=='current_registered' and
        ↪ evaluation_stratum=='conditional_scenario'
        returncode=ex.get('returncode');actual_end=number(check.get('actual_last_time_seconds'
        ↪ ,m.get('last_time_s')))
        requested=number(case.get('end_seconds'));complete=actual_end is not None and requested
        ↪ is not None and actual_end>=requested-.01
        field=fields.get(rid);field_ok=ins.check_bound(field) if field else False
        case_path=C/'runs'/rid/'case.json'
        case_sha=digest(ins.bytes[str(case_path.resolve())]) if str(case_path.resolve()) in
        ↪ ins.bytes else None
        execution_case_matches=bool(not ex or ex.get('case_sha256')==case_sha)
        log_path=Path(ex.get('log_path',C/'runs'/rid/'run.log'))
        log_matches=ins.check_bound({'path':str(log_path),'bytes':log_path.stat().st_size,'sha
        ↪ 256':ex['log_sha256'],'role':'native execution log bound to actual attempt'}) if
        ↪ ex.get('log_sha256') and log_path.exists() else None
        execution_path=C/'runs'/rid/'execution.json'
        execution_sha=digest(ins.bytes[str(execution_path.resolve())]) if
        ↪ str(execution_path.resolve()) in ins.bytes else None
        def bound_attempt_matches(record):
            comparisons=[record.get('execution_sha256')==execution_sha if
            ↪ record.get('execution_sha256') else None,
                record.get('log_sha256')==ex.get('log_sha256') if record.get('log_sha256') else
                ↪ None,
                record.get('source_fields_sha256')==field.get('sha256') if
                ↪ record.get('source_fields_sha256') and field else None]
            return False if False in comparisons else True if any(x is not None for x in
            ↪ comparisons) else None
        parsed_attempt_matches=bound_attempt_matches(check)
        metric_attempt_matches=bound_attempt_matches(m)
        field_attempt_matches=bound_attempt_matches(field or {})
        bed_path=Path(case['erodible_bed']) if case.get('erodible_bed') else None
        bed_hash_matches=ins.check_bound({'path':str(bed_path),'bytes':bed_path.stat().st_size,
```

```
            'sha256':case['erodible_bed_sha256'],'role':'preregistered finite erodible-bed thickness input'}) if bed_path and bed_path.exists() and case.get('erodible_bed_sha256') else None
        stale_prior_retry=rid==retry_budget.get('run_id') and 'active at report time' in retry_budget.get('actual_execution_context','') and log_matches is False
        cancellation=cancellation_by_id.get(rid) or ins.js(C/'runs'/rid/'cancellation.json','separate retry cancellation receipt',False) or {}
        cancelled_retry_confirmed=bool(cancellation.get('status')=='cancelled_superseded_compute_priority'
            and cancellation.get('native_stop_returncode')==0 and cancellation.get('prior_attempt_execution_sha256')==execution_sha
            and cancellation.get('retry_log_sha256_at_cancellation')==file_digest(log_path)) if cancellation and log_path.exists() else False
        parsed_current=bool(check) and (not check.get('case_sha256') or check['case_sha256']==case_sha)
        if 'exit_code' in check:parsed_current=parsed_current and check['exit_code']==returncode
        parsed_current=parsed_current and execution_case_matches and log_matches is not False and parsed_attempt_matches is not False and metric_attempt_matches is not False and field_attempt_matches is not False
        field_case_matches=not field or field.get('case_sha256')==case_sha
        finite=truth(check.get('finite'));nonnegative=truth(check.get('nonnegative_to_1e-8'))
        if rid in excluded_ids and not attempted:status='not_run_material_applicability_exclusion'
        elif 'cancel' in str(ex.get('status','')).lower() or ex.get('cancelled'):status='cancelled'
        elif stale_prior_retry and cancelled_retry_confirmed:status='cancelled_superseded_compute_priority'
        elif stale_prior_retry:status='running_retry_with_stale_prior_attempt_metadata'
        elif quarantine:status='quarantined_configuration'
        elif not ex and attempted:status='running_unfinalized'
        elif not ex:status='planned_without_execution'
        elif returncode is None:status=str(ex.get('status') or 'missing_execution_result')
        elif returncode!=0:status='pipeline_failure'
        elif not parsed_current:status='finished_unanalysed'
        elif check.get('solver_success_field')==0 or check.get('log_contains_numerical_failure') or finite is False or nonnegative is False:status='numerical_failure'
        elif not complete:status='early_terminated_before_requested_horizon'
        elif not field_ok or not field_case_matches:status='field_missing_or_index_stale'
        else:status='completed'
        q2=case.get('upstream_fullmask_record') or case.get('upstream_release_record') or upstream.get(case.get('scenario_id'),{})
        onset=case.get('upstream_onset_status',q2.get('onset_status'))
        if_released=truth(case.get('if_released_propagation'))
        if onset=='never_crossed':if_released=True
        if if_released is None and stratum(case.get('evidence_role'))=='conditional_scenario':if_released=True
        geometry=case.get('geometry_version') or ('q4-r3-spatial-copdem-v1' if case.get('terrain_id')=='copdem' else None)
        qual='excluded_single_phase_surface2' if quarantine else 'conditional_native_execution_only' if status=='completed' else 'unqualified_'+status
        base={'run_id':rid,'producer':'official','model_id':case['model'],'scenario_id':case.get('scenario_id'),
          'base_scenario_id':case.get('base_scenario_id',case.get('scenario_id')),
          'parameter_version':case.get('upstream_parameter_version') or case.get('official_protocol_version','q4-r3-official-v1'),
          'geometry_version':geometry,'upstream_model_id':case.get('upstream_model_id',q2.get('model_id')),
```

```
        'upstream_parameter_version':case.get('upstream_parameter_version',q2.get('parameter_version')),
        'upstream_onset_status':onset,'release_mapping_version':case.get('release_mapping_version', 'q4-r3-q2-mapped-release-v1' if rid.startswith('q2') else 'common_reference'),
        'mechanism_id':q2.get('mechanism_id') or case.get('upstream_mechanism_id') or ('pore_pressure' if case['model']=='dclaw' else 'multiphase_coupling' if case['model']=='ravaflow_multi' else 'homogeneous_frictional_mixture'),
        'propagation_mechanism_id':'pore_pressure' if case['model']=='dclaw' else 'multiphase_coupling' if case['model']=='ravaflow_multi' else 'homogeneous_frictional_mixture',
        'experiment_id':'q2_release_transfer' if rid.startswith('q2') else 'domain_sensitivity' if 'extended' in rid else 'official_regional_or_benchmark',
        'evaluation_stratum':stratum(case.get('evidence_role')),'plan_scope':scope,
        'applicability_decision_version':applicability.get('version'),'applicability_class':applicability_class,
        'applicable_required':required,'completion_required':completion_required,
        'not_run_exclusion_but_execution_attempted':rid in excluded_ids and attempted,
        'comparison_model_subgroup':'composite_solid_water' if rid.startswith('composite_') else 'generic_three_material_proxy' if case['model']=='ravaflow_multi' else 'homogeneous_frictional_mixture' if case['model']=='ravaflow_single' else 'solid_fluid_porepressure' if case['model']=='dclaw' else case['model'],
        'comparison_reference_run_id':replacement.get('reference_run_id') if rid==replacement.get('new_run_id') else case.get('comparison_reference_run_id',case.get('reference_comparison_run_id',case.get('domain_comparison_parent_run_id'))),
        'ablation_scope':replacement.get('label') if rid==replacement.get('new_run_id') else None,
        'current_plan_sources':active_sources.get(rid,[]),'execution_attempted':attempted,
        'native_solver_started':True if stale_prior_retry else False if ex.get('solver_started') is False else None,
        'execution_status':status,'raw_execution_status':raw.get('status',ex.get('status')),
        'quality_gate':gate,'requested_horizon_seconds':requested,'actual_end_time_seconds':actual_end,
        'wall_seconds':ex.get('elapsed_seconds'),'initial_velocity_east_north_m_s':case.get('initial_velocity_east_north_m_s'),
        'initiation_to_release_delay_seconds':None,'if_released_propagation':if_released,
        'time_origin':'bulk_release','state_horizon_complete':complete,'pipeline_returncode':None if stale_prior_retry else returncode,
        'prior_attempt_returncode':returncode if stale_prior_retry else None,'current_retry_with_prior_execution_record':stale_prior_retry,
        'separate_cancellation_receipt':cancellation or None,
        'cancellation_receipt_hashes_confirmed':cancelled_retry_confirmed,
        'cancellation_is_numerical_failure':False if cancelled_retry_confirmed else None,
        'native_success_field':check.get('solver_success_field'),'finite_state':finite,'nonnegative_state':nonnegative,
        'fields_hash_matches_index':field_ok,'field_case_hash_matches':field_case_matches,'parsed_case_hash_matches':parsed_current,'field_reference':field,
        'numerical_qualification':qual,'include_in_scientific_parameter_comparison':status=='completed' and not quarantine and rid not in diagnostic_ids|excluded_ids and bed_hash_matches is not False,
        'include_in_generic_material_diagnostic_comparison':status=='completed' and rid in diagnostic_ids,
        'include_in_arrival_envelope':status=='completed' and not quarantine and stratum(case.get('evidence_role'))!='numerical_benchmark' and rid not in diagnostic_ids|excluded_ids and bed_hash_matches is not False,
        'event_timing_validated':False,'independent_event_validation_count':0,
        'regional_numerical_accuracy_qualified':False,'scientifically_adopted_for_prediction':False,
        'arrival_envelope_gate_scope':'execution, bound fields, actual time and boundary threshold only; scientific qualification still pending',
```

```
750           'csurface':csurface,'image_id':ex.get('image_id'),'container_id':ex.get('container_i
              ↪ d'),
751           'base_image_id':ex.get('base_image_id'),'runtime_image_identity_complete':bool(ex.ge
              ↪ t('image_id')),
752           'solver_build':case.get('solver_build','official_original_frozen_source'),
753           'resourcefix_parent_run_id':case.get('resourcefix_parent_run_id'),
754           'solver_binary_sha256':ex.get('resourcefix_binary_sha256'),
755           'historical_adapter_sha256_at_start':ex.get('adapter_sha256_at_start'),
756           'historical_adapter_hash_status':'recorded_at_execution_start' if
              ↪ ex.get('adapter_sha256_at_start') else 'unknown_no_contemporaneous_hash',
757           'entrainment_configuration':{**{k:v for k,v in case.items() if any(term in k.lower()
              ↪ for term in ['entrain','erodible','bed_scenario','max_available_bed'])},'native_
              ↪ extra_parameters':{k:v for k,v in case.get('extras',{}).items() if 'entrain' in
              ↪ k.lower()}},
758           'entrainment_bed_input_hash_matches':bed_hash_matches,
759           'entrainment_evidence_scope':'prescribed bounded scenario; added material requires
              ↪ separate native source and boundary ledger; no inference from domain mass
              ↪ change',
760           'execution_case_hash_matches':execution_case_matches,'execution_log_hash_matches':lo
              ↪ g_matches,
761           'parsed_execution_attempt_hash_matches':parsed_attempt_matches,'metric_execution_att
              ↪ empt_hash_matches':metric_attempt_matches,
762           'field_execution_attempt_hash_matches':field_attempt_matches,'execution_sha256':exec
              ↪ ution_sha,
763           'boundary_definition':'three-cell edge-store/absorbing native domain' if
              ↪ case['model'].startswith('ravaflow') else 'zero-gradient/extrapolation native
              ↪ ghost cells; signed net flow',
764           'parameters':{k:case.get(k) for k in
              ↪ ['resolution_m','release_thickness_m','mu_basal','xi','extras','drag','virtualma
              ↪ ss','cmelt','entrain','entrainment','entrainmax','entrainment_coefficient','cent
              ↪ rainment','erodible_bed','erodible_bed_sha256','max_available_bed_volume_m3','er
              ↪ odible_bed_density_kg_m3','phase_volume_fractions','densities_kg_m3','bulk_densi
              ↪ ty_kg_m3','material_volume_fractions','passive_material_volume_fractions']},
765           'raw_case_json':case,'raw_execution_json':ex,'raw_validation_json':check,'raw_metric
              ↪ s_json':m,'raw_runs_row_json':raw}
766         runs.append(base)
767         outmetrics.append({'run_id':rid,'model_id':case['model'],'scenario_id':base['scenario_
            ↪ id'],'solver_build':base['solver_build'],
768           'evaluation_stratum':base['evaluation_stratum'],'max_depth_m':number(m.get('max_dept
              ↪ h_over_saved_outputs_m')),
769           'speed_peak_m_s':number(m.get('max_speed_over_saved_outputs_m_s')),'speed_definition
              ↪ ':m.get('speed_definition'),
770           'speed_wet_depth_cutoff_m':.1,'horizontal_speed_peak_m_s':number(m.get('max_horizont
              ↪ al_speed_over_saved_outputs_m_s')),
771           'native_cumulative_speed_scalar_peak_m_s':number(m.get('native_global_cumulative_spe
              ↪ ed_scalar_m_s',m.get('native_cumulative_max_speed_m_s'))),
772           'native_cumulative_max_depth_m':number(m.get('native_cumulative_max_depth_m')),
773           'peak_sampling_definition':m.get('peak_sampling_definition'),'impact_area_m2':number
              ↪ (m.get('impact_area_m2')),
774           'impact_area_definition':'planar area where saved-output hmax>=.1m; undersamples
              ↪ native maxima',
775           'initial_volume_m3':check.get('first_output_volume_m3'),'final_domain_volume_m3':num
              ↪ ber(m.get('end_volume_m3')),
776           'initial_mass_kg':number(initial_history.get(rid,{}).get('mass_kg')),'final_domain_m
              ↪ ass_kg':number(m.get('end_mass_kg')),
777           'domain_mass_change_fraction':number(m.get('mass_change_fraction')),
778           'significant_mass_change_without_recorded_boundary_depth':abs(number(m.get('mass_cha
              ↪ nge_fraction')) or 0)>.01 and check.get('first_boundary_threshold_time_seconds')
              ↪ is None,
779           'initial_volume_relative_difference':check.get('initial_volume_relative_difference'),
780           'initial_depth_vs_supplied_max_difference_m':check.get('initial_depth_vs_supplied_ma
              ↪ x_difference_m'),
```

```
        'initial_depth_normalized_L1_difference':check.get('initial_depth_normalized_L1_difference'),
        'initial_supplied_support_cells':check.get('initial_supplied_support_cells'),'initial_native_support_cells':check.get('initial_native_support_cells'),
        'initial_effective_bed_vs_input_mae_m':check.get('initial_effective_bed_vs_input_mae_m'),
        'initial_effective_bed_vs_input_rmse_m':check.get('initial_effective_bed_vs_input_rmse_m'),
        'initial_effective_bed_vs_input_max_abs_m':check.get('initial_effective_bed_vs_input_max_abs_m'),
        'first_boundary_threshold_time_seconds':check.get('first_boundary_threshold_time_seconds'),
        'native_mass_budget_relative_residual':check.get('native_mass_budget_relative_residual'),
        'estimated_boundary_adjusted_mass_residual_fraction':check.get('estimated_boundary_adjusted_mass_residual_fraction'),
        'mass_budget_definition':check.get('boundary_estimate_scope') if case['model']=='dclaw' else check.get('native_budget_definition'),
        'native_entrainment_source_budget_evidence':{scope:{k:v for k,v in obj.items() if any(term in k.lower() for term in ['entrain','erosion','source_mass','source_volume','mass_budget','volume_budget'])} for scope,obj in [('validation',check),('metrics',m)]},
        'entrainment_reference_run_id':base['comparison_reference_run_id'] if case.get('erodible_bed') else None,
        'prescribed_erodible_bed_capacity_volume_m3':number(case.get('max_available_bed_volume_m3')),
        'prescribed_erodible_bed_density_kg_m3':number(case.get('erodible_bed_density_kg_m3')),
        'prescribed_erodible_bed_capacity_mass_kg':number(case['max_available_bed_volume_m3'])*number(case['erodible_bed_density_kg_m3']) if case.get('max_available_bed_volume_m3') is not None and case.get('erodible_bed_density_kg_m3') is not None else None,
        'native_conservation_verified':False if case['model']=='dclaw' else None,
        'boundary_flux_sampling_sensitivity':check.get('boundary_flux_sampling_sensitivity_residual_fractions'),
        'numerical_qualification':qual,'raw_metric_json':m})
by={x['run_id']:x for x in runs};arrivals=[]
additional=ins.csv(R/'results/spatial_targets_v2.csv','all currently registered OSM target representatives',False)
saved_arrival_cache={}
def independent_saved_check(base,row):
    """Bind an observer row to actual immutable NPZ time/depth, without solving."""
    import numpy as np
    rid=base['run_id'];tid=row.get('target',row.get('target_id'))
    if rid not in saved_arrival_cache:
        target_list=list(cases[rid].get('targets',[]))+[{**x,'id':x['target_id'],'xy':[float(x['x_m']),float(x['y_m'])],'buffer_m':100} for x in additional]
        with np.load(base['field_reference']['path'],allow_pickle=False) as a:
            t=a['time_s'];h=a['depth_frames'];tr=a['transform'];ny,nx=h.shape[1:]
            ins.note_array_read(base['field_reference']['path'],'NumPy time_s/depth_frames/transform loaded to independently re-evaluate registered target masks and saved-output arrival intervals')
            rr,cc=np.indices((ny,nx));xx=tr[0]*(cc+.5)+tr[1]*(rr+.5)+tr[2];yy=tr[3]*(cc+.5)+tr[4]*(rr+.5)+tr[5]
            found={}
            for target in target_list:
                mask=(xx-target['xy'][0])**2+(yy-target['xy'][1])**2<=target.get('buffer_m',100)**2
                hit=np.flatnonzero((h[:,mask]>=.1).any(axis=1)) if mask.any() else []
                k=int(hit[0]) if len(hit) else None
                found[target['id']]={'outside':not bool(mask.any()),'time':float(t[k]) if k is not None else None,
```

```
                        'lower':float(t[k-1]) if k is not None and k>0 else 0. if k==0 and t[0]==0 else None,'actual_end':float(t[-1]),
                        'last_saved_output_at_or_before_requested_horizon':float(t[t<=base['requested_horizon_seconds']][-1]) if (t<=base['requested_horizon_seconds']).any() else None}
            saved_arrival_cache[rid]=found
        expected=saved_arrival_cache[rid].get(tid)
        if expected is None:return False,'target_missing_from_preregistered_geometry',None
        raw_time=number(row.get('arrival_seconds',row.get('arrival_time_seconds')))
        time_match=(raw_time==expected['time'])
        observer_end=number(row.get('evaluation_end_time_seconds'))
        end_match=observer_end is None or abs(observer_end-expected['actual_end'])<.01
        run_end_match=abs(expected['actual_end']-(base['actual_end_time_seconds'] or -1))<.01
        return bool(time_match and end_match and run_end_match),'independently_recomputed_from_hashed_saved_depth_and_actual_times',expected
    for row in raw_arrivals+extra:
        rid=row['run_id']
        if rid not in by:continue
        base=by[rid];check=base['raw_validation_json'];raw_time=number(row.get('arrival_seconds',row.get('arrival_time_seconds')))
        observed=raw_time is not None;boundary=number(row.get('first_boundary_threshold_time_seconds',check.get('first_boundary_threshold_time_seconds')))
        affected=boundary is not None and (raw_time is None or boundary<raw_time)
        target_field_matches=row.get('source_fields_sha256')==fields.get(rid,{}).get('sha256') if row.get('source_fields_sha256') else None
        target_attempt_matches=(not row.get('execution_sha256') or row['execution_sha256']==base.get('execution_sha256')) and (not row.get('log_sha256') or row['log_sha256']==base['raw_execution_json'].get('log_sha256'))
        binding_method='observer_explicit_fields_sha256' if target_field_matches is not None else 'missing_observer_field_hash'
        expected=None
        if base['execution_status']=='completed' and base['field_reference']:
            independently_matches,binding_method,expected=independent_saved_check(base,row)
            target_field_matches=independently_matches and target_field_matches is not False and target_attempt_matches
        raw_status=row.get('status');valid=base['execution_status']=='completed' and target_field_matches
        if not valid:status=base['execution_status'];travel=None
        elif raw_status in ['out_of_domain','outside_domain','missing']:status='outside_domain' if 'domain' in raw_status else 'missing';travel=None
        elif observed:status='arrived';travel=raw_time
        else:status='not_arrived_by_horizon';travel=None
        arrivals.append({'run_id':rid,'scenario_id':base['scenario_id'],'base_scenario_id':base['base_scenario_id'],
          'model_id':base['model_id'],'upstream_model_id':base['upstream_model_id'],'upstream_parameter_version':base['upstream_parameter_version'],
          'solver_build':base['solver_build'],
          'upstream_onset_status':base['upstream_onset_status'],'if_released_propagation':base['if_released_propagation'],
          'geometry_version':base['geometry_version'],'target_geometry_version':row.get('geometry_version') or 'q4-r3-spatial-copdem-v1',
          'release_mapping_version':base['release_mapping_version'],'parameter_version':base['parameter_version'],
          'target_id':row.get('target',row.get('target_id')),'entity_id':row.get('entity_id'),'target_kind':row.get('target_kind'),
          'raw_status':raw_status,'arrival_status':status if target_field_matches is not False else 'stale_target_field_reference','observed_hit':observed,'observed_hit_time_seconds':raw_time,
          'travel_time_seconds':travel,'arrival_bracket_lower_seconds':expected['lower'] if expected and valid else number(row.get('previous_below_threshold_time_seconds')),
```

```
          'arrival_bracket_upper_seconds':raw_time,'arrival_definition':row.get('arrival_defin⌋
          ↪  ition'),
          'requested_horizon_seconds':base['requested_horizon_seconds'],'evaluation_end_time_s⌋
          ↪  econds':base['actual_end_time_seconds'],
          'first_boundary_threshold_time_seconds':boundary,'domain_affected_before_arrival':af⌋
          ↪  fected,
          'include_in_arrival_envelope':bool(base['include_in_arrival_envelope'] and not
          ↪  affected and status in ['arrived','not_arrived_by_horizon']),
          'numerical_qualification':base['numerical_qualification'],'evaluation_stratum':base[⌋
          ↪  'evaluation_stratum'],
          'regional_numerical_accuracy_qualified':False,'scientifically_adopted_for_prediction⌋
          ↪  ':False,
          'arrival_envelope_gate_scope':base['arrival_envelope_gate_scope'],
          'event_timing_validated':False,'initiation_to_release_delay_seconds':None,'time_orig⌋
          ↪  in':'bulk_release',
          'probability_weight':None,'target_field_hash_matches':target_field_matches,'target_b⌋
          ↪  inding_method':binding_method,
          'independent_saved_field_arrival_check':expected,'source_fields_sha256':fields.get(r⌋
          ↪  id,{}).get('sha256'),'raw_source_row_json':row})
    # Preserve target rows for planned/unanalysed/failure cases with no parsed output.
    seen={(x['run_id'],x['target_id']) for x in arrivals}
    for rid,base in by.items():
        target_list=list(cases[rid].get('targets',[]))
        if base['evaluation_stratum']=='conditional_scenario':target_list +=
        ↪  [{**x,'id':x['target_id']} for x in additional]
        for target in target_list:
            if (rid,target['id']) in seen:continue
            arrivals.append({'run_id':rid,'scenario_id':base['scenario_id'],'base_scenario_id'⌋
            ↪  :base['base_scenario_id'],
              'model_id':base['model_id'],'target_id':target['id'],'entity_id':target.get('ent⌋
              ↪  ity_id'),'target_kind':target.get('target_kind'),
              'target_geometry_version':target.get('geometry_version') or
              ↪  'q4-r3-spatial-copdem-v1','geometry_version':base['geometry_version'],
              'parameter_version':base['parameter_version'],'upstream_onset_status':base['upst⌋
              ↪  ream_onset_status'],
              'if_released_propagation':base['if_released_propagation'],'arrival_status':base[⌋
              ↪  'execution_status'],
              'observed_hit':False,'observed_hit_time_seconds':None,'travel_time_seconds':None,
              'arrival_definition':'no parsed saved fields; target definition retained in raw
              ↪  case',
              'requested_horizon_seconds':base['requested_horizon_seconds'],'evaluation_end_ti⌋
              ↪  me_seconds':base['actual_end_time_seconds'],
              'include_in_arrival_envelope':False,'numerical_qualification':base['numerical_qu⌋
              ↪  alification'],
              'evaluation_stratum':base['evaluation_stratum'],'initiation_to_release_delay_sec⌋
              ↪  onds':None,'time_origin':'bulk_release',
              'event_timing_validated':False,'probability_weight':None})
    return runs,outmetrics,arrivals,validation,active

def archived_attempts(ins,runs,arrivals):
    """Keep distinct adapter/native attempts; a duplicate archive is one attempt."""
    by={x['run_id']:x for x in runs};extra=[];extra_arrivals=[]
    def identity(ex):
        ex=ex.get('original_execution_record') or ex
        return canonical({k:ex.get(k) for k in ['command','started_utc','case_sha256']})
    seen={identity(x['raw_execution_json']) for x in runs if x.get('raw_execution_json') and
    ↪  not x.get('current_retry_with_prior_execution_record')}
    for path in sorted((C/'runs').glob('*/attempts/*/execution.json')):
        ex=ins.js(path,'archived actual execution attempt');key=identity(ex)
        archive_log=path.parent/'run.log'
        if archive_log.exists():
```

```
            ins.check_bound({'path':str(archive_log),'bytes':archive_log.stat().st_size,'sha25
            ↪  6':ex.get('log_sha256',file_digest(archive_log)),'role':'archived attempt
            ↪  log'})
        if key in seen:continue
        seen.add(key);parent=path.parents[2].name;base=by.get(parent)
        if not base:raise RuntimeError('Archived execution has no case inventory: '+str(path))
        rid=parent+'::attempt::'+path.parent.name
        row={**base,'run_id':rid,'record_kind':'archived_execution_attempt','parent_case_run_i
        ↪  d':parent,
            'plan_scope':'historical_preparation','execution_attempted':True,
            'native_solver_started':False if 'mapcalc_adapter_failure' in path.parent.name else
            ↪  ex.get('solver_started'),
            'execution_status':'pipeline_failure' if ex.get('returncode') else
            ↪  'historical_attempt_unanalysed',
            'pipeline_returncode':ex.get('returncode'),'current_retry_with_prior_execution_rec
            ↪  ord':False,
            'prior_attempt_returncode':None,'actual_end_time_seconds':None,'state_horizon_comp
            ↪  lete':False,
            'wall_seconds':ex.get('elapsed_seconds'),'field_reference':None,'fields_hash_match
            ↪  es_index':False,
            'include_in_scientific_parameter_comparison':False,'include_in_arrival_envelope':F
            ↪  alse,
            'numerical_qualification':'archived_adapter_failure' if 'mapcalc_adapter_failure'
            ↪  in path.parent.name else 'historical_attempt_unqualified',
            'raw_execution_json':ex,'raw_validation_json':{},'raw_metrics_json':{},'raw_runs_r
            ↪  ow_json':{},
            'archived_attempt_metadata_path':str(path),'image_id':ex.get('image_id'),'base_ima
            ↪  ge_id':ex.get('base_image_id'),
            'execution_sha256':digest(ins.bytes[str(path.resolve())]),'execution_log_hash_matc
            ↪  hes':file_digest(archive_log)==ex.get('log_sha256') if archive_log.exists()
            ↪  else None}
        extra.append(row)
        for target in [x for x in arrivals if x['run_id']==parent]:
            extra_arrivals.append({**target,'run_id':rid,'arrival_status':row['execution_statu
            ↪  s'],'observed_hit':False,
                'observed_hit_time_seconds':None,'travel_time_seconds':None,'arrival_bracket_l
                ↪  ower_seconds':None,
                'arrival_bracket_upper_seconds':None,'include_in_arrival_envelope':False,'eval
                ↪  uation_end_time_seconds':None,
                'numerical_qualification':row['numerical_qualification'],'raw_source_row_json'
                ↪  :{},'target_field_hash_matches':None})
    return extra,extra_arrivals

def benchmarks(ins):
    evidence=ins.js(R/'results/reduced_numerical_validation.json','frozen reduced numerical
    ↪  benchmarks');rows=[]
    for case in evidence['cases']:
        for i,run in enumerate(case.get('runs',[{'summary':case.get('summary',{})}])):
            s=run.get('summary',{});rows.append({'run_id':f"reduced_benchmark::{case['case_id'
            ↪  ]}::{i}",
              'producer':'reduced','model_id':'independent_rigid_ode' if 'rigid' in
              ↪  case['case_id'] else 'independent_fv_numerical_benchmark',
              'scenario_id':case['case_id'],'evaluation_stratum':'numerical_benchmark','plan_s
              ↪  cope':'current_registered',
              'execution_attempted':True,'execution_status':s.get('execution_status','complete
              ↪  d_configuration'),
              'actual_end_time_seconds':s.get('duration_actual_seconds'),'requested_horizon_se
              ↪  conds':s.get('duration_requested_seconds'),
              'wall_seconds':s.get('wall_seconds'),'check_pass':case['pass'],'numerical_qualif
              ↪  ication':'declared_property_check_only',
              'include_in_arrival_envelope':False,'event_timing_validated':False,'raw_case_jso
              ↪  n':case,'raw_config_json':run})
```

```
    for p in sorted((C/'benchmarks').glob('**/*execution.json')):
        ex=ins.js(p,'external official benchmark execution');name=ex.get('name',p.parent.name)
        rows.append({'run_id':'official_external_benchmark::'+str(p.relative_to(C/'benchmarks'
        ↪  )),
          'producer':'official_external_benchmark','model_id':'dclaw' if 'gully' in name else
          ↪  'ravaflow',
          'scenario_id':name,'evaluation_stratum':'numerical_benchmark','plan_scope':'current_
          ↪  registered',
          'execution_attempted':True,'execution_status':'completed_pipeline_only' if
          ↪  ex.get('returncode')==0 else 'pipeline_failure',
          'pipeline_returncode':ex.get('returncode'),'wall_seconds':ex.get('elapsed_seconds'),
          'include_in_arrival_envelope':False,'event_timing_validated':False,'numerical_qualif
          ↪  ication':'inspect_official_benchmark_fields_separately','raw_execution_json':ex})
    return rows,evidence

def within_framework_pairs(runs,metrics):
    mm={x['run_id']:x for x in metrics};pairs=[]
    eligible=[x for x in runs if x.get('evaluation_stratum')=='conditional_scenario' and
    ↪  x.get('execution_status')=='completed' and x.get('plan_scope')=='current_registered']
    def group_key(x):return (x['model_id'],x.get('comparison_model_subgroup',x['model_id']),x.
    ↪  get('solver_build','independent_frozen_r3'))
    for (model,subgroup,build),items in
    ↪  itertools.groupby(sorted(eligible,key=group_key),key=group_key):
        group=list(items)
        explicit_refs={x.get('comparison_reference_run_id') for x in group}
        refs=[x for x in group if re.search(r'(?:_ref(?:_|$)|_P[123]_ref$)',x['run_id']) or
        ↪  x['run_id'] in explicit_refs]
        for ref in refs:
            for other in group:
                if other['run_id']==ref['run_id']:continue
                # Same release/base key or explicit domain-parent comparison; never a
                ↪  cross-framework gain.
                parent=other.get('comparison_reference_run_id') or
                ↪  other.get('raw_case_json',{}).get('domain_comparison_parent_run_id')
                same_release_key=other.get('base_scenario_id')==ref.get('base_scenario_id')
                common_ofat=ref.get('release_mapping_version')=='common_reference' and
                ↪  other.get('release_mapping_version')=='common_reference'
                if not same_release_key and parent!=ref['run_id'] and not common_ofat:continue
                if ref.get('path_id')!=other.get('path_id'):continue
                a=mm.get(ref['run_id'],{});b=mm.get(other['run_id'],{})
                pair={'model_id':model,'comparison_model_subgroup':subgroup,'solver_build':bui
                ↪  ld,'reference_run_id':ref['run_id'],'other_run_id':other['run_id'],
                  'ablation_scope':other.get('ablation_scope'),
                  'comparison_scope':'within-framework conditional sensitivity; not calibrated
                  ↪  model gain',
                  'scientifically_qualified_for_prediction':False,'mass_conservation_qualified
                  ↪  ':False,
                  'reference_conditional_diagnostic_range_eligible':ref.get('conditional_diagn
                  ↪  ostic_range_eligible'),
                  'other_conditional_diagnostic_range_eligible':other.get('conditional_diagnos
                  ↪  tic_range_eligible'),
                  'reference_parameters':ref.get('parameters'),'other_parameters':other.get('p
                  ↪  arameters'),
                  'same_geometry_version':ref.get('geometry_version')==other.get('geometry_ver
                  ↪  sion'),
                  'same_base_scenario_key':same_release_key,
                  'same_initial_volume_within1e_minus6':abs(a.get('initial_volume_m3',0)-b.get
                  ↪  ('initial_volume_m3',0))/max(abs(a.get('initial_volume_m3') or 0),1)<1e-6
                  ↪  if a.get('initial_volume_m3') is not None and b.get('initial_volume_m3')
                  ↪  is not None else None,
```

```
                  'same_initial_mass_within1e_minus6':abs(a.get('initial_mass_kg',0)-b.get('in
                  ↪  itial_mass_kg',0))/max(abs(a.get('initial_mass_kg') or 0),1)<1e-6 if
                  ↪  a.get('initial_mass_kg') is not None and b.get('initial_mass_kg') is not
                  ↪  None else None,
                  'changed_parameter_fields':[k for k in
                  ↪  set(ref.get('parameters',{}))|set(other.get('parameters',{})) if
                  ↪  ref.get('parameters',{}).get(k)!=other.get('parameters',{}).get(k)],
                  'ref_numerical_qualification':ref.get('numerical_qualification'),'other_nume
                  ↪  rical_qualification':other.get('numerical_qualification')}
                for key in ['max_depth_m','impact_area_m2','stationary_residual_area_m2','stat
                ↪  ionary_residual_volume_m3']:
                    av,bv=number(a.get(key)),number(b.get(key));pair[key+'_difference']=bv-av
                    ↪  if av is not None and bv is not None else None
                pairs.append(pair)
    return pairs

def decorate_time_contract(row):
    """Separate actual saved-output detection from the requested evaluation horizon."""
    upper=number(row.get('observed_hit_time_seconds'));lower=number(row.get('arrival_bracket_l
    ↪  ower_seconds'))
    horizon=number(row.get('requested_horizon_seconds'));travel=number(row.get('travel_time_se
    ↪  conds'))
    valid=row.get('arrival_status')=='arrived' and travel is not None
    row['arrival_interval_definition']='between previous saved output below threshold and first
    ↪  saved output above; no solver-step precision claim'
    if upper==0:
        row['arrival_bracket_lower_seconds']=0.;row['arrival_bracket_upper_seconds']=0.
        row['arrival_interval_type']='initial_condition_already_above_threshold'
    elif upper is not None and lower is None:
        row['arrival_interval_type']='left_censored_at_first_saved_output'
    else:row['arrival_interval_type']='saved_output_detection_interval' if upper is not None
    ↪  else 'right_censored_or_missing_as_status'
    row['arrival_within_requested_horizon']=bool(valid and horizon is not None and
    ↪  travel<=horizon) if valid else None
    row['formal_requested_travel_time_seconds']=travel if
    ↪  row['arrival_within_requested_horizon'] is True else None
    if valid and horizon is not None and travel>horizon:
        row['formal_requested_horizon_status']='arrival_interval_straddles_requested_horizon'
        ↪  if lower is None or lower<horizon else
        ↪  'not_arrived_at_saved_outputs_through_requested_horizon'
        row['include_in_arrival_envelope']=False
    elif valid:row['formal_requested_horizon_status']='arrived_by_requested_horizon'
    elif row.get('arrival_status')=='not_arrived_by_horizon':row['formal_requested_horizon_sta
    ↪  tus']='not_arrived_at_saved_outputs_through_requested_horizon'
    else:row['formal_requested_horizon_status']='unqualified_or_missing_observation'
    row['requested_horizon_claim_scope']='formal requested horizon fields; actual evaluation
    ↪  end may exceed requested time'

def early_export(ins,runs,metrics,arrivals,numerical,summary,version,selection='dclaw'):
    """Freeze a selected completed partial packet; rolling root tables stay provisional."""
    is_delta=selection=='q2compactdelta'
    if not re.fullmatch(r'compact_delta_v[0-9]+' if is_delta else
    ↪  r'propagation_v[0-9]+',version):raise ValueError('compact delta requires
    ↪  compact_delta_vN; other early packets require propagation_vN')
    dest=R/'exports'/version
    if dest.exists():raise FileExistsError('Never overwrite an early or final frozen version')
    packet_status={'q2mapped':'frozen_partial_q2mapped_for_adapter_review','q2fullmask':'froze
    ↪  n_partial_q2fullmask_for_adapter_review','q2compactdelta':'frozen_partial_q2compact_de
    ↪  lta_for_adapter_review','dclaw':'frozen_partial_for_q3_review'}[selection]
    q2_selection=selection in ['q2mapped','q2fullmask','q2compactdelta']
```

```
    q2_review=ins.js(R/'config/q2_compact_quality_review_04.json','registered compact-source numerical qualification review',selection=='q2mapped') if selection=='q2mapped' else None
    requested_ids=set(q2_review['run_ids']) if q2_review else None
    if selection=='q2fullmask':
        plan=ins.js(R/'config/official_cases_q2fullmask_v2.json','registered fullmask S60/ST60/ST120 early handoff selector')
        if not FULLMASK_PARTIAL_IDS<={x['run_id'] for x in plan['cases']}:raise RuntimeError('Required three fullmask runs are not all registered')
        requested_ids=set(FULLMASK_PARTIAL_IDS)
    if is_delta:
        plan=ins.js(R/'config/official_cases_q2mapped_v2.json','registered original compact Q2 mapping cases; delta does not create new model cases')
        if not COMPACT_DELTA_IDS<={x['run_id'] for x in plan['cases']}:raise RuntimeError('Both compact Q2 delta cases must already be registered')
        requested_ids=set(COMPACT_DELTA_IDS)
    selected=[dict(x,integration_status=packet_status) for x in runs
        if x.get('producer')=='official' and (x['run_id'] in requested_ids if requested_ids is not None else x.get('model_id')=='dclaw')
        and x.get('evaluation_stratum')=='conditional_scenario' and x.get('execution_status')=='completed']
    if not selected or requested_ids is not None and {x['run_id'] for x in selected}!=requested_ids:raise RuntimeError('Selected partial packet cases are not all completed/parsed/hash-bound')
    ids={x['run_id'] for x in selected}
    selected_metrics=[dict(x,integration_status=packet_status) for x in metrics if x['run_id'] in ids]
    selected_arrivals=[dict(x,integration_status=packet_status) for x in arrivals if x['run_id'] in ids]
    if selection=='q2fullmask' or is_delta:
        if len(selected_arrivals)!=7*len(ids) or any(sum(x['run_id']==rid for x in selected_arrivals)!=7 for rid in ids):
            raise RuntimeError('Q2 lineage handoff requires all seven targets for every selected run')
        expected_targets={'paper_port_symbol','paper_junction'}|{x['target_id'] for x in ins.csv(R/'results/spatial_targets_v2.csv','fullmask early packet registered target identity check')}
        if any({x['target_id'] for x in selected_arrivals if x['run_id']==rid}!=expected_targets for rid in ids):
            raise RuntimeError('Q2 lineage handoff target identity set must match two paper targets plus five OSM representatives')
    if not all(x.get('target_field_hash_matches') is True for x in selected_arrivals):
        raise RuntimeError('Early package requires every selected target to match immutable saved fields')
    # Only selected case/attempt/log/fields and physical input dependencies gate this partial export.
    external=[];metadata=[];metadata_origins={}
    controls_packet=ins.js(R/'results/official_native_control_index.json','actual native controls index for selected early packet')
    controls_by={x['run_id']:x for x in controls_packet['runs']}
    for run in selected:
        rid=run['run_id'];case=run['raw_case_json'];ex=run['raw_execution_json']
        for path,sha,role in [(case.get('dem'),case.get('dem_sha256'),'immutable native input terrain'),
                              (case.get('release'),case.get('release_sha256'),'immutable native release thickness')]:
            if path and sha:
                p=Path(path)
                if not p.exists() or not ins.check_bound({'path':path,'bytes':p.stat().st_size,'sha256':sha,'role':role}):
                    raise RuntimeError('Selected physical input hash mismatch: '+str(path))
```

```
            external.append({**ins.records[str(p.resolve())],'storage':'immutable_external_cache'})
        for p,role in [(C/'runs'/rid/'case.json','native case'),(C/'runs'/rid/'execution.json','native execution record')]:
            b=ins.read(p,role)
            if digest(p.read_bytes())!=digest(b):raise RuntimeError('Selected metadata changed during early snapshot: '+str(p))
            metadata.append((Path('provenance')/rid/p.name,b,role))
        field=run['field_reference'];p=Path(field['path'])
        if not ins.check_bound(field) or file_digest(p)!=field['sha256']:raise RuntimeError('Selected field bytes changed: '+str(p))
        external.append({**field,'storage':'immutable_external_cache'})
        if ex.get('log_path'):
            p=Path(ex['log_path'])
            if not ex.get('log_sha256') or not p.exists() or file_digest(p)!=ex['log_sha256']:
                raise RuntimeError('Selected execution log no longer matches actual attempt')
            external.append({'path':str(p),'bytes':p.stat().st_size,'sha256':ex['log_sha256'],
                             'role':'immutable native execution log','storage':'immutable_external_cache'})
            if q2_selection:
                relative=Path('provenance')/rid/'run.log';metadata.append((relative,p.read_bytes(),'actual native execution log snapshot'));metadata_origins[str(relative)]=str(p)
        if q2_selection:
            if rid not in controls_by:raise RuntimeError('Selected native control index missing case: '+rid)
            for control in controls_by[rid]['native_input_controls']:
                if not ins.check_bound(control):raise RuntimeError('Selected actual native control hash mismatch: '+control['path'])
                p=Path(control['path']);b=p.read_bytes()
                if len(b)<=256*1024:
                    try:b.decode('utf-8')
                    except UnicodeDecodeError:external.append({**control,'storage':'immutable_external_cache'})
                    else:
                        relative=Path('provenance')/rid/'native_controls'/p.name;metadata.append((relative,b,control.get('role','actual native control text')));metadata_origins[str(relative)]=str(p)
                else:external.append({**control,'storage':'immutable_external_cache'})
            saved=controls_by[rid].get('saved_executed_adapter_path');saved_sha=controls_by[rid].get('saved_executed_adapter_sha256')
            if saved and saved_sha:
                p=Path(saved)
                if file_digest(p)!=saved_sha:raise RuntimeError('Saved actual adapter hash mismatch')
                relative=Path('provenance')/rid/'adapter_executed.py';metadata.append((relative,p.read_bytes(),'saved actual executed adapter; contemporaneous hash retained'));metadata_origins[str(relative)]=str(p)
            parser=run['raw_validation_json'].get('parser_source_snapshot_path');parser_sha=run['raw_validation_json'].get('parser_sha256_at_start')
            if parser and parser_sha:
                p=Path(parser)
                if file_digest(p)!=parser_sha:raise RuntimeError('Selected actual parser snapshot hash mismatch')
                relative=Path('provenance/postprocessing')/p.name;metadata.append((relative,p.read_bytes(),'actual parser source snapshot bound to selected process fields'));metadata_origins[str(relative)]=str(p)
            check=run['raw_validation_json']
            if check.get('native_summary_path'):
                p=Path(check['native_summary_path'])
                if file_digest(p)!=check.get('native_summary_sha256'):raise RuntimeError('Selected native internal summary hash mismatch')
```

```
                relative=Path('provenance')/rid/'native_internal_summary.txt';metadata.append(
                ↪  (relative,p.read_bytes(),'actual native internal unscaled volume
                ↪  summary'));metadata_origins[str(relative)]=str(p)
                for nout in p.parent.glob('*nout1.txt'):
                    relative=Path('provenance')/rid/nout.name;metadata.append((relative,nout.r
                    ↪  ead_bytes(),'actual native nout success/source/edge accounting
                    ↪  output'));metadata_origins[str(relative)]=str(nout)
    fields=list({x['path']:x for x in external}.values())
    selected_by={x['run_id']:x for x in selected}
    for row in selected_arrivals:
        row['source_snapshot_stable']=True
        row['source_snapshot_stability_scope']='selected run case/execution/log/native
        ↪  input/field hashes; unrelated rolling observer tables may advance'
        row['include_in_arrival_envelope']=bool(selected_by[row['run_id']]['include_in_arrival
        ↪  _envelope'] and not row.get('domain_affected_before_arrival') and
        ↪  row['arrival_status'] in ['arrived','not_arrived_by_horizon'])
    attach_quality_contract(ins,selected,selected_metrics,selected_arrivals)
    if selection=='q2mapped':
        apply_q2_compact_quality(ins,selected,selected_metrics,selected_arrivals)
        if not all(x.get('source_quality_evidence_binding_matches') is True for x in
        ↪  selected):raise RuntimeError('Selected compact-source budget evidence must match
        ↪  actual attempt')
        for name in ['config/q2_compact_quality_review_04.json',q2_review['evidence']['path'].
        ↪  removeprefix('track_b/research/q4/round3/')]:
            source=R/name;relative=Path('quality')/source.name
            metadata.append((relative,ins.read(source,'selected compact-source numerical
            ↪  qualification evidence'),'compact source quality
            ↪  review/evidence'));metadata_origins[str(relative)]=str(source)
    if selection=='q2mapped' or is_delta:
        upstream=R/'inputs/q2/release_bundle_q2-r3-release-v1'
        for source in [upstream/'bundle.json',upstream/'upstream_bundle.json',upstream/'files/
        ↪  results/release_states.csv']:
            relative=Path('provenance/upstream_q2')/source.name
            metadata.append((relative,ins.read(source,'actual delivered Q2 compact release
            ↪  lineage'),'delivered Q2 compact source
            ↪  lineage'));metadata_origins[str(relative)]=str(source)
        if is_delta:
            source=R/'config/official_cases_q2mapped_v2.json';relative=Path('provenance/compac
            ↪  t_registration')/source.name
            metadata.append((relative,ins.read(source,'original compact Q2 registration; this
            ↪  packet adds no new native runs'),'original compact Q2 native case
            ↪  registration'));metadata_origins[str(relative)]=str(source)
    if selection=='q2fullmask':
        ready_path=R/'results/official_q2full_ready_receipt.json'
        ready=ins.js(ready_path,'owner exactready receipt for the three fullmask native
        ↪  attempts and fixed-parser target rows')
        ready_bytes=ins.bytes[str(ready_path.resolve())]
        if digest(ready_bytes)!='33b78cf3a25da72acfaf0462c74cd1fd8de8004d1c485561f3e6dc8d33cca
        ↪  f25':
            raise RuntimeError('Authorized fullmask exactready receipt bytes changed')
        if not ready.get('native_execution_and_postprocessing_ready') or
        ↪  ready.get('selected_case_count')!=3 or ready.get('selected_target_rows')!=21:
            raise RuntimeError('Fullmask receipt does not declare the authorized
            ↪  three-case/21-target ready scope')
        ready_by={x['run_id']:x for x in ready['records']}
        if set(ready_by)!=ids:raise RuntimeError('Fullmask ready receipt selected IDs
        ↪  mismatch')
        for run in selected:
            rid=run['run_id'];rec=ready_by[rid]
            if (rec['source_fields_sha256']!=run['field_reference']['sha256']
                or rec['case_sha256']!=digest(ins.read(C/'runs'/rid/'case.json','actual native
                ↪  case for exactready binding'))
```

```
                or rec['execution_sha256']!=run['execution_sha256']
                or rec['log_sha256']!=run['raw_execution_json']['log_sha256']
                or abs(rec['actual_last_time_s']-run['actual_end_time_seconds'])>1e-9
                or ready['parser_sha256']!=run['raw_validation_json']['parser_sha256_at_start']):
                raise RuntimeError('Fullmask exactready receipt disagrees with selected actual attempt: '+rid)
        relative=Path('provenance/fullmask_registration')/ready_path.name
        metadata.append((relative,ready_bytes,'owner exactready receipt bound to actual three native attempts, fixed parser and21 targets'));metadata_origins[str(relative)]=str(ready_path)
        upstream=R/'inputs/q2/release_bundle_v2_q2-r3-release-v2-spatial-copdem-v1'
        for relative_source in ['bundle.json','upstream_bundle.json','files/results/release_states_geometry_v2.csv','files/results/geometry_integration_summary.json','files/results/geometry_integration_terrain.csv','files/GEOMETRY_INTEGRATION.md']:
            source=upstream/relative_source;relative=Path('provenance/upstream_q2_fullmask')/Path(relative_source).name
            metadata.append((relative,ins.read(source,'actual delivered Q2 fullmask geometry-v2 release lineage'),'delivered Q2 fullmask geometry-v2 lineage'));metadata_origins[str(relative)]=str(source)
        for source in [R/'config/official_cases_q2fullmask_v2.json',R/'config/q2_fullmask_integration_protocol.json',R/'config/official_q2_v2_protocol.json']:
            if source.exists():
                relative=Path('provenance/fullmask_registration')/source.name;metadata.append((relative,ins.read(source,'registered fullmask mapping and native execution protocol'),'registered fullmask protocol'));metadata_origins[str(relative)]=str(source)
    if q2_selection:
        decision=summary['scientific_acceptance_decision']
        quality_sources=[R/'config/scientific_acceptance_decision_05.json',ROOT/decision['evidence']['path']]
        evidence=ins.js(ROOT/decision['evidence']['path'],'qualification checkpoint evidence05 for partial review')
        quality_sources += [ROOT/x['path'] for x in evidence.get('fixed_evidence',[])]
        for source in quality_sources:
            relative=Path('quality')/source.name;metadata.append((relative,ins.read(source,'scientific acceptance decision05 and fixed counterevidence snapshot'),'scientific acceptance and qualification evidence'));metadata_origins[str(relative)]=str(source)
        for name in summary.get('scheduling_resource_context',{}).get('source_paths',[]):
            source=ROOT/name;relative=Path('provenance/scheduling')/source.name
            metadata.append((relative,ins.read(source,'actual mixed concurrency/CPU quota context snapshot'),'scheduling/resource context; no controlled performance benchmark'));metadata_origins[str(relative)]=str(source)
    apply_scientific_acceptance(summary['scientific_acceptance_decision'],selected,selected_metrics,selected_arrivals)
    specific_limits=[
        'Only completed Q2 compact p0000/p0036 mapped-release diagnostics; full-source geometry and other required native cases remain outside this partial packet',
        'All selected runs and targets excluded from arrival envelope; computed raw travel and saved-output brackets retained solely for adapter/diagnostic review',
        'Compact supports are8/16 raster cells and2.7778/11.1111 equivalent full cells; source underresolution and mass discrepancy remain unresolved',
        'ASCII source-adjusted budget residuals +21.1534%/+12.6130% and separate native-internal summary residuals +21.3729%/+12.7163%; no tolerance declared to certify pass',
        'Upstream never_crossed remains if-released propagation; absolute bulk-release UTC and initiation-to-release delay remain unknown'] if selection=='q2mapped' else [
        'Only the two already executed compact geometry-v1 p0072 S_H and p0255 S_T_H parent mappings are included; this packet adds no new model cases or independent events',
```

```
        'Original Q2 v1 release catalogue, actual case/execution/log/fields/native controls and all14 target rows are retained; this delta does not duplicate the earlier v2/v3 cases',
        'Decision05 accepted-run list remains empty; all selected arrival-envelope, regional accuracy, mass-conservation and predictive qualification flags remain false',
        'Compact-source budget discrepancies are unresolved; ASCII and native-internal residuals remain separate diagnostics, with no passing tolerance declared',
        'Travel/brackets are relative to modeled bulk release; initiation-to-release delay and absolute release UTC remain unknown'] if is_delta else [
        'Only preregistered fullmask geometry-v2 S60/ST60/ST120 scenarios are included; this partial lineage handoff does not imply all-model completion',
        'Source masks and per-grid depth normalization are the delivered full-source geometry-v2 mapping, distinct from the compact-source v1 mapping',
        'Decision05 accepted-run list is empty: all selected target arrival envelope, regional accuracy, mass-conservation and prediction qualification flags remain false',
        'Computed travel/brackets, actual end/requested horizon and independent execution diagnostics are retained; no percentage residual is certified as passing',
        'Upstream never_crossed remains conditional if-released propagation; absolute bulk-release UTC and initiation-to-release delay are unknown'] if selection=='q2fullmask' else [
        'Early partial DClaw packet only; all-route and Q2 mapping comparisons remain pending',
        'Boundary-affected arrivals are conditional diagnostics, excluded from the arrival envelope',
        'copdem_120_dclaw_ref historical build-container image ID is unavailable; base image/container and exact execution/log/fields retained']
    partial_summary={'status':packet_status,'early_selection':selection,'generated_at_utc':utc(),
        'partial':True,'provisional_compare_scope':True,'selected_run_ids':sorted(ids),
        'selected_run_count':len(ids),'all_routes_completed':False,
        'full_inventory_snapshot_count':summary['run_inventory_count'],
        'full_actual_regional_attempts_snapshot':summary['actual_regional_execution_attempted_count'],
        'adds_new_model_case_count':0,'adds_independent_event_validation_count':0,
        'remaining_and_excluded_run_statuses':[{k:x.get(k) for k in ['run_id','model_id','evaluation_stratum','plan_scope','execution_attempted','execution_status','solver_build']} for x in runs if x['run_id'] not in ids],
        'arrival_status_counts':dict(collections.Counter(x['arrival_status'] for x in selected_arrivals)),
        'arrival_envelope_eligible_count':sum(x['include_in_arrival_envelope'] for x in selected_arrivals),
        'conditional_diagnostic_range_eligible_target_count':sum(x['conditional_diagnostic_range_eligible'] for x in selected_arrivals),
        'scientific_acceptance_decision_version':summary['scientific_acceptance_decision_version'],
        'arrival_envelope_semantics_version':summary['arrival_envelope_semantics_version'],
        'scheduling_resource_context':summary.get('scheduling_resource_context'),
        'within_framework_pairs':within_framework_pairs(selected,selected_metrics),
        'independent_local_training_events':0,'independent_local_validation_events':0,
        'probability_weights':None,'coverage_probability_claimed':False,
        'time_origin':'bulk_release','initiation_to_release_delay_seconds':None,
        'limitations':specific_limits+[
            'Saved-output target detection was independently re-evaluated against hashed native NPZ times/depth; not solver-step arrival precision',
            'Five OSM representatives refer to three distinct candidate entities, not five independent validation targets',
            'No independent validation events, statistical weighting, predictive adoption or probability coverage claim']}
    selected_numerical={'status':partial_summary['status'],'generated_at_utc':utc(),
```

```
        'qualification_by_run':[{k:x.get(k) for k in ['run_id','execution_status','numerical_qualification','state_horizon_complete','finite_state','nonnegative_state','fields_hash_matches_index','execution_case_hash_matches','execution_log_hash_matches','runtime_image_identity_complete','source_quality_review_version','source_support_cells','source_equivalent_full_cells','source_quality_evidence_binding_matches','scientifically_qualified_for_prediction','mass_conservation_qualified']} for x in selected],
        'official':{'checks':[x for x in numerical['official']['checks'] if x['run_id'] in ids or selection=='dclaw' and x['run_id']=='flatrest_60_dclaw']},
        'scientific_acceptance_decision':summary['scientific_acceptance_decision'],
        'independent_local_event_validation_count':0,'native_conservation_verified':False,
        'declared_property_pass_is_not_regional_accuracy':True}
    unique_metadata={}
    for relative,b,role in metadata:
        if str(relative) in unique_metadata and unique_metadata[str(relative)][1]!=b:raise RuntimeError('Conflicting early metadata destination: '+str(relative))
        unique_metadata[str(relative)]=(relative,b,role)
    dest.mkdir(parents=True);manifest=[]
    def record(path,role,source_path=None):
        manifest.append({'path':str(path.relative_to(ROOT)),'bytes':path.stat().st_size,'sha256':file_digest(path),'role':role,'storage':'copied_versioned_metadata',**({'source_path':source_path} if source_path else {})})
    for name,rows in [('model_runs.csv',selected),('comparison_metrics.csv',selected_metrics),('target_arrivals.csv',selected_arrivals)]:
        p=dest/name;dump_csv(p,rows);record(p,name.removesuffix('.csv'))
    for name,obj in [('comparison_summary.json',partial_summary),('numerical_validation.json',selected_numerical),('fields_index.json',{'fields':[x for x in fields if x.get('role')=='actual_official_process_fields']})]:
        p=dest/name;write_json(p,obj);record(p,name.removesuffix('.json'))
    for relative,b,role in unique_metadata.values():
        p=dest/relative;p.parent.mkdir(parents=True,exist_ok=True);p.write_bytes(b);record(p,role,metadata_origins.get(str(relative)))
    if q2_selection:
        p=dest/'native_controls_index.json';write_json(p,{'runs':[controls_by[rid] for rid in sorted(ids)],'source_index_path':'results/official_native_control_index.json','scope':'selected actual saved generated controls; source_path in manifest maps copied control bytes'});record(p,'selected actual native controls provenance index')
    for name in ['spatial_targets_v2.csv','computational_geometries.geojson']:
        source=R/'results'/name;b=ins.read(source,'shared target and spatial definition',False)
        if b is not None:
            p=dest/'geometry'/name;p.parent.mkdir(parents=True,exist_ok=True);p.write_bytes(b);record(p,'target geometry')
    code=dest/'provenance/integrate_comparison.py';code.write_bytes(ins.bytes[str(Path(__file__).resolve())]);record(code,'exact early-package postprocessor source')
    index={'files':manifest+fields,'all_entries_have_path_bytes_sha256_role':True,'file_list_excludes_self':True,
        'source_observer_snapshot_hashes':[rec for rec in ins.records.values() if rec['path'].startswith('track_b/research/q4/round3/results/official_')],
        'dependency_scope':'copied selected-run metadata; immutable byte-hash-bound external native fields, logs and initial rasters',
        'rolling_root_input_index_at_export':{'path':'results/comparison_input_index.json','sha256':file_digest(R/'results/comparison_input_index.json')}}
    p=dest/'input_index.json';write_json(p,index);record(p,'selected dependency and observer snapshot index')
    bundle={'bundle_version':'q4-r3-'+version.replace('_','-'),'producer_question':'q4',
        'input_version':INPUT_VERSION,'status':partial_summary['status'],
        'common_context':common_context(selected,selected_arrivals),
        'early_selection':selection,
        'scientific_acceptance_decision_version':summary['scientific_acceptance_decision_version'],
        'arrival_envelope_semantics_version':summary['arrival_envelope_semantics_version'],
```

```
        'accepted_arrival_envelope_run_ids':summary['scientific_acceptance_decision']['accepted_arrival_envelope_run_ids'],
        'lineage_by_run':[{k:x.get(k) for k in ['run_id','scenario_id','base_scenario_id','upstream_model_id','upstream_parameter_version','mechanism_id','parameter_version','release_mapping_version','geometry_version','upstream_onset_status','if_released_propagation','initiation_to_release_delay_seconds']} for x in selected],
        'scientifically_qualified_for_prediction':False,'mass_conservation_qualified':False,'regional_accuracy_qualified':False,
        'scheduling_resource_context':summary.get('scheduling_resource_context'),
        'partial':True,'provisional_compare_scope':True,'all_routes_completed':False,
        'adds_new_model_case_count':0,'adds_independent_event_validation_count':0,
        'generated_at_utc':utc(),'selected_run_ids':sorted(ids),'evaluation_stratum':'conditional_scenario',
        'files':manifest+fields,'file_list_excludes_self':True,
        'table_paths':{name:str((dest/(name+'.csv')).relative_to(ROOT)) for name in ['model_runs','comparison_metrics','target_arrivals']},
        'probability_weights':None,'independent_local_validation_events':0,'coverage_probability_claimed':False,
        'time_origin':'bulk_release','initiation_to_release_delay_seconds':None,
        'limitations':partial_summary['limitations'],'human_scientific_adoption':'pending_team_review'}
    write_json(dest/'bundle.json',bundle)
    if not is_delta:write_json(R/'exports/propagation_bundle.json',bundle)
    return {'bundle_path':str((dest/'bundle.json').relative_to(ROOT)),'bundle_sha256':file_digest(dest/'bundle.json'),'bundle_bytes':(dest/'bundle.json').stat().st_size,
        'files':len(bundle['files']),'run_ids':sorted(ids),'arrivals':partial_summary['arrival_status_counts'],
        'arrival_envelope_eligible':partial_summary['arrival_envelope_eligible_count']}

def main():
    parser=argparse.ArgumentParser();mode=parser.add_mutually_exclusive_group();mode.add_argument('--freeze',action='store_true');mode.add_argument('--early-freeze',action='store_true');parser.add_argument('--early-selection',choices=['dclaw','q2mapped','q2fullmask','q2compactdelta'],default='dclaw');parser.add_argument('--version',default='propagation_v1');parser.add_argument('--active-config',action='append');parser.add_argument('--control-index',action='append');args=parser.parse_args()
    if args.freeze:
        match=re.fullmatch(r'propagation_v([0-9]+)',args.version)
        if not match or int(match.group(1))<4:raise ValueError('Final all-model joint-review freeze requires a NEW propagation_v4 or higher; v1/v2 and reserved fullmask partial v3 remain immutable')
    ins=Inputs();ins.read(Path(__file__),'integration source code');policy=ins.js(R/'config/comparison_policy.json','preregistered comparison policy')
    reduced,rm,ra=build_reduced(ins);official,om,oa,ov,active=collect_official(ins,args.active_config or ACTIVE);br,rv=benchmarks(ins)
    archived,archived_arrivals=archived_attempts(ins,official,oa);official+=archived;oa+=archived_arrivals
    runs=reduced+official+br;metrics=rm+om;arrivals=ra+oa
    run_by_id={x['run_id']:x for x in runs}
    for row in runs:
        row['q2_parent_lineage_status']='not_applicable_numerical_benchmark' if row.get('evaluation_stratum')=='numerical_benchmark' else 'explicit_delivered_Q2_parent' if row.get('upstream_model_id') else 'unknown_Q2_parent'
    for row in metrics+arrivals:row['q2_parent_lineage_status']=run_by_id[row['run_id']]['q2_parent_lineage_status']
    target_entities={x['target_id']:x.get('entity_id') for x in ins.csv(R/'results/spatial_targets_v2.csv','all currently registered OSM target representatives',False)}
    for row in arrivals:
        row['inundation_or_runout_reference']=run_by_id[row['run_id']].get('field_reference')
```

```
        if not row.get('entity_id'):row['entity_id']=target_entities.get(row.get('target_id'))
        decorate_time_contract(row)
    stationary_residual_metrics(ins,runs,metrics)
    enrich_native_semantics_and_budget(ins,runs,metrics)
    replay_evidence=bind_existing_replays(ins)
    native_controls=bind_native_controls(ins,args.control_index)
    scheduling=bind_scheduling_context(ins)
    for row in runs:
        row['controlled_performance_benchmark_eligible']=False
        row['wall_time_interpretation']=scheduling['wall_time_interpretation']
    scientific_acceptance=load_scientific_acceptance(ins)
    fullmask_volume_annotation=bind_fullmask_volume_annotation(ins,runs,metrics)
    q2_catalogue_provenance=bind_q2_upstream_catalogues(ins)
    official_final_receipt=bind_official_final_receipt(ins)
    ins.js(R/'results/dclaw_mass_audit.json','independent raw-state DClaw mass audit',False)
    ins.read(R/'sources/dclaw_mass_source_audit.json','fixed-source DClaw mass audit provenance',False)
    ins.js(R/'config/q2_compact_quality_review_04.json','registered compact-source numerical qualification review',False)
    ins.js(R/'sources/q2_compact_budget_review_v1.json','independently crosschecked Q2 compact ASCII/native budget evidence',False)
    changed=ins.changed();provisional=not args.freeze
    for row in arrivals:
        row['source_snapshot_stable']=not changed
        if changed:row['include_in_arrival_envelope']=False
    mass_audit=attach_quality_contract(ins,runs,metrics,arrivals)
    q2_quality=apply_q2_compact_quality(ins,runs,metrics,arrivals)
    apply_scientific_acceptance(scientific_acceptance,runs,metrics,arrivals)
    statuses=collections.Counter(x['execution_status'] for x in runs)
    summary={'integration_version':'q4-r3-comparison-v1','generated_at_utc':utc(),'status':'provisional' if provisional else 'final_freeze_requested',
      'input_version':dict(INPUT_VERSION),'common_context':common_context(runs,arrivals),
      'scientific_acceptance_decision':scientific_acceptance,
      'scientific_acceptance_decision_version':scientific_acceptance['version'],
      'arrival_envelope_semantics_version':'q4-r3-arrival-envelope-accepted-runs-v05',
      'accepted_arrival_envelope_run_count':sum(x.get('include_in_arrival_envelope',False) for x in runs),
      'fullmask_volume_reference_semantics_annotation':fullmask_volume_annotation,
      'delivered_Q2_catalogue_provenance':q2_catalogue_provenance,
      'official_final_delivery_receipt':official_final_receipt,
      'accepted_arrival_envelope_target_count':sum(x.get('include_in_arrival_envelope',False) for x in arrivals),
      'passes_execution_time_boundary_gate_target_count':sum(x.get('passes_execution_time_boundary_gates',False) for x in arrivals),
      'conditional_diagnostic_range_eligible_target_count':sum(x.get('conditional_diagnostic_range_eligible',False) for x in arrivals),
      'diagnostic_observation_available_target_count':sum(x.get('diagnostic_observation_available',False) for x in arrivals),
      'source_input_index':'results/comparison_input_index.json','source_inputs_changed_during_read':changed,
      'run_inventory_count':len(runs),'reduced_regional_count':len(reduced),'official_case_inventory_count':sum(x.get('record_kind')!='archived_execution_attempt' for x in official),'archived_distinct_attempt_records':len(archived),'numerical_configuration_records':len(br),
      'actual_execution_attempted_count':sum(bool(x.get('execution_attempted')) for x in runs),
      'actual_regional_execution_attempted_count':sum(bool(x.get('execution_attempted')) for x in runs if x.get('evaluation_stratum')=='conditional_scenario'),
      'official_current_registered_count':sum(x['plan_scope']=='current_registered' for x in official),
      'official_current_conditional_registered_count':sum(x['plan_scope']=='current_registered' and x['evaluation_stratum']=='conditional_scenario' for x in official),
```

```
    'official_current_conditional_status_counts':dict(collections.Counter(x['execution_statu
    ↪  s'] for x in official if x['plan_scope']=='current_registered' and
    ↪  x['evaluation_stratum']=='conditional_scenario')),
    'registered_applicability_completion':{'registered_count':sum(x['plan_scope']=='current_
    ↪  registered' for x in official),
        'registered_conditional_count':sum(x['plan_scope']=='current_registered' and
        ↪  x['evaluation_stratum']=='conditional_scenario' for x in official),
        'applicable_required_count':sum(x.get('applicable_required',False) for x in
        ↪  official),
        'applicable_required_run_ids':[x['run_id'] for x in official if
        ↪  x.get('applicable_required')],
        'applicable_required_completed_count':sum(x.get('applicable_required',False) and
        ↪  x['execution_status']=='completed' for x in official),
        'quarantined_not_run_count':sum(x['execution_status']=='not_run_material_applicabili
        ↪  ty_exclusion' for x in official),
        'quarantined_not_run_run_ids':[x['run_id'] for x in official if
        ↪  x['execution_status']=='not_run_material_applicability_exclusion'],
        'generic_diagnostic_only_statuses':{x['run_id']:x['execution_status'] for x in
        ↪  official if
        ↪  x.get('applicability_class')=='generic_material_proxy_diagnostic_only'},
        'applicability_review_status':'registered decision03; mathematical implementation
        ↪  invalidity not claimed for excluded material proxies',
        'completed_registered_count':sum(x['plan_scope']=='current_registered' and
        ↪  x['execution_status']=='completed' for x in official)},
    'existing_case_reproduction':{'replayed_existing_case_count':len((replay_evidence or
    ↪  {}).get('replayed_cases',[])),
        'adds_new_model_case_count':0,'adds_independent_event_validation_count':0,
        'evidence_path':'results/replay_execution_comparison.json' if replay_evidence else
        ↪  None},
    'actual_native_input_control_provenance':native_controls,
    'scheduling_resource_context':scheduling,
    'native_output_semantics':'cumulative total depth uses native hflow_max ASCII max_t(sum
    ↪  phase depths); raw nout phase-max sum and max-phase horizontal speed are separately
    ↪  labelled; no cross-engine peak-speed ranking',
    'unresolved_source_binding_failures':[x['run_id'] for x in runs if
    ↪  x.get('parsed_source_snapshot_hash_matches') is False or
    ↪  x.get('actual_native_time_matches_npz') is False],
    'unresolved_parsed_field_or_budget_consistency_failures':[x['run_id'] for x in metrics if
    ↪  x.get('native_cumulative_depth_matches_hashed_npz') is False or
    ↪  x.get('parsed_source_adjusted_budget_matches_recomputation') is False],
    'official_plan_scope_counts':dict(collections.Counter(x['plan_scope'] for x in
    ↪  official)),
    'execution_status_counts':dict(statuses),'arrival_status_counts':dict(collections.Counte
    ↪  r(x['arrival_status'] for x in arrivals)),
    'execution_status_by_stratum':{scope:dict(collections.Counter(x['execution_status'] for x
    ↪  in runs if x.get('evaluation_stratum')==scope)) for scope in
    ↪  sorted(set(x['evaluation_stratum'] for x in runs))},
    'official_execution_status_counts':dict(collections.Counter(x['execution_status'] for x
    ↪  in official)),
    'conditional_valid_hit_count':sum(x['arrival_status']=='arrived' and
    ↪  x.get('travel_time_seconds') is not None for x in arrivals),
    'boundary_affected_valid_hit_count':sum(x['arrival_status']=='arrived' and
    ↪  x.get('domain_affected_before_arrival') is True for x in arrivals),
    'current_pending_runs':[x['run_id'] for x in official if x.get('completion_required') and
    ↪  x['execution_status'] in ['planned_without_execution','running_unfinalized','running
    ↪  _retry_with_stale_prior_attempt_metadata','finished_unanalysed','field_missing_or_in
    ↪  dex_stale','missing_execution_result','historical_attempt_unanalysed']],
    'actual_attempts_unsettled':[x['run_id'] for x in official if x['execution_attempted']
    ↪  and x['execution_status'] in ['running_unfinalized','running_retry_with_stale_prior_
    ↪  attempt_metadata','finished_unanalysed','field_missing_or_index_stale','missing_exec
    ↪  ution_result','historical_attempt_unanalysed']],
```

```
    'within_framework_pairs':within_framework_pairs(runs,metrics),'cross_framework_incremental_physics_gain_claimed':False,
    'statistical_weights':None,'independent_local_training_events':0,'independent_local_validation_events':0,
    'confidence_interval_or_95pct_coverage_claim':False,'human_scientific_adoption':'pending_team_review',
    'arrival_envelope_gate_scope':'explicit accepted-run membership plus execution/time/field/material/boundary gates; diagnostic gates alone never grant accepted arrival envelope membership',
    'limitations':['inventory is not number of completed formal experiments; plan_scope and actual attempts separated',
      'reduced 1D/2D unqualified for thin-flow arrival timing; 1D area is equivalent-width only',
      'ravaflow scalar terrain-corrected speed differs from horizontal magnitude; sampling/cutoffs retained',
      'DClaw nodal-to-cell effective terrain/release differ; signed sampled boundary flux is not native conservation verification',
      'DClaw extended reference has about 4.15% domain mass loss without recorded boundary depth >=.1m; do not attribute all mass discrepancy to outflow',
      'Raw-state audit finds extended DClaw mass -2.1612% at180s and permeable +11.6012% at20s while saved boundary h/hu/hv are zero; cause remains unresolved without native per-step ledger',
      'decision05 has empty accepted-run list: all regional arrival envelope flags false; separately recorded diagnostic gates do not confer mass, regional or predictive qualification',
      'failed-pipeline and early-terminated hits retained as observed_hit only, valid travel time null',
      'all travel times relative to scenario bulk release; initiation-to-release delay unknown']}
freeze_blocked=bool(changed or not official_final_receipt or summary['current_pending_runs'] or summary['actual_attempts_unsettled'] or native_controls['status']!='indexed_actual_generated_controls' or summary['unresolved_source_binding_failures'] or summary['unresolved_parsed_field_or_budget_consistency_failures'])
summary['freeze_block_reasons']=[label for label,condition in [
    ('source_files_changed_during_read',bool(changed)),('required_or_active_diagnostic_runs_unfinished',bool(summary['current_pending_runs'])),
    ('owner_final_exactready_receipt_missing',not official_final_receipt),
    ('actual_attempts_unsettled',bool(summary['actual_attempts_unsettled'])),('native_input_control_index_missing_or_mismatched',native_controls['status']!='indexed_actual_generated_controls'),
    ('source_time_binding_mismatch',bool(summary['unresolved_source_binding_failures'])),('parsed_field_or_budget_recomputation_mismatch',bool(summary['unresolved_parsed_field_or_budget_consistency_failures']))] if condition]
if args.freeze and not freeze_blocked:
    if not re.fullmatch(r'propagation_v[0-9]+',args.version):raise ValueError('version must be propagation_vN')
    if (R/'exports'/args.version).exists():raise FileExistsError('Frozen version already exists; choose new version')
    summary['status']='frozen_for_joint_review'
elif args.freeze:summary['status']='provisional'
numerical={'generated_at_utc':utc(),'status':summary['status'],'reduced':rv,'official':ov,
  'dclaw_raw_state_mass_audit':mass_audit,
  'existing_case_reproduction_evidence':replay_evidence,
  'actual_native_input_control_provenance':native_controls,
  'scientific_acceptance_decision':scientific_acceptance,
```

```
    'qualification_by_run':[{k:x.get(k) for k in ['run_id','execution_status','numerical_qua
    ↪ lification','state_horizon_complete','finite_state','nonnegative_state','fields_hash
    ↪ _matches_index','event_timing_validated','scientific_acceptance_decision_version','a
    ↪ rrival_envelope_semantics_version','include_in_arrival_envelope','conditional_diagno
    ↪ stic_range_eligible','passes_execution_time_boundary_gates','diagnostic_observation_
    ↪ available','regional_accuracy_qualified','mass_conservation_qualified','scientifical
    ↪ ly_qualified_for_prediction','human_scientific_adoption']} for x in runs],
    'independent_local_event_validation_count':0,'declared_property_pass_is_not_regional_acc
    ↪ uracy':True,
    'source_inputs_changed_during_read':changed}
for row in runs:row['integration_status']=summary['status']
for row in metrics:row['integration_status']=summary['status']
for row in arrivals:row['integration_status']=summary['status']
dump_csv(R/'results/model_runs.csv',runs);dump_csv(R/'results/comparison_metrics.csv',metr
↪ ics);dump_csv(R/'results/target_arrivals.csv',arrivals)
write_json(R/'results/numerical_validation.json',numerical);write_json(R/'results/comparis
↪ on_summary.json',summary)
index={'generated_at_utc':utc(),'integration_status':summary['status'],'files':list(ins.re
↪ cords.values()),'no_self_reference':True,'mutability_note':'read-time snapshot; rerun
↪ integration after official observer tables have settled'}
write_json(R/'results/comparison_input_index.json',index)
log={'utc':utc(),'command_mode':'freeze' if args.freeze else
↪ 'provisional','source_index_sha256':digest((R/'results/comparison_input_index.json').r
↪ ead_bytes()),'source_count':len(ins.records),'source_files_changed':changed,'outputs_w
↪ ritten':['model_runs.csv','comparison_metrics.csv','target_arrivals.csv','numerical_va
↪ lidation.json','comparison_summary.json'],'final_bundle_written':False}
if args.freeze:
    if freeze_blocked:
        log['freeze_refused']=summary['freeze_block_reasons'];write_json(R/'results/compar
        ↪ ison_integration_log.json',log)
        raise RuntimeError('Cannot freeze: '+', '.join(summary['freeze_block_reasons'])+';
        ↪ tables remain provisional')
    dest=R/'exports'/args.version
    dest.mkdir(parents=True);files=[]
    for name in ['model_runs.csv','comparison_metrics.csv','target_arrivals.csv','numerica
    ↪ l_validation.json','comparison_summary.json','comparison_input_index.json']:
        source=R/'results'/name;target=dest/'files/results'/name;target.parent.mkdir(paren
        ↪ ts=True,exist_ok=True);shutil.copy2(source,target)
        b=target.read_bytes();files.append({'path':str(target.relative_to(ROOT)),'bytes':l
        ↪ en(b),'sha256':digest(b),'role':name.removesuffix('.csv').removesuffix('.json'
        ↪ )})
    for key,rec in ins.records.items():
        if key in ins.bytes:
            original=Path(rec['path'])
            relative=original if not original.is_absolute() else
            ↪ Path('external')/str(original).lstrip('/')
            target=dest/'source_snapshots'/relative;target.parent.mkdir(parents=True,exist
            ↪ _ok=True);target.write_bytes(ins.bytes[key])
            files.append({'path':str(target.relative_to(ROOT)),'bytes':target.stat().st_si
            ↪ ze,'sha256':file_digest(target),
                'role':rec['role'],'storage':'copied_read_time_source_snapshot','source_pa
                ↪ th':rec['path']})
        else:
            files.append({**rec,'storage':'immutable_external_byte_hash_bound_dependency'})
    bundle={'bundle_version':'q4-r3-'+args.version.replace('_','-'),'producer_question':'q
    ↪ 4','input_version':INPUT_VERSION,
      'common_context':summary['common_context'],
      'partial':False,'all_routes_completed':True,
      'completion_scope':'all applicable required cases and actual attempts settled;
      ↪ explicit material-applicability not-run exclusions retained',
      'table_paths':{name:str((dest/'files/results'/(name+'.csv')).relative_to(ROOT)) for
      ↪ name in ['model_runs','comparison_metrics','target_arrivals']},
```

```
        'root_table_snapshot_sha256':{name:file_digest(R/'results'/(name+'.csv')) for name in ['model_runs','comparison_metrics','target_arrivals']},
        'scientific_acceptance_decision_version':scientific_acceptance['version'],
        'arrival_envelope_semantics_version':summary['arrival_envelope_semantics_version'],
        'accepted_arrival_envelope_run_ids':scientific_acceptance['accepted_arrival_envelope_run_ids'],
        'scientifically_qualified_for_prediction':False,'regional_accuracy_qualified':False,'mass_conservation_qualified':False,
        'evaluation_stratum':['conditional_scenario','numerical_benchmark'],
        'evaluation_stratum_note':'conditional_scenario; numerical_benchmark rows explicitly separated','files':files,
        'status':'frozen_for_joint_review','probability_weights':None,'independent_local_validation_events':0,
        'time_origin':'bulk release','initiation_to_release_delay_seconds':None,'limitations':summary['limitations'],
        'human_scientific_adoption':'pending_team_review','file_list_excludes_self':True}
      write_json(dest/'bundle.json',bundle);write_json(R/'exports/propagation_bundle.json',bundle);log['final_bundle_written']=True;log['bundle_version']=bundle['bundle_version']
    if args.early_freeze:
      log['early_partial_bundle']=early_export(ins,runs,metrics,arrivals,numerical,summary,args.version,args.early_selection)
      log['early_partial_bundle_written']=True
    write_json(R/'results/comparison_integration_log.json',log)
    print(canonical({'status':summary['status'],'inventory':len(runs),'official_current_pending':len(summary['current_pending_runs']),'actual_regional_attempts':summary['actual_regional_execution_attempted_count'],'arrivals':dict(summary['arrival_status_counts']),'final_bundle_written':log['final_bundle_written'],'source_input_index_sha256':log['source_index_sha256'],'early_partial_bundle':log.get('early_partial_bundle')}))

if __name__=='__main__':main()
```

程序 48：audit_dclaw_mass.py

来源：track_b/research/q4/round3/scripts/audit_dclaw_mass.py
SHA256: 4cfcc891bded122944557d7e0b337a78b2435af4e909246e685c8af03e1bcc9c

```
#!/usr/bin/env python3
"""Read-only audit of completed fixed D-Claw outputs; never invokes a solver.

Output is confined to this audit's two new JSON files. Native data and the
official postprocessor are inputs. No numerical-source accounting is inferred
from coarse saved frames.
"""
from pathlib import Path
import datetime
import hashlib
import json
import sys

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official/runs')
SOURCE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/runtime/sources/dclaw-1.0.0')
RUNS = ['dclaw_ref_extended120', 'copdem_120_dclaw_permeable', 'copdem_120_dclaw_ref']
SOURCE_SCOPE = {
    'docs/src/theory.md': {'read_lines': [[1, 175]], 'purpose': 'State semantics, core equations, density and dilation source terms'},
    'src/2d/dig/digclaw_module.f90': {'read_lines': [[70, 105], [267, 465]], 'purpose': 'State indices, ordinary qfix, constant-mass qfix_cmass, actual permeability law'},
    'src/2d/dig/src2.f90': {'read_lines': [[1, 300]], 'purpose': 'Complete source integrator; rhoh before qfix, source method 2, dry cycle, entrainment switch'},
    'src/2d/dig/mp_update.f90': {'read_lines': [[1, 165]], 'keyword_scan_only_elsewhere': ['rhoh', 'qfix', 'dry_tolerance', 'subroutine', 'h =', 'hm =', 'm =', 'mass'], 'purpose': 'Method descriptions, actual method-2 coefficients and constant-mass calls; not a complete branch-by-branch proof'},
    'src/2d/dig/b4step2.f90': {'read_lines': [[1, 80]], 'purpose': 'Pre-step qfix calls for interior and ghost cells'},
    'src/2d/dig/rpn2_dclaw.f': {'read_lines': [[1, 392]], 'purpose': 'Normal Riemann qfix, wet-dry wall handling, zero-speed fluctuation branch'},
    'src/2d/dig/riemannsolvers_dclaw.f': {'read_lines': [[1, 508]], 'purpose': 'Flux wave identities including hm flux; not replayed as a solver'},
    'src/2d/dig/flux2fw.f': {'read_lines': [[1, 240]], 'purpose': 'First-order flux-difference assembly; active first-order run avoids second-order limiter claims'},
    'src/2d/dig/valout.f90': {'read_lines': [[1, 565]], 'purpose': 'Native binary layout: seven states and eta, ghost cells, tiny-value output rounding'},
}


def record(path, scope=None):
    path = Path(path)
    digest = hashlib.sha256()
    with path.open('rb') as f:
        for block in iter(lambda: f.read(8 * 1024 * 1024), b''):
            digest.update(block)
    row = {'path': str(path), 'bytes': path.stat().st_size, 'sha256': digest.hexdigest()}
    if scope is not None:
        row['actual_read_scope'] = scope
    return row


```

```
def parse_data(path):
    """Read generated named scalar controls; not a setrun reconstruction."""
    values = {}
    for line in path.read_text().splitlines():
        if '=:' in line:
            before, after = line.split('=:', 1)
            values[after.split()[0]] = before.strip()
    return values


def raw_frame(folder, suffix):
    """Independent binary64 decoder from valout's Fortran stream layout."""
    t_path, q_path, b_path = [folder / (prefix + suffix) for prefix in ['fort.t', 'fort.q',
    ↪  'fort.b']]
    tokens = [line.split()[0] for line in t_path.read_text().splitlines() if line.strip()]
    time, nq, ngrids, ghost, fmt = float(tokens[0]), int(tokens[1]), int(tokens[2]),
    ↪  int(tokens[5]), tokens[6]
    if (nq, ngrids, fmt.lower()) != (8, 1, 'binary64'):
        raise ValueError(f'Unsupported audit layout in {t_path}: {tokens}')
    head = [line.split()[0] for line in q_path.read_text().splitlines() if line.strip()]
    if len(head) != 8 or int(head[1]) != 1:
        raise ValueError('Expected one level-1 header, no AMR overlap')
    nx, ny = int(head[2]), int(head[3])
    dx, dy = float(head[6]), float(head[7])
    raw = np.fromfile(b_path, dtype='<f8')
    expected = nq * (nx + 2 * ghost) * (ny + 2 * ghost)
    if raw.size != expected:
        raise ValueError(f'Byte count does not match native header: {b_path}')
    full = raw.reshape(ny + 2 * ghost, nx + 2 * ghost, nq)
    q = full[ghost:ghost + ny, ghost:ghost + nx, :]
    # The native array is south-to-north. Integrals need no vertical flip.
    return time, q, dx, dy, [t_path, q_path, b_path]


def source_rate_at_state(q, controls, dry):
    """Instantaneous mathematical rates, not the actual substep ledger."""
    rs, rf = float(controls['rho_s']), float(controls['rho_f'])
    mu, kref, mref = float(controls['mu']), float(controls['kref']), float(controls['mref'])
    h, hm, p = q[:, :, 0], q[:, :, 3], q[:, :, 4]
    wet = h > dry
    m = np.divide(hm, h, out=np.zeros_like(h), where=wet)
    rho = rf + (rs - rf) * m
    k = kref * np.exp(-(m - mref) / .04)  # Actual setvars, not the sign in a prose formula.
    excess = p - rf * 9.81 * h
    dilation = np.divide(-2 * k * excess, mu * h, out=np.zeros_like(h), where=wet)
    phi1 = np.where(wet, (rho - rf) / rho * dilation, 0.)
    phi4 = np.where(wet, -dilation * m * rf / rho, 0.)
    return phi1, phi4


def audit_run(name):
    folder = CACHE / name
    case_path = folder / 'case.json'
    case = json.loads(case_path.read_text())
    controls = parse_data(folder / 'dclaw.data')
    geo = parse_data(folder / 'geoclaw.data')
    dry = float(geo['dry_tolerance'])
    rs, rf = float(controls['rho_s']), float(controls['rho_f'])
    summaries = sorted(folder.glob('summary_fields_db13be185e93.npz'))
    if len(summaries) != 1:
        raise ValueError('Exact audited summary version missing')
    summary_path = summaries[0]
```

```
    files = [record(folder / f, 'Complete named control/JSON read' if f in ['case.json',
    ↪  'dclaw.data', 'geoclaw.data'] else 'Identity hash only within audit script; associated
    ↪  controls reviewed separately') for f in [
        'case.json', 'dclaw.data', 'geoclaw.data', 'claw.data', 'amr.data',
        'execution.json', 'timing.txt']]
    files.append(record(summary_path, 'All named arrays relevant to depth, m, phase volumes,
    ↪  mass, time and boundary flux; no rewrite'))
    rows, raw_files = [], []
    comparisons = {key: 0. for key in ['mass_max_abs_difference_kg',
    ↪  'mass_max_relative_difference',
        'phase_volume_max_abs_difference_m3', 'depth_float32_max_abs_difference_m',
        'wet_m_float32_max_abs_difference', 'boundary_mass_rate_max_abs_difference_kg_s']}
    with np.load(summary_path) as summary:
        stored_t = summary['time_s']
        stored_mass = summary['mass_kg']
        stored_phases = summary['phase_volume_m3']
        stored_depth = summary['depth_frames']
        stored_m = summary['solid_fraction_frames']
        stored_boundary = summary['boundary_mass_flux_kg_s']
        if len(stored_t) != 91 or stored_t[0] != 0 or stored_t[-1] != 1800:
            raise ValueError('Unexpected completed horizon')
        footprint = None
        for index, tf in enumerate(sorted(folder.glob('fort.t[0-9][0-9][0-9][0-9]'))):
            time, q, dx, dy, paths = raw_frame(folder, tf.name[-4:])
            if time != stored_t[index]:
                raise ValueError('Native time disagrees with summary')
            if not np.isfinite(q).all():
                raise ValueError(f'Nonfinite native state in {name}, t={time}')
            raw_files.extend(record(path, 'Complete header or all binary64 interior state
            ↪  values decoded; ghosts excluded from integrals') for path in paths)
            h, hm, hu, hv = q[:, :, 0], q[:, :, 3], q[:, :, 1], q[:, :, 2]
            area = dx * dy
            volume = float(h.sum() * area)
            solid_volume = float(hm.sum() * area)
            fluid_volume = float((h - hm).sum() * area)
            # Never apply the 0.1 m impact threshold to total or phase mass.
            cell_mass = rf * h + (rs - rf) * hm
            mass = float(cell_mass.sum() * area)
            density = rf + (rs - rf) * np.divide(hm, h, out=np.zeros_like(h), where=h > 1e-12)
            edge_h = np.concatenate((h[0], h[-1], h[1:-1, 0], h[1:-1, -1]))
            edge_momentum = np.concatenate((q[0, :, 1:3].ravel(), q[-1, :, 1:3].ravel(),
                                            q[1:-1, 0, 1:3].ravel(), q[1:-1, -1, 1:3].ravel()))
            # E/N signs evaluated in native south-to-north coordinates.
            boundary_rate = float(dy * (np.sum(density[:, -1] * hu[:, -1]) - np.sum(density[:,
            ↪  0] * hu[:, 0]))
                                  + dx * (np.sum(density[-1] * hv[-1]) - np.sum(density[0] *
                                  ↪  hv[0])))
            next_h = np.where(h <= dry, 0., h)
            next_hm = np.where(h <= dry, 0., np.clip(hm, 0., h))
            qfix_delta = (rf * next_h + (rs - rf) * next_hm - cell_mass) * area
            thin = (h > 0) & (h <= dry)
            footprint = h > dry if footprint is None else footprint | (h > dry)
            row = {
                'time_s': time, 'volume_m3': volume, 'solid_volume_hm_m3': solid_volume,
                'fluid_volume_h_minus_hm_m3': fluid_volume, 'mass_kg': mass,
                'h_min_m': float(h.min()), 'hm_min_m': float(hm.min()),
                'hm_negative_cells': int(np.count_nonzero(hm < 0)),
                'hm_gt_h_cells': int(np.count_nonzero(hm > h)),
                'thin_positive_cells_h_le_dry': int(thin.sum()),
                'thin_positive_mass_kg': float(cell_mass[thin].sum() * area),
                'hypothetical_one_qfix_mass_loss_kg': float(-qfix_delta[qfix_delta < 0].sum()),
                'hypothetical_one_qfix_mass_gain_kg': float(qfix_delta[qfix_delta > 0].sum()),
                'boundary_h_max_m': float(edge_h.max()),
```

```
            'boundary_abs_hu_hv_max_m2_s': float(np.abs(edge_momentum).max()),
            'saved_boundary_mass_rate_kg_s': boundary_rate,
            'wet_cells_h_gt_dry': int((h > dry).sum()),
            'erosion_bdif_max_abs_m': float(np.abs(q[:, :, 6]).max()),
        }
        if index == 0:
            phi1, phi4 = source_rate_at_state(q, controls, dry)
            row['instantaneous_source_integrals_t0'] = {
                'phi1_volume_rate_m3_s': float(phi1.sum() * area),
                'phi4_solid_volume_rate_m3_s': float(phi4.sum() * area),
                'combined_total_mass_rate_kg_s': float((rf * phi1 + (rs - rf) * phi4).sum()
                ↪  * area),
                'combined_absolute_pointwise_residual_kg_s': float(np.abs(rf * phi1 + (rs -
                ↪  rf) * phi4).sum() * area),
                'scope': 't=0 instantaneous theoretical rates integrated over cells, actual
                ↪  native substep source history unavailable',
            }
        mass_diff = abs(mass - float(stored_mass[index]))
        comparisons['mass_max_abs_difference_kg'] =
        ↪  max(comparisons['mass_max_abs_difference_kg'], mass_diff)
        comparisons['mass_max_relative_difference'] =
        ↪  max(comparisons['mass_max_relative_difference'], mass_diff / abs(mass))
        comparisons['phase_volume_max_abs_difference_m3'] =
        ↪  max(comparisons['phase_volume_max_abs_difference_m3'],
            float(np.max(np.abs(stored_phases[index] - [solid_volume, fluid_volume]))))
        comparisons['depth_float32_max_abs_difference_m'] =
        ↪  max(comparisons['depth_float32_max_abs_difference_m'],
            float(np.max(np.abs(stored_depth[index] - h[::-1]))))
        # The summary stores m only above its 0.1 m display/impact threshold.
        shown = h[::-1] > .1
        primitive_m = np.divide(hm, h, out=np.zeros_like(h), where=h != 0)[::-1]
        if shown.any():
            comparisons['wet_m_float32_max_abs_difference'] =
            ↪  max(comparisons['wet_m_float32_max_abs_difference'],
                float(np.max(np.abs(stored_m[index][shown] - primitive_m[shown]))))
        comparisons['boundary_mass_rate_max_abs_difference_kg_s'] =
        ↪  max(comparisons['boundary_mass_rate_max_abs_difference_kg_s'],
            abs(boundary_rate - float(stored_boundary[index])))
        rows.append(row)
    m0, v0, s0 = [rows[0][key] for key in ['mass_kg', 'volume_m3', 'solid_volume_hm_m3']]
    for row in rows:
        row['mass_change_fraction'] = row['mass_kg'] / m0 - 1
        row['volume_change_fraction'] = row['volume_m3'] / v0 - 1
        row['solid_volume_change_fraction'] = row['solid_volume_hm_m3'] / s0 - 1
    rates = np.array([row['saved_boundary_mass_rate_kg_s'] for row in rows])
    times = np.array([row['time_s'] for row in rows])
    integrated = float(np.trapezoid(rates, times))
    first_threshold = next((row['time_s'] for row in rows if row['boundary_h_max_m'] >=
    ↪  .1), None)
    final_loss = m0 - rows[-1]['mass_kg']
    max_thin = max(row['thin_positive_mass_kg'] for row in rows)
    result = {
        'run_id': name, 'solver_binary_sha256': record(folder / 'xdclaw')['sha256'],
        'native_single_patch_shape_y_x': list(q.shape[:2]), 'native_frames': len(rows),
        'all_eight_output_components_finite_at_all_saved_interior_cells': True,
        'rho_s_kg_m3': rs, 'rho_f_kg_m3': rf, 'dry_tolerance_m': dry,
        'source_method': int(controls['src2method']), 'entrainment':
        ↪  int(controls['entrainment']),
        'mass_formula': 'rho_f*h + (rho_s-rho_f)*hm; integrate all interior cells without
        ↪  wet threshold',
        'decoder_comparison': comparisons,
        'first_saved_boundary_h_ge_0_1_seconds': first_threshold,
```

```
            'all_saved_boundary_h_exactly_zero': all(row['boundary_h_max_m'] == 0 for row in rows),
            'all_saved_boundary_hu_hv_exactly_zero': all(row['boundary_abs_hu_hv_max_m2_s'] == 0 for row in rows),
            'estimated_saved_boundary_mass_out_kg': integrated,
            'estimated_boundary_adjusted_final_residual_fraction': (rows[-1]['mass_kg'] + integrated - m0) / m0,
            'max_saved_thin_positive_mass_kg': max_thin,
            'max_saved_thin_positive_mass_fraction_of_initial': max_thin / m0,
            'max_saved_thin_mass_fraction_of_final_mass_deficit': max_thin / final_loss if final_loss > 0 else None,
            'saved_ever_wet_cells_h_gt_dry': int(footprint.sum()),
            'hypothetical_one_cleanup_all_cells_upper_bound_kg': float(q.shape[0] * q.shape[1] * dx * dy * dry * rs),
            'dry_budget_limit': 'Snapshot thin mass and hypothetical single-call bounds are not cumulative deletion; intermediate qfix calls/states are not recorded',
            'qualification': 'conditional_diagnostic; no independent validation and no native conservative mass ledger',
            'rows': rows,
        }
    return result, files + raw_files


def main():
    when = datetime.datetime.now(datetime.timezone.utc).isoformat()
    results, all_inputs = [], []
    for name in RUNS:
        result, inputs = audit_run(name)
        results.append(result)
        all_inputs.extend(inputs)
    output = {
        'audit_version': 'q4-r3-dclaw-mass-audit-v1', 'created_utc': when,
        'baseline_commit': '904d7257b899b9f71d1ac9e0eab7479e459eab52',
        'baseline_manifest_sha256': 'fd755fab38df285a67c986026859be5f635a314aa3f79f2d247a2626246c3a53',
        'execution': {'argv': sys.argv, 'python': sys.executable, 'numpy': np.__version__,
                      'native_solver_invoked': False, 'inputs_modified': False},
        'state_semantics': {'q1': 'h', 'q2': 'hu', 'q3': 'hv', 'q4': 'hm', 'q5': 'pb',
                            'q6': 'hchi', 'q7': 'bdif', 'q8_output_only': 'eta'},
        'theoretical_identity': 'rho_f*phi1+(rho_s-rho_f)*phi4 = 0, since rho-rho_f=(rho_s-rho_f)*m',
        'solid_phase_scope': 'hm has nonzero phi4 source in the stated model; its integral is audited separately and cannot be presumed invariant without source and boundary ledgers',
        'causal_attribution': 'unresolved: stored states reproduce deficits/gains, but no native per-step flux/source/qfix ledger exists',
        'runs': results,
    }
    output_path = ROOT / 'results/dclaw_mass_audit.json'
    output_path.write_text(json.dumps(output, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
    source_records = []
    for path, scope in SOURCE_SCOPE.items():
        # Hash complete files; the cited scope is the actual human/model review,
        # not a claim that hashing constituted a complete scientific reading.
        nlines = len((SOURCE / path).read_text().splitlines())
        scope = dict(scope)
        scope['read_lines'] = [[first, min(last, nlines)] for first, last in scope['read_lines']]
        source_records.append(record(SOURCE / path, scope))
    source_audit = {
        'created_utc': when, 'audit_version': output['audit_version'],
```

```
        'source_version': 'D-Claw 1.0.0; fixed local runtime source snapshot',
        'source_review': source_records,
        'postprocessor_review': record(ROOT / 'scripts/official_analyse.py', 'Full source read, independent decoder does not import or execute it'),
        'runtime_identity_snapshot': record(ROOT / 'sources/official_source_identity.json', 'Full structured read; inherited binary/source checks, not recompiled in this audit'),
        'native_and_summary_inputs': all_inputs,
        'raw_hash_scope': 'All fort.t/fort.q headers and fort.b binary streams of all 91 frames for each of the three listed runs; no output mutation',
        'audit_script': record(Path(__file__), 'Executed read-only audit'),
        'audit_result': record(output_path, 'New audit result'),
        'read_limitations': ['No native timestep mass flux ledger exists in these outputs',
                             'No per-call qfix before/after states exist',
                             'No native source-integrator substep history exists',
                             'No solver rerun, recompilation or patching undertaken'],
    }
    (ROOT / 'sources/dclaw_mass_source_audit.json').write_text(json.dumps(source_audit, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
    print(json.dumps({'result': str(output_path), 'runs': [
        {'run_id': row['run_id'], 'mass_change_at_20s': row['rows'][1]['mass_change_fraction'],
         'mass_change_at_180s': row['rows'][9]['mass_change_fraction'],
         'mass_change_at_1800s': row['rows'][-1]['mass_change_fraction'],
         'mass_decode_max_abs_difference_kg':
         row['decoder_comparison']['mass_max_abs_difference_kg']}
        for row in results]}, ensure_ascii=False, indent=2))


if __name__ == '__main__':
    main()
```

程序 49：compare_native_sensitivity.py

来源：track_b/research/q4/round3/scripts/compare_native_sensitivity.py
SHA256: 2b4c22e67317c4f40c8f9ed921e58f2d016b1ccdf84528efb580cb859c803367

```
#!/usr/bin/env python3
"""Compare registered single-phase native runs; no solver executions or fitting."""
from pathlib import Path
from datetime import datetime, timezone
import argparse
import csv
import hashlib
import json

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[5]
R = ROOT/'track_b/research/q4/round3'
CACHE = Path('<LOCAL_HOME>/Documents/MathorCup/research_cache/round3/q4/official/runs')
CONFIGS = [
    ('copdem_120_single_ref__sf0', 'Reference 120m', 'reference'),
    ('copdem_120_single_cfl025__sf0', 'CFL 0.25', 'numerical'),
    ('copdem_60_single_ref__sf0', 'Reference 60m', 'numerical'),
    ('copdem_120_single_h5__sf0', 'Thickness 5m', 'physical'),
    ('copdem_120_single_h30__sf0', 'Thickness 30m', 'physical'),
    ('copdem_120_single_h136__sf0', 'Thickness 136m', 'physical'),
    ('copdem_120_single_mu0p08__sf0', 'Friction mu=0.08', 'physical'),
    ('copdem_120_single_mu0p25__sf0', 'Friction mu=0.25', 'physical'),
    ('copdem_120_single_xi500__sf0', 'Drag xi=500', 'physical'),
    ('copdem_120_single_xi2000__sf0', 'Drag xi=2000', 'physical'),
]


def sha(p):
    digest = hashlib.sha256()
    with p.open('rb') as stream:
        for block in iter(lambda: stream.read(1024*1024), b''):
            digest.update(block)
    return digest.hexdigest()


def rows(name):
    return list(csv.DictReader((R/'results'/name).open()))


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--require-complete', action='store_true')
    args = parser.parse_args()
    runs = {r['run_id']: r for r in rows('official_runs.csv')}
    metrics = {r['run_id']: r for r in rows('official_metrics.csv')}
    fields = {r['run_id']: r for r in
    ↪  json.loads((R/'results/official_fields.json').read_text())['fields']}
    arrivals = rows('official_arrivals.csv')
    expected = [r[0] for r in CONFIGS]
    missing = [rid for rid in expected if rid not in runs or runs[rid]['status'] !=
    ↪  'completed']
    if args.require_complete:
        assert not missing, missing
```

```
    loaded, output = {}, []
    for rid, label, kind in CONFIGS:
        if rid in missing:
            continue
        rec = fields[rid]
        path = Path(rec['path'])
        assert sha(path) == rec['sha256']
        cp = CACHE/rid/'case.json'
        assert sha(cp) == rec['case_sha256']
        case = json.loads(cp.read_text())
        with np.load(path) as d:
            loaded[rid] = {k: d[k] for k in d.files}
        d = loaded[rid]
        t, h = d['time_s'], d['depth_frames']
        assert np.isfinite(t).all() and np.all(np.diff(t) > 0) and t[-1] >= case['end_seconds']
        m = metrics[rid]
        def number(key):
            return float(m[key]) if m.get(key) not in [None, ''] else None
        output.append({'run_id': rid, 'label': label, 'comparison_role': kind,
                       'fields': rec, 'actual_end_time_s': float(t[-1]),
                       'depth_max_saved_m': number('max_depth_over_saved_outputs_m'),
                       'horizontal_speed_max_saved_m_s':
                       ↪  number('max_horizontal_speed_over_saved_outputs_m_s'),
                       'native_cumulative_speed_scalar_m_s':
                       ↪  number('native_global_cumulative_speed_scalar_m_s'),
                       'saved_cumulative_footprint_area_m2': number('impact_area_m2'),
                       'native_internal_summary_budget_relative_residual':
                       ↪  number('native_internal_summary_mass_budget_relative_residual'),
                       'ASCII_integral_budget_relative_residual':
                       ↪  number('native_mass_budget_relative_residual'),
                       'paper_target_records': [a for a in arrivals if a['run_id'] == rid],
                       'case': case})
    reference = output[0]
    for row in output:
        row['relative_to_reference'] = {
            key: row[key]/reference[key]-1 for key in [
                'depth_max_saved_m', 'horizontal_speed_max_saved_m_s',
                ↪  'saved_cumulative_footprint_area_m2']}
    # A strict registration-level contrast; saved actual native controls are indexed
    ↪  separately.
    cflid = CONFIGS[1][0]
    cfl_check = None
    if cflid in loaded:
        a, b = [next(r['case'] for r in output if r['run_id'] == rid).copy() for rid in
        ↪  [expected[0], cflid]]
        for case in (a, b):
            for k in ['run_id', 'parent_configuration_run_id', 'extras']:
                case.pop(k, None)
        assert a == b
        assert next(r['case'] for r in output if r['run_id'] == cflid)['extras'] == {'cfl':
        ↪  .25}
        cfl_check = {'registration_other_fields_identical': True,
                     'reference_native_default_CFL': .5, 'challenge_CFL': .25,
                     'execution_control_provenance':
                     ↪  'results/official_native_control_index.json; registration equality
                     ↪  alone is not proof of executed control equality'}
        cross_path = R/'results/official_cfl_control_crosscheck.json'
        cross = json.loads(cross_path.read_text())
        for rec in cross['native_parameter_files']:
            assert sha(Path(rec['path'])) == rec['sha256']
        cfl_check['actual_native_control_crosscheck'] = {
            'path': str(cross_path.relative_to(ROOT)), 'bytes': cross_path.stat().st_size,
```

```
            'sha256': sha(cross_path), 'only_numeric_control_difference': 'CFL criterion 0.50 to 0.25',
            'actual_binary_sha256': cross['actual_binary_sha256']}
    doc = {'generated_at_utc': datetime.now(timezone.utc).isoformat(),
           'status': 'complete_registered_single_comparison' if not missing else 'provisional_completed_subset',
           'required_ids': expected, 'pending_ids': missing, 'runs': output, 'CFL_pair': cfl_check,
           'interpretation': 'Numerical and physical variations are separated. Two spatial grids or two CFL values do not establish asymptotic convergence. Peaks from saved fields can depend on temporal sampling; native cumulative scalar is a different quantity. The native cumulative speed also changes under the CFL-only contrast, so stored-output sampling cannot explain the whole difference.',
           'independent_local_event_validation_count': 0, 'predictive_qualification': False}
    (R/'results/native_sensitivity_comparison.json').write_text(json.dumps(doc, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
    fig, axes = plt.subplots(2, 3, figsize=(15, 9), layout='constrained')
    for rid, label, kind in CONFIGS[:3]:
        if rid not in loaded:
            continue
        d = loaded[rid]
        t, h = d['time_s'], d['depth_frames']
        cellarea = abs(float(d['transform'][0]*d['transform'][4]))
        speed = np.where(h >= .1, d['horizontal_speed_frames'], 0).reshape(len(t), -1).max(1)
        for ax, values in zip(axes[0], [d['volume_m3']/1e6, speed, (h >= .1).sum((1, 2))*cellarea/1e6]):
            ax.plot(t, values, label=label, lw=1.3)
    for ax, label in zip(axes[0], ['Saved domain volume (Mm³)', 'Wet horizontal speed peak (m/s)', 'Instantaneous wet area (km²)']):
        ax.set(xlabel='Actual seconds after bulk release', ylabel=label)
        ax.grid(alpha=.2)
        ax.legend(fontsize=8)
    colors = {'reference': '#676767', 'numerical': '#c47b20', 'physical': '#3d76a2'}
    for ax, key, label in zip(axes[1],
                              ['depth_max_saved_m', 'horizontal_speed_max_saved_m_s', 'saved_cumulative_footprint_area_m2'],
                              ['Saved depth peak', 'Saved horizontal speed peak', 'Cumulative wet footprint']):
        values = [100*row['relative_to_reference'][key] for row in output]
        ax.barh([r['label'] for r in output], values, color=[colors[r['comparison_role']] for r in output])
        ax.invert_yaxis()
        ax.axvline(0, color='black', lw=.6)
        ax.set(xlabel='Change from 120m reference (%)', title=label)
        ax.grid(axis='x', alpha=.2)
    scope = 'All ten registered comparisons' if not missing else 'Completed subset only'
    fig.suptitle('Registered single-phase sensitivity: orange numerical settings; blue conditional physical inputs\n'+scope+'; no convergence or independently validated prediction claim', fontsize=11)
    fig.savefig(R/'figures/comparison_single_sensitivity.png', dpi=160)
    plt.close(fig)
    print(json.dumps({'completed_single': len(output), 'pending_single': missing,
    ↪  'registration_CFL_contrast_verified': bool(cfl_check)}))


if __name__ == '__main__':
    main()
```

程序 50：official_result_matrix.py

来源：track_b/research/q4/round3/scripts/official_result_matrix.py
SHA256: 3d3c567a8bf86ab4f3191253f12016ed11cc57a4fd58772c2dd617e6fa6055d7

```
#!/usr/bin/env python3
"""Summarize closed, registered runs without qualifying their numerical accuracy."""
from pathlib import Path
import csv, datetime, hashlib, json
import numpy as np
from official_catalogue import registered_cases, applicability, EXCLUDED, DIAGNOSTIC

P = Path(__file__).resolve().parents[1]
R = P / 'results'

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def main():
    metrics_path = R / 'official_metrics.csv'
    arrivals_path = R / 'official_arrivals.csv'
    validation_path = R / 'official_validation.json'
    metrics = {x['run_id']: x for x in csv.DictReader(metrics_path.open())}
    arrivals = {}
    for row in csv.DictReader(arrivals_path.open()):
        arrivals.setdefault(row['run_id'], {})[row['target']] = row
    checks = {x['run_id']: x for x in json.loads(validation_path.read_text())['checks']}
    decisions = applicability()
    rows, phase_ledgers = [], []
    for item in registered_cases():
        rid = item['run_id']
        decision = decisions.get(rid, {}).get('decision', 'required')
        c = json.loads(Path(item['case']).read_text())
        row = {'run_id': rid, 'scenario_id': c.get('scenario_id'),
               'model': c['model'], 'resolution_m': c['resolution_m'],
               'applicability_decision': decision, 'case_sha256': sha(item['case']),
               'requested_release_volume_m3': c.get('release_grid_volume_m3'),
               'requested_horizon_seconds': c.get('end_seconds'),
               'mu_basal': c.get('mu_basal'), 'xi': c.get('xi'),
               'release_thickness_m': c.get('release_thickness_m'),
               'numerical_accuracy_qualified': False,
               'local_event_validation_count': 0}
        if decision in EXCLUDED:
            row['execution_state'] = decision
            rows.append(row)
            continue
        m, check = metrics.get(rid), checks.get(rid)
        if not m or not check:
            row['execution_state'] = 'not_yet_in_closed_postprocessed_cohort'
            rows.append(row)
            continue
        path = Path(m['source_fields_path'])
        assert sha(path) == m['source_fields_sha256'] == check['source_fields_sha256']
        fields = np.load(path)
        area = abs(float(fields['transform'][0]) * float(fields['transform'][4]))
        h = fields['depth_final'].astype(float)
        v = fields['stationary_criterion_speed_final'].astype(float)
        wet = h >= .1
        stationary = wet & np.isfinite(v) & (v < .1)
        row.update({
            'execution_state': m['status'],
            'actual_end_seconds': float(fields['time_s'][-1]),
```

```
            'source_fields_path': str(path),
            'source_fields_sha256': m['source_fields_sha256'],
            'parser_sha256_at_start': m['parser_sha256_at_start'],
            'execution_sha256': m['execution_sha256'],
            'log_sha256': m['log_sha256'],
            'solver_build': m['solver_build'],
            'actual_binary_sha256': m['actual_binary_sha256'],
            'execution_image_id': m['execution_image_id'],
            'stationary_residual_volume_m3': float(np.sum(h[stationary]) * area),
            'stationary_residual_cells': int(stationary.sum()),
            'final_wet_min_horizontal_speed_m_s': float(np.min(v[wet])) if wet.any() else None,
            'initial_domain_volume_m3': float(fields['volume_m3'][0]),
            'final_domain_volume_m3': float(fields['volume_m3'][-1]),
            'initial_domain_mass_kg': float(fields['mass_kg'][0]),
            'final_domain_mass_kg': float(fields['mass_kg'][-1]),
        })
        for key in ['max_depth_over_saved_outputs_m',
                    'max_horizontal_speed_over_saved_outputs_m_s',
                    'native_cumulative_max_depth_m',
                    'native_cumulative_max_speed_m_s', 'impact_area_m2',
                    'native_mass_budget_relative_residual',
                    'native_internal_summary_mass_budget_relative_residual',
                    'estimated_boundary_adjusted_mass_residual_fraction',
                    'mass_change_fraction']:
            row[key] = float(m[key]) if m.get(key) else None
        for target in ['paper_port_symbol', 'paper_junction']:
            a = arrivals.get(rid, {}).get(target, {})
            if a:
                assert a['source_fields_sha256'] == m['source_fields_sha256']
                assert float(a['evaluation_end_time_seconds']) == row['actual_end_seconds']
            for key in ['status', 'arrival_seconds', 'previous_below_threshold_time_seconds',
                        'first_boundary_threshold_time_seconds',
                        'domain_affected_before_arrival']:
                row[target + '_' + key] = a.get(key) or None
        ledger = {'run_id': rid, 'model': c['model'], 'source_fields_sha256':
        m['source_fields_sha256'],
                  'phase_volume_initial_m3': fields['phase_volume_m3'][0].tolist(),
                  'phase_volume_final_m3': fields['phase_volume_m3'][-1].tolist(),
                  'phase_material_names': fields['phase_material_names'].tolist(),
                  'initial_total_mass_kg': row['initial_domain_mass_kg'],
                  'final_total_mass_kg': row['final_domain_mass_kg'],
                  'native_edge_store_phase_volumes_m3':
                  check.get('native_edge_store_outflow_phase_volumes_m3'),
                  'native_signed_entrainment_phase_volumes_m3':
                  check.get('native_signed_entrainment_phase_volumes_m3'),
                  'native_added_entrainment_mass_kg':
                  check.get('native_entrained_added_mass_kg'),
                  'estimated_boundary_mass_out_kg':
                  check.get('estimated_boundary_mass_out_kg'),
                  'native_boundary_phase_budget_available': c['model'] != 'dclaw',
                  'native_conservation_qualified': False,
                  'empty_phase_max_abs_depth_m':
                  check.get('prescribed_empty_phase_max_abs_depth_m'),
                  'empty_phase_max_abs_volume_m3':
                  check.get('prescribed_empty_phase_max_abs_volume_m3')}
        phase_ledgers.append(ledger)
        rows.append(row)
    columns = list(dict.fromkeys(k for row in rows for k in row))
    with (R / 'official_result_matrix.csv').open('w') as f:
        w = csv.DictWriter(f, fieldnames=columns); w.writeheader(); w.writerows(rows)
    excluded = sum(x['applicability_decision'] in EXCLUDED for x in rows)
    closed = sum(x['execution_state'] not in EXCLUDED |
    {'not_yet_in_closed_postprocessed_cohort'} for x in rows)
```

```
    output = {'created_utc': datetime.datetime.now(datetime.timezone.utc).isoformat(),
              'registered_count': len(rows), 'material_excluded_not_run_count': excluded,
              'closed_postprocessed_count': closed,
              'all_applicable_and_diagnostic_closed': closed + excluded == len(rows),
              'sources': [{'path': str(p), 'sha256': sha(p)} for p in [metrics_path,
              ↪  arrivals_path, validation_path]],
              'stationary_residual_definition': 'last actual saved output: h>=0.1m and max
              ↪  horizontal speed over wet native phases (phase h>=1e-6m)<0.1m/s; finite v
              ↪  only; not permanent deposition',
              'phase_budget_definition': 'saved-output domain phase volumes; DClaw solid/fluid
              ↪  from h*m and h*(1-m), ravaflow active native phases; no melt inference',
              'dclaw_boundary_budget_limitation': 'boundary mass estimate is trapezoidal
              ↪  integration of saved output, not native conservative phase flux; no
              ↪  qualification from this estimate',
              'accuracy_or_conservation_qualification': False, 'local_event_validation_count':
              ↪  0,
              'phase_ledgers': phase_ledgers}
    (R / 'official_final_diagnostics.json').write_text(json.dumps(output, indent=2) + '\n')
    print(json.dumps({k: output[k] for k in ['registered_count', 'closed_postprocessed_count',
    ↪  'material_excluded_not_run_count', 'all_applicable_and_diagnostic_closed']}))

if __name__ == '__main__':
    main()
```

程序 51：q4_plot_results.py

来源：track_b/paper/scripts/q4_plot_results.py
SHA256: e27b7003ab9b797f1edca548517bab3f622b0e7af355c5d511d5828372741c27

```
#!/usr/bin/env python3
"""Replot frozen Q4 process fields for the paper; never run a physical model."""
from pathlib import Path
import hashlib
import json
from datetime import datetime, timezone
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.colors import LightSource, LogNorm, SymLogNorm
from matplotlib import font_manager
from pyproj import Transformer

ROOT = Path(__file__).resolve().parents[3]
RESEARCH = ROOT / 'track_b/research/q4/round3'
PAPER = ROOT / 'track_b/paper'
OUT = PAPER / 'figures/q4'
PRIMARY = 'q2full_q2r3_p0036_S_T_linear__geomv2_120m__sf0'
FINE = 'q2full_q2r3_p0036_S_T_linear__geomv2_60m__sf0'
PRESSURE = 'dclaw_ref_extended120'
PHASE = 'composite_RI_water_120__sf0'
USED = []


def sha(path):
    with path.open('rb') as stream:
        return hashlib.file_digest(stream, 'sha256').hexdigest()


def record(path, role):
    return {'path': str(path.relative_to(ROOT)) if path.is_relative_to(ROOT) else str(path),
            'sha256': sha(path), 'bytes': path.stat().st_size, 'role': role}


def load_fields(run_id):
    index = RESEARCH / 'results/official_fields.json'
    row = next(x for x in json.loads(index.read_text())['fields'] if x['run_id'] == run_id)
    path = Path(row['path'])
    assert sha(path) == row['sha256'] and path.stat().st_size == row['bytes']
    USED.append({**record(path, 'frozen native saved process fields'), 'run_id': run_id,
                 'bound_by_repo_index': record(index, 'native field index')})
    with np.load(path) as data:
        return {key: data[key].copy() for key in data.files}


def extent(data):
    tr = data['transform']; ny, nx = data['depth_frames'].shape[1:]
    return [tr[2]/1000, (tr[2]+tr[0]*nx)/1000,
            (tr[5]+tr[4]*ny)/1000, tr[5]/1000]


def setup():
    path = font_manager.findfont(font_manager.FontProperties(family='Arial Unicode MS',
    ↪  weight='normal'))
    font_manager.fontManager.addfont(path)
    plt.rcParams.update({'font.family': 'Arial Unicode MS', 'font.size': 8.0,
```

```
        'axes.titlesize': 9, 'axes.labelsize': 8, 'xtick.labelsize': 7,
        'ytick.labelsize': 7, 'axes.unicode_minus': False, 'pdf.fonttype': 42,
        'ps.fonttype': 42, 'mathtext.fontset': 'stix', 'axes.linewidth': .6})
    return path


def decorate(ax, title, targets, background, ex):
    ax.imshow(background, extent=ex, origin='upper', cmap='gray', vmin=0, vmax=1,
              interpolation='nearest', rasterized=True)
    ax.set(xlim=(339.48, 357.48), ylim=(3127.68, 3136.92), title=title,
           xlabel='东坐标 E（km）', ylabel='北坐标 N（km）')
    ax.set_xticks([340, 345, 350, 355]); ax.set_yticks([3128, 3132, 3136])
    ax.tick_params(length=2, pad=2)
    if ax.get_subplotspec().is_last_col():
        ax.set_ylabel('')
    ax.set_aspect('equal')
    for key, marker in [('J', 'o'), ('P', '^')]:
        x, y = targets[key]
        ax.plot(x, y, marker, ms=3.5, mfc='white', mec='#20242a', mew=.7, zorder=6)
        ax.annotate(key, (x, y), xytext=(4, 4), textcoords='offset points',
                    fontsize=7.5, weight='bold', color='#1b2027', zorder=7)


def save(fig, name):
    pdf = OUT / f'{name}.pdf'; png = OUT / f'{name}.png'
    fig.savefig(pdf, dpi=300, bbox_inches='tight', pad_inches=.03, metadata={'Creator': 'Q4
    ↪  frozen-field paper plot',
                                       'CreationDate': None, 'ModDate': None})
    fig.savefig(png, dpi=300, bbox_inches='tight', pad_inches=.03)
    plt.close(fig)
    return [record(pdf, 'paper figure PDF'), record(png, '300dpi visual review image')]


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    font_path = setup()
    primary, fine, pressure, phase = [load_fields(rid) for rid in [PRIMARY, FINE, PRESSURE,
    ↪  PHASE]]
    grid = RESEARCH/'results/spatial_inputs_120m.npz'
    USED.append(record(grid, 'registered common terrain grid'))
    with np.load(grid) as data:
        terrain = data['z'].copy()
    background = LightSource(azdeg=315, altdeg=45).hillshade(terrain, vert_exag=1, dx=120,
    ↪  dy=120)
    ex = extent(primary)
    geo_path = RESEARCH/'results/computational_geometries.geojson'
    geoms = json.loads(geo_path.read_text())['features']; USED.append(record(geo_path,
    ↪  'candidate target geometry'))
    transform = Transformer.from_crs(4326, 32645, always_xy=True)
    targets = {}
    for key, sid in [('J', 'paper_junction'), ('P', 'paper_port_symbol')]:
        obj = next(g for g in geoms if g['properties'].get('space_id') == sid)
        x, y = transform.transform(*obj['geometry']['coordinates']); targets[key] = [x/1000,
        ↪  y/1000]
    norm = LogNorm(.1, 100)
    fig, axes = plt.subplots(2, 2, figsize=(6.65, 3.9))
    fig.subplots_adjust(left=.08, right=.89, bottom=.11, top=.94, wspace=.22, hspace=.47)
    frames = []
    for letter, ax, target in zip('abcd', axes.flat, [0, 180, 600, 1800]):
        k = int(np.argmin(np.abs(primary['time_s']-target)))
        tau = float(primary['time_s'][k]); h = primary['depth_frames'][k]
        decorate(ax, f'（{letter}）τ = {tau:.1f} s', targets, background, ex)
        im = ax.imshow(np.ma.masked_less(h, .1), extent=ex, origin='upper', cmap='viridis',
```

```
                    norm=norm, interpolation='nearest', rasterized=True, zorder=2)
        frames.append({'requested_time_s': target, 'actual_time_s': tau,
                    'max_saved_depth_m': float(h.max()), 'wet_planar_area_km2':
                    ↪ float((h>=.1).sum()*120**2/1e6)})
    cb = fig.colorbar(im, cax=fig.add_axes([.915, .18, .015, .67]), ticks=[.1, 1, 10, 100])
    cb.set_label('流深 h（m）', labelpad=3); cb.ax.tick_params(labelsize=7)
    outputs = save(fig, 'process_fullmask_st120')

    fig, axes = plt.subplots(2, 2, figsize=(6.65, 4.25))
    fig.subplots_adjust(left=.08, right=.94, bottom=.10, top=.94, wspace=.35, hspace=.48)
    diagnostics = []
    for ax, data, title in [(axes[0,0], primary, '（a）ST 全源面：120 m'),
                            (axes[0,1], fine, '（b）ST 全源面：60 m')]:
        keep = data['time_s'] <= 1800
        hmax = data['depth_frames'][keep].max(axis=0)
        decorate(ax, title, targets, background, ex)
        im = ax.imshow(np.ma.masked_less(hmax, .1), extent=extent(data), origin='upper',
                    cmap='viridis', norm=norm, interpolation='nearest', rasterized=True,
                    ↪ zorder=2)
        cb = fig.colorbar(im, ax=ax, pad=.015, fraction=.035, ticks=[.1, 1, 10, 100])
        cb.ax.tick_params(labelsize=6.5); cb.set_label('最大流深（m）', fontsize=7, labelpad=2)
        diagnostics.append({'panel': title, 'field': 'max saved total depth for actual times
        ↪ <=1800s',
                            'last_selected_time_s': float(data['time_s'][keep][-1]),
                            'threshold_footprint_km2':
                            ↪ float((hmax>=.1).sum()*data['transform'][0]**2/1e6)})
    k = int(np.argmin(np.abs(pressure['time_s']-180)))
    h = pressure['depth_frames'][k]; p = pressure['pressure_frames'][k]/1000
    ax = axes[1,0]; decorate(ax, f'（c）D-Claw：τ = {pressure["time_s"][k]:.0f} s', targets,
    ↪ background, ex)
    p = np.ma.masked_where((h < .1) | ~np.isfinite(p), p)
    im = ax.imshow(p, extent=extent(pressure), origin='upper', cmap='magma',
    ↪ norm=SymLogNorm(linthresh=1, vmin=0, vmax=600),
                interpolation='nearest', rasterized=True, zorder=2)
    cb = fig.colorbar(im, ax=ax, pad=.015, fraction=.035, ticks=[0,1,10,100,600]);
    ↪ cb.set_label('基底压力（kPa）', fontsize=7, labelpad=2)
    cb.ax.tick_params(labelsize=6.5)
    k = int(np.argmin(np.abs(phase['time_s']-180)))
    h = phase['depth_frames'][k]; depths = phase['phase_depth_frames'][k]
    assert depths.shape[0] == 3 and np.max(np.abs(depths[1])) == 0
    water = np.divide(depths[2], depths.sum(axis=0), out=np.zeros_like(h),
    ↪ where=depths.sum(axis=0)>0)
    ax = axes[1,1]; decorate(ax, f'（d）复合固水：τ = {phase["time_s"][k]:.1f} s', targets,
    ↪ background, ex)
    im = ax.imshow(np.ma.masked_where(h<.1, water), extent=extent(phase), origin='upper',
                cmap='Blues', vmin=0, vmax=1, interpolation='nearest', rasterized=True,
                ↪ zorder=2)
    cb = fig.colorbar(im, ax=ax, pad=.015, fraction=.035, ticks=[0, .5, 1]);
    ↪ cb.set_label('水相厚度份额', fontsize=7, labelpad=2)
    cb.ax.tick_params(labelsize=6.5)
    outputs += save(fig, 'model_comparison')
    report = {'schema_version':'q4-paper-plot-manifest-v1', 'command':'track_b/.venv/bin/python
    ↪ track_b/paper/scripts/q4_plot_results.py',
        'source_files':USED, 'outputs':outputs, 'main_process_frames':frames,
        'comparison_panels':diagnostics, 'target_markers_utm_km':targets,
        'font_file':font_path, 'comparison_horizon_seconds':1800,
        'scope':'Frozen conditional flow fields. ST panels use V31.184139Mm3;
        ↪ pressure/composite panels use distinct baseline V13.878Mm3. Geographic panel limits
        ↪ show common core; expanded pressure run solved larger domain.',
        'threshold_depth_m':.1, 'target_buffer_radius_m':100,
        'pressure_color_scale':'SymLogNorm: linear0-1kPa, logarithmic1-600kPa',
        'pressure_actual_time_s':180.0, 'water_fraction_actual_time_s':180.5,
        ↪ 'time_origin':'given bulk release',
```

```
        'initiation_to_release_delay_seconds':None,
        'independent_local_event_validation_count':0,
        'physical_model_runs_added':0}
    (PAPER/'evidence/q4_plot_manifest.json').write_text(json.dumps(report, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
    print(json.dumps({'process_frames':frames,'comparison_panels':diagnostics,'outputs':[x['path'] for x in outputs]},ensure_ascii=False))


if __name__ == '__main__':
    main()
```

程序 52：论文论断与数值来源核对

来源：track_b/paper/scripts/build_evidence.py
SHA256: 9ffc9e321f6fe5e49aec6420522c447a3300d44213e50a15cb81cc6f304cbb02

```
#!/usr/bin/env python3
# AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24.
# Contributions and model identity are recorded in AI_DISCLOSURE.md.
"""Merge chapter evidence and verify immutable repository source hashes.

No model is run.  Explicit ``checks`` permit JSON-pointer or CSV-cell numerical
checks; other values remain author assertions linked to hash-verified sources.
"""
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import math
from pathlib import Path


PAPER = Path(__file__).resolve().parents[1]
REPO = PAPER.parents[1]


def digest(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            h.update(block)
    return h.hexdigest()


def write_json(path: Path, value) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")


def source_record(value) -> dict:
    if isinstance(value, str):
        return {"path": value, "sha256": None}
    if not isinstance(value, dict):
        raise ValueError("source_files entries must be paths or objects")
    return {**value, "path": value.get("path", value.get("repo_path", value.get("file",
    ↪  value.get("source_path")))),
            "sha256": value.get("sha256", value.get("hash", value.get("sha")))}


def repository_path(root: Path, name: str) -> Path:
    if not isinstance(name, str) or not name:
        raise ValueError("empty source path")
    if Path(name).is_absolute():
        raise ValueError(f"repository-relative path required: {name}")
    resolved = (root / name).resolve()
    if not resolved.is_relative_to(root.resolve()):
        raise ValueError(f"source escapes repository: {name}")
    return resolved


def read_claims(path: Path) -> list:
```

```
    data = json.loads(path.read_text(encoding="utf-8"))
    if isinstance(data, list):
        return data
    if isinstance(data, dict):
        for key in ("claims", "evidence", "records"):
            if isinstance(data.get(key), list):
                return data[key]
    raise ValueError(f"{path.name}: expected a list or a claims list")


def pointer(data, expr: str):
    if expr in ("", "/"):
        return data
    if not expr.startswith("/"):
        raise ValueError("JSON pointer must begin with /")
    for part in expr[1:].split("/"):
        part = part.replace("~1", "/").replace("~0", "~")
        data = data[int(part)] if isinstance(data, list) else data[part]
    return data


def equal(actual, expected, rtol: float, atol: float) -> bool:
    if isinstance(expected, bool) or expected is None:
        return actual == expected
    if isinstance(expected, (int, float)):
        try:
            return math.isclose(float(actual), expected, rel_tol=rtol, abs_tol=atol)
        except (TypeError, ValueError):
            return False
    if isinstance(expected, list):
        return isinstance(actual, list) and len(actual) == len(expected) and all(
            equal(a, b, rtol, atol) for a, b in zip(actual, expected))
    if isinstance(expected, dict):
        return isinstance(actual, dict) and all(k in actual and equal(actual[k], v, rtol, atol)
                                                for k, v in expected.items())
    return str(actual) == str(expected)


def check_value(check: dict, root: Path, known_sources: set[str]) -> dict:
    record = dict(check)
    try:
        name = check.get("source_path", check.get("path"))
        if name not in known_sources:
            raise ValueError("numeric check source must occur in claim source_files")
        path = repository_path(root, name)
        if "json_pointer" in check:
            actual = pointer(json.loads(path.read_text(encoding="utf-8")),
            ↪  check["json_pointer"])
        elif "column" in check:
            with path.open(encoding="utf-8-sig", newline="") as stream:
                rows = list(csv.DictReader(stream, delimiter=check.get("delimiter", "\t" if
                ↪  path.suffix == ".tsv" else ",")))
            where = check.get("where", {})
            matches = [row for row in rows if all(str(row.get(k)) == str(v) for k, v in
            ↪  where.items())]
            if "row" in check:
                matches = [matches[int(check["row"])]]
            if len(matches) != 1:
                raise ValueError(f"CSV selector selected {len(matches)} rows; exactly one
                ↪  required")
            actual = matches[0][check["column"]]
        else:
            raise ValueError("numeric check requires json_pointer or column")
```

```
        if "expected" not in check:
            raise ValueError("numeric check requires explicit expected value")
        record["actual"] = actual
        record["passed"] = equal(actual, check["expected"], float(check.get("rtol", 1e-8)),
        ↪  float(check.get("atol", 1e-12)))
    except (OSError, ValueError, KeyError, IndexError, TypeError) as exc:
        record.update(passed=False, error=str(exc))
    return record


def build(paper: Path, root: Path) -> tuple[dict, dict]:
    claims, issues, source_checks, value_checks, files = [], [], {}, [], []
    seen_ids = set()
    input_paths = sorted((paper / "evidence").glob("q[1-4]_claims.json"))
    for shared_name in ("common_claims.json", "public_claims.json"):
        if (paper / "evidence" / shared_name).exists():
            input_paths.append(paper / "evidence" / shared_name)
    for path in input_paths:
        files.append(str(path.relative_to(paper)))
        try:
            rows = read_claims(path)
        except (OSError, ValueError) as exc:
            issues.append(str(exc))
            continue
        for i, original in enumerate(rows):
            if not isinstance(original, dict):
                issues.append(f"{path.name}[{i}] is not an object")
                continue
            row = dict(original)
            claim_id = row.get("claim_id", row.get("id"))
            if not claim_id or claim_id in seen_ids:
                issues.append(f"missing or duplicate claim_id: {claim_id!r} in {path.name}")
            seen_ids.add(claim_id)
            row["claim_id"] = claim_id
            row["chapter_evidence"] = str(path.relative_to(paper))
            raw_sources = row.get("source_files", row.get("sources", []))
            if isinstance(raw_sources, dict):
                raw_sources = [raw_sources] if any(k in raw_sources for k in ("path",
                ↪  "repo_path", "file", "source_path")) else [
                    {"path": k, "sha256": v} for k, v in raw_sources.items()]
            normalized, names = [], set()
            if not raw_sources:
                issues.append(f"{claim_id}: no source_files")
            for raw in raw_sources:
                try:
                    source = source_record(raw)
                    name, expected = source["path"], source["sha256"]
                    source_path = repository_path(root, name)
                    if source_path in (paper / "evidence/claim_index.json", paper /
                    ↪  "qa/evidence_check.json"):
                        raise ValueError("generated evidence index/report cannot be its own
                        ↪  input source")
                    actual = digest(source_path)
                    passed = isinstance(expected, str) and expected.lower() == actual
                    key = (name, expected)
                    source_checks[key] = {"path": name, "expected_sha256": expected,
                    ↪  "actual_sha256": actual, "passed": passed}
                    if not passed:
                        issues.append(f"{claim_id}: missing or mismatching source hash:
                        ↪  {name}")
                    normalized.append({**source, "sha256": expected, "verified": passed})
                    names.add(name)
                except (ValueError, OSError, TypeError) as exc:
```

```
                    issues.append(f"{claim_id}: {exc}")
            row["source_files"] = normalized
            checks = [check_value(c, root, names) for c in row.get("checks", [])]
            for check in checks:
                value_checks.append({"claim_id": claim_id, **check})
                if not check["passed"]:
                    issues.append(f"{claim_id}: explicit value check failed")
            row["value_verification"] = "explicit_source_value_checks" if checks else
            ↪  "source_hash_verified_author_value_mapping"
            row["checks"] = checks
            claims.append(row)
    if not files:
        issues.append("no q1-q4 chapter claim files found")
    chapters = {Path(p).name.split("_")[0] for p in files if Path(p).name.startswith("q")}
    missing = sorted({"q1", "q2", "q3", "q4"} - chapters)
    report = {
        "schema_version": 1, "passed": not issues, "input_files": files,
        "chapters_present": sorted(chapters), "chapters_pending": missing,
        "claim_count": len(claims), "source_count": len(source_checks),
        "explicit_value_check_count": len(value_checks),
        "author_value_mapping_count": sum(not r["checks"] for r in claims),
        "value_check_meaning": "Only explicit checks compare source values automatically;
        ↪  source hashes alone establish provenance, not numeric agreement.",
        "source_checks": sorted(source_checks.values(), key=lambda x: (x["path"],
        ↪  str(x["expected_sha256"]))),
        "value_checks": value_checks, "issues": issues,
    }
    index = {"schema_version": 1, "repository_root": "../..", "claims": claims}
    return index, report


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--paper", type=Path, default=PAPER)
    parser.add_argument("--repo", type=Path, default=REPO)
    parser.add_argument("--require-all", action="store_true", help="fail if a chapter has not
    ↪  delivered its evidence")
    args = parser.parse_args()
    paper, root = args.paper.resolve(), args.repo.resolve()
    index, report = build(paper, root)
    if args.require_all and report["chapters_pending"]:
        report["issues"].append("missing chapters: " + ", ".join(report["chapters_pending"]))
        report["passed"] = False
    write_json(paper / "evidence/claim_index.json", index)
    write_json(paper / "qa/evidence_check.json", report)
    print(json.dumps({k: report[k] for k in ("passed", "claim_count", "source_count",
    ↪  "explicit_value_check_count", "chapters_pending", "issues")}, ensure_ascii=False))
    return 0 if report["passed"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

程序 53：参考文献按首次引用排序

来源：track_b/paper/scripts/build_references.py
SHA256: 10f0f421843ed048b0672e85ed77a397556cea1dffef3d392357eeda69b68d2c

```
#!/usr/bin/env python3
# AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24.
# Contributions and model identity are recorded in AI_DISCLOSURE.md.
"""Build a bibliography in actual first-citation order from chapter bibitems."""
from __future__ import annotations

import argparse
import json
import re
from pathlib import Path

PAPER = Path(__file__).resolve().parents[1]


def uncomment(text: str) -> str:
    return re.sub(r"(?<!\\)%[^\n]*", "", text)


def tex_files(paper: Path, entry: str = "main.tex", seen=None) -> list[Path]:
    """TeX input paths are relative to the compilation root, not each section."""
    seen = set() if seen is None else seen
    path = (paper / entry).resolve()
    if path in seen or not path.is_file():
        return []
    if not path.is_relative_to(paper.resolve()):
        raise ValueError(f"TeX input escapes paper root: {entry}")
    seen.add(path)
    result = [path]
    for match in re.finditer(r"\\(?:input|include)\s*\{([^}]+)\}",
    ↪  uncomment(path.read_text(encoding="utf-8"))):
        name = match.group(1)
        result.extend(tex_files(paper, name if Path(name).suffix else name + ".tex", seen))
    return result


def expanded_tex(paper: Path, entry: str = "main.tex", stack=None) -> str:
    stack = set() if stack is None else stack
    path = (paper / entry).resolve()
    if not path.is_file():
        return ""
    if not path.is_relative_to(paper.resolve()):
        raise ValueError(f"TeX input escapes paper root: {entry}")
    if path in stack:
        raise ValueError(f"recursive TeX input: {entry}")
    text = uncomment(path.read_text(encoding="utf-8"))
    def replace(match):
        name = match.group(1)
        return expanded_tex(paper, name if Path(name).suffix else name + ".tex", stack |
        ↪  {path})
    return re.sub(r"\\(?:input|include)\s*\{([^}]+)\}", replace, text)


def citations(text: str) -> list[str]:
    keys = []
    pattern = r"\\(?:cite\w*|nocite)\*?(?:\s*\[[^\]]*\]){0,2}\s*\{([^}]+)\}"
    for match in re.finditer(pattern, text):
        for key in match.group(1).split(","):
```

```
            key = key.strip()
            if key and key not in keys:
                keys.append(key)
    return keys


def bibitems(path: Path) -> list[tuple[str, str]]:
    text = uncomment(path.read_text(encoding="utf-8"))
    text = re.sub(r"\\end\{thebibliography\}.*", "", text, flags=re.S)
    matches = list(re.finditer(r"\\bibitem(?:\s*\[[^\]]*\])?\s*\{([^}]+)\}", text))
    return [(m.group(1), text[m.start():matches[i + 1].start() if i + 1 < len(matches) else len(text)].strip())
            for i, m in enumerate(matches)]


def build(paper: Path) -> tuple[str, dict]:
    sources = sorted((paper / "evidence").glob("q[1-4]_references.tex"))
    common = paper / "evidence/common_references.tex"
    if common.exists():
        sources.append(common)
    definitions, origins, duplicates, conflicts = {}, {}, [], []
    for path in sources:
        for key, item in bibitems(path):
            if key in definitions:
                identical = re.sub(r"\s+", " ", definitions[key]) == re.sub(r"\s+", " ", item)
                duplicates.append({"key": key, "sources": [origins[key], str(path.relative_to(paper))], "identical": identical})
                if not identical:
                    conflicts.append(key)
            else:
                definitions[key], origins[key] = item, str(path.relative_to(paper))
    order = citations(expanded_tex(paper))
    missing = [key for key in order if key not in definitions]
    content = "% Generated by scripts/build_references.py; edit evidence/*_references.tex.\n"
    content += "\\begin{thebibliography}{99}\n"
    content += "\n\n".join(definitions[key] for key in order if key in definitions) + "\n"
    content += "\\end{thebibliography}\n"
    report = {"schema_version": 1, "passed": not missing and not conflicts,
              "citation_order": order, "reference_count": len(order) - len(missing),
              "sources": [str(p.relative_to(paper)) for p in sources],
              "duplicate_keys": duplicates, "conflicting_keys": conflicts, "missing_keys": missing,
              "unused_keys": sorted(set(definitions) - set(order))}
    return content, report


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--paper", type=Path, default=PAPER)
    args = parser.parse_args()
    paper = args.paper.resolve()
    content, report = build(paper)
    (paper / "qa").mkdir(parents=True, exist_ok=True)
    (paper / "qa/references_check.json").write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    if report["passed"]:
        (paper / "references.tex").write_text(content, encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False))
    return 0 if report["passed"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

程序 54：自编程序清单与完整源码附录

来源：track_b/paper/scripts/build_program_appendix.py
SHA256: 798e0906e8702b14cba0704d3e4cdb656b9b8f330dc8eab43ace7c7762f7b46e

```
#!/usr/bin/env python3
# AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24.
# Contributions and model identity are recorded in AI_DISCLOSURE.md.
"""Copy adopted original programs and build a portable complete-source appendix."""
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path
import re
import shutil

from build_evidence import repository_path, write_json

PAPER = Path(__file__).resolve().parents[1]
REPO = PAPER.parents[1]


def digest(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()


def anonymize_paths(data: bytes) -> tuple[bytes, int]:
    """Display-only path prefix substitution; original bytes stay available."""
    text = data.decode("utf-8")
    text, count = re.subn(r"/Users/[A-Za-z0-9_.-]+(?=/|\b)", "<LOCAL_HOME>", text)
    return text.encode("utf-8"), count


def escape(text) -> str:
    replacements = {"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#",
                    "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^":
                    ↪  r"\textasciicircum{}"}
    return "".join(replacements.get(c, c) for c in str(text))


def normalize_entry(value) -> dict:
    if isinstance(value, str):
        return {"path": value}
    if not isinstance(value, dict):
        raise ValueError("program entry must be path or object")
    return {**value, "path": value.get("path", value.get("repo_path", value.get("file",
    ↪  value.get("source_path"))))}


def software_label(record: dict) -> str:
    """Human-readable identity only; full metadata remains in the JSON index."""
    parts = [escape(record.get("name", " 第三方软件"))]
    for key, label in (("version", " 版本"), ("release", " 发布")):
        if isinstance(record.get(key), (str, int, float)):
            parts.append(label + " " + escape(record[key]))
    commit = record.get("commit", record.get("revision"))
    if isinstance(commit, str) and commit:
        parts.append(" 提交 " + r"\texttt{" + escape(commit[:12]) + "}")
    return ", ".join(parts)

```

```

def third_party_item(record: dict) -> str:
    text = r"\item " + software_label(record)
    dependency = record.get("numerical_dependency")
    if isinstance(dependency, dict):
        text += "；数值依赖：" + software_label(dependency)
    dependencies = record.get("dependencies", [])
    if isinstance(dependencies, dict):
        dependencies = [{"name": name, **value} if isinstance(value, dict)
                        else {"name": name, "version": value} for name, value in
                        ↪  dependencies.items()]
    if isinstance(dependencies, list):
        labels = [software_label(value) for value in dependencies if isinstance(value, dict)]
        if labels:
            text += "；配套依赖：" + "；".join(labels)
    url = record.get("source_url", record.get("url"))
    if isinstance(url, str) and url:
        text += "。源码：" + r"\url{" + url + "}"
    return text + "。\n"


def entries(path: Path) -> tuple[list, list]:
    data = json.loads(path.read_text(encoding="utf-8"))
    if isinstance(data, list):
        return data, []
    if not isinstance(data, dict):
        raise ValueError(f"{path.name}: expected list or object")
    original = None
    accepted_keys = ("code_files", "files", "programs", "self_authored", "self_authored_files")
    for key in accepted_keys:
        if key in data:
            if not isinstance(data[key], list):
                raise ValueError(f"{path.name}: {key} must be a list")
            original = data[key]
            break
    if original is None:
        if data:
            raise ValueError(f"{path.name}: unrecognized self-authored program list; expected
            ↪  one of {', '.join(accepted_keys)}")
        original = []
    third_party = data.get("third_party", data.get("third_party_solvers", []))
    if isinstance(third_party, dict):
        third_party = [{"name": k, **v} if isinstance(v, dict) else {"name": k, "version": v}
                       for k, v in third_party.items()]
    derivative = data.get("third_party_derivative_build")
    if isinstance(derivative, dict):
        third_party.append(derivative)
    return original, third_party


def build(paper: Path, repo: Path) -> tuple[str, dict]:
    records, third_party, issues, seen = [], [], [], {}
    listings = sorted((paper / "evidence").glob("q[1-4]_code_files.json"))
    if (paper / "evidence/common_code_files.json").exists():
        listings.append(paper / "evidence/common_code_files.json")
    for listing in listings:
        chapter = listing.name.split("_")[0]
        try:
            programs, third = entries(listing)
            third_party.extend({"chapter": chapter, **(r if isinstance(r, dict) else {"name":
            ↪  str(r)})} for r in third)
            for item in programs:
                entry = normalize_entry(item)
```

```
            if entry.get("type", entry.get("kind", "")) in ("third_party", "external", "solver"):
                third_party.append({"chapter": chapter, **entry})
                continue
            name = entry["path"]
            source = repository_path(repo, name)
            if not source.is_file():
                raise ValueError(f"missing program: {name}")
            if name.startswith("track_b/paper/code/") or name.startswith("track_b/paper/appendices/"):
                raise ValueError(f"generated code/appendix cannot source itself: {name}")
            data = source.read_bytes()
            data.decode("utf-8")  # Complete source appendix must be text.
            actual = digest(data)
            display, substitutions = anonymize_paths(data)
            expected = entry.get("sha256")
            if expected and expected != actual:
                raise ValueError(f"program hash mismatch: {name}")
            if name in seen:
                seen[name]["chapters"].append(chapter)
                continue
            record = {"source_path": name, "portable_path": "code/" + name,
                      "display_path": "code_print/" + name, "display_sha256": digest(display),
                      "path_anonymization_count": substitutions,
                      "sha256": actual, "bytes": len(data), "lines": len(data.splitlines()),
                      "chapters": [chapter], "title": entry.get("title", entry.get("description", entry.get("role", source.name)))}
            seen[name] = record
            records.append(record)
    except (ValueError, OSError, TypeError, UnicodeError) as exc:
        issues.append(f"{listing.name}: {exc}")
if not listings:
    issues.append("no q1-q4 code lists found")
report = {"schema_version": 1, "passed": not issues, "sources": [str(p.relative_to(paper)) for p in listings],
          "program_count": len(records), "source_lines": sum(r["lines"] for r in records),
          "chapter_program_counts": {chapter: sum(chapter in r["chapters"] for r in records)
                                     for chapter in sorted({p.name.split('_')[0] for p in listings})},
          "programs": records, "third_party": third_party, "issues": issues,
          "copy_policy": "Full UTF-8 original sources copied byte-for-byte; third-party solvers indexed by declared version/configuration only.",
          "display_copy_policy": "Only /Users/[A-Za-z0-9_.-]+ path prefixes are replaced with <LOCAL_HOME>; equations, parameters and program logic are otherwise unchanged. Original portable copies preserve exact source bytes."}
chunks = ["% Generated by scripts/build_program_appendix.py.\n",
          r"\section{论文采用的自编程序}" + "\n",
          "本附录按采用清单列出完整自编源码。
          完整原程序在配套工程的\\texttt{code/}中按原字节保存；
          本附录的显示副本仅将本机用户目录前缀替换为\\texttt{<LOCAL\\_HOME>}，公式、
          参数和程序逻辑保持原样。源码行号与折行便于核查；第三方求解器以版本和配置索引。\n",
          r"\begin{longtable}{p{0.07\textwidth}p{0.72\textwidth}r}" + "\n",
          r"\toprule 序号 & 程序与用途 & 行数\\\midrule\endhead" + "\n"]
for i, record in enumerate(records, 1):
    title = escape(record["title"])
    chunks.append(f"{i} & {title} & {record['lines']}\\\\\n")
chunks.append(r"\bottomrule\end{longtable}" + "\n")
if third_party:
    chunks.extend([r"\subsection*{第三方框架版本索引}" + "\n", r"\begin{itemize}" + "\n"])
```

```
        for record in third_party:
            chunks.append(third_party_item(record))
        chunks.append(r"\end{itemize}" + "\n")
        chunks.append("完整版本元数据、源记录路径与哈希见配套工程的" +
        ↪  r"\nolinkurl{evidence/program_manifest.json}" + "。\n")
    for i, record in enumerate(records, 1):
        chunks.append(r"\clearpage" + "\n")
        chunks.append(r"\subsection*{" + f" 程序 {i}: " + escape(record["title"]) + "}\n")
        chunks.append(r"\noindent 来源：\nolinkurl{" + record["source_path"] + "}\\\\\n")
        chunks.append(r"\noindent SHA256：\texttt{" + record["sha256"][:32] +
        ↪  r"}\allowbreak\texttt{" + record["sha256"][32:] + "}\\\\\n")
        # Direct VerbatimInput keeps copied source exact and does not rely on a path
        # relative to appendices/. Compilation always takes place at paper root.
        chunks.append(r"\VerbatimInput[breaklines=true,breakanywhere=true,numbers=left,numbers」
        ↪  ep=5pt,fontsize=\footnotesize,tabsize=4]{" + record["display_path"] + "}\n")
    return "".join(chunks), report


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--paper", type=Path, default=PAPER)
    parser.add_argument("--repo", type=Path, default=REPO)
    args = parser.parse_args()
    paper, repo = args.paper.resolve(), args.repo.resolve()
    text, report = build(paper, repo)
    write_json(paper / "qa/programs_check.json", report)
    if report["passed"]:
        for record in report["programs"]:
            target = paper / record["portable_path"]
            target.parent.mkdir(parents=True, exist_ok=True)
            shutil.copyfile(repo / record["source_path"], target)
            display_target = paper / record["display_path"]
            display_target.parent.mkdir(parents=True, exist_ok=True)
            display_target.write_bytes(anonymize_paths(target.read_bytes())[0])
        write_json(paper / "evidence/program_manifest.json", report)
        (paper / "appendices").mkdir(parents=True, exist_ok=True)
        (paper / "appendices/programs.tex").write_text(text, encoding="utf-8")
    print(json.dumps({k: report[k] for k in ("passed", "program_count", "source_lines",
    ↪  "issues")}, ensure_ascii=False))
    return 0 if report["passed"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

程序 55：参数范围与机制分类附表生成

来源：track_b/paper/scripts/build_supplement_tables.py
SHA256: 61e99652db2555ba907b459fb48401522104a03d093957257397caaee6c41f5d

```
#!/usr/bin/env python3
"""Build supplemental tables from frozen results. AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24."""
from pathlib import Path
import csv,json,hashlib
REPO=Path(__file__).resolve().parents[3]
PAPER=REPO/'track_b/paper'
def esc(s):
    return str(s).replace('_',r'\_').replace('%',r'\%').replace('&',r'\&')
def fmt(v):
    return f'{float(v):.5g}'
names={
'theta_deg':('结构面倾角',r'$^\circ$'),
'thickness_normal_m':('法向厚度','m'),
'rock_volume_fraction':('岩体体积分数','1'),
'bridge_fraction':('岩桥承载面积比例','1'),
'bridge_cohesion_kpa':('岩桥有效黏聚力','kPa'),
'rock_mu':('岩面摩擦系数','1'),
'initial_temp_c':('初始裂隙温度',r'$^\circ$C'),
'diffusivity_m2_s':('热扩散率',r'm$^2$/s'),
'interface_depth_m':('热路径深度','m'),
'station_source_offset_c':('站点至源区温度偏移',r'$^\circ$C'),
'storativity':('储水量/水头比','1'),
'drainage_hours':('排水时间常数','h'),
'connected_depth_m':('最大连通水头','m'),
'rain_multiplier':('有效入渗倍率','1'),
'melt_mm_deg_day':('度日融化系数',r'mm/($^\circ$C$\cdot$d)'),
'damage_rate_h_inv':('损伤速率系数',r'h$^{-1}$'),
'damage_exponent':('应力比幂次','1'),
'initial_damage':('初始损伤','1'),
'drainage_damage_coupling':('损伤排水耦合系数','1'),
}
protocol_path='track_b/research/q2/round3/config/experiment_v1.json'
comparison_path='track_b/research/q2/round3/results/analytic_thermal/model_comparison.csv'
d=json.loads((REPO/protocol_path).read_text())
rows=list(csv.DictReader((REPO/comparison_path).open()))
chunks=[r'\subsection{结构水热共同参数设计}\label{app:parameter-table}'+'\n',
'下表给出空间填充设计的19项参数上下限及取样尺度。四机制共享256个参数点，两强度政策形成2048个设计情景；参数设计与概率先验分别登记。实际各参数点见相应参数表。\n',
r'\begingroup\small\begin{longtable}{p{3.8cm}p{2.2cm}rrp{1.6cm}}'+'\n',
r'\caption{共同参数设计范围。}\label{tab:app-parameters}\\'+'\n',
r'\toprule 参数 & 单位 & 下限 & 上限 & 取样尺度\\\midrule\endfirsthead'+'\n',
r'\toprule 参数 & 单位 & 下限 & 上限 & 取样尺度\\\midrule\endhead'+'\n']
for k,v in d['ranges'].items():
    n,u=names[k];chunks.append(f"{n} & {u} & {fmt(v['min'])} & {fmt(v['max'])} & "+('对数' if v['scale']=='log' else '线性')+r'\\'+'\n')
chunks += [r'\bottomrule\end{longtable}\endgroup'+'\n',
r'\subsection{解析热机制分类与裕度补充}'+'\n',
'分类按相应初态与首次过阈定义，在同一设计表内汇总；初始越阈与未来越阈分列。\n',
r'\begin{table}[htbp]\centering\small\caption{最终解析热版本的机制分类与最小稳定裕度中位数。}\label{tab:app-mechanisms}'+'\n',
r'\begin{tabular}{llrrrr}\toprule 机制 & 强度政策 & 未来越阈 & 初始越阈 & 时域内保持 & 裕度中位数\\\midrule'+'\n']
for r in rows:
    policy='线性延伸' if r['law_policy']=='linear_extension' else '暖端岩接触'
    mech=r['mechanism_id'].replace('_', '+')
```

```
    chunks.append(f"{mech} & {policy} & {r['interval_crossing_n']} & {r['left_censored_n']} & {r['never_crossed_n']} & {float(r['median_min_margin']):.4f}"+r'\\'+'\n')
chunks += [r'\bottomrule\end{tabular}\end{table}'+'\n']
(PAPER/'appendices/parameter_tables.tex').write_text(''.join(chunks))
record={'generated_from':[{'path':n,'sha256':hashlib.sha256((REPO/n).read_bytes()).hexdigest()} for n in [protocol_path,comparison_path]],'parameter_rows':len(d['ranges']),'mechanism_rows':len(rows),'all_classification_counts_per_row_256':all(sum(int(r[k]) for k in ['interval_crossing_n','left_censored_n','never_crossed_n','missing_n'])==256 for r in rows)}
(PAPER/'qa/supplement_tables_check.json').write_text(json.dumps(record,ensure_ascii=False,indent=2)+'\n')
print(json.dumps(record,ensure_ascii=False))
```

程序 56：论文结构、引用、匿名和研究完整性检查

来源：track_b/paper/scripts/verify_paper.py
SHA256: 84b7d3eda775025a75e3a72369eb71c2e71842fbd019c4272d76ba07efd4f095

```
#!/usr/bin/env python3
# AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24.
# Contributions and model identity are recorded in AI_DISCLOSURE.md.
"""Read-only source/PDF QA; writes only reports beneath paper/qa."""
from __future__ import annotations

import argparse
import hashlib
import json
import re
from pathlib import Path
import subprocess

from build_evidence import build as build_evidence, digest
from build_references import bibitems, citations, expanded_tex, tex_files, uncomment

PAPER = Path(__file__).resolve().parents[1]
REPO = PAPER.parents[1]
FORBIDDEN = ["尚未验证", "未达到", "未实现", "资格未接受", "未通过", "尚不满足", "不能声称",
↪  "门禁", "未完成"]
HISTORY_BASE = "eecf07db7873b3c2c63275fa0b419867d8bdd3b2"
PROTECTED_ROOTS = ["00_资料索引.txt", "01_比赛规范", "02_提交模板", "03_A赛道", "04_B赛道",
↪  "DeepSeek_A",
                   "track_b/research", "track_b/data", "track_b/audit", "track_b/config",
                   ↪  "track_b/scripts",
                   "track_b/requirements-lock.txt", "track_b/requirements.in"]


def identity_patterns() -> dict[str, str]:
    """Derive the actual local username without printing it in program source."""
    return {"email": r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
            "local_user_path": r"/Users/[A-Za-z0-9_.-]+(?:/|\b)",
            "local_username": r"(?i)" + re.escape(Path.home().name),
            "identity_field": r"(?:参赛队号|参赛队员|指导教师|作者姓名|所属学校)\s*[:： ]\s*\S+"}


def history_integrity(repo: Path, baseline: str) -> dict:
    """Compare protected material to the paper baseline, including working edits."""
    def git(*args):
        return subprocess.run(["git", *args], cwd=repo, text=True, capture_output=True,
        ↪  check=False)
    result = {"baseline_commit": baseline, "protected_roots": PROTECTED_ROOTS,
              "changed_protected_paths": [], "untracked_protected_paths": [], "branches": [],
              ↪  "issues": []}
    if git("cat-file", "-e", baseline + "^{commit}").returncode:
        result["issues"].append("paper baseline commit unavailable")
        result["passed"] = False
        return result
    diff = git("diff", "--name-only", "-z", "--no-renames", baseline, "--", *PROTECTED_ROOTS)
    untracked = git("ls-files", "--others", "--exclude-standard", "-z", "--", *PROTECTED_ROOTS)
    result["changed_protected_paths"] = [v for v in diff.stdout.split("\0") if v]
    result["untracked_protected_paths"] = [v for v in untracked.stdout.split("\0") if v]
    if diff.returncode or untracked.returncode:
        result["issues"].append("git integrity comparison error")
    if result["changed_protected_paths"] or result["untracked_protected_paths"]:
        result["issues"].append("protected research or original material changed")
    registry = repo / "track_b/audit/stage4/round3/workspaces.json"
```

```
    if registry.exists():
        for record in json.loads(registry.read_text(encoding="utf-8")).get("workspaces", []):
            ref, expected = record["git_branch"], record["source_commit"]
            resolved = git("rev-parse", "--verify", "refs/heads/" + ref)
            actual = resolved.stdout.strip()
            branch = {"branch": ref, "expected_source_commit": expected, "actual_commit":
            ↪  actual,
                      "passed": resolved.returncode == 0 and actual == expected}
            result["branches"].append(branch)
            if not branch["passed"]:
                result["issues"].append(f"historical R3 branch moved or missing: {ref}")
    result["passed"] = not result["issues"]
    return result


def group(text: str, start: int) -> tuple[str, int]:
    if start >= len(text) or text[start] != "{":
        raise ValueError("expected brace group")
    level = 0
    for i in range(start, len(text)):
        if text[i] == "{" and (i == 0 or text[i - 1] != "\\"):
            level += 1
        if text[i] == "}" and (i == 0 or text[i - 1] != "\\"):
            level -= 1
            if level == 0:
                return text[start + 1:i], i + 1
    raise ValueError("unclosed brace group")


def last_main_page(aux: Path) -> int | None:
    if not aux.exists():
        return None
    text = aux.read_text(encoding="utf-8", errors="replace")
    match = re.search(r"\\newlabel\{LastMainPage\}\s*", text)
    if not match:
        return None
    value, _ = group(text, match.end())
    _, end = group(value, 0)
    page, _ = group(value, end)
    return int(page) if page.isdigit() else None


def check_pdf(pdf: Path, aux: Path) -> dict:
    result = {"path": pdf.name, "exists": pdf.exists(), "last_main_page": last_main_page(aux),
              "checks": [], "issues": []}
    def add(name, passed, detail):
        result["checks"].append({"name": name, "passed": bool(passed), "detail": detail})
        if not passed:
            result["issues"].append(name)
    add("body_page_limit", result["last_main_page"] is not None and 1 <=
    ↪  result["last_main_page"] <= 30,
        {"LastMainPage": result["last_main_page"], "limit": 30})
    if not pdf.exists():
        result["issues"].append("PDF missing")
        return result
    try:
        from pypdf import PdfReader
        reader = PdfReader(pdf)
        pages = [page.extract_text() or "" for page in reader.pages]
        result.update(page_count=len(pages), sha256=digest(pdf))
        first = re.sub(r"\s+", "", pages[0]) if pages else ""
        second = re.sub(r"\s+", "", pages[1]) if len(pages) > 1 else ""
```

```
        add("first_page_abstract_keywords", "摘要" in first and "关键词" in first, "abstract and keywords on physical page 1")
        add("second_page_contents", " 目录" in second, "contents on physical page 2")
        author = str((reader.metadata or {}).get("/Author", "")).strip()
        add("anonymous_pdf_author", not author, {"Author": author})
        body_count = result["last_main_page"] or 0
        body_pages = pages[2:2 + body_count]
        # Footer extraction may reorder glyphs; a page number token in the final
        # 120 characters is a conservative automatic check, plus render QA.
        footer = []
        for i, page_text in enumerate(body_pages, 1):
            ok = bool(re.search(rf"(?<!\d){i}(?!\d)", page_text[-120:]))
            footer.append({"body_page": i, "footer_token_present": ok})
        add("body_footer_page_numbers", len(footer) == body_count and all(r["footer_token_present"] for r in footer), footer)
        identifiers = []
        patterns = identity_patterns()
        for number, text in enumerate(pages, 1):
            for name, pattern in patterns.items():
                for match in re.finditer(pattern, text):
                    identifiers.append({"page": number, "kind": name, "text": match.group(0)})
        add("anonymous_entire_pdf", not identifiers, identifiers)
        prose_hits = [{"page": number, "phrase": phrase} for number, text in enumerate(pages[:2 + body_count], 1)
                      for phrase in FORBIDDEN if phrase in re.sub(r"\s+", "", text)]
        add("positive_results_wording", not prose_hits, prose_hits)
    except (ImportError, OSError, ValueError, KeyError) as exc:
        result["issues"].append(f"PDF inspection error: {exc}")
    return result


def verify(paper: Path, repo: Path, pdf: Path | None, history_base: str = HISTORY_BASE) -> dict:
    checks, issues = [], []
    def add(name, passed, detail):
        checks.append({"name": name, "passed": bool(passed), "detail": detail})
        if not passed:
            issues.append(name)
    files = tex_files(paper)
    source = expanded_tex(paper)
    add("main_tex_exists", (paper / "main.tex").is_file(), "main.tex")
    missing_inputs = []
    missing_figures = []
    prose_hits = []
    for path in files:
        text = uncomment(path.read_text(encoding="utf-8"))
        for match in re.finditer(r"\\(?:input|include)\s*\{([^}]+)\}", text):
            name = match.group(1)
            target = paper / (name if Path(name).suffix else name + ".tex")
            if not target.is_file():
                missing_inputs.append({"file": str(path.relative_to(paper)), "input": name})
        for match in re.finditer(r"\\includegraphics\*?(?:\s*\[[^\]]*\])?\s*\{([^}]+)\}", text):
            name = match.group(1)
            target = paper / name
            exists = target.is_file() if target.suffix else any(target.with_suffix(s).is_file() for s in (".pdf", ".png", ".jpg", ".jpeg"))
            if not exists:
                missing_figures.append({"file": str(path.relative_to(paper)), "figure": name})
        if "appendices" not in path.parts and path.name not in ("preamble.tex", "references.tex"):
            for phrase in FORBIDDEN:
                if phrase in text:
```

```
                    prose_hits.append({"file": str(path.relative_to(paper)), "phrase": phrase})
    add("all_tex_inputs_exist", not missing_inputs, missing_inputs)
    add("all_figures_exist", not missing_figures, missing_figures)
    labels = re.findall(r"\\label\s*\{([^}]+)\}", source)
    duplicate_labels = sorted({label for label in labels if labels.count(label) > 1})
    refs = re.findall(r"\\(?:ref|eqref|autoref|pageref|cref|Cref)\*?\s*\{([^}]+)\}", source)
    missing_labels = sorted({key.strip() for ref in refs for key in ref.split(",")} -
    ↪  set(labels))
    add("unique_labels", not duplicate_labels, duplicate_labels)
    add("all_label_references_resolve", not missing_labels, missing_labels)
    keys = citations(source)
    bib = bibitems(paper / "references.tex") if (paper / "references.tex").exists() else []
    bibkeys = [key for key, _ in bib]
    add("all_citations_resolve", not (set(keys) - set(bibkeys)), sorted(set(keys) -
    ↪  set(bibkeys)))
    add("bibliography_first_citation_order", bibkeys == keys, {"citations": keys,
    ↪  "bibliography": bibkeys})
    add("positive_results_source_wording", not prose_hits, prose_hits)
    _, evidence_report = build_evidence(paper, repo)
    add("source_evidence_hashes", evidence_report["passed"], {k: evidence_report[k] for k in
    ↪  ("claim_count", "source_count", "issues", "chapters_pending")})
    add("four_chapter_evidence", not evidence_report["chapters_pending"],
    ↪  evidence_report["chapters_pending"])
    history = history_integrity(repo, history_base)
    add("historical_research_integrity", history["passed"], history)
    manifest_path = paper / "evidence/program_manifest.json"
    program_checks = []
    if manifest_path.exists():
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
        for record in manifest.get("programs", []):
            target = paper / record["portable_path"]
            display = paper / record.get("display_path", record["portable_path"])
            passed = target.is_file() and digest(target) == record["sha256"]
            display_passed = display.is_file() and digest(display) ==
            ↪  record.get("display_sha256", record["sha256"])
            program_checks.append({"path": record["portable_path"], "display_path":
            ↪  str(display.relative_to(paper)),
                                   "original_passed": passed, "display_passed": display_passed,
                                   "passed": passed and display_passed})
    add("portable_original_program_hashes", bool(program_checks) and all(r["passed"] for r in
    ↪  program_checks), program_checks)
    aux = (pdf.with_suffix(".aux") if pdf else paper / "main.aux")
    log = aux.with_suffix(".log")
    if log.exists():
        text = log.read_text(encoding="utf-8", errors="replace")
        warnings = [line for line in text.splitlines() if ("undefined" in line.lower() or
        ↪  "multiply defined" in line.lower()) and "warning" in line.lower()]
        add("resolved_latex_references", not warnings, warnings)
        overflow = re.findall(r"Overfull \\[hv]box[^\n]*", text)
        add("no_overfull_boxes", not overflow, overflow)
    else:
        add("latex_log_exists", False, str(log))
    pdf_report = check_pdf(pdf or paper / "main.pdf", aux)
    add("pdf_content_checks", not pdf_report["issues"], pdf_report)
    return {"schema_version": 1, "passed": not issues, "check_count": len(checks),
            "passed_count": sum(r["passed"] for r in checks), "checks": checks, "issues":
            ↪  issues,
            "visual_review": "Automatic checks complement page-by-page rendered inspection;
            ↪  mathematical applicability is reviewed through chapter evidence and handoffs."}


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
```

```
    parser.add_argument("--paper", type=Path, default=PAPER)
    parser.add_argument("--repo", type=Path, default=REPO)
    parser.add_argument("--pdf", type=Path, help="compiled PDF, default paper/main.pdf; aux/log expected alongside")
    parser.add_argument("--history-base", default=HISTORY_BASE, help="paper common baseline for protected research/material comparison")
    args = parser.parse_args()
    paper, repo = args.paper.resolve(), args.repo.resolve()
    report = verify(paper, repo, args.pdf.resolve() if args.pdf else None, args.history_base)
    (paper / "qa").mkdir(parents=True, exist_ok=True)
    (paper / "qa/paper_check.json").write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(json.dumps({k: report[k] for k in ("passed", "check_count", "passed_count", "issues")}, ensure_ascii=False))
    return 0 if report["passed"] else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

程序 57：PDF 逐页渲染与总览图生成

来源：track_b/paper/scripts/render_qa.py
SHA256: ccdd2287b11c905b974c63b39abf0a3c030f9584af19f28d902201d13502ef0d

```
#!/usr/bin/env python3
"""Render every selected PDF page and contact sheets. AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24."""
from pathlib import Path
import argparse,hashlib,json,shutil,subprocess
from PIL import Image,ImageOps,ImageDraw
PAPER=Path(__file__).resolve().parents[1]
parser=argparse.ArgumentParser()
parser.add_argument('--pdf',type=Path,required=True)
parser.add_argument('--tag',required=True)
parser.add_argument('--dpi',type=int,default=85)
parser.add_argument('--first-page',type=int,default=1)
parser.add_argument('--last-page',type=int)
args=parser.parse_args()
pdf=args.pdf.resolve()
pages=PAPER/'qa/pages'/args.tag
sheets=PAPER/'qa/contact_sheets'
pages.mkdir(parents=True,exist_ok=True);sheets.mkdir(parents=True,exist_ok=True)
for old in pages.glob('page-*.png'):old.unlink()
for old in sheets.glob(args.tag+'-*.jpg'):old.unlink()
tool=shutil.which('pdftoppm')
if not tool:raise SystemExit('pdftoppm is required for rendering')
cmd=[tool,'-r',str(args.dpi),'-f',str(args.first_page)]
if args.last_page:cmd+=['-l',str(args.last_page)]
cmd+=['-png',str(pdf),str(pages/'page')]
subprocess.run(cmd,check=True)
images=sorted(pages.glob('page-*.png'))
contacts=[]
for start in range(0,len(images),12):
    group=images[start:start+12]
    canvas=Image.new('RGB',(1080,1560),'#d8d8d8')
    draw=ImageDraw.Draw(canvas)
    for j,fp in enumerate(group):
        im=Image.open(fp).convert('RGB')
        im.thumbnail((340,480))
        x=(j%3)*360+10;y=(j//3)*390+23
        im.thumbnail((340,360))
        canvas.paste(im,(x+(340-im.width)//2,y))
        draw.text((x,y-17),args.tag+' / '+fp.stem,fill='black')
    out=sheets/f'{args.tag}-{start//12+1:03d}.jpg'
    canvas.save(out,quality=88)
    contacts.append(str(out.relative_to(PAPER)))
report={'pdf':str(pdf.relative_to(PAPER)),'sha256':hashlib.sha256(pdf.read_bytes()).hexdigest(),'dpi':args.dpi,'first_page':args.first_page,'last_page_requested':args.last_page,'page_images':len(images),'contact_sheets':contacts,'review_status':'rendered_for_visual_inspection'}
(PAPER/'qa'/f'render_{args.tag}.json').write_text(json.dumps(report,ensure_ascii=False,indent=2)+'\n')
print(json.dumps(report,ensure_ascii=False))
```

程序 58：论文可重复构建入口

来源：track_b/paper/scripts/build_paper.py
SHA256: 36f1972024b76869cb1b5563c8aaa021959f0d8ab4bb7c594caaf42b03224b1f

```
#!/usr/bin/env python3
"""Rebuild the paper from existing frozen research results. AI-assisted: OpenAI Codex / GPT-6 family, OpenAI, 2026-10-24."""
from pathlib import Path
import argparse,json,os,shutil,subprocess,sys
PAPER=Path(__file__).resolve().parents[1]
REPO=PAPER.parents[1]
parser=argparse.ArgumentParser()
parser.add_argument('--render',action='store_true')
args=parser.parse_args()
def run(command,cwd=REPO,log=None):
    if log:
        with log.open('w') as f:r=subprocess.run(command,cwd=cwd,stdout=f,stderr=subprocess.STDOUT)
        if r.returncode:
            print(log.read_text(errors='replace')[-5000:])
            raise SystemExit(r.returncode)
    else:subprocess.run(command,cwd=cwd,check=True)
for script,tail in [('build_supplement_tables.py',[]),('build_evidence.py',['--require-all']),('build_references.py',[]),('build_program_appendix.py',[])]:
    run([sys.executable,'-B',str(PAPER/'scripts'/script),*tail])
latexmk=shutil.which('latexmk')
if not latexmk:raise SystemExit('latexmk / XeLaTeX required')
for stem,sub in [('main','main'),('ai_details','ai')]:
    work=PAPER/'build'/sub;work.mkdir(parents=True,exist_ok=True)
    run([latexmk,'-xelatex','-interaction=nonstopmode','-halt-on-error',f'-outdir=build/{sub}',f'{stem}.tex'],cwd=PAPER,log=work/'latexmk_console.log')
run([sys.executable,'-B',str(PAPER/'scripts/verify_paper.py'),'--pdf',str(PAPER/'build/main/main.pdf')])
output=PAPER/'output';output.mkdir(exist_ok=True)
shutil.copyfile(PAPER/'build/main/main.pdf',output/'赛道 B 论文初稿.pdf')
shutil.copyfile(PAPER/'build/ai/ai_details.pdf',output/'AI 工具使用详情说明.pdf')
if args.render:
    run([sys.executable,'-B',str(PAPER/'scripts/render_qa.py'),'--pdf',str(PAPER/'build/main/main.pdf'),'--tag','main','--dpi','70'])
    run([sys.executable,'-B',str(PAPER/'scripts/render_qa.py'),'--pdf',str(PAPER/'build/ai/ai_details.pdf'),'--tag','ai','--dpi','100'])
print(json.dumps({'paper':str(output/'赛道B论文初稿.pdf'),'ai_details':str(output/'AI工具使用详情说明.pdf'),'model_rerun':False},ensure_ascii=False))
```